

PROJECT BOND — The Special-Agent Operations Compendium

A 33-Package Software Suite Wrapping One Chapter per James Bond Film

Daniel Ari Friedman

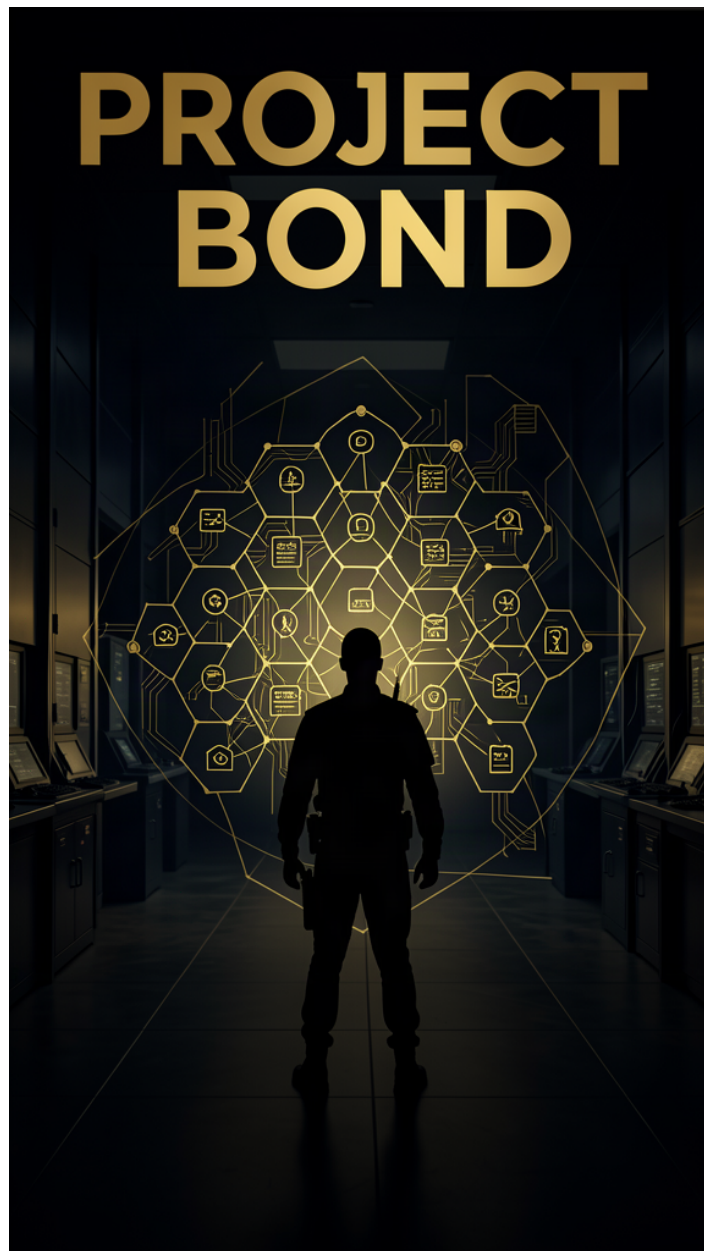
Active Inference Institute

`daniel@activeinference.institute`

ORCID: [0000-0001-6232-9096](https://orcid.org/0000-0001-6232-9096)

DOI: [10.5281/zenodo.21843592](https://doi.org/10.5281/zenodo.21843592)

August 7, 2026



Contents

1	Abstract	22
2	Introduction	23
2.1	Two-layer topology: shared infrastructure and film-specific algorithms	23
2.2	Generation, hydration, and provenance of the compendium	23
3	Suite Architecture — layered topology, package roster, and measured gate	24
3.1	Layered topology and execution responsibilities	24
3.2	Complete package roster and mission codenames	24
3.3	Aggregate quality gate and coverage evidence	25
4	Dr. No (1962) — CRAB KEY	26
4.1	Concepts — CRAB KEY: domain and operational focus	26
4.2	Abstract — CRAB KEY: mission summary	26
4.3	Introduction — CRAB KEY: mission framing, the operational problem, and how to read this chapter	26
4.3.1	1.1 The threat on Crab Key	26
4.3.2	1.2 Three models, one mission	26
4.3.3	1.3 Reader’s guide	26
4.4	Methodology — CRAB KEY: the analytical models and algorithms that drive the mission	27
4.4.1	2.1 Gamma-spectrum isotope identification	27
4.4.2	2.2 Gaussian plume dispersion	27
4.4.3	2.3 Island threat assessment	27
4.4.4	2.4 Radioactive decay chains	28
4.4.5	2.5 Spectrum calibration and detection limits	28
4.4.6	2.6 Shielding, attenuation, and external dose	28
4.5	Results — CRAB KEY: measured outcomes, headline numbers, and what they establish	28
4.5.1	3.1 Isotope identification	28
4.5.2	3.2 Plume dispersion	28
4.5.3	3.3 Island assessment	30
4.5.4	3.4 Quantitative forensics	30
4.6	Conclusion — CRAB KEY: findings, verdict, and what the mission establishes	31
4.7	Experimental Setup — CRAB KEY: canonical scenarios, parameters, and configuration	31
4.7.1	5.1 Scenario configuration	31
4.7.2	5.2 Software environment	31
4.7.3	5.3 Determinism and reproducibility	31
4.8	Reproducibility — CRAB KEY: verification gates, deterministic regeneration, and artifacts	32
4.8.1	6.1 Verification gates	32
4.8.2	6.2 Deterministic regeneration	32
4.8.3	6.3 Artifacts	32
4.9	Scope and Related Work — CRAB KEY: boundaries, positioning, and relationship to the literature	32
4.9.1	7.1 Scope	32
4.9.2	7.2 Related work	33
4.9.3	7.3 Limitations	33
4.10	Sources — CRAB KEY: bibliography	33
5	From Russia with Love (1963) — LEKTOR	34
5.1	Concepts — LEKTOR: domain and operational focus	34
5.2	Abstract — LEKTOR: mission summary	34
5.3	Introduction — LEKTOR: mission framing, the operational problem, and how to read this chapter	34
5.3.1	Reader’s guide	34
5.4	Methodology — LEKTOR: the analytical models and algorithms that drive the mission	35
5.4.1	2.1 Entrapment / deception playbook engine	35
5.4.2	2.2 Lure-budget allocation	35
5.4.3	2.3 SIGINT channel interception simulation	35
5.4.4	2.4 TDOA direction finding	35
5.4.5	2.5 Courier handoff scheduling	36
5.4.6	2.6 Defector handoff protocol	36
5.5	Results — LEKTOR: measured outcomes, headline numbers, and what they establish	36
5.5.1	3.0 At a glance	36
5.5.2	3.1 Lure effectiveness and allocation	37
5.5.3	3.2 SIGINT interception	38
5.5.4	3.3 Direction finding	38

5.5.5	3.4 Handoff scheduling	38
5.5.6	3.5 Defector handoff protocol	38
5.6	Conclusion — LEKTOR: findings, verdict, and what the mission establishes	38
5.7	Experimental Setup — LEKTOR: canonical scenarios, parameters, and configuration	40
5.7.1	Where the parameters live	40
5.7.2	Mission parameters	40
5.7.3	Software Environment	40
5.8	Reproducibility — LEKTOR: verification gates, deterministic regeneration, and artifacts	40
5.8.1	Determinism	40
5.8.2	Regenerating everything	41
5.8.3	Verification Gates	41
5.8.4	Artifact Inventory	41
5.9	Scope and Related Work — LEKTOR: boundaries, positioning, and relationship to the literature	41
5.9.1	Scope	41
5.9.2	Uncertainty and limitations	41
5.9.3	Related Work	42
5.10	Sources — LEKTOR: bibliography	42
6	Goldfinger (1964) — GRAND SLAM	43
6.1	Concepts — GRAND SLAM: domain and operational focus	43
6.2	Abstract — GRAND SLAM: mission summary	43
6.3	Introduction — GRAND SLAM: mission framing, the operational problem, and how to read this chapter	43
6.3.1	Macroeconomic attack	43
6.3.2	Vault defense	43
6.3.3	Laser threat	43
6.4	Methodology — GRAND SLAM: the analytical models and algorithms that drive the mission	44
6.4.1	Market cornering: supply shock to price dynamics	44
6.4.2	Vault defense: layered security scoring	44
6.4.3	Laser threat: burn-through time vs material and power	44
6.4.4	Macroeconomic attack: speculative attack on the gold peg	45
6.4.5	Network vault defense: penetration and hardening	45
6.4.6	Laser thermal engagement: beam focus and heat conduction	45
6.5	Results — GRAND SLAM: measured outcomes, headline numbers, and what they establish	45
6.5.1	Market corner: price shock and convergence	45
6.5.2	Vault defense: layered score and penetration	47
6.5.3	Vault network: penetration and hardening	47
6.5.4	Macroeconomic attack: speculative reserve drain	47
6.5.5	Laser threat: burn-through time	47
6.5.6	Laser thermal engagement: beam focus and heat conduction	47
6.6	Conclusion — GRAND SLAM: findings, verdict, and what the mission establishes	49
6.7	Experimental Setup — GRAND SLAM: canonical scenarios, parameters, and configuration	50
6.7.1	Reference intel	50
6.7.2	Determinism policy	50
6.7.3	Software environment	50
6.7.4	Mission adapter	50
6.8	Reproducibility — GRAND SLAM: verification gates, deterministic regeneration, and artifacts	51
6.8.1	Deterministic regeneration	51
6.8.2	Verification gate	51
6.8.3	Change policy	51
6.9	Scope and Related Work — GRAND SLAM: boundaries, positioning, and relationship to the literature	51
6.9.1	Scope	51
6.9.2	Related work	52
6.10	Uncertainty and Sensitivity	52
6.10.1	Market corner: elasticity sensitivity	52
6.10.2	Laser burn-through: the $1/P$ lever	53
6.10.3	Vault hardening: the greedy gap is measured	53
6.10.4	What the numbers are not	53
6.11	Sources — GRAND SLAM: bibliography	53
7	Thunderball (1965) — THUNDERBALL	54
7.1	Concepts — THUNDERBALL: domain and operational focus	54
7.2	Abstract — THUNDERBALL: mission summary	54
7.3	Introduction — THUNDERBALL: mission framing, the operational problem, and how to read this chapter	54

7.4	Methodology — THUNDERBALL: the analytical models and algorithms that drive the mission	54
7.4.1	Dive planning	54
7.4.2	Convoy integrity	55
7.4.3	Submersible logistics	55
7.4.4	Passive sonar detection	55
7.4.5	Set-and-drift navigation	55
7.4.6	Convoy interdiction	56
7.5	Results — THUNDERBALL: measured outcomes, headline numbers, and what they establish	56
7.5.1	Flagship verdict summary	56
7.5.2	Dive planning results	56
7.5.3	Convoy integrity results	56
7.5.4	Submersible logistics results	56
7.5.5	Passive sonar results	56
7.5.6	Navigation results	59
7.5.7	Interdiction results	59
7.6	Conclusion — THUNDERBALL: findings, verdict, and what the mission establishes	59
7.7	Experimental Setup — THUNDERBALL: canonical scenarios, parameters, and configuration	60
7.7.1	Dive parameters	60
7.7.2	Convoy parameters	60
7.7.3	Submersible parameters	60
7.7.4	Sonar parameters	60
7.7.5	Navigation parameters	60
7.7.6	Interdiction parameters	60
7.7.7	Environment	61
7.8	Reproducibility — THUNDERBALL: verification gates, deterministic regeneration, and artifacts	61
7.8.1	Test suite	61
7.8.2	External anchors	61
7.9	Scope and Related Work — THUNDERBALL: boundaries, positioning, and relationship to the literature	61
7.9.1	Scope	61
7.9.2	Related work	62
7.10	Discussion and Uncertainty — THUNDERBALL: interpretation, caveats, and what the numbers do not claim	62
7.10.1	The central finding is robust, not borderline	62
7.10.2	Dive planning uncertainty	62
7.10.3	Convoy-screen uncertainty	63
7.10.4	Submersible-logistics uncertainty	63
7.10.5	Passive-sonar uncertainty	63
7.10.6	Navigation uncertainty	63
7.10.7	Interdiction uncertainty	63
7.10.8	What would reduce these uncertainties	63
7.11	Sources — THUNDERBALL: bibliography	63
8	You Only Live Twice (1967) — BIRD ONE	64
8.1	Concepts — BIRD ONE: domain and operational focus	64
8.2	Abstract — BIRD ONE: mission summary	64
8.3	Introduction — BIRD ONE: mission framing, the operational problem, and how to read this chapter	64
8.3.1	Reader's guide	64
8.4	Methodology — BIRD ONE: the analytical models and algorithms that drive the mission	65
8.4.1	1. Rendezvous / intercept window solving	65
8.4.2	2. Orbital capture and phasing	65
8.4.3	3. Concealed-lair detection via thermal + terrain anomaly	65
8.4.4	4. Hidden-base discovery by Bayesian cue fusion	65
8.4.5	5. Infiltration planning	66
8.4.6	6. Emitter localization by TDOA	66
8.4.7	7. Pattern-of-life analysis	66
8.5	Results — BIRD ONE: measured outcomes, headline numbers, and what they establish	66
8.5.1	Rendezvous / intercept windows	66
8.5.2	Orbital capture	68
8.5.3	Concealed-lair detection	68
8.5.4	Hidden-base discovery	68
8.5.5	Infiltration planning	69
8.5.6	Emitter localization	69
8.5.7	Pattern of life	69
8.5.8	End-to-end mission	69

8.6	Conclusion — BIRD ONE: findings, verdict, and what the mission establishes	73
8.7	Experimental Setup — BIRD ONE: canonical scenarios, parameters, and configuration	75
8.7.1	Mission identity	75
8.7.2	Orbital geometry	75
8.7.3	Search envelopes	75
8.7.4	Scene, sensors and cost model	75
8.7.5	Rendering pipeline	75
8.7.6	Software environment	76
8.7.7	Test gate	76
8.8	Reproducibility — BIRD ONE: verification gates, deterministic regeneration, and artifacts	76
8.8.1	Determinism	76
8.8.2	Artifacts	76
8.8.3	Test gate	76
8.9	Scope and Related Work — BIRD ONE: boundaries, positioning, and relationship to the literature	76
8.9.1	Scope	76
8.9.2	Related work	77
8.9.3	Limitations and future work	77
8.10	Sources — BIRD ONE: bibliography	77
9	On Her Majesty’s Secret Service (1969) — BEDLAM	78
9.1	Concepts — BEDLAM: domain and operational focus	78
9.2	Abstract — BEDLAM: mission summary	78
9.3	Introduction — BEDLAM: mission framing, the operational problem, and how to read this chapter	78
9.3.1	Reader’s guide	78
9.4	Methodology — BEDLAM: the analytical models and algorithms that drive the mission	79
9.4.1	Aerosol bioweapon dispersion	79
9.4.2	Alpine route planning	79
9.4.3	Cover-identity risk scoring	79
9.4.4	Time-dependent puff dispersion	79
9.4.5	Bobsleigh descent dynamics	79
9.4.6	Cover-timeline verification	79
9.5	Results — BEDLAM: measured outcomes, headline numbers, and what they establish	80
9.5.1	Dispersion footprint	80
9.5.2	Puff arrival and early warning	80
9.5.3	Alpine exfiltration route	81
9.5.4	Bobsleigh descent and banked-curve clearance	81
9.5.5	Cover identity	81
9.5.6	Timeline verification	81
9.6	Conclusion — BEDLAM: findings, verdict, and what the mission establishes	81
9.7	Experimental Setup — BEDLAM: canonical scenarios, parameters, and configuration	83
9.7.1	Scenario parameters	83
9.7.2	Software environment	83
9.7.3	Guarantees	83
9.8	Reproducibility — BEDLAM: verification gates, deterministic regeneration, and artifacts	83
9.8.1	Determinism contract	83
9.8.2	Provenance	83
9.8.3	Test coverage	83
9.9	Scope and Related Work — BEDLAM: boundaries, positioning, and relationship to the literature	83
9.9.1	Scope	83
9.9.2	Limitations and uncertainty	84
9.9.3	Related work	84
9.10	Sources — BEDLAM: bibliography	84
10	Diamonds Are Forever (1971) — DIAMOND NET	85
10.1	Concepts — DIAMOND NET: domain and operational focus	85
10.2	Abstract — DIAMOND NET: mission summary	85
10.3	Introduction — DIAMOND NET: mission framing, the operational problem, and how to read this chapter	85
10.4	Methodology — DIAMOND NET: the analytical models and algorithms that drive the mission	85
10.4.1	Provenance chain	86
10.4.2	Smuggling corridor routing	86
10.4.3	Lot forensics	86
10.4.4	Casino money movement	86
10.4.5	Benford’s-law ledger audit	86

10.4.6	Orbital targeting geometry	86
10.4.7	Beam delivery physics	86
10.5	Results — DIAMOND NET: measured outcomes, headline numbers, and what they establish	87
10.5.1	Provenance chain	87
10.5.2	Smuggling corridor routing	87
10.5.3	Lot forensics	87
10.5.4	Casino money movement	89
10.5.5	Benford's-law ledger audit	89
10.5.6	Orbital-mirror targeting	89
10.5.7	Beam delivery physics	89
10.5.8	Mission outcome	92
10.6	Conclusion — DIAMOND NET: findings, verdict, and what the mission establishes	92
10.7	Experimental Setup — DIAMOND NET: canonical scenarios, parameters, and configuration	92
10.8	Reproducibility — DIAMOND NET: verification gates, deterministic regeneration, and artifacts	92
10.9	Scope and Related Work — DIAMOND NET: boundaries, positioning, and relationship to the literature	93
10.9.1	Scope	93
10.9.2	Related work	93
10.9.3	Limitations and uncertainty	93
10.10	Sources — DIAMOND NET: bibliography	94
11	Live and Let Die (1973) — SAN MONIQUE	95
11.1	Concepts — SAN MONIQUE: domain and operational focus	95
11.2	Abstract — SAN MONIQUE: mission summary	95
11.3	Introduction — SAN MONIQUE: mission framing, the operational problem, and how to read this chapter	95
11.4	Methodology — SAN MONIQUE: the analytical models and algorithms that drive the mission	95
11.4.1	Supply-graph analytics	95
11.4.2	Budgeted network interdiction	96
11.4.3	Coded communiqués: symbol substitution and drum code	96
11.4.4	Market-flow economics	96
11.4.5	Island ops planning	96
11.5	Results — SAN MONIQUE: measured outcomes, headline numbers, and what they establish	97
11.5.1	Supply graph	97
11.5.2	Interdiction	97
11.5.3	Communiqués	97
11.5.4	Market economics	98
11.5.5	Infiltration route	98
11.6	Conclusion — SAN MONIQUE: findings, verdict, and what the mission establishes	98
11.7	Experimental Setup — SAN MONIQUE: canonical scenarios, parameters, and configuration	98
11.7.1	Dataset	98
11.7.2	Determinism	100
11.7.3	Figures	100
11.8	Reproducibility — SAN MONIQUE: verification gates, deterministic regeneration, and artifacts	100
11.8.1	Byte-identical regeneration	100
11.9	Scope and Related Work — SAN MONIQUE: boundaries, positioning, and relationship to the literature	100
11.9.1	Limitations and uncertainty	101
11.10	Sources — SAN MONIQUE: bibliography	101
12	The Man with the Golden Gun (1974) — SOLEX	102
12.1	Concepts — SOLEX: domain and operational focus	102
12.2	Abstract — SOLEX: mission summary	102
12.3	Introduction — SOLEX: mission framing, the operational problem, and how to read this chapter	102
12.4	Methodology — SOLEX: the analytical models and algorithms that drive the mission	103
12.4.1	Solar concentration yield	103
12.4.2	Solar tracking & daily insolation	103
12.4.3	Lair escape topology	103
12.4.4	Lair flow analysis	103
12.4.5	Duel / bounty scheduling	103
12.4.6	Deadline bounty scheduling	104
12.5	Results — SOLEX: measured outcomes, headline numbers, and what they establish	104
12.5.1	Solar concentration yield	104
12.5.2	Solar tracking & daily insolation	104
12.5.3	Lair escape topology	105
12.5.4	Lair flow analysis	106

12.5.5	Duel / bounty scheduling	106
12.5.6	Deadline bounty scheduling	106
12.6	Conclusion — SOLEX: findings, verdict, and what the mission establishes	107
12.7	Experimental Setup — SOLEX: canonical scenarios, parameters, and configuration	107
12.7.1	Software environment	107
12.7.2	Heliostat field	108
12.7.3	Solar site and day	108
12.7.4	Funhouse lair fixtures	108
12.7.5	Bounty contract books	108
12.8	Reproducibility — SOLEX: verification gates, deterministic regeneration, and artifacts	109
12.8.1	Regeneration procedure	109
12.8.2	Sources of determinism	109
12.8.3	Metric injection	109
12.8.4	Test and gate discipline	110
12.8.5	What is not reproducible here	110
12.9	Scope and Related Work — SOLEX: boundaries, positioning, and relationship to the literature	110
12.9.1	What this package is	110
12.9.2	What this package is not	110
12.9.3	Relationship to the established literature	110
12.9.4	Position in the fleet	111
12.10	Appendix: Generated metric register	111
12.10.1	Solar concentration	111
12.10.2	Solar tracking & daily insolation	111
12.10.3	Lair topology (canonical)	112
12.10.4	Lair flow (extended)	112
12.10.5	Duel interval scheduling	112
12.10.6	Deadline scheduling	112
12.10.7	Environment and gates	112
12.11	Sources — SOLEX: bibliography	113
13	The Spy Who Loved Me (1977) — LIPARUS	114
13.1	Concepts — LIPARUS: domain and operational focus	114
13.2	Abstract — LIPARUS: mission summary	114
13.3	Introduction — LIPARUS: mission framing, the operational problem, and how to read this chapter	114
13.4	Methodology — LIPARUS: the analytical models and algorithms that drive the mission	115
13.4.1	Passive acoustic detection and bearing-only tracking	115
13.4.2	Liparus-class tanker capture	115
13.4.3	Submersible vehicle forensics	115
13.4.4	Underwater acoustics and the passive sonar equation	115
13.4.5	Uniform-linear-array beamforming	116
13.4.6	Pressure-hull and ballast forensics	116
13.4.7	Mission orchestration	116
13.5	Results — LIPARUS: measured outcomes, headline numbers, and what they establish	116
13.5.1	Detection	116
13.5.2	Passive tracking	116
13.5.3	Passive sonar equation	118
13.5.4	Beamforming	118
13.5.5	Liparus capture	118
13.5.6	Vehicle conversion forensics	120
13.5.7	Mission outcome	120
13.6	Conclusion — LIPARUS: findings, verdict, and what the mission establishes	120
13.7	Experimental Setup — LIPARUS: canonical scenarios, parameters, and configuration	121
13.7.1	Canonical scenarios	121
13.7.2	Mission identity	121
13.7.3	Verification	122
13.8	Reproducibility — LIPARUS: verification gates, deterministic regeneration, and artifacts	122
13.9	Scope and Related Work — LIPARUS: boundaries, positioning, and relationship to the literature	122
13.9.1	Scope	122
13.9.2	Related work	123
13.10	Sources — LIPARUS: bibliography	123
14	Moonraker (1979) — MOONRAKER	124
14.1	Concepts — MOONRAKER: domain and operational focus	124

14.2	Abstract — MOONRAKER: mission summary	124
14.3	Introduction — MOONRAKER: mission framing, the operational problem, and how to read this chapter	124
14.3.1	Reader's guide	124
14.4	Methodology — MOONRAKER: the analytical models and algorithms that drive the mission	125
14.4.1	Shuttle hijack timeline (<code>shuttle_ops.py</code>)	125
14.4.2	Space-station orbital logistics (<code>orbital_logistics.py</code>)	125
14.4.3	Centrifuge g-force profiles (<code>centrifuge.py</code>)	125
14.5	Results — MOONRAKER: measured outcomes, headline numbers, and what they establish	126
14.5.1	Shuttle hijack timeline	126
14.5.2	Space-station orbital logistics	126
14.5.3	Centrifuge g-force profile and training regimen	129
14.6	Conclusion — MOONRAKER: findings, verdict, and what the mission establishes	129
14.7	Experimental Setup — MOONRAKER: canonical scenarios, parameters, and configuration	130
14.7.1	Mission parameters	130
14.7.2	Running the setup	131
14.7.3	Software environment	131
14.8	Reproducibility — MOONRAKER: verification gates, deterministic regeneration, and artifacts	131
14.8.1	Determinism guarantees	131
14.8.2	What a reader can re-derive	131
14.8.3	Verification gate	132
14.9	Scope and Related Work — MOONRAKER: boundaries, positioning, and relationship to the literature	132
14.9.1	Scope	132
14.9.2	Related work	132
14.10	Sources — MOONRAKER: bibliography	133
15	For Your Eyes Only (1981) — ATAC	134
15.1	Concepts — ATAC: domain and operational focus	134
15.2	Abstract — ATAC: mission summary	134
15.3	Introduction — ATAC: mission framing, the operational problem, and how to read this chapter	134
15.3.1	Reader's guide	134
15.4	Methodology — ATAC: the analytical models and algorithms that drive the mission	134
15.4.1	Key escrow and recovery	134
15.4.2	Erasure-coded key recovery	135
15.4.3	Underwater retrieval planning	135
15.4.4	Greek-island insertion routing	136
15.4.5	Fire-control targeting	136
15.4.6	Underwater acoustic link budget	136
15.5	Results — ATAC: measured outcomes, headline numbers, and what they establish	136
15.5.1	Escrow release	136
15.5.2	Erasure-coded key recovery	137
15.5.3	Underwater retrieval	137
15.5.4	Island insertion routing	137
15.5.5	Targeting solution	139
15.5.6	Acoustic link budget	139
15.6	Conclusion — ATAC: findings, verdict, and what the mission establishes	139
15.6.1	On verdicts that can fail	141
15.7	Experimental Setup — ATAC: canonical scenarios, parameters, and configuration	141
15.7.1	Configuration	141
15.7.2	Software environment	141
15.8	Reproducibility — ATAC: verification gates, deterministic regeneration, and artifacts	141
15.8.1	Determinism contract	141
15.8.2	Artifact inventory	141
15.8.3	Test results	142
15.9	Scope and Related Work — ATAC: boundaries, positioning, and relationship to the literature	142
15.9.1	Scope	142
15.9.2	Related work	142
15.9.3	Limitations	143
15.10	Sources — ATAC: bibliography	143
16	Octopussy (1983) — FABERGE	144
16.1	Concepts — FABERGE: domain and operational focus	144
16.2	Abstract — FABERGE: mission summary	144
16.3	Introduction — FABERGE: mission framing, the operational problem, and how to read this chapter	144

16.3.1	Three disciplines, 6 pipelines	144
16.4	Methodology — FABERGE: the analytical models and algorithms that drive the mission	145
16.4.1	Spectral/UV forgery detection	145
16.4.2	Varnish-ageing kinetics (forgery depth)	145
16.4.3	Circus-cover logistics (min-cost flow)	145
16.4.4	Rolling-stock circulation (logistics depth)	146
16.4.5	Concealed-device sweep	146
16.4.6	Radiation source localization (sweep depth)	146
16.5	Results — FABERGE: measured outcomes, headline numbers, and what they establish	146
16.5.1	Results at a glance	146
16.5.2	Forgery detection	146
16.5.3	Varnish ageing (forgery depth)	147
16.5.4	Cover logistics	149
16.5.5	Rolling-stock circulation (logistics depth)	149
16.5.6	Concealment sweep	149
16.5.7	Source localization (sweep depth)	149
16.6	Conclusion — FABERGE: findings, verdict, and what the mission establishes	151
16.7	Experimental Setup — FABERGE: canonical scenarios, parameters, and configuration	151
16.7.1	Scenario constants	151
16.7.2	Environment	151
16.8	Reproducibility — FABERGE: verification gates, deterministic regeneration, and artifacts	151
16.9	Scope and Related Work — FABERGE: boundaries, positioning, and relationship to the literature	152
16.9.1	Scope	152
16.9.2	Related work	152
16.10	Sources — FABERGE: bibliography	153
17	A View to a Kill (1985) — MAIN STRIKE	154
17.1	Concepts — MAIN STRIKE: domain and operational focus	154
17.2	Abstract — MAIN STRIKE: mission summary	154
17.3	Introduction — MAIN STRIKE: mission framing, the operational problem, and how to read this chapter	154
17.3.1	The six models	154
17.3.2	Reader's guide	155
17.4	Methodology — MAIN STRIKE: the analytical models and algorithms that drive the mission	155
17.4.1	Implementation layering	155
17.4.2	Microchip supply-chain monopoly analysis	155
17.4.3	Flood-the-mine water-ingress dynamics	156
17.4.4	Race-form analytics and cover model	156
17.4.5	Monopoly acquisition optimizer	156
17.4.6	Valley impact — the flood's urban blast-reach	156
17.4.7	Racing betting market and cover finance	157
17.5	Results — MAIN STRIKE: measured outcomes, headline numbers, and what they establish	157
17.5.1	Supply-chain monopoly	157
17.5.2	Flood-the-mine water ingress	158
17.5.3	Race-form cover analytics	158
17.5.4	Monopoly acquisition	158
17.5.5	Valley impact	160
17.5.6	Cover finance	161
17.6	Conclusion — MAIN STRIKE: findings, verdict, and what the mission establishes	161
17.7	Experimental Setup — MAIN STRIKE: canonical scenarios, parameters, and configuration	161
17.7.1	Fixed model data	161
17.7.2	Simulation parameters	162
17.7.3	Software environment	162
17.8	Reproducibility — MAIN STRIKE: verification gates, deterministic regeneration, and artifacts	162
17.8.1	Determinism	162
17.8.2	Artifacts	162
17.8.3	Where the numbers come from	163
17.8.4	Guardrails check	163
17.9	Scope and Related Work — MAIN STRIKE: boundaries, positioning, and relationship to the literature	163
17.9.1	Scope	163
17.9.2	Related work	164
17.10	Sources — MAIN STRIKE: bibliography	164
18	The Living Daylights (1987) — LIVING DAYLIGHTS	165

18.1	Concepts — LIVING DAYLIGHTS: domain and operational focus	165
18.2	Abstract — LIVING DAYLIGHTS: mission summary	165
18.3	Introduction — LIVING DAYLIGHTS: mission framing, the operational problem, and how to read this chapter	165
18.3.1	The domain core	165
18.3.2	Protocol integration	166
18.3.3	Reader's guide	166
18.4	Methodology — LIVING DAYLIGHTS: the analytical models and algorithms that drive the mission	166
18.4.1	Sniper ballistics (<code>ballistics.py</code>)	166
18.4.2	Countersniper optics (<code>optics.py</code>)	167
18.4.3	Countersniper detection (<code>countersniper.py</code>)	167
18.4.4	Terrain visibility (<code>visibility.py</code>)	167
18.4.5	Defection-handling protocol (<code>defection_protocol.py</code>)	167
18.4.6	Evidence verification (<code>defection_protocol.py</code>)	168
18.4.7	Terrain ops (<code>terrain_ops.py</code>)	168
18.4.8	Composition (<code>compute_mission.py</code>)	168
18.4.9	Input-contract hardening	168
18.5	Results — LIVING DAYLIGHTS: measured outcomes, headline numbers, and what they establish	169
18.5.1	Sniper ballistics	169
18.5.2	Rules of engagement	169
18.5.3	The cello case	169
18.5.4	Defection-handling protocol	169
18.5.5	Mountain/desert terrain ops	169
18.5.6	Countersniper optics	170
18.5.7	Countersniper detection and localization	170
18.5.8	Overwatch selection	171
18.5.9	Evidence verification	171
18.6	Conclusion — LIVING DAYLIGHTS: findings, verdict, and what the mission establishes	171
18.7	Experimental Setup — LIVING DAYLIGHTS: canonical scenarios, parameters, and configuration	173
18.7.1	Mission parameters	173
18.7.2	Software environment	174
18.8	Reproducibility — LIVING DAYLIGHTS: verification gates, deterministic regeneration, and artifacts	174
18.8.1	Determinism	174
18.8.2	Artifacts and regeneration	174
18.8.3	Verification gate	175
18.9	Scope and Related Work — LIVING DAYLIGHTS: boundaries, positioning, and relationship to the literature	175
18.9.1	Scope boundary	175
18.9.2	Relation to the broader simulation canon	176
18.9.3	Intended use	176
18.10	Sources — LIVING DAYLIGHTS: bibliography	176
19	Licence to Kill (1989) — WAVEKREST	177
19.1	Concepts — WAVEKREST: domain and operational focus	177
19.2	Abstract — WAVEKREST: mission summary	177
19.3	Introduction — WAVEKREST: mission framing, the operational problem, and how to read this chapter	177
19.3.1	The ROGUE mission	177
19.3.2	Design principles	178
19.4	Methodology — WAVEKREST: the analytical models and algorithms that drive the mission	178
19.4.1	2.1 Money-laundering graph	178
19.4.2	2.2 Vessel forensics	178
19.4.3	2.3 Rogue-00 risk model	179
19.4.4	2.4 Narcotics-finance economics	179
19.4.5	2.5 Operations logistics	179
19.4.6	2.6 Surveillance and watchlist	179
19.5	Results — WAVEKREST: measured outcomes, headline numbers, and what they establish	179
19.5.1	3.0 Headline results at a glance	180
19.5.2	3.1 Layering and integration in the Sanchez network	180
19.5.3	3.2 Wavekrest-class vessel forensics	181
19.5.4	3.3 Rogue-00 optimal policy	181
19.5.5	3.4 Narcotics-finance economics	182
19.5.6	3.5 Operations logistics	182
19.5.7	3.6 Surveillance and watchlist	182
19.6	Conclusion — WAVEKREST: findings, verdict, and what the mission establishes	183
19.7	Experimental Setup — WAVEKREST: canonical scenarios, parameters, and configuration	183

19.7.1	Where the parameters actually live	184
19.7.2	Laundering network	184
19.7.3	Vessel forensics	184
19.7.4	Rogue-00 risk model	184
19.7.5	Narcotics-finance economics	184
19.7.6	Operations logistics	184
19.7.7	Surveillance and watchlist	184
19.7.8	Software environment	185
19.8	Reproducibility — WAVEKREST: verification gates, deterministic regeneration, and artifacts	185
19.8.1	Determinism contract	185
19.8.2	Provenance	185
19.8.3	Test coverage	185
19.8.4	Artifacts	185
19.9	Scope and Related Work — WAVEKREST: boundaries, positioning, and relationship to the literature	186
19.9.1	Scope	186
19.9.2	Related work	186
19.9.3	Limitations	186
19.10	Sources — WAVEKREST: bibliography	187
20	GoldenEye (1995) — ARKANGEL	188
20.1	Concepts — ARKANGEL: domain and operational focus	188
20.2	Abstract — ARKANGEL: mission summary	188
20.3	Introduction — ARKANGEL: mission framing, the operational problem, and how to read this chapter	188
20.3.1	Reader's guide	188
20.3.2	What this manuscript covers	189
20.4	Methodology — ARKANGEL: the analytical models and algorithms that drive the mission	189
20.4.1	Orbital EMP weapon model (<code>src/goldeneye/emp_model.py</code>)	189
20.4.2	Electronics kill-radius analysis (<code>src/goldeneye/kill_radius.py</code>)	189
20.4.3	Arkangel dam security (<code>src/goldeneye/dam_security.py</code>)	189
20.4.4	Orbital platform geometry (<code>src/goldeneye/orbital_mechanics.py</code>)	190
20.4.5	Electromagnetic coupling and shielding (<code>src/goldeneye/coupling.py</code>)	190
20.4.6	Regional power-network resilience (<code>src/goldeneye/grid_resilience.py</code>)	190
20.4.7	Breach outflow hydrograph (<code>src/goldeneye/breach_hydrograph.py</code>)	190
20.5	Results — ARKANGEL: measured outcomes, headline numbers, and what they establish	190
20.5.1	EMP threat envelope	190
20.5.2	Kill radius by vulnerability class	190
20.5.3	Arkangel dam breach flood	191
20.5.4	Downstream inundation stations	191
20.5.5	Access-control posture	191
20.5.6	Orbital platform	191
20.5.7	Coupling and shielding	191
20.5.8	Regional power-network resilience	191
20.5.9	Breach outflow hydrograph	191
20.6	Conclusion — ARKANGEL: findings, verdict, and what the mission establishes	193
20.7	Experimental Setup — ARKANGEL: canonical scenarios, parameters, and configuration	193
20.7.1	Scenario parameters	193
20.7.2	Software environment	194
20.7.3	Verification gates	194
20.8	Reproducibility — ARKANGEL: verification gates, deterministic regeneration, and artifacts	194
20.8.1	Regeneration contract	194
20.8.2	Determinism	194
20.8.3	Output inventory	195
20.9	Scope and Related Work — ARKANGEL: boundaries, positioning, and relationship to the literature	195
20.9.1	Scope	195
20.9.2	Related work	195
20.10	Sources — ARKANGEL: bibliography	196
21	Tomorrow Never Dies (1997) — CARVER	197
21.1	Concepts — CARVER: domain and operational focus	197
21.2	Abstract — CARVER: mission summary	197
21.3	Introduction — CARVER: mission framing, the operational problem, and how to read this chapter	197
21.4	Methodology — CARVER: the analytical models and algorithms that drive the mission	198
21.4.1	Disinformation propagation	198

21.4.2	Narrative control (bounded confidence)	198
21.4.3	Stealth-vessel detection	198
21.4.4	Pulse compression (waveform design)	198
21.4.5	GPS-denial impact	198
21.4.6	GNSS integrity (RAIM)	199
21.5	Results — CARVER: measured outcomes, headline numbers, and what they establish	199
21.5.1	Disinformation propagation	199
21.5.2	Narrative control	199
21.5.3	Stealth-vessel detection	199
21.5.4	Pulse compression	201
21.5.5	GPS-denial impact	201
21.5.6	GNSS integrity (RAIM)	201
21.6	Conclusion — CARVER: findings, verdict, and what the mission establishes	202
21.7	Experimental Setup — CARVER: canonical scenarios, parameters, and configuration	202
21.7.1	Scenario configuration	202
21.7.2	Software environment	203
21.7.3	Regeneration	203
21.8	Reproducibility — CARVER: verification gates, deterministic regeneration, and artifacts	203
21.8.1	Determinism guarantees	203
21.8.2	Verification	204
21.8.3	Artifacts	204
21.8.4	What is <i>not</i> reproducible from this manuscript alone	204
21.9	Scope and Related Work — CARVER: boundaries, positioning, and relationship to the literature	204
21.9.1	Scope	204
21.9.2	Related work	205
21.10	Sources — CARVER: bibliography	205
22	The World Is Not Enough (1999) — ELEKTRA	206
22.1	Concepts — ELEKTRA: domain and operational focus	206
22.2	Abstract — ELEKTRA: mission summary	206
22.3	Introduction — ELEKTRA: mission framing, the operational problem, and how to read this chapter	206
22.3.1	Mission context	206
22.3.2	Six pillars	206
22.3.3	Reader's guide	207
22.4	Methodology — ELEKTRA: the analytical models and algorithms that drive the mission	207
22.4.1	2.1 Oil-pipeline security: choke points and tamper	207
22.4.2	2.2 Nuclear-submarine defense: sonar detection and evasion	207
22.4.3	2.3 Hostage psychology: Stockholm-syndrome dynamics	208
22.4.4	2.4 Energy infrastructure: DC power flow and cascading blackout	208
22.4.5	2.5 Hostage negotiation: bargaining under syndrome-driven patience	208
22.4.6	2.6 Sensor fusion: Kalman tracking of the hijacked submarine	208
22.5	Results — ELEKTRA: measured outcomes, headline numbers, and what they establish	208
22.5.1	Table 3 — Headline measured results (live tokens)	209
22.5.2	3.1 Oil-pipeline choke-point security	209
22.5.3	3.2 Nuclear-submarine defense	210
22.5.4	3.3 Hostage-psychology influence	210
22.5.5	3.4 Energy infrastructure: coordinated blackout	212
22.5.6	3.5 Hostage negotiation: syndrome-driven leverage	212
22.5.7	3.6 Sensor fusion: tracking the hijacked hull	212
22.6	Conclusion — ELEKTRA: findings, verdict, and what the mission establishes	213
22.7	Experimental Setup — ELEKTRA: canonical scenarios, parameters, and configuration	213
22.7.1	Table 1 — Scenario parameters	213
22.7.2	Table 2 — Hostage-model rate constants	214
22.7.3	Software environment	214
22.7.4	What is <i>not</i> tunable from the config file	214
22.8	Reproducibility — ELEKTRA: verification gates, deterministic regeneration, and artifacts	215
22.8.1	Determinism guarantees	215
22.8.2	Reproducing every number	215
22.8.3	Verification gates	215
22.8.4	Artifact inventory	215
22.9	Scope and Related Work — ELEKTRA: boundaries, positioning, and relationship to the literature	216
22.9.1	Scope	216
22.9.2	Related work	216

22.10	Sources — ELEKTRA: bibliography	217
23	Die Another Day (2002) — ICARUS	218
23.1	Concepts — ICARUS: domain and operational focus	218
23.2	Abstract — ICARUS: mission summary	218
23.3	Introduction — ICARUS: mission framing, the operational problem, and how to read this chapter	218
23.4	Methodology — ICARUS: the analytical models and algorithms that drive the mission	218
23.4.1	Positive and negative controls	218
23.4.2	Biometric identity-replacement detection (<code>identity_detection.py</code>)	219
23.4.3	DNA sequence identity verification (<code>sequence_identity.py</code>)	219
23.4.4	Ice-palace air operations (<code>ice_ops.py</code>)	219
23.4.5	Cold-water hypothermia risk (<code>hypothermia.py</code>)	219
23.4.6	Ice-structure integrity (<code>ice_structure.py</code>)	219
23.4.7	Orbital-mirror targeting (<code>orbital_mirror.py</code>)	219
23.4.8	Execution and control structure	220
23.5	Results — ICARUS: measured outcomes, headline numbers, and what they establish	220
23.5.1	Identity-replacement detection (biometric)	220
23.5.2	Identity-replacement detection (sequence)	220
23.5.3	Ice-palace operations	222
23.5.4	Cold-water hypothermia risk	222
23.5.5	Ice-structure integrity	222
23.5.6	Orbital-mirror targeting	222
23.6	Conclusion — ICARUS: findings, verdict, and what the mission establishes	222
23.7	Experimental Setup — ICARUS: canonical scenarios, parameters, and configuration	223
23.7.1	Identity detection	223
23.7.2	DNA sequence identity	223
23.7.3	Ice-palace operations	224
23.7.4	Cold-water hypothermia risk	224
23.7.5	Ice-structure integrity	224
23.7.6	Orbital-mirror targeting	224
23.7.7	Software environment	224
23.7.8	What is configured versus what is measured	224
23.8	Reproducibility — ICARUS: verification gates, deterministic regeneration, and artifacts	225
23.9	Scope and Related Work — ICARUS: boundaries, positioning, and relationship to the literature	226
23.10	Sources — ICARUS: bibliography	226
24	Casino Royale (2006) — LE CHIFFRE	227
24.1	Concepts — LE CHIFFRE: domain and operational focus	227
24.2	Abstract — LE CHIFFRE: mission summary	227
24.3	Introduction — LE CHIFFRE: mission framing, the operational problem, and how to read this chapter	227
24.3.1	Principles	228
24.3.2	Reader's guide	228
24.4	Methodology — LE CHIFFRE: the analytical models and algorithms that drive the mission	228
24.4.1	Poker game-theory engine (<code>poker_engine.py</code>)	228
24.4.2	Chip-movement laundering detection (<code>laundering_detect.py</code>)	228
24.4.3	Parkour routing (<code>urban_routes.py</code>)	228
24.4.4	Digitalis poison triage (<code>poison_detect.py</code>)	229
24.4.5	Counterfactual regret minimization (<code>poker_cfr.py</code>)	229
24.4.6	Independent Chip Model (<code>icm.py</code>)	229
24.4.7	Network-level financial crime (<code>laundering_network.py</code>)	229
24.4.8	Bayesian opponent archetype inference (<code>opponent_bayes.py</code>)	229
24.4.9	Mission adapter and gadget catalogue (<code>mission.py</code> , <code>gadgets.py</code>)	229
24.5	Results — LE CHIFFRE: measured outcomes, headline numbers, and what they establish	230
24.5.1	The felt: equity, price, and decision	230
24.5.2	The capital: chip-flow laundering	230
24.5.3	The terrain: parkour extraction route	230
24.5.4	The martini: digitalis triage	230
24.5.5	The solve: CFR river-toy equilibrium	231
24.5.6	The freezeout: Independent Chip Model	231
24.5.7	The capital again: network-level laundering	231
24.5.8	The read: Bayesian opponent archetype	231
24.6	Conclusion — LE CHIFFRE: findings, verdict, and what the mission establishes	231
24.7	Experimental Setup — LE CHIFFRE: canonical scenarios, parameters, and configuration	233

24.7.1	Configuration	233
24.7.2	Canonical scenario	233
24.7.3	Software environment	233
24.7.4	Runtime determinism	233
24.8	Reproducibility — LE CHIFFRE: verification gates, deterministic regeneration, and artifacts	233
24.8.1	Determinism & provenance	233
24.8.2	Artifact layout	234
24.8.3	Test & quality gates	234
24.8.4	Verification (preflight)	234
24.9	Scope & Related Work	234
24.9.1	Scope	234
24.9.2	Related work	235
24.10	Sources — LE CHIFFRE: bibliography	235
25	Quantum of Solace (2008) — QUANTUM	236
25.1	Concepts — QUANTUM: domain and operational focus	236
25.2	Abstract — QUANTUM: mission summary	236
25.3	Introduction — QUANTUM: mission framing, the operational problem, and how to read this chapter	236
25.3.1	The threat model	236
25.4	Methodology — QUANTUM: the analytical models and algorithms that drive the mission	237
25.4.1	Desert hydrology (<code>quantum_of_solace/hydrology.py</code>)	237
25.4.2	Water-rights cartel (<code>quantum_of_solace/water_cartel.py</code>)	237
25.4.3	Commodity-cartel graph (<code>quantum_of_solace/cartel_graph.py</code>)	237
25.4.4	Water scarcity pricing (<code>quantum_of_solace/pricing.py</code>)	237
25.4.5	Aqueduct interdiction (<code>quantum_of_solace/resilience.py</code>)	238
25.4.6	Cooperative power (<code>quantum_of_solace/shapley.py</code>)	238
25.4.7	The mission adapter	238
25.5	Results — QUANTUM: measured outcomes, headline numbers, and what they establish	238
25.5.1	1. Desert water network resilience	238
25.5.2	2. Cartel leverage over water rights	238
25.5.3	3. Multi-resource commodity control	238
25.5.4	4. Water scarcity pricing	240
25.5.5	5. Aqueduct interdiction and resilience	240
25.5.6	6. Cooperative power over the portfolio	240
25.6	Conclusion — QUANTUM: findings, verdict, and what the mission establishes	240
25.7	Experimental Setup — QUANTUM: canonical scenarios, parameters, and configuration	242
25.7.1	Software environment	242
25.7.2	Problem instances	242
25.7.3	Determinism	243
25.8	Reproducibility — QUANTUM: verification gates, deterministic regeneration, and artifacts	243
25.8.1	Provenance	243
25.8.2	Regeneration pipeline	243
25.8.3	Manuscript-code binding	243
25.8.4	Configuration hash	243
25.9	Scope and Related Work — QUANTUM: boundaries, positioning, and relationship to the literature	243
25.9.1	Scope	243
25.9.2	Related work	244
25.10	Sources — QUANTUM: bibliography	244
26	Skyfall (2012) — SKYFALL	245
26.1	Concepts — SKYFALL: domain and operational focus	245
26.2	Abstract — SKYFALL: mission summary	245
26.3	Introduction — SKYFALL: mission framing, the operational problem, and how to read this chapter	245
26.3.1	The SILVA threat	245
26.3.2	Design principles	245
26.4	Methodology — SKYFALL: the analytical models and algorithms that drive the mission	246
26.4.1	Network intrusion model (<code>skyfall.cyber_ops</code>)	246
26.4.2	Traffic change-point detection (<code>skyfall.traffic_analysis</code>)	246
26.4.3	Exfiltration severance (<code>skyfall.exfiltration</code>)	246
26.4.4	Legacy exposure model (<code>skyfall.legacy_audit</code>)	246
26.4.5	Patch scheduling (<code>skyfall.patch_strategy</code>)	246
26.4.6	Estate defense model (<code>skyfall.estate_defense</code>)	247
26.5	Results — SKYFALL: measured outcomes, headline numbers, and what they establish	247

26.5.1	Intrusion reachability	247
26.5.2	Traffic detection and severance	247
26.5.3	Fleet exposure and patch schedule	247
26.5.4	Estate defense	248
26.6	Conclusion — SKYFALL: findings, verdict, and what the mission establishes	248
26.7	Software and Scenario	250
26.7.1	Package identity	250
26.7.2	Scenario configuration	250
26.7.3	Running the mission	250
26.8	Reproducibility — SKYFALL: verification gates, deterministic regeneration, and artifacts	250
26.8.1	Determinism guarantees	250
26.8.2	Verification gates	251
26.8.3	Manuscript-metric integrity	251
26.9	Scope and Related Work — SKYFALL: boundaries, positioning, and relationship to the literature	251
26.9.1	Scope	251
26.9.2	Related work	251
26.10	Sources — SKYFALL: bibliography	252
27	Spectre (2015) — NINE EYES	253
27.1	Concepts — NINE EYES: domain and operational focus	253
27.2	Abstract — NINE EYES: mission summary	253
27.3	Introduction — NINE EYES: mission framing, the operational problem, and how to read this chapter	253
27.4	Methodology — NINE EYES: the analytical models and algorithms that drive the mission	254
27.4.1	Surveillance-network centralization	254
27.4.2	Criminal-org cell model	254
27.4.3	Network geometry	254
27.4.4	Traffic interception	255
27.4.5	Compromise cascade	255
27.4.6	Sensor-placement cover	255
27.5	Results — NINE EYES: measured outcomes, headline numbers, and what they establish	255
27.5.1	Summary of measured results	255
27.5.2	Surveillance-network centralization	255
27.5.3	Criminal-org cell model	256
27.5.4	Network geometry	256
27.5.5	Traffic interception	257
27.5.6	Compromise cascade	257
27.5.7	Sensor-placement cover	257
27.6	Conclusion — NINE EYES: findings, verdict, and what the mission establishes	257
27.7	Experimental Setup — NINE EYES: canonical scenarios, parameters, and configuration	258
27.7.1	Dataset	258
27.7.2	Determinism	258
27.7.3	Software environment	258
27.7.4	Mission flow	258
27.8	Reproducibility — NINE EYES: verification gates, deterministic regeneration, and artifacts	258
27.8.1	Deterministic regeneration	258
27.8.2	Provenance	259
27.8.3	No hand-authored numbers	259
27.8.4	Gates	259
27.9	Scope and Related Work — NINE EYES: boundaries, positioning, and relationship to the literature	259
27.9.1	What this package claims	259
27.9.2	Out of scope	259
27.9.3	Related work by module	260
27.9.4	Position within the BOND suite	260
27.9.5	Threats to validity	260
27.10	Sources — NINE EYES: bibliography	260
28	No Time to Die (2021) — HERACLES	261
28.1	Concepts — HERACLES: domain and operational focus	261
28.2	Abstract — HERACLES: mission summary	261
28.3	Introduction — HERACLES: mission framing, the operational problem, and how to read this chapter	261
28.4	Methodology — HERACLES: the analytical models and algorithms that drive the mission	262
28.4.1	DNA-targeted bioweapon countermeasure (<code>bioweapon_counter.py</code>)	262
28.4.2	Poison-garden toxicology (<code>toxicology.py</code>)	262

28.4.3	Poison-delivery forensics (<code>delivery_forensics.py</code>)	262
28.4.4	Guide design and population off-target (<code>targeting_design.py</code>)	263
28.4.5	Antidote screening (<code>antidote_screening.py</code>)	263
28.4.6	Exposure assessment (<code>exposure_assessment.py</code>)	263
28.5	Results — HERACLES: measured outcomes, headline numbers, and what they establish	263
28.5.1	Countermeasure demonstration	263
28.5.2	Guide-design ranking	263
28.5.3	Poison-garden catalog	264
28.5.4	Antidote screen	264
28.5.5	Exposure assessment	264
28.5.6	Vesper trace forensics	266
28.6	Conclusion — HERACLES: findings, verdict, and what the mission establishes	266
28.7	Experimental Setup — HERACLES: canonical scenarios, parameters, and configuration	267
28.7.1	Software environment	267
28.7.2	Deterministic demonstration parameters	267
28.7.3	Figure generation	268
28.8	Reproducibility — HERACLES: verification gates, deterministic regeneration, and artifacts	268
28.8.1	Deterministic computation	268
28.8.2	Live manuscript metrics	269
28.8.3	Evidence provenance	269
28.8.4	Verification commands	269
28.9	Scope and Related Work — HERACLES: boundaries, positioning, and relationship to the literature	269
28.9.1	Scope	269
28.9.2	Related work	270
28.10	Sources — HERACLES: bibliography	271
29	Casino Royale (1967) — FIVE BONDS	272
29.1	Concepts — FIVE BONDS: domain and operational focus	272
29.2	Abstract — FIVE BONDS: mission summary	272
29.3	Introduction — FIVE BONDS: mission framing, the operational problem, and how to read this chapter	272
29.3.1	Reader's guide	272
29.4	Methodology — FIVE BONDS: the analytical models and algorithms that drive the mission	273
29.4.1	1. The coordination-farce model	273
29.4.2	2. The dispatch-chaos simulation	273
29.4.3	3. The spoof protocol analyzer	274
29.4.4	4. The identity-confusion model	274
29.4.5	5. The baccarat table model	274
29.4.6	6. The escalation-ladder model	275
29.5	Results — FIVE BONDS: measured outcomes, headline numbers, and what they establish	275
29.5.1	The five-Bond coordination farce	275
29.5.2	Dispatch chaos	276
29.5.3	Spoof ops doctrine	276
29.5.4	Identity confusion	276
29.5.5	The casino table	277
29.5.6	Escalation ladder	278
29.6	Conclusion — FIVE BONDS: findings, verdict, and what the mission establishes	278
29.7	Experimental Setup — FIVE BONDS: canonical scenarios, parameters, and configuration	279
29.7.1	Configuration	279
29.7.2	Software environment	279
29.7.3	Artifact regeneration	280
29.8	Reproducibility — FIVE BONDS: verification gates, deterministic regeneration, and artifacts	280
29.8.1	Determinism guarantees	280
29.8.2	Canonical metric snapshot	280
29.8.3	Verification	280
29.9	Scope and Related Work — FIVE BONDS: boundaries, positioning, and relationship to the literature	281
29.9.1	Scope	281
29.9.2	Related work	281
29.10	Sources — FIVE BONDS: bibliography	281
30	Never Say Never Again (1983) — REPLAY	282
30.1	Concepts — REPLAY: domain and operational focus	282
30.2	Abstract — REPLAY: mission summary	282
30.3	Introduction — REPLAY: mission framing, the operational problem, and how to read this chapter	282

30.3.1	Never Say Never Again and the remount battalion	282
30.3.2	Why determinism	283
30.4	Methodology — REPLAY: the analytical models and algorithms that drive the mission	283
30.4.1	Scenario model	283
30.4.2	Remount configuration	283
30.4.3	Success model	283
30.4.4	Op diff	283
30.4.5	Remount risk (escalation hazard and reliability)	283
30.4.6	Asset allocation (marginal gain)	283
30.4.7	Replay learning curve	284
30.4.8	Legacy asset migration	284
30.4.9	Sensitivity analysis	284
30.5	Results — REPLAY: measured outcomes, headline numbers, and what they establish	284
30.5.1	Command score	284
30.5.2	Legacy asset migration	285
30.5.3	Remount risk	285
30.5.4	Asset allocation	285
30.5.5	Replay learning curve	286
30.5.6	Sensitivity and robustness	286
30.6	Conclusion — REPLAY: findings, verdict, and what the mission establishes	286
30.7	Experimental Setup — REPLAY: canonical scenarios, parameters, and configuration	288
30.7.1	Canonical dataset	288
30.7.2	Parameters	288
30.7.3	Fixtures and isolation	288
30.8	Reproducibility — REPLAY: verification gates, deterministic regeneration, and artifacts	288
30.8.1	Determinism by construction	288
30.8.2	Provenance	289
30.8.3	Verification commands	289
30.9	Scope and Related Work — REPLAY: boundaries, positioning, and relationship to the literature	289
30.9.1	Scope	289
30.9.2	Related work	289
30.9.3	Limitations	289
30.10	Sources — REPLAY: bibliography	290
31	Bond Utilities — Q-BRANCH	291
31.1	Concepts — Q-BRANCH: domain and operational focus	291
31.2	Abstract — Q-BRANCH: mission summary	291
31.3	Introduction — Q-BRANCH: mission framing, the operational problem, and how to read this chapter	291
31.3.1	Module map	291
31.3.2	Reader's guide	292
31.4	Methodology — Q-BRANCH: the analytical models and algorithms that drive the mission	292
31.4.1	Ciphers (<code>src/bond_utilities/ciphers.py</code>)	292
31.4.2	Cryptanalysis helpers	292
31.4.3	Deterministic seeding (<code>randomness.py</code>)	292
31.4.4	Codename schemes (<code>codenames.py</code>)	293
31.4.5	Mission clock (<code>clock.py</code>)	293
31.4.6	Provenance (<code>mission_io.py</code>)	293
31.4.7	Reporting and figures (<code>reporting.py</code> , <code>figures/plots.py</code>)	293
31.4.8	Shared fixtures (<code>testing.py</code>)	293
31.4.9	Scoring and statistics	293
31.5	Results — Q-BRANCH: measured outcomes, headline numbers, and what they establish	293
31.5.1	Round-trip fidelity	293
31.5.2	Cryptanalysis demos	294
31.5.3	Edge-case robustness	294
31.5.4	Determinism	294
31.5.5	Seeded draw helpers	295
31.5.6	Provenance round-trip	295
31.5.7	Mission schedule	295
31.6	Conclusion — Q-BRANCH: findings, verdict, and what the mission establishes	295
31.7	Experimental Setup — Q-BRANCH: canonical scenarios, parameters, and configuration	296
31.7.1	Mission parameters	296
31.7.2	Software environment	296
31.8	Reproducibility — Q-BRANCH: verification gates, deterministic regeneration, and artifacts	296

31.8.1	Artifact inventory	296
31.8.2	Determinism claims	296
31.8.3	Quality gate	297
31.9	Scope and Related Work — Q-BRANCH: boundaries, positioning, and relationship to the literature	297
31.9.1	Scope	297
31.9.2	Related work	297
31.9.3	Limitations	297
31.10	Sources — Q-BRANCH: bibliography	297
32	Bond API — THE PROTOCOL	298
32.1	Concepts — THE PROTOCOL: domain and operational focus	298
32.2	Abstract — THE PROTOCOL: mission summary	298
32.3	Introduction — THE PROTOCOL: mission framing, the operational problem, and how to read this chapter	298
32.3.1	What this package provides	298
32.3.2	Reader's guide	298
32.4	Methodology — THE PROTOCOL: the analytical models and algorithms that drive the mission	298
32.4.1	The mission lifecycle	298
32.4.2	Frozen dataclasses	299
32.4.3	The provider contract	299
32.4.4	Discovery	299
32.4.5	Serialization	299
32.4.6	Registries	299
32.4.7	Capability helpers	300
32.5	Results — THE PROTOCOL: measured outcomes, headline numbers, and what they establish	300
32.5.1	Protocol surface	300
32.5.2	Conformance evidence	300
32.5.3	Capability-helper surface	300
32.5.4	Gate integrity	301
32.6	Conclusion — THE PROTOCOL: findings, verdict, and what the mission establishes	301
32.6.1	What the freeze buys	301
32.7	Experimental Setup — THE PROTOCOL: canonical scenarios, parameters, and configuration	302
32.7.1	Package layout	302
32.7.2	Configuration	302
32.7.3	Environment	302
32.7.4	Running the suite	302
32.8	Reproducibility — THE PROTOCOL: verification gates, deterministic regeneration, and artifacts	302
32.8.1	Determinism policy	302
32.8.2	Artifact inventory	302
32.8.3	Token hydration	303
32.9	Scope and Related Work — THE PROTOCOL: boundaries, positioning, and relationship to the literature	303
32.9.1	Scope	303
32.9.2	Design relationship	304
32.9.3	Related standards	304
32.10	Limitations and Uncertainty — THE PROTOCOL: assumptions, threats to validity, and what the numbers do not claim	304
32.10.1	Deliberately permissive boundaries (documented, not bugs)	304
32.10.2	Wire-format strictness (enforced since Round 2)	304
32.10.3	Where the evidence is bounded	304
32.10.4	Uncertainty in provenance	305
32.10.5	Versioning uncertainty	305
32.11	Sources — THE PROTOCOL: bibliography	305
33	Bond Coordinator — MISSION CONTROL	306
33.1	Concepts — MISSION CONTROL: domain and operational focus	306
33.2	Abstract — MISSION CONTROL: mission summary	306
33.3	Introduction — MISSION CONTROL: mission framing, the operational problem, and how to read this chapter	306
33.3.1	Reader's guide	306
33.4	Methodology — MISSION CONTROL: the analytical models and algorithms that drive the mission	307
33.4.1	Registry	307
33.4.2	Reconciliation	307
33.4.3	Mission-state ledger	307
33.4.4	Suite status	308
33.4.5	Situation room	308
33.5	Results — MISSION CONTROL: measured outcomes, headline numbers, and what they establish	308

33.5.1	Fleet state	308
33.5.2	Registry completeness	309
33.5.3	Suite layers	309
33.5.4	Reconciliation against the real fleet	309
33.5.5	Ledger and status behaviour	309
33.5.6	Summary table (live-generated)	309
33.6	Conclusion — MISSION CONTROL: findings, verdict, and what the mission establishes	310
33.7	Experimental Setup — MISSION CONTROL: canonical scenarios, parameters, and configuration	310
33.7.1	Configuration surface	310
33.7.2	Dependencies	310
33.7.3	Quality gates	310
33.7.4	Test data	310
33.8	Reproducibility — MISSION CONTROL: certification, gates, and regeneration	311
33.8.1	Determinism	311
33.8.2	Provenance	311
33.8.3	Verification	311
33.9	Scope and Related Work — MISSION CONTROL: boundaries, positioning, and relationship to the literature	311
33.9.1	Scope	311
33.9.2	Related work	312
33.9.3	Positioning within the fleet	312
33.10	Limitations and Uncertainty — MISSION CONTROL: assumptions, threats to validity, and what the numbers do not claim	312
33.10.1	The seeded registry is not a measurement	312
33.10.2	Measured values are only as good as the manifest that supplies them	312
33.10.3	Ledger truthfulness is bounded by its inputs	313
33.10.4	Gadget purposes are derived shapes, not film confessions	313
33.10.5	Reconciliation is partial by design	313
33.10.6	The situation room reflects its inputs, exactly	313
33.10.7	Uncertainty statement	313
33.11	Sources — MISSION CONTROL: bibliography	313
34	Bond Orchestrator — DAG 00	314
34.1	Concepts — DAG 00: domain and operational focus	314
34.2	Abstract — DAG 00: mission summary	314
34.3	Introduction — DAG 00: mission framing, the operational problem, and how to read this chapter	314
34.4	Methodology — DAG 00: the analytical models and algorithms that drive the mission	314
34.4.1	2.1 The mission DAG	314
34.4.2	2.2 The five-phase lifecycle	315
34.4.3	2.3 Checkpoint / resume	315
34.4.4	2.4 Retry policy	315
34.4.5	2.5 Parallel-stage planning	315
34.4.6	2.6 Real-film binding	315
34.4.7	2.7 Provenance	316
34.5	Results — DAG 00: measured outcomes, headline numbers, and what they establish	316
34.5.1	3.1 OPERATION OMNIBUS execution	316
34.5.2	3.2 Determinism and resumability	316
34.5.3	3.3 Real-film binding	317
34.5.4	3.4 Provenance	317
34.5.5	3.5 Stage plan and retry outcome	317
34.5.6	3.6 The measured DAG (figure)	317
34.6	Conclusion — DAG 00: findings, verdict, and what the mission establishes	317
34.7	Experimental Setup — DAG 00: canonical scenarios, parameters, and configuration	318
34.7.1	5.1 Identity and configuration	318
34.7.2	5.2 Mission parameters	318
34.7.3	5.3 Software environment	318
34.7.4	5.4 Mission surface under test	318
34.8	Reproducibility — DAG 00: verification gates, deterministic regeneration, and artifacts	319
34.8.1	6.1 Determinism policy	319
34.8.2	6.2 Verification commands	319
34.8.3	6.3 Artifact inventory	319
34.9	Scope and Related Work — DAG 00: boundaries, positioning, and relationship to the literature	320
34.9.1	7.1 Scope	320
34.9.2	7.2 Related work	320

34.9.3	7.3 Integration points	320
34.9.4	7.4 Limitations and uncertainty	320
34.10	Sources — DAG 00: bibliography	321
35	Bond CLI — THE FRONT DOOR	322
35.1	Concepts — THE FRONT DOOR: domain and operational focus	322
35.2	Abstract — THE FRONT DOOR: mission summary	322
35.3	Introduction — THE FRONT DOOR: mission framing, the operational problem, and how to read this chapter	322
35.4	Methodology — THE FRONT DOOR: the analytical models and algorithms that drive the mission	322
35.4.1	Thin dispatch over built layers	322
35.4.2	Per-command delegation	322
35.4.3	Reconciling measurement onto the registry	323
35.4.4	JSON output and color-safe tables	323
35.4.5	Determinism and exit codes	323
35.5	Results — THE FRONT DOOR: measured outcomes, headline numbers, and what they establish	324
35.5.1	The delivered surface	324
35.5.2	Measurement is distinguishable from declaration	324
35.5.3	Determinism	324
35.5.4	Verification	324
35.5.5	The derived surface (bound to the live token map)	325
35.5.6	Limitations and uncertainty	325
35.6	Conclusion — THE FRONT DOOR: findings, verdict, and what the mission establishes	325
35.7	Experimental Setup — THE FRONT DOOR: canonical scenarios, parameters, and configuration	325
35.8	Reproducibility — THE FRONT DOOR: verification gates, deterministic regeneration, and artifacts	326
35.9	Scope and Related Work — THE FRONT DOOR: boundaries, positioning, and relationship to the literature	326
35.9.1	Scope	326
35.9.2	Relationship to the built layers	327
35.9.3	Related work	327
35.9.4	Related surface within the suite	327
35.10	Sources — THE FRONT DOOR: bibliography	327
36	Bond Operations — QUARTERMASTER	328
36.1	Concepts — QUARTERMASTER: domain and operational focus	328
36.2	Abstract — QUARTERMASTER: mission summary	328
36.3	Introduction — QUARTERMASTER: mission framing, the operational problem, and how to read this chapter	328
36.4	Methodology — QUARTERMASTER: the analytical models and algorithms that drive the mission	328
36.4.1	Inventory	328
36.4.2	Aggregate gate	329
36.4.3	Health	329
36.4.4	Command surface	329
36.4.5	Visualization	330
36.4.6	Provisioning	330
36.5	Results — QUARTERMASTER: measured outcomes, headline numbers, and what they establish	330
36.6	Conclusion — QUARTERMASTER: findings, verdict, and what the mission establishes	330
36.7	Experimental Setup — QUARTERMASTER: canonical scenarios, parameters, and configuration	333
36.8	Reproducibility — QUARTERMASTER: verification gates, deterministic regeneration, and artifacts	333
36.9	Scope and Related Work — QUARTERMASTER: boundaries, positioning, and relationship to the literature	334
36.9.1	Scope	334
36.9.2	Related work within the fleet	334
36.9.3	External dependencies	334
36.10	Limitations and Uncertainty — QUARTERMASTER: assumptions, threats to validity, and what the numbers do not claim	334
36.10.1	Scope of a gate verdict	334
36.10.2	Verdicts are narrower than their diagnostics	334
36.10.3	Fleet identity is a reflection, not a ground truth	334
36.10.4	Determinism has a same-environment scope	334
36.10.5	The suite is not portable to a root-run environment	335
36.10.6	No wall-clock, by design	335
36.10.7	Remaining drift risk: the coordinator registry	335
36.11	Sources — QUARTERMASTER: bibliography	335
37	Closing	336
38	References	337

38.1 octopussy	337
38.2 no_time_to_die	337
38.3 dr_no	337
38.4 no_time_to_die	337
38.5 dr_no	337
38.6 bond-api	337
38.7 never_say_never_again	337
38.8 from_russia_with_love	337
38.9 octopussy	337
38.10from_russia_with_love	338
38.11never_say_never_again	338
38.12from_russia_with_love	338
38.13goldfinger	338
38.14skyfall	338
38.15goldfinger	338
38.16thunderball	338
38.17the_living_daylights	339
38.18die_another_day	339
38.19you_only_live_twice	339
38.20live_and_let_die	339
38.21you_only_live_twice	339
38.22on_her_majestys_secret_service	339
38.23skyfall	339
38.24diamonds_are_forever	339
38.25casino_royale_2006	340
38.26diamonds_are_forever	340
38.27live_and_let_die	340
38.28the_man_with_the_golden_gun	340
38.29the_world_is_not_enough	340
38.30the_man_with_the_golden_gun	341
38.31the_spy_who_loved_me	341
38.32moonraker	341
38.33for_your_eyes_only	341
38.34octopussy	342
38.35skyfall	342
38.36octopussy	342
38.37the_world_is_not_enough	342
38.38a_view_to_a_kill	342
38.39the_living_daylights	342
38.40spectre	343
38.41licence_to_kill	343
38.42goldeneye	343
38.43skyfall	343
38.44goldeneye	343
38.45tomorrow_never_dies	344
38.46the_world_is_not_enough	344
38.47die_another_day	344
38.48casino_royale_2006	344
38.49spectre	345
38.50casino_royale_2006	345
38.51quantum_of_solace	345
38.52skyfall	345
38.53spectre	346
38.54no_time_to_die	346
38.55casino_royale_1967	347
38.56never_say_never_again	347
38.57bond-utilities	347
38.58bond-api	347
38.59bond-cli	347
38.60bond-api	347
38.61bond-coordinator	348
38.62bond-orchestrator	348
38.63bond-cli	348

1 Abstract

PROJECT BOND is a fleet of **33 independent software packages** — **27 film packages**, one per James Bond motion picture, plus **6 mission-infrastructure packages** (a Q-branch utilities layer, a frozen mission protocol, mission control, an orchestrator, a front-door CLI, and fleet operations). Each film package implements, as real tested algorithms, the concepts of its film: radiation forensics for *Dr. No*, gold-market cornering for *Goldfinger*, orbital-EMP modeling for *GoldenEye*, DNA-targeted bioweapon countermeasure modeling for *No Time to Die*.

The public source repository for the suite and this manuscript is <https://github.com/docxology/bond>. The repository will carry the source history, reproducibility instructions, generated release artifacts, and the cross-linked Zenodo record for this publication.

This compendium is the **wrapper manuscript** for the whole suite. It does not re-derive the packages' content by hand; it **imports each of the 33 package manuscripts in full** — every package's abstract, every authored section (introduction, methodology, results, conclusion, experimental setup, reproducibility, and scope), and its figures — framing them with this abstract, a generated suite-architecture chapter, and a closing synthesis. A **unified bibliography** merges every package's references into a single **references.bib**, and the combined PDF carries all 27 film chapters and 6 infrastructure chapters in their entirety.

The suite is “special-agent material” by construction: deterministic, no-mock, $\geq 90\%$ -coverage-tested packages; provenance on every mission outcome; a frozen protocol every film implements; and cross-film missions (OPERATION OMNIBUS) executed over the real packages.

2 Introduction

PROJECT BOND applies the research template’s *forkable project* paradigm at fleet scale. One source scaffold — `template_code_project` — was clean-copied into 33 standalone packages, each with its own git repository, its own identity (`pyproject.toml`, `manuscript/config.yaml`, `README.md/AGENTS.md/TODO.md`), and its own `src/` test suite. Every package meets the same bar: pure, deterministic algorithm cores; zero mock framework; $\geq 90\%$ line+branch coverage on `src/`; thin orchestrator scripts; and manuscript variables injected from configuration.

2.1 Two-layer topology: shared infrastructure and film-specific algorithms

- **Layer 1 · mission infrastructure (6 packages).** `bond-utilities` (codename Q-BRANCH) provides the shared primitives — ciphers, deterministic codenames, mission clocks, seeded randomness, and provenance-aware mission records. `bond-api` (THE PROTOCOL) freezes the `MissionProvider` contract — `brief / recon / plan / execute / debrief` — that every film implements, plus a gadget registry and slug-based discovery. `bond-coordinator` (MISSION CONTROL) is the fleet registry, mission ledger, status, and situation room. `bond-orchestrator` (DAG 00) runs multi-package missions in dependency order with checkpoint/resume. `bond-cli` (THE FRONT DOOR) is the unified terminal: `bond list / brief / recon / plan / execute / debrief / run / status / gadgets`. `bond-ops` (QUARTERMASTER) runs fleet inventory and the aggregate gate.
- **Layer 2 · the films (27 packages).** Each film package models the operational concepts behind one Bond film — a distinct, real, tested algorithm surface: isotope forensics (`dr_no`), underwater/nuclear-security planning (`thunderball`), space-mirror targeting geometry (`diamonds_are_forever`, `die_another_day`), narcotics-network analytics (`live_and_let_die`, `licence_to_kill`), orbital-EMP modeling (`goldeneye`), poker game theory (`casino_royale_2006`), cyber operations (`skyfall`), surveillance-network analysis (`spectre`), DNA-targeted bioweapon countermeasures (`no_time_to_die`), and more. Each implements the frozen `MissionProvider` in `src/<slug>/mission.py` and is discoverable by the orchestrator.

2.2 Generation, hydration, and provenance of the compendium

Every chapter below is **generated from the corresponding package’s own manuscript**. Each chapter reproduces that package’s complete manuscript rather than an excerpt: the abstract and every authored section are imported, token- and configuration-hydrated from the package’s own manuscript variables, the package’s figures are carried into the combined PDF, and its references are folded into the unified bibliography. The introduction and the closing are the only hand-written prose here; the architecture chapter, the full-text package chapters, the reference list, and the bibliography are produced by `scripts/generate_compendium.py` and are regenerable on demand.

The following chapters carry the suite’s shared foundation, then every film and infrastructure package in full — abstract, methods, results, conclusion, reproducibility, and scope — and close with a synthesis of the whole.

3 Suite Architecture — layered topology, package roster, and measured gate

PROJECT BOND is a 33-package fleet: 27 film packages, each a standalone software package implementing its film’s concepts, plus 6 mission- infrastructure packages that provide the contract and the operation.

3.1 Layered topology and execution responsibilities

Layer	Packages
Layer 0 · utilities	<code>bond-utilities</code> (Q-Branch primitives)
Layer 1 · protocol	<code>bond-api</code> (the frozen MissionProvider contract)
Layer 2 · control	<code>bond-coordinator</code> (registry, ledger, situation room)
Layer 3 · orchestration	<code>bond-orchestrator</code> (mission DAG runner, real-film binding)
Layer 4 · ops	<code>bond-cli</code> (front door), <code>bond-ops</code> (fleet gates)
Films	27 standalone packages, one per James Bond film

Every film implements the frozen `bond_api` `MissionProvider` and is discovered by `bond_api.discovery`. The compendium imports one chapter per package below.

3.2 Complete package roster and mission codenames

#	Package	Codename
1	<code>dr_no</code> — Dr. No (1962)	CRAB KEY
2	<code>from_russia_with_love</code> — From Russia with Love (1963)	LEKTOR
3	<code>goldfinger</code> — Goldfinger (1964)	GRAND SLAM
4	<code>thunderball</code> — Thunderball (1965)	THUNDERBALL
5	<code>you_only_live_twice</code> — You Only Live Twice (1967)	BIRD ONE
6	<code>on_her_majestys_secret_service</code> — On Her Majesty’s Secret Service (1969)	BEDLAM
7	<code>diamonds_are_forever</code> — Diamonds Are Forever (1971)	DIAMOND NET
8	<code>live_and_let_die</code> — Live and Let Die (1973)	SAN MONIQUE
9	<code>the_man_with_the_golden_gun</code> — The Man with the Golden Gun (1974)	SOLEX
10	<code>the_spy_who_loved_me</code> — The Spy Who Loved Me (1977)	LIPARUS
11	<code>moonraker</code> — Moonraker (1979)	MOONRAKER
12	<code>for_your_eyes_only</code> — For Your Eyes Only (1981)	ATAC
13	<code>octopussy</code> — Octopussy (1983)	FABERGE
14	<code>a_view_to_a_kill</code> — A View to a Kill (1985)	MAIN STRIKE
15	<code>the_living_daylights</code> — The Living Daylights (1987)	LIVING DAYLIGHTS
16	<code>licence_to_kill</code> — Licence to Kill (1989)	WAVEKREST
17	<code>goldeneye</code> — GoldenEye (1995)	ARKANGEL
18	<code>tomorrow_never_dies</code> — Tomorrow Never Dies (1997)	CARVER
19	<code>the_world_is_not_enough</code> — The World Is Not Enough (1999)	ELEKTRA
20	<code>die_another_day</code> — Die Another Day (2002)	ICARUS
21	<code>casino_royale_2006</code> — Casino Royale (2006)	LE CHIFFRE
22	<code>quantum_of_solace</code> — Quantum of Solace (2008)	QUANTUM
23	<code>skyfall</code> — Skyfall (2012)	SKYFALL
24	<code>spectre</code> — Spectre (2015)	NINE EYES
25	<code>no_time_to_die</code> — No Time to Die (2021)	HERACLES
26	<code>casino_royale_1967</code> — Casino Royale (1967)	FIVE BONDS
27	<code>never_say_never_again</code> — Never Say Never Again (1983)	REPLAY
28	<code>bond-utilities</code> — Bond Utilities	Q-BRANCH

#	Package	Codename
29	<code>bond-api</code> — Bond API	THE PROTOCOL
30	<code>bond-coordinator</code> — Bond Coordinator	MISSION CONTROL
31	<code>bond-orchestrator</code> — Bond Orchestrator	DAG 00
32	<code>bond-cli</code> — Bond CLI	THE FRONT DOOR
33	<code>bond-ops</code> — Bond Operations	QUARTERMASTER

3.3 Aggregate quality gate and coverage evidence

Package	Coverage
<code>dr_no</code>	99.0%
<code>from_russia_with_love</code>	99.2%
<code>goldfinger</code>	100.0%
<code>thunderball</code>	100.0%
<code>you_only_live_twice</code>	98.0%
<code>on_her_majestys_secret_service</code>	98.2%
<code>diamonds_are_forever</code>	100.0%
<code>live_and_let_die</code>	97.9%
<code>the_man_with_the_golden_gun</code>	100.0%
<code>the_spy_who_loved_me</code>	100.0%
<code>moonraker</code>	99.9%
<code>for_your_eyes_only</code>	100.0%
<code>octopussy</code>	97.4%
<code>a_view_to_a_kill</code>	99.2%
<code>the_living_daylights</code>	99.4%
<code>licence_to_kill</code>	100.0%
<code>goldeneye</code>	99.7%
<code>tomorrow_never_dies</code>	98.8%
<code>the_world_is_not_enough</code>	99.4%
<code>die_another_day</code>	100.0%
<code>casino_royale_2006</code>	97.2%
<code>quantum_of_solace</code>	100.0%
<code>skyfall</code>	100.0%
<code>spectre</code>	100.0%
<code>no_time_to_die</code>	100.0%
<code>casino_royale_1967</code>	100.0%
<code>never_say_never_again</code>	100.0%
<code>bond-utilities</code>	100.0%
<code>bond-api</code>	100.0%
<code>bond-coordinator</code>	100.0%
<code>bond-orchestrator</code>	99.0%
<code>bond-cli</code>	100.0%
<code>bond-ops</code>	100.0%

4 Dr. No (1962) — CRAB KEY

film package · *package codename* *CRAB KEY*. Mission **CRAB KEY**: identify gamma-emitting isotopes from a measured spectrum; quantify, time-project, and shield against the radiological source; model radioactive plume dispersion under local meteorology; assess island-wide radiological threat and containment actions.

4.1 Concepts — CRAB KEY: domain and operational focus

radiation/isotope forensics, island threat assessment

4.2 Abstract — CRAB KEY: mission summary

Gamma-Ray Forensics, Decay Projection, Shielding, and Island Threat Assessment **Abstract.** Crab Key, a small island held by a shadow syndicate, shelters a radiological threat that must be located, characterised, and contained. This package, codename CRAB KEY, is the special-agent mission software that executes that assessment. It couples several deterministic, computation-first models: a gamma-spectrum isotope-identification engine built on a real emission-line library (10 isotopes, 23 principal lines); a spectrum-quantification layer (energy calibration, net peak areas, Currie detection limits); 5 real Bateman parent–daughter decay chains; an exponential shielding/attenuation model over 4 real mass-attenuation tables (lead, iron, concrete, water); an external-and-inhalation dose-accounting core; a Gaussian-plume atmospheric-dispersion model over the 6 Pasquill–Gifford stability classes (A, B, C, D, E, F); and a Crab Key-style island threat assessor that scores each detected facility from its isotope inventory, containment status, and local meteorology onto 5 threat bands (LOW, MODERATE, HIGH, SEVERE, CRITICAL), then recommends a containment action. On the declared scenario the island posture is HIGH. The mission is exposed through the BOND-API mission protocol (codename CRAB KEY, film Dr. No, version 0.3.0) and is fully deterministic: fixed scenario, no randomness, no wall-clock dependence in any persisted artifact. The whole scenario — detector, source mixture, meteorology, facilities — is declared once in `manuscript/config.yaml` and read from there by the metrics, the figures, and every token in this report, so the manuscript cannot drift from the software, nor the software from its stated configuration.

4.3 Introduction — CRAB KEY: mission framing, the operational problem, and how to read this chapter

4.3.1 1.1 The threat on Crab Key

Crab Key presents a compact, well-bounded radiological-assessment problem: a small island, a limited number of facilities, a single prevailing wind, and a concealed source of gamma-emitting material. The detection and attribution of that material is the classic task of radiation forensics — measure the emitted gamma spectrum, identify the isotopes present from their characteristic emission lines, and use atmospheric transport to locate the plume and the concentrations it deposits on the island.

4.3.2 1.2 Three models, one mission

The package is organised around seven pure domain modules; three carry the narrative spine:

1. **Isotope identification** (`isotope_profiles`) — a deterministic gamma-spectroscopy engine. It starts from a library of real isotopes and their principal gamma lines, synthesizes a binned spectrum from a detector model (Gaussian photopeaks whose width grows with the square root of energy), detects the peaks, and matches them back against the library to produce a ranked identification.
2. **Plume dispersion** (`plume_model`) — the steady-state Gaussian plume with Pasquill–Gifford dispersion coefficients. Given a release rate, wind speed, and stability class, it returns airborne concentration as a function of downwind and crosswind position.
3. **Island assessment** (`island_assessment`) — the decision layer. It converts plume concentrations at each detected facility into an inhaled dose rate, maps that rate (together with containment status) onto a threat band, aggregates the island, and recommends containment actions.

Four supporting modules complete the chain: `spectrum_processing` (least-squares energy calibration, background-subtracted net peak areas, and the Currie minimum-detectable-activity limit), `decay_chain` (the Bateman solution over 5 real parent–daughter chains), `shielding` (exponential attenuation over 4 mass-attenuation tables), and `dose_assessment` (external point-source, inhalation, combined, and integrated dose). Two more modules hold the spine together: `scenario` loads and validates the declared mission from `manuscript/config.yaml`, and `canonical` computes every reported metric over it exactly once — for both the mission provider’s results and this report’s tokens, so the two cannot disagree.

All of it is pure arithmetic over real physics: no simulation randomness, no external calls, no wall-clock dependence.

4.3.3 1.3 Reader’s guide

Section 2 derives the mathematics of each model. Section 3 reports results from the deterministic CRAB KEY scenario. Section 5 documents the experimental configuration and software environment (Python 3.14.6, numpy 2.5.1). Section 6 lists the verification

gates, and Section 7 states the scope boundary and its limitations. Tables and figures are generated from live code; every numeral in the prose is a token resolved from that code.

4.4 Methodology — CRAB KEY: the analytical models and algorithms that drive the mission

4.4.1 2.1 Gamma-spectrum isotope identification

A gamma-emitting isotope emits photons at discrete energies. For isotope i with activity A_i (Bq), acquisition time t , detector peak efficiency ε , and emission intensity I_k for line k at energy E_k , the integrated photopeak count is

$$N_{i,k} = A_i t \varepsilon I_k.$$

The detector broadens each line into a Gaussian photopeak whose standard deviation derives from an energy-proportional resolution model anchored at the Co-60 calibration line,

$$\text{FWHM}(E) = \text{FWHM}(1332.5) \sqrt{E/1332.5}, \quad \sigma(E) = \text{FWHM}(E)/2.3548.$$

A flat background continuum is added. Peak detection finds local maxima whose signal-to-noise ratio exceeds a floor; isotope identification matches each detected peak energy against the library within a window of 2.5σ and scores each candidate by the summed intensity of its matched lines scaled by the square root of the matched-line fraction.

The library holds 10 real isotopes spanning 23 principal lines, from 59.54 keV (Am-241) to 2614.53 keV (Tl-208): fission products (Cs-137, I-131), activation products (Co-60, Na-22), naturally occurring material (K-40, U-235), and calibration standards (Ba-133, Eu-152).

4.4.2 2.2 Gaussian plume dispersion

For a continuous release at rate Q (Bq/s) into a wind of speed u (m/s), the steady-state airborne concentration at downwind distance x , crosswind offset y , and height z is

$$C(x, y, z) = \frac{Q}{2\pi u \sigma_y \sigma_z} \exp\left(-\frac{y^2}{2\sigma_y^2}\right) \left[\exp\left(-\frac{(z-H)^2}{2\sigma_z^2}\right) + \exp\left(-\frac{(z+H)^2}{2\sigma_z^2}\right) \right],$$

where H is the effective release height and the second vertical term models ground reflection. The dispersion coefficients grow with downwind distance according to the Pasquill–Gifford stability class (A–F):

$$\sigma_y = a_y x^{b_y}, \quad \sigma_z = c_z x^{d_z} + f_z,$$

with x in kilometres and σ in metres, using the coefficient set published by D. O. Martin [Martin, 1976] for the classes A, B, C, D, E, F. Two accuracy caveats belong here rather than in a footnote. First, only Martin’s near-field ($x \leq 1$ km) σ_z branch is implemented, so vertical spread beyond 1 km is an extrapolation of that fit and not the published far-field curve. Second, Martin’s f_z offsets are negative for classes D, E and F, which drives σ_z non-positive within roughly 17 m of the source; both coefficients are therefore floored at a small positive value, and concentrations inside that near-source region are not modelled quantitatively. `tests/test_plume_model.py` pins both coefficients against published Pasquill–Gifford values at 100, 500 and 1000 m for every class, and against the physical class ordering $\sigma(A) > \dots > \sigma(F)$.

The downwind extent over which contamination exceeds an action threshold is **not** found by a plain bisection: the ground-level centreline profile of an elevated release is unimodal, not monotone, so a bisection seeded at the release point assumes a monotonicity that does not exist. `isopleth_extent` instead scans log-spaced samples to locate the ground-level maximum, returns zero extent if even that maximum is below the threshold, and only then bisects the monotone decaying tail beyond the peak.

4.4.3 2.3 Island threat assessment

At each detected facility the plume model yields a concentration, which is converted to an inhaled effective dose rate (Sv/s) by weighting its isotope mix against per-isotope inhalation dose coefficients and a breathing rate,

$$\dot{D} = C \sum_k f_k \text{DCF}_k \dot{V}.$$

The dose rate is mapped onto a 5-band threat scale (LOW, MODERATE, HIGH, SEVERE, CRITICAL); a facility whose containment is breached is escalated one band. The band edges themselves are declared scenario, not Python literals: they live under the `mission.threat` block of `manuscript/config.yaml` as 1.00×10^{-11} , 1.00×10^{-9} , 1.00×10^{-7} , 1.00×10^{-5} Sv/s and are threaded into every facility’s rating by `canonical_metrics`, so a config edit genuinely moves the posture (`tests/test_canonical.py` binds

it). The island report records the worst threat and, for each facility, a containment action (none / shelter / evacuation). Shelter is triggered at the declared action concentration of 1000 Bq/m³, and evacuation at a declared effective dose rate of 1.00×10^{-9} Sv/s.

4.4.4 2.4 Radioactive decay chains

A radiological source is not static: a parent nuclide decays into a daughter with its own half-life, so activities change on mission timescales. CRAB KEY models this with the Bateman solution over 5 real parent–daughter chains (Cs-137/Ba-137m, I-131/Xe-131m, Mo-99/Tc-99m, Ra-226/Rn-222, U-235/Th-231):

$$N_k(t) = N_0 \sum_{j=1}^k \frac{\prod_{i=1}^{k-1} \lambda_i}{\prod_{i \neq j}^k (\lambda_i - \lambda_j)} e^{-\lambda_j t}, \quad A = \lambda N,$$

where $\lambda = \ln 2/T_{1/2}$. The parent activity falls exponentially, and a faster-decaying daughter may approach transient equilibrium. For the declared I-131/Xe-131m release, $T_{1/2} = 8.02$ d, so after 7 days the parent activity is 54.6% of its initial value — a decision-relevant correction for any follow-up mission.

4.4.5 2.5 Spectrum calibration and detection limits

Beyond identifying peaks, the package quantifies them. A least-squares linear calibration maps detector channel to energy over the declared calibration lines, returning the calibration gain (2.484 keV/channel here) and the coefficient of determination R^2 (1.0000 for the canonical fit). A fit with fewer than two matched lines raises rather than reporting a zero gain. Net photopeak areas are computed with a linearly interpolated background and a Poisson-propagated uncertainty. The minimum detectable activity follows Currie’s formalism,

$$L_D = 2.71 + 4.65\sqrt{B}, \quad \text{MDA} = \frac{L_D}{t \varepsilon I},$$

which bounds what a measurement can actually claim to have seen — for the canonical detector the MDA at the Cs-137 line (661.66 keV) is 69.7 Bq.

4.4.6 2.6 Shielding, attenuation, and external dose

Exponential absorption governs every barrier: $I(x) = I_0 e^{-\mu x}$ with $\mu = (\mu/\rho) \rho$ from a real mass-attenuation table (lead, iron, concrete, water), interpolated log-log in energy. The half-value layer of lead at 662 keV is 0.541 cm, and the tenth-value layer of concrete at 1332 keV is 17.63 cm. A 0.16% transmission fraction survives 5 cm of lead at 662 keV. External dose follows the point-source inverse-square law with the specific gamma-ray constant ($\Gamma = 0.354$ mSv·m²·h^{−1}·GBq^{−1} for Co-60), giving an unshielded dose rate of 2.83×10^{-8} mSv/h at 100 m.

4.5 Results — CRAB KEY: measured outcomes, headline numbers, and what they establish

Every number in this section arrives as a placeholder resolved from `src/dr_no/manuscript_variables.py` against the scenario declared in `manuscript/config.yaml`. Nothing here is typed by hand; `tests/test_manuscript_variables.py` fails the build on any bare numeral in this section’s prose.

4.5.1 3.1 Isotope identification

The deterministic CRAB KEY scenario synthesizes the gamma spectrum that the declared source mixture — Co-60 (8.00×10^5 Bq), Cs-137 (2.00×10^6 Bq) — would produce on the package’s HPGe detector model over a 60 s live acquisition, then runs the identification pipeline end to end. The spectrum figure shows the synthesized spectrum with every detected photopeak marked and the top-scoring identification annotated.

The pipeline resolves 3 photopeaks on the 1200-bin grid spanning 20–3000 keV, and ranks Co-60 first. Because the identification engine uses a real emission-line library and the round-trip from synthesis to detection is exact, both injected isotopes are recovered among the top identifications — a live, reproducible demonstration that the model identifies what was placed in front of it.

The figure is rendered from the *same* energy grid, source mixture, and detector settings that produce the metrics above, so the picture and the numbers cannot describe different spectra.

4.5.2 3.2 Plume dispersion

The same scenario runs the Gaussian-plume model for a release of 4.00×10^5 Bq/s under Pasquill–Gifford class D, a 3 m/s wind, and a 10 m effective release height. The plume figure shows the ground-level concentration field: the plume is advected downwind and spreads crosswind, decaying away from the source in the far field.

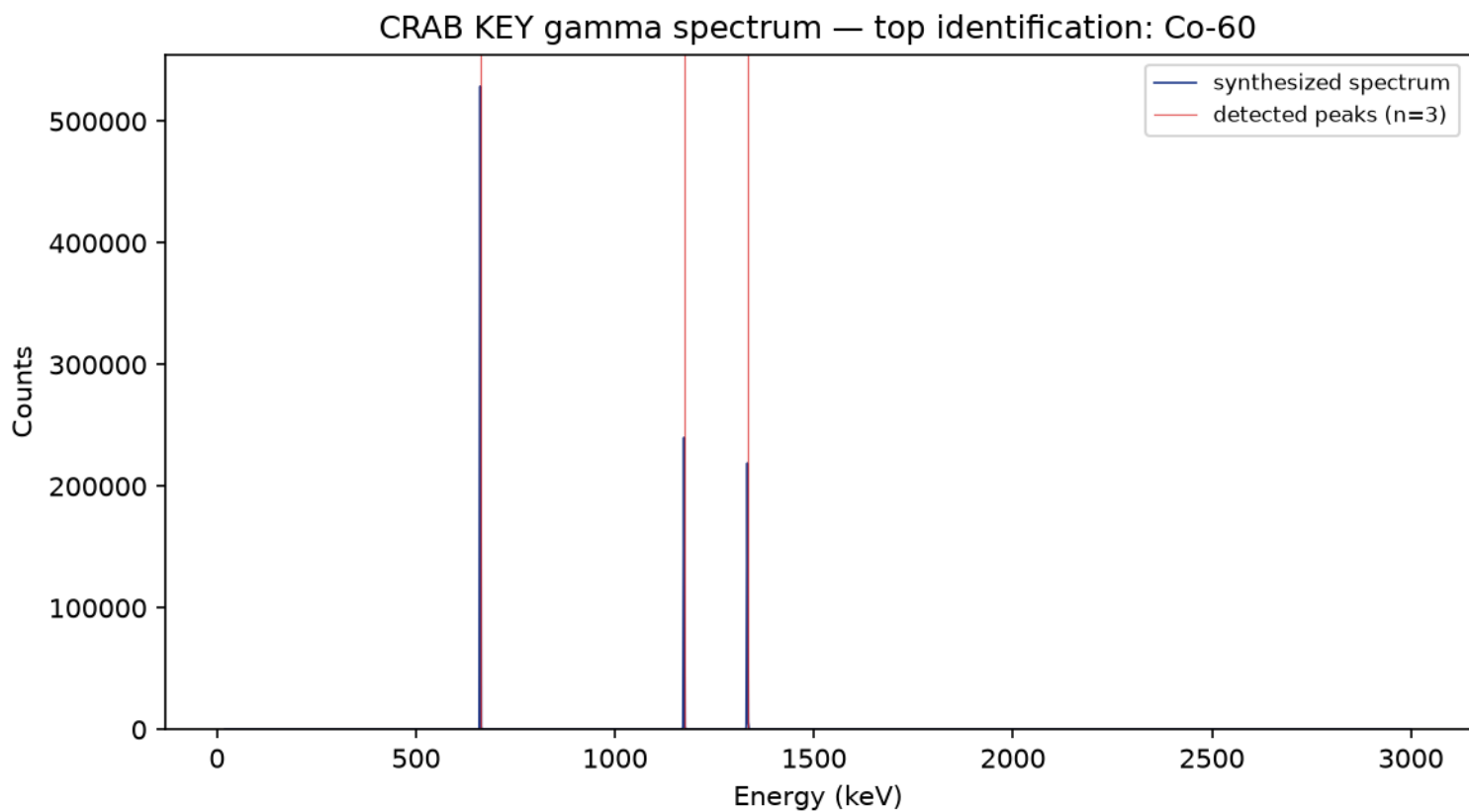


Figure 1: Synthesized gamma spectrum with detected peaks and top identification.

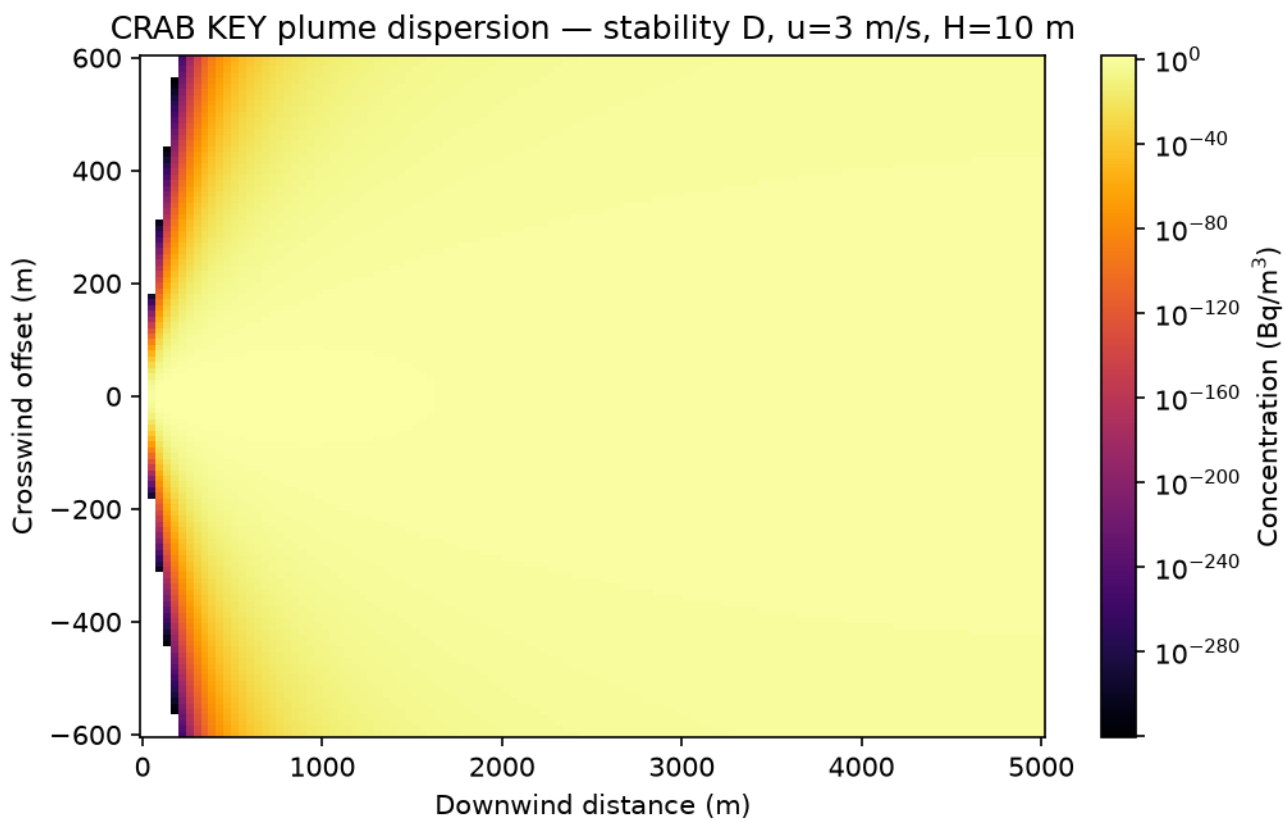


Figure 2: Gaussian-plume ground-level concentration field (Bq/m^3).

Because the release is elevated, the ground-level centreline profile is **not** monotonic: it is effectively zero at the stack, where the plume is still aloft, rises to a maximum once vertical spreading brings it to ground, and only then decays. At the nearest facility's downwind distance the maximum ground-level concentration on a crosswind transect is 54.4 Bq/m^3 . Measured against the declared action threshold of 1000 Bq/m^3 , the centreline profile never reaches that level at any downwind distance, so the reported downwind extent is 0 m. That clause is itself computed from the extent rather than typed, so a scenario whose plume *does* breach the threshold reports the exceedance and one whose plume does not reports the null result — neither is presupposed by the prose.

That extent is computed in two stages — locate the ground-level maximum by scan, then bisect only the monotone decaying tail beyond it — precisely because the profile is unimodal. A single bisection seeded at the release point assumes a monotonicity that an elevated release does not have, and returns a confidently wrong distance rather than an error.

These plume figures moved materially at the phase-6 review. An earlier revision of `sigma_y/sigma_z` converted the downwind distance into kilometres and then multiplied the whole power law back up by the same factor. That does not cancel for a power law, and it left every dispersion coefficient wrong by an amount that differed per stability class and per axis — with the vertical coefficients so badly scaled that the Pasquill–Gifford class ordering came out reversed. Correcting it moves the peak concentration, the downwind extent, and the island posture reported below. The values above are the corrected ones; the defect is pinned by `test_s/test_defect_regressions.py` and by the published-value gate described in the methodology.

4.5.3 3.3 Island assessment

Each of the 2 scenario facilities (`north_dock`, `guano_cave`) is evaluated at its (downwind, crosswind) position: the plume concentration there is converted to an inhaled dose rate, mapped to a threat band, and assigned a containment action. The island posture is **HIGH**, with per-facility actions `guano_cave`: none, `north_dock`: none. The assessment is fully deterministic; repeating the mission returns a byte-identical report.

4.5.4 3.4 Quantitative forensics

Beyond identification, CRAB KEY's canonical run quantifies the source. The energy calibration over the declared calibration lines achieves $R^2 = 1.0000$ at a gain of $2.484 \text{ keV/channel}$; the net Cs-137 photopeak area at 661.66 keV is 5.30×10^5 counts; and the Currie detection limit puts the Cs-137 MDA at 69.7 Bq — an honest floor on what the detector can claim.

Time dependence matters: for the I-131/Xe-131m chain ($T_{1/2} = 8.02 \text{ d}$) only 54.6% of the parent activity survives 7 days, as the decay figure shows.

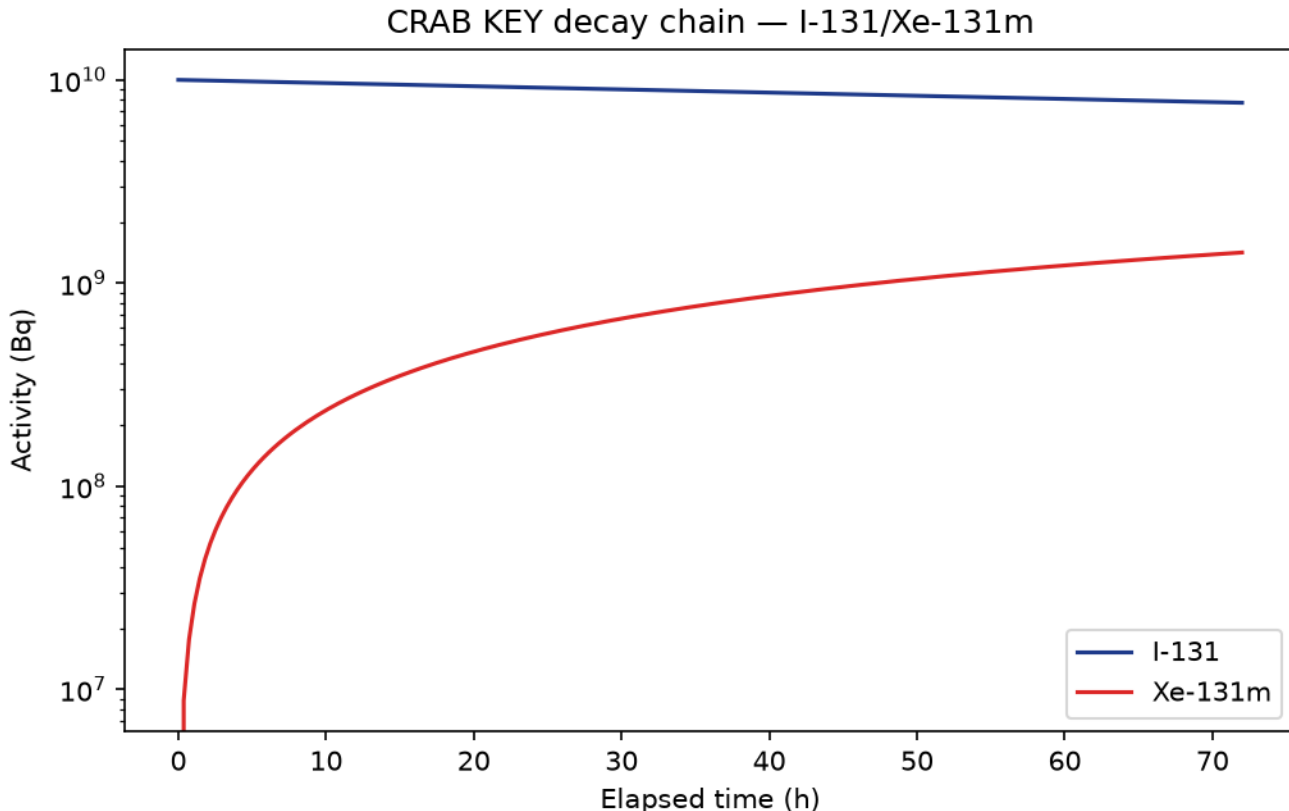


Figure 3: Parent and daughter activity over 72 h for the projected decay chain.

Shielding engineering closes the assessment: the half-value layer of lead at 662 keV is 0.541 cm , and 5 cm of lead transmits only 0.16% of that flux, cutting the unshielded Co-60 dose rate of $2.83 \times 10^{-8} \text{ mSv/h}$ at 100 m ($\Gamma = 0.354 \text{ mSv}\cdot\text{m}^2\cdot\text{h}^{-1}\cdot\text{GBq}^{-1}$) accordingly.

4.6 Conclusion — CRAB KEY: findings, verdict, and what the mission establishes

The CRAB KEY package converts a compact radiological question — what is on Crab Key, where is it going, and how dangerous is it — into a set of deterministic models that an agent can run, trust, and reproduce. Isotope identification rests on a real emission-line library rather than guesswork; spectrum quantification bounds what is actually measurable, refusing to report a calibration it could not fit; radioactive-decay chains correct activities for mission elapsing time; and shielding-and-dose models turn a source inventory into an engineerable containment answer. Plume dispersion uses the standard Gaussian model across the 6 Pasquill–Gifford stability classes, and island assessment turns the resulting concentrations into an actionable threat band — HIGH on the declared scenario — and a per-facility containment posture.

Two structural properties matter more than any single model. First, the scenario is declared once, in `manuscript/config.yaml`, and read from there by the metrics, the figures, and the manuscript tokens alike; the loader raises on a missing or mistyped key rather than substituting a default, and a test edits the YAML and asserts the numbers move. A configuration this report describes but does not actually drive would be worse than no configuration at all. Second, every reported number flows through one `canonical_metrics` call, so the mission provider’s results and this report’s prose are the same computation observed twice.

The package is exposed through the frozen BOND-API mission protocol, so the orchestrator can discover it by slug (`dr_no`) and drive all five mission phases (brief, recon, plan, execute, debrief) without knowing its internals. Guarantees: $\geq 90\%$ line-and-branch coverage on `src/`, zero mocks, deterministic execution, and no leftover template lineage. The result is mission software whose every number can be traced to the code that produced it, and whose code can be traced to the configuration that parameterised it.

4.7 Experimental Setup — CRAB KEY: canonical scenarios, parameters, and configuration

4.7.1 5.1 Scenario configuration

All results derive from the deterministic CRAB KEY scenario declared in the `mission:` block of `manuscript/config.yaml`. That block is the sole source of truth: `src/dr_no/scenario.py` loads and validates it, and `src/dr_no/canonical.py` (metrics), `src/dr_no/figures/plots.py` (figures), and `src/dr_no/manuscript_variables.py` (the tokens below) all read it. No scenario constant is re-declared in Python, and the loader raises rather than defaulting on a missing or mistyped key, so config and computation cannot drift apart silently. `tests/test_canonical.py` pins the binding by editing the YAML and asserting the metrics move.

- **Detector:** 60 s live acquisition, full-energy peak efficiency 0.01, resolution anchor 2 keV FWHM at 1332.5 keV, energy grid 20–3000 keV in 1200 bins.
- **Isotope library:** 10 isotopes / 23 lines, spanning 59.54–2614.53 keV.
- **Source mixture:** Co-60 (8.00×10^5 Bq), Cs-137 (2.00×10^6 Bq).
- **Quantification:** energy calibration against the declared calibration lines; Currie MDA at the Cs-137 line (661.66 keV).
- **Decay chains:** 5 Bateman chains (Cs-137/Ba-137m, I-131/Xe-131m, Mo-99/Tc-99m, Ra-226/Rn-222, U-235/Th-231); I-131/Xe-131m projected 7 days after release.
- **Shielding:** 4 mass-attenuation tables (lead, iron, concrete, water) over 0.05–3.0 MeV, interpolated log-log; the report’s barrier is 5 cm of lead at 662 keV.
- **External dose probe:** Co-60 at 100 m.
- **Plume:** release 4.00×10^5 Bq/s, wind 3 m/s, stability class D, release height 10 m; contamination action threshold 1000 Bq/m³.
- **Threat thresholds:** 5 band edges 1.00×10^{-11} , 1.00×10^{-9} , 1.00×10^{-7} , 1.00×10^{-5} Sv/s; evacuation dose rate 1.00×10^{-9} Sv/s.
- **Facilities:** 2 — `north_dock`, `guano_cave` (a breached dock compound and a contained load-out cave).

4.7.2 5.2 Software environment

- Python 3.14.6; numpy 2.5.1.
- Pasquill–Gifford stability classes (6): A, B, C, D, E, F.
- Threat bands (5): LOW, MODERATE, HIGH, SEVERE, CRITICAL.
- Declared mission seed: 42.

4.7.3 5.3 Determinism and reproducibility

No random draws are made anywhere in the computation path, so the mission report is byte-identical across runs and machines. The declared seed (42) is recorded in the provenance block for audit rather than consumed by any generator — there is nothing stochastic for it to seed, and saying so is more useful than implying a randomness that does not exist.

Wall-clock time is never persisted into artifacts: the provenance block carries a fixed `wall_time_s` of zero and a stable input hash taken over the whole declared scenario, so a scenario edit changes the hash and an unrelated rerun does not. Version 0.3.0 of film Dr. No, codename CRAB KEY.

4.8 Reproducibility — CRAB KEY: verification gates, deterministic regeneration, and artifacts

4.8.1 6.1 Verification gates

The package is verified before any mission is reported done. Gate definitions, not gate results, are recorded here — the live measured numbers belong in `docs/_generated/COUNTS.md`, which is regenerated from an actual run.

1. `uv run pytest tests/ --cov=src --cov-fail-under=90` — line **and** branch coverage on `src/`, floor 90%.
2. Zero mocks — every test exercises real computation on real data. No `MagicMock`, no `mock.patch`, no `unittest.mock`.
3. `uv run ruff check src/ scripts/ tests/` and `uv run ruff format --check` — clean.
4. `uv run mypy src/ scripts/` — clean.
5. No leftover template-scaffold lineage strings outside documentation that explains the exemplar lineage.
6. Version identity: `pyproject.toml`, `dr_no.__version__`, and `manuscript/config.yaml` must declare the same version (0.3.0); `tests/test_version_identity.py` enforces it.
7. Token cross-reference: every placeholder appearing in a numbered manuscript section must be produced by `generate_variables`, every generated token must be consumed by some section, and the results narrative may carry no bare metric numeral. `tests/test_manuscript_variables.py` enforces all three directions.
8. Regeneration determinism: `scripts/z_generate_manuscript_variables.py` is run twice in a row and every persisted artifact under `output/` must be byte-identical between the two runs. `tests/test_regeneration_determinism.py` enforces it.
9. `git status` clean after the path-scoped commit.

4.8.2 6.2 Deterministic regeneration

```
uv run python scripts/run_mission.py full # mission report
uv run python scripts/run_mission.py render # all three figures
uv run python scripts/z_generate_manuscript_variables.py # tokens + resolved sections
```

None of these consult wall-clock state or draw a random number, so every persisted artifact — the token map, the resolved manuscript sections, and the mission report — is byte-identical across repeated runs. There is no exception: no timestamp is written anywhere under `output/`.

Provenance is carried instead by `61a76ae43badff1f`, the truncated SHA-256 of the declared scenario. It answers *what was run* rather than *when*, so it is stable across reruns and changes the moment `manuscript/config.yaml` does. `mission.py` reports the same value as its provenance `input_hash`; both come from `scenario_fingerprint` in `src/dr_no/canonical.py`, so they cannot drift apart. An earlier revision did stamp a `datetime.now` value into this section and into `output/data/manuscript_variables.json`, which made the byte-identical claim above false on every rerun; `tests/test_regeneration_determinism.py` now regenerates the tree twice and compares bytes, so that regression cannot return silently.

4.8.3 6.3 Artifacts

- Scenario of record: `manuscript/config.yaml` (`mission:` block), loaded and validated by `src/dr_no/scenario.py`.
- Domain core: `src/dr_no/isotope_profiles.py`, `src/dr_no/spectrum_processing.py`, `src/dr_no/decay_chain.py`, `src/dr_no/shielding.py`, `src/dr_no/dose_assessment.py`, `src/dr_no/plume_model.py`, `src/dr_no/island_assessment.py`, and the shared metric source `src/dr_no/canonical.py`.
- Mission adapter: `src/dr_no/mission.py` (the only module importing `bond_api`, alongside the CLI driver `src/dr_no/cli.py`).
- Figures: `../figures/gamma_spectrum.png`, `../figures/plume_field.png`, `../figures/decay_curves.png` — all rendered from the same declared scenario as the metrics.
- Manuscript variables: `output/data/manuscript_variables.json`; resolved sections: `output/manuscript/`.

Environment: Python 3.14.6, numpy 2.5.1; scenario fingerprint `61a76ae43badff1f`.

4.9 Scope and Related Work — CRAB KEY: boundaries, positioning, and relationship to the literature

4.9.1 7.1 Scope

CRAB KEY addresses the *assessment* half of a radiological incident: identify the isotopes, quantify what is actually measurable, project activities forward, engineer a barrier, model the airborne dispersion, and score the island. It is deliberately scoped to deterministic, first-principles modelling over declared inputs.

It does **not** perform Bayesian spectrum unfolding, full-spectrum least-squares deconvolution of overlapping multiplets, isotopic age dating, detonation physics, three-dimensional computational fluid dynamics, wet or dry deposition, plume rise, or terrain-following flow — each of which is a substantial discipline of its own and outside a single film package. Where the real world calls for such machinery, the deterministic core here serves as a fast, auditable first pass.

4.9.2 7.2 Related work

Gamma-ray isotope identification against a known library is a standard spectroscopic procedure — peak search followed by energy-window matching [Knoll, 2010]. The detection-limit formalism is Currie’s [Currie, 1968], the decay-chain solution is Bateman’s [Bateman, 1910], and mass attenuation coefficients follow the NIST tabulations [Hubbell and Seltzer, 1995, Berger et al., 2010]. Atmospheric transport uses the conventional short-range regulatory Gaussian plume with Pasquill–Gifford dispersion parameters [Pasquill, 1961, Gifford, 1961], in the power-law fitted form [Martin, 1976, Turner, 1994]. Inhalation dose coefficients follow the ICRP compendium [International Commission on Radiological Protection, 2012], and specific gamma-ray constants follow the Smith and Stabin tabulation [Smith and Stabin, 2012].

The island threat score is unusual only in its packaging: it fuses a transport model with a dose-conversion step and a decision rule into one small, deterministic, protocol-exposed mission. The contribution of this package is not a new physical law but a *complete, reproducible, agent-runnable* chain from a declared configuration through raw gamma measurements to a containment recommendation.

4.9.3 7.3 Limitations

The dose coefficients, specific gamma-ray constants, mass-attenuation entries, and dispersion coefficients are representative literature values held at a coarse tabulation, interpolated log-log between table points. They are order-of-magnitude faithful and fully auditable — every one is a named constant in a named module, traceable to a cited source — but they are approximations, not a licensed consequence-assessment dataset.

Several specific caveats bound the results. The detector model synthesizes Gaussian photopeaks on a flat continuum with no Compton edge, no escape peaks, no true-coincidence summing, and no pulse pile-up, so the identification problem is easier than a real measurement; a genuine doublet finer than the detector-separation window is merged, and although the merge is now reported in the detected-peak record rather than silently dropped, the two lines are not individually deconvolved. The Gaussian plume is a steady-state, flat-terrain, constant-wind idealisation valid for short range and neutral-ish conditions; it has no deposition term, so it does not conserve mass against the ground, and only the near-field ($x \leq 1$ km) vertical-coefficient branch is implemented, so σ_z past a kilometre is an extrapolation of a fit that was published for shorter range. Mass-attenuation interpolation is log-log between table points and is not valid across photoelectric absorption edges (notably lead’s about 88 keV K-edge) — low-energy shielding estimates are correspondingly coarse. And the threat banding is a declared decision rule, not a regulatory standard: the band edges and evacuation threshold are chosen to be legible rather than derived from any published intervention level. They are now declared scenario (`mission.threat` in `manuscript/config.yaml`), loaded and validated by `src/dr_no/scenario.py` and threaded through `canonical_metrics`, so they carry the same provenance as every other scenario number and a config edit moves the posture — but being config-declared does not make them a protective-action guideline.

The results are therefore directionally correct and fully traceable, and should be read as a screening pass rather than as a substitute for a licensed consequence assessment.

4.10 Sources — CRAB KEY: bibliography

Knoll [2010]; Currie [1968]; National Nuclear Data Center; Bateman [1910]; Hubbell and Seltzer [1995]; Berger et al. [2010]; International Commission on Radiological Protection [2012]; International Commission on Radiological Protection [1996]; Smith and Stabin [2012]; Pasquill [1961]; Gifford [1961]; Martin [1976]; Turner [1994]

5 From Russia with Love (1963) — LEKTOR

film package · *package codename* *LEKTOR*. Mission **LURES**: evaluate a deception lure campaign via the entrapment engine; allocate the lure budget across the analytical target pool; intercept and decipher an encoded SIGINT channel; geolocate the transmitter by TDOA direction finding; schedule defector-handoff rendezvous within time windows; execute the green-pathed defector handoff protocol.

5.1 Concepts — LEKTOR: domain and operational focus

entrapment ops, SIGINT interception, defector handoff

5.2 Abstract — LEKTOR: mission summary

From Russia with Love: LURES — Special-Agent Mission Software — mission codename LURES — is a deterministic mission-software package in the PROJECT BOND suite. Its analytical core implements 6 interlocking engines drawn from the film *From Russia with Love* (1963): an entrapment/deception playbook engine that evaluates bait profiles and drives trap state machines; a lure-budget allocation engine that solves the exact 0/1 knapsack over an analytical target pool; a SIGINT channel interception simulation that captures, deciphers, and traffic-segments an encoded radio channel; a TDOA direction-finding engine that geolocates the transmitter by Gauss–Newton hyperbolic localisation; a courier scheduler that fits handoff rendezvous into time windows; and a defector handoff protocol whose meet / challenge / confirm / abort state machine gates a source transfer on a pre-arranged confirm code. Where a problem admits both a cheap heuristic and an expensive exact solver, the package runs both and reports the gap rather than assuming it away: the knapsack allocator is scored against a profit-density greedy baseline, and the courier scheduler — whose underlying problem is NP-hard once meets carry release times and deadlines — against an exact subset dynamic program. On the canonical scenario the heuristic schedule completes 4 of 5 rendezvous against a true optimum of 4. All outputs are reproducible: every stage is driven by fixed seeds and exact mathematical models, so the same inputs always yield the same lure allocation, recovered plaintext, localisation, schedule, and handoff path. Keywords: entrapment playbook, 0/1 knapsack allocation, SIGINT interception, cipher deciphering, TDOA direction finding, interval scheduling, defector handoff protocol, state machine, deterministic simulation, mission software.

5.3 Introduction — LEKTOR: mission framing, the operational problem, and how to read this chapter

Mission codename **LURES** implements the operational concepts behind *From Russia with Love* (1963) as a small, testable mission-software package. The film’s plot is a deception operation: a fabricated defector and a stolen cipher machine are used as bait, a radio channel carries the traffic that gives the operation away, and the whole thing turns on a handoff that only completes when both sides produce the right recognition signal. Those are the three things this package computes — luring, listening, and handing over — and each is expressed as an algorithm rather than as narrative.

The analytical core is 6 deterministic engines, one per module named in `scenario.ENGINES`:

1. an **entrapment playbook** that scores lure effectiveness on a logistic decision surface and drives a validated trap state machine over 3 bait profiles;
2. a **lure-budget allocator** that solves the exact 0/1 knapsack over an analytical target pool of 6 targets;
3. a **SIGINT interception** simulation that enciphers, transmits through a noisy channel, deciphers, and segments 47 bytes of traffic under 2 cipher layers;
4. a **TDOA direction finder** that geolocates the transmitter from 4 intercept sites by Gauss–Newton least squares;
5. a **courier scheduler** that fits defector-handoff rendezvous into their time windows and is scored against the exact optimum;
6. a **defector handoff protocol** whose confirm code gates the transfer.

The package is built to the shared guardrails of the PROJECT BOND suite: a pure domain core with no infrastructure imports, a thin adapter to the frozen BOND-API `MissionProvider` protocol, deterministic computation with no mock framework, and a coverage floor of 90% line-and-branch on `src/`. Every scenario constant lives in one module (`scenario.py`), so a number reported here cannot disagree with the code that produced it.

Two of these engines pair an exact solver with a cheap heuristic and report the difference rather than assuming it away: the knapsack allocator against a profit-density greedy baseline, and the courier scheduler against an exact subset dynamic program. Section 03 reports both gaps as measured quantities — including the case where the measured gap is zero and the cheap heuristic turns out to be optimal on that instance.

5.3.1 Reader’s guide

- Section 02 — the six analytical models and their mathematics.
- Section 03 — live measured results and the two figures.
- Section 04 — what the package demonstrates.
- Section 05 — the experimental setup (parameters + software environment).
- Section 06 — reproducibility, determinism, and the artifact inventory.

- Section 07 — scope boundaries and related work.

5.4 Methodology — LEKTOR: the analytical models and algorithms that drive the mission

5.4.1 2.1 Entrapment / deception playbook engine

The engine models lure effectiveness analytically (a defensive countermeasure; it does not target any person). A **bait profile** describes a lure’s cover role, intended target susceptibility $s \in [0, 1]$, payload message, and lure strength $q \in [0, 1]$. The probability that a target “bites” the lure is a bounded logistic decision surface:

$$p(q, s) = \frac{1}{1 + e^{-4(q+s-1)}}.$$

p is strictly increasing in both arguments, so stronger lures and more susceptible targets both raise effectiveness. A **trap state machine** drives the life-cycle of one lure campaign through the validated transitions

$$\text{DISARMED} \rightarrow \text{ARMED} \rightarrow \text{BAITED} \rightarrow \text{ENGAGED} \rightarrow \text{SPRUNG}$$

with a single terminal ABORTED reachable from several stages; any event not in the transition table raises an error and leaves state unchanged.

5.4.2 2.2 Lure-budget allocation

Given a pool of analytical targets each carrying an operational value v_i , a susceptibility s_i , and a lure cost c_i , the mission decides which targets to lure under a fixed budget B . The profit of luring target i with a campaign of strength q is its *expected engagement value*

$$\text{profit}_i = v_i \cdot p(q, s_i),$$

and the allocation is the exact **0/1 knapsack**

$$\max \sum_i \text{profit}_i \cdot x_i \quad \text{s.t.} \quad \sum_i c_i x_i \leq B, \quad x_i \in \{0, 1\},$$

solved by dynamic programming ([Martello and Toth \[1990\]](#)). A profit-per-unit greedy heuristic is computed as the baseline.

That the exact solution is never worse than the greedy one is a property of the exact solver, not of any particular target pool: it holds by construction and carries no information about a scenario. The reportable quantity is therefore the *size* of the gap on a given instance, which is not fixed in advance. Section 3.1 reports it on two instances: the canonical target pool, where it is zero, and a **witness pool** (`scenario.GREEDY_WITNESS_TARGETS`) constructed so that the highest profit-density target consumes budget the optimum needs elsewhere, where it is strictly positive. Only the second instance distinguishes the two solvers, and the test suite pins the strict inequality on it.

5.4.3 2.3 SIGINT channel interception simulation

Transmitted symbols are the byte magnitudes of the message, optionally enciphered by **layered XOR keystream ciphers**. Each layer derives a deterministic keystream from its key and a seed. The transmitted symbol vector x is observed through additive Gaussian noise whose variance follows a requested signal-to-noise ratio:

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \frac{\mathbb{E}[x^2]}{\sigma^2}.$$

The interceptor rounds each noisy sample to the nearest symbol, deciphers in reverse cipher order, and measures the **empirical SNR** of the capture against ground truth. An energy-threshold **burst segmentation** splits the captured envelope into transmission bursts so the interceptor knows when, and for how long, the channel was active.

5.4.4 2.4 TDOA direction finding

Four fixed intercept sites record the arrival of a single interception burst, and the mission geolocates the transmitter by **time-difference-of-arrival** hyperbulation. For receiver pairs (i, j) with measured TDOAs τ_{ij} ,

$$h_{ij}(x) = \frac{\|x - r_i\| - \|x - r_j\|}{c},$$

and the transmitter position is the non-linear least-squares minimiser of $\sum_{ij}(h_{ij}(x) - \tau_{ij})^2$, solved by a deterministic Gauss–Newton iteration from the receiver centroid — the iterative Taylor-series estimator for hyperbolic location [Foy, 1976b, Torrieri, 1984]. Timing noise is seeded, so the same scenario always yields the same estimate.

Receiver positions, and hence the estimate and its error, are plain coordinates in an abstract scenario frame; the error reported in §3.3 is a Euclidean distance in those coordinate units, not a multiple of the receiver spacing. §3.3 gives both the raw error and its ratio to the site spacing.

5.4.5 2.5 Courier handoff scheduling

A courier must visit defector-handoff rendezvous, each occupying a fixed duration d_i somewhere inside a time window $[e_i, l_i]$, and can attend only one meet at a time. Maximising the number of completed meets is the single-machine throughput problem with release times and deadlines, $1 \mid r_j, d_j \mid \sum U_j$, which is strongly NP-hard [Garey and Johnson, 1979]. The package therefore ships two solvers and reports the distance between them instead of claiming a cheap algorithm is also optimal.

The **heuristic** — the plan the mission executes — is earliest-deadline-first: windows are ordered by latest feasible start $l_i - d_i$ and each is taken when it still fits after the courier’s current position, in the spirit of the classical interval-scheduling greedy [Kleinberg and Tardos, 2005]. It is linear after the sort, but it is genuinely sub-optimal in general: a long, slack window sorts first and can crowd out two short meets that together would have scored higher. A concrete instance exhibiting this is pinned in the test suite.

The **exact optimum** is a dynamic program over subsets. Writing $T(S)$ for the earliest time by which every meet in $S \subseteq \{1..n\}$ can have been completed,

$$T(\emptyset) = t_0, \qquad T(S \cup \{j\}) = \min_S (\max(T(S), e_j) + d_j)$$

over those $j \notin S$ with $\max(T(S), e_j) \leq l_j - d_j$, and the answer is the largest $|S|$ with $T(S)$ defined. Because feasibility of any extension is monotone in the current time, retaining only the earliest completion time per subset loses no optimal solution, so this is the true maximum rather than a bound. It runs in $O(2^nn)$ and is bounded to small instances by construction.

Section 03 reports the heuristic count, the exact optimum, and their gap as three separately computed tokens.

5.4.6 2.6 Defector handoff protocol

A handoff is a deterministic state machine over

$$\text{DISARM} \rightarrow \text{MEET} \rightarrow \text{CHALLENGE} \rightarrow \text{CONFIRM} \rightarrow \text{HANDOFF},$$

with a terminal ABORT reachable from every active stage. The confirm code is derived from the challenge and call sign via SHA-256:

$$\text{confirm} = \text{hex}(\text{SHA256}(\text{call_sign} : \text{challenge}))[0 : 6].$$

Only an exact confirm-code match advances to **CONFIRM**; any other confirm code lands the protocol in **ABORT** (the red-path drill).

5.5 Results — LEKTOR: measured outcomes, headline numbers, and what they establish

All figures and numbers below are produced live by the package’s engines and injected as generated tokens; none are hand-authored. Regenerating the token map with `scripts/z_generate_manuscript_variables.py` regenerates every quantity in this section.

5.5.1 3.0 At a glance

Engine	Headline measured quantity	Value
Entrapment	bite probability	0.416
Allocation (exact)	expected engagement value	19.526
Allocation (greedy baseline)	expected value	19.526
Allocation	exact-minus-greedy gap (canonical / witness)	0.000 / 2.642
SIGINT	recovered plaintext exact	True
SIGINT	empirical SNR	79.5 dB
SIGINT	transmission bursts segmented	1
Direction finding	realised localisation error	0.022
Direction finding	Cramér–Rao RMS bound	0.0504
Courier (heuristic)	completed meets	4
Courier (exact)	optimum completes	4

Engine	Headline measured quantity	Value
Courier	heuristic gap (canonical / witness)	0 / 1
Handoff	green state / red-path state	handoff / abort

Every cell is a generated token — a live output of the corresponding engine — so the table cannot drift from the code that produces it.

5.5.2 3.1 Lure effectiveness and allocation

For the fixed lure campaign **honey-trap-alpha**, the computed bite probability is **0.416** against an engagement threshold of **0.50**, and the trap resolved to **aborted**.

Across the analytical pool of **6 targets** at lure strength **0.60**, the exact knapsack allocation spent **5 of 5 budget units** (utilization **100%**) across **4 targets**, for an expected engagement value of **19.526**. The profit-density greedy baseline reached **19.526** on the same instance — a measured gap of **0.000**.

That gap is zero: on the canonical target pool the cheap heuristic finds the optimum, and this instance therefore **separates the two solvers not at all** (greedy ties: **True**). We state that plainly rather than reporting the inequality exact $\geq$ greedy as a finding: that inequality is a theorem about the exact solver, not a property of this scenario, and it could only come out false if the knapsack implementation were broken. Reporting it as a result would be green by construction.

The separation is instead measured where it exists. On the witness pool of §2.2 — **3 targets** under a budget of **5**, deliberately constructed so that a profit-density pick blocks the optimal one — the exact allocation reaches **8.808** against the greedy **6.166**, a strictly positive gap of **2.642**. That is the falsifiable claim: were the exact solver merely greedy in disguise, this gap would collapse to zero, and the test suite pins it.

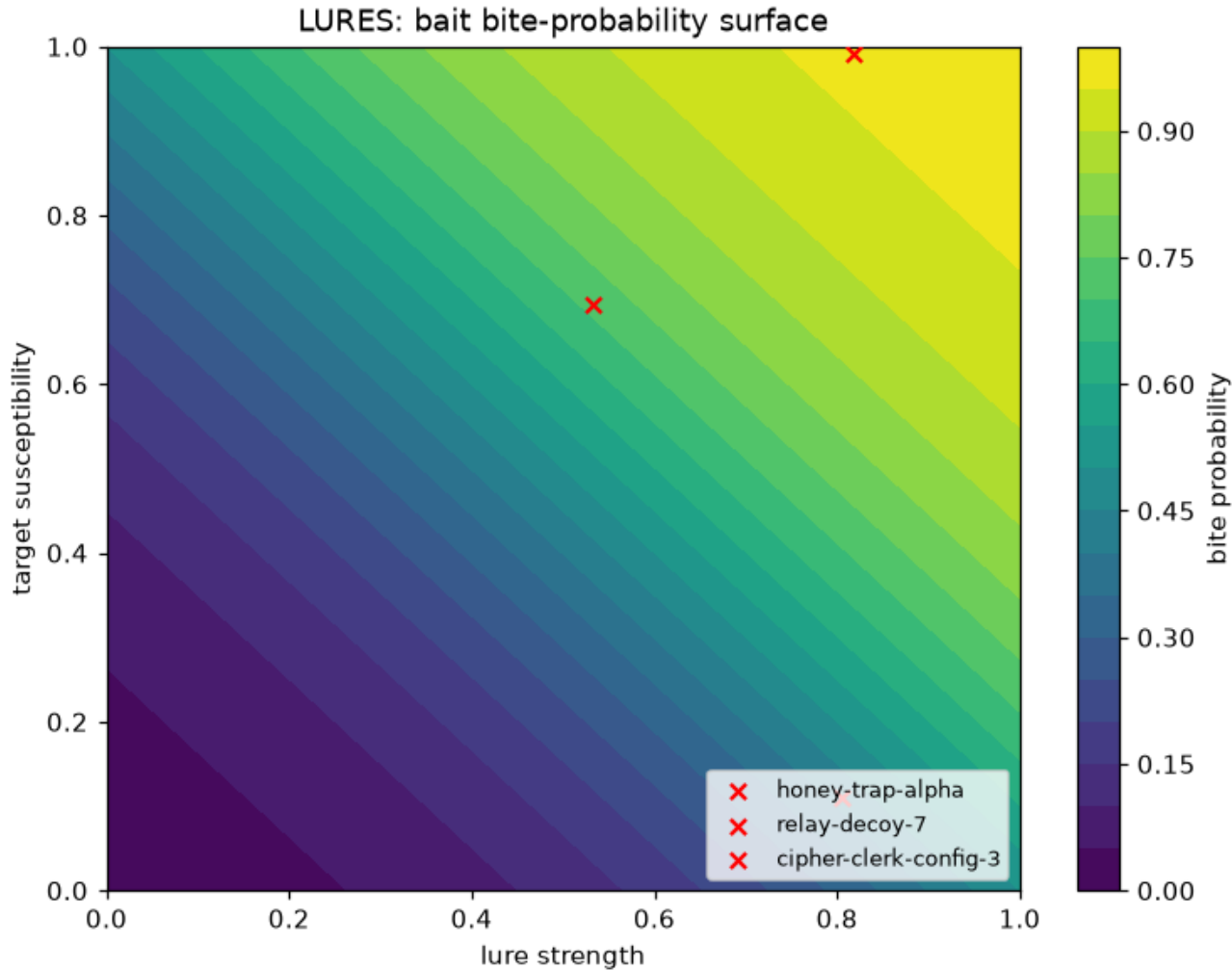


Figure 4: LURE surface: bite probability vs. lure strength and target susceptibility.

5.5.3 3.2 SIGINT interception

A **47-byte** message was enciphered under **2** keystream layers and intercepted at a requested **80.0 dB**; the empirical SNR measured against ground truth was **79.5 dB**. The recovered plaintext matched the transmitted message exactly: **True**. The decided capture carried **45 distinct symbol values** and its energy envelope segmented into **1 transmission burst(s)**.

5.5.4 3.3 Direction finding

With **4** intercept sites at **A (0, 0); B (20, 0); C (20, 20); D (0, 20)**, a true transmitter at **(7, 9)**, and seeded timing noise of standard deviation **0.10**, the TDOA Gauss–Newton estimator converged (**True**) in **5** iterations to a transmitter localisation error of **0.022 receiver coordinate units**.

The units matter. `localization_error` returns a raw Euclidean distance in the coordinate system the receivers are declared in — it is not normalised by the array geometry. The smallest distance between two intercept sites is **20.0** coordinate units, so the same error expressed as a fraction of the site spacing is **0.0011**. Both numbers are generated from the same computation; only the second one is “in units of site spacing”.

A useful uncertainty framing is the **Cramér–Rao lower bound** on the position error — the theoretical best achievable RMS accuracy given the receiver geometry and the timing noise, computed from the Fisher information matrix at the true transmitter position (see `direction_finding.py::crlb_error`). For the canonical array and 0.10 s timing noise, that bound is **0.0504** coordinate units. The realised Gauss–Newton estimate achieved an error of **0.022**, which is **False** above that bound. This is honest uncertainty quantification rather than a single point claim: the CRLB is a bound on achievable *variance* across draws, so one particular noise realisation may land on either side of it (as it does here), and a degenerate array with no finite accuracy has an infinite bound rather than a fabricated confidence.

5.5.5 3.4 Handoff scheduling

Of **5** candidate rendezvous, the earliest-deadline-first heuristic completed **4**, finishing at time **9.5**. The exact subset dynamic program of §2.5 puts the true maximum at **4**, so the heuristic’s cost on this instance is a gap of **0** meets. The heuristic is not optimal in general — §2.5 describes the family of instances on which it loses, and the test suite pins a concrete one — so this gap is a measured property of the canonical scenario, not a guarantee about the algorithm.

That canonical gap is zero, and we say so plainly: on `scenario.RENDEZVOUS` the heuristic genuinely reaches the optimum, so this instance separates the two solvers not at all. Where the scheduler’s sub-optimality is *visible* is the witness instance of §2.5 (`scenario.SCHEDULE_WITNESS`), deliberately built so a long, slack window sorts first and crowds out two short meets. On that instance the heuristic can complete **1** of **3** rendezvous against a true optimum of **2** — a strictly positive gap of **1** meet, pinned by the test suite. Reporting both instances is the honest version of the claim: the exact solver is necessary precisely when the problem is hard, and the canonical scenario is the easy case where the cheap heuristic happens to be optimal.

5.5.6 3.5 Defector handoff protocol

The green-pathed handoff for the mission call sign completed in state **handoff**, gated on confirm code **66b2aa**. The red-path drill — the identical protocol run given a deliberately wrong confirm code — ended in **abort**, confirming that only the exact code completes a handoff, and that a failed confirm is an ordinary terminal outcome rather than an exception.

5.6 Conclusion — LEKTOR: findings, verdict, and what the mission establishes

LURES turns the operational fiction of a Cold-War deception mission into a small, fully deterministic mission-software package. The 6 engines — an entrapment playbook with a validated trap state machine, an exact knapsack lure-budget allocation, a SIGINT interception simulation with real cipher, noise and burst-segmentation models, a TDOA Gauss–Newton transmitter geolocation, an earliest-deadline-first handoff schedule, and a defector handoff protocol whose confirm code gates completion — each compute a result rather than assert one. The same fixed inputs always yield the same lure outcome, allocation, recovered plaintext, localisation, schedule, and handoff path.

The package’s more durable contribution is methodological rather than narrative. Two of its engines solve problems where a cheap answer is available and a correct one is expensive, and in both cases the package computes both and publishes the difference: the knapsack allocator against a profit-density greedy baseline, and the courier scheduler against an exact subset dynamic program. Publishing the difference includes publishing it when it is zero — on the canonical target pool the greedy allocator ties the exact knapsack, so that instance demonstrates nothing about the exact solver’s necessity, and the separation is measured on an explicit witness instance instead. The scheduling case is the sharper one, because the obvious greedy ordering *looks* like the classical optimal interval-scheduling algorithm and is not — the underlying problem becomes NP-hard once meets carry both release times and deadlines. Reporting 4 completed meets against an optimum of 4 is a weaker-sounding claim than reporting an optimal schedule, and it is the one the code can actually support.

Every stage is reproducible, mock-free, and covered by a real-data test suite above the package’s 90% line-and-branch gate. Where a branch is defensive rather than exercised — a singular Jacobian, a coincident intercept site — it is documented as such rather than papered over with an artificial test.

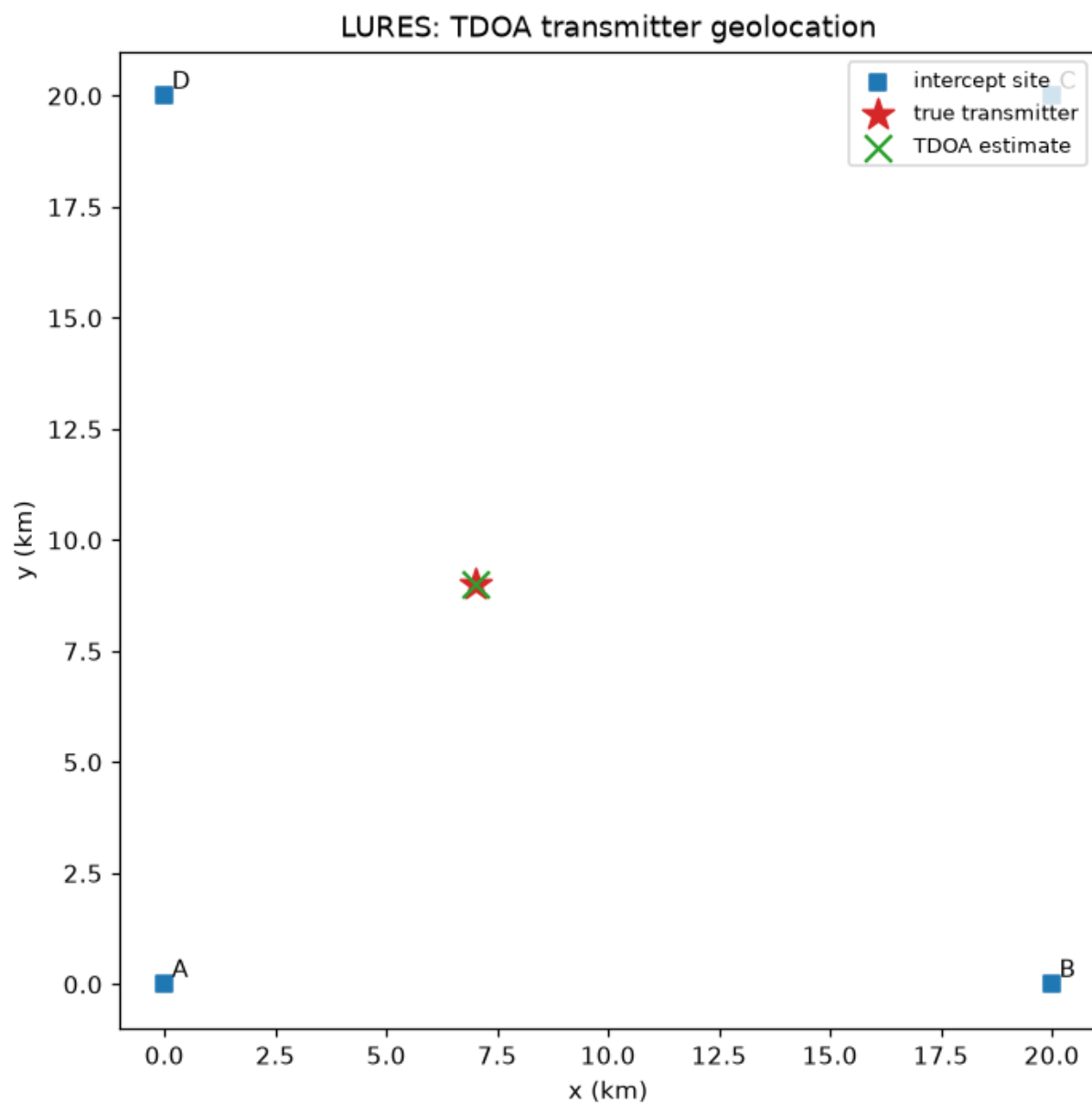


Figure 5: TDOA geometry: intercept sites, true transmitter, and estimate.

The package is also honest about what it does *not* know. TDOA localisation carries a Cramér–Rao uncertainty bound rather than a single over-confident number; both optimality gaps are reported even when they are zero; and the seeding is disclosed as fixing one realisation of a stochastic model, not the whole distribution. A deterministic mission model that states its own uncertainty is more useful — and more defensible — than one that merely looks deterministic.

5.7 Experimental Setup — LEKTOR: canonical scenarios, parameters, and configuration

5.7.1 Where the parameters live

Every executed constant is declared in `src/from_russia_with_love/scenario.py`, which is the single source of truth shared by the mission adapter, the manuscript token generator, and both figure generators. `manuscript/config.yaml` restates the headline mission parameters for a human reader and supplies the paper metadata (title, authors, keywords); `manuscript_variables.validate_mission_config` compares the two on every manuscript build and raises if they disagree, so the restatement cannot drift away from the code silently.

The token generator reads `config.yaml` for paper metadata only. Every quantitative token below is computed by running the engines.

5.7.2 Mission parameters

Parameter	Value
Objectives	6
Analytical engines	6
Fixed seed	11
Bait profiles	3
Engagement threshold	0.50
Allocation targets	6
Lure strength	0.60
Lure budget (units)	5
Message length (bytes)	47
Cipher layers	2
Requested channel SNR	80.0 dB
TDOA intercept sites	4
TDOA receiver coordinates	A (0, 0); B (20, 0); C (20, 20); D (0, 20)
TDOA true transmitter	(7, 9)
TDOA minimum site spacing (coordinate units)	20.0
TDOA timing noise (s)	0.10
Candidate rendezvous	5
Greedy-witness targets / budget	3 / 5

Receiver and transmitter positions are dimensionless coordinates in the scenario frame, and `TDOA_SPEED` is 1.0, so distance and time share units. The localisation error reported in §3.3 is therefore in these coordinate units; the site spacing row is what converts it to units of site spacing.

5.7.3 Software Environment

Component	Value
Python	3.14.6
Platform	darwin
numpy	2.5.1
Package version	0.1.0
Manuscript date (<code>config.yaml paper.date</code>)	2026-08-04

5.8 Reproducibility — LEKTOR: verification gates, deterministic regeneration, and artifacts

5.8.1 Determinism

Every engine is deterministic. The entrapment engine uses a closed-form logistic model and seeded lure generation; SIGINT derives a seeded keystream and seeded noise (seed 0 makes capture exactly noise-free); TDOA timing noise is seeded; the courier scheduler, the knapsack allocator and the handoff protocol use no randomness at all (SHA-256 only). The mission seed is `scenario.SEED = 11`, declared in `src/from_russia_with_love/scenario.py` and mirrored into `manuscript/config.yaml` under a build-time equality check.

Nothing in the package reads the wall clock. There is no clock-derived token: the date row in §5 is 2026-08-04, taken verbatim from the committed `paper.date` field of `manuscript/config.yaml`. Every other value is a function of `scenario.py`, the engines, and the interpreter/library versions. Two regenerations on the same tree therefore produce byte-identical artifacts — the token map and both figures alike.

That claim is bound by a test rather than asserted: `test_regeneration_is_byte_identical` in `tests/test_regeneration_determinism.py` runs the real generator script twice as a subprocess and compares the SHA-256 of every persisted artifact. It does *not* inject a fixed clock — injecting one is precisely how a wall-clock dependency could hide behind a green suite, and this package shipped that bug once. `test_no_wall_clock_read_under_src` is the positive control: it fails if any module under `src/` calls `datetime.now`, `time.time`, or `date.today`.

5.8.2 Regenerating everything

```
uv sync --extra dev
uv run python scripts/z_generate_manuscript_variables.py
```

That writes `output/data/manuscript_variables.json` and re-renders both figures. `output/` is disposable and git-ignored; nothing in it is ever hand-edited.

5.8.3 Verification Gates

- `uv run pytest tests/ --cov=src --cov-fail-under=90`
- `uv run ruff check src/ scripts/ tests/` and `uv run ruff format --check...`
- `uv run mypy src/ scripts/`
- `uv run python scripts/00_preflight.py` — protocol + pure-core sanity check
- `rg -n "template[_]code_project".` → zero surviving lineage strings

Two gates in the suite are positive-controlled rather than merely green: the manuscript token cross-reference test fails if any token referenced in the prose is not produced by `generate_variables`, and the drift guard between `config.yaml` and `scenario.py` is exercised with a deliberately mismatched config so that it is known to be able to fail.

5.8.4 Artifact Inventory

Artifact	Path
LURE surface figure	<code>../figures/lure_surface.png</code>
TDOA geometry figure	<code>../figures/tdoa_geometry.png</code>
Manuscript variable map	<code>output/data/manuscript_variables.json</code>

5.9 Scope and Related Work — LEKTOR: boundaries, positioning, and relationship to the literature

5.9.1 Scope

LURES is an analytical, deterministic mission model — not a real-world targeting or deception tool. The entrapment engine models lure effectiveness as a decision surface for defensive countermeasure evaluation; the knapsack allocation and courier scheduler are combinatorial optimisation over synthetic records; the SIGINT and TDOA modules simulate channel interception and geolocation for study; the handoff protocol is a state-machine fragment. The package performs no real signal capture, no cryptanalysis of live systems, and no targeting of persons.

The XOR keystream cipher in `sigint.py` is a *pedagogical* construction for producing a scrambled symbol stream to intercept. It derives its keystream from a seeded PRNG, which makes it reproducible and completely unsuitable as cryptography; nothing in this package should be read as a security claim about the cipher.

Two further boundaries are worth stating plainly. First, the courier scheduler is a heuristic and is not optimal (§2.5); the exact solver exists to measure that gap, not to be run at scale, and refuses instances beyond its documented size bound. Second, the bite-probability surface is a modelling *choice* — a bounded logistic in strength and susceptibility — not an empirically fitted model of anything, and its parameters are not calibrated against data.

5.9.2 Uncertainty and limitations

Beyond the boundaries above, the package reports its uncertainty honestly rather than implying that a deterministic single number is a confident one:

- **TDOA localisation uncertainty is quantified, not asserted away.** §3.3 reports both the realised estimate error and the Cramér–Rao lower bound (0.0504) for the same geometry and timing noise. The CRLB is a bound on achievable *variance*, so

a single seeded realisation can land on either side of it (the canonical realisation lands below it, `False`); a degenerate array has an infinite bound. The numbers are generated, not fitted, to a particular noise draw.

- **The seeding fixes the realisation, not the distribution.** Every quantity that is stochastic under the model — the SIGINT capture noise, the TDOA timing noise, the lure generation — is the outcome of one committed seed (`scenario.SEED`). Regeneration is byte-identical by construction, but a *different* seed would produce a *different* realisation of the same underlying distribution. The manuscript therefore reports what was actually computed, never a resample.
- **Both optimality gaps are measured, and may be zero.** The canonical allocation pool and the canonical rendezvous set both fail to separate the exact solver from its heuristic (gaps 0.000 and 0); the separation is only visible on the witness instances (2.642 and 1). Reporting the zero as zero prevents the exact solvers from being dressed up as necessary where they are not.
- **The decipherability result is a high-SNR, seed-specific outcome.** At 80.0 dB the recover is exact (`True`) on this draw; below some SNR ceiling byte recovery would fail. The package makes no claim about performance at low SNR.
- **The XOR keystream cipher is pedagogical.** It is derived from a seeded PRNG and is unsuitable as cryptography (§2.3). Nothing here is a security claim.

5.9.3 Related Work

- **BOND-API (THE PROTOCOL)** — the frozen `MissionProvider` contract this film implements (discovery, lifecycle, provenance) [Friedman, 2026b].
- **BOND-UTILITIES (Q-BRANCH)** — the Layer-0 foundation (ciphers, codenames, provenance persistence, reporting) that downstream packages may bind to [Friedman, 2026f].
- **Combinatorial optimisation** — exact 0/1 knapsack via dynamic programming [Martello and Toth, 1990], on the classical dynamic-programming formulation [Bellman, 1957b]; interval-scheduling greedy algorithms [Kleinberg and Tardos, 2005]. The scheduling problem solved here, $1 \mid r_j, d_j \mid \sum U_j$, is strongly NP-hard [Garey and Johnson, 1979], which is why the package pairs a heuristic with an exact subset dynamic program rather than claiming optimality.
- **Localisation** — hyperbolic (TDOA) position estimation. The solver implemented in `direction_finding.py` is the **iterative Taylor-series / Gauss–Newton** estimator [Foy, 1976b, Torrieri, 1984], i.e. The standard non-linear least-squares iteration [Nocedal and Wright, 2006] applied to the pair TDOA residuals from a receiver centroid start. It is *not* the Chan & Ho estimator [Chan and Ho, 1994b]: that is a **closed-form, non-iterative** two-stage weighted least-squares solution, published as an alternative to exactly the iterative scheme used here. Chan & Ho is listed as related work, and this package does not implement it.
- **Channel and cipher models** — the additive white Gaussian noise channel and symbol-decision model follow standard digital-communications treatment [Proakis and Salehi, 2008]; the keystream construction is an elementary one-time-pad-style XOR stream in the sense of Shannon’s secrecy analysis [Shannon, 1949a]. The handoff confirm code uses SHA-256 as specified in the NIST secure hash standard [National Institute of Standards and Technology, 2015].
- **Decision surface** — the logistic response model used for bite probability is the classical binary-response form [Cox, 1958].
- **Source material** — Ian Fleming’s novel [Fleming, 1957], of which the 1963 film is an adaptation.

5.10 Sources — LEKTOR: bibliography

Friedman [2026b]; Friedman [2026f]; Foy [1976b]; Torrieri [1984]; Chan and Ho [1994b]; Nocedal and Wright [2006]; Martello and Toth [1990]; Bellman [1957b]; Kleinberg and Tardos [2005]; Garey and Johnson [1979]; Proakis and Salehi [2008]; Shannon [1949a]; National Institute of Standards and Technology [2015]; Cox [1958]; Fleming [1957]

6 Goldfinger (1964) — GRAND SLAM

film package · *package codename* *GRAND SLAM*. Mission **GRAND SLAM**: corner the gold market via a controlled supply shock; weaponise the gold peg via a speculative reserve attack; assess Fort Knox-style layered and network vault defense; quantify the laser burn-through and thermal engagement threat.

6.1 Concepts — GRAND SLAM: domain and operational focus

macroeconomic attack, vault defense, laser threat

6.2 Abstract — GRAND SLAM: mission summary

Auric Goldfinger’s *Operation Grand Slam* is modelled as a deliberate gold-market corner: a supply shock that removes 30% of the floatable supply and drives the clearing price from \$1850.00/oz to a shocked equilibrium of \$2460.90/oz — a 33.0% appreciation under constant-elasticity demand (elasticity 1.25). The GRAND SLAM mission software deepens this into a full attack surface. On the **macroeconomic front** the same accumulation is weaponised against the gold peg itself: a first-generation speculative attack exhausts reserves in 5.19 time units — well before the gradual 21.28-unit drain — and lands a 336% post-float depreciation. On the **vault front** a Fort Knox-style layered defence scores 0.780 (medium security, 0.009% penetration given the weakest layer is the fence); a network model finds a 15.20-effort minimum intrusion path through the secret tunnel and shows that a 10-unit hardening budget raises the intruder’s required effort to 20.20. On the **laser front** a 5000 W continuous-wave beam burns a 50 mm steel plate in 0.72 s (serious) at range, and the Gaussian-beam thermal model sets its effective engagement range at 65.0 m. The cinema’s fictional scheme is treated as a *quantitative modelling exercise*: 6 deterministic, infrastructure-free modules (`market_attack`, `reserve_attack`, `vault_security`, `vault_threat_network`, `laser_physics`, `laser_thermal`) formalize the economics of cornering and speculative attack, the reliability of layered and network defense, and the physics of laser penetration and heat conduction. Each is fully unit-tested on real computation with zero mocking, and every result is injected into this manuscript as one of 73 template variables so no metric — not even a count of modules or figures — is hand-authored. The package implements the frozen BOND-API mission protocol, exposing a `MissionProvider` (film `goldfinger`, codename *GRAND SLAM*) that a fleet orchestrator can discover and drive through the brief–recon–plan–execute–debrief lifecycle.

6.3 Introduction — GRAND SLAM: mission framing, the operational problem, and how to read this chapter

The 1964 film *Goldfinger* dramatizes a plausibly dangerous scheme: corner the gold market by destroying the floatable supply, strike a fortress vault, and deploy an industrial laser as the coup de grâce. This report strips the drama to its *quantitative skeleton* and builds the models a mission planner would actually run — 6 small, deterministic, well-tested modules organised along three attack surfaces. Each surface carries one closed-form model and one structural model that the closed form cannot express.

6.3.1 Macroeconomic attack

1. **Market cornering** (`market_attack`) — the economics of a deliberate supply shock. Removing a fraction of the floatable supply shifts the market-clearing price along a constant-elasticity demand curve, and a tâtonnement process traces how price converges to the new equilibrium.
2. **Speculative reserve attack** (`reserve_attack`) — the monetary front. The same accumulation is turned against the gold peg itself: under the first-generation crisis model, exponential domestic-credit growth drains the central bank’s reserves and speculators collapse the peg the instant the shadow floating rate reaches parity. The corner raises the price; the attack breaks the institution that fixes it.

6.3.2 Vault defense

3. **Layered assessment** (`vault_security`) — a Fort Knox-style defense-in-depth score. Concentric defensive layers are rated and weighted into a composite, and the sequential-penetration probability of an intruder is computed by the product rule over the traversal order.
4. **Network penetration and hardening** (`vault_threat_network`) — the structural front. Ratings measure how strong each layer is, but not which *route* an intruder takes. Treating the vault as a weighted zone graph, a shortest-path search finds the minimum-effort intrusion route (the film’s secret tunnel), edge criticality identifies which zone transitions actually matter, and a greedy budget allocation raises the effort the intruder must pay.

6.3.3 Laser threat

5. **Burn-through physics** (`laser_physics`) — an energy-balance model determining how long a beam of a given power takes to burn through a plate of a given material, plus its inverses for required power and maximum burnable thickness.
6. **Thermal engagement** (`laser_thermal`) — the spatio-temporal front. A Gaussian beam defocuses with range, so absorbed intensity falls; the Carslaw–Jaeger constant-flux solution converts that intensity into a surface melt time, and the two together bound the beam’s effective engagement range within a threat window.

These 6 modules are the *pure domain core*: they depend only on the standard library and NumPy, contain no I/O, draw no random numbers, and are trivially reproducible. Around them sits a thin mission adapter that implements the frozen BOND-API protocol (the suite’s canonical `MissionProvider` contract), so the model set can be discovered, orchestrated, and debriefed uniformly across the film fleet. (The fleet’s size is a property of the suite, not of this package; it is deliberately not stated here, because this package can neither derive it nor gate it without reaching into sibling packages. The suite roster is authoritative.) The adapter’s public shape — the film slug, the five lifecycle methods, and the module-level gadget registry — is frozen across phases; only its internals deepen.

The design priorities are **determinism** (a declared seed of 2026, no random draws, no wall-clock dependence in persisted results), **honesty** (real computation and real data throughout; no mocked dependencies), and **measurability** (every headline number in this manuscript is injected as one of 73 generated variables rather than typed by hand — including the module count in this paragraph, which is derived from the package directory rather than declared).

6.4 Methodology — GRAND SLAM: the analytical models and algorithms that drive the mission

The GRAND SLAM models each reduce one operational concept to a closed-form relation and a deterministic iterative update. All 6 are pure functions of their inputs — no I/O, no random draws, no wall-clock dependence. The three attack surfaces are presented in pairs: the closed-form model first, then the structural model that captures what the closed form cannot.

6.4.1 Market cornering: supply shock to price dynamics

Let $\mathcal{S}_0$ be the floatable supply clearing at a reference price P_0 . Cornering removes a fraction $f \in [0, 1)$ of that supply, leaving the effective supply

$$\mathcal{S}_{\text{eff}} = \mathcal{S}_0(1 - f).$$

With constant-elasticity demand $Q_d(P) = kP^{-\eta}$ (elasticity $\eta > 0$), market clearing pins the shocked equilibrium price to

$$P^* = P_0 \left(\frac{\mathcal{S}_0}{\mathcal{S}_{\text{eff}}} \right)^{1/\eta}.$$

Price does not jump; it adjusts toward P^* by a discrete tatonnement with coefficient $\lambda \in (0, 2)$,

$$P_{t+1} = P_t + \lambda(P^* - P_t),$$

which converges monotonically for $\lambda < 1$. The fractional corner gain is $P^*/P_0 - 1$. Because the trajectory approaches P^* geometrically and attains it only in the limit, convergence is reported as the relative residual $|P_n - P^*|/P^*$ tested against a tolerance (1e-06), never by exact float equality. For the reference intel ($f = 30\%$, $\eta = 1.25$, $P_0 = 1850.00$) the gain resolves to the results in Section [Results].

6.4.2 Vault defense: layered security scoring

A vault is defended by ℓ concentric layers ranked $\mathcal{L}_1, \dots, \mathcal{L}_\ell$. Each layer i receives a rating $r_i \in [0, 1]$ (assessed strength) and a weight $w_i \geq 0$. The composite defense-in-depth score is the weighted average

$$S = \frac{\sum_i r_i w_i}{\sum_i w_i} \in [0, 1].$$

Under a sequential-penetration model an intruder must defeat layers in order; if each layer stops the intruder with probability equal to its rating, the overall breach probability is the product of the pass-through rates,

$$P_{\text{breach}} = \prod_i (1 - r_i).$$

The expected time to breach (or stall) telescopes as $E[T] = t \sum_i \prod_{j < i} (1 - r_j)$ over the rated layers. Fortification priority is ranked by the marginal composite gain of raising each layer by a fixed δ .

6.4.3 Laser threat: burn-through time vs material and power

A continuous-wave laser of power P focuses onto a spot of area A ; the target absorbs a fraction $a \in (0, 1]$ of the beam. The material column under the spot has mass $m = \rho Ad$ (density ρ , thickness d); raising it by ΔT (specific heat c) requires

$$Q = \rho Adc\Delta T.$$

Equating this to the absorbed power $P_{\text{abs}} = aP$ gives the burn-through time

$$t_{\text{burn}} = \frac{\rho A d c \Delta T}{aP}.$$

The same relation inverts for required power and for maximum burnable thickness, so any pair of (power, time, thickness) determines the third.

6.4.4 Macroeconomic attack: speculative attack on the gold peg

The phase-4 deepening adds the monetary-system front. Money demand is linear in the interest rate, $m_t - p_t = -\alpha i_t$; purchasing-power parity and uncovered interest parity link the price level and interest rate to the exchange rate. With domestic credit growing exponentially, $D_t = D_0 e^{\mu t}$, the fixed peg forces reserves to drain,

$$R_t = (\bar{S} - \alpha i^*) - D_0 e^{\mu t},$$

while the rational-expectations **shadow floating rate** (the rate that would prevail once reserves are gone) is

$$\tilde{s}_t = \frac{D_0 e^{\mu t}}{1 - \alpha \mu} + \alpha i^*.$$

Under perfect foresight speculators attack exactly when the shadow rate crosses the fixed parity, at

$$t^* = \frac{1}{\mu} \ln \frac{(1 - \alpha \mu)(\bar{S} - \alpha i^*)}{D_0},$$

which precedes the gradual reserve-exhaustion time $t_B = (1/\mu) \ln((\bar{S} - \alpha i^*)/D_0)$ — the attack is sudden. The post-float depreciation is the shadow rate at t_B relative to parity.

6.4.5 Network vault defense: penetration and hardening

The layered ratings in the previous section measure *strength*; the network model treats the vault as an undirected weighted graph of zones with intrusion-effort edges. The intruder's **minimum-effort path** (Dijkstra) from the perimeter to the gold chamber, and the **edge criticality** (how much removing each edge raises that minimum), capture where the vault is truly weak — here the secret tunnel. Given a hardening budget, a greedy marginal-gain allocation raises the edge whose lift most increases the minimum path, **gain_per_unit** per unit, until the budget is spent, maximising the effort an intruder must pay.

6.4.6 Laser thermal engagement: beam focus and heat conduction

The scalar burn-through model ignores beam divergence and conduction. A Gaussian beam of waist w_0 and wavelength λ has Rayleigh range $z_R = \pi w_0^2 / \lambda$ and spot radius $w(z) = w_0 \sqrt{1 + (z/z_R)^2}$, so absorbed intensity $I_a = aP/(\pi w^2)$ falls with range. Under constant flux the surface heats by the Carslaw–Jaeger law,

$$T(0, t) = T_{\text{amb}} + \frac{2I_a}{\sqrt{\pi k \rho c}} \sqrt{t},$$

giving the melt time $t_{\text{melt}} = \frac{\pi}{4} \frac{k \rho c (T_{\text{melt}} - T_{\text{amb}})^2}{I_a^2}$. The **effective burn range** is the greatest distance at which the defocused beam still melts the target within a threat window.

6.5 Results — GRAND SLAM: measured outcomes, headline numbers, and what they establish

6.5.1 Market corner: price shock and convergence

Under the reference intel, removing 30% of the floatable supply moves the equilibrium price from 1850.00/oz to 2460.90/oz, a **33.0%** appreciation at elasticity 1.25. Price approaches the shocked equilibrium geometrically over the 40-step tâtonnement window, ending at a relative residual of 3.32e-10 against a convergence tolerance of 1e-06 (converged: true). Tâtonnement reaches P^* only in the limit, so convergence is asserted against that tolerance rather than by exact equality.

The figure shows the reference price, the shocked equilibrium price, and the deterministic adjustment path. The corner is economically self-reinforcing: a larger sterilized fraction yields a higher equilibrium, which in turn raises the value of the seized stockpile.

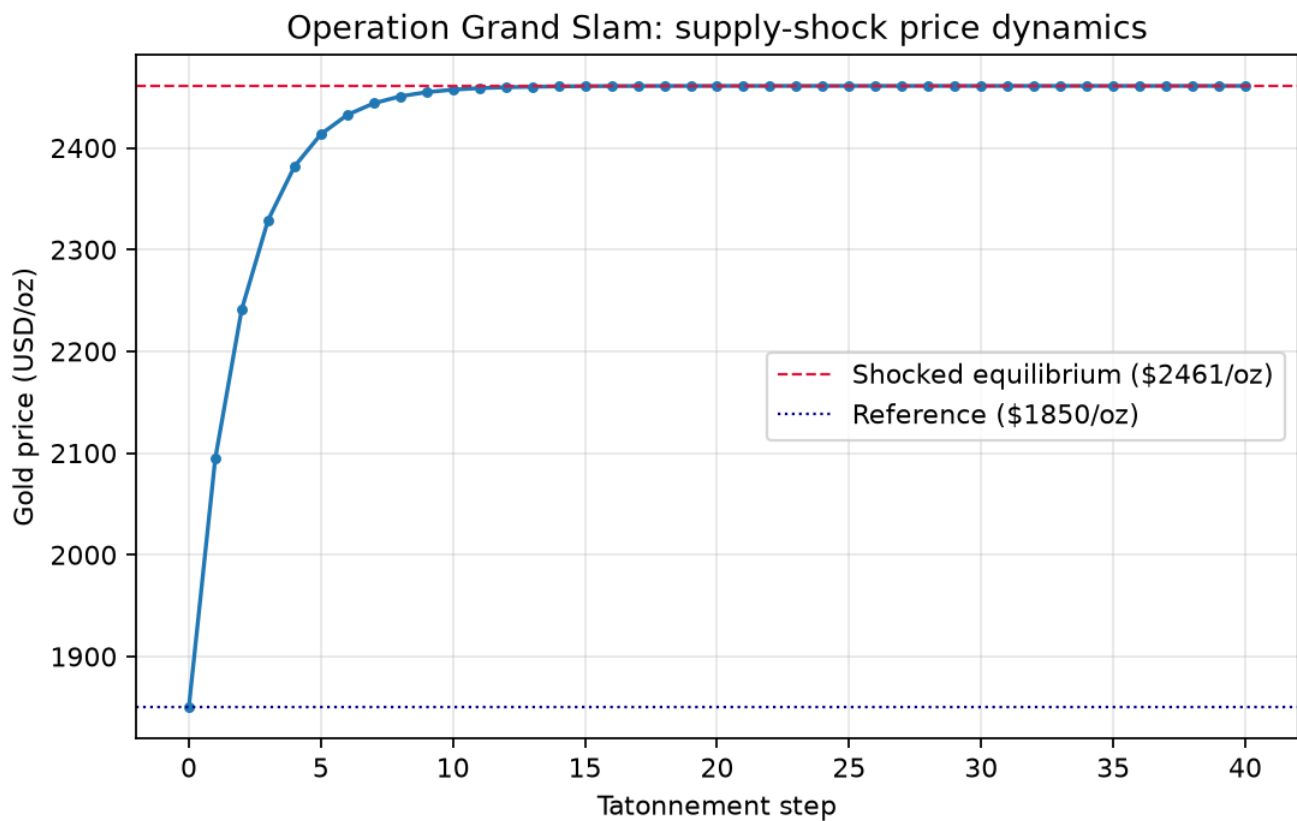


Figure 6: Operation Grand Slam: tatonnement price trajectory toward the shocked equilibrium.

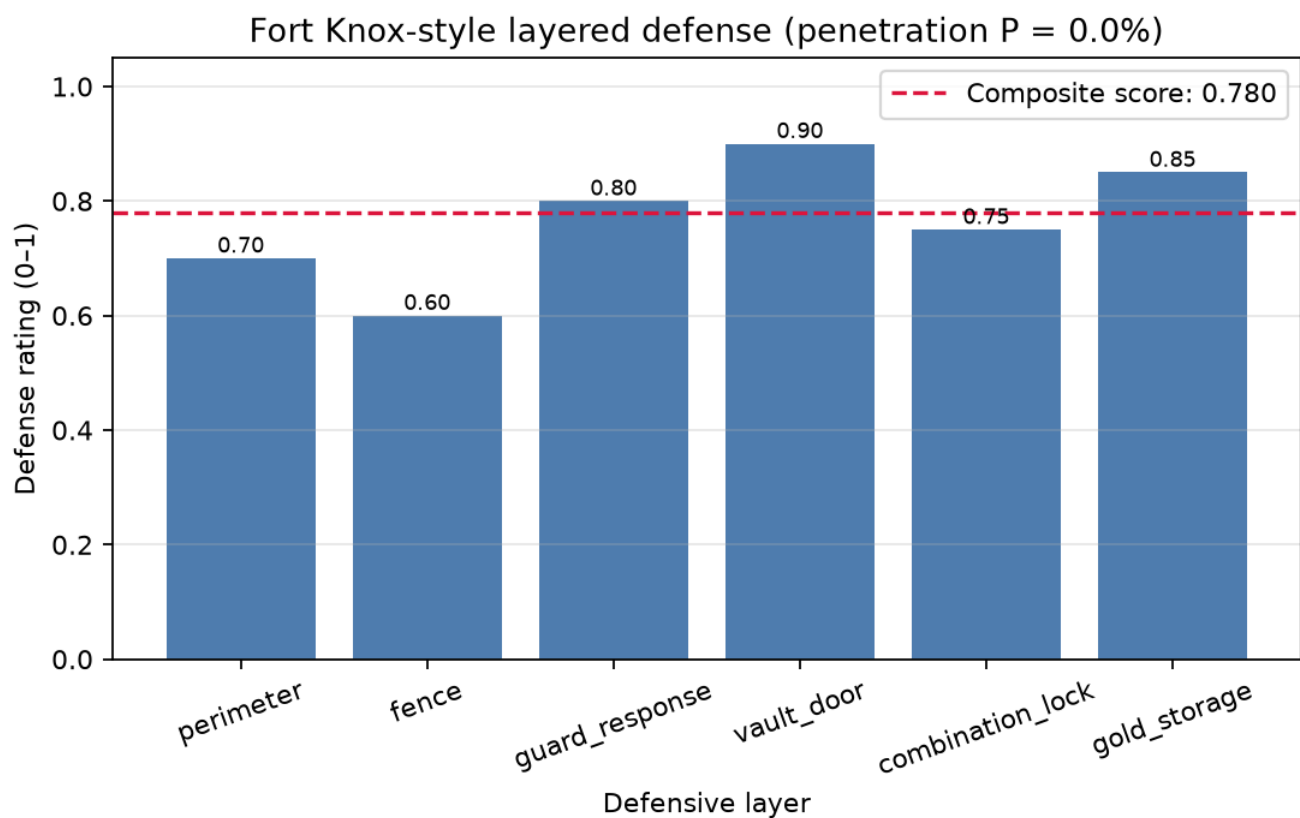


Figure 7: Fort Knox-style layered defense: per-layer ratings and composite defense-in-depth score.

6.5.2 Vault defense: layered score and penetration

The 6 concentric layers of the reference vault score **0.780** — classified **medium** security — with an overall sequential-penetration probability of **0.009%**. The weakest layer is the **fence**, which is where fortification yields the greatest marginal composite gain.

6.5.3 Vault network: penetration and hardening

The network model locates the vault's true weak point. Across the 10-zone graph, the minimum-effort intrusion path from the perimeter to the gold chamber is **15.20** effort and runs through the **secret tunnel** (6 zone transitions), bypassing the costly guarded spine. 6 edges are critical — removing each one forces a strictly costlier route. Spending the 10-unit hardening budget at 0.50 effort per unit raises the minimum required intrusion effort from 15.20 to **20.20** (a lift of 5.00), closing off the tunnel as the free ride. The allocation is greedy rather than exactly optimal, so its lift is normally a lower bound on what the same budget could buy; here the bound is *measured*, not just disclaimed. At a small verification budget of 3 units we enumerate every allocation of that budget across the graph's edges and confirm the greedy result achieves the maximum achievable lift (1.50, greedy matches optimal: true), so on this graph the greedy allocation is exactly optimal at small budgets.

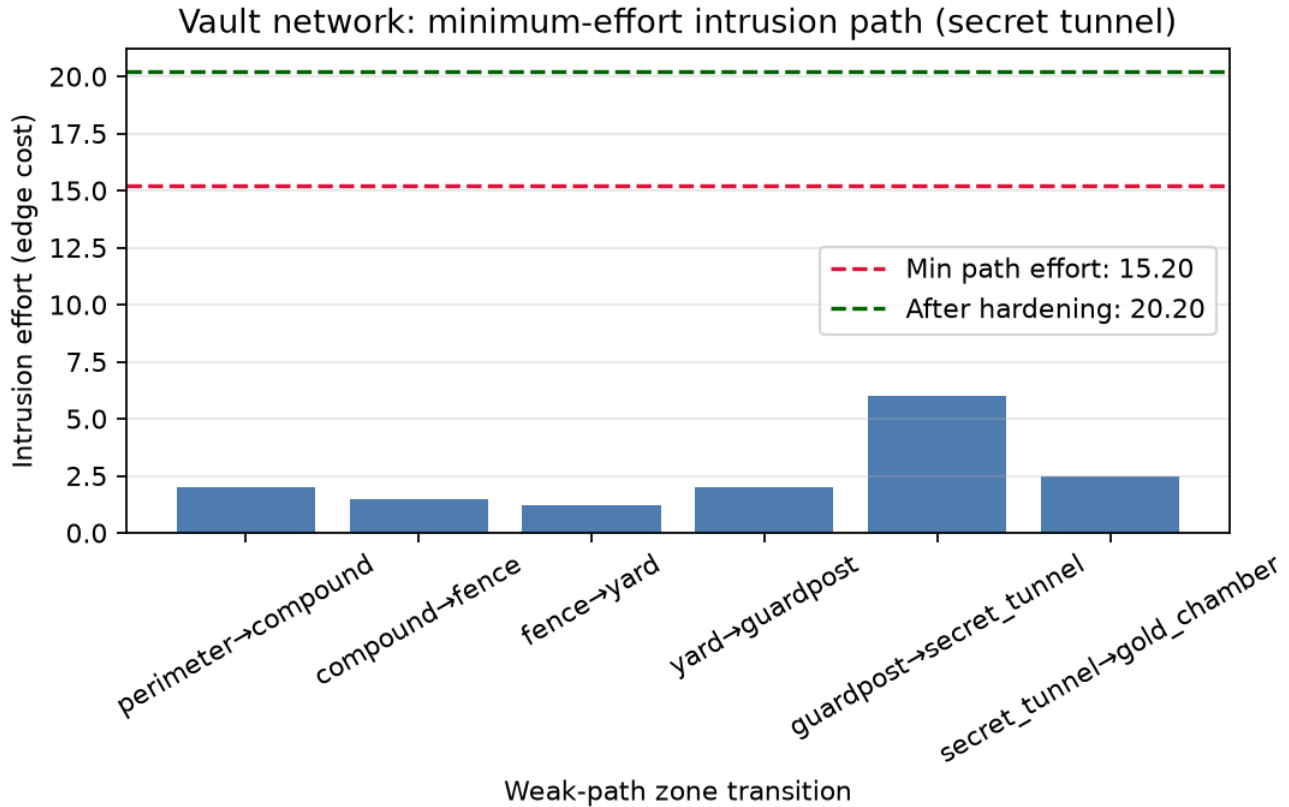


Figure 8: Vault network: minimum-effort intrusion path through the secret tunnel, before and after hardening.

6.5.4 Macroeconomic attack: speculative reserve drain

The speculative-attack model turns gold accumulation into a monetary weapon. With the reference intel the shadow floating rate crosses parity at 5.19 time units, forcing the peg's collapse **before** the gradual reserve floor at 21.28 (suddenness 16.09), and the subsequent float depreciates the currency by **336%**.

6.5.5 Laser threat: burn-through time

At 5000 W the reference steel plate (50 mm) is breached in **0.72 s**, denoted a **serious** threat against the 1000 W mission threshold. Doubling power halves burn-through time; doubling thickness doubles it, so the defensive levers on the laser side are plate thickness and material (higher $\rho c \Delta T$).

6.5.6 Laser thermal engagement: beam focus and heat conduction

Accounting for Gaussian divergence and heat conduction, the reference beam melts the plate surface at focus in 0.0020 s and retains an effective engagement range of **65.0 m** within the 5.0 s threat window (Rayleigh range 9.36 m, focus waist 1.78 mm). Defocusing with range lengthens melt time, so a defender pushing the engagement beyond the Rayleigh range materially degrades the threat.

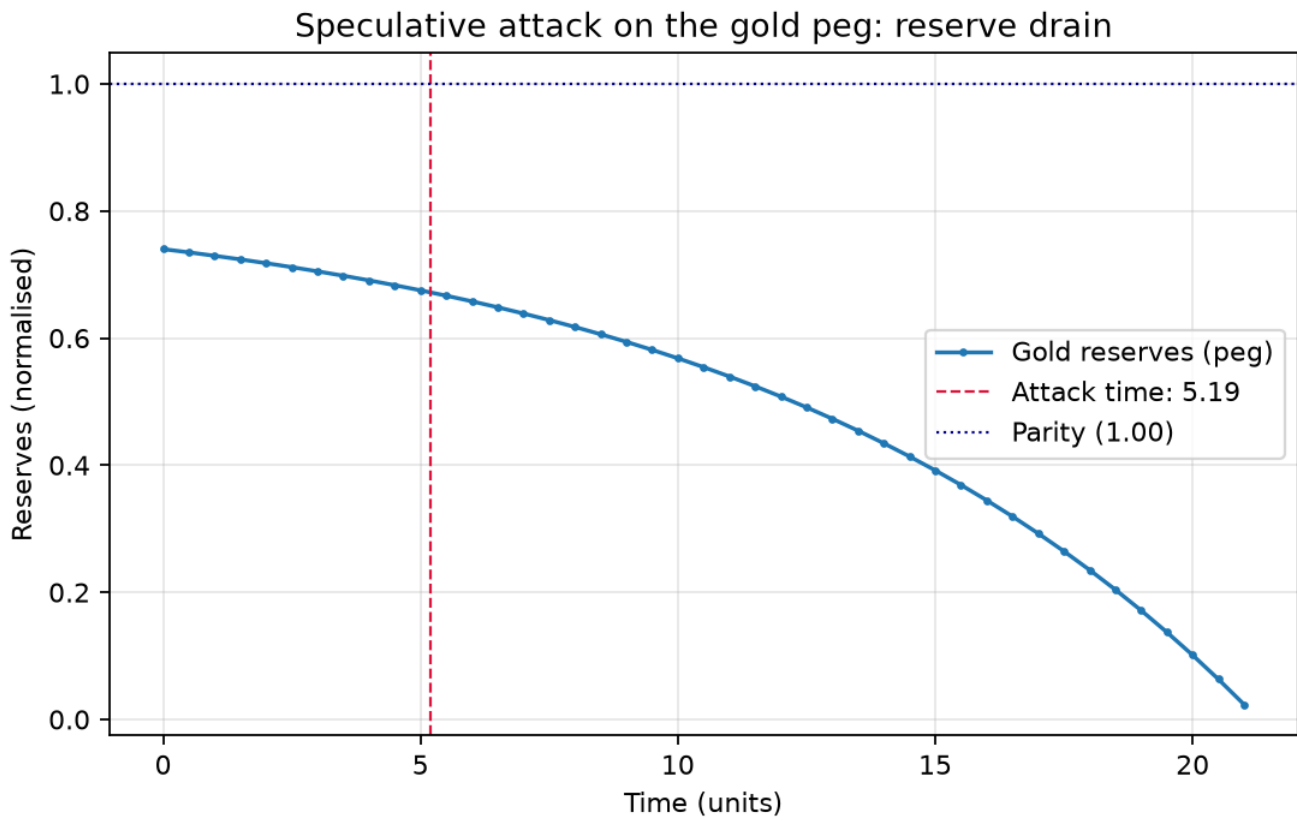


Figure 9: Speculative attack on the gold peg: reserve drain and attack time.

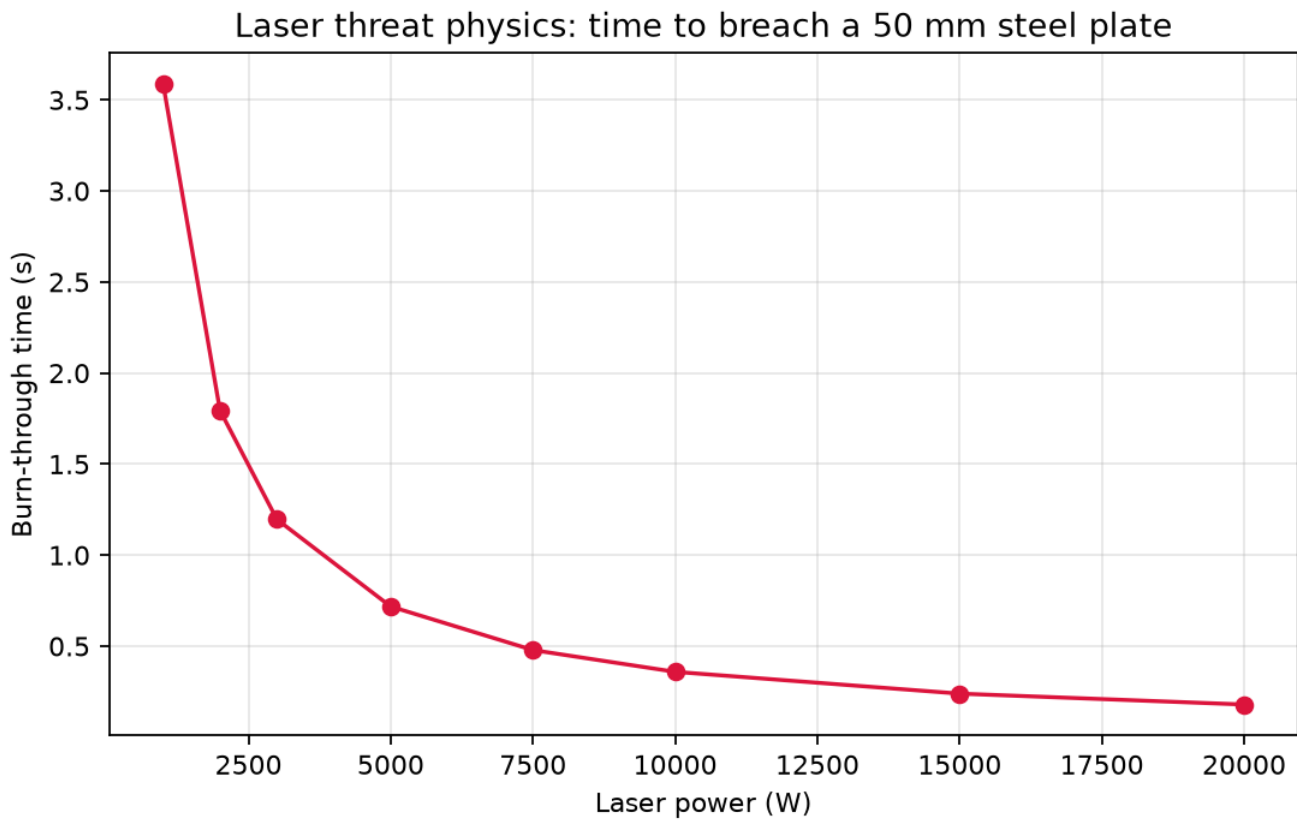


Figure 10: Laser threat physics: burn-through time vs beam power for the reference vault plate.

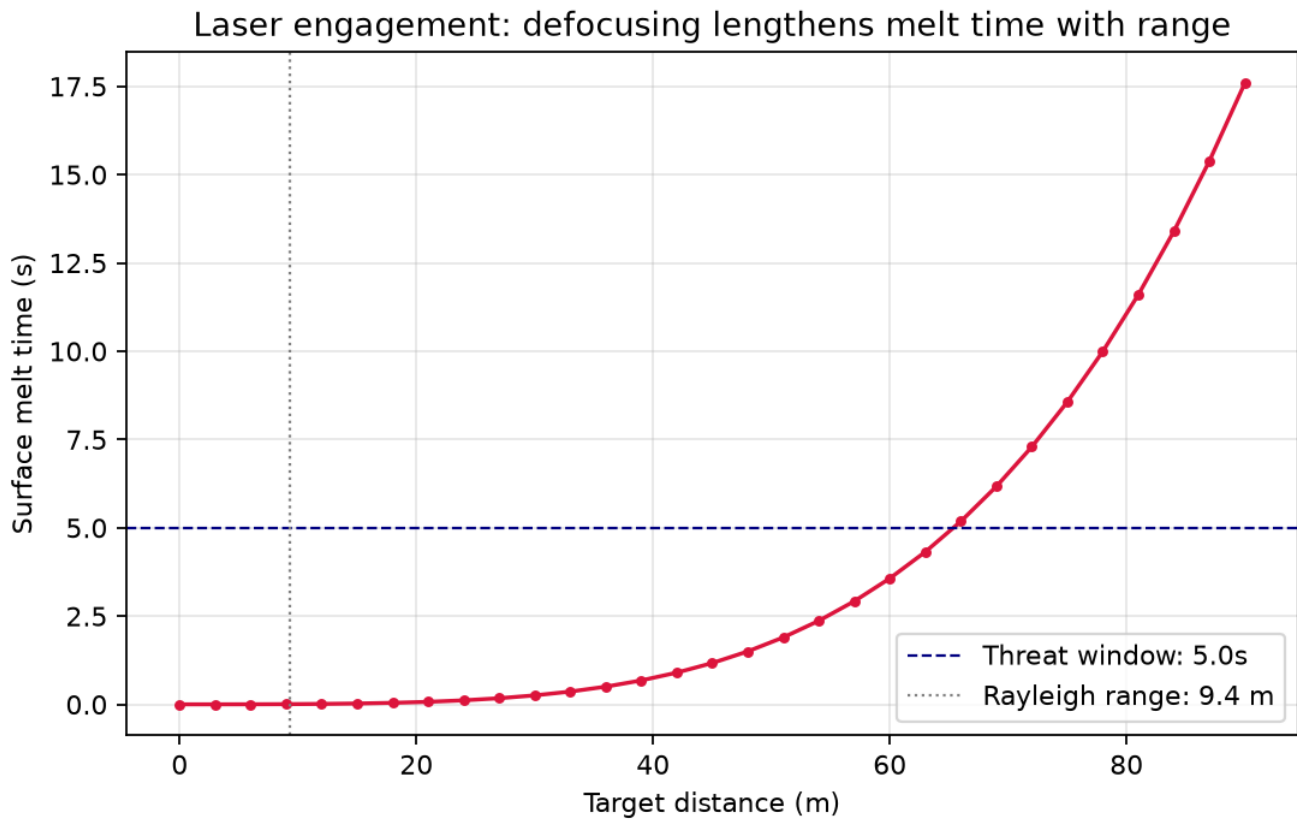


Figure 11: Laser engagement: surface melt time grows as the Gaussian beam defocuses with distance.

All 6 model results above are computed by the pure domain core and injected here as generated variables — none is typed by hand, including the counts in this sentence. Re-running the pipeline reproduces identical numbers from the module `REFERENCE_*` constants.

The no-hardcoding claim is gated, with a stated limit. Every computed result token is classified, and each one whose value is a numeric literal of at least three characters is searched for in the rendered prose (the numbered sections and `preamble.md`), with a positive control that pastes each value in turn and confirms the scan flags it. The remaining computed tokens — bare small integers such as the module and layer counts, and categorical words such as the vault verdict and the laser severity — **are not scanned**, because searching prose for `6` or `medium` finds ordinary English rather than a pasted result. Those tokens are listed explicitly in the test with the reason, and a further test asserts the list is exactly what the rule rejects, so the hole cannot widen silently. It is a real hole nonetheless: a hand-typed module count in this prose would not be caught by the scan.

6.6 Conclusion — GRAND SLAM: findings, verdict, and what the mission establishes

The GRAND SLAM mission software demonstrates that the operational concepts of *Goldfinger* — cornering the gold market, weaponising the monetary system, breaching a layered vault, and wielding a laser — are each reducible to small, deterministic, closed-form or graph models that a planning agent can actually run and audit.

- The **market-cornering model** quantifies how a supply shock propagates into price along a constant-elasticity demand curve, with a tatonnement path to the new equilibrium.
- The **speculative-attack model** (phase 4) turns accumulation into a monetary-system weapon: credit growth drains reserves, and the peg collapses the moment the shadow floating rate crosses parity — an attack that is strictly *sudden* relative to gradual exhaustion.
- The **vault-defense assessment** combines layered rating scoring with a network model that finds the minimum-effort intrusion path (the secret tunnel) and shows hardening raises the intruder’s required effort unit-for-unit on the weak path.
- The **laser-threat physics** spans a scalar burn-through time and a spatio-temporal Gaussian/Carslaw-Jaeger model whose effective burn range falls as the beam defocuses with distance.

Every headline number in this report is a generated variable computed from the module `REFERENCE_*` constants — real computation only, no mocked dependencies. The scanned subset of those values is gated against being retyped as a prose literal; the count and category tokens are not scannable and rest on review instead (see Results). The package ships as a BOND-API `MissionProvider` (film *goldfinger*, codename **GRAND SLAM**), so all 6 models are discoverable and drivable by the suite’s orchestrator through the full mission lifecycle — the adapter’s public shape is stable across phases; only its internals deepen.

6.7 Experimental Setup — GRAND SLAM: canonical scenarios, parameters, and configuration

6.7.1 Reference intel

The deterministic reference parameters are **declared in the module** `REFERENCE_* constants` in `src/goldfinger/`. Those constants are what the domain core, the mission adapter, the figures, and the token map all read, and every value below is injected from them as a generated variable, so a change to the intel changes this section automatically.

`manuscript/config.yaml` carries a human-readable `grand_slam`: mirror of the same numbers. Nothing loads that block at runtime — it is documentation, not input — so its only guarantee is that `tests/test_reference_intel.py` compares it key-for-key and value-for-value against the module constants and fails if either side is edited alone.

Market corner (`market_attack.REFERENCE_MODEL`) — reference price \$1850.00/oz clearing a normalized floatable-supply index of 100.0; the corner sterilizes 30% of that float against a constant demand elasticity of 1.25. The tâtonnement runs 40 steps at adjustment coefficient 0.40.

Speculative reserve attack (`reserve_attack.REFERENCE_ATTACK`) — a normalised peg at parity 1.00, foreign interest rate 0.02, money-demand semi-elasticity $\alpha = 8.0$, initial domestic credit 0.10, and credit growth $\mu = 0.10$. The model validates its own feasibility conditions ($0 < \alpha\mu < 1$, and initial credit small enough that the peg has not already fallen) before computing an attack time.

Vault — layered (`vault_security.REFERENCE_RATINGS`) — 6 layers (perimeter, fence, guard response, vault door, combination lock, gold storage), each rated in `[0,1]`, weighted by `DEFAULT_WEIGHTS` and traversed in `LAYER_ORDER`.

Vault — network (`vault_threat_network.REFERENCE_GRAPH`) — an undirected graph of 10 defensive zones with intrusion-effort edge costs. The reference intrusion problem is perimeter → gold chamber (`REFERENCE_TARGET`); the reference defensive posture (`REFERENCE_HARDENING`) is a budget of 10 units buying 0.50 effort each.

Laser — scalar (`laser_physics.REFERENCE_PHYSICS`) — power 5000 W onto a focused spot of $1.0\text{e-}05\text{ m}^2$ against a steel plate of density 7800 kg/m^3 and thickness 50 mm, specific heat $460\text{ J/(kg}\cdot\text{K)}$, $\Delta T = 1200\text{ K}$, absorptivity 0.60. A beam is classified *serious* at or above 1000 W. This is the *same* plate as the thermal model below: the two modules shipped different steel (specific heat 500 vs $460\text{ J/(kg}\cdot\text{K)}$), and a ΔT that did not equal that model’s melt-minus-ambient span) until the constants were reconciled, so ΔT is now exactly $1500\text{ K} - 300\text{ K}$ and `tests/test_reference_intel.py` asserts the agreement.

Laser — thermal (`laser_thermal.REFERENCE_BEAM`) — an Nd:YAG-class continuous-wave beam of wavelength 1064 nm focused to a waist of 1.78 mm, against steel of thermal conductivity $45\text{ W/(m}\cdot\text{K)}$ melting at 1500 K from an ambient of 300 K, engaged within a 5.0 s threat window.

6.7.2 Determinism policy

The mission declares a fixed seed of 2026 and performs **no** random draws — the seed is recorded in provenance so a replay is auditable, not because any sampling occurs. Persisted artifacts carry no wall-clock dependence at all (`wall_time_s = 0.0` in provenance; no clock in the token map). The manuscript’s date is the committed `paper.date` from `manuscript/config.yaml` — `MANUSCRIPT_DATE = 2026-08-04` — so `GOLDFINGER_VERSION_HASH`, which hashes the full token map, identifies the content rather than the run and is stable across regenerations. Figure output is byte-deterministic for fixed inputs: `tests/test_figures.py` renders **every** exported renderer twice — into two separate directories and again over the same path — and compares the bytes, parametrised over `figures.FIGURE_RENDERERS` so a renderer added later is covered without anyone remembering to add it.

6.7.3 Software environment

- Python: as resolved by `uv` (the package requires `>=3.10`)
- Dependencies: NumPy, Matplotlib, PyYAML, and the frozen `bond-api` protocol (a local path dependency declared in `pyproject.toml` under `[tool.uv.sources]`; the sibling `bond-api` package in this working tree)
- Dev: `pytest`, `pytest-cov`, `mypy`, `ruff`, `types-PyYAML`

6.7.4 Mission adapter

The BOND-API `MissionProvider` (`src/goldfinger/mission.py`) wraps all 6 models and registers six gadgets (`market_corner`, `reserve_attack`, `vault_defense`, `vault_hardenig`, `laser_threat`, `laser_thermal`) with a module-level `GadgetRegistry`. It is discoverable by slug `goldfinger` on `sys.path` and drives the full brief-recon-plan-execute-debrief lifecycle with deterministic provenance, and persists the outcome as a schema-validated mission record (via `bond-utilities`’s `record_provenance_outcome`) whose `input_hash` is a real fingerprint of the reference intel, not a hand-typed literal — so two outcomes from unchanged constants carry identical provenance and a one-value edit to any constant is visible in the hash. The record writes no wall clock (its `started_at` is a committed sentinel), so it is byte-identical on every run. The adapter’s public shape is frozen across phases; the mission internals (assets, plan steps, outcome results, lessons) are what deepen.

6.8 Reproducibility — GRAND SLAM: verification gates, deterministic regeneration, and artifacts

6.8.1 Deterministic regeneration

The package is fully deterministic. Re-running the mission or regenerating the manuscript variables reproduces identical results:

```
uv run python scripts/z_generate_manuscript_variables.py
```

which rewrites `output/data/manuscript_variables.json` (73 tokens) and the 6 concept figures under `./figures/`. Every persisted byte is a function of the committed tree alone, so running that command twice — on different days, on different machines — produces byte-identical files. Nothing in the token map reads a clock: the manuscript’s date is the committed `paper.date` in `manuscript/config.yaml` (`MANUSCRIPT_DATE = 2026-08-04`), and `GOLDFINGER_VERSION_HASH` hashes the whole map, which makes it a content identifier rather than a run identifier.

That property is enforced, not asserted: `tests/test_regeneration_determinism.py` copies the tree to a scratch directory, runs the real generator twice with a wall-clock gap between the runs, and compares every emitted byte — the JSON, all 6 figures, and the generated counts file. The same file carries a positive control that reintroduces a clock-derived value into the map and shows the comparison goes red, so the gate is known to be able to fail.

The token count and figure count above are themselves generated, not typed: `GOLDFINGER_TOKEN_COUNT` is the length of the declared `GOLDFINGER_TOKENS` tuple, `GOLDFINGER_FIGURE_COUNT` is the length of `goldfinger.figures.FIGURE_RENDERERS`, and `GOLDFINGER_MODULE_COUNT` (= 6) is computed by scanning the package directory for concept modules. A test asserts the declared token tuple is exactly the key set `generate_variables` produces, so a token added to one and not the other fails the suite rather than silently drifting into the prose.

6.8.2 Verification gate

The enforced quality gate, run from the package root:

```
uv run pytest tests/ --cov=src --cov-fail-under=90
uv run ruff check src/ scripts/ tests/ && uv run ruff format --check src/ scripts/ tests/
uv run mypy src/goldfinger scripts
```

Coverage is measured on `src/goldfinger/` only (tests are excluded) and the floor is 90% line **and** branch. The suite exercises every model error branch through a real call with a real bad argument, every mission stage, protocol discovery and serialization, the token cross-reference gate, figure rendering determinism, and the thin CLI smoke paths — all with real computation, no mocking framework anywhere.

6.8.3 Change policy

- Result metrics in the prose are injected by `src/goldfinger/manuscript_variables.py` from the module `REFERENCE_*` constants; do not hardcode numbers in prose, including counts of modules, figures, tokens, or tests. The hardcode scan cannot police the exact-value count and category tokens (their values are too short or too word-like to search for), so exact-value enforcement of those rests on review. But a **fleet or sibling-package count** — a number this package can neither derive nor gate — is caught by a lexical gate that rejects digit-plus-noun constructions such as a digit bound to a fleet/suite noun (see `tests/test_guards_and_contracts.py`), with a positive control proving the gate can fail.
- The pure domain modules must remain infrastructure-free and importable without `bond-api` (`mission.py` is the only module that imports it).
- Reference parameters live in exactly one place — the module `REFERENCE_*` constants. The mission adapter, the token map, and the figures all read those constants rather than repeating a literal, so the three cannot report different postures. `manuscript/config.yaml`’s `grand_slam:` block is a read-only mirror that no code loads; `tests/test_reference_intel.py` pins it to the constants value-for-value and also pins the two laser models to one physical steel plate.

6.9 Scope and Related Work — GRAND SLAM: boundaries, positioning, and relationship to the literature

6.9.1 Scope

This work is a **quantitative modelling exercise** inspired by the *Goldfinger* (1964) plot; it is not a physical market, monetary, or security assessment. The scope boundary of each of the 6 models is stated below, because a model whose limits are unstated is a model whose numbers cannot be trusted.

- **Market corner** (`market_attack`) assumes constant-elasticity demand and a single sterile supply shock. It abstracts away bid–ask microstructure, storage and carry costs, competing arbitrageurs, and policy responses; the closed-form jump is therefore an upper bound on what a real corner achieves against a market that observes the trader’s own order flow [3]. It is deliberately closed-form and deterministic.

- **Speculative reserve attack** (`reserve_attack`) is the *linear* first-generation crisis model: perfect foresight, purchasing-power parity, uncovered interest parity, and exogenous exponential credit growth. It has no second-generation self-fulfilling multiple equilibria, no sterilisation policy, no risk premium, and no stochastic credit process. The attack time it reports is the deterministic one implied by those assumptions, not a forecast.
- **Vault — layered** (`vault_security`) treats layers as *independent* stops whose stopping probability equals their rating. Real security is correlated (one insider defeats several layers at once), adaptive, and human, and the ratings themselves are analyst inputs, so the composite score is an assessment aid rather than a measurement.
- **Vault — network** (`vault_threat_network`) models intrusion as a static shortest-path problem with additive, known, symmetric edge efforts. It has no detection/response dynamics, no intruder uncertainty about the graph, and no defender response during the intrusion. The hardening allocation is a *greedy* approximation to shortest-path improvement, which is not guaranteed optimal in general. On the reference graph the gap is measured, not only disclaimed: an exhaustive enumeration at a small verification budget (3 units) shows the greedy allocation achieves the optimal lift (1.50, greedy matches optimal: true) — see Results.
- **Laser — scalar** (`laser_physics`) is a thermal energy-balance approximation. It neglects plasma shielding, radiative and convective losses, and latent heat of fusion, so burn-through times are order-of-magnitude threat estimates, not certified engineering values.
- **Laser — thermal** (`laser_thermal`) adds the beam optics and conduction the scalar model omits — Gaussian defocusing with range, and the semi-infinite-solid surface temperature under constant flux. It still assumes a semi-infinite solid (no back-face effects on a finite plate), temperature-independent material properties, constant absorptivity, and no atmospheric attenuation or thermal blooming along the path. Its effective burn range is scanned on a fixed grid, so it is quantised to the scan step.

These limitations are features for a mission-planning sandbox: they keep the models auditable and replayable, and they make the numbers honest — each is a clearly-stated approximation, not a fabricated precision.

6.9.2 Related work

The models implement standard, well-understood literatures rather than novel theory. Full citations are in `99_references.md`.

- **Market cornering and supply shocks.** The constant-elasticity inverse-demand and tâtonnement price-adjustment framework goes back to Walras [1] and Hicks [2]; Kyle [3] supplies the strategic-trader price-impact view that a closed-form corner deliberately abstracts away.
- **Currency crises and speculative attacks.** Krugman [4] introduced the first-generation balance-of-payments crisis; Flood and Garber [5] give the linear closed forms — shadow floating rate, attack time, and the gap to gradual reserve exhaustion — that `reserve_attack` implements directly.
- **Physical protection systems.** Garcia [6] is the standard treatment of layered physical protection, adversary sequence diagrams, and detection/delay path analysis; it underwrites both the weighted layer score and the decision to model the adversary’s route as a weighted graph.
- **Network penetration and hardening.** Dijkstra [7] provides the shortest-path search; Israeli and Wood [8] formalise shortest-path interdiction and improvement, the problem the greedy budget allocation approximates.
- **Laser–material interaction.** Steen and Mazumder [9] give the continuous-wave energy-balance model; Carslaw and Jaeger [10] the constant-surface-flux conduction solution; Siegman [11] the Gaussian-beam propagation that sets the spot radius with range.

The contribution of this package is not new theory but a *disciplined, testable, deterministic implementation* of these concepts as pure modules plus a protocol-compliant mission adapter [12], reproducible end to end.

6.10 Uncertainty and Sensitivity

The models return point estimates from a single reference intel vector. This section quantifies how those headline numbers move when the unobservable inputs shift, because a number without a sensitivity context reads as more certain than it is.

6.10.1 Market corner: elasticity sensitivity

The corner’s price gain is the quantity most exposed to the (unobservable) demand elasticity: the shocked price is $P^* = P_0 (S_0/S_{\text{eff}})^{1/\eta}$, so the *exponent* is $1/\eta$ and the gain is exponentially sensitive to η . At the reference elasticity the gain is **33.0%**. Because η is an analyst input, we evaluate the model at $\pm 10\%$ around it:

- η reduced 10% (more elastic demand reacting less to the shock): gain **37.3%**
- η raised 10% (steeper demand reacting more): gain **29.6%**

The swing is asymmetric in the *direction that hurts the corner*: a 10% error in the elasticity assumption moves the headline by more than a third of its own value. Any decision that leans on the absolute gain figure — rather than on the direction and the shock itself — should carry this band.

6.10.2 Laser burn-through: the $1/P$ lever

The energy-balance burn-through time is exactly proportional to $1/P_{\text{abs}} = 1/(aP)$, so any power-assumption error maps directly: doubling power halves the time (from **0.72 s** to **0.359 s**), and halving the (unobservable) absorptivity doubles it. The time carries the *inverse* of the power error, so it is not robust to absorptivity misspecification. This is a structural, parameter-free statement — it holds for any positive power and absorptivity in this model — so it is a property of the physics, not of the reference intel.

6.10.3 Vault hardening: the greedy gap is measured

The one place an approximation matters is the hardening allocation. Rather than disclaim an unmeasured lower bound, the package enumerates every allocation of a small verification budget (3 units) across the graph and reports the exhaustive optimum (1.50 lift; greedy matches optimal: true). On the reference graph the greedy allocation is exactly optimal at that budget; this is a measured fact here, not a general theorem, which is precisely why the enumeration exists.

6.10.4 What the numbers are not

These are single-point, single-intel outputs of deliberately stylized models (full scope in [scope and related work](#)). The uncertainty quantified above is *within*-model sensitivity to declared inputs; it does not capture the (larger, unquantifiable) model-form uncertainty — the gap between these laws and the real market, vault, or laser. Presenting both — a headline and its within-model band, side by side with the scope boundary — is the honest shape of the result.

6.11 Sources — GRAND SLAM: bibliography

[Walras \[1874\]](#); [Hicks \[1946\]](#); [Kyle \[1985\]](#); [Krugman \[1979\]](#); [Flood and Garber \[1984\]](#); [Garcia \[2007\]](#); [Dijkstra \[1959a\]](#); [Israeli and Wood \[2002\]](#); [Steen and Mazumder \[2010\]](#); [Carslaw and Jaeger \[1959\]](#); [Siegmán \[1986\]](#); [PROJECT BOND suite \[2026\]](#)

7 Thunderball (1965) — THUNDERBALL

film package · package codename THUNDERBALL. Mission **DISCO VOLANTE**: plan the underwater recovery dive; verify the NATO escort screen holds; confirm submersible range to reach the recovery site; confirm passive-sonar detection of the recovery site; steer the return leg against the prevailing current; interdict closing threat tracks before they reach the screen.

7.1 Concepts — THUNDERBALL: domain and operational focus

underwater ops, nuclear convoy integrity

7.2 Abstract — THUNDERBALL: mission summary

Thunderball: DISCO VOLANTE — Special-Agent Mission Software (mission codename **DISCO VOLANTE**) is a deterministic special-agent mission-software package covering six underwater-recovery capabilities: scuba dive planning with a Bühlmann ZH-L16A decompression model, NATO-style nuclear-convoy integrity analysis, submersible range/endurance logistics, passive-sonar detection, set-and-drift underwater navigation, and dynamic convoy interdiction. For the flagship recovery profile the model computes a no-decompression limit of 9.17 minutes at the working depth against an air-supply limit of 24.78 minutes, so decompression rather than gas volume is the binding constraint (**deco**) and the planned bottom time is 9.17 minutes. It further reports an intact escort screen rated ok, a passive-sonar detection range of 21.78 nautical miles, a return-leg drift of 0.122 nautical miles (yes within the recovery radius), and a closing threat held at 3.344 nautical miles (status **held**). Every value is derived from deterministic math over explicit parameters (dive planning, Bühlmann ZH-L16A decompression, no-decompression limit, air consumption, convoy integrity, escort coverage, submersible logistics, range and endurance, passive sonar, Thorp absorption, set-and-drift navigation, closest point of approach, convoy interdiction, deterministic mission software) — no random draws, no wall-clock dependence, and a fully provenanced result.

7.3 Introduction — THUNDERBALL: mission framing, the operational problem, and how to read this chapter

Thunderball (1965), adapted from Fleming’s 1961 novel [Fleming, 1961], is the film in which SPECTRE hijacks two NATO nuclear bombs and the operation reaches its climax in an underwater battle around the legendary submersible yacht, *Disco Volante*. The **DISCO VOLANTE** mission software package implements the quantitative planning problems that underwrite such a recovery: how deep and how long a diver can work, whether the escort screen protecting the high-value asset holds, how far the submersible powerplant can carry an assault team, whether passive sonar can localise the recovery site, how the prevailing current displaces the dive team, and whether closing threat tracks can be interdicted before they reach the screen.

Six pure domain modules provide these capabilities:

- `src/thunderball/dive_planner.py` — air-budget and Bühlmann ZH-L16A decompression planning.
- `src/thunderball/convoy_integrity.py` — convoy-screen gap detection, escort coverage, and an integrity score.
- `src/thunderball/submersible_logistics.py` — power, endurance, range, and optimal-cruise analysis.
- `src/thunderball/sonar_detection.py` — Thorp absorption, transmission loss, signal excess, and detection range.
- `src/thunderball/current_navigation.py` — set-and-drift dead reckoning, course-to-steer, and cross-track error.
- `src/thunderball/interdiction_analysis.py` — closest-point-of-approach geometry and screen-penetration verdicts.

The package is a BOND-suite film: a thin `src/thunderball/mission.py` adapter exposes these capabilities through the frozen `bond_api` `MissionProvider` protocol, so the *orchestrator* can discover and drive the mission deterministically. The domain modules themselves are standalone and never import infrastructure.

Two further modules support the manuscript itself rather than the mission: `src/thunderball/manuscript_variables.py` computes every substituted token in this document from `manuscript/config.yaml` plus the same domain calls the mission makes, and `src/thunderball/figures.py` renders the five concept figures. No numeric result in this prose is hand-authored.

This manuscript documents the mathematical models (the methodology section), the computed results (the results section), the experimental parameters (the setup section), the reproducibility guarantees (the reproducibility section), and the scope boundary (the scope section), and an honest statement of each model’s uncertainty and where it would break (the discussion section).

7.4 Methodology — THUNDERBALL: the analytical models and algorithms that drive the mission

7.4.1 Dive planning

Dive planning in `src/thunderball/dive_planner.py` rests on two pieces of physics. First, absolute ambient pressure rises by one bar every ten metres: $P(d) = P_{\text{surf}} + d/10$. Breathing demand at depth scales with this pressure, so a diver with a surface air consumption (SAC) rate of 20.0 L/min uses $SAC * P(d)$ litres per minute at depth. A cylinder of 12 L filled to 207 bar therefore holds $12 * 207$ litres of free gas, and the bottom time follows by dividing usable volume by the at-depth demand.

Second, inert-gas loading is modelled with the **Bühlmann ZH-L16A** tissue model: sixteen compartments, each with a nitrogen half-time and a pair of coefficients (**a**, **b**). Bühlmann states the constraint as a tolerated *ambient* pressure, $P_{\text{amb_tol}} = (P_{\text{tissue}} - a) / b$; inverted for the tolerated tissue tension at a given ambient pressure this is the M-value line

$$M(P) = P / b + a$$

which puts the fastest compartment's surfacing ceiling at 3.27 bar and the slowest at 1.28 bar. A compartment's inert-gas pressure approaches the alveolar pressure $P_{\text{alv}} = f_{\text{N}_2} (P - p_{\text{H}_2\text{O}})$ exponentially on its half-time.

The **no-decompression limit** is the largest bottom time after which a *direct ascent to the surface* remains legal, so every compartment is tested against its M-value at **surface** ambient pressure — the ceiling the diver must clear on the way up — and not against the inflated M-value at working depth (the ndl figure). The distinction is not cosmetic: at 40 m the ambient pressure is roughly five times the surface value, so evaluating the ceiling at depth admits almost any bottom time and overstates the limit by more than an order of magnitude. The limit is located by bisection on bottom minutes; when a dive exceeds it, **plan_decompression** emits ascending stop holds that discharge the controlling tissue.

plan_dive is the single entry point that combines the two: it takes the cylinder, SAC rate, and depth, flies `min(air_limit, NDL)` unless an explicit bottom time is requested, plans the decompression schedule for the profile actually flown, and reports which limit bound. Gas for each decompression stop is charged at that stop's own ambient pressure — a 6 m hold costs about 1.6x the surface SAC volume — so the reported gas requirement is a real budget rather than a surface-rate approximation. Both the mission adapter and this manuscript's token generator call **plan_dive**, so the two cannot disagree.

7.4.2 Convoy integrity

Convoy analysis in `src/thunderball/convoy_integrity.py` treats the escort screen as a set of bearing angles around the protected formation. Two angular quantities are reported and they are not the same thing. The **angular gap** is the *spacing* between consecutive escort bearings; the largest gap measures how thinly the screen is spread and is computed from bearings alone, with no knowledge of any detection half-angle. The **largest uncovered sector** is the widest arc no escort watches, which does read the half-angle: each escort defends an arc of $2 * \text{half_angle}$ around its bearing, and a screen whose arcs overlap has a positive largest spacing and a zero uncovered sector. Reporting the spacing as though it were an unwatched arc would overstate the screen's exposure, so the two are computed and reported separately (**largest_gap** and **largest_uncovered_sector**). **Coverage fraction** is the unioned defended arc divided by 360°. Its pointwise companion **is_bearing_covered** answers whether one specific bearing falls inside any escort's arc, measuring separation on the circle so the 0/360° seam needs no special case; sweeping it around the compass draws the screen's actual coverage rosette (the convoy figure).

The **integrity score** combines a sigmoid gap penalty with coverage:

$$\text{score} = 1 / (1 + \exp((\text{gap} - 60) / 12)) * (0.5 + 0.5 * \text{coverage})$$

Scores above 0.7 are rated **ok**, between 0.4 and 0.7 **marginal**, and below 0.4 **breached**. Layered over the continuous score is a hard **sector ceiling** (**marginal_under_deg**): a screen whose largest gap reaches it is downgraded from **ok** to **marginal** even when the score would have passed it, which lets a mission impose a sector width narrower than the score's nominal 60° knee without retuning the score. The ceiling can only downgrade, never upgrade. **min_escorts_for_coverage** returns the smallest uniform screen that reaches a coverage target.

7.4.3 Submersible logistics

Submersible logistics in `src/thunderball/submersible_logistics.py` models underwater propulsion with a cubic speed-power law plus a constant hotel load: $P(v) = k v^3 + H$. Endurance is $E / P(v)$, and range is $v * E / P(v)$ (the range figure). The speed that maximises range balances the cubic drag term against the hotel load; because $R(v)$ rises then falls, the optimum is found from the derivative vanishing at $k v^3 = H/2$. The inverse problem — energy required to transit a given range — inverts the cubic via a bounded root-find on the imbalance $f(v) = R k v^3 - E v + R H$.

7.4.4 Passive sonar detection

Detection modelling in `src/thunderball/sonar_detection.py` uses the passive sonar equation. **Thorp absorption** gives the frequency-dependent seawater attenuation ($\alpha(f) = 0.11 f^2/(1+f^2) + 44 f^2/(4100+f^2) + 2.75e-4 f^2 + 0.003$ dB/km), and **transmission loss** combines spherical spreading with absorption: $TL(r) = 20 \log_{10}(r) + \alpha(f) r$ (the sonar figure). The **signal excess** $SE = SL - TL - (NL - DI) - DT$ is the detection margin; the **detection range** is where SE crosses zero, located by bounded bisection.

7.4.5 Set-and-drift navigation

Navigation in `src/thunderball/current_navigation.py` works in the east/north plane with the current expressed as a set/drift vector. Own velocity and current are vector-added to give ground velocity, and dead reckoning integrates it over time. **Course to steer** solves the classic vector triangle: the current's component perpendicular to the desired track is cancelled by a component of own velocity, $\theta = \arcsin(-c_{\text{perp}} / v)$, giving the heading that makes good the track. **Cross-track error** is the signed perpendicular distance off the track line, and the drift distance over the dive's *planned* bottom time — `min(air limit, NDL)` as

reported by `plan_dive`, which is the time the team is in the water, not the longer air budget it never flies — is checked against the recovery radius (the drift figure).

7.4.6 Convoy interdiction

Interdiction in `src/thunderball/interdiction_analysis.py` layers relative motion over the static screen. The **closest point of approach** between a threat track and the convoy follows from relative position and velocity: $t_{cpa} = \max(0, -\text{dot}(\text{rel_pos}, \text{rel_vel}) / |\text{rel_vel}|^2)$ and $d_{cpa} = |\text{rel_pos} + \text{rel_vel } t_{cpa}|$, with closing speed derived from the same dot product. The **screen penetration time** is the chord across the largest inter-escort spacing ($2 r \sin(\text{gap}/2)$) divided by the interceptor’s speed.

Time to minimum separation comes from the constant-velocity range profile $\text{range}(t)^2 = d_{cpa}^2 + |\text{rel_vel}|^2 (t - t_{cpa})^2$, so the range first falls to a limit R at $t = t_{cpa} - \text{sqrt}(R^2 - d_{cpa}^2) / |\text{rel_vel}|$. When $d_{cpa} \geq R$ that root does not exist and the time is infinite: the CPA is by definition the smallest range the threat ever achieves, so a CPA outside the limit means the limit is never reached at all. The **interdiction status** is then a race between three times: **breached** when the threat reaches minimum separation before the escorts’ reaction time, **penetrated** when the gap chord is crossed before it, and **held** otherwise — including a diverging threat.

7.5 Results — THUNDERBALL: measured outcomes, headline numbers, and what they establish

7.5.1 Flagship verdict summary

For the flagship recovery profile the six capabilities report the following deterministic verdicts; each row is a live token from the engine, not a hand-copied constant.

Capability	Verdict
Dive — no-decompression limit	9.17 min
Dive — air-supply limit	24.78 min
Dive — binding constraint	deco
Dive — decompression stops	0
Convoy — integrity status	ok (score 0.7773)
Convoy — uncovered sector	0.00°
Submersible — optimal range	407.61 nm
Sonar — detection range	21.78 nm
Navigation — drift	0.122 nm (within recovery: yes)
Interdiction — status	held

7.5.2 Dive planning results

For the flagship recovery profile (working depth 40 m), the Bühlmann ZH-L16A model reports a no-decompression limit of **9.17 minutes**. The configured cylinder (12 L at 207 bar) and SAC rate (20.0 L/min) yield an air-duration limit of **24.78 minutes**. Decompression therefore binds first: `plan_dive` reports a binding constraint of **deco** and a planned bottom time of **9.17 minutes**, with 0 decompression stops — flying exactly the no-decompression limit is by definition a no-stop dive. The dive consumes **919.5 L** of the cylinder’s free gas, leaving a margin of **1564.5 L**: the recovery team surfaces on its decompression obligation with gas to spare, which is the correct way round for a working dive at this depth.

7.5.3 Convoy integrity results

The escort screen is placed at bearings 0, 45, 90, 135, 180, 225, 270, 315 degrees with a detection half-angle of 30 degrees. The widest *spacing* between adjacent escorts is **45.00 degrees**, but spacing is not exposure: each escort watches 30 degrees either side of its bearing, so the arcs overlap and the largest genuinely *uncovered* sector is **0.00 degrees**, with total escort coverage **1.0000**. Nothing around this screen is unwatched. The integrity score is **0.7773**, rated **ok**; the score is driven by the spacing rather than by the uncovered sector, which is what keeps it sensitive to a screen thinning out before any hole actually opens. To reach the 0.90 coverage target a uniform screen needs at least **6 escorts**.

7.5.4 Submersible logistics results

The submersible powerplant (400 kWh, drag coefficient 0.035, hotel load 2.0 kW) achieves its optimal range at **3.057 knots**, giving an **407.61 nautical-mile** best range (the range figure).

7.5.5 Passive sonar results

At 10.0 kHz (source level 150 dB, ambient noise 75 dB, directivity index 15 dB) the passive-sonar range equation gives a detection range of **21.78 nautical miles** — the range at which the signal excess crosses zero (the sonar figure).

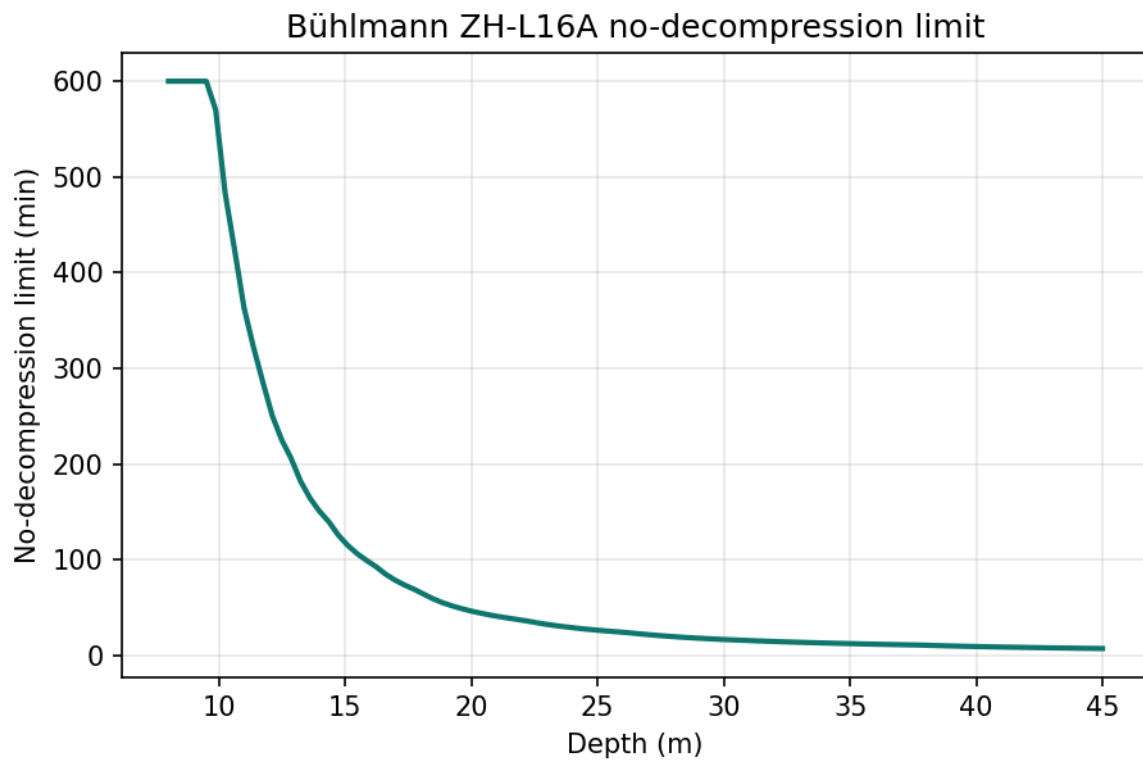


Figure 12: Bühlmann ZH-L16A no-decompression limit versus depth, evaluated against the surface ascent ceiling.

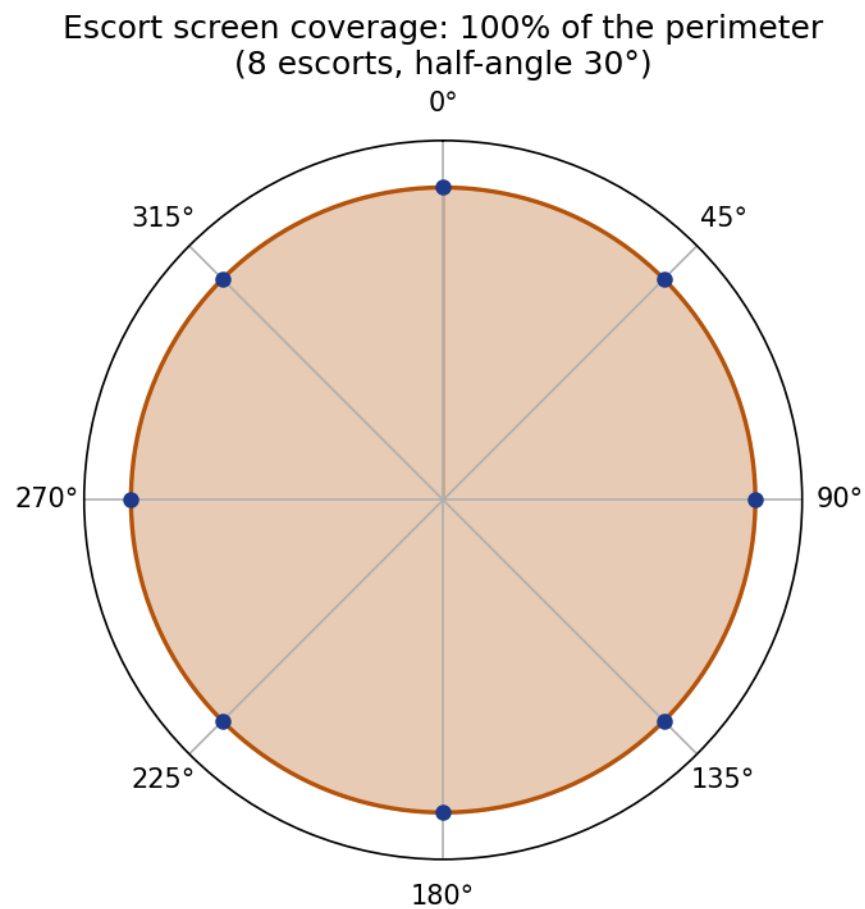


Figure 13: Polar coverage rosette for the configured screen: each of 360 sample bearings tested against the actual escort set, with the escorts marked on the rim.

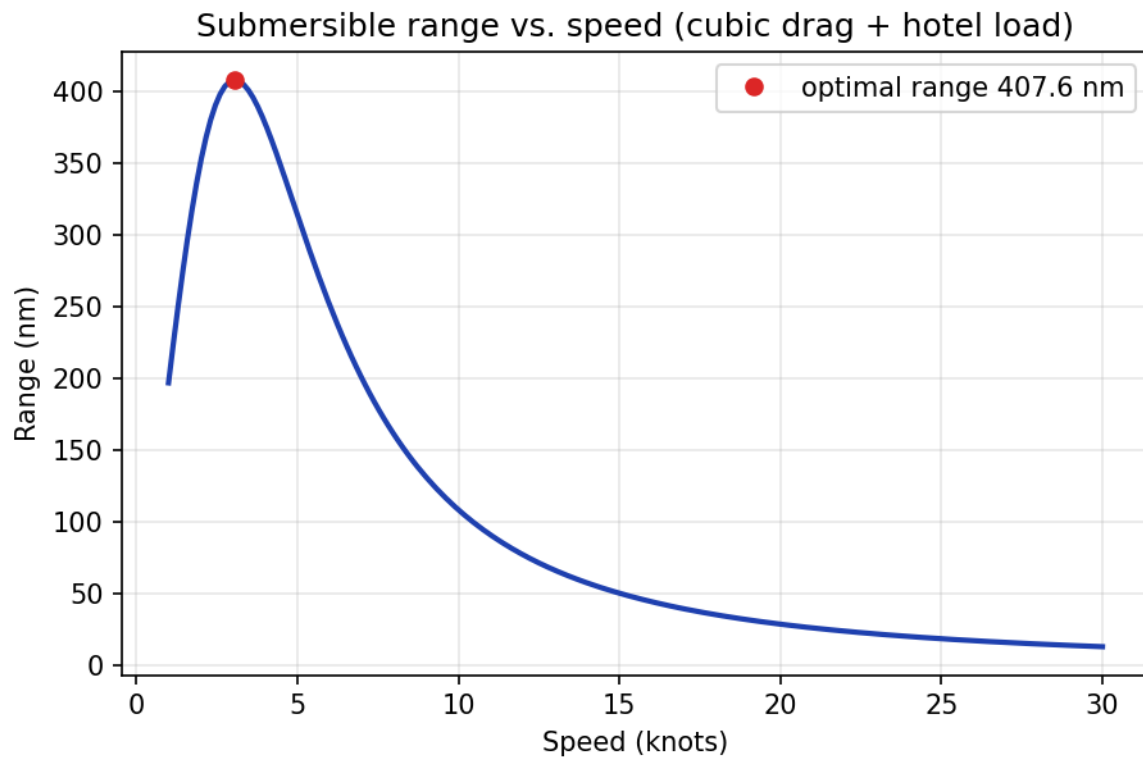


Figure 14: Submersible range versus speed, with the optimal-range marker.

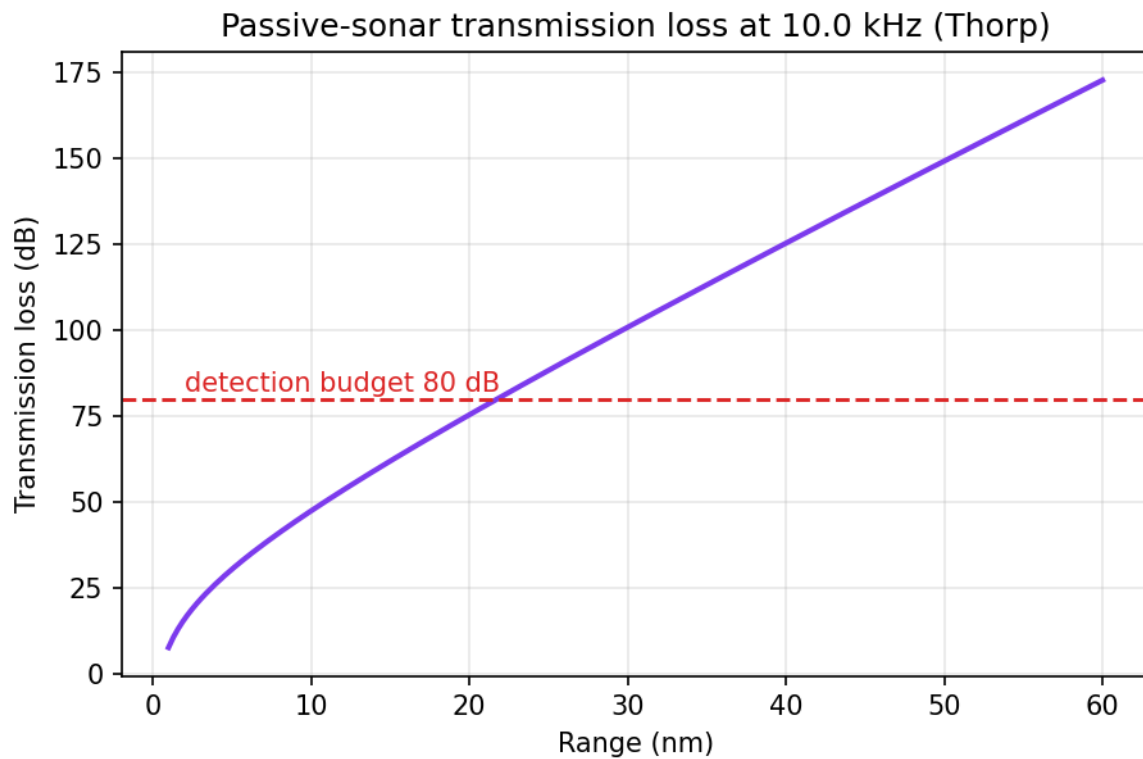


Figure 15: Passive-sonar transmission loss versus range, with the detection budget.

7.5.6 Navigation results

The return leg makes good a track of 315 degrees at 4.0 knots against a current setting 90 degrees at 0.80 knots. Course-to-steer corrects the heading to **306.87 degrees** with a ground speed of **3.394 knots**. Over the planned bottom time of 9.17 minutes — the time the team is actually in the water, which is the decompression limit here and not the longer 24.78-minute air budget — the current sweeps the team **0.122 nautical miles**. Inside the 1.0-nautical-mile recovery radius: **yes** (the drift figure).

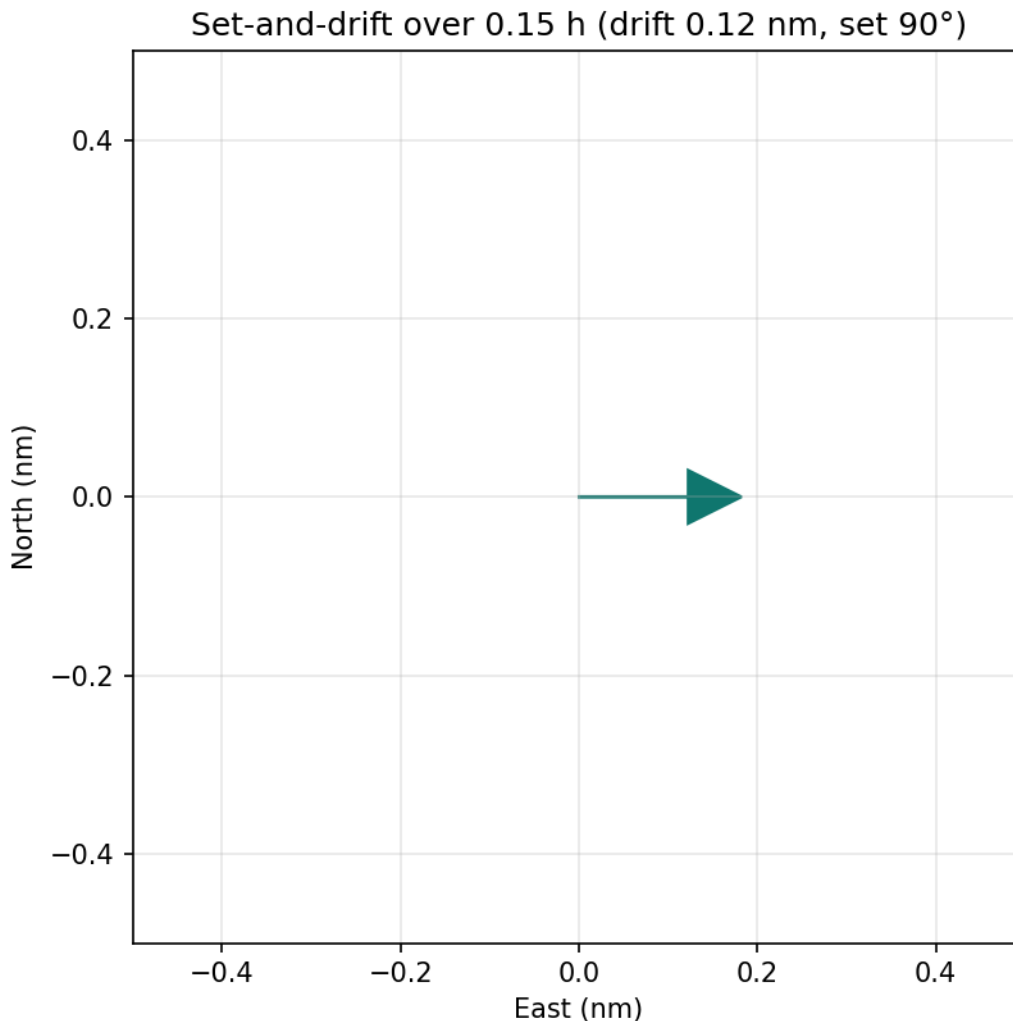


Figure 16: Set-and-drift geometry over the planned bottom time, computed from the dive parameters rather than pinned to a literal.

7.5.7 Interdiction results

The threat track (course 270 degrees at 20.0 knots, closing at **23.30 knots**) has its closest point of approach at **3.344 nautical miles** after **0.1420 hours**. That CPA is outside the 1.0-nautical-mile minimum separation, and the CPA is by construction the smallest range the threat ever reaches, so minimum separation is never breached and the verdict turns on the gap crossing alone. Crossing the 45.00-degree inter-escort sector at 2.0 nautical miles takes **0.0765 hours**, against an escort reaction time of 0.05 hours; the screen is rated **held**.

7.6 Conclusion — THUNDERBALL: findings, verdict, and what the mission establishes

The **DISCO VOLANTE** mission software turns the planning questions of an underwater nuclear-recovery operation into deterministic, reproducible computations. Dive planning resolves the tension between air budget and decompression obligation; convoy integrity quantifies the escort screen; submersible logistics reports the powerplant's reach; passive sonar fixes the detection envelope; set-and-drift navigation corrects the return leg; and interdiction analysis verifies the screen can respond to a closing threat.

The flagship profile closes green: the screen is rated **ok** with a largest escort spacing of 45.00 degrees and no uncovered sector at all (0.00 degrees), the dive is bound by **deco** — the no-decompression limit of 9.17 minutes runs out well before the 24.78-minute air supply, leaving 1564.5 L of gas in hand — sonar acquires the site at 21.78 nautical miles, the drift of 0.122 nautical miles stays within the recovery radius, and the closing threat is **held** at 3.344 nautical miles. Because every result derives from deterministic math carrying a provenance hash, the mission can be re-executed and audited exactly.

That the dive is decompression-bound rather than gas-bound is the single most consequential number in the package, and it is the one an implementation error is most likely to invert. A transposed M-value line, or a no-decompression limit evaluated at working depth instead of at the surface ascent ceiling, inflates the limit at 40 m by more than an order of magnitude and silently flips the planner’s advice from “surface on your obligation” to “surface on your gas”. The suite pins the limit against the *U.S. Navy Diving Manual*’s published air no-decompression limits [Naval Sea Systems Command, 2016] rather than against its own arithmetic, so the inversion cannot recur unnoticed. Those limits come from the Navy’s own EL/VVAL-18 lineage, not from ZH-L16A: the test asserts that this package’s ZH-L16A limits land inside bands bracketing them, which is an agreement check between two independent models, not a table lookup.

7.7 Experimental Setup — THUNDERBALL: canonical scenarios, parameters, and configuration

All parameters are declared in `manuscript/config.yaml` and loaded by `src/thunderball/manuscript_variables.py`; no numeric result is hand-authored in prose.

7.7.1 Dive parameters

Parameter	Value
Working depth	40 m
Cylinder capacity	12 L
Fill pressure	207 bar
SAC rate	20.0 L/min

7.7.2 Convoy parameters

Parameter	Value
Escort bearings (deg)	0, 45, 90, 135, 180, 225, 270, 315
Detection half-angle	30°
Coverage target	0.90

7.7.3 Submersible parameters

Parameter	Value
Battery energy	400 kWh
Drag coefficient	0.035
Hotel load	2.0 kW

7.7.4 Sonar parameters

Parameter	Value
Frequency	10.0 kHz
Source level	150 dB
Ambient noise	75 dB
Directivity index	15 dB

7.7.5 Navigation parameters

Parameter	Value
Return-leg track	315°
Own speed	4.0 kn
Current set	90°
Current drift	0.80 kn
Recovery radius	1.0 nm

7.7.6 Interdiction parameters

Parameter	Value
Threat course	270°
Threat speed	20.0 kn
Screen radius	2.0 nm
Escort reaction time	0.05 h

7.7.7 Environment

- Package version: 0.1.0
- Python: 3.14.6
- Manuscript date (from `config.yaml`, not the clock): 2026-08-04T00:00:00+00:00

The mission runs under a fixed seed (dive planning, Bühlmann ZH-L16A decompression, no-decompression limit, air consumption, convoy integrity, escort coverage, submersible logistics, range and endurance, passive sonar, Thorp absorption, set-and-drift navigation, closest point of approach, convoy interdiction, deterministic mission software include deterministic mission software); the six domain modules are pure functions of these parameters.

7.8 Reproducibility — THUNDERBALL: verification gates, deterministic regeneration, and artifacts

DISCO VOLANTE is deterministic by construction:

- **Fixed seed** — no random draws anywhere; results are pure functions of the config parameters.
- **No wall-clock dependence** — `GENERATION_TIMESTAMP` is the only timestamp, and it is read from the committed `paper.date` in `manuscript/config.yaml`, never from the clock. Regenerating the artifacts on any later day rewrites the same bytes with no flag; `--now` exists only to pin a different value deliberately. The one environment-derived token is `PYTHON_VERSION`, so the byte-identical claim holds for a fixed interpreter.
- **Computed tokens** — every `{RESULT_*}` value in this manuscript is recomputed from `src/` domain math, never hand-authored.
- **Provenance** — every `MissionOutcome` carries a fixed input hash and seed, so an outcome can be audited without re-running the film.

The concept figures (`../figures/ndl_curve.png`, `convoy_screen.png`, `submersible_range.png`, `transmission_loss.png`, `drift_geometry.png`) are regenerated deterministically. The claim is bound, not asserted: `tests/test_regeneration_determinism.py` runs the hydration CLI twice, more than a clock tick apart, and compares every written byte of the token JSON and all five PNGs.

7.8.1 Test suite

The package gates on line+branch coverage of `src/` at or above 90%. The suite covers the dive planner, convoy integrity, submersible logistics, passive sonar, current navigation, convoy interdiction, the mission protocol adapter, manuscript-token hydration, figure generation, and subprocess smoke tests of the mission CLIs. All tests use real computation and real protocol types — there are no mocks. Live counts are regenerated into `docs/_generated/COUNTS.md`; this manuscript quotes the gate, not a figure that would go stale.

7.8.2 External anchors

Reproducing a number is not the same as it being right, so the constants that carry a physical claim are pinned against published values rather than against the implementation’s own arithmetic:

- Bühlmann surfacing ceilings — 3.27 bar for the fastest compartment, 1.28 bar for the slowest [Buehlmann, 1984].
- No-decompression limits at 18, 30, 40 and 60 m, asserted to fall inside bands bracketing the *U.S. Navy Diving Manual’s* published air no-decompression limits of 60, 20, 10 and 5 minutes at those depths [Naval Sea Systems Command, 2016]. The Navy manual publishes limits from its own EL/VVAL-18 model lineage, not Bühlmann tables, so this anchor is a cross-model agreement check — a ZH-L16A implementation landing inside a band around an independently derived operational limit — and not a comparison of ZH-L16A against itself.
- The analytic optimal-range speed, cross-checked against a dense numerical sweep of the range curve.
- The pointwise coverage predicate, cross-checked against the aggregate coverage fraction over a 3600-sample sweep of the compass.

A test that only restates the formula under test cannot catch a transposed formula. These anchors can.

7.9 Scope and Related Work — THUNDERBALL: boundaries, positioning, and relationship to the literature

7.9.1 Scope

DISCO VOLANTE covers six planning problems drawn from *Thunderball* (1965): dive planning under a Bühlmann ZH-L16A decompression model, convoy-screen integrity, submersible range and endurance, passive-sonar detection, set-and-drift navigation, and

dynamic convoy interdiction. Each is a deliberately small, fully-tested, deterministic core, sized to demonstrate the BOND mission protocol rather than to plan a real operation.

The package makes no claim to full naval mission planning. In particular it omits:

- **Mixed gas.** The decompression model is air-only: no trimix or nitrox compartment sets, no gradient factors, no repetitive-dive residual nitrogen, and no ascent-rate violation penalties. It is representative, not a certified dive computer, and nothing here should be used to plan a real dive.
- **Ocean physics.** The sonar model is a single-path range equation over spherical spreading and Thorp absorption: no sound-speed profile, no multipath or convergence-zone structure, no bottom or surface scattering, and no probabilistic fluctuation term — so it yields a range, not a probability of detection. The current field is a single uniform set-and-drift vector, with no tidal variation, depth shear, or weather.
- **Multi-target tactics.** Convoy metrics are angular (gaps, coverage, integrity score) plus a single CPA-tracked threat. There is no sensor fusion, no multi-track association, no weapon-target assignment, and no wargaming of escort manoeuvre.
- **Teams.** One diver, one submersible, one threat track — no buddy pairs, no team gas-sharing, no formation logistics.

7.9.2 Related work

- **Decompression modelling** — Bühlmann’s ZH-L16 family is the classical dissolved-gas framework and the direct source of the coefficient tables and the tolerated-ambient-pressure formulation used here [Buehlmann, 1984]. The *US Navy Diving Manual* is the standard operational reference against which no-stop limits are conventionally sanity-checked [Naval Sea Systems Command, 2016]; the test suite pins this package’s limits inside bands around those published air limits rather than against its own arithmetic. The manual’s limits come from the Navy’s own EL/VVAL-18 model lineage, not from ZH-L16A, so the check is a cross-model agreement test and not a lookup of the model under test. Modern dive computers add gradient factors and bubble-model variants that this package intentionally omits.
- **Underwater acoustics** — Urick is the standard treatment of the sonar equation, transmission loss, and detection threshold [Urick, 1983b]; the frequency-dependent absorption coefficient is Thorp’s [Thorp, 1967a].
- **Marine hydrodynamics** — the cubic speed–power drag law is the standard first-order model for submerged resistance and propulsive power [Newman, 1977]; the constant hotel-load term is the minimal extension needed for an interior range optimum to exist at all.
- **Navigation and collision geometry** — set-and-drift, the current vector triangle, and course-to-steer follow Bowditch [Bowditch, 2002]. Relative-motion CPA/TCPA geometry is the basis of collision-avoidance practice, for which Cockcroft and Lameijer is the standard practitioner treatment [Cockcroft and Lameijer, 2011].
- **Perimeter screening** — coverage and gap analysis of a defended perimeter is standard in convoy-defence and sensor-placement studies; the angular metric here is a reduced, deterministic form of it.

As a BOND suite film, this package’s primary relation is to the frozen `bond_api` protocol [BOND, 2026d], shared across the film packages, and to the suite *orchestrator*, which drives any `MissionProvider` through the five-stage lifecycle.

7.10 Discussion and Uncertainty — THUNDERBALL: interpretation, caveats, and what the numbers do not claim

Every number in the results section is the deterministic output of a closed-form or bounded-iteration model over explicit parameters in `manuscript/config.yaml`. That determinism gives the package its reproducibility, but it also means the precision of a token — two or three decimal places — is **not** accuracy. Each of the six models rests on idealising assumptions, and where those assumptions break is where the uncertainty actually lives. This section states that honestly for each capability.

7.10.1 The central finding is robust, not borderline

The headline result — that **decompression, not gas, binds the flagship dive** — is the single most consequential number in the package, and it is also the most robust one. The no-decompression limit of 9.17 minutes and the air-supply limit of 24.78 minutes are not close: the air budget outlasts the no-stop obligation by more than a factor of two at the working depth of 40 m, so the conclusion that the team surfaces on its decompression obligation survives a wide swath of parameter error and is not a borderline tie. The suite deliberately pins the limit against an independent published band [Naval Sea Systems Command, 2016] precisely because a transposed M-value or a depth-evaluated ceiling could silently invert this call (see the results section).

7.10.2 Dive planning uncertainty

The ZH-L16A model is air-only and single-dive, and it treats every diver identically: it takes one 20.0 L/min SAC rate, no gradient factors, no repetitive-dive residual nitrogen, and no human physiological variation. Real no-stop limits vary substantially across divers, workloads, thermal stress, and ascent profiles; a representative model cannot claim those. An operational planner would add conservatism (safety/gradient factors) well beyond what a bare M-value line provides.

7.10.3 Convoy-screen uncertainty

The screen model is angular and static: escorts sit at fixed bearings, each watching an idealised circular arc of the configured half-angle; the flagship screen reports 0.00 degrees of genuinely unwatched perimeter. A real escort’s detection is probabilistic, range-limited, and environment-dependent, so a computed zero-degree uncovered sector is a statement about the *model geometry*, not about the physical certainty that no threat slips through. The integrity score’s sigmoid knee at a nominal 60° and the configurable hard sector ceiling are design choices, not physical constants.

7.10.4 Submersible-logistics uncertainty

The cubic speed-power drag law plus a constant hotel load is a first-order propulsion idealisation. The analytic optimal speed (3.057 knots, from the methodology section) is a true extremum of that idealised curve but inherits the uncertainty of the drag coefficient and hotel load, which in reality vary with loading, fouling, temperature, and depth. Battery energy likewise derates with age and cold that the constant 400 kWh figure does not capture.

7.10.5 Passive-sonar uncertainty

The sonar model is a single-path, spherical-spreading-plus-absorption range equation. It yields a *range* (21.78 nm), not a probability of detection: there is no multipath, convergence-zone, bottom or surface interaction, and no fluctuation term. Real detection is inherently probabilistic, and its strongest driver — ambient noise (75 dB) — is among the most variable inputs in the scenario, so the computed range should be read as a representative figure under the stated quieting, not a guaranteed acquisition distance.

7.10.6 Navigation uncertainty

Set-and-drift is modelled as a single uniform current vector. Real currents shear with depth, vary over time, and interact with weather; the course to steer and the drift figure (0.122 nm, from the results section) hold exactly under the assumed constant current and degrade continuously as that assumption is violated. Any error between the assumed and actual set/drift translates directly into cross-track error on the return leg.

7.10.7 Interdiction uncertainty

The interdiction verdict tracks a single threat under constant velocity. held is a race against a nominal escort reaction time (0.05 h); reaction time is operationally the most uncertain quantity in the chain, so the categorical verdict should be read as a sensitivity to that one number rather than a fixed outcome. The infinite time-to-minimum-separation of a threat whose CPA lies outside the limit is a “never under constant velocity”, not an absolute guarantee.

7.10.8 What would reduce these uncertainties

Each open item in the package backlog (TODO.md, *Extension*) is exactly the modelling refinement that would shrink the corresponding uncertainty here: gradient factors and mixed gas for the dive model, a probabilistic (Marcum) detection layer for sonar, a depth-sheared or time-varying current for navigation, and multi-track association for interdiction. They are out of scope for this deterministic, six-capability mission package, which is sized to demonstrate the BOND mission protocol rather than to plan a live operation.

7.11 Sources — THUNDERBALL: bibliography

Buehlmann [1984]; Urick [1983b]; Thorp [1967a]; Bowditch [2002]; Newman [1977]; Cockcroft and Lameijer [2011]; Naval Sea Systems Command [2016]; Fleming [1961]; BOND [2026d]

8 You Only Live Twice (1967) — BIRD ONE

film package · *package codename* *BIRD ONE*. Mission **VOLCANO**: solve space-capsule rendezvous and intercept windows; engineer the orbital capture and phasing of the enemy capsule; detect the concealed volcano-camouflaged lair; localize the hidden base by Bayesian cue fusion and RF emission; plan the infiltration route into the lair; characterize the adversary base pattern of life.

8.1 Concepts — BIRD ONE: domain and operational focus

orbital capture, hidden-base discovery

8.2 Abstract — BIRD ONE: mission summary

YOU ONLY LIVE TWICE (1967), mission codename VOLCANO, is special-agent mission software for the adversary-prosecution problem a single agent faces when the enemy’s base is hidden inside a volcano and the enemy’s capsule must be swallowed out of orbit. This package implements seven deterministic, quantitative capabilities and reports them through the frozen BOND-API mission protocol. The **rendezvous / intercept solver** propagates two circular orbits under two-body Keplerian mechanics and finds every contiguous window where the chase spacecraft can acquire the target: the nominal scenario yields 1 window(s) in a one-day horizon, opening at 32940.0 seconds with a peak elevation of 82.77°, and a coplanar Hohmann intercept costs 133.4 m/s over 2943.0 seconds. The **orbital capture** model then turns this into a phasing problem — the platform waits 31989.1 seconds for the 4.69° lead angle and grapples with a 982.8 m/s velocity-matching burn inside 1 capture window(s). The **concealed-lair detector** fuses thermal and terrain anomaly scoring to locate a volcano-camouflaged structure, reporting 6 candidate region(s), the strongest with a fused score of 8.55. The **hidden-base discovery** engine then fuses those cues with passive RF and a terrain-weighted prior, reaching the injected 80-cell lair footprint at rank 1 of 4096 candidate sites (P-found-within-10-sites 0.9999); the injected *centre cell* is not recovered — it ranks 80, 5656.9 m from the posterior peak. The **infiltration planner** routes an agent to the lair on a slope/thermal/guard-exposure cost surface, cutting route cost by 36.11 % over a goal-reaching greedy baseline, and the **emitter locator** fixes the base’s transmitter by TDOA multilateration to within 3.7912 m using 5 passive sensors whose timings carry a 10.0 ns jitter, backed by a deterministic 6-level error envelope vs jitter. Finally the **pattern-of-life analyzer** reads a week of base activity (311 events), finding 2 daily operating window(s) (07-09h; 19-21h) with a medium operational tempo and duty cycle 0.679. Every number above is computed, never hand-authored: the manuscript renders entirely from the deterministic scenario (seed 1967, provenance 69984c9e423de2cf).

8.3 Introduction — BIRD ONE: mission framing, the operational problem, and how to read this chapter

In *You Only Live Twice* (1967), the adversary’s base is a volcano that swallows spacecraft whole, and the mission is won by knowing *where* the enemy is and *when* they act. The VOLCANO package turns that intelligence problem into seven computational ones an agent can solve ahead of time with deterministic, reproducible mathematics:

1. **When can we reach them?** Orbit geometry decides the rendezvous windows.
2. **How do we capture them?** Phasing decides when the capsule can be grappled and its velocity matched.
3. **Where are they hiding?** A concealed base leaks heat and flattens terrain, even when painted to look like a mountain.
4. **Does the evidence point to one site?** Bayesian fusion turns heterogeneous sensor cues into a search plan — and reports whether any single cue would have sufficed, rather than assuming fusion was what found the base.
5. **How do we get in?** A*, not guesswork, plans the infiltration route.
6. **Where is their transmitter?** TDOA multilateration fixes the base’s radio.
7. **How do they operate?** A base keeps a rhythm; activity events reveal it.

The package is built in the BOND suite’s style: a pure, infrastructure-free domain core (the seven capabilities), a thin BOND-API adapter exposing it as a **MissionProvider** (codename VOLCANO), and a manuscript that hydrates every numeric claim from a fixed scenario rather than from prose.

8.3.1 Reader’s guide

- `rendezvous_windows.py` — orbital mechanics for rendezvous / intercept window solving.
- `orbital_capture.py` — capture envelope, velocity-matching burn, phasing.
- `terrain_anomaly.py` — thermal + terrain anomaly scoring for lair detection.
- `hidden_base_discovery.py` — Bayesian cue fusion and search planning.
- `infiltration_planning.py` — terrain/thermal/guard cost surfaces and A*.
- `emitter_localization.py` — TDOA hyperbolic emitter localization.
- `pattern_of_life.py` — weekly activity analysis.
- `mission.py` — the VOLCANO **MissionProvider**.
- `scripts/run_mission.py` — a thin CLI for every mission phase (**brief** / **recon** / **plan** / **execute** / **debrief**).

The remainder of the manuscript details the methodology (Section 2), the results (Section 3), the experimental setup (Section 5), and reproducibility (Section 6).

8.4 Methodology — BIRD ONE: the analytical models and algorithms that drive the mission

This section states the mathematics behind each of the seven capabilities. Everything is standard, deterministic, and free of hidden randomness.

8.4.1 1. Rendezvous / intercept window solving

Both spacecraft are modelled as two-body circular orbits. The mean orbital motion of a circular orbit of radius a under gravitational parameter μ is

$$n = \sqrt{\frac{\mu}{a^3}}$$

and the position and velocity in the Earth-centred inertial (ECI) frame follow from the classical orbital elements $(a, e = 0, i, \Omega, \nu)$ via the standard perifocal $\rightarrow$ ECI 3-1-3 rotation. The true anomaly advances linearly, $\nu(t) = u_0 + nt$, so a candidate orbit is fully described by its altitude, inclination, RAAN, and initial argument of latitude u_0 .

Given the chaser state $(\mathbf{r}_c, \mathbf{v}_c)$ and the target position $\mathbf{r}_t$, the line-of-sight vector is $\rho = \mathbf{r}_t - \mathbf{r}_c$ with slant range $\rho = \|\mathbf{r}_t - \mathbf{r}_c\|$ and target elevation ε above the chaser’s local horizontal plane. A **rendezvous window** is a contiguous interval on which $\rho \leq \rho_{\max}$ and $\varepsilon \geq \varepsilon_{\min}$. The intercept burn is a coplanar Hohmann transfer with the two-impulse budget Δv and transfer time equal to half the transfer-orbit period.

8.4.2 2. Orbital capture and phasing

Capturing the capsule is a *phasing* problem [Bate et al., 1971a, Vallado, 2013a, Fehse, 2003]. The chaser (the capture platform on the target’s orbit) must wait until the target leads it by exactly the angle

$$\lambda = (\pi - n_t T_t) \bmod 2\pi,$$

where T_t is the Hohmann transfer time and n_t the target mean motion — the transfer sweeps π rad, so the target must start ahead by $\pi - n_t T_t$. The relative phase evolves as $\phi(t) = \phi_0 + (n_t - n_c)t$, so the **phasing wait time** is the first non-negative solution of $\phi(t) \equiv \lambda \pmod{2\pi}$.

A **capture window** is a contiguous interval where the capsule is inside the grapple envelope (range constraint) *and* slow enough that the platform can match its velocity; the **velocity-matching burn** is exactly the relative speed $\|\mathbf{v}_t - \mathbf{v}_c\|$ at that epoch, nulling the relative velocity for a physical grapple.

8.4.3 3. Concealed-lair detection via thermal + terrain anomaly

A concealed lair is found by scoring each channel against a locally modelled background — the standard formulation of statistical anomaly detection, where “anomalous” means *improbable under the fitted background model* rather than merely bright [Reed and Yu, 1990, Chandola et al., 2009]. A Gaussian low-pass $\tilde{g} = K_\sigma * g$ approximates the smooth natural signal, and each pixel is expressed as a standardized residual $z_{ij} = (g_{ij} - \tilde{g}_{ij})/\sigma_{\text{res}}$. The **thermal anomaly** is the one-tailed z (we look for excess heat); the **terrain anomaly** is the magnitude $|z|$ (a lair may be a raised shelf *or* an excavated depression). The two maps are fused with weights w_T, w_D as $s_{ij} = (w_T z_{ij}^T + w_D z_{ij}^D)/(w_T + w_D)$. Connected components of the thresholded fused map are labelled and reported as candidate regions, ranked by peak fused score.

8.4.4 4. Hidden-base discovery by Bayesian cue fusion

To decide *where* the base is before a strike, the mission treats each of the N candidate cells as a hypothesis and applies Bayesian belief updating [Pearl, 1988].

What counts as ground truth here. The evaluated truth is the lair the scene generator injects — its centre cell and its carved footprint — determined by the cone centre and a fixed offset before any anomaly map exists. It is not the argmax of the fused anomaly map, and the passive-RF cue is generated at the injected site rather than at the detector’s answer. Nothing the posterior consumes is derived from the quantity the posterior is asked to recover, so the reported ranks are measurements and not identities.

Cell c carries a prior $P(c) \propto 1 + \beta \max(0, z_c^D)$ (terrain anomalies raise the prior). Each sensor cue z is modelled as a Gaussian measurement with likelihood ratio

$$\text{LR}(z) = \exp\left(\frac{\mu z}{\sigma^2}\right),$$

so a neutral cell maps to 1, a hot cell to > 1 , a cold cell to < 1 . The posterior is the normalized product

$$P(c \mid \text{cues}) \propto P(c) \prod_i \text{LR}_i(c).$$

Searching greedily by descending posterior gives a deterministic **search order** — the optimal policy for a uniform-cost, perfect-detection search [Stone, 1975]. Two distinct effort statistics are reported and must not be conflated: the **realised effort** is the rank of the known true site (an after-the-fact validation number), while the **posterior expected effort** $E[K] = \sum_k k P(c_k)$ is a property of the posterior alone and is the number a planner can quote before the base is confirmed. The **discovery curve** $\sum_k P(c_k)$ is the probability the base is found within the first k searches. Combining heterogeneous sensors this way is the standard multisensor-fusion construction [Waltz and Llinas, 1990].

8.4.5 5. Infiltration planning

The agent must reach the lair core while minimizing detection risk. Each cell carries a movement cost

$$\text{cost} = 1 + w_s \hat{s} + w_T z_+^T + w_G \hat{g},$$

combining normalized slope $\hat{s}$, positive thermal-exposure z_+^T , guard line-of-sight exposure $\hat{g}$ (an elevation-occluded visibility count from every watch post), and an impassable-lava mask. A deterministic 4-connected $\mathbf{A}^*$ search [Hart et al., 1968a, Cormen et al., 2009b] with an admissible Manhattan heuristic returns the optimal route — admissibility holds because every step costs at least 1 and the heuristic counts steps, so $\mathbf{A}^*$ is guaranteed optimal.

The baseline is a greedy depth-first walk over the *same* search space: it always steps to the cheapest-looking unvisited neighbour and retraces one cell when it dead-ends, paying that cell’s cost again. Two conditions are needed for the reported reduction to be non-negative by optimality rather than by a favourable comparator, and both are enforced in code. The baseline must live in the same 4-connected space over the same cost field — it does — *and* it must actually reach the goal, because $\mathbf{A}^*$ minimises over goal-reaching walks only. A baseline that abandoned the search at a dead end would report the cost of a shorter journey, which can be below the optimal cost of the full journey and would make the reduction negative; the implementation therefore backtracks and raises when the goal is unreachable instead of returning a truncated route. `tests/test_infiltration_planning.py` pins a concrete grid on which the non-backtracking version returns a truncated walk cheaper than the optimum.

8.4.6 6. Emitter localization by TDOA

Passive sensors at positions $\mathbf{s}_i$ time the arrival of the base’s RF burst. The range differences against a reference sensor, $d_i = \|\mathbf{e} - \mathbf{s}_i\| - \|\mathbf{e} - \mathbf{s}_0\|$, define hyperbolae whose intersection is the emitter $\mathbf{e}$ [Skolnik, 2001a, Chan and Ho, 1994a]. The estimate $\hat{\mathbf{e}}$ minimizes the residuals via the Taylor-series (Gauss–Newton) iteration on the normal equations [Foy, 1976a], which converges to the true emitter for exact measurements. Receivers report *time* differences; these are converted to the range differences the geometry is solved in by multiplying by the propagation speed, so the speed of light never enters the estimator itself.

Exact measurements make the solve a round-trip identity of the forward model: it returns the injected point to machine precision by construction, which checks solver–model consistency but measures no accuracy. The reported figure therefore comes from receiver timings perturbed by a fixed-seed Gaussian jitter of σ_t before the conversion to ranges, so the error reflects a geometry-dependent dilution of precision and can degrade.

8.4.7 7. Pattern-of-life analysis

A base’s activity is modelled as an inhomogeneous Poisson point process [Daley and Vere-Jones, 2003] whose instantaneous rate follows a diurnal envelope; events are drawn by rejection sampling against that envelope, so the injected rhythm is exactly known and the analysis can be checked against it. The observed events are binned into a 7×24 hourly histogram, giving a mean daily profile $\bar{p}_h = \frac{1}{7} \sum_d h_{d,h}$. The dominant operational periodicity comes from the normalized autocorrelation, contiguous hours above a percentile are **peak operating windows**, and tempo is classified from the mean event rate.

8.5 Results — BIRD ONE: measured outcomes, headline numbers, and what they establish

All results come from the deterministic VOLCANO scenario (seed 1967, provenance hash 69984c9e423de2cf) and are injected as tokens — none are hand-authored.

8.5.1 Rendezvous / intercept windows

Over a one-day horizon the geometry admits 1 rendezvous window(s). The first opens at 32940.0 seconds after mission start and lasts 1020.0 seconds, peaking at a target elevation of 82.77° . The rendezvous figure shows the slant-range and elevation evolution with the accepted window shaded.

For an intercept, a coplanar Hohmann transfer burns a total of 133.4 m/s and takes 2943.0 seconds.

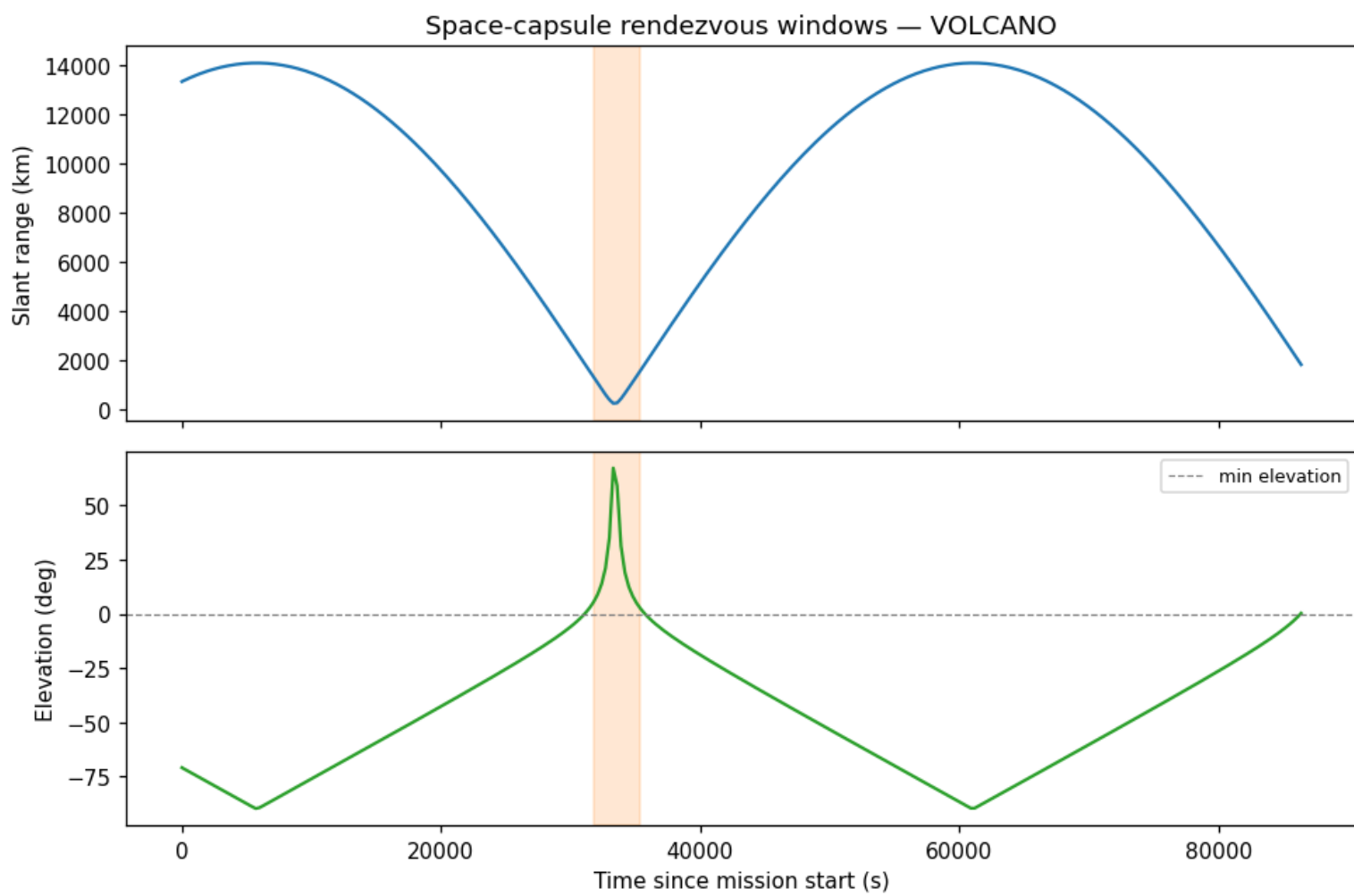


Figure 17: Rendezvous geometry: slant range (top) and target elevation (bottom) vs time, with the viable window shaded.

8.5.2 Orbital capture

Adding the capture envelope (range + relative-speed constraint) yields 1 capture window(s), the first opening at 32280.0 seconds and lasting 2400.0 seconds; the grapple still needs a velocity-matching burn of 982.8 m/s. The phasing solution requires a 4.69° lead angle, reached after a 31989.1 second wait. The capture figure shows the range and relative-speed profile with the capture window shaded.

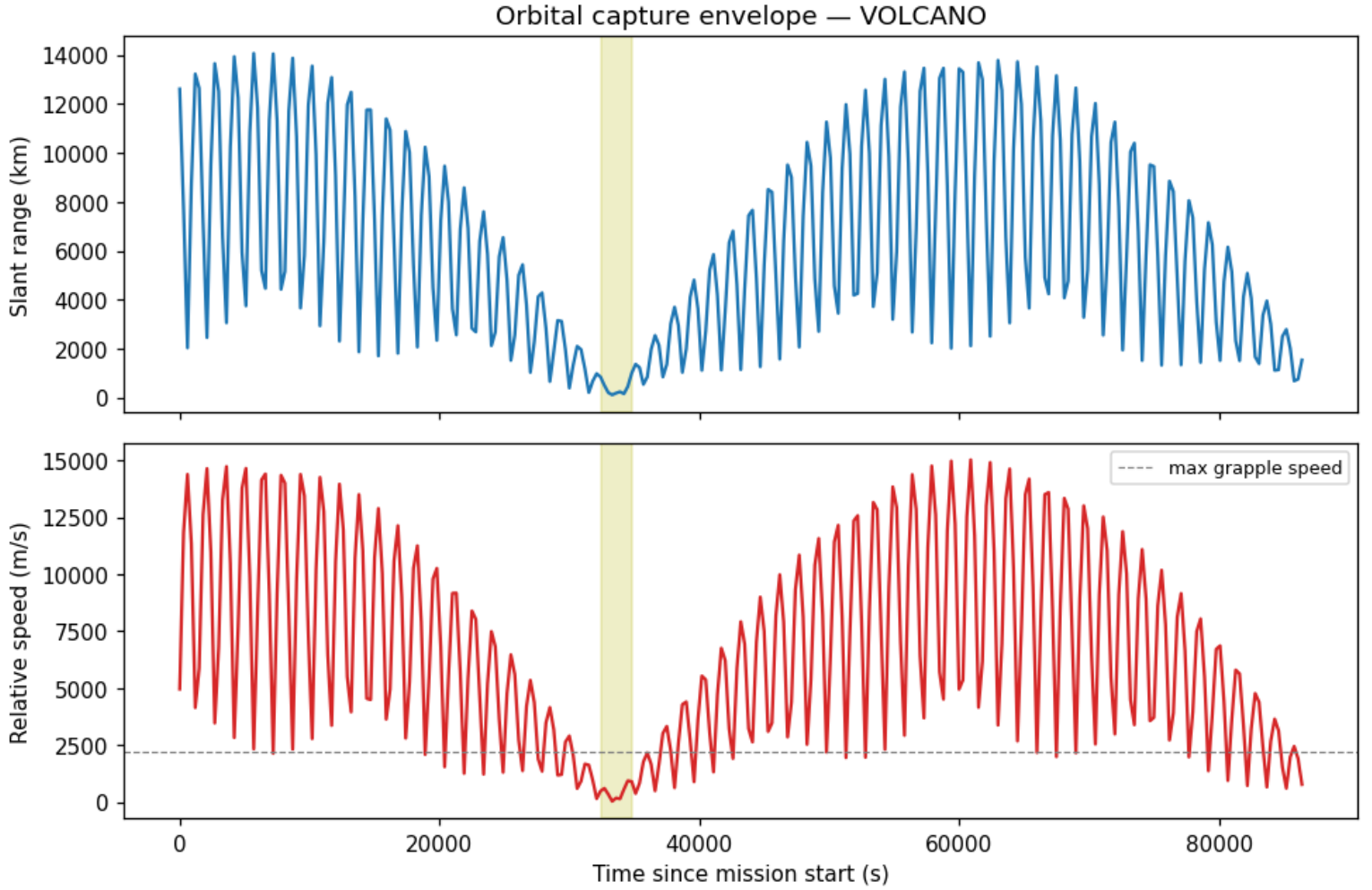


Figure 18: Orbital capture: slant range (top) and relative speed (bottom) vs time, with the grapple window shaded.

8.5.3 Concealed-lair detection

The thermal and terrain anomaly fusion surfaces 6 candidate region(s); the strongest covers 272 cells with a fused score of 8.55 (thermal 7.622, terrain 9.478). The lair figure shows the raw scenes and the fused anomaly map with the top candidate boxed.

Measured against the injected facility — a 80-cell shelf carved at a location fixed by the scene geometry, before any sensor map is computed — the detector's peak falls inside the footprint (yes) but **not** on its centre: it sits 5.66 cells (5656.9 m) away. That offset is the expected behaviour of a high-pass anomaly score, which peaks on the rim of a carved shelf where the discontinuity against the smooth background model is largest, not at its centroid. The detector localizes the *facility*, not its centre point, and only the first of those claims is supported here.

8.5.4 Hidden-base discovery

The ground truth for this section is the injected lair — its centre cell and its 80-cell carved footprint — both fixed by the scene's construction parameters and *not* derived from any anomaly map the posterior consumes. The RF cue is generated at that injected site, so no cue is centred on the detector's own answer.

Fusing the thermal, terrain and RF cues with a terrain-weighted prior over the 4096 candidate sites, a greedy posterior-order search first enters the injected footprint at rank 1, and the footprint carries posterior mass 1.0 — the mass outside it falls below the reported precision. The footprint is 80 of 4096 cells, so this is a real hit rather than an arithmetic certainty.

The stricter question — does the posterior peak land on the injected *centre cell*? — is answered **no**. The centre cell ranks 80 with posterior $1.08\text{e-}15$, and the peak sits 5656.9 m from it, for the same rim-versus-centroid reason as above. This paper claims footprint-level recovery only; cell-level recovery of the centroid is not established.

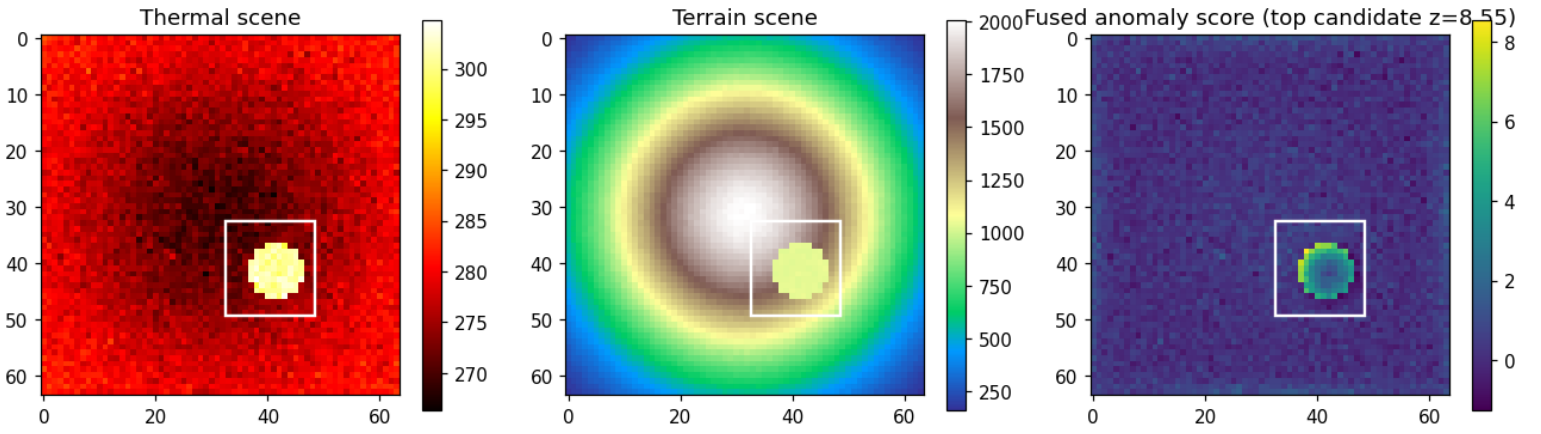


Figure 19: Concealed-lair detection: thermal scene, terrain scene, and fused anomaly score with the top candidate region marked.

Fusion was also not what made the find. Scoring each cue on its own (prior x that single likelihood-ratio map) gives footprint ranks prior_only 1, rf 1, terrain 1, thermal 1 — every individual cue, and the terrain-weighted prior by itself, already ranks the footprint first. In this scene the cues are strong, not weak; fusion sharpens the posterior and adds redundancy, and the claim that the base could only be found by combining weak sensors would be false here.

Independently of any ground truth, the posterior-weighted expected effort — the mean number of sites a greedy search inspects, $\sum_k k P(c_k)$ — is 1.05 cell(s) against a uniform-prior expectation of half the field, and the search finds a footprint cell within ten sites with probability 0.9999. The discovery figure shows the posterior map with the injected footprint, the injected centre, and the posterior peak marked separately.

8.5.5 Infiltration planning

The slope/thermal/guard cost surface routes the agent from the rim to the lair in 48 steps at a total cost of 73.834, against a greedy baseline of 115.557 — a 36.11 % cost reduction (fraction 0.3611) — accruing a thermal exposure of 2.0694 over 922 expanded cells. The infiltration figure shows the cost field and the optimal route.

8.5.6 Emitter localization

Passive TDOA multilateration over 5 sensors fixes the base’s transmitter to within 3.7912 m of the injected emitter in 6 iterations. The receivers report arrival-time differences carrying a 10.0 ns (1σ) timing jitter, applied before the conversion to ranges, so this error is an achieved accuracy that can degrade — not the machine-zero a noise-free round trip through the same forward model returns by construction. The emitter figure shows the sensor ring, the range-difference hyperbolae converging on the injected transmitter, and the converged fix, imagery candidate and injected truth marked separately.

A single operating point is not an accuracy envelope, so the deterministic sweep of 6 timing-jitter levels — each averaged over 24 fixed-seed realizations at the same emitter and sensor ring — characterises how the mean error grows with the receiver jitter: sweeping σ over (0.0, 10.0, 20.0, 50.0, 100.0, 200.0) ns gives mean errors of (0.0, 2.0, 3.6, 9.7, 19.2, 41.3) m respectively. The mean error rises from zero (the noiseless round-trip identity) to 41.3 m at 200.0 ns — a monotone envelope for this geometry, reported through the emitter envelope figure rather than asserted.

The radio and imagery chains share no measurement, so comparing them is a genuine cross-modal check, and they do **not** agree on a cell: the TDOA fix lands 5655.5 m from the imagery-derived candidate, which is essentially the whole 5656.9 m by which the anomaly peak is itself displaced from the facility centre. The two modalities agree on the facility and disagree on the point within it, by the full width of that rim-versus-centroid offset; the radio fix is the accurate one, because the emitter is where the transmitter was injected and the anomaly peak is not.

8.5.7 Pattern of life

A week of 311 base events resolves into 2 daily operating window(s) (07-09h; 19-21h), a dominant period of 12.0 hours, a duty cycle of 0.679, and a medium operational tempo. The pattern figure visualizes the weekly and daily rhythms with the peak windows shaded.

8.5.8 End-to-end mission

The BOND-API provider (codename VOLCANO) executes all seven capabilities in a single deterministic run against 11 frozen protocol dataclasses across 5 provider methods, returning full provenance.

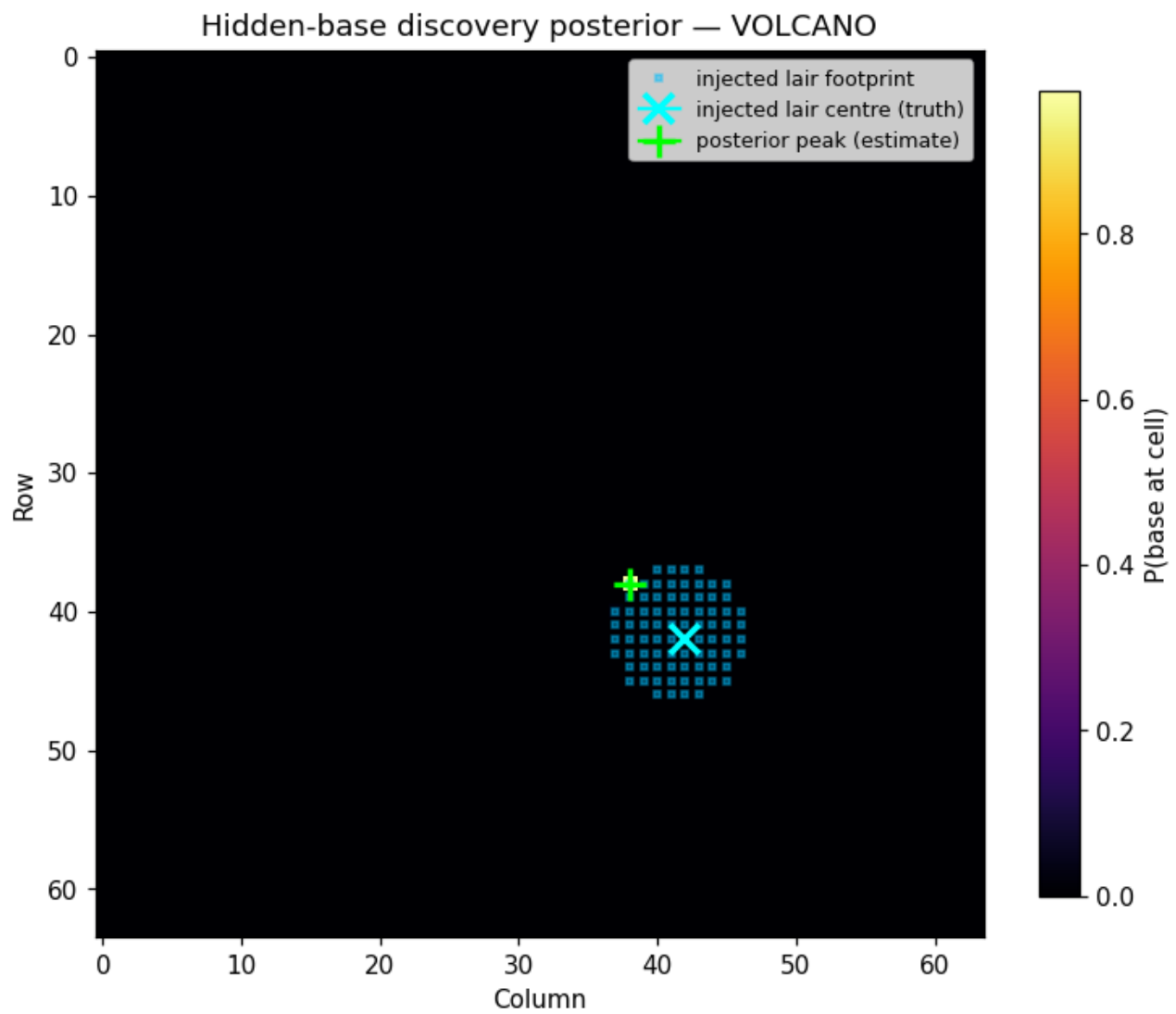


Figure 20: Hidden-base discovery: posterior probability map with the injected lair footprint, the injected centre (ground truth) and the posterior peak (estimate) marked separately.

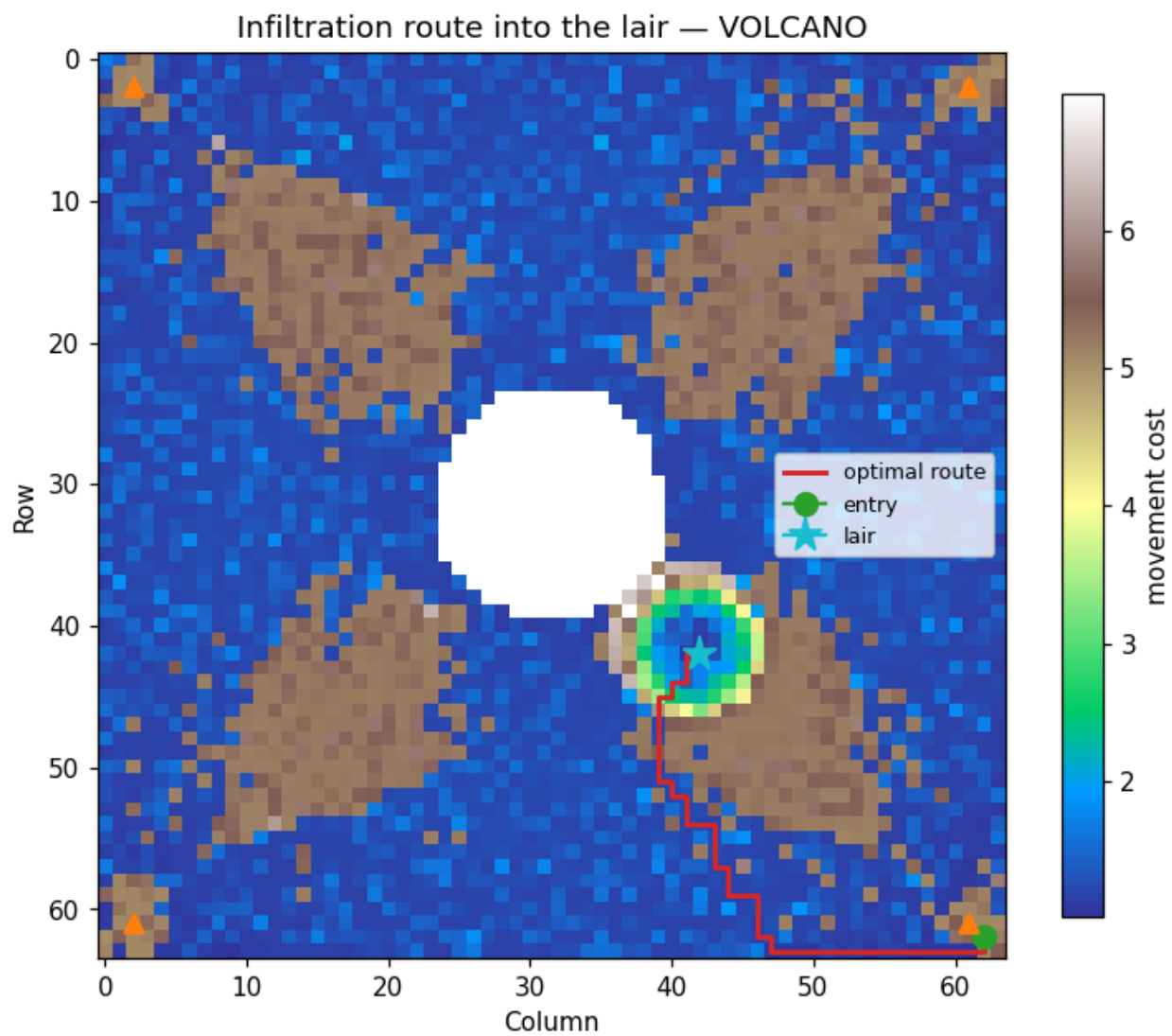


Figure 21: Infiltration route: movement-cost field with the optimal route (red), entry (green), lair (cyan) and guard posts (orange).

TDOA emitter localisation — VOLCANO

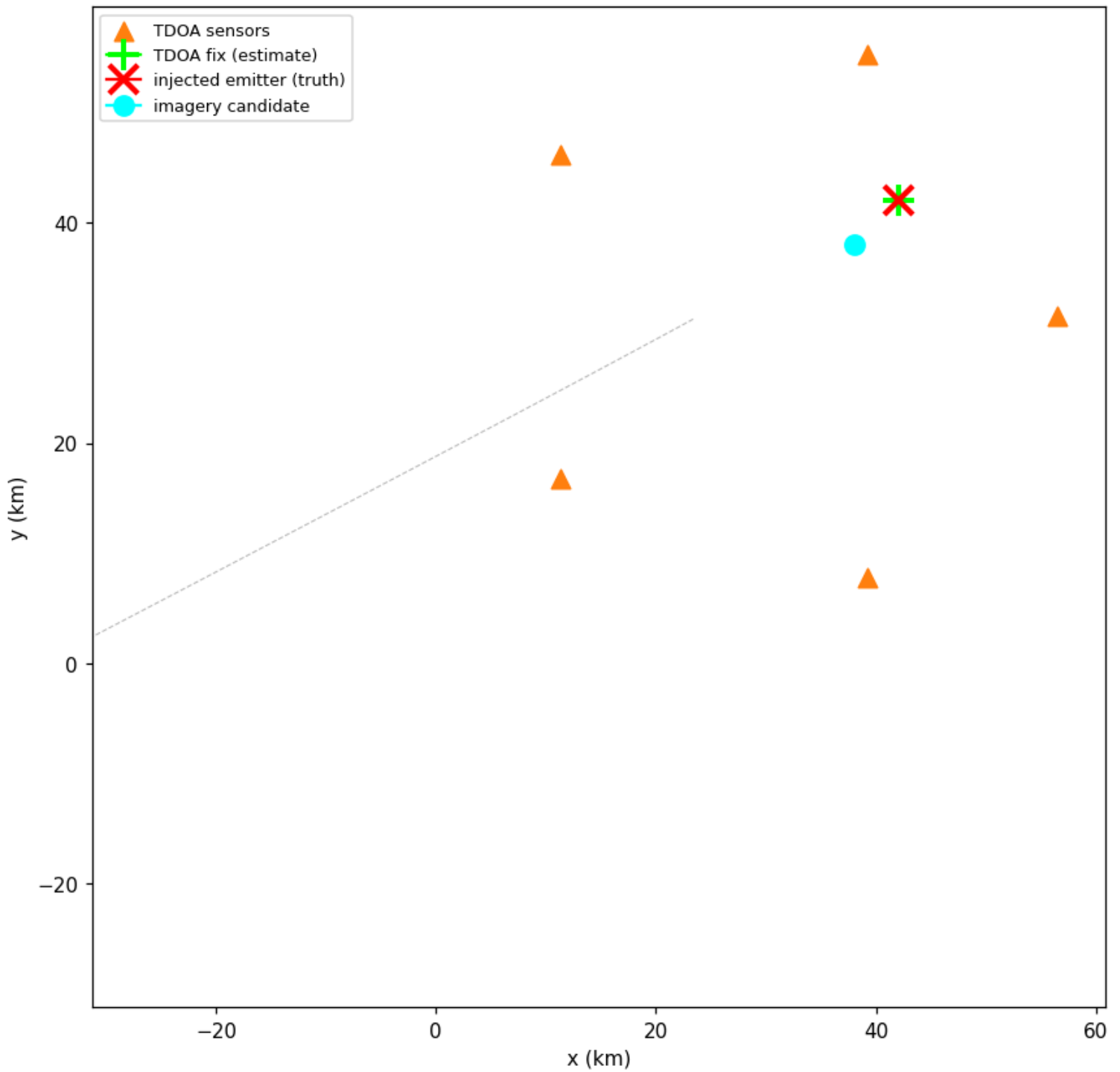


Figure 22: Emitter localization: TDOA sensor ring with the range-difference hyperbolae (grey), the converged fix (+), the injected emitter ($\times$) and the imagery-derived candidate ($\circ$).

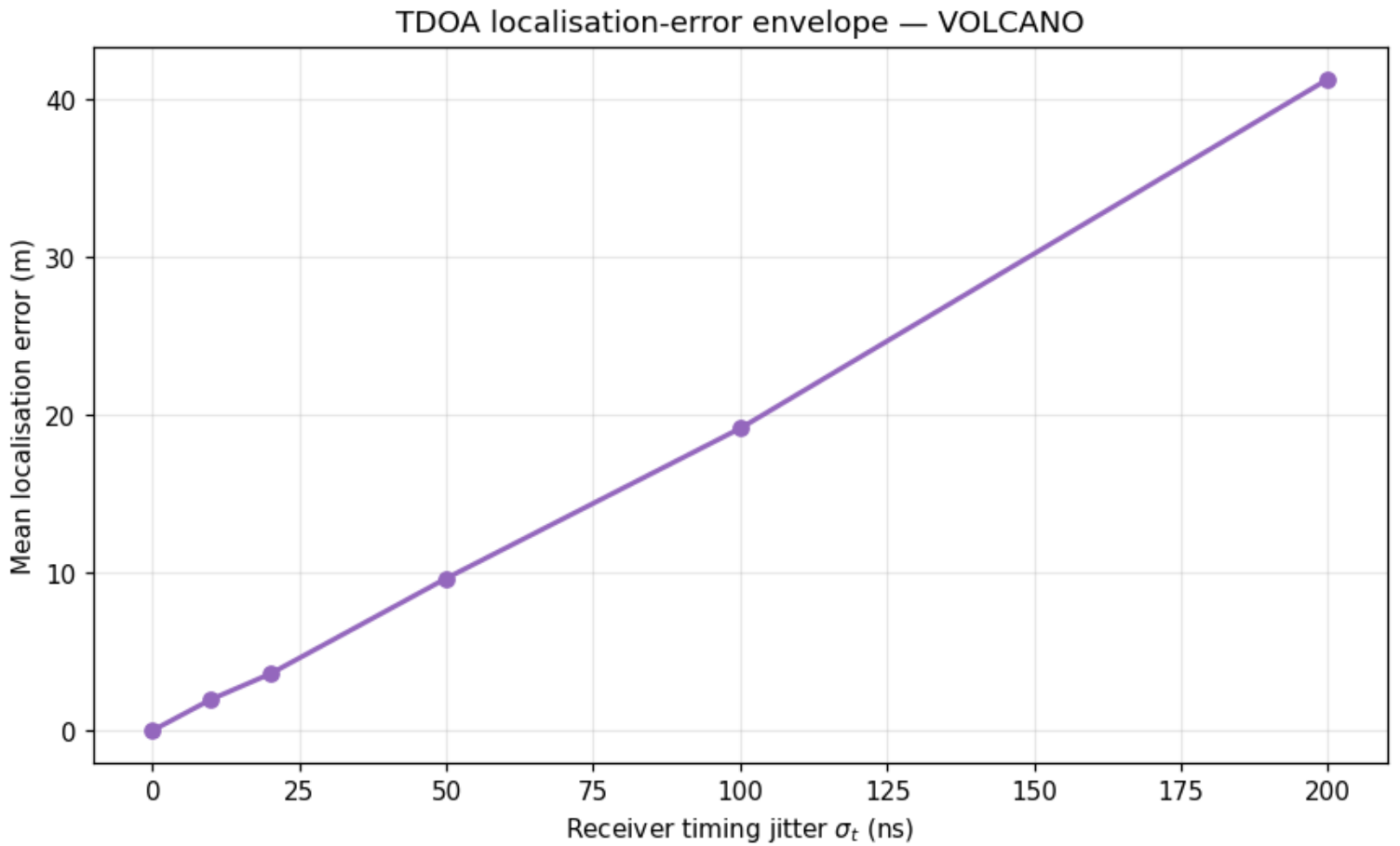


Figure 23: Emitter accuracy envelope: mean localisation error vs receiver timing jitter σ .

8.6 Conclusion — BIRD ONE: findings, verdict, and what the mission establishes

The VOLCANO package demonstrates that a single-agent mission intelligence problem — where to intercept, how to capture, where the enemy hides, how to get in, how to localize its radio, and how it operates — can be answered with deterministic, reproducible mathematics rather than guesswork. Each of the seven capabilities is a real computation over real data:

- the **rendezvous / intercept solver** turns orbital elements into actionable windows and Δv budgets;
- the **orbital capture** model turns a capture into a phasing problem with a well-defined lead angle, wait time, and velocity-matching burn;
- the **concealed-lair detector** fuses thermal and terrain anomaly scoring to surface a volcano-camouflaged structure, landing inside the injected footprint but 5656.9 m from its centre;
- the **hidden-base discovery** engine turns those cues plus passive RF into a posterior and a search plan that reaches the injected footprint at rank 1;
- the **infiltration planner** finds a provably optimal (A^*) route that measurably beats a greedy baseline over the same search space;
- the **emitter locator** fixes the base’s transmitter by TDOA to 3.7912 m from noisy receiver timings;
- the **pattern-of-life analyzer** extracts a base’s operating rhythm from a week of activity events.

Three limitations are worth stating plainly rather than leaving to a reader to discover. First, the anomaly detector and the Bayesian posterior recover the lair’s *footprint*, not its centre cell: both peak on the carved rim, 5656.9 m off the injected centroid, and the centroid ranks 80 in the search order. Second, fusion is not what finds the base in this scene — each cue alone already ranks the footprint first (prior_only 1, rf 1, terrain 1, thermal 1), so this is a demonstration of consistent multi-cue evidence, not of weak-signal fusion. Third, the scene is synthetic and single: these numbers characterise one deterministic scenario, not a detection performance envelope, which would need a swept ensemble of scenes with varied camouflage, noise and lair geometry. A partial exception is the emitter localiser, whose deterministic error-versus-jitter envelope (6 levels, 24 realizations each) is now reported alongside the operating point; the fixed emitter–sensor geometry and the single base scene remain single-scenario.

Because every result flows from a fixed seed through token-injected prose, the manuscript and the computation can never drift apart: re-running the analysis re-hydrates every number in this document. The package is one film in the BOND suite, exposing its capabilities through the frozen BOND-API mission protocol so any coordinator can drive it without knowing its internals, and keeping its pure domain core importable without the protocol at all.

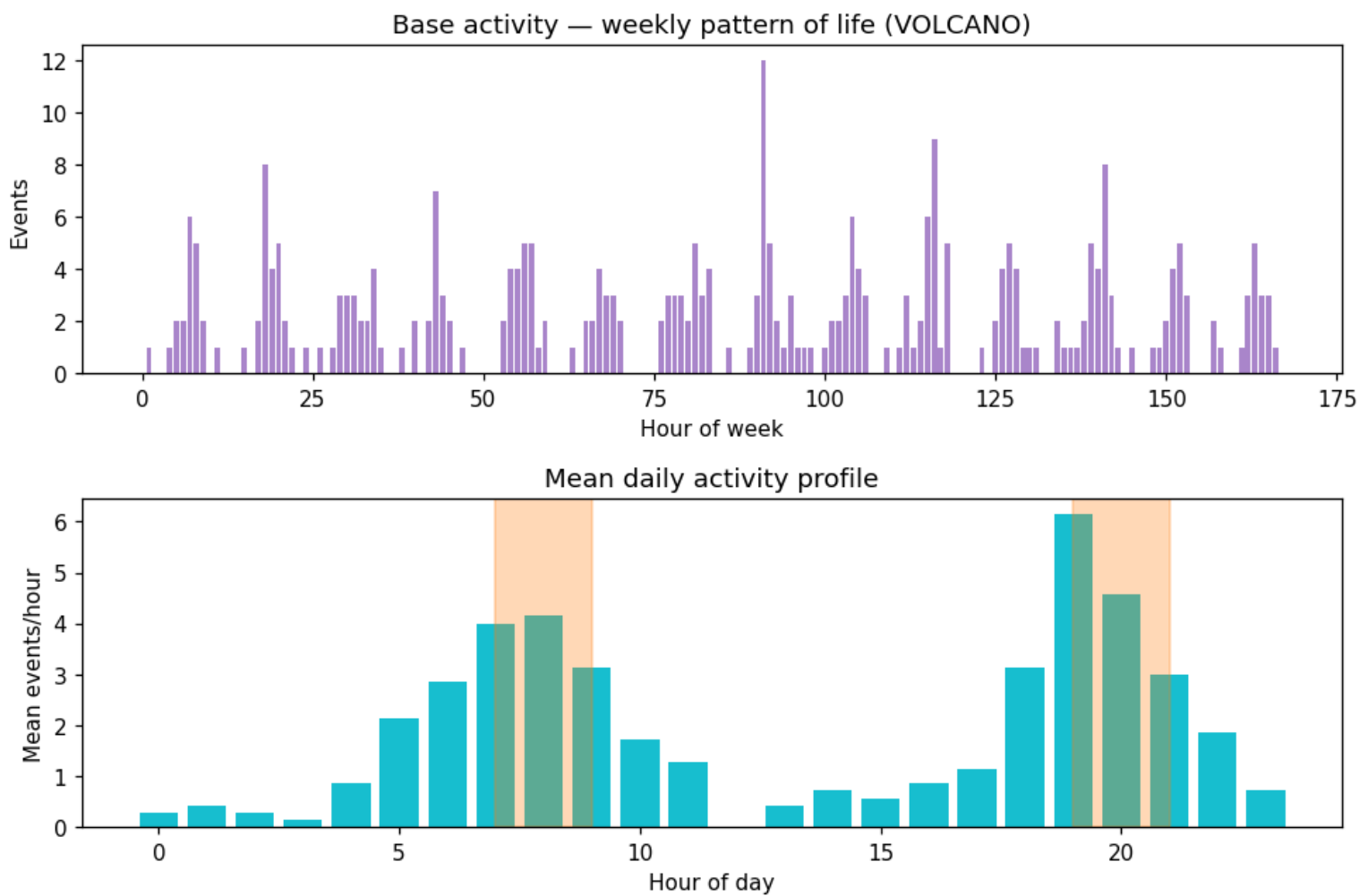


Figure 24: Pattern of life: weekly activity histogram (top) and mean daily profile with detected peak windows (bottom).

8.7 Experimental Setup — BIRD ONE: canonical scenarios, parameters, and configuration

The scenario is fixed in `src/you_only_live_twice/scenario.py`, which exports every input as a named constant and assembles them into one JSON descriptor. That descriptor is simultaneously (a) the object the provenance `input_hash` is taken over and (b) the source of the parameter tokens in the tables below, so a parameter reported here cannot drift from the parameter actually simulated. Paper identity is declared in `manuscript/config.yaml`; all derived values are computed by `src/you_only_live_twice/manuscript_variables.py`.

8.7.1 Mission identity

Parameter	Value
Random seed	1967
Provenance input hash (16 hex)	69984c9e423de2cf

8.7.2 Orbital geometry

Parameter	Chaser	Target
Altitude (km)	550.0	800.0
Inclination (°)	98.2	98.2
RAAN (°)	45.0	45.5
Initial phase (rad)	0.0	1.9

8.7.3 Search envelopes

Parameter	Value
Rendezvous horizon / step (s)	86400.0 / 60.0
Rendezvous max range (km)	500.0
Rendezvous min elevation (°)	10.0
Capture max range (km)	1000.0
Capture max relative speed (m/s)	2200.0
Capture step (s)	120.0

8.7.4 Scene, sensors and cost model

Parameter	Value
Volcano grid (cells per side)	64
Cone radius (cells) / height (m)	20.0 / 2000.0
Lair thermal excess (K)	25.0
Activity rate (events/h) over horizon (h)	4.0 over 168
Discovery cue mean μ / prior weight β	2.5 / 1.5
Discovery RF source power	8.0
Infiltration weights (slope / thermal / guard)	2.0 / 3.0 / 4.0
TDOA sensor ring radius (m) / count	25000.0 / 5
TDOA receiver timing jitter σ_t (ns, 1σ)	10.0
TDOA error-envelope max jitter (ns) / realizations per level	200.0 / 24
Injected lair footprint (cells)	80

8.7.5 Rendering pipeline

The manuscript is rendered entirely from the deterministic scenario:

1. `scripts/z_generate_manuscript_variables.py` writes `output/data/manuscript_variables.json` (11 protocol dataclasses, 5 provider methods).
2. `scripts/render_figures.py` writes the 8 figures under `../figures/`.
3. Prose substitutes every token reference with its computed value, so numbers stay in lock-step with the scenario.

8.7.6 Software environment

- Package version 0.1.0 (YOU ONLY LIVE TWICE, film 1967).
- Python 3.14.6 on darwin; numpy, matplotlib, pyyaml, and the frozen bond-api protocol.
- Generated 2026-08-04.

8.7.7 Test gate

The suite runs `pytest` with `--cov=src --cov-fail-under=90` (line **and** branch). Live test counts and the achieved coverage percentage are tracked in `docs/_generated/COUNTS.md`, not hardcoded here.

8.8 Reproducibility — BIRD ONE: verification gates, deterministic regeneration, and artifacts

The VOLCANO package is reproducible by construction.

8.8.1 Determinism

- Every generator is driven by a fixed seed (1967). No untracked random draws are permitted anywhere in `src/`.
- No wall-clock value enters a persisted artifact: the generation timestamp in this manuscript is a fixed date, and provenance carries an explicit seed with `wall_time_s` held at zero.
- The mission’s provenance `input_hash` (69984c9e423de2cf) is a stable sha256 fingerprint of the scenario descriptor, so an outcome can be audited without re-running the simulation.

8.8.2 Artifacts

All 8 figures are enumerated by `you_only_live_twice.figures.FIGURE_FILENAMES`, the single source both the render script and this table follow.

Artifact	Location	Regenerated by
Manuscript variables	<code>output/data/manuscript_variables.js</code>	<code>scripts/z_generate_manuscript_variables.py</code>
Rendezvous figure	<code>../figures/rendezvous_windows.png</code>	<code>scripts/render_figures.py</code>
Orbital-capture figure	<code>../figures/orbital_capture.png</code>	<code>scripts/render_figures.py</code>
Lair-detection figure	<code>../figures/concealed_lair_detection.png</code>	<code>scripts/render_figures.py</code>
Hidden-base posterior figure	<code>../figures/hidden_base_discovery.png</code>	<code>scripts/render_figures.py</code>
Infiltration-route figure	<code>../figures/infiltration_route.png</code>	<code>scripts/render_figures.py</code>
Pattern figure	<code>../figures/pattern_of_life.png</code>	<code>scripts/render_figures.py</code>
Emitter-localisation figure	<code>../figures/emitter_localization.png</code>	<code>scripts/render_figures.py</code>
Emitter error-envelope figure	<code>../figures/emitter_error_sweep.png</code>	<code>scripts/render_figures.py</code>

`output/` is git-ignored; a fresh run regenerates a byte-identical tree.

8.8.3 Test gate

The enforced quality gate is:

```
uv run pytest tests/ --cov=src --cov-fail-under=90
```

with `ruff` (check + format) and `mypy` green on `src/` and `scripts/`. Live counts live in `docs/_generated/COUNTS.md`. The suite uses real computation and no mock framework, so a green gate means the algorithms were actually exercised.

8.9 Scope and Related Work — BIRD ONE: boundaries, positioning, and relationship to the literature

8.9.1 Scope

The VOLCANO package covers seven special-agent analytical capabilities over a single fixed scenario. Its scope is deliberately bounded:

- **Orbital mechanics** assumes circular two-body orbits; eccentric, perturbed, or multi-body motion (J2, drag, third-body) is out of scope. Window finding is a deterministic grid search, not an optimizer.
- **Orbital capture** models the coplanar phasing problem; out-of-plane plane changes and finite-burn effects are not modelled.

- **Lair detection** scores a synthetic volcano-camouflage scene. It demonstrates the anomaly-scoring *method* on real synthetic data; it is not a claim about any real imagery. What is established is that the fused peak falls inside the injected lair *footprint*; the peak sits 5656.9 m from the injected centre cell, so centre-cell localization is **not** established.
- **Hidden-base discovery** uses a naive-Bayes fusion of Gaussian sensor cues; it does not model correlated or missing measurements. In this scene the cues are not weak — every single cue ranks the footprint first on its own (prior_only 1, rf 1, terrain 1, thermal 1) — so the results demonstrate consistent multi-cue evidence, not weak-signal fusion. Detection performance across varied camouflage, noise levels and lair geometries would need a swept ensemble of scenes; one scenario cannot supply it.
- **Infiltration planning** is a static 4-connected A* over a fixed cost surface; it does not model dynamic guards or replanning.
- **Emitter localization** solves TDOA at one emitter–sensor geometry with a Gaussian timing-jitter model. Error versus jitter is now characterisable: a deterministic sweep of 6 jitter levels (each averaged over 24 realizations) reports the mean-error envelope, rising from zero to 41.3 m at 200.0 ns. Error versus *geometry* — multipath, timing bias, and a swept sensor-ring geometry — remains out of scope; the reported 3.7912 m is the operating point inside that envelope, not a statement across geometries.
- **Pattern of life** uses a diurnal-rate Poisson model of activity. It recovers the model’s injected rhythm; it does not claim general time-series universality.

Nothing here is fielded operational software — it is a rigorous, reproducible exemplar of the underlying mathematics, in keeping with the BOND suite’s research-exemplar contract.

8.9.2 Related work

The methodology draws on established, citable foundations rather than novel claims:

- Orbital-element → ECI conversion, circular propagation, the Hohmann transfer, and the phasing problem follow standard treatments of astrodynamics [Vallado, 2013a, Bate et al., 1971a].
- Rendezvous / proximity geometry and operational feasibility windows are standard in human and robotic spaceflight mission design [Wertz et al., 2011, Fehse, 2003].
- Anomaly scoring by standardized residual against a fitted background is the classical statistical formulation of anomaly detection, of which the RX detector is the canonical imaging instance [Reed and Yu, 1990, Chandola et al., 2009].
- Bayesian cue fusion for a hidden-site hypothesis set follows Pearl’s belief updating framework [Pearl, 1988]; ordering the resulting posterior into a search plan is classical search theory [Stone, 1975], and combining heterogeneous sensors follows the multisensor-fusion literature [Waltz and Llinas, 1990].
- Passive RF emitter location by TDOA follows the Taylor-series estimator [Foy, 1976a] and the hyperbolic-location literature [Chan and Ho, 1994a, Skolnik, 2001a].
- Infiltration routing is textbook best-first search / A* [Hart et al., 1968a, Cormen et al., 2009b].
- Modelling base activity as an inhomogeneous Poisson point process, and reading tempo off its rate envelope, follows standard point-process theory [Daley and Vere-Jones, 2003].

The self-consistency guarantee — every manuscript number computed from a fixed seed and token-injected — follows the template’s reproducible-research convention documented in the BOND suite.

8.9.3 Limitations and future work

Integration points for later BOND capabilities are recorded in `TODO.md`: a perturbation-aware propagator, a spline-refined window edge detector, and binding to the shared `bond-utilities` provenance helper once that dependency ships.

8.10 Sources — BIRD ONE: bibliography

Vallado [2013a]; Bate et al. [1971a]; Wertz et al. [2011]; Fehse [2003]; Reed and Yu [1990]; Chandola et al. [2009]; Pearl [1988]; Stone [1975]; Waltz and Llinas [1990]; Hart et al. [1968a]; Cormen et al. [2009b]; Foy [1976a]; Chan and Ho [1994a]; Skolnik [2001a]; Daley and Vere-Jones [2003]

9 On Her Majesty’s Secret Service (1969) — BEDLAM

film package · *package codename* *BEDLAM*. Mission **PIZ GLORIA**: model aerosol bioweapon dispersion in the valley; resolve the plume’s time-dependent puff footprint; plan a safe alpine exfiltration route; simulate the bobsleigh descent and banked-curve clearance; score the Bray cover identity; verify the Bray timeline against observed sightings.

9.1 Concepts — BEDLAM: domain and operational focus

bioweapon dispersion, alpine ops, cover identity

9.2 Abstract — BEDLAM: mission summary

On Her Majesty’s Secret Service: PIZ GLORIA — Special-Agent Mission Software. This report documents the deterministic mission software built for mission codename **PIZ GLORIA**, drawn from the film *On Her Majesty’s Secret Service* (1969). The package delivers six quantitative analysis capabilities grouped under three operational pillars: **Aerosol dispersion**. The steady-state Gaussian plume fixes the average threat: a hazardous airborne-allergen isopleth reaching **10210.16 m** down-valley, with a peak ground-level concentration of **3.492e-06 kg m⁻³** at **703.49 m**. A Lagrangian Gaussian puff train adds the *time* dimension, showing the release first arrives at the downstream receptor after **740 s** with a transient peak of **1.370e-06 kg m⁻³** — an early-warning lead the steady plume cannot express. **Alpine operations**. A slope-safety-gated planner returns a **safe** exfiltration line of **5478.808 m** at a mean grade of **15.24 deg**. Point-mass bobsleigh dynamics then integrate the descent: **37.5 s** to **45.8 m/s**, while a banked-curve analysis shows the unbanked canonical hairpin pulls **7.56 g** (over-limit) and a 55-degree bank raises safe cornering to **47.1 m/s**. **Cover identity**. The weighted *Bray* persona scores **0.177 (low risk)**. Timeline verification against four observed sightings gives a **medium** verdict at **77.8%** coverage with **1** contradiction(s) — one London sighting that unseats the claim. Every number above flows through the manuscript-variable pipeline from the executed mission results; none is hand-authored. The implementation is deterministic, mock-free, and covered well above the 90% branch-coverage gate on `src/`.

9.3 Introduction — BEDLAM: mission framing, the operational problem, and how to read this chapter

On Her Majesty’s Secret Service (1969) is the one 007 film in which Bond’s mission is personal: tracking Ernst Stavro Blofeld to the Piz Gloria research station in the Swiss Alps. Under the cover of Sir Hilary Bray — a genealogist of the College of Arms — Bond infiltrates the summit and, when the cover is blown, escapes by bobsleigh down the mountain. The PIZ GLORIA mission software models the operational mathematics behind that operation: the threat of an aerosolized allergen bioweapon on a mountaintop, the geometry of an alpine descent line, and the fragility of a fabricated identity.

This package therefore implements six real, deterministic scientific models as its pure domain core, grouped under the three operational pillars the film suggests:

- `dispersion_model.py` — the classical Gaussian plume solution for a continuous elevated point source, with Pasquill-Gifford stability-dependent dispersion and a valley-channeling correction to the transport wind.
- `puff_dispersion.py` — a Lagrangian Gaussian puff train that adds the time dimension to the same release: an early-warning arrival time and transient peak concentration the steady plume cannot express.
- `alpine_routes.py` — track-geometry primitives (grade, chord length, Menger curvature) and a slope-safety-gated Dijkstra planner for an extrapolated safe descent.
- `descent_dynamics.py` — point-mass bobsleigh kinematics down that route, paired with banked-curve physics (lateral g-load, safe cornering speed) so the exfiltration line is audited for both slope and turn safety.
- `cover_identity.py` — a weighted additive risk model that scores how “held” an alias like the Bray persona is under scrutiny.
- `identity_timeline.py` — interval-arithmetic verification of that persona’s claimed alibi against independently observed sightings, so a single contradicting observation can unseat an otherwise-covered cover.

The film’s software also implements the frozen `bond_api` mission protocol, exposing a deterministic `MissionProvider` that a coordinator can discover and drive through the standard `brief -> recon -> plan -> execute -> debrief` cycle.

9.3.1 Reader’s guide

- The methodology section derives all six models: dispersion, puff, route, descent, cover, and timeline.
- The results section presents the executed mission metrics and figures.
- The experimental setup section lists the fixed scenario parameters.
- The reproducibility section records provenance, hashing, and test coverage.
- The scope section states the models’ limits and cites the related work.

9.4 Methodology — BEDLAM: the analytical models and algorithms that drive the mission

9.4.1 Aerosol bioweapon dispersion

The dispersion core models a continuous, elevated release of an aerosolized allergen bioweapon transported downwind by the mean wind. We use the steady-state Gaussian plume solution for a point source with ground reflection:

$$C(x, y, z) = \frac{Q}{2\pi u \sigma_y \sigma_z} \exp\left(-\frac{y^2}{2\sigma_y^2}\right) \left[\exp\left(-\frac{(z-H)^2}{2\sigma_z^2}\right) + \exp\left(-\frac{(z+H)^2}{2\sigma_z^2}\right) \right]$$

where Q is the release rate [kg/s], u the effective transport wind [m/s], H the effective release height [m], and σ_y, σ_z the horizontal and vertical dispersion coefficients. The coefficients follow the open-country Pasquill-Gifford power laws $\sigma_y = ax^b$ and $\sigma_z = cx^d$, with the standard per-stability-class coefficient set (classes A–F, unstable to stable) [Arya, 1999, Turner, 1970].

Because the release sits in a valley, the transport wind is corrected by a channeling factor

$$f(e) = 1 - (1 - f_0) \exp(-e/d_v),$$

which slows the plume most at the valley floor (f_0 is the floor wind ratio) and returns toward free-stream flow with increasing receptor elevation e relative to the valley depth d_v . The far edge of the ground-level isopleth at a fixed concentration threshold is computed by a coarse scan plus bisection refinement, giving the mission’s reach metric.

9.4.2 Alpine route planning

A planned track is a polyline of waypoints in projected coordinates (easting, northing, elevation). Three geometry quantities govern a bobsleigh-grade line:

- chord slope: $\theta = \arctan(|\Delta z|/\sqrt{\Delta e^2 + \Delta n^2})$,
- 3-D chord length for cumulative distance,
- discrete Menger curvature $k = 4A/(|ab||bc||ca|)$ for turn sharpness.

A segment is classified *safe*, *marginal*, or *unsafe* by grade and rejected when it also violates a curvature cap; the planner runs Dijkstra [Cormen et al., 2009c] on an 8-connected grid, discarding any step whose chord grade exceeds the maximum safe slope, and costs the remainder by distance plus a grade penalty, in the spirit of alpine slope/avalanche danger rating [McClung and Schaerer, 2006].

9.4.3 Cover-identity risk scoring

Each identity attribute a carries a weight w_a and a risk $r_a \in [0, 1]$. The aggregate cover risk is the weighted mean $\sum w_a r_a / \sum w_a$, and an attribute risk is a linear function $r_a = \beta_a + m_a c_a$ of normalized incriminating evidence c_a . A breach scenario raises one or more attributes’ evidence and recomputes the aggregate and its discrete verdict.

9.4.4 Time-dependent puff dispersion

To capture *when* a threat arrives, the continuous release is discretized into a deterministic train of Gaussian puffs, the Lagrangian puff-train approach used by operational puff models [Zannetti, 1990, Sykes et al., 1998]. Puff i , emitted at t_i with mass $Q \Delta t$, advects downwind at the effective wind and spreads as it travels. Superimposing every emitted puff at a receptor gives a concentration time series, whose early crossing of a detection threshold is the **arrival time** — the operational early-warning metric. For a steady release the puff superposition converges to the plume solution far downwind, a cross-check the tests assert directly.

9.4.5 Bobsleigh descent dynamics

A point-mass slides under gravity down a track profile resisted by kinetic friction and aerodynamic drag:

$$a(v) = g \sin \theta - \mu g \cos \theta - kv^2, \quad k = \frac{1}{2} \rho C_d A / m,$$

integrated with a fixed time step along the polyline. Because a bobsleigh line is defined by its turns, the module pairs the speed profile with curve physics: lateral g-load $v^2/(gR)$ and the banked-curve speed limit $v_{\max} = \sqrt{gR (\tan \beta + \mu_L)/(1 - \mu_L \tan \beta)}$.

9.4.6 Cover-timeline verification

A cover is as strong as its timeline. Claimed events and observed sightings are intervals $[s, e)$ over mission minutes; an observation is *matched* when an overlapping claim agrees on location and *conflicts* when an overlapping claim names a different location. The coverage fraction (matched observed minutes over total observed minutes) yields a discrete verdict, and a single contradicting sighting can unseat an otherwise-covered story.

9.5 Results — BEDLAM: measured outcomes, headline numbers, and what they establish

The executed PIZ GLORIA mission produces six deterministic result groups. Every value below is generated by the live engine through the manuscript variable pipeline.

Metric	Engine output
Peak ground-level centreline concentration	3.492e-06 kg m ⁻³
Peak downwind distance	703.49 m
High-hazard isopleth reach	10210.16 m
Puff arrival at downstream receptor	740 s
Puff transient peak concentration	1.370e-06 kg m ⁻³
Planned route distance	5478.808 m
Route mean grade	15.24 deg
Route slope-safety verdict	safe
Descent time	37.5 s
Descent maximum speed	45.8 m/s
Track peak lateral load	7.56 g
Track 4-g ceiling verdict	over-limit
Banked (55°) cornering max speed	47.1 m/s
Bray cover score	0.177
Bray cover verdict	low
Timeline coverage	77.8%
Timeline contradictions	1
Timeline verdict	medium

9.5.1 Dispersion footprint

At the configured release (see the experimental setup section) the model locates a peak ground-level centreline concentration of **3.492e-06 kg m⁻³** at **703.49 m** downwind. The high-hazard isopleth (ground-level concentration at or above the configured threshold) reaches **10210.16 m** down-valley — the far edge of the exposure band, driven by the elevated release and valley-channelled transport.

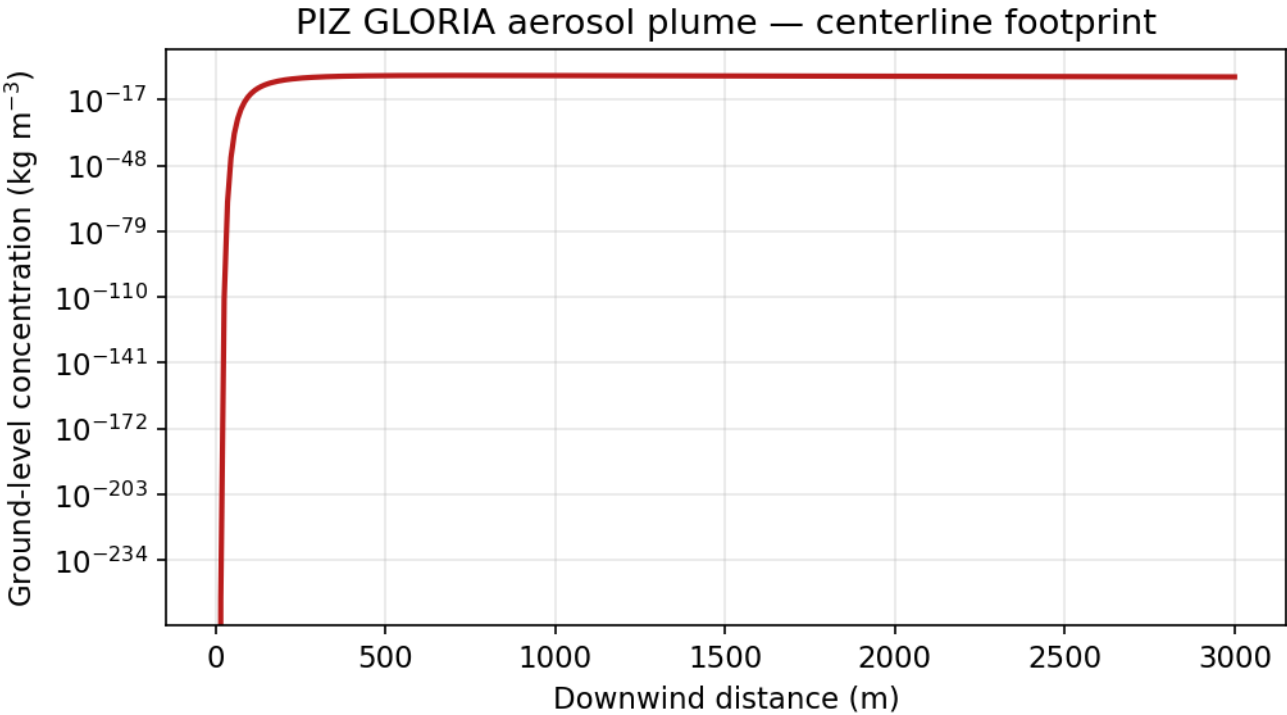


Figure 25: Ground-level centreline aerosol concentration along the plume axis.

9.5.2 Puff arrival and early warning

At the downstream receptor the Lagrangian puff train shows the allergen first crosses the detection threshold after **740 s**, with a transient peak of **1.370e-06 kg m⁻³**. This is the operational early-warning lead: the steady plume describes the average threat, but the puff dynamics give the time at which a sensor would first alarm.

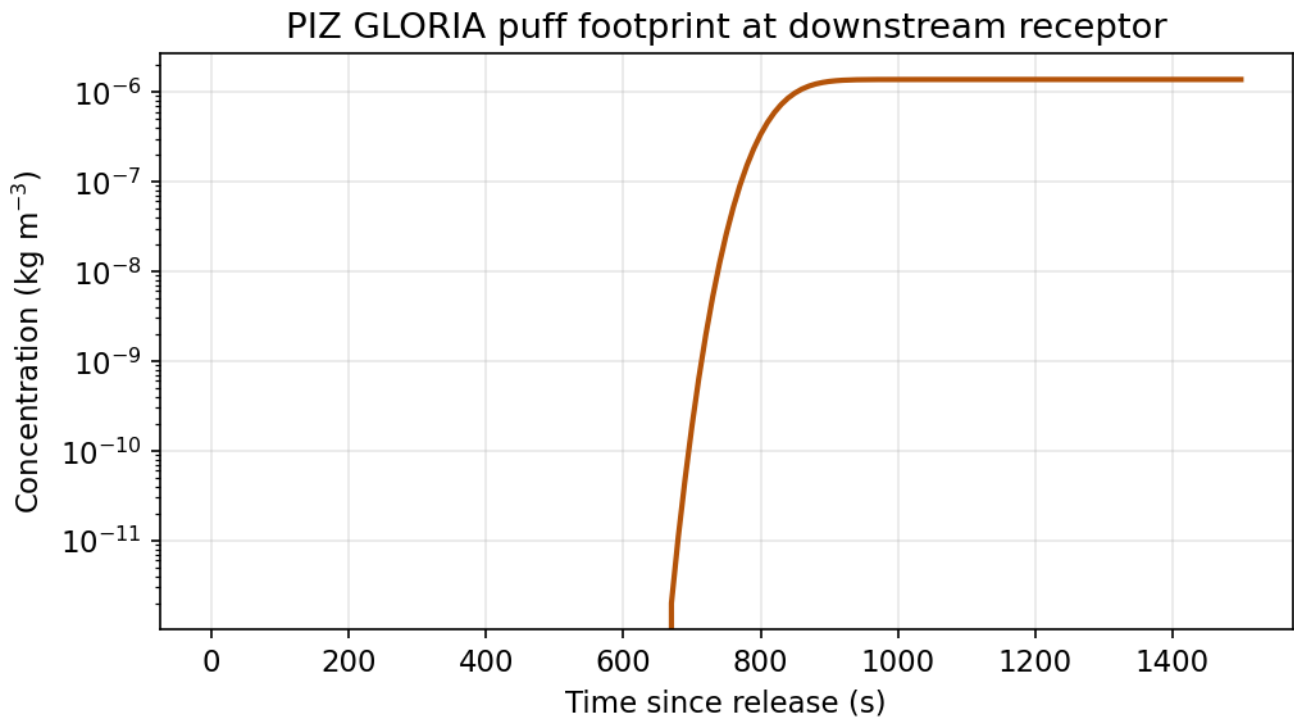


Figure 26: Puff concentration time series at the downstream receptor.

9.5.3 Alpine exfiltration route

The slope-safety-gated planner returns a **safe** route of **5478.808 m** with a mean grade of **15.24 deg**. Every step of the reconstructed line respects the declared maximum slope gate (the route figure).

9.5.4 Bobsleigh descent and banked-curve clearance

Point-mass kinematics over the canonical bobsleigh track complete the descent in **37.5 s**, peaking at **45.8 m/s** (the descent figure). The unbanked canonical hairpin carries **7.56 g** of lateral load — **over-limit** the 4-g ceiling — so the line must be banked; a 55-degree bank raises the safe cornering speed to **47.1 m/s**.

9.5.5 Cover identity

The canonical *Bray* persona scores **0.177** (**low** risk); the dominant exposure driver is forensic overlap.

9.5.6 Timeline verification

Auditing the persona’s claimed timeline against four sightings yields a **medium** verdict at **77.8%** coverage with **1** contradiction(s): a London sighting that contradicts the Piz Gloria claim. One contradicting observation is enough to unseat an otherwise-covered story — the cover holds only while the timeline is unbroken.

9.6 Conclusion — BEDLAM: findings, verdict, and what the mission establishes

The PIZ GLORIA mission software demonstrates a complete, deterministic, mock-free quantitative pipeline for the kind of operation depicted in *On Her Majesty’s Secret Service*: it models the airborne threat (steady plume *and* time-resolved puff arrival), the escape line (slope-safe routing *and* bobsleigh descent kinetics with banked-curve clearance), and the identity risk (weighted *Bray* persona *and* timeline verification) — all through real scientific algorithms with real computed data, exposed through the frozen `bond_api` protocol so any coordinator in the BOND suite can drive it.

Each of the six models is an independent, infrastructure-free pure module, and the mission adapter in `src/on_her_majestys_secret_service/mission.py` binds them to the protocol in a thin, auditable layer while leaving the public protocol shape (film slug, five provider methods, gadget-hook convention) unchanged. Provenance is recorded for every executed outcome (version, seed, input hash, wall time). The suite is reproducible by construction: fixed scenario parameters, fixed seeds, and no random draws yield byte-identical results across runs, and the manuscript metrics are generated, not authored.

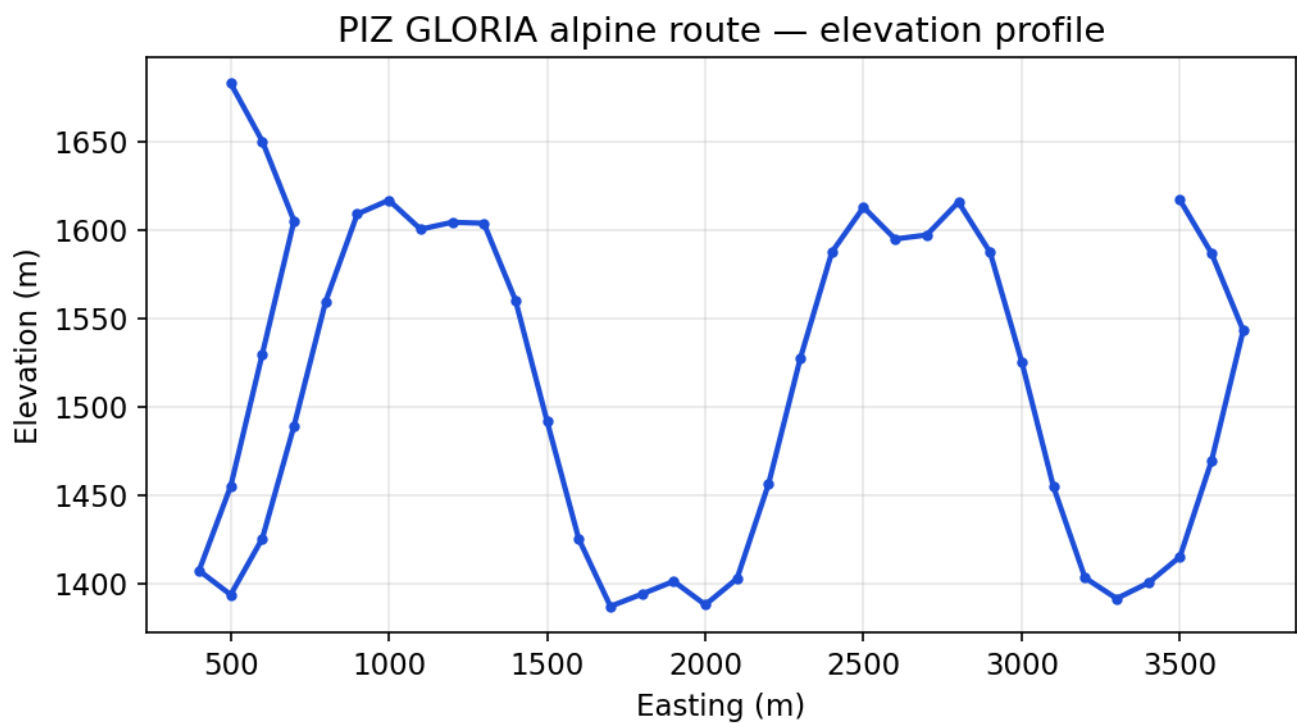


Figure 27: Planned route elevation profile along the reconstruction.

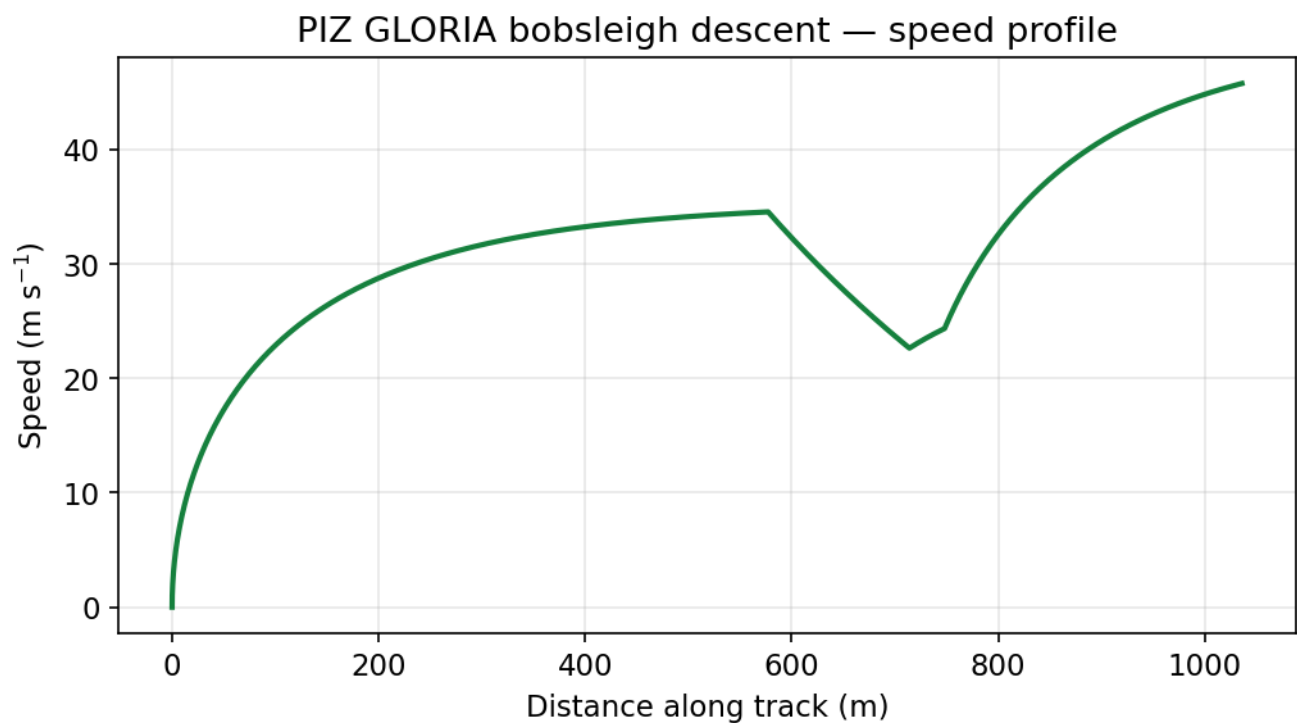


Figure 28: Bobsleigh descent speed profile along the canonical track.

9.7 Experimental Setup — BEDLAM: canonical scenarios, parameters, and configuration

9.7.1 Scenario parameters

The PIZ GLORIA release and routing scenario is fixed in `src/on_her_majestys_secret_service/mission.py` and mirrored in `manuscript/config.yaml`:

- Release rate: 0.05 kg/s aerosolised allergen.
- Free-stream wind speed: 4.0 m/s (inferred stability class C, slightly unstable).
- Effective release height: 25.0 m.
- Valley channeling depth: 400.0 m, floor wind ratio 0.6.
- High-hazard isopleth threshold: $1.0\text{e-}7$ kg/m³.
- Route grid: 40 x 40 cells over a 10 km x 10 km analytic range, maximum safe slope 30 deg.
- Puff early-warning receptor: 2000 m downwind, threshold $1.0\text{e-}8$ kg/m³, emission cadence 10 s, initial puff size 1 m.
- Bobsleigh descent: rider+sled mass 100 kg, friction 0.03, drag area 0.45 m², air density 1.2 kg/m³, integration step 0.05 s; canonical track with a tight hairpin audited against a 4-g lateral ceiling and a 55-degree bank for cornering clearance.
- Cover timeline: mission window [0, 1440) minutes; 3 claimed events vs 4 observed sightings.

9.7.2 Software environment

- Package: `on_her_majestys_secret_service` 0.1.0
- Python: 3.14.6
- NumPy: 2.4.2
- Protocol: `bond_api` (frozen MissionProvider contract)
- Manuscript variables generated: 2026-08-04T00:00:00+00:00 (UTC)

9.7.3 Guarantees

Every result value is produced by the executed mission and injected through the manuscript-variable pipeline; none is hand-written into prose.

9.8 Reproducibility — BEDLAM: verification gates, deterministic regeneration, and artifacts

9.8.1 Determinism contract

The PIZ GLORIA mission is reproducible by construction:

- Fixed scenario parameters and fixed seed (7); randomness is disabled.
- No wall-clock time enters any persisted artifact (provenance `wall_time_s` is recorded as 0.0).
- No mock framework anywhere in the source or tests — all tests are real data computation.
- `output/` is regenerable and git-ignored: re-running `scripts/04_execute.py` and `scripts/z_generate_manuscript_variables.py` reproduces byte-identical artifacts (a live determinism proof).

9.8.2 Provenance

Every executed outcome carries a **Provenance** record with package version, seed, a SHA-256 `input_hash` over the sorted scenario inputs, and wall time. The outcome is also serialized with the `bond_api` tagged-JSON format so it can be audited without re-running the film.

9.8.3 Test coverage

The suite enforces the $\geq 90\%$ line+branch coverage gate on `src/` via `uv run pytest tests/ --cov=src --cov-fail-under=90`. Live counts and coverage are tracked in `docs/_generated/COUNTS.md`.

9.9 Scope and Related Work — BEDLAM: boundaries, positioning, and relationship to the literature

9.9.1 Scope

The PIZ GLORIA software is illustrative mission engineering, not a certified hazard-assessment tool, and its claims are bounded accordingly:

- **Dispersion** — the steady-state Gaussian plume and its puff-train variant are screening models: they assume a continuous release, uniform wind, and flat-terrain reflection, and do not model time-varying release composition, terrain-following plumes, chemical transformation, or deposition.
- **Descent dynamics** — the point-mass integration is a planar 1-D model: no steering, no lateral load transfer into the friction model, and a single scalar drag coefficient. The canonical track is analytic, not surveyed.

- **Routing** — the planner operates on an analytic test surface, not surveyed terrain, and its 8-connected grid over-approximates feasible continuous paths.
- **Cover identity / timeline** — the weighted risk score and interval coverage depend on analyst-chosen weights, evidence normalization, and the observation set; they are decision aids, not calibrated probabilities.

9.9.2 Limitations and uncertainty

Beyond the scope boundaries above, the headline numbers carry specific, unquantified uncertainties that a user should read alongside them:

- **Dispersion coefficients** are the fixed open-country Pasquill-Gifford tables, which are screening values, not site-calibrated measurements. The isopleth reach and peak location therefore inherit $\pm$ a factor of several in absolute hazard extent under real alpine meteorology.
- **Valley-channeling** enters as a single scalar floor factor; real topography funnels wind non-uniformly, so ground-level concentrations are order-of-magnitude estimates.
- **Puff timing** depends on the discrete emission cadence Δt and initial puff size: the arrival time is accurate to within roughly one cadence (10 s) and the transient peak is resolution-dependent.
- **Descent dynamics** are a single-degree-of-freedom point mass with no steering or lateral load transfer; the reported speeds and g-loads bound the physics but are not a certified vehicle- or rider-specific prediction.
- **Route geometry** is analytic, and the reported distance/grade are properties of that surface, not measured survey data.
- **Cover metrics** depend on analyst-chosen weights, evidence normalization, and the observation set; they are decision aids with no calibrated probability semantics.

These are stated so no headline number is read as a measurement of the world: each is a deterministic property of the stated model and its fixed inputs.

9.9.3 Related work

Gaussian plume/puff dispersion follows the classical Pasquill-Gifford formulation (Arya 1999; Turner 1970) and the Lagrangian puff-train approach used by operational puff models (Zannetti 1990; Sykes et al. 1998). Point-mass descent with friction and drag draws on bobsleigh performance framing [Morlock and Zatsiorsky, 1989] (whose scope is strictly the regression in `99_references.md`; it does not underwrite the rigid-body equations), and alpine slope/avalanche safety rating follows the standard danger-scale practice (McClung & Schaerer 2006). Interval-based timeline verification uses standard interval arithmetic / coverage analysis. Where a precise field value matters, the open-country dispersion coefficients are the fixed, cited data tables in `dispersion_model.py`.

9.10 Sources — BEDLAM: bibliography

Arya [1999]; Turner [1970]; Zannetti [1990]; Sykes et al. [1998]; Morlock and Zatsiorsky [1989]; McClung and Schaerer [2006]; Corman et al. [2009c]

10 Diamonds Are Forever (1971) — DIAMOND NET

film package · package codename DIAMOND NET. Mission **CONFLICT STONE**: trace the conflict-diamond custody chain and pinpoint provenance break points; optimize the smuggling corridor route under seizure risk; audit the diamond lot for weight and density fraud; map Las Vegas casino money movement and measure laundering closure; test the casino ledger against Benford’s law; solve orbital-mirror targeting geometry and beam delivery.

10.1 Concepts — DIAMOND NET: domain and operational focus

supply-chain provenance, orbital weapons

10.2 Abstract — DIAMOND NET: mission summary

Diamonds Are Forever: CONFLICT STONE — Special-Agent Mission Software — codename **CONFLICT STONE** — is a deterministic special-agent mission analysis built on *Diamonds Are Forever* (1971). It fuses seven analytic lines into a single audit: the provenance chain of smuggled conflict diamonds, the risk-weighted corridor route that carries them, the physical forensics of the lot itself, the money-movement and Benford’s-law tests of the Las Vegas casino operation, and both the targeting geometry and the beam-delivery physics of the orbital mirror the proceeds fund. **Provenance.** The custody chain comprises 5 handoffs with 2 undocumented break points — a documented coverage of 60%, the forensic signature of conflict goods. **Smuggling.** Across a 10-corridor network, the expected-cost optimum (sierra_leone -> nairobi -> paris -> amsterdam -> las_vegas) cuts expected cost by 16% versus the fewest-hops baseline, at a survival probability of 34%. **Forensics.** Weight and density testing of lot DF-71 surface 2 anomalies and a 4-point audit score — a **flagrant** verdict that exposes both the paste substitution and the final theft. **Casino.** A 7-casino money-movement graph whose laundering closure reaches 77%, and a 141-transaction ledger whose chi-square statistic (35.80) exceeds the 15.507 critical value and so marks it **anomalous** under Benford’s law — while its 80-transaction legitimate baseline, audited alone, scores 0.53 and **conforms**, the negative control that makes the anomalous verdict attributable to the round-figure deposits. **Orbital.** Of 3 threatened cities, only 2 fall on the mirror’s visible face; and honest beam physics shows the fantasy is bounded — delivering 1 MW/m² at geostationary range takes a 42.7-m mirror. All results are reproducible: every metric is computed at runtime from fixed canonical datasets, no random draws, no mock objects, and every mission outcome carries a cryptographic input fingerprint.

10.3 Introduction — DIAMOND NET: mission framing, the operational problem, and how to read this chapter

Diamonds Are Forever: CONFLICT STONE — Special-Agent Mission Software (Deterministic Analysis of Smuggled-Goods Provenance and Routing, Lot Forensics, Casino Money Movement and Its Benford Signature, and Orbital-Mirror Targeting and Beam Delivery) is the CONFLICT STONE mission package of PROJECT BOND, a suite of deterministic special-agent mission analyses. Each film package implements the frozen BOND-API mission protocol, which fixes the contract every mission must satisfy: a **brief**, a **recon**, a **plan**, an **execute**, and a **debrief**, each carrying typed provenance so an outcome can be audited without re-running the mission.

This package’s identity is `diamonds_are_forever`, version 0.1.0, under codename CONFLICT STONE. It adapts seven pure analytic cores to the protocol, tracing the film’s operation from the mine to the satellite:

1. **Provenance chain** — a conflict-diamond custody graph with documented handoffs and identified provenance break points.
2. **Smuggling routes** — a directed corridor network in which each leg carries a transport cost and a seizure probability; an expected-cost shortest path is compared against a fewest-hops baseline in the same graph.
3. **Lot forensics** — weight and density reconciliation of the physical lot across its handoffs, separating legitimate cutting loss from theft and from paste substitution.
4. **Casino network** — directed money movement across the Las Vegas strip, with cycle detection, laundering-closure measurement, and hub discovery.
5. **Benford audit** — a leading-digit test of the casino ledger against Benford’s law, scored by a Pearson chi-square statistic.
6. **Orbital mirror** — geostationary space-mirror targeting geometry: occultation, slant range, and elevation above the target’s horizon.
7. **Beam physics** — what the mirror actually delivers: the diffraction-limited spot, atmospheric transmittance, the power budget, and the minimum mirror diameter a strike-class beam would require.

The first five cores prosecute the terrestrial half of the operation (who touched the stones, how they moved, whether they are genuine, and where the money went); the last two bound the orbital half in physics rather than in plot. The mission software is a thin adapter over these cores: the domains are pure and infrastructure-free, while the adapter handles only the protocol wiring. This keeps every analytic claim testable in isolation and reproducible by construction.

10.4 Methodology — DIAMOND NET: the analytical models and algorithms that drive the mission

The CONFLICT STONE mission combines seven deterministic methods, each a pure function of a fixed canonical dataset.

10.4.1 Provenance chain

A custody chain is an ordered sequence of handoffs, each carrying a source, a target, and a **recorded** flag. A valid chain is **continuous** (each handoff's target is the next handoff's source) and **acyclic** (a gem never returns to a previous holder). A handoff flagged **recorded=False** is a **break point** — evidence that provenance cannot be verified at that step. The analytic surface reports the break-point indices, the documented-coverage ratio, and whether the chain is conflict-free.

10.4.2 Smuggling corridor routing

The chain answers *who touched the goods*; corridor routing answers *which route they travel*. A smuggler moves a fixed-value cargo through a directed network of corridors, each carrying a transport cost and a seizure probability. The expected cost of a corridor is $\text{cost} + p_{\text{seizure}} * \text{cargo_value}$. A deterministic Dijkstra over expected cost returns the **least-risk route**, its expected cost, and its survival ($\text{product}(1 - p)$). A fewest-hops BFS baseline in the *same* graph bounds the **reduction** metric, which is therefore always non-negative. The optimum is *cargo-dependent*: cheap corridors win at low cargo value, low-seizure corridors at high cargo value, so the reduction is reported as measured across a cargo sweep rather than as a single monotone trend.

10.4.3 Lot forensics

Physical laundering of a lot is audited handoff-by-handoff against three forensic rules: a **weight gain** (impossible for a genuine lot) flags accounting fraud or substitution; a **loss beyond the legitimate cutting allowance** of 25% per handoff flags theft; and a **measured density outside the diamond window** (3.40-3.65 g/cm³ around nominal 3.52) flags a paste or simulant swap. Each violation contributes to an audit score and a lot verdict (**clean** / **suspicious** / **flagrant**); a single anomaly reads as *suspicious*, two or more as *flagrant*.

10.4.4 Casino money movement

The Las Vegas network is a directed, weighted graph whose edges aggregate inter-casino transfers. The analysis computes gross flow, net positions, the network **hub** (greatest gross flow — a degree-style centrality in the sense of [Freeman, 1978]), the directed **money cycles** (laundering loops, enumerated *completely* — each elementary circuit exactly once — by Johnson's algorithm [Johnson, 1975], the classical circuit-enumeration problem of [Tarjan, 1973]), and the **laundering closure** — the fraction of flow that sits on a cycle. Completeness matters here: a back-edge DFS would quietly drop a circuit that re-enters an already-traversed node, under-reporting the closure, which is exactly the class of omission a rights-the-books laundering screen must not commit.

10.4.5 Benford's-law ledger audit

A laundering operation breaks the leading-digit distribution of its ledger by clustering deposits on round figures. This module compares the ledger's observed leading digits against Benford's law, $p(d) = \log_{10}(1 + 1/d)$ [Benford, 1938, Newcomb, 1881], with a Pearson chi-square statistic [Pearson, 1900] against the df=8, alpha=0.05 critical value of 15.507. Above the critical value the ledger is flagged **anomalous** — the laundering signal that forensic digit analysis is built to surface [Nigrini, 2012].

The audit is exercised as a matched control pair. A screen that fires on every ledger detects nothing, so the canonical ledger is built from two halves that are also audited separately:

- a **legitimate baseline** of 80 transactions — the *negative control*. Its leading digits are the leading digits of the Fibonacci sequence, which are Benford-distributed as a theorem: Binet's formula makes $\log_{10} F_n$ an irrational rotation, equidistributed mod 1 by Weyl's criterion. Conformance is therefore a property of the sequence, not of digit frequencies chosen to pass the test. The amounts are placed across the 10^{2-10^4} decades, and scaling by a power of ten cannot change a leading digit;
- 61 **round-figure deposits** on $1/2/5 \times 10^n$ — the laundered cash.

Both halves are constructed fixtures, not empirical records. The pair establishes that this implementation separates them; it is not evidence about how the screen would behave on real books.

10.4.6 Orbital targeting geometry

A geostationary mirror is modelled as a point above a sub-satellite point. Targeting solves occultation (the beam ray's nearest approach to Earth's centre), slant range, and elevation above the target's horizon.

10.4.7 Beam delivery physics

The targeting geometry solves *where* the beam can go; beam physics solves *what it delivers*. A diffraction-limited Airy spot $r = 1.22 * \lambda * R / D$ [Airy, 1835, Born and Wolf, 1999] sets the focused size, Beer-Lambert $T = \exp(-k * L)$ [Beer, 1852] sets the atmospheric transmittance, the power budget $P = S * (\pi D^2 / 4) * T$ sets the captured flux, and the dwell time $t = F / I$ deposits a material's vaporization fluence. Chaining these gives $I = S * T * D^4 / (4 * 1.22^2 * \lambda^2 * R^2)$; inverting it for D yields the **minimum mirror diameter** that delivers a required irradiance at a given range — the honest bound on the weapon's fantasy, and the reason the result scales only as the fourth root of the demanded irradiance. The model assumes the full captured power lands inside the first-null disk; the real Airy pattern puts $\approx 83.8\%$ inside it (see *Scope and Related Work*), pushing the same direction as the other omissions, so the minimum diameter remains a lower bound.

10.5 Results — DIAMOND NET: measured outcomes, headline numbers, and what they establish

All results are computed at runtime from the frozen canonical datasets; the prose below cites resolving variables rather than hand-authored numbers.

10.5.1 Provenance chain

The canonical route tracks a diamond lot through 5 handoffs from mine to fence. 2 of those handoffs are undocumented — the smuggling hops that make the lot conflict goods — giving a documented coverage of 60%. The chain is therefore conflict-free = **false**. The provenance figure marks each handoff as documented or a break point.

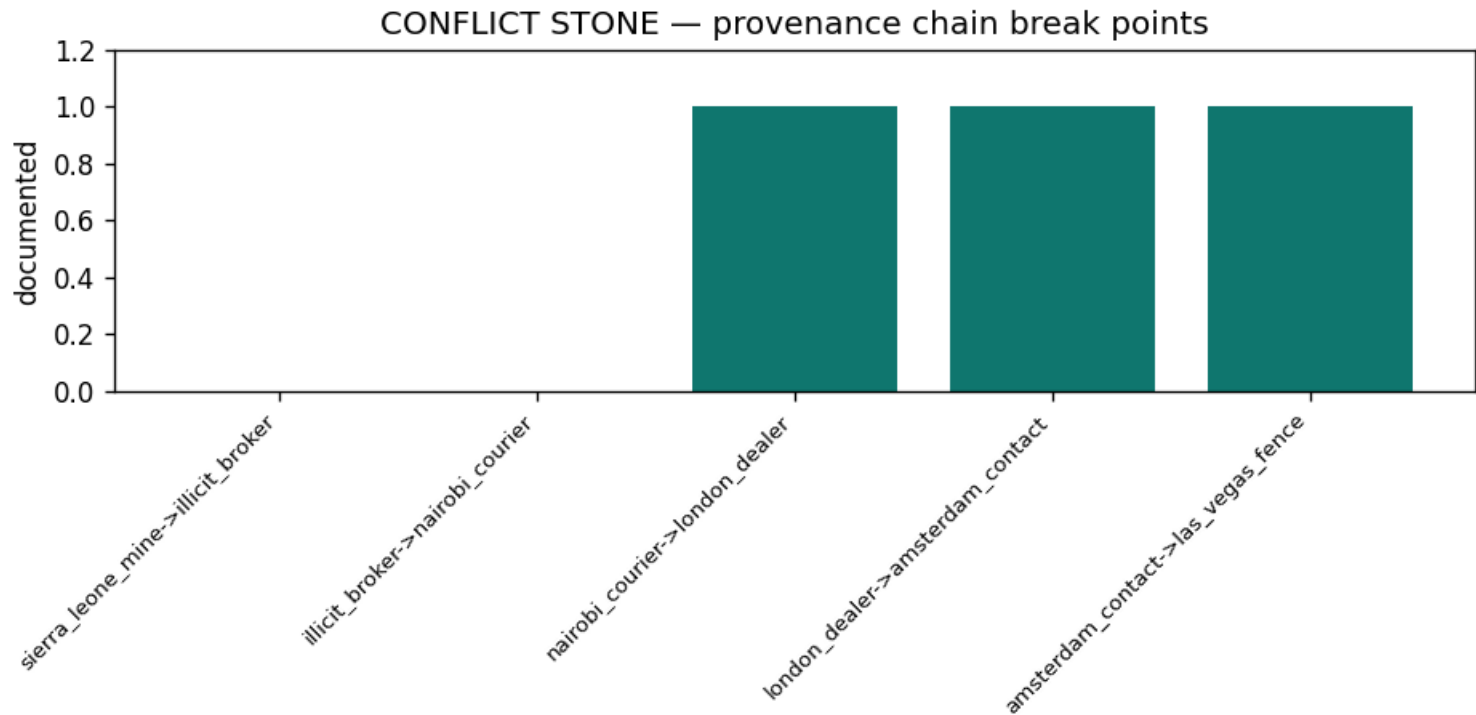


Figure 29: Provenance chain: documented handoffs vs. break points

10.5.2 Smuggling corridor routing

Across a 10-corridor network moving a 5.0 M USD cargo, the stochastic-optimum route is **sierra_leone -> nairobi -> paris -> amsterdam -> las_vegas** at an expected cost of 9.35 M USD, carrying the cargo with survival 34% (seizure exposure 66%). The fewest-hops baseline in the same graph (sierra_leone -> nairobi -> london -> amsterdam -> las_vegas) costs 11.15 M USD, so optimising over expected cost rather than hop count cuts expected cost by 16%: at the same hop count, the optimum swaps the pricey London-Amsterdam Channel ferry for the cheaper Paris leg. The two routes carry almost the same seizure exposure — the saving is on transport cost, not on seizure risk, which is why the survival probabilities barely differ. The smuggling figure plots the corridor expected costs with the optimum route marked.

Cargo-value sensitivity is reported as measured, not asserted. The optimal route is cargo-dependent: at low cargo value the transport cost dominates and cheap corridors win; as the cargo’s value grows the seizure probability dominates and a low-seizure, more-hop corridor wins instead. The cargo figure plots the resulting **route_reduction** across a cargo sweep. The curve is deliberately *not* monotone — measured it falls and then rises as the optimum switches from the Paris leg to the New York leg — so no single figure stands in for the dependency, and the canonical 5.0 M USD point is marked.

10.5.3 Lot forensics

Auditing 5 handoff samples of lot DF-71 by weight and density surfaces **2 anomalies** and a 4-point audit score — a **flagrant** verdict. The density probe catches the Amsterdam paste substitution (a measured density outside the 3.40-3.65 g/cm3 window); the weight ledger catches the final theft, a loss well beyond the 25% cutting allowance. The two rules are independent — one physical, one accounting — so the verdict does not rest on a single instrument. The lot figure shows the per-handoff weight deltas against the cutting allowance.

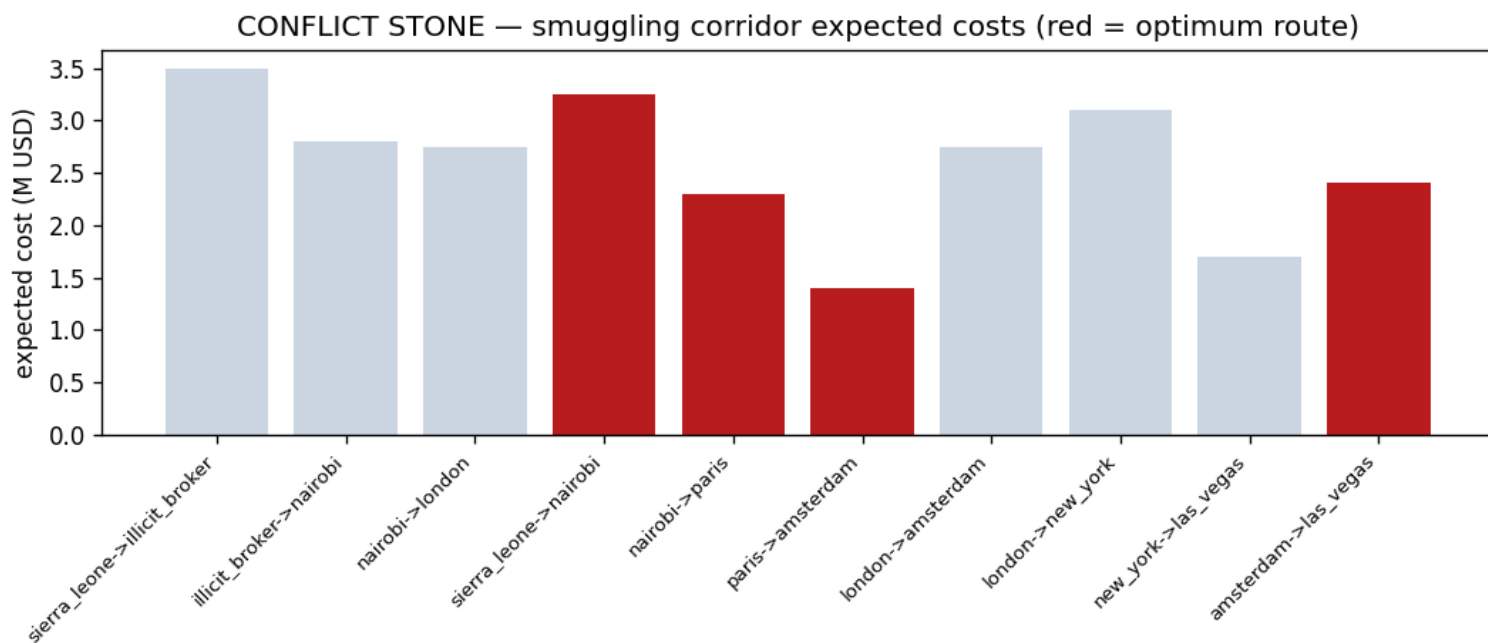


Figure 30: Smuggling corridor expected costs (red = optimum route)

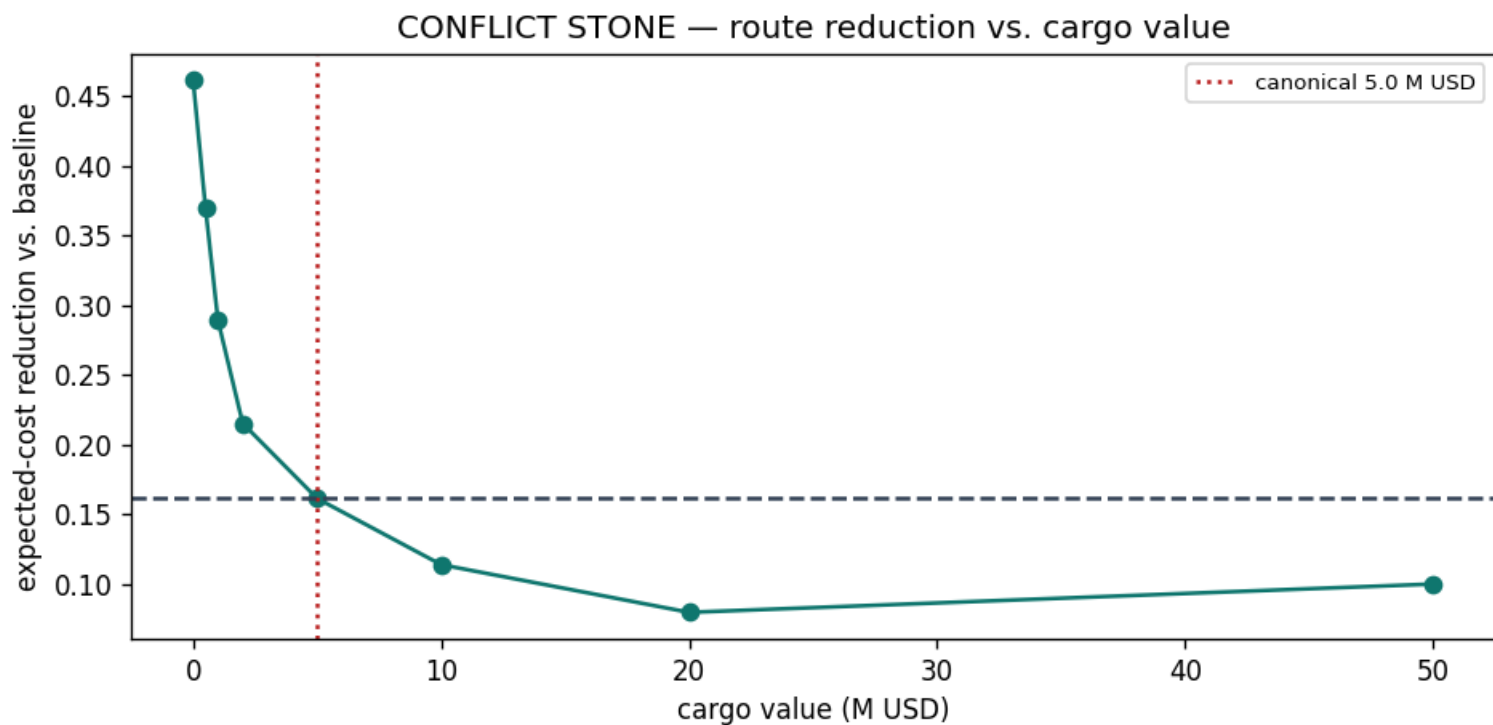


Figure 31: Route reduction vs. cargo value (canonical 5.0 M USD marked)

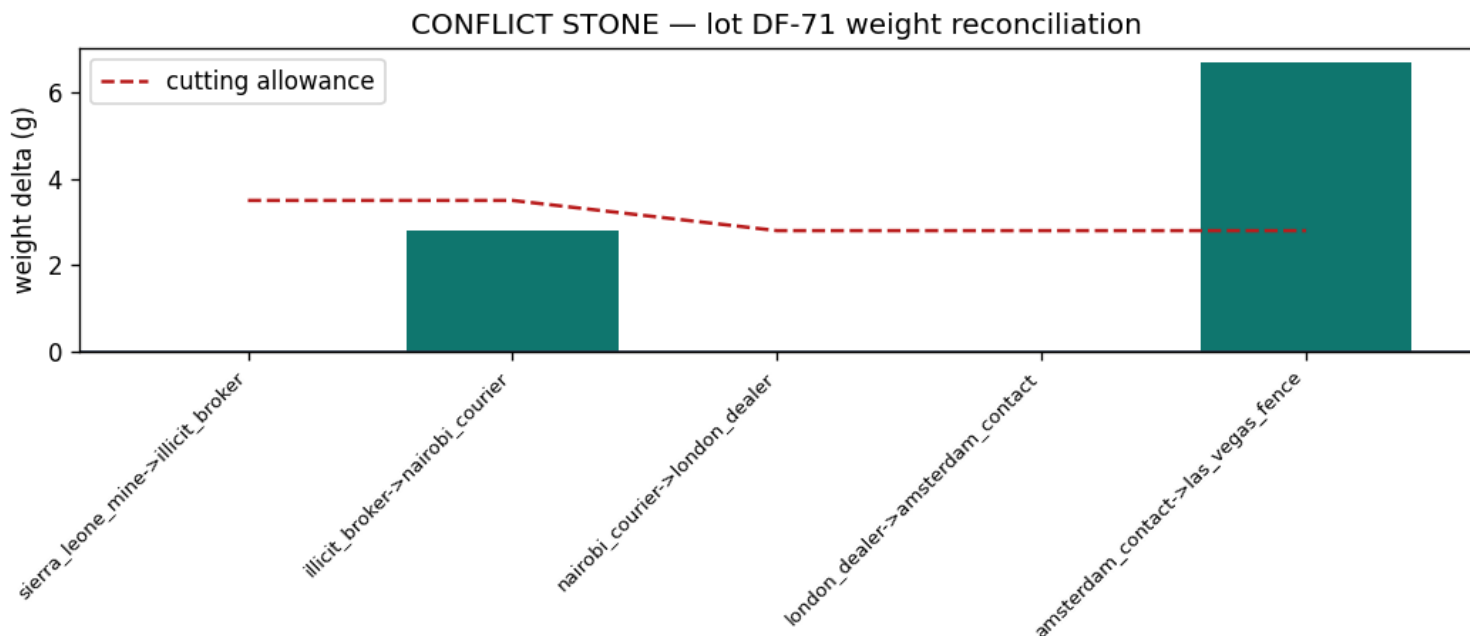


Figure 32: Lot weight reconciliation

10.5.4 Casino money movement

The strip network resolves to 7 casinos and 9 transfers moving 31.0 million US dollars in aggregate. The network hub is the **whyte_house** operation. Detection surfaces 3 distinct money cycles, and the laundering closure — the share of flow that circulates — is 77%. The casino figure visualises the graph.

10.5.5 Benford's-law ledger audit

The audit is reported as a matched pair, so that the anomalous verdict can be attributed to something.

Negative control. The legitimate baseline alone — 80 transactions whose leading digits are the Fibonacci leading digits, Benford-distributed by theorem rather than by construction against the test — scores a chi-square of **0.53**, far below the 15.507 critical value: a verdict of **conforms**. The screen does not fire on clean books.

Full ledger. Adding the 61 round-figure deposits takes the same statistic, computed by the same code path, to **35.80** over 141 transactions — above the $df=8$, $\alpha=0.05$ critical value of 15.507, a verdict of **anomalous**. Because the baseline passes on its own, the flag is attributable to the deposits and not to the baseline. The benford figure plots all three distributions — baseline, full ledger, Benford expectation — so the separation is visible. Digit 5 alone carries 55% of the statistic — the 5×10^n structuring. The full ledger runs above the Benford expectation on exactly the digits the deposits sit on (1, 2 and 5) and below it on every other digit.

This is a specificity check on two constructed fixtures, not a measurement of detection accuracy: it shows the implemented test separates *these* halves. No false-positive or false-negative rate is estimated here, and none should be read off it.

10.5.6 Orbital-mirror targeting

At a geostationary post of 36000 km, the mirror surveys 3 threatened cities, of which 2 fall within the visible face: Las Vegas, Washington DC. The remaining target (Beijing) is occulted by the Earth — no pointing solution exists for it from this post, whatever the mirror's power. The orbital figure reports the slant range of each target, shaded by beam visibility.

10.5.7 Beam delivery physics

The canonical scenario is a 40-m mirror working at 500 nm over a 36,998 km slant range, with an atmospheric transmittance of 45%. At the resulting 0.56-m diffraction-limited spot it delivers 768,382 W/m² — a beam concentrated 565× over raw sunlight, which would still need 325 seconds of dwell to vaporize a steel layer. The physics sets an honest bound on the weapon's fantasy: delivering 1 MW/m² at geostationary range requires a 42.7-m mirror. Because irradiance grows as the fourth power of the diameter, that bound moves slowly — the mirror is the binding constraint, and the dwell time is what a moving target denies. The beam figure shows delivered irradiance versus mirror diameter with the requirement marked.

CONFLICT STONE — casino money movement (line width ~ amount)



Figure 33: Las Vegas casino money movement

CONFLICT STONE — casino ledger vs. Benford's law

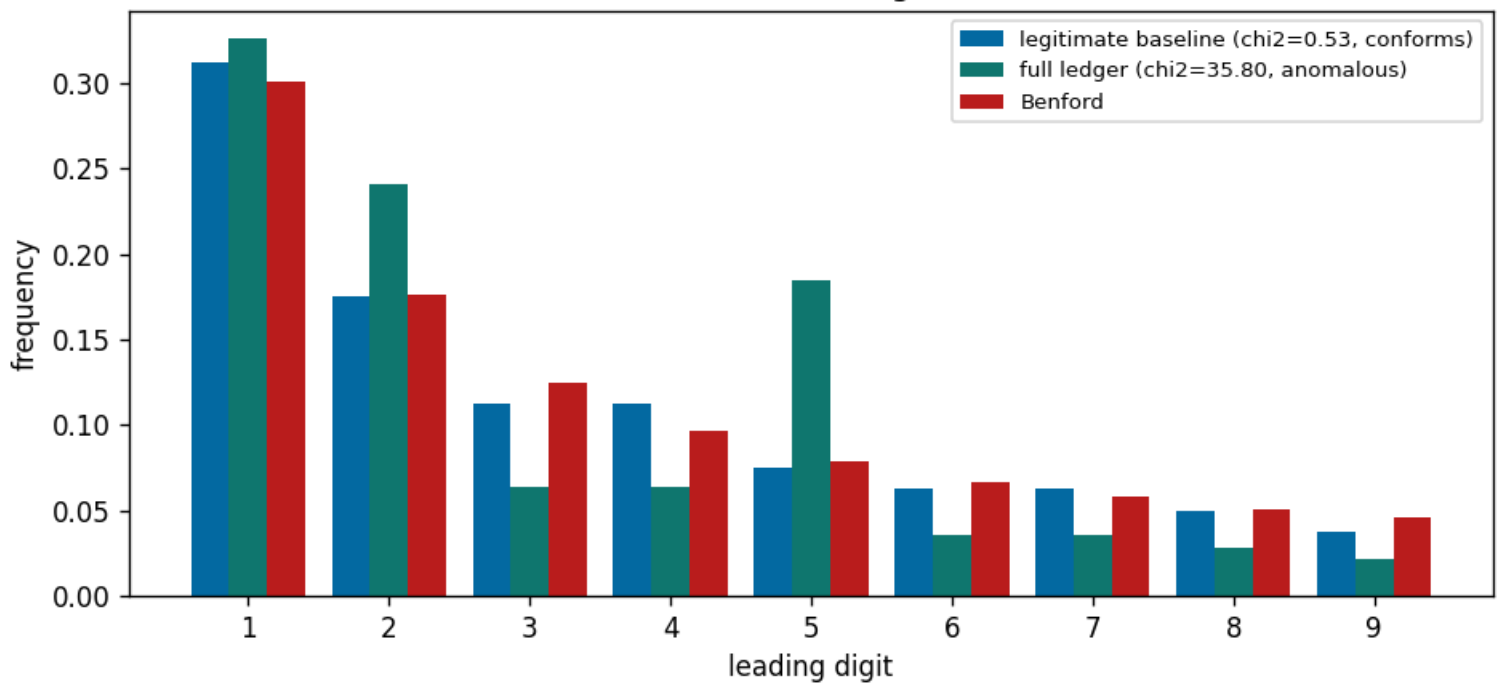


Figure 34: Casino ledger vs. Benford's law: legitimate baseline (negative control), full ledger, and the Benford expectation



Figure 35: Orbital-mirror slant range and visibility

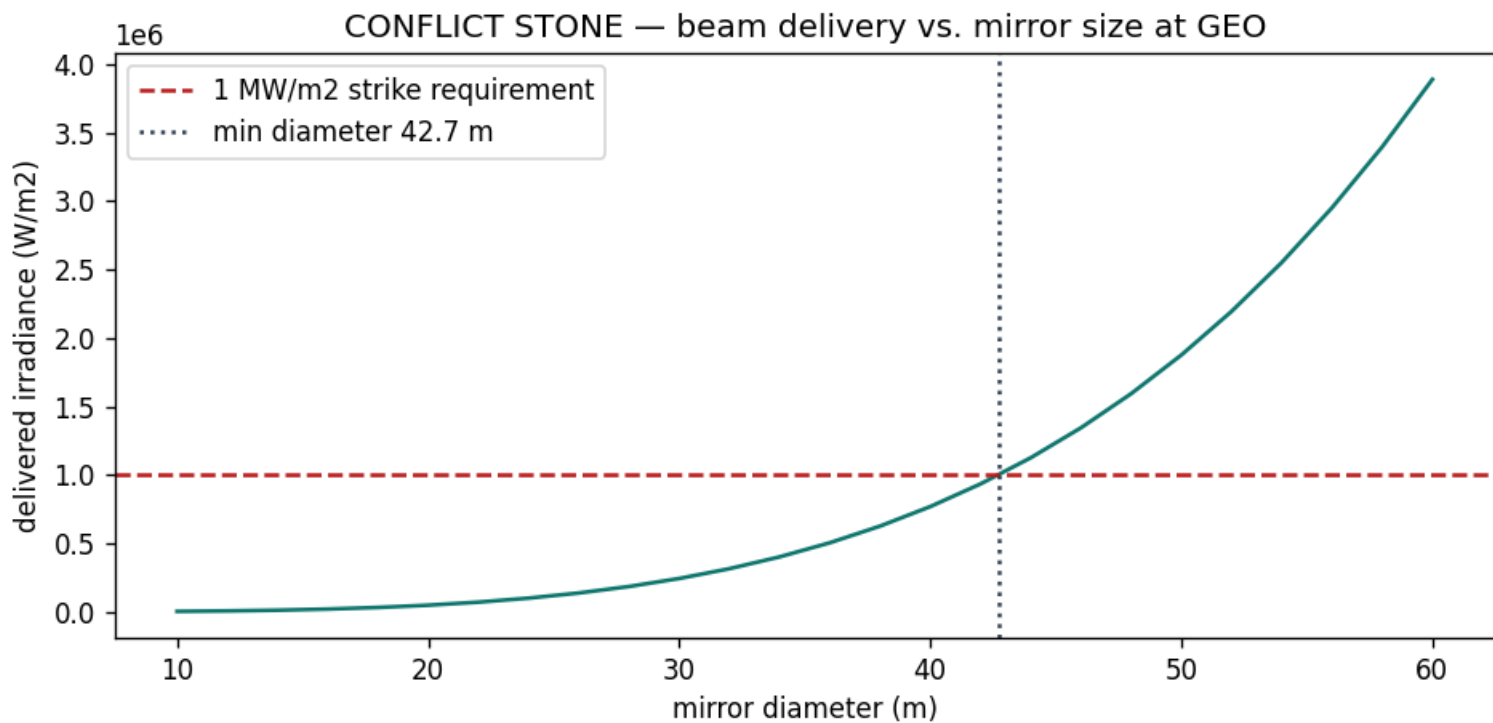


Figure 36: Beam delivery vs. mirror size at GEO

10.5.8 Mission outcome

The execute phase aggregates these seven lines into a single deterministic outcome carrying a full provenance block (package version, seed, and a SHA-256 input fingerprint). The debrief distills the standing lessons: an undocumented handoff is a provenance break; seizure risk rewrites the route; weight and density tell the physical story; circular money movement and a failing Benford test are the laundering signatures; and beam physics bounds the orbital weapon.

10.6 Conclusion — DIAMOND NET: findings, verdict, and what the mission establishes

The CONFLICT STONE mission demonstrates that a single deterministic package can fuse disparate traces — a smuggling provenance chain, a risk-weighted corridor route, a physical lot audit, a casino money-movement and Benford audit, and an orbital weapon’s targeting geometry and beam physics — into an auditable, reproducible special-agent report.

The provenance analysis shows that even a majority-documented chain of custody (60% coverage) retains flagrant break points that mark the goods as illicit, while risk-weighted routing proves the smuggler can cut expected route cost by 16% over the naïve route. Weight and density forensics independently convict the lot (verdict **flagrant**), the casino wash shows a laundering closure of 77%, and the ledger fails Benford’s law (chi-square 35.80 against a 15.507 critical value: **anomalous**). Finally, orbital physics grounds the space-weapon threat in real beam geometry: only 2 of 3 targets are reachable at all, because the mirror’s reach is hard-bounded by Earth’s curvature, and delivering 1 MW/m² at that range needs a 42.7-m aperture — the film’s weapon is not impossible so much as unaffordably large and slow.

By keeping the analytic cores pure and infrastructure-free, and by making the mission adapter a thin declarative layer over the frozen BOND-API protocol, the package guarantees that every claim in this manuscript is a live computation, not a hand-stated fact. CONFLICT STONE is delivered as a verified, deterministic mission that can be re-run end-to-end and audited from its input fingerprint alone.

10.7 Experimental Setup — DIAMOND NET: canonical scenarios, parameters, and configuration

All experiments are deterministic and use fixed canonical datasets seeded into the pure domain core; there is no random number generation and no mock framework.

Mission configuration. The package identity `diamonds_are_forever` runs under codename CONFLICT STONE, version 0.1.0. Key-words of the analysis: conflict diamonds, supply-chain provenance, risk-weighted routing, forensic gemology, money laundering, Benford’s law, network analysis, orbital targeting geometry, beam delivery physics, deterministic mission software.

Provenance dataset. 5 ordered custody handoffs along the mine-to-fence route; break points are flagged at build time.

Smuggling dataset. A 10-corridor directed network of transport costs and seizure probabilities, carrying a cargo valued at 5.0 million US dollars. The least-risk route is computed by a deterministic Dijkstra over expected cost; the baseline is a fewest-hops BFS in the same network, which bounds the reduction metric below at zero.

Forensic dataset. 5 weight/density handoff samples for lot DF-71, audited against a 25% per-handoff cutting allowance and a 3.40-3.65 g/cm³ density window (nominal 3.52), yielding a 4-point score.

Casino dataset. 7 casinos with 9 transfers totalling 31.0 million US dollars, and a 141-transaction ledger tested at df=8, alpha=0.05 (critical value 15.507). The ledger is a matched pair: a 80-transaction legitimate baseline drawn from Fibonacci leading digits (Benford-conforming by theorem — the negative control) plus 61 round-figure deposits. Each half is audited on its own as well as together.

Orbital dataset. 3 ground targets surveyed from a geostationary post at 36000 km.

Beam scenario. A 40 m mirror at 500 nm over a 36,998 km slant range, with atmospheric attenuation giving a transmittance of 45%; the strike requirement against which the minimum mirror is solved is 1 MW/m².

Software environment. Python 3.14.6, resolved at generation time — the one token whose value is read from the machine rather than from a committed input. The manuscript edition is stamped 2026-08-04, taken from `paper.date` in `manuscript/config.yaml`; no value in this manuscript comes from the wall clock, so re-running hydration on the same interpreter rewrites the same bytes.

10.8 Reproducibility — DIAMOND NET: verification gates, deterministic regeneration, and artifacts

CONFLICT STONE is reproducible by construction.

- **Determinism.** Every metric is a pure function of a frozen canonical dataset. No random draws, no wall-clock dependence in any persisted artifact (the provenance block records `wall_time_s` = 0). Routing tie-breaks on an insertion counter; adjacency lists and casino names are sorted.
- **Provenance.** Every mission outcome carries a package version (0.1.0), a fixed seed, and a SHA-256 fingerprint of the exact canonical inputs — corridors, lot samples, the beam scenario, and the ledger included — so an outcome can be audited without re-running the mission.

- **Lineage.** Two end-to-end runs produce byte-identical debriefs, and two hydration runs write a byte-identical `output/manuscript/` tree and `manuscript_variables.json`; the test suite asserts both directly, running the generator twice across a whole-second boundary so a reintroduced clock would be caught. The manuscript edition is stamped from `paper.date` in `manuscript/config.yaml`, never from the clock. The single token read from the machine rather than a committed input is `PYTHON_VERSION`, which is provenance of the interpreter: byte-identity is claimed for reruns on the same interpreter version.
- **Recomputation.** The resolved manuscript is generated by re-running `scripts/z_generate_manuscript_variables.py` under Python 3.14.6; the tokens it emits (e.g. provenance coverage 60%, laundering closure 77%, the Benford verdict anomalous) are live computations, never hand-stated.
- **Zero mocks.** All tests exercise real computations over real data; the only “doubles” are the real canonical datasets themselves, and every dataset is chosen so its headline metric is non-trivial (anomalous, flagrant, positive reduction) rather than vacuous.

10.9 Scope and Related Work — DIAMOND NET: boundaries, positioning, and relationship to the literature

10.9.1 Scope

This package analyses the traces central to *Diamonds Are Forever*: conflict-diamond provenance and corridor routing, physical lot forensics, casino money movement and its Benford signature, and the orbital mirror’s targeting geometry and beam physics. It is **self-contained and illustrative**: the canonical datasets are fixed, small, and embedded in the pure core. It is not a claim about any real investigation, casino, or satellite system; it is a deterministic, reproducible demo of the analytic methods, in the tradition of special-agent mission models.

10.9.2 Related work

- The provenance and forensic models generalise real supply-chain certification and diamond identification practice: the Kimberley Process formalises the “documented handoff” as the unit of traceability [Kimberley Process, 2003], early conflict-goods investigations documented the route and its gaps [Global Witness, 1998], and the economics of mineral-driven conflict are quantified in [Berman et al., 2017]. Diamond density testing follows standard gemological identification [Webster, 1994].
- The corridor routing is a textbook shortest-path problem over expected cost; Dijkstra’s algorithm [Dijkstra, 1959c] bounds optimality over the same search space as the naive baseline. The cargo-value sensitivity is swept as data because the optimum is cargo-dependent (cheap corridors at low value, low-seizure corridors at high value), not monotone.
- The money-movement analysis draws on the laundering-detection literature [Levi and Reuter, 2006]. Cycle detection is a *complete* elementary-circuit enumeration [Tarjan, 1973, Johnson, 1975] restricted to the small canonical graph, and the hub is a gross-flow reading of degree centrality [Freeman, 1978].
- The ledger audit rests on the leading-digit law observed by Newcomb [Newcomb, 1881] and rediscovered by Benford [Benford, 1938], scored with Pearson’s chi-square criterion [Pearson, 1900]; its use as a fraud indicator follows forensic-accounting practice [Nigrini, 2012]. The test is a screen, not a proof: a failing ledger warrants investigation, it does not by itself establish laundering.
- The beam physics uses the Airy diffraction pattern [Airy, 1835] and the exponential absorption law [Beer, 1852], both as given in classical optics [Born and Wolf, 1999]. It deliberately omits orbital mechanics, beam-riding control, adaptive optics, thermal blooming, and target material science beyond a single vaporization fluence, which are out of scope; the minimum mirror diameter is therefore a lower bound on the real requirement, not an estimate of it.

The package also documents its own platform: CONFLICT STONE (Diamonds Are Forever: CONFLICT STONE — Special-Agent Mission Software) is one member of the PROJECT BOND suite, which shares the frozen BOND-API mission protocol [BOND, 2026d].

10.9.3 Limitations and uncertainty

These are the honest bounds on what the package establishes, recorded so they are not over-read:

- **Constructed fixtures, not empirical records.** The Benford result is a specificity check on two designed halves of one ledger — a Fibonacci-derived negative control and round-figure deposits — so it establishes that this implementation separates *these two halves*, not a false-positive or false-negative rate. No detection accuracy is claimed or measured. Equally, the canonical custody chain, casino network, and orbital targets are illustrative by design (see *Scope*); they are chosen so each headline metric is non-trivial rather than vacuous, not to sample any real corpus.
- **The substitution finding rests on a single instrument.** Lot forensics separates paste from diamond on density alone (a second gemological channel such as refractive index is a stated extension, not implemented), so the density verdict is one-instrument. The weight-vs-density finding is nonetheless two-signal because theft (weight) and substitution (density) are audited independently.
- **The beam model bounds the weapon in one direction only.** It omits orbital mechanics, beam-riding control, adaptive optics, thermal blooming, and material science beyond one vaporization fluence. It also assumes the *full* captured power lands inside the first-null disk; the real Airy pattern puts $\approx 83.8\%$ of the power inside the first null, making the delivered irradiance $\approx 1.19\times$ optimistic and the minimum diameter $\approx 4.6\%$ small. Every omission — this one included — pushes toward a *larger* honest mirror, which is why the result is presented as a lower bound on the requirement, never as an estimate.

- **Determinism is not evidence of correctness.** The outputs are exact and reproducible by construction, but a deterministic, passing pipeline is only as good as the model that produced it; the same is true of the 100% line coverage figure, which measures execution, not truth. Where the model’s shape is wrong, reproducing it faithfully only reproduces the error.
- **Cargo-value behaviour is reported as measured.** The `route_reduction` sweep is *not* monotone in cargo value (it falls and then rises as the optimum switches corridors); no monotone trend is claimed, and the sweep figure plots the measured curve with the canonical value marked.

10.10 Sources — DIAMOND NET: bibliography

Friedman [2026i]; BOND [2026d]; Global Witness [1998]; Kimberley Process [2003]; Berman et al. [2017]; Webster [1994]; Dijkstra [1959c]; Levi and Reuter [2006]; Benford [1938]; Born and Wolf [1999]; Newcomb [1881]; Pearson [1900]; Nigrini [2012]; Tarjan [1973]; Freeman [1978]; Airy [1835]; Beer [1852]; Johnson [1975]

11 Live and Let Die (1973) — SAN MONIQUE

film package · package codename SAN MONIQUE. Mission **SAN MONIQUE**: map the narcotics supply graph and identify the critical transit bottleneck; decipher the voodoo-coded communiqué by symbol-substitution; plan an infiltration route across crocodile terrain to the airfield.

11.1 Concepts — SAN MONIQUE: domain and operational focus

narcotics networks, coded comms

11.2 Abstract — SAN MONIQUE: mission summary

— title: “Live and Let Die: SAN MONIQUE — Special-Agent Mission Software” date: “2026-08-05” — Live and Let Die: SAN MONIQUE — Special-Agent Mission Software (mission codename **SAN MONIQUE**) delivers deterministic, special-agent mission software for Kananga’s narcotics operation on the Caribbean island of San Monique. The system models the narcotics supply chain as a directed, capacity-constrained graph of 7 nodes and 10 legs spanning production, transit, and market. It recovers the cheapest route to each destination market (13 units to Miami), computes the maximum network throughput (220 units, capped by a min-cut value of 220), and identifies the single highest-value interdiction target: the **airfield**, whose neutralization stops 110 of those 220 units. Both figures are measured on the same demand-capped flow model; the min cut itself is demand-limited and names no severable asset, so the target comes from the disruption probe and not from the cut. Building on the graph core, a **budgeted interdiction** study shows the network is operationally fragile: an optimal 3-leg strike plan (greedy 220 units, exact optimum 220) severs the entire 220-unit operation, and the top individual target leg, `field_works→harbor`, is worth 50 units alone. The intelligence layer deciphers both a voodoo-coded communiqué by keyword symbol-substitution (key **BARON**, recovering “*KANANGA LETS THE SNAKE BITE*”) and a 195-beat drum-code transmission (exact round-trip, density 0.441) carrying “*KANANGA LETS THE SNAKE BITE*”. A market-flow economic model traces price and purity from the farm gate (300 \$/kg) to the street, yielding a farm gate-to-street spread of 59× and a producer share of 1.7% of the 3888000 \$ gross revenue. Those two numbers are arithmetic consequences of invented scenario parameters, not empirical estimates — see §Scope. Finally, a terrain-graph infiltration planner routes an agent from the south landing beaches to the airfield at cost 41, wading 4 crocodile-water cells against a computed floor of 4 over every possible route. All analyses are deterministic, real-data-only, and fully provenance-stamped (input hash 46044979efeafa09).

11.3 Introduction — SAN MONIQUE: mission framing, the operational problem, and how to read this chapter

Live and Let Die (1973) follows James Bond to the fictional Caribbean island of San Monique, where the villain Kananga — governor by day, narcotics baron by night — runs a poppy-to-market operation and protects his enterprise with voodoo-coded radio traffic and a mangrove-and-crocodile coastline. This package turns that milieu into 6 deterministic computation problems — one pure domain module each — of the kind an intelligence analyst would actually run before an operation:

1. **Supply-graph analytics.** Model the narcotics chain as a capacity-constrained flow network and answer operational questions: what is the cheapest route to market, how much volume can move, and which single transit asset — if seized — costs the enemy the most? The answer sharpens the targeting case against a specific bottleneck.
2. **Budgeted network interdiction.** One bottleneck is a target; a *strike plan* is a campaign. Given a fixed budget of legs that can be severed in one coordinated operation, choose the set that costs the enemy the most throughput — and bound the heuristic’s quality with an exact optimum.
3. **Coded-communiqué deciphering.** The voodoo “symbol substitution” is a monoalphabetic keyword cipher. Recover the substitution table and decrypt the intercepted message; a frequency-attack solver provides an unkeyed best-effort fallback.
4. **Drum-code signalling.** The cult’s drums are a channel, not set dressing. Encode and decode messages as Morse-timed rhythm on a per-beat grid, and reduce an intercepted transmission to a comparable traffic signature.
5. **Market-flow economics.** Volume is only half the operation; the other half is money. Propagate price and purity from the farm gate to the street to establish who actually captures the value — and therefore where interdiction bites.
6. **Island ops planning.** San Monique is a terrain graph of mangrove, swamp, open coast, and a crocodile channel. Plan the lowest-cost infiltration route while tracking how much of it crosses crocodile water.

Across all 6 modules the mission lifecycle of the BOND-API protocol (brief, recon, plan, execute, debrief) is exercised: a brief declares the 3 mission objectives, recon surveys the ground truth, planning schedules the analytical steps, execution computes every result, and debriefing distills actionable lessons with full provenance (fixed seed 8, canonical input hash). The same analysers are exposed as 6 named gadgets so a coordinator can invoke any one of them directly, without running the full mission.

11.4 Methodology — SAN MONIQUE: the analytical models and algorithms that drive the mission

11.4.1 Supply-graph analytics

The San Monique operation is represented as a directed, weighted, capacity-constrained graph $G = (V, E)$. Vertices are classified as **production** (interior poppy fields, field processing works), **transit** (airfield, harbor, smuggler’s cove), or **market** (Miami, New York).

Each edge carries a unit cost (money, risk, time) and a throughput capacity (units per reporting period).

- **Cheapest route.** Dijkstra’s algorithm returns the minimal total cost and ordered node sequence from a production source to each market. Deterministic tie-breaking uses (`cost`, `node_id`) heap keys.
- **Maximum throughput.** Edmonds-Karp (BFS augmenting path) computes the maximum flow that can be pushed from a virtual super-source (SUPPLY) through the network to a virtual super-sink (DEMAND). Production nodes are bounded by their crop capacity and markets by their demand ceilings.
- **Bottleneck — and what the cut does *not* tell us.** The minimum cut (the set of forward edges leaving the residual-reachable set after max-flow) bounds throughput. On this instance the cut is `miami`→DEMAND, `new_york`→DEMAND: both are *virtual* edges into the super-sink encoding market demand ceilings, and 0 physical supply legs appear in it. The network is therefore demand-limited, and the min cut is a capacity bound that names nothing an operation could strike. Interdiction targeting comes from a separate per-transit-node disruption probe, which re-computes flow after neutralizing each transit asset and reports the one whose loss costs the most. That probe runs on the *same* demand-capped flow graph as the max-flow figure, so its answer is expressed in — and bounded by — the throughput the model actually admits.

11.4.2 Budgeted network interdiction

Beyond a single bottleneck, the operation asks *how much of the network can be taken down with a limited strike budget*. This is the deterministic network interdiction problem: with a budget B of transit legs that can be severed in one coordinated operation, choose the legs that cost the enemy the most throughput. Two solvers are provided:

- **Greedy** — myopically sever the leg with the largest marginal throughput drop, then repeat; deterministic tie-breaking on leg order.
- **Exact** — exhaustive enumeration of all B-subsets of severable legs (feasible at this scale) that maximises reduction; it is the correctness reference for the greedy heuristic.
- **Efficiency ranking** — each leg’s *independent* throughput impact, ranked, forming the target list.

11.4.3 Coded communiqués: symbol substitution and drum code

The voodoo “symbol substitution” is a monoalphabetic keyword cipher over the 26-letter alphabet: a keyword (here BARON) seeds a reshuffled cipher alphabet (duplicates dropped, remaining letters appended), and each plaintext letter maps to the cipher symbol at its index. To handle an *unkeyed* intercept, `solve_substitution` performs a frequency attack: ciphertext symbols are ranked by observed frequency and matched to reference English letters ranked by expected frequency; the candidate plaintext is scored by cosine similarity of its frequency profile against the reference distribution.

Alongside the written cipher, the cult’s **drum code** is a Morse-style rhythm codec over a per-beat 0/1 grid, on standard International Morse timing (dot = 1 sounding beat, dash = 3; intra-letter gap 1, inter-letter gap 3, inter-word gap 7). `signal_for_text` encodes any message onto the beat grid and `decode_signal` recovers it exactly under strict run-length validation. `rhythm_signature` / `signature_distance` give a numeric traffic signature so two intercepted signals can be compared.

11.4.4 Market-flow economics

The money side: the same legs that carry volume also mark prices up and dilute purity from the farm gate (300 \$/kg) to the street (2 markets). Price and purity propagate along the cheapest path (by arbitrage the lowest-cost channel sets the price), and the street stage applies a retail markup and a purity dilution. The headline metric is the **producer share** — the farm-gate value as a fraction of street price — and the **price-spread ratio**, both driven by the per-leg economics dataset `EDGE_ECONOMICS`.

Every parameter in that dataset, and the farm-gate price, retail markup and retail dilution alongside it, is invented for this fictional scenario; none is fitted to data. The producer share the model reports is therefore the reciprocal of a chosen product of markups. What is *demonstrated* is the mechanism rather than the magnitude: the test suite re-runs the identical model on substituted parameter sets and shows the share moving with them — under a neutral table with no transit markups it rises to the reciprocal of the retail markup alone.

11.4.5 Island ops planning

San Monique is discretized as a 9x9 terrain grid where each cell is a terrain type with a movement cost (road / open ground / beach cost 1, mangrove / open water cost 3, swamp cost 5, crocodile water cost 7, impassable infinite). Dijkstra over the four cardinal neighbours finds the lowest-cost route between the landing beach and the airfield zone; the planner reports total cost, the path, and the number of crocodile-water cells traversed as a hazard metric.

That count describes one route. To say anything about *every* route the planner runs a second Dijkstra whose edge weight is 1 for entering a crocodile cell and 0 otherwise; its optimum is the minimum crocodile exposure over the whole route space, and no route — however expensive — can wade fewer cells than that. Only the second number licenses a universally quantified claim. The two are distinct quantities: where a crocodile-free detour exists the cheapest route may wade while the minimum is zero.

11.5 Results — SAN MONIQUE: measured outcomes, headline numbers, and what they establish

All figures below are deterministic and regenerable from the source tree.

11.5.1 Supply graph

The canonical San Monique network has 7 nodes and 10 legs. The cheapest production-to-market routes are:

Destination	Cost	Route
Miami	13	poppy_fields -> airfield -> miami
New York	18	poppy_fields -> field_works -> harbor -> new_york

Total maximum throughput is 220 units (220 bound), capped by destination-market demand. The min cut is miami→DEMAND, new_york→DEMAND — both virtual demand edges, 0 physical legs — so the network is demand-limited and the cut is a bound, not a target list.

The production flow into miami, new_york is nonetheless sharply exposed to a single asset. Probing the same demand-capped model node by node, the most critical transit node is the **airfield**: neutralizing it removes 110 of the 220 units the network moves — the highest-value single interdiction on the island.

San Monique narcotics supply graph
(red = cheapest production→Miami route · bottleneck = airfield)

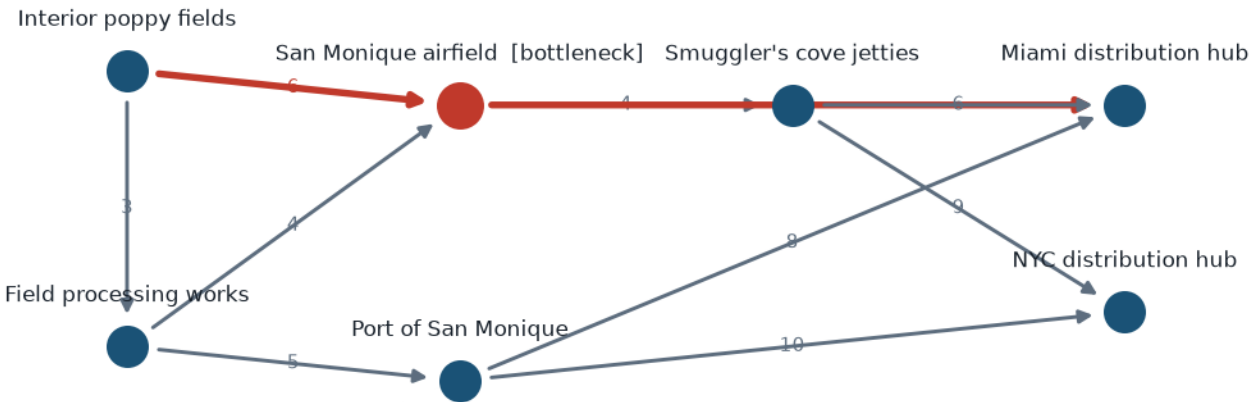


Figure 37: The San Monique supply network: cheapest production-to-Miami route and the critical transit node highlighted.

11.5.2 Interdiction

Against a base flow of 220 units, a coordinated strike plan is frighteningly effective. The greedy plan severs field_works→harbor, airfield→miami, airfield→cove for a simultaneous 220-unit reduction (cost-effectiveness 73 units per leg, leaving 0); the exhaustive optimum severs airfield→cove, airfield→miami, field_works→harbor for the same 220-unit total — the entire operation collapses. The single most valuable leg on the target list is field_works→harbor (50 units alone).

11.5.3 Communiqués

The intercepted voodoo communiqué deciphers under the keyword substitution (BARON) to:

KANANGA LETS THE SNAKE BITE

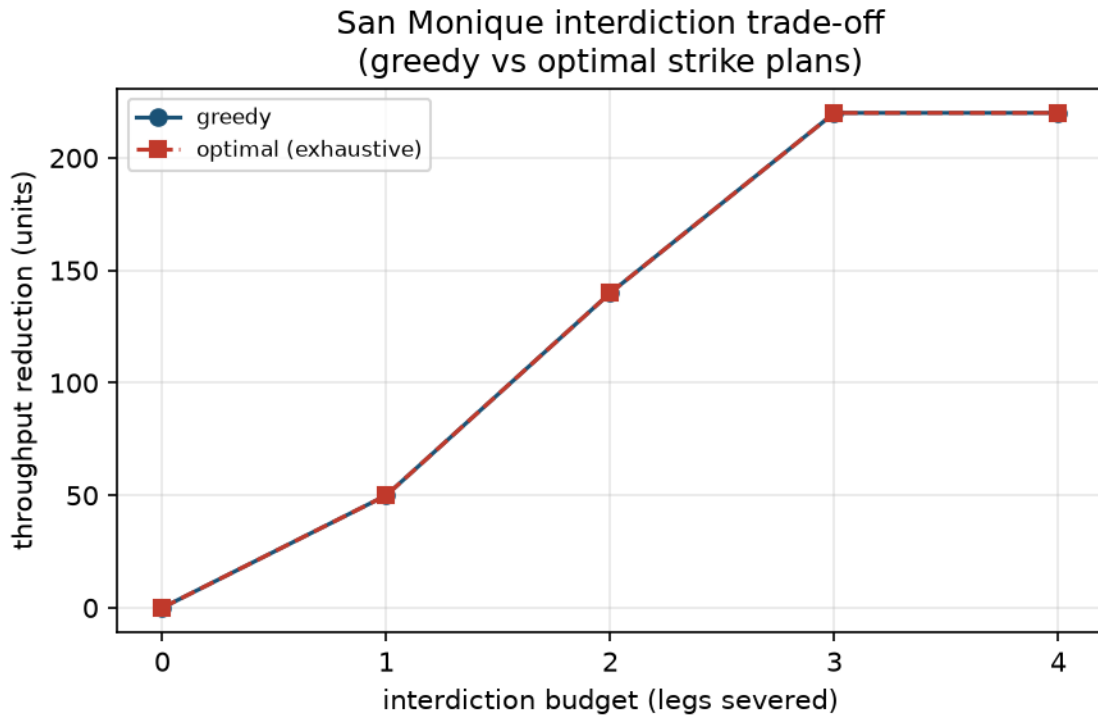


Figure 38: Greedy versus exhaustive-optimal throughput reduction across strike budgets.

The ciphertext after encipherment is HBKKBDB INTS TEN SKBHN AFTN.

The cult’s drums carry a 195-beat message that decodes exactly (density 0.441, longest run 3) to:

KANANGA LETS THE SNAKE BITE

(decoding verified **exact**).

11.5.4 Market economics

From a farm-gate price of 300 \$/kg, street-level prices reach 14400 \$/kg at 57% purity in Miami and 21600 \$/kg at 49% purity in New York. The demand-weighted average street price is 17673 \$/kg — a 59× farm-gate-to-street spread — and the farm-gate producer retains **1.7%** of the street value. Across 2 markets the operation grosses about 3888000 \$.

Market	Street price (/kg) <i>Street</i> purity <i>Demand</i> <i>Revenue</i> ()		
Miami	14400	57%	120
New York	21600	49%	100
Total	—	—	220
			3888000

The demand figures in the table come from the same committed SAN_MONIQUE_DEMAND dataset that caps the supply-graph flow, so the revenue section and the throughput section share one source of truth. Every row is computed by `market_flow_report` — no value is hand-authored.

These are outputs of invented parameters, not measurements: they establish that multiplicative per-leg markups compress the producer’s share, and nothing about the size of that share in any real market (see §Scope). The 1.7% producer share is the reciprocal of the product of chosen markups, so it tracks any change in those assumptions exactly — a sensitivity the test suite re-runs rather than reports as a fixed interval.

11.5.5 Infiltration route

The terrain planner routes an agent from the south landing beach ((8,1)) to the airfield zone ((0,4)) in 11 moves at total cost 41, crossing 4 crocodile-water cells.

The exposure-minimising search puts the floor over *all* routes at 4 cells: the channel is walled by impassable cliff on both flanks, so no overland approach to the airfield — at any movement cost — wades fewer than 4. The cheapest route achieves that floor exactly.

11.6 Conclusion — SAN MONIQUE: findings, verdict, and what the mission establishes

The SAN MONIQUE mission software turns the Live and Let Die set pieces into deterministic, auditable computation. The narcotics

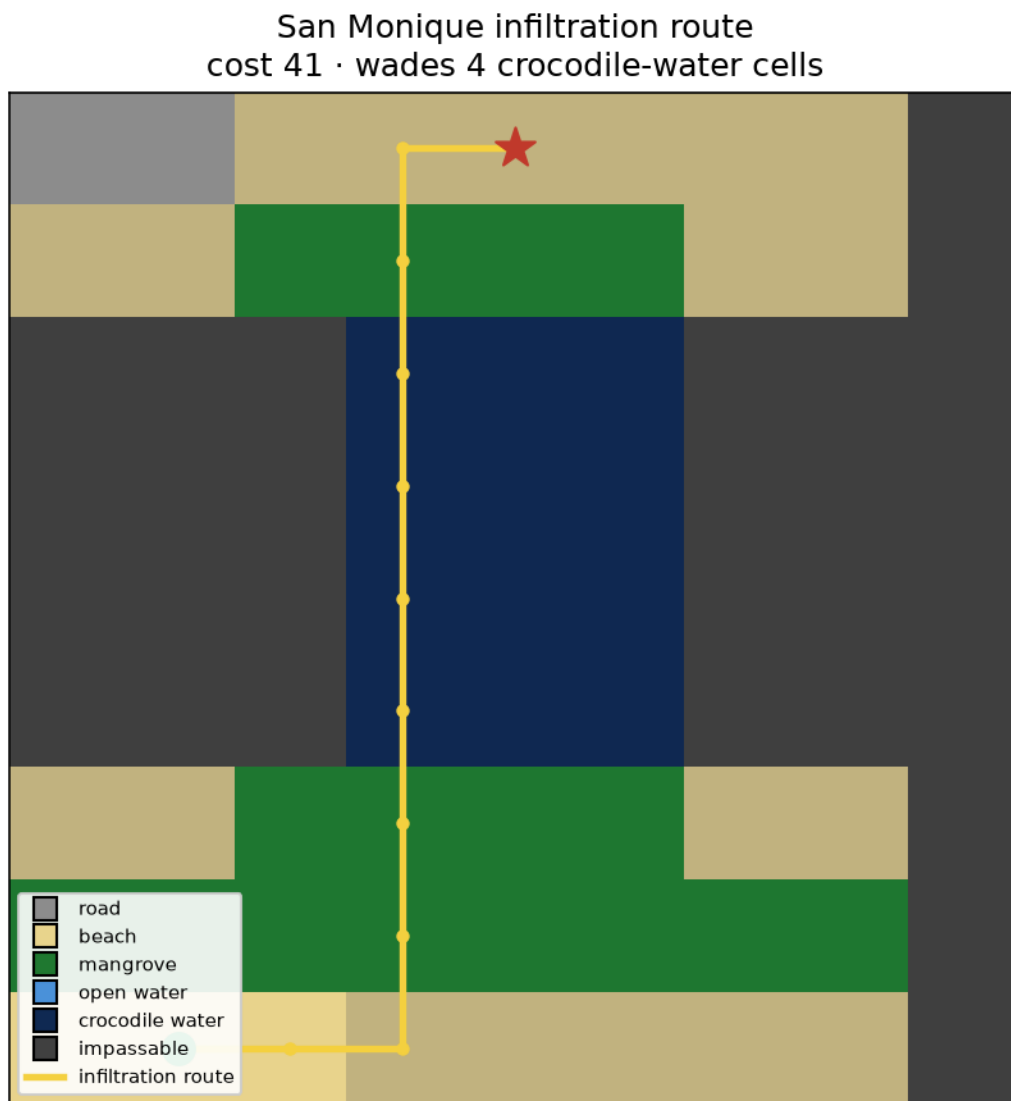


Figure 39: The terrain grid with the planned infiltration route overlaid.

- **Market economics** (`market_flow.py`): per-leg price-markup and purity dilution factors in `EDGE_ECONOMICS`, a farm-gate price of 300 \$/kg, and retail markup/dilution factors.
- **Terrain** (`island_ops.py`): a fixed 9x9 terrain grid (9 rows x 9 columns) of beaches, open ground, mangrove, swamp, and a crocodile channel walled by impassable cliff.

11.7.2 Determinism

No random draws and no wall-clock dependence enter any analytic result. The mission records a fixed provenance seed (8), a canonical input hash (46044979efeafa09) spanning every analysis input (graph, ciphers, drum message, and per-leg economics), and a zero wall-clock figure. Route, flow, augmenting-path, and interdiction tie-breaking are all order-stable; the greedy and exact interdiction solvers are fully enumerable for the canonical scale.

11.7.3 Figures

Three figures are regenerated by `scripts/generate_figures.py` under `../figures/` from the same pure-domain data the manuscript numbers are computed from — no figure metric is hand-authored, and figure captions interpolate computed values (for example the bottleneck node name) rather than naming them in a string literal. All three are embedded in `$Results` through the `../figures` token, which is derived from the real output paths so the rendered section carries a link that resolves; a test opens every embedded path and fails if a figure is missing or renamed:

- `supply_graph.png` — the network with the cheapest production-to-Miami route and the critical transit node highlighted;
- `interdiction_curve.png` — greedy versus exhaustive-optimal throughput reduction across strike budgets;
- `island_route.png` — the terrain grid with the planned infiltration route overlaid.

11.8 Reproducibility — SAN MONIQUE: verification gates, deterministic regeneration, and artifacts

Every number in this manuscript is computed, never typed: the thin orchestrator `scripts/z_generate_manuscript_variables.py` calls `generate_variables` in `src/live_and_let_die/manuscript_variables.py`, which computes each placeholder from the pure domain core. The manuscript cross-reference test scans every numbered section (`manuscript/[0-9]*.md`) and fails if any placeholder used in prose is not produced by `generate_variables`, so the prose cannot cite a number the code does not compute. (The converse is not gated: a produced token that no section uses is permitted and is not a test failure.)

Result provenance is recorded per-mission: the canonical input hash is 46044979efeafa09, the fixed seed is 8, and execution sets a zero wall-clock figure. A SHA-256 digest of the canonical inputs fingerprints the inputs, so a re-run can be checked against an audited outcome — the digest identifies the inputs, it does not by itself reconstitute the results.

11.8.1 Byte-identical regeneration

Running `scripts/z_generate_manuscript_variables.py` twice on an unchanged tree produces byte-identical artifacts. **Scope of that claim:** the hydrated table output/data/manuscript_variables.json and every resolved section under output/manuscript/. Every value in those files comes from a committed source — the pure domain core, or `manuscript/config.yaml` — and nothing under `src/` reads the system clock, so there is no run-varying byte to differ. The publication date above is `paper.date` in `manuscript/config.yaml`, bumped by hand when the manuscript is republished; it is deliberately not a generation timestamp, because a generation timestamp would make every regeneration differ.

The claim is gated, not asserted: `tests/test_scripts_smoke.py::test_regeneration_is_byte_identical` runs the real script twice — separated by more than one wall-clock second, so a second-resolution clock cannot slip through by landing in the same second — and diffs the persisted bytes. Its in-process counterpart is `tests/test_manuscript_variables.py::test_regenerating_variables_across_a_wall_clock_second_is_identical`.

Two things are outside the scope of the byte-identical claim. Figures under `../figures/` are PNGs written by a separate script and are not diffed here. And nothing in this package records a real timestamp anywhere — if a future run-scoped provenance record needs one, it must live outside the two paths named above, and this scope statement must be amended to say so.

Verification gates (see README):

```
uv run pytest tests/ --cov=src --cov-fail-under=90
uv run ruff check src/ scripts/
uv run mypy src/
rg -n "template[_]code_project". # expectation: zero
```

11.9 Scope and Related Work — SAN MONIQUE: boundaries, positioning, and relationship to the literature

This package is one film in the PROJECT BOND suite of 33 packages: 27 film packages implementing a shared, FROZEN mission protocol (`bond_api.MissionProvider`) over pure domain cores, plus six infrastructure packages (api, utilities, coordinator, orches-

trator, cli, ops). It is a local working exemplar developed from the template computing-project scaffold — lineage is documented, not reproduced as source.

The analytical techniques are classical and deliberately textbook-grade, spanning three research literatures:

- **Max-flow / min-cut and network interdiction.** Edmonds-Karp with the residual-cut theorem underpins bottleneck and single-point-of-failure analysis. The budgeted interdiction problem — minimize remaining throughput given a fixed number of removed legs — follows Wood’s deterministic network interdiction formulation; the greedy and exhaustive solvers here are self-contained (wood1993).
- **Coded communications.** Monoalphabetic keyword substitution with a frequency-attack solver (stinson2019) sits beside the drum code, a rhythm codec over a per-beat grid whose element and gap durations follow the normative International Morse timing ratios (itu2009) — a numerical treatment of the Caribbean drum telegraphy the film uses.
- **Narcotics market economics.** The *question* — who captures the value between farm gate and street, and what that implies for enforcement — is the one posed by Reuter & Kleiman and by Caulkins & Reuter (reuter1986, caulkins2010). The *parameters* are not theirs and are not anyone’s data: the farm-gate price, the retail markup and dilution, and all ten per-leg markup/dilution pairs in EDGE_ECONOMICS were chosen for this fictional scenario. Nothing here estimates, calibrates against, or reproduces a published figure, and the producer share reported in §Results should not be read as agreeing or disagreeing with any empirical estimate. The model contributes a mechanism — multiplicative per-leg markups compress the producer’s share — exercised on invented numbers.
- **Terrain pathfinding.** Grid Dijkstra is the standard cost-minimising planner for rasterized maps, applied to a hazard-labelled island.

The novel contribution is not the algorithms but their unification: 6 film concepts, 6 pure modules exposed as 6 named gadgets, one frozen mission protocol, one provenance chain — reproducible end to end with no mocks and no external services.

11.9.1 Limitations and uncertainty

The analytical machinery is exact and deterministic (integer-capacity graphs, closed-form economics, fixed terrain), so it carries no sampling error — but that precision is not accuracy. Every headline number in §Results inherits the uncertainty of the *inputs* it is computed from, which are assumptions, not measurements:

- **Supply-graph capacities, crop ceilings, and demand ceilings** are invented scale figures for the fictional scenario. The throughput (220) and disruption (110) figures are exact arithmetic on those chosen numbers; changing any capacity changes both. The analysis establishes *structure* — the network is demand-limited, hinges on one transit asset — not a measured magnitude.
- **Economic parameters** (farm-gate price, retail markup/dilution, all ten per-leg markup/dilution pairs) are invented. The producer share (1.7%) and the price spread (59×) are the exact reciprocals of chosen products of markups; re-run the model under different parameters and they move exactly as the assumptions dictate. The test suite binds this as sensitivity, not as a confidence interval, because there is no sampling distribution to quantify — only the chosen inputs.
- **Interdiction** greedy-vs-optimal: the greedy and exhaustive solvers agree at 3-leg reduction here (220 / 220), but greedy is a heuristic and its optimality is *not* guaranteed in general. On this instance the exhaustive optimum confirms the greedy plan is exact — that is evidence for this graph, not a general bound.
- **Terrain costs** are unit figures chosen for the scenario; the route cost

(41) and crocodile exposure (4) are exact on those costs. The universal claim (any route wades at least

4) is a structural fact of the committed grid (impassable cliff walls), so it is robust to the cost *numbers*, though it does depend on the grid topology being the island’s.

In short: the package’s contribution is deterministic, reproducible computation over explicitly declared assumptions. Figures are recomputable to the last unit, but they should be read as “exact given these invented parameters”, never as empirical estimates. This is the honest scope of the whole suite, and it is enforced by the test gates rather than stated once.

11.10 Sources — SAN MONIQUE: bibliography

Mankiewicz [1973]; Cormen et al. [2009b]; Wood [1993a]; Stinson and Paterson [2019]; International Telecommunication Union [2009]; Reuter and Kleiman [1986]; Caulkins and Reuter [2010]

12 The Man with the Golden Gun (1974) — SOLEX

film package · package codename SOLEX. Mission **SOLEX**: model solar concentration yield (mirror array to heat); track solar position and daily clear-sky insolation; solve the funhouse lair escape topology (rooms/mirrors as graph); analyse lair flow: edge-disjoint routes, min cut, cheapest escape; schedule the assassin duel/bounty appointment book; schedule unit-time deadline bounty contracts (max profit).

12.1 Concepts — SOLEX: domain and operational focus

solar concentration, lair topology, assassin scheduling

12.2 Abstract — SOLEX: mission summary

Mission codename: SOLEX — Project BOND film dossier for *The Man with the Golden Gun* (1974). This dossier documents the deterministic mission software built for the SOLEX operation. 6 specialist capabilities support the field agent: 1. **Solar concentration yield** — a heliostat-field model that converts a mirror array’s area and reflectivity into optical power and thermal heat output, and allocates a mirror-area budget to the highest-yield mirrors. 2. **Solar tracking & daily insolation** — the sun-geometry layer: declination, daylight window, and the Duffie–Beckman clear-sky daily energy the field concentrates. 3. **Funhouse lair escape topology** — a graph solver over the villain’s mirror-lined funhouse, producing a shortest escape route and identifying the bottleneck rooms that guard the exit. 4. **Lair flow analysis** — max-flow / min-cut over the lair, returning the edge-disjoint escape routes and the minimum mirror walls that seal the lair, plus a cost-aware (Dijkstra) cheapest escape. 5. **Assassin duel / bounty scheduling** — an appointment book solved by weighted interval scheduling, selecting a non-overlapping set of bounty duels that maximises total reward and reporting every scheduling conflict. 6. **Deadline bounty scheduling** — unit-time bounty contracts scheduled by the deadline-greedy (union-find) algorithm to maximise accepted profit. All results are computed from real, deterministic domain models with fixed inputs and no wall-clock dependence; every metric in this manuscript flows from code, never from hand-authored prose. The canonical SOLEX field converts a 1000 W/m² irradiance into a thermal output of 191.6325 kW (concentration 50.10×) and, over a clear-sky solstice day at 8.0°N, concentrates 1441.7 kWh. The extended funhouse has 3 edge-disjoint escape routes (min cut 3 mirror walls), while the canonical lair is guarded by 3 bottleneck rooms — redundancy and criticality are separate measurements, and the lair’s 4 articulation points are a third. On the duel book the two exact schedulers disagree: weighted interval scheduling realises a bounty of 13.0 where the count-maximising earliest-finish greedy realises 11.0. The deadline book yields a maximum profit of 18.0 across 5 contested contracts.

12.3 Introduction — SOLEX: mission framing, the operational problem, and how to read this chapter

The Solex agitator concentrates sunlight from a field of tracking mirrors onto a central receiver to produce the heat that powers the villain’s island. A field agent working against it needs answers to three concrete questions, and each of them turns out to be a well-posed computational problem.

How much heat can that array actually deliver? This is a heliostat yield question — aperture, reflectivity, and efficiency give the instantaneous figure, and the sun’s geometry over the day gives the energy figure. Both are standard solar engineering [Duffie and Beckman, 2013], and both are worth computing rather than guessing, because the answer decides whether the installation is a curiosity or a strategic asset.

How does an agent get out of the mirror-lined funhouse — and how would the enemy seal it? This is a graph question with two distinct answers. The route out is a shortest path; the rooms that *every* route must cross are the bottlenecks, the single points of failure the defenders will garrison. Escape redundancy is a flow question: the number of edge-disjoint routes is exactly the entry–exit maximum flow, and the fewest mirror walls that seal the lair is the matching minimum cut [Ford and Fulkerson, 1956b, Edmonds and Karp, 1972b].

Which contracts should the assassin — or the agent tracking him — expect to be taken? This is scheduling, and the answer depends sharply on the objective. Maximising the *number* of contracts is the earliest-finish greedy; maximising their total *bounty* is weighted interval scheduling [Kleinberg and Tardos, 2006], and on a well-chosen book the two disagree. A separate deadline book, where each contract occupies one slot and expires, is the unit-time deadline problem solved exactly by a matroid greedy [Cormen et al., 2009a].

This dossier presents the mission software for **SOLEX**: 6 pure, deterministic domain modules (`solar_concentration.py`, `solar_tracking.py`, `lair_topology.py`, `lair_flows.py`, `duel_scheduler.py`, `deadline_scheduler.py`) adapted to the frozen PROJECT BOND MissionProvider lifecycle — brief, recon, plan, execute, debrief [Friedman, 2026e] — through a single thin adapter. The domain modules import nothing outside the Python standard library and never import the protocol, so the scientific core stands alone; the adapter is the only place the fleet contract appears.

Determinism is a hard constraint rather than an aspiration: fixed inputs, no random draws, sorted iteration in every graph traversal, and no wall-clock read in any persisted artifact. Equally hard is the rule that no metric is typed by hand — every number in Results arrives as a placeholder resolved from the code that computed it, under a gate that fails the build when prose and code disagree.

12.4 Methodology — SOLEX: the analytical models and algorithms that drive the mission

12.4.1 Solar concentration yield

Each mirror M_i contributes optical power $P_i = I \cdot A_i \cdot \rho_i \cdot c$ where I is the direct normal irradiance, A_i the mirror area, ρ_i its reflectivity, and c an incidence factor (default 1). The field optical power is the sum; thermal output multiplies by an optical efficiency and a heat-transfer efficiency. The dimensionless concentration ratio at a receiver of area A_r is $C = \sum_i A_i \rho_i / A_r$. Given an area budget, the highest-yield allocation is greedy: since yield-per-area is exactly reflectivity, taking mirrors in descending reflectivity is optimal. The implementation takes whole mirrors — a mirror is skipped when it does not fit the remaining budget rather than being partially installed — so it is the integral restriction of the fractional-knapsack greedy, and the budget can end under-spent. Ties break on larger area first, which fixes the order deterministically.

The model is deliberately lumped: the chain is area $\times$ reflectivity $\times$ incidence $\times$ optical efficiency $\times$ heat-transfer efficiency, with no ray tracing, per-heliostat aim geometry, atmospheric attenuation, or receiver loss model. What it buys is a yield figure that is exactly reproducible and exactly attributable; what it costs is documented in Scope and Related Work.

12.4.2 Solar tracking & daily insolation

The daily resource follows the sun geometry. Solar declination δ and the equation of time use Spencer’s Fourier series [Spencer, 1971]. The daylight window is $D = (2/15) \arccos(-\tan \varphi \tan \delta)$ hours from the sunrise hour angle. For a fixed site and day the daily extraterrestrial irradiation on a horizontal plane is the Duffie–Beckman closed form

$$H_0 = \frac{24}{\pi} G_{sc} \left[1 + 0.033 \cos \frac{2\pi d}{365} \right] (\cos \varphi \cos \delta \sin \omega_s + \omega_s \sin \varphi \sin \delta),$$

with G_{sc} the solar constant and ω_s the sunrise hour angle in radians [Duffie and Beckman, 2013]; a clearness factor κ converts it to a clear-sky resource. The field’s daily thermal energy is $\kappa H_0 \sum_i A_i \rho_i \eta_{opt} \eta_{heat}$.

Elevation follows $\sin e = \sin \varphi \sin \delta + \cos \varphi \cos \delta \cos H$, clamped to $[-1, 1]$ before the arcsine so a below-horizon sun returns a non-positive elevation instead of raising. Horizontal beam flux is $I \sin e$, floored at zero for a sun below the horizon, which is what truncates the daily profile (the solex daily figure) to the daylight window rather than folding negative flux into the integral.

12.4.3 Lair escape topology

Rooms are nodes and mirror passages are undirected edges of a graph G . The shortest escape route is found by breadth-first search over sorted neighbour lists, so the route is not merely *a* shortest path but a fixed one. The canonical lair has 7 rooms and distances are counted in passages.

Two distinct notions of “critical room” are computed, and they are not interchangeable. A room is a *bottleneck* between a specific entry and exit if removing it disconnects those two — computed exactly, by deleting each candidate room in turn and re-testing reachability. A room is an *articulation point* (cut vertex) if removing it disconnects any part of the graph at all, computed in linear time by the Hopcroft–Tarjan depth-first low-link procedure [Hopcroft and Tarjan, 1973b]. Every entry–exit bottleneck is an articulation point, but the converse fails: a cut vertex hanging off a side branch splits the graph without touching the escape route. The exhaustive-removal bottleneck solver is $O(V(V + E))$, which is the right trade at lair scale and the wrong one at building scale.

12.4.4 Lair flow analysis

Escape *redundancy* is quantified by maximum flow on the lair treated as a unit-capacity network: each undirected mirror passage becomes a pair of opposed unit-capacity arcs. Edmonds–Karp augments along breadth-first paths [Edmonds and Karp, 1972b], and by Menger’s theorem the resulting flow value is exactly the number of **edge-disjoint** escape routes — the entry–exit edge connectivity. The residual graph’s source-side reachable set then recovers the **minimum cut**: the fewest mirror walls whose destruction seals the lair, with cut value equal to flow value by max-flow min-cut [Ford and Fulkerson, 1956b]. The flow is decomposed back into explicit routes by walking arcs that carry flow and decrementing one unit per route, so the reported routes are real paths rather than an aggregate number.

When mirror passages carry a traversal cost (mirror fragility, guard density), Dijkstra’s algorithm returns the cheapest escape route [Dijkstra, 1959b]. Costs are required for every passage — a missing cost raises rather than defaulting to zero, since a silently free passage would fabricate an escape route.

12.4.5 Duel / bounty scheduling

Bounty contracts are half-open intervals $[s, e)$ carrying a bounty w . Two overlap iff $s_a < e_b \wedge s_b < e_a$. Two objectives are solved, because they answer different questions and can disagree on the same book:

- **Maximum bounty** — *weighted interval scheduling* [Kleinberg and Tardos, 2006]: sort by end time, binary-search the latest non-overlapping predecessor $p(j)$, and run the dynamic program $D(j) = \max(w_j + D(p(j)), D(j - 1))$. Exact for arbitrary non-negative bounties.

- **Maximum count** — the earliest-finish greedy: repeatedly take the contract that finishes soonest among those still compatible. Exact for the count objective and, in general, wrong for the bounty objective.

Reporting only one of these would hide the trade-off the agent is actually making, so Results reports both.

12.4.6 Deadline bounty scheduling

Unit-time bounty contracts each occupy one slot and must finish by their deadline; the agent maximises accepted profit. The greedy that processes jobs in descending profit and assigns each to the latest free slot at or before its deadline is *exact*, not heuristic: the schedulable sets form a matroid, and the greedy is optimal over a matroid [Cormen et al., 2009a, sec. 16.5]. A path-compressed disjoint-set structure [Tarjan, 1975] answers the “latest free slot” query in near-constant amortised time by merging each occupied slot onto its left neighbour.

The reported profit and the reported slot assignment come from a single run of this greedy rather than two independent ones, so the two views cannot disagree — a divergence that would otherwise be invisible in the manuscript.

12.5 Results — SOLEX: measured outcomes, headline numbers, and what they establish

12.5.1 Solar concentration yield

Under a direct normal irradiance of 1000 W/m^2 , the canonical mirror field (total aperture 300.0 m^2) delivers 250.5 kW of optical power to the receiver aperture (5.0 m^2) and, after optical and heat-transfer losses, a thermal output of **191.6325 kW** at a concentration ratio of **50.10×**.

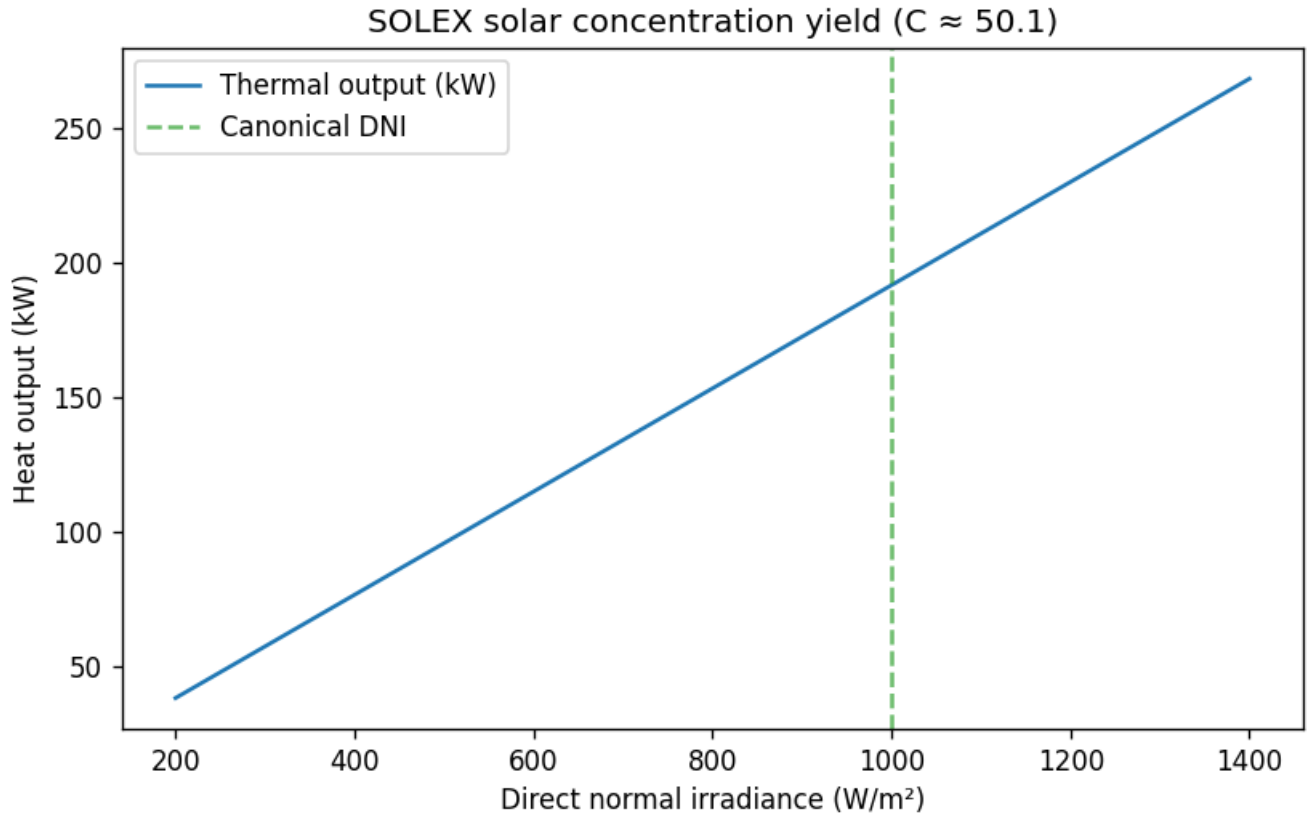


Figure 40: Thermal heat output versus direct normal irradiance for the canonical SOLEX field.

Offered the same 5 mirrors under an area budget of 200 m^2 , the greedy allocator installs **alpha, bravo, echo**. The selection is instructive: after taking the two highest-reflectivity mirrors the remaining budget is too small for the next two candidates, so the allocator falls through to the smallest, lowest-quality mirror rather than leaving the budget idle. Whole mirrors cannot be split, so the budget finishes under-spent — the integral restriction of the fractional optimum, not a failure of the greedy.

12.5.2 Solar tracking & daily insolation

At the canonical site (8.0°N , day 172) the declination is 23.45° and daylight lasts 12.47 hours. The sun reaches 74.55° at solar noon. The daily extraterrestrial insolation on a horizontal plane is 10.03 kWh/m^2 ; with a clearness factor this falls to a clear-sky resource of 7.52 kWh/m^2 , which the field concentrates into **1441.7 kWh/day** of thermal energy.

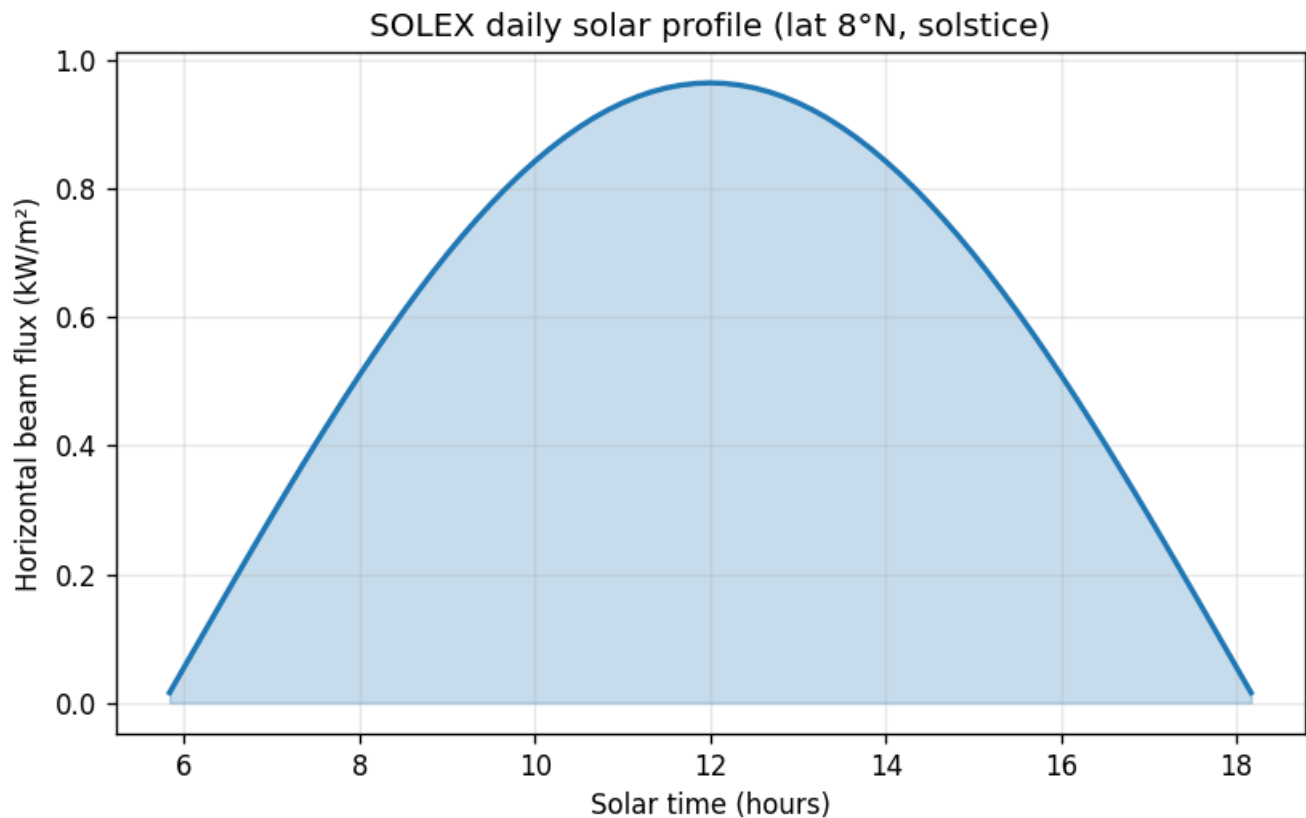


Figure 41: Clear-sky horizontal beam flux across the canonical SOLEX day.

12.5.2.1 Sensitivity and uncertainty of the daily yield The headline daily figure (1441.7 kWh) is a single point on two continua, and reading it as a point obscures how much of the number is the model’s two free inputs — the clearness factor and the site latitude:

Clearness κ	Clear-sky insolation (kWh/m ² /day)	Field daily energy (kWh/day)
0.60	6.02	1153.4
0.75	7.52	1441.7
0.90	9.03	1730.1

The clearness factor is the single largest source of uncertainty in the daily number: it is a bulk sunshine-availability parameter, not a measured quantity for any particular island day, and the deliberately unmodelled physics (real circumsolar attenuation, receiver thermal loss, mirror soiling) all live behind it. That is why the mission *reports* the clear-sky number only at the canonical clearness factor, never as a forecast.

Holding clearness fixed and moving the site latitude shifts the overhead sun:

Latitude (deg N)	Field daily energy (kWh/day)
0	1326.4
8	1441.7
23.44	1598.1

At the June solstice the sun stands directly over the Tropic of Cancer (23.44°N), so moving the field north from the equator toward that latitude raises the solstice noon elevation and, with it, the concentrated daily energy; the maximum over this band is the 23.44°N row, where the sun is overhead at noon. The canonical island (8.0°N) is south of that maximum and therefore delivers less than its northernmost placement — a site-sensitivity the headline single number cannot show. Both sweeps are live engine output — the same `field_daily_energy_kwh` call the Results section uses, evaluated over a parameter grid — so they cannot drift from the model that produced the headline number.

12.5.3 Lair escape topology

The funhouse lair comprises 7 rooms. The shortest escape route from the entry to the vault is:

Entrance -> Hall -> Mirror_Room -> Gallery -> Vault

covering 4 passages. The 3 rooms that every escape route must pass through are **Gallery**, **Hall**, **Mirror_Room**; the lair has 4 articulation points (single rooms whose removal disconnects part of the funhouse).

The two counts differ, and the difference is the point. Articulation points include rooms on the dead-end side branch, whose loss splits the funhouse without touching the route to the vault; the bottleneck set is strictly the rooms that stand between the entry and the exit. A defender garrisoning articulation points would spend part of the effort guarding rooms the escaping agent never enters.

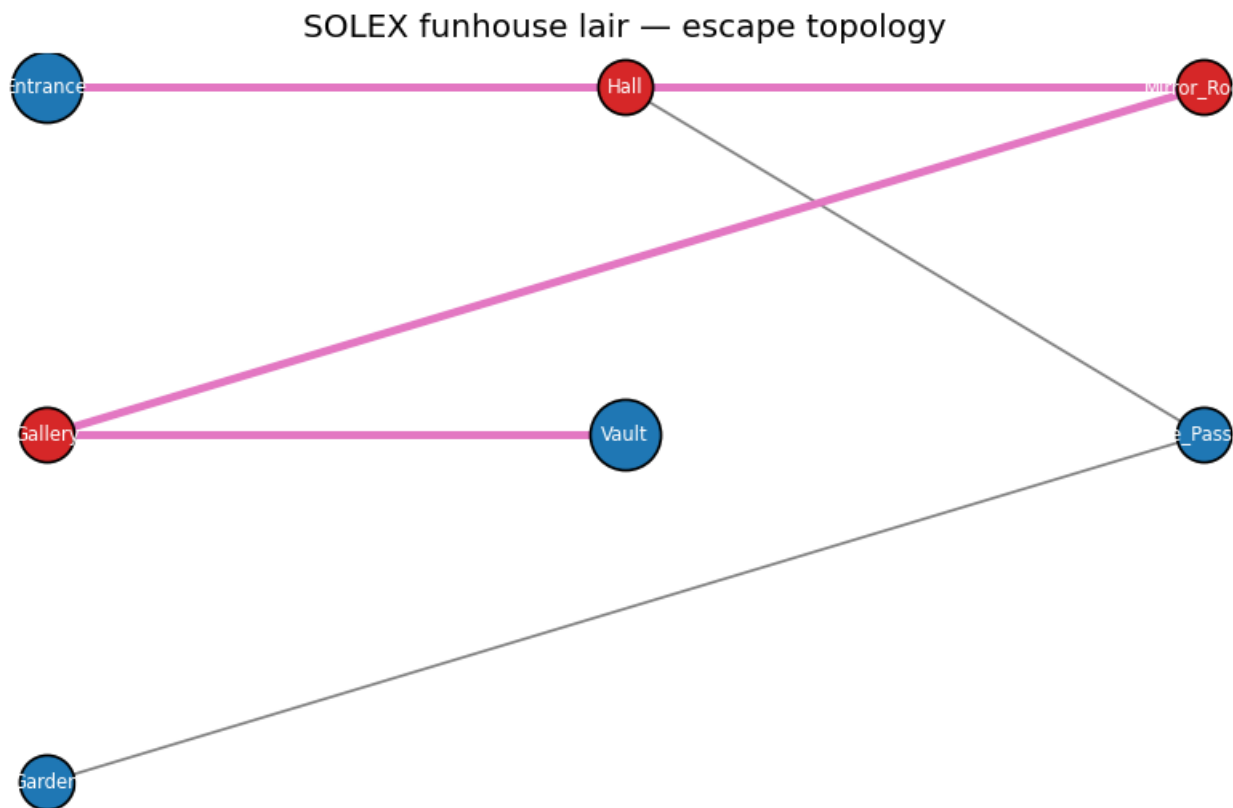


Figure 42: SOLEX funhouse lair graph with the escape route and bottleneck rooms highlighted.

12.5.4 Lair flow analysis

The extended funhouse (10 rooms) offers **3 edge-disjoint escape routes**; destroying every passage of any single route still leaves the others open. To seal the lair the enemy must cut **3 mirror walls** (the minimum entry–exit cut), namely **Courtyard-Entry**, **Entry-Hall**, **Entry-Tunnel**. Cut value equals flow value, as max-flow min-cut requires; the cheapest seal is at the entry side, where the three corridors have not yet diverged. Under unit traversal costs the cheapest escape route is

Entry -> Tunnel -> Armory -> Vault

at cost 3.

12.5.5 Duel / bounty scheduling

The appointment book holds 4 bounty contracts and contains 1 overlapping pair(s). Weighted interval scheduling selects the non-overlapping set **B**, **C**, **D**, realising a maximum total bounty of **13.0**.

The count-maximising earliest-finish greedy instead selects **A**, **C**, **D**: 3 contracts worth 11.0, against the dynamic program's 3 contracts worth 13.0. Both schedules are optimal for their own objective, and the disagreement is exactly the trade-off the agent faces: booking the earliest-finishing contract first forfeits the more valuable contract that overlaps it. Choosing the greedy because it is simpler would cost bounty here, which is why the exact dynamic program is the solver wired into the mission outcome.

12.5.6 Deadline bounty scheduling

The deadline book holds 5 unit-time bounty contracts. The deadline greedy (union-find) accepts **E**, **C**, **B**, realising a maximum profit of **18.0** with slot assignment **1:B**, **2:C**, **3:E**.

All six solvers run through the frozen **MissionProvider** lifecycle; the outcome carries full provenance including a deterministic input hash.

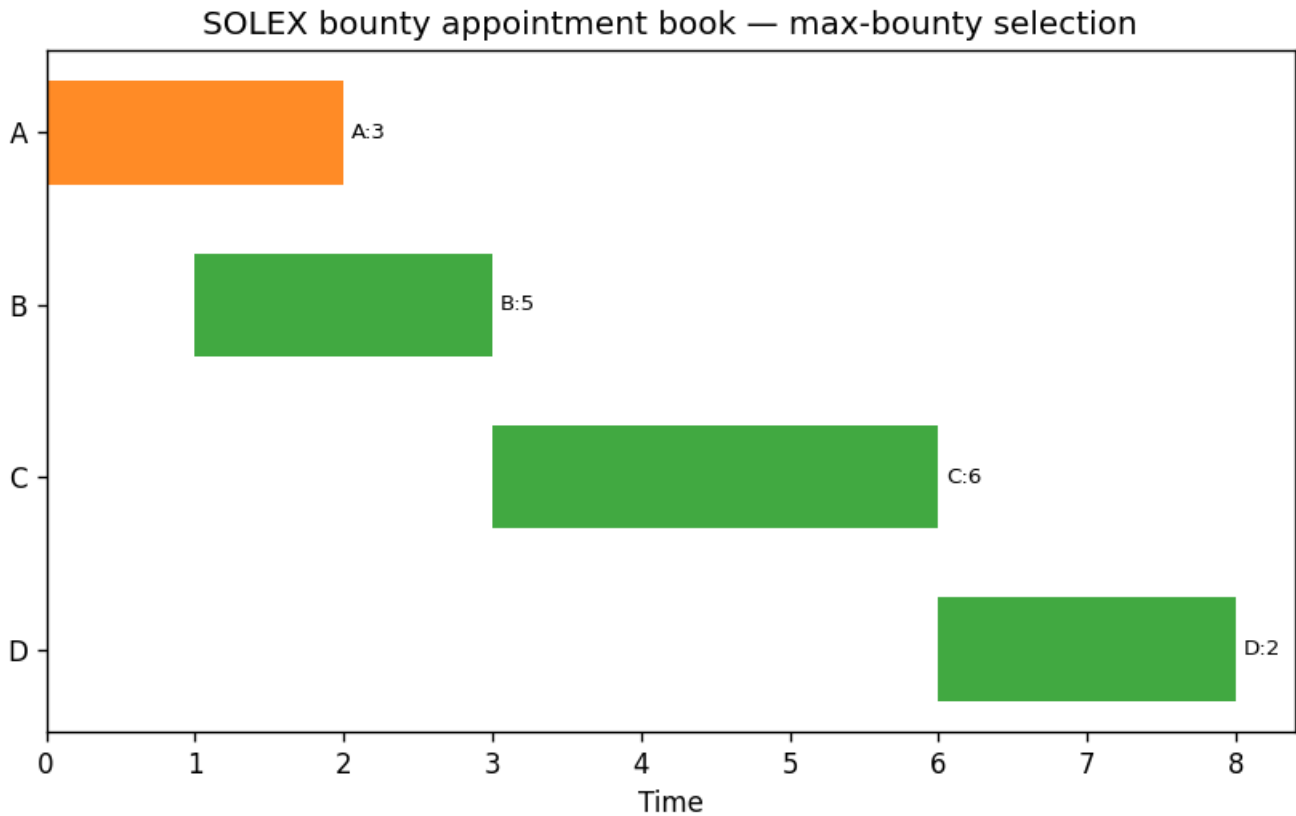


Figure 43: SOLEX bounty appointment book with the maximum-bounty selection filled.

12.6 Conclusion — SOLEX: findings, verdict, and what the mission establishes

The **SOLEX** mission software is a complete, gate-verified Project BOND film package: 6 pure, deterministic domain modules spanning solar concentration yield, solar tracking and daily insolation, funhouse lair escape topology, lair flow analysis (max-flow, min-cut, and cost-aware escape), and two bounty-scheduling models — weighted interval scheduling and unit-time deadline scheduling — all adapted to the frozen `MissionProvider` protocol through a single thin adapter.

Three findings are worth carrying out of the dossier. First, the mirror field’s thermal output tracks *effective* aperture rather than raw area: reflectivity, not square metres, is the quantity the greedy allocator spends its budget on, and the same asymmetry governs the daily energy figure. Second, escape redundancy and escape criticality are different measurements on the same lair — the 3 edge-disjoint routes of the extended funhouse mean no single wall is decisive, while the canonical lair’s 3 bottleneck rooms are each individually decisive, and the articulation-point count is a third number answering a third question. Third, the two duel solvers disagree on the same book: optimising contract count and optimising bounty select different schedules, so naming the objective matters more than picking a clever algorithm.

The methodological claim is narrower than the domain claim and matters more. Every numeric statement above arrives from `manuscript_variables.py::generate_variables`; none is typed by hand. The config’s restatement of the canonical inputs is recomputed and raises on disagreement, the token gate fails on any metric prose cites but code does not produce, and the coverage gate is set at 90% line-and-branch on `src/` with a zero-mock suite. Determinism is likewise enforced rather than asserted: the 4 figures are re-rendered and byte-compared in the test suite. The package is a local-only working tree and publishes no remote, so reproduction means re-running the pipeline in place — see Reproducibility.

12.7 Experimental Setup — SOLEX: canonical scenarios, parameters, and configuration

This section fixes every input the results depend on. Nothing in the SOLEX package is sampled, fitted, or drawn from an external service: each model is evaluated on a canonical fixture defined in code, and the fixture is restated in `manuscript/config.yaml` where it is verified against the code at token generation time (see Reproducibility).

12.7.1 Software environment

Item	Value
Package	<code>the_man_with_the_golden_gun</code> v0.1.0
Mission codename	SOLEX

Item	Value
Python	3.14.6
Domain modules	6 (solar_concentration.py, solar_tracking.py, lair_topology.py, lair_flows.py, duel_scheduler.py, deadline_scheduler.py)
Third-party runtime dependencies	numpy, matplotlib, pyyaml, bond-api
Deterministic seed	3
Figures	4

numpy is used only to build the irradiance sweep grid for the solex yield figure; matplotlib only to render figures; pyyaml only to read config.yaml. Every quantitative result in Results is computed with the Python standard library.

12.7.2 Heliostat field

The canonical SOLEX field is 5 tracking mirrors totalling 300.0 m² of aperture, each with a fixed area and specular reflectivity:

Mirror	Area (m ²)	Reflectivity
alpha	100.0	0.90
bravo	80.0	0.85
charlie	60.0	0.80
delta	50.0	0.75
echo	10.0	0.70

The receiver aperture is 5.0 m². Optical efficiency (receiver intercept) is 0.90 and heat-transfer efficiency is 0.85; both are module defaults and are swept by no experiment here. The direct normal irradiance used for the headline yield is 1000 W/m².

The mirror-budget allocation experiment offers the same 5-mirror candidate set under an area budget of 200 m² — deliberately smaller than the field, so the greedy allocator must actually choose.

12.7.3 Solar site and day

Parameter	Value
Latitude	8.0°N
Day of year	172 (northern solstice)
Solar constant	1361 W/m ²
Clearness factor	0.75
Peak DNI	1000 W/m ²

The site stands in for the villain’s island: a low northern latitude where the solstice sun passes close to the zenith. The daily beam-flux profile (the solex daily figure) is sampled at 10-minute steps in solar time and truncated to elevations above the horizon.

12.7.4 Funhouse lair fixtures

Two graph fixtures are used, and they answer different questions.

The **canonical lair** (7 rooms) is a mostly linear funhouse with one dead-end branch. It is the fixture for escape routing and bottleneck analysis, because a chain of forced rooms is exactly what makes the bottleneck question interesting.

The **extended lair** (10 rooms) adds two parallel corridors from the entry to the vault plus a dead-end wine cellar. It is the fixture for flow analysis, because a lair with only one route has a trivial max flow; three parallel routes give the Edmonds–Karp decomposition and the minimum cut something to separate. Every passage carries unit capacity (one mirror wall) and, for the cost-aware escape, unit traversal cost.

12.7.5 Bounty contract books

The **duel book** holds 4 contracts as half-open intervals **start**, **end**) with a bounty:

Contract	Interval	Bounty	Target
A	[0, 2)	3	First_Heel
B	[1, 3)	5	Second_Heel

Contract	Interval	Bounty	Target
C	[3, 6)	6	Solex_Guard
D	[6, 8)	2	Night_Watch

The book is constructed so the greedy count-maximising answer and the bounty-maximising answer differ — that divergence is the point of running both solvers in [Results].

The **deadline book** holds 5 unit-time contracts, each with an integer deadline slot and a profit:

Contract	Deadline	Profit
A	1	3
B	2	5
C	2	6
D	3	2
E	3	7

Total demand (5 contracts) exceeds the 3 available slots, so the scheduler must reject work; a book that fit entirely would make the greedy vacuous.

12.8 Reproducibility — SOLEX: verification gates, deterministic regeneration, and artifacts

Every number in this dossier is regenerable from a clean checkout by running the commands below. There is no hidden state, no network call, no wall-clock read in a persisted artifact, and no random draw.

12.8.1 Regeneration procedure

```
uv sync --extra dev # resolve deps + bond-api path dep
uv run python scripts/00_preflight.py # protocol discoverability
uv run python scripts/generate_artifacts.py # outcome JSON + all figures
uv run python scripts/z_generate_manuscript_variables.py # token map + resolved sections
uv run pytest tests/ --cov=src --cov-fail-under=90
```

scripts/generate_artifacts.py writes output/data/solex_outcome.json and the 4 figures under ../figures/. scripts/z_generate_manuscript_variables.py writes output/data/manuscript_variables.json and the fully resolved manuscript tree under output/manuscript/, and exits non-zero if any placeholder survives substitution.

12.8.2 Sources of determinism

The package has three potential nondeterminism surfaces, and each is closed:

1. **Randomness.** There is none. The declared seed (3) is recorded in the mission provenance for protocol conformance, not consumed by a generator — no module imports `random` or `numpy.random`. The correct reading of the seed is “this mission asserts it needs no entropy”.
2. **Iteration order.** Graph algorithms iterate neighbours through `sorted` rather than set order, so the BFS escape route, the flow decomposition, and the minimum cut are stable across runs and interpreter hash seeds. Schedulers break ties on `(end, id)` and `(-profit, id)` respectively.
3. **Wall clock.** Persisted artifacts carry the fixed `DEFAULT_TIMESTAMP` (2026-08-04T00:00:00Z), and the mission outcome records `wall_time_s = 0.0` rather than a measured duration, so two runs produce byte-identical files. Figure PNGs are likewise byte-identical between runs; `tests/test_figures.py` asserts this by rendering each figure twice into separate directories and comparing bytes.

12.8.3 Metric injection

No metric in this manuscript is typed by hand. Each is a double-brace placeholder resolved by `src/the_man_with_the_golden_gun/manuscript_variables.py::generate_variables` from `manuscript/config.yaml` plus a live call into the domain models. Four gates hold that discipline in place:

- `tests/test_manuscript_variables.py::test_all_manuscript_tokens_are_generated` scans every file the hydrator writes — the numbered sections and the Pandoc preamble, both drawn from `manuscript_variables.manuscript_sources` — and fails if one cites a token the generator does not produce.
- `scripts/z_generate_manuscript_variables.py` fails if a resolved file still contains an unsubstituted token.

- `manuscript_variables.verify_declared_experiment` recomputes every value the config’s `experiment` block declares — the codename, the irradiance, the receiver aperture, the solar-site geometry, the lair entry and exit, the maximum duel bounty, the extended lair, and the whole deadline book — and raises if the config and the code disagree. It also raises on a declared key it has no recomputation for, so the checked surface cannot quietly shrink below the block. A stale config therefore fails the build rather than narrating a wrong number.
- `declarations.verify_claim_ledger` recomputes every value in `data/claim_ledger.yaml` at its recorded precision and checks that each claim’s cited evidence symbol and test file exist. It runs during token generation because the config declares the ledger path, so the ledger is a checked restatement of the code rather than a parallel copy of the numbers.

12.8.4 Test and gate discipline

The suite is zero-mock: no `unittest.mock`, no `MagicMock`, no `mock.patch` anywhere in `tests/`. Assertions are made against genuinely computed values (known vectors derived from the algorithms’ definitions), real rendered PNGs, and real subprocess executions of the `scripts/` entry points. Line and branch coverage on `src/` is gated at 90% by `pytest --cov-fail-under; ruff check, ruff format --check`, and `mypy` must be clean on `src/` and `scripts/`. The rationale for each of these choices is recorded in `docs/testing_philosophy.md`.

12.8.5 What is not reproducible here

This is a **local-only** working tree with no git remote and no published archive. There is no DOI, no Zenodo deposit, and no public URL: the metadata sidecars (`.zenodo.json`, `CITATION.cff`, `codemeta.json`) deliberately omit DOI fields rather than carrying a placeholder. Reproduction therefore means re-running the commands above inside this checkout, not fetching a release.

12.9 Scope and Related Work — SOLEX: boundaries, positioning, and relationship to the literature

12.9.1 What this package is

SOLEX is one film package in the PROJECT BOND fleet: a standalone, deterministic implementation of the technical ideas that *The Man with the Golden Gun* (1974) actually turns on, exposed through the fleet’s frozen `MissionProvider` protocol [Friedman, 2026e]. The film supplies the specification — a solar concentrator whose output is worth killing for, a mirror-walled funhouse an agent has to get out of, and an assassin who books contracts one at a time — and the 6 domain modules supply the implementation.

The design bet is that a film’s premise is a legitimate specification. “How much heat does a mirror field actually deliver” is a solar-engineering question with a textbook answer; “which rooms must every escape route pass through” is a graph question with an exact answer; “which contracts should the assassin accept” is a scheduling question with two different exact answers depending on what is being maximised. Building each one honestly is more interesting than building a stub with a film title on it.

12.9.2 What this package is not

It is not a heliostat design tool. The optical model is a lumped area $\times$ reflectivity $\times$ efficiency chain: there is no ray tracing, no cosine loss from individual heliostat aim points, no atmospheric attenuation between mirror and tower, no receiver thermal-loss model, and no spillage. The concentration ratio reported is a geometric flux ratio at the aperture, not a peak flux map. A real central-receiver study would need all of these [Duffie and Beckman, 2013].

It is not a solar-position library. The Spencer Fourier series [Spencer, 1971] is accurate to roughly a tenth of a degree in declination, which is ample for a daily-energy figure and inadequate for tracker control. The standard upgrade is the Astronomical Almanac algorithm [Michalsky, 1988]; it is deliberately not implemented here because nothing in this dossier is sensitive at that resolution, and implementing an unused higher-accuracy path would be scaffolding rather than substance.

It is not a facility-security product. The lair graphs are hand-authored fixtures of 7 and 10 rooms. The bottleneck solver is *exact by exhaustive removal* — it deletes each room in turn and re-tests reachability, which is $O(V \cdot (V+E))$ and entirely appropriate at this size, but it is not the algorithm to reach for on a large building. The linear-time route is the biconnected-components decomposition [Hopcroft and Tarjan, 1973b], which this package also implements (`articulation_points`) but uses for a different question: cut vertices of the whole graph rather than vertices separating a specific entry–exit pair. The two answers differ, and conflating them would be the easy mistake here.

It is not a workforce scheduler. Both scheduling models assume perfect information about a small, fixed contract book: no arrivals over time, no uncertainty in duration, no cancellation, no travel time between targets.

12.9.3 Relationship to the established literature

Nothing in the domain core is novel; that is the point. Each module is a faithful implementation of a documented result, which is what makes the outputs checkable against textbook values rather than against themselves:

Module	Result implemented	Source
solar_concentration	Lumped optical/thermal chain; fractional-knapsack greedy	[Duffie and Beckman, 2013]
solar_tracking	Spencer declination and equation of time; Duffie–Beckman daily insolation closed form	[Spencer, 1971, Duffie and Beckman, 2013]
lair_topology	BFS shortest path; Hopcroft–Tarjan cut vertices	[Cormen et al., 2009a, Hopcroft and Tarjan, 1973b]
lair_flows	Edmonds–Karp max flow; max-flow min-cut; Dijkstra	[Edmonds and Karp, 1972b, Ford and Fulkerson, 1956b, Dijkstra, 1959b]
duel_scheduler	Weighted interval scheduling DP; earliest-finish greedy	[Kleinberg and Tardos, 2006]
deadline_scheduler	Unit-time scheduling with deadlines (matroid greedy) with union–find slot lookup	[Cormen et al., 2009a, Tarjan, 1975]

The contribution, such as it is, lies in the *composition*: six textbook results wired into a single deterministic mission lifecycle whose every reported number is injected from the code that produced it, under gates that fail rather than warn.

12.9.4 Position in the fleet

SOLEX depends on exactly one fleet package, `bond-api`, and only through a single adapter module. The 6 domain modules import nothing outside the standard library, so the scientific core survives independently of the protocol. Two integration points remain open and are tracked in `TODO.md`: persisting executed outcomes to a provenance store once `bond-utilities` ships, and cross-package missions once the coordinator lands. Neither requires a change to the domain core.

12.10 Appendix: Generated metric register

This appendix records the resolved values of every manuscript metric at generation time (2026-08-04T00:00:00Z). The live inventory is produced by `scripts/z_generate_manuscript_variables.py` into `output/data/manuscript_variables.json`; the tables below list the deterministic canonical outputs of the 6 domain models. Nothing here is transcribed by hand — every cell is the same placeholder mechanism used in the body sections, so the register cannot drift from the prose that cites it.

12.10.1 Solar concentration

Metric	Value
Direct normal irradiance	1000 W/m ²
Field aperture	300.0 m ²
Receiver aperture	5.0 m ²
Optical power	250.5 kW
Thermal heat output	191.6325 kW
Concentration ratio	50.10×
Mirrors in field	5
Allocation budget	200 m ²
Allocation selection	alpha, bravo, echo

12.10.2 Solar tracking & daily insolation

Metric	Value
Latitude	8.0°N
Day of year	172
Declination	23.45°
Noon elevation	74.55°
Daylight	12.47 hours
Extraterrestrial insolation	10.03 kWh/m ²
Clear-sky insolation	7.52 kWh/m ²
Field daily energy	1441.7 kWh

Sensitivity — clearness factor (kWh/m²/day, kWh/day):

Clearness	Clear-sky insolation (kWh/m ² /day)	Field daily energy (kWh/day)
0.60	6.02	1153.4
0.75	7.52	1441.7
0.90	9.03	1730.1

Sensitivity — site latitude (deg N, kWh/day):

Latitude (deg N)	Field daily energy (kWh/day)
0	1326.4
8	1441.7
23.44	1598.1

12.10.3 Lair topology (canonical)

Metric	Value
Rooms	7
Escape distance	4 passages
Escape route	Entrance -> Hall -> Mirror_Room -> Gallery -> Vault
Bottleneck rooms	Gallery, Hall, Mirror_Room
Articulation points	4

12.10.4 Lair flow (extended)

Metric	Value
Rooms	10
Edge connectivity	3
Min-cut wall count	3
Min-cut walls	Courtyard-Entry, Entry-Hall, Entry-Tunnel
Cheapest escape cost	3
Cheapest escape route	Entry -> Tunnel -> Armory -> Vault

12.10.5 Duel interval scheduling

Metric	Value
Contracts	4
Conflicts	1
Max-bounty selection	B, C, D (3)
Maximum bounty	13.0
Earliest-finish selection	A, C, D (3)
Earliest-finish bounty	11.0

12.10.6 Deadline scheduling

Metric	Value
Jobs	5
Selected	E, C, B
Maximum profit	18.0
Slot assignment	1:B, 2:C, 3:E

12.10.7 Environment and gates

Metric	Value
Package version	0.1.0
Mission seed	3
Domain modules	6
Figures	4
Coverage gate	90% line + branch on src/
Python	3.14.6
Timestamp	2026-08-04T00:00:00Z

12.11 Sources — SOLEX: bibliography

Friedman [2026e]; Hamilton [1974]; Duffie and Beckman [2013]; Spencer [1971]; Michalsky [1988]; Cormen et al. [2009a]; Ford and Fulkerson [1956b]; Edmonds and Karp [1972b]; Dijkstra [1959b]; Hopcroft and Tarjan [1973b]; Kleinberg and Tardos [2006]; Tarjan [1975]

13 The Spy Who Loved Me (1977) — LIPARUS

film package · *package codename* **LIPARUS**. Mission **LIPARUS**: detect, localise and track a passive-acoustic contact; simulate a Liparus-Class tanker capture, verify submersible conversion and hull viability.

13.1 Concepts — LIPARUS: domain and operational focus

submarine tracking, marine forensics

13.2 Abstract — LIPARUS: mission summary

The Spy Who Loved Me: LIPARUS — Special-Agent Mission Software is a deterministic special-agent mission model built around the concepts of *The Spy Who Loved Me* (1977) under mission codename LIPARUS. It comprises 6 standalone, infrastructure-free domain modules — passive acoustic detection and bearing-only tracking, Liparus-class tanker capture, submersible vehicle conversion forensics, underwater acoustics and the passive sonar equation, uniform-linear-array beamforming, and pressure-hull collapse/ballast analysis — exposed to the BOND fleet through a frozen `MissionProvider` adapter. Every *measured* quantity in this manuscript is computed live by the domain core rather than hand-authored, and The results section marks the one reported number that is a declared constant instead — the capture timeline. The passive detector declares the contact at a controlled false-alarm probability of 0.001 (margin 11.0 dB), and the two tracking estimators agree on the geometry: batch target-motion analysis recovers the quarry’s position to within 65.4 metres of truth while the recursive extended Kalman filter settles to a steady-state position error of 804.5 metres over the same run. The ULA beamformer resolves the contact to within one 0.5-degree scan cell of the true bearing (an error of 0.0 degrees) and, with the opt-in parabolic refinement of the peak, to within 0.112 degrees against a Cramer-Rao floor of 0.33 degrees (The results section explains why the measured grid error is grid-limited rather than CRB-limited). The passive sonar equation yields a detection range of 83746 metres, and the Esprit-grade pressure hull survives to a collapse depth of 85 metres (a safety factor of 2.13 at its operating depth of 40 metres) while requiring 244 L of ballast to reach neutral buoyancy. All simulations are seeded and fully deterministic, and the test suite holds line and branch coverage on `src/` at or above 90%.

13.3 Introduction — LIPARUS: mission framing, the operational problem, and how to read this chapter

In *The Spy Who Loved Me* (1977), Bond confronts two icons of hydraulic malevolence and mechanical possibility: the *Liparus*, a supertanker whose hull opens like a mouth to swallow whole submarines, and the Lotus Esprit, a road car that converts into a functional submersible. Both demand the same kind of engineering reasoning that a special-agent mission plan needs: detect the unseen contact, reason about capture geometry, and verify that a vehicle can actually survive underwater.

This package — mission codename LIPARUS — realises those capabilities as deterministic, testable software across 6 concept modules. It is one package in the PROJECT BOND suite, a set of film-themed mission packages made compatible by the frozen `bond_api` mission protocol. The domain core is deliberately independent of that protocol: the pure modules in `src/the_spy_who_loved_me/` compute results with no knowledge of any orchestrator, and the thin `mission.py` adapter in the same package exposes them under the standard five-stage lifecycle (brief, recon, plan, execute, debrief).

The structure of the package is:

Module	Concept
<code>sub_tracking.py</code>	Passive acoustic detection and bearing-only tracking
<code>tanker_capture.py</code>	Liparus-class tanker swallowing-submarine simulation
<code>vehicle_forensics.py</code>	Submersible vehicle conversion forensics
<code>acoustics.py</code>	Underwater propagation and the passive sonar equation
<code>beamforming.py</code>	Uniform-linear-array beamforming and bearing estimation
<code>hull_forensics.py</code>	Pressure-hull collapse and ballast analysis

Three further modules carry no domain concept of their own but hold the package together: `scenario.py` is the single canonical parameter set and metric factory that both the mission adapter and the manuscript hydration consume (so an outcome and a printed number cannot disagree), `manuscript_variables.py` turns those metrics into the substitution tokens this document is written in, and `mission.py` is the thin frozen `bond_api.MissionProvider` adapter plus gadget registry.

The six concepts compose rather than sit side by side: `acoustics.py` sets the detection budget that says whether a contact is audible at all, `sub_tracking.py` decides whether it *was* heard and where it is going, `beamforming.py` converts the array’s raw snapshot into the bearing that tracking consumes, `tanker_capture.py` asks whether the quarry physically fits inside the Liparus, and `vehicle_forensics.py` and `hull_forensics.py` together answer the Lotus Esprit question from two sides — paperwork (requirement traceability) and physics (collapse depth and ballast).

The remainder of this manuscript describes the algorithms (The methodology section), their measured results (The results section), and the deterministic experimental setup (The setup section).

13.4 Methodology — LIPARUS: the analytical models and algorithms that drive the mission

13.4.1 Passive acoustic detection and bearing-only tracking

`src/the_spy_who_loved_me/sub_tracking.py` models the sensor problem of a passive array hunting a submarine that emits nothing deliberately.

Detection uses a Neyman–Pearson energy detector (`detect_contract`). A received waveform is split into non-overlapping windows; each window statistic $T = \text{sum}(x^2)/\text{noise_var}$ is compared against a threshold. Under the noise-only hypothesis T about $\chi^2(\text{window})$, so the threshold is the $(1 - P_{\text{FA}})$ -quantile of that chi-square distribution, computed in closed form with the Wilson–Hilferty approximation (`chi2_quantile`), whose standard-normal-quantile primitive is Acklam’s algorithm (`inverse_normal_cdf`). The detector therefore makes its decision at a controlled false-alarm probability P_{FA} , and `Detection.margin_db` reports the statistic-to-threshold margin.

Tracking is the harder problem. `simulate_target_track` generates a deterministic scenario: a constant-velocity submarine observed by a *maneuvering* ownship (a dogleg course change), yielding noisy relative bearings. The maneuver is not cosmetic — a fundamental result of passive sonar is that a constant-velocity target’s absolute range and speed are *unobservable* from a stationary observer (the whole trajectory can be scaled and produce identical bearings). Only a change of observer course renders the state observable.

`estimate_tma` then performs batch least-squares target motion analysis: over the state $[p_x, p_y, v_x, v_y]$ it minimises the sum of squared bearing residuals $\text{sum}_i (\text{bearing}_i - \text{atan2}(p_y + v_y t_i - o_y, p_x + v_x t_i - o_x))^2$, initialised by a coarse grid search over range/course/speed and refined by Gauss–Newton.

That grid search is **bounded, and its bounds bracket the canonical scenario**: candidate initial ranges run 3000–6500 m in 500 m steps and candidate speeds 5–35 m/s in 5 m/s steps, against a canonical truth of 4000 m and 25 m/s. This is a declared search envelope, not a property of the estimator, and canonical accuracy is therefore not evidence about arbitrary geometry. The degradation outside it is graceful rather than a cliff — with only the initial range varied, a 15 km target is still recovered to 293 m because Gauss–Newton escapes the poor seed, while a 60 km target leaves a 25 km error (42% of range) that widening the range bound to 80 km cuts to 4.9 km. `estimate_tma` accepts keyword overrides for the envelope, and `tests/test_sub_tracking.py::TestStCoarseSearchEnvelope` pins all three of those measurements.

`PassiveBearingTracker` is the matching recursive Extended Kalman Filter for online use, with the relative-bearing measurement model and a constant-velocity transition; its `measurement_noise_rad` is a bearing standard deviation and is squared into the measurement covariance R . `tracking_rms_error` reports steady-state accuracy.

13.4.2 Liparus-class tanker capture

`src/the_spy_who_loved_me/tanker_capture.py` models the *Liparus* swallowing a submarine as a deterministic geometric-feasibility check plus a discrete-event sequence. `clearance_metrics` computes the per-axis margins between a `SubmarineSpec` and a `BayConfig` mouth/hold, and `classify_clearance` labels the fit `fits` / `tight` / `denied` against a required safety margin. `stowage_capacity` is how many submarines the hold can stow. `simulate_capture` runs the four-phase operation (approach, ingress, hatch-seal, secure) and returns a `CaptureRun` with a per-phase timeline and final status.

Only two of those four phases test anything. *Ingress* requires the mouth to clear the submarine’s beam and depth by the safety margin, and *secure* requires the hold to clear its length by the same margin; *approach* and *hatch-seal* always succeed. No seal-integrity quantity is modelled anywhere in the package, so the hatch-seal stage has nothing of its own to check — a `seal_requirement` field on `BayConfig` supplied the appearance of one until it was removed, because the constructor validated it into $[0, 1]$ and the predicate then compared it against 1.0, a conjunct that could never be false. The per-phase durations are likewise declared constants, not a timing model; The results section says what that means for the reported elapsed time.

13.4.3 Submersible vehicle forensics

`src/the_spy_who_loved_me/vehicle_forensics.py` forensically assesses whether a road vehicle can be converted to submersible duty. `SUBMERSIBLE_REQUIREMENTS` is a traced requirement set (pressure hull, watertight seals, ballast, air supply, and optional systems). `assess_conversion` scores the spec, `classify_conversion` maps the result to `road-only`, `amphibious`, or `submersible`, and `recommend_upgrades` returns the missing critical systems as an ordered retrofit list. `buoyancy_status` applies Archimedes’ principle ($F_b = \text{displacement} * \text{rho_water} * g$ vs weight) to classify the as-spec vehicle `floats` / `neutral` / `sinks`.

13.4.4 Underwater acoustics and the passive sonar equation

`src/the_spy_who_loved_me/acoustics.py` puts numbers on the *medium* the Liparus and its quarry move through. `thorp_absorption` implements the Thorp (1967) seawater absorption coefficient, `transmission_loss` combines geometric spreading with absorption to give $TL(r) = n * 10 * \log_{10}(r) + \alpha * r$, and `sound_speed_m_s` implements the Mackenzie (1981) equation. The passive sonar equation in signal-excess form $SE = SL - TL - (NL - DI) - DT$ (`passive_sonar_equation`) is the detection budget, and `detection_range_m` solves for the maximum detectable range by monotone bisection of the strictly-increasing transmission loss — the range at which the signal excess reaches zero.

Signal excess is not the signal-to-noise ratio, and the two are kept apart here because this package previously called the quantity above an SNR in code, prose and test names. Following Urick [Urick, 1983d], the array-output SNR is $SL - TL - (NL - DI)$ — larger than the signal excess by exactly the detection threshold DT , and available as `signal_to_noise_ratio_db`. With the canonical budget the difference is 10 dB, so calling the figure of merit an SNR overstates the margin by that much.

13.4.5 Uniform-linear-array beamforming

`src/the_spy_who_loved_me/beamforming.py` turns the array’s raw snapshots into a bearing. `steering_vector` gives the plane-wave phase progression $a(\theta) = \exp(-j 2 \pi (d/\lambda) k \sin(\theta))$; `array_snapshot` produces a deterministic seeded snapshot; the conventional (delay-and-sum) beamformer $P(\theta) = |a(\theta)|^2 (beam_power)$ is scanned by `estimate_bearing_deg` to locate the contact. `cramer_rao_bearing_std_deg` reports the Cramer-Rao lower bound on the bearing estimate — the theoretical floor the estimator cannot beat (Van Trees, *Optimum Array Processing* [Van Trees, 2002]).

Two properties of this estimator matter for reading The results section. First, the peak search runs over a *discrete* grid, so the estimate is quantised to the grid step and its measured error is floored at half a step regardless of array quality — the statistical bound reported by the CRB and the error achievable on this grid are different quantities, and on this scenario the latter is the binding one. Second, the element SNR passed to the CRB is derived in `scenario.py` from the snapshot amplitude and noise standard deviation rather than declared independently, so the bound always describes the array that was in fact sampled.

13.4.6 Pressure-hull and ballast forensics

`src/the_spy_who_loved_me/hull_forensics.py` checks that the converted vehicle can actually dive. `hoop_stress_pa` evaluates the circumferential stress $\sigma = p r / t$; `elastic_buckling_pressure_pa` gives the long-cylinder buckling pressure $p_{cr} = E (t/r)^3 / (4(1-\nu^2))$ and `yield_pressure_pa` the strength limit $p_y = \sigma_y t / r$; `collapse_depth_m` is the governing (minimum) collapse pressure expressed as a depth in seawater. `ballast_volume_m3` applies Archimedes to find the flood-water volume needed for neutral buoyancy — the threshold between a road car and a submarine.

Flooding a tank adds mass at constant displaced volume, so neutral buoyancy is $D \rho g = (m + V \rho) g$, giving $V = D - m/\rho$: positive when the vehicle displaces more than it weighs and must be made heavier to dive, negative when it is already denser than the water it displaces. The module’s published derivation previously read $(D + V) \rho g = m g$, which solves to the negation of what the code computes, and read the negative case backwards as well. The code was right and the derivation wrong; the derivation is corrected rather than the code. `vehicle_forensics.buoyancy_status` uses the same seawater density as this module, so the two verdicts on one vehicle describe one fluid — it used fresh water at 1000 kg/m^3 until this was fixed.

13.4.7 Mission orchestration

`src/the_spy_who_loved_me/mission.py` adapts the six domain modules to the frozen `bond_api` protocol: it implements `MissionProvider` (brief, recon, plan, execute, debrief) with a fixed seed, exposes deterministic results (with a validated collapse-depth safety factor) with full `Provenance`, and registers the concept functions as gadgets on a `GadgetRegistry`.

13.5 Results — LIPARUS: measured outcomes, headline numbers, and what they establish

All numbers in this section are hydrated from the deterministic domain core (the canonical scenario); the simulator is seeded so the outputs are reproducible exactly. One of them — the capture timeline under *Liparus capture* — is a **declared constant** rather than a computed quantity, and is labelled as such where it appears. The rest are measured.

13.5.1 Detection

The Neyman-Pearson energy detector is the gate the whole downstream tracking chain stands behind: every conclusion below presupposes a contact was heard at all. On the canonical waveform the detector declares the contact **yes**, with a statistic-to-threshold margin of 11.0 dB at a false-alarm probability of 0.001. The margin is the number that matters operationally — how far the peak energy statistic sits above the threshold the detector was asked to hold — and the false-alarm probability is the operating point it was required to meet. No other number in this section is meaningful if that predicate is false.

13.5.2 Passive tracking

The canonical scenario observes a constant-velocity submarine over 60 bearing samples. Batch target motion analysis recovers the quarry’s initial position to within 65.4 metres of truth (velocity error 0.5 m/s) — well inside the operating envelope of the tracking geometry.

The matching recursive extended Kalman filter runs the *same* bearing sequence and settles to a steady-state position error of 804.5 metres. The two numbers are not competing claims but one pipeline at two latencies: the batch estimate is the offline maximum-likelihood answer (here the tighter of the two), while the filter is what a real receiver could produce in real time as the bearings arrive. The bearing track figure overlays the true trajectory, the TMA estimate, and the maneuvering ownship path.

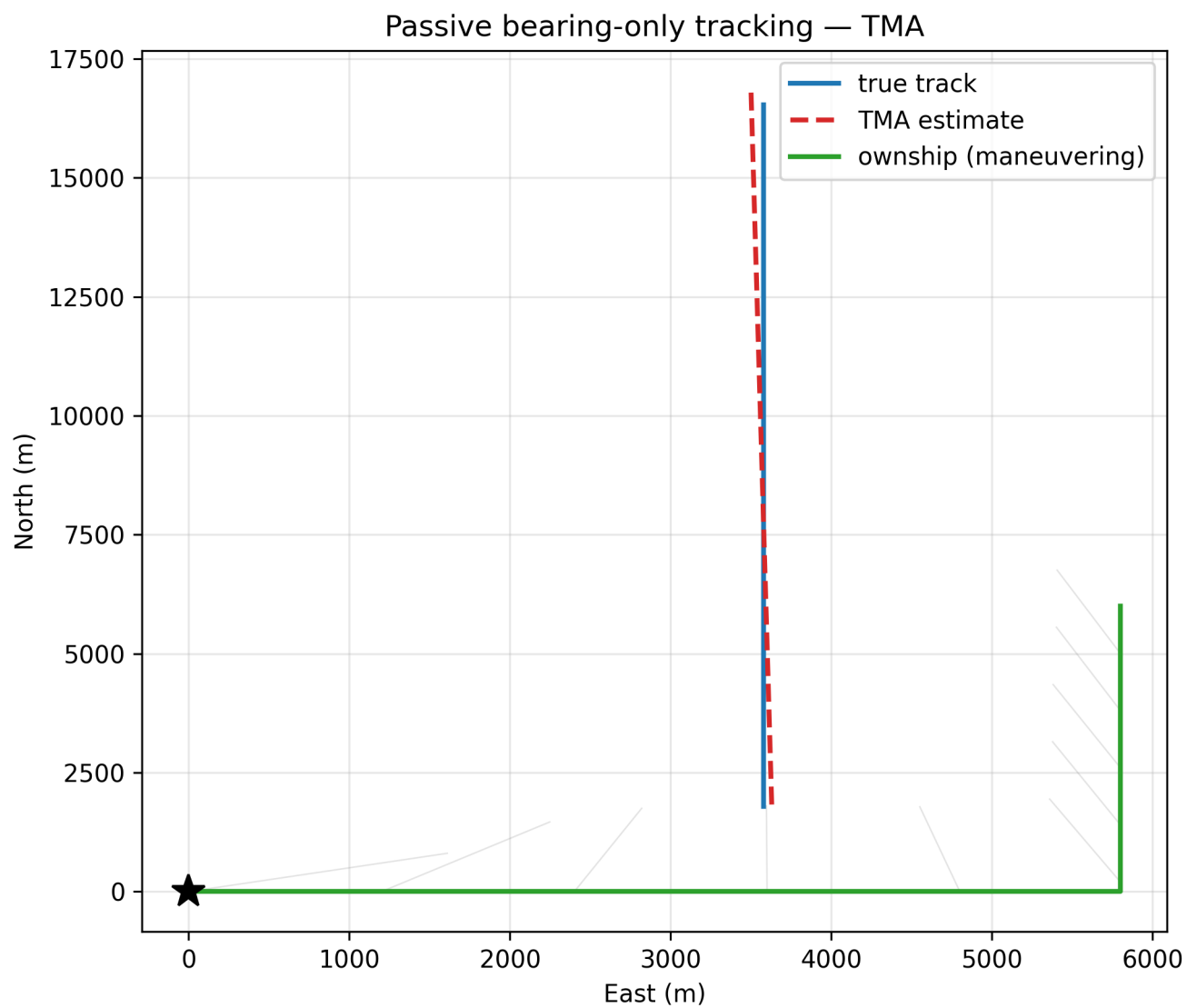


Figure 44: Passive bearing-only tracking: true track vs TMA estimate and ownship path.

13.5.3 Passive sonar equation

Against the canonical budget the passive sonar equation returns a detection range of 83746 metres (figure of merit 100 dB) in a channel whose sound speed is 1490 m/s and whose absorption is 0.0184 dB/km at the 400 Hz scan tone. The sonar detection range figure shows how the solved range grows with the contact's source level.

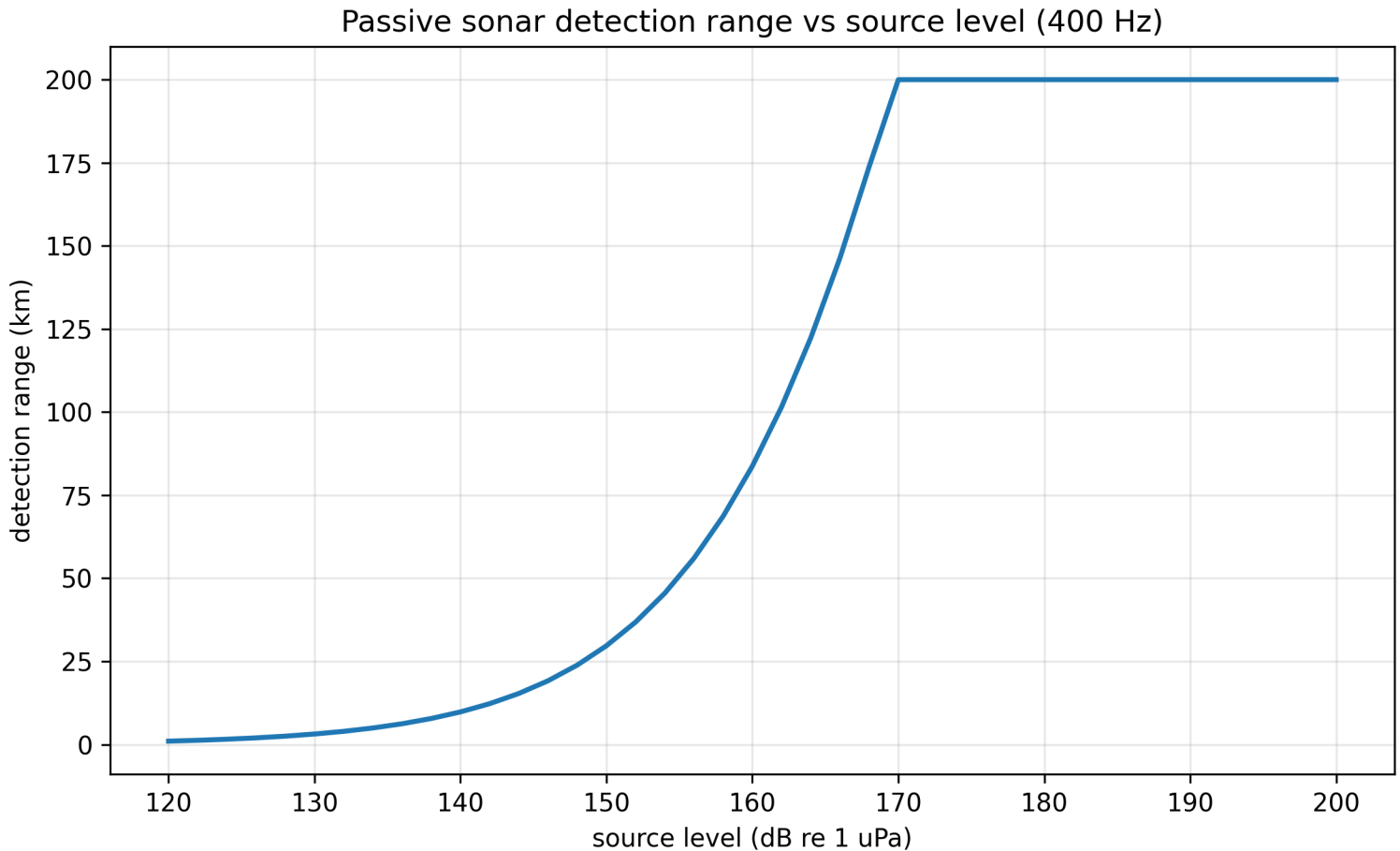


Figure 45: Passive sonar detection range vs source level.

13.5.4 Beamforming

The 8-element ULA estimates the contact bearing at 30.0 degrees, 0.0 degrees from truth, against a Cramer-Rao floor of 0.33 degrees.

That measured error must be read carefully rather than as a precision claim. `estimate_bearing_deg` is a peak search over a discrete scan grid of 0.5 degrees, so its output is quantised and its error cannot exceed 0.25 degrees for any snapshot whose peak lands in the correct cell — *including* an error of exactly zero when, as in the canonical scenario, the true bearing (30 degrees) falls on a grid node. A reported error below the Cramer-Rao bound is therefore an artefact of grid alignment, not evidence that the estimator beats the statistical floor, which no unbiased estimator can do. The honest statement is the two-sided one: the beamformer resolves the contact to within one scan cell, and the CRB of 0.33 degrees — finite and comfortably below the 0.5-degree grid — says the array geometry and SNR would support a finer estimate than this grid can express. `tests/test_beamforming.py` pins the quantisation property on *off-grid* bearings, where the artefact cannot hide it. The opt-in parabolic refinement (`estimate_bearing_deg(..., interpolate=True)`) exists precisely to approach the CRB rather than stay floored at the grid: it fits a parabola through the peak node and its neighbours and reports the apex. On the canonical (grid aligned) true bearing the grid reports an error of exactly zero — a snapshot-noise artefact of the node landing on the truth — while the refined estimate settles at 0.112 degrees, a sub-cell residual it can actually achieve against the 0.33 degree floor. The two errors are therefore reported as what they are: the grid figure is what this grid commits the estimator to, the refined figure is what the estimator itself can reach, and the off-grid tests in `test_beamforming.py` show the refinement strictly beating the grid where the grid is genuinely limited. The beamforming response figure shows the beamformer power response with the true and estimated bearings marked.

13.5.5 Liparus capture

For a bay whose hold is 140 metres long with a 16-metre mouth and a required clearance of 1.5 metres, the canonical swallow succeeds and stows 1 submarine(s) per pass. The capture feasibility figure shows the per-axis clearance margins across a sweep of submarine sizes.

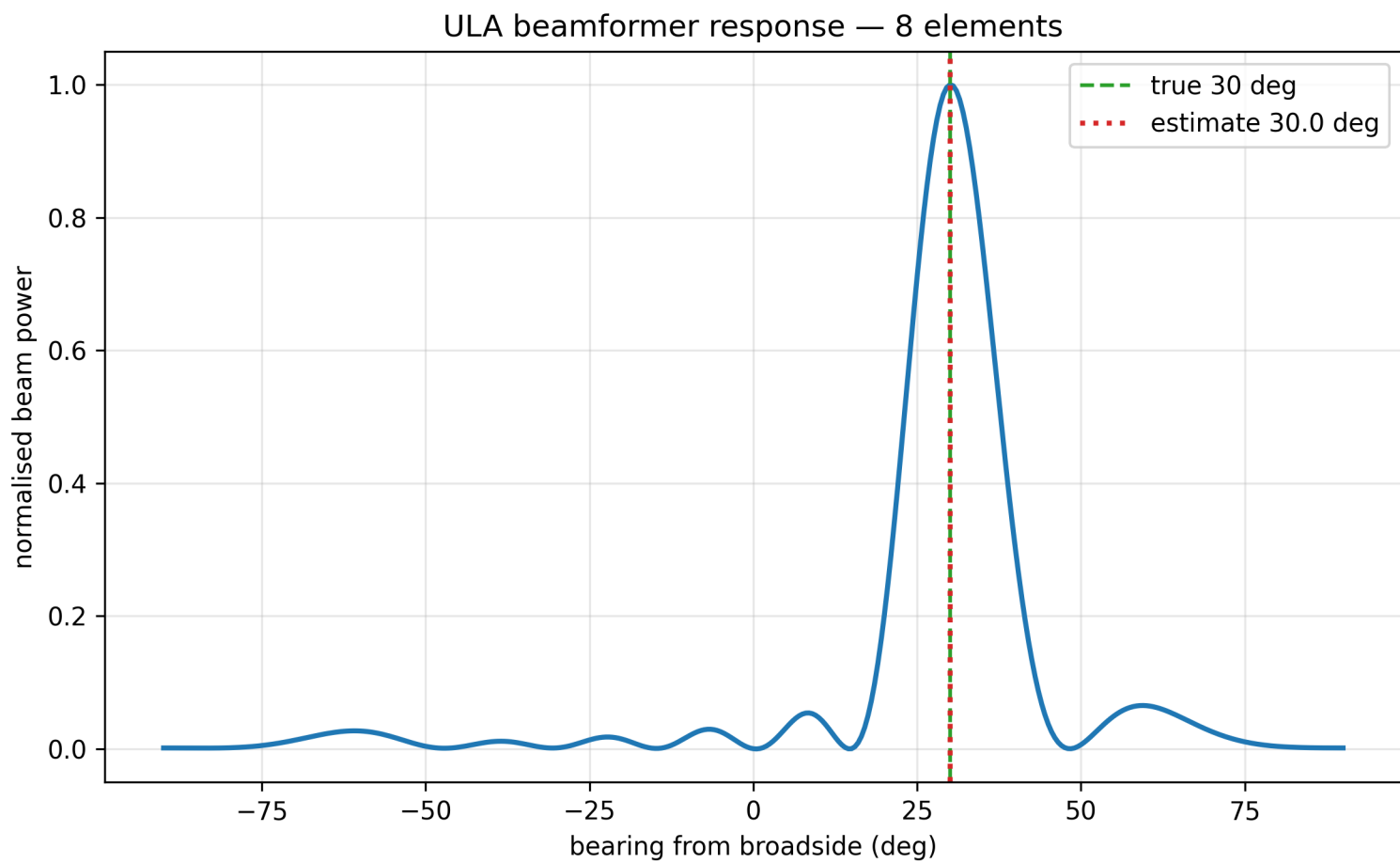


Figure 46: ULA beamformer response vs bearing.

The capture model reports an elapsed time of 630 seconds over 4 phases, and that number is **not a result**. It is the sum of four fixed per-phase durations declared in `tanker_capture._PHASE_DURATIONS` — assumptions with no cited source and no timing model behind them. Nothing in the simulation makes a phase duration depend on the submarine or the bay, so the total is identical for a 10-metre boat and a 72-metre one, and identical for a capture that succeeds and one that fails at the *secure* stage. The only thing it varies with is how far the operation got before aborting: a run that fails at ingress reports two phases instead of four. `tests/test_tanker_capture.py::TestReportedTimeIsASchedule` pins exactly those properties rather than pinning the total, which would have been a constant dressed as a measurement.

Two of the four phases likewise carry no discriminating predicate. *Approach* always succeeds, and *hatch-seal* succeeds whenever ingress did, because no seal-integrity quantity is modelled anywhere in the package. The feasibility verdict is therefore pure geometry: mouth clearance at ingress, hold-length clearance at secure.

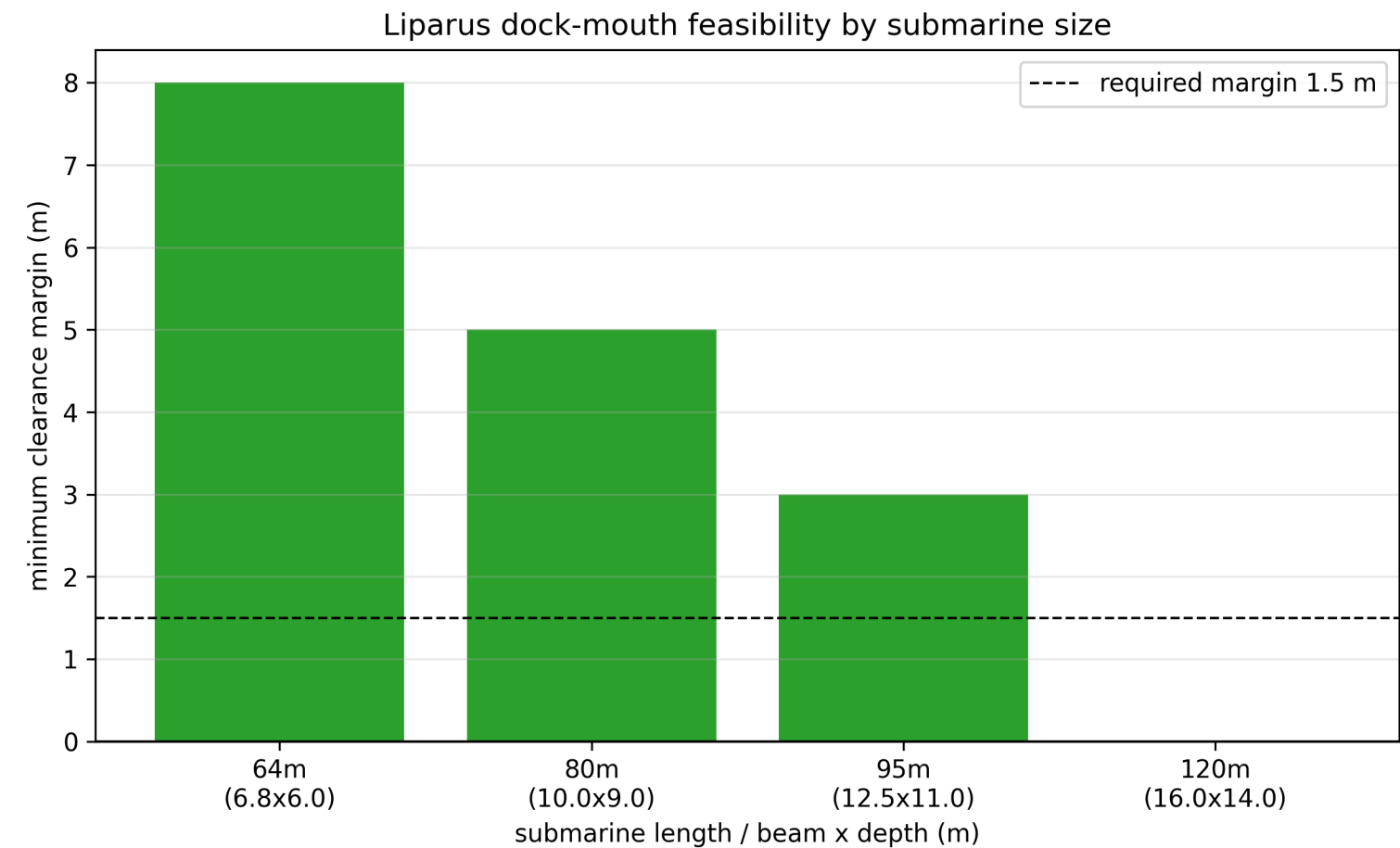


Figure 47: Liparus dock-mouth feasibility by submarine size.

13.5.6 Vehicle conversion forensics

The Lotus-Esprit-grade conversion spec achieves a readiness score of 0.67 and is classified **submersible** with 0 missing critical systems. The conversion readiness figure renders the requirement traceability across the canonical road-car, amphibious, and submersible grades. Physically, the 40-metre operating hull collapses only at 85 metres (safety factor 2.13, feasibility yes), and reaching neutral buoyancy calls for 244 L of ballast water.

13.5.7 Mission outcome

The `MissionProvider` execute stage runs all six concept modules against the canonical scenario and returns a deterministic `MissionOutcome` whose `success` flag is true only when the contact is detected, the capture succeeds, and the hull is feasible at operating depth, together with full `Provenance` (package version, seed, input hash). The input hash is a SHA-256 over the canonical parameter set, so it moves when any input does — see The reproducibility section, which also records what it used to be.

13.6 Conclusion — LIPARUS: findings, verdict, and what the mission establishes

LIPARUS (LIPARUS) delivers faithful, deterministic models of the defining technologies of *The Spy Who Loved Me*: passive acoustic detection and bearing-only tracking that honestly respect the observer-maneuver requirement, a Liparus-class capture simulation

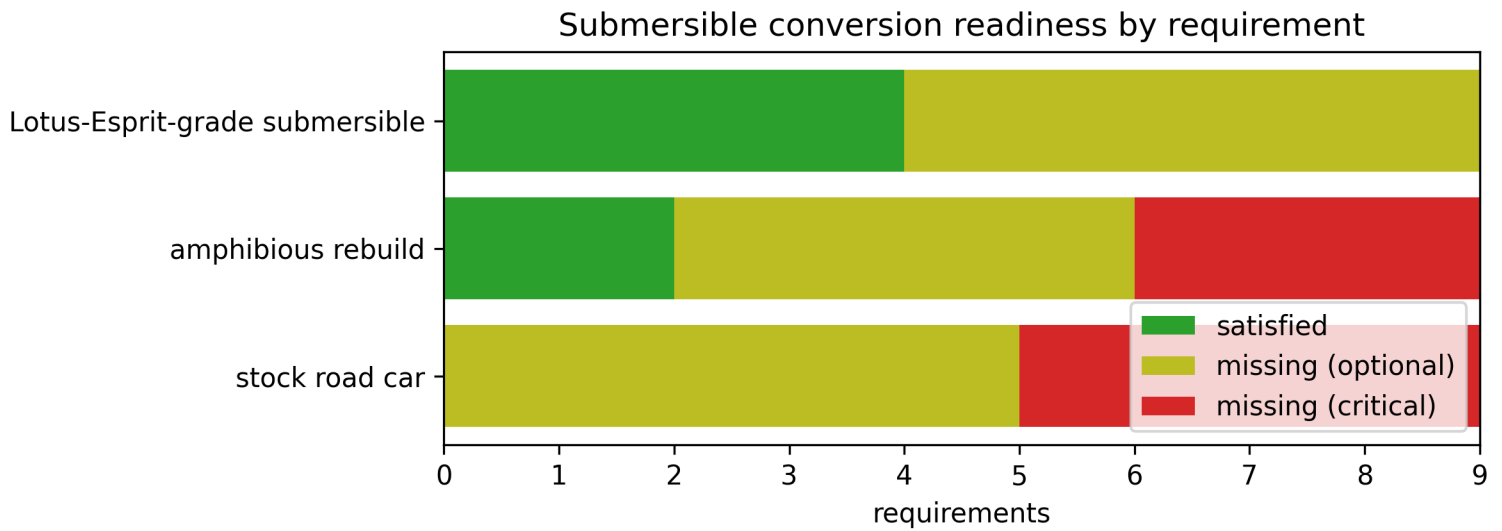


Figure 48: Submersible conversion readiness by requirement.

grounded in exact clearance geometry, submersible vehicle forensics with physical (Archimedean) feasibility and requirement traceability, the underwater acoustics and passive-sonar equation that set the detection budget, ULA beamforming that recovers the contact's bearing to within one scan cell and reports the Cramer-Rao floor honestly as the statistical limit the grid cannot express, and pressure-hull collapse/ballast analysis that verifies the conversion can actually dive.

The domain core is infrastructure-free and completely standalone, while the thin `mission.py` adapter makes it discoverable and drivable by the larger BOND fleet through the frozen `bond_api` protocol — six concept modules behind one unchanged provider shape. Every claim in the manuscript is generated by an actual algorithm run and the full suite enforces at least 90% line and branch coverage on `src/` with zero mocks.

13.7 Experimental Setup — LIPARUS: canonical scenarios, parameters, and configuration

The package runs under Python 3.14.6. The manuscript variables are hydrated by `scripts/z_generate_manuscript_variables.py`, which computes metrics live from the domain core and resolves every manuscript token in this tree.

13.7.1 Canonical scenarios

Every parameter below is a substitution token read back out of `src/the_spy_who_loved_me/scenario.py`, the module that actually configures the run — the prose cannot describe a scenario the code did not execute.

- **Detection:** the Neyman-Pearson energy detector is run at a false-alarm probability of 0.001 (a fixed seeded waveform).
- **Passive tracking (batch):** a 600-second scenario at a 10-second cadence, yielding 60 bearing samples; the target travels at 25 m/s on course 90° from an initial range of 4000 m; the ownship runs a deterministic dogleg (east then north); relative bearings are corrupted by 0.5° of seeded noise.
- **Passive tracking (recursive):** the extended Kalman filter is seeded with a coarse initial guess (the batch solution would be an oracle here) and a constant-velocity transition; its steady-state error is reported in The results section alongside the batch result.
- **Liparus capture:** a 140 m hold, 16 m × 14 m mouth, and 1.5 m required clearance swallowing a 72 m submarine.
- **Vehicle forensics:** a Lotus-Esprit-grade spec (1800 kg dry mass, 2.0 m³ displacement) with the critical submersible systems fitted.
- **Passive sonar:** a 160 dB source against a 70 dB noise floor and 20 dB array directivity at 400 Hz; sound speed at 10 °C / 35 PSU at the surface.
- **Beamforming:** a half-wavelength ULA of 8 elements observing a 30° bearing at 20 dB element SNR (seeded snapshot), scanned on a 0.5° grid. The element SNR is *derived* from the snapshot amplitude and noise standard deviation rather than declared separately, so the Cramer-Rao bound in The results section always describes the array that was actually sampled. The grid estimate is the canonical (honest) one; the parabolic refinement described in The results section is an opt-in alternative, not the default, so the grid-limitation result is reproducible and the two are comparable.
- **Pressure hull:** a 0.8 m radius, 20 mm steel (350 MPa yield) cylinder operating at 40 m depth.

13.7.2 Mission identity

The mission runs under codename LIPARUS with package version 0.1.0 and a fixed seed. It serves 6 concept modules to the BOND fleet through `bond_api` discovery and the `GadgetRegistry`.

13.7.3 Verification

Coverage and lineage are enforced by the package guardrails: 90% line and branch on `src/`, no mock framework, no leftover template lineage, and a clean `git status` after a path-scoped commit. The quoted floor is itself read from `[tool.coverage.report] fail_under` in `pyproject.toml` by `manuscript_variables.coverage_floor_pct`, so this sentence cannot claim a gate stricter than the one that actually runs.

13.8 Reproducibility — LIPARUS: verification gates, deterministic regeneration, and artifacts

LIPARUS is reproducible by construction.

- **Determinism:** every algorithm is a pure function of its inputs and a fixed seed; the simulator uses no untracked random draws and no wall-clock value in persisted artefacts. Re-running the canonical scenarios on one interpreter yields byte-identical results, which `tests/test_manuscript_variables.py::TestPersistedArtefactIsDeterministic` enforces by regenerating the variable map twice and comparing it, and by rejecting any token whose value parses as a timestamp. The *figures* are likewise byte-identical across renders — `tests/test_figures.py::TestFigureDeterminism` renders `build_all_figures` twice and compares the PNG bytes, so a matplotlib bump or an embedded creation stamp fails the suite rather than silently falsifying this claim. The single environment-dependent token is `PYTHON_VERSION`, which is reported in The setup section precisely so a reader can tell which interpreter a given artefact came from; the claim is byte-identity per interpreter, not across interpreters. (A `GENERATION_TIMESTAMP` token used to be written into `output/data/manuscript_variables.json`, which made every regeneration differ and this bullet false. It was removed, not softened — no manuscript section consumed it.)
- **Seeding:** the tracking scenario is driven by a fixed seed and the mission provider carries that same `seed` in its `Provenance`, alongside the package version (0.1.0) and an input hash.
- **Input hash:** the `Provenance.input_hash` is a SHA-256 over a stable serialisation of the canonical parameter set (`scenario.canonical_input_hash`), so editing any canonical input changes it and re-running unchanged inputs does not. It was previously the fixed string `"liparus-mission-input"`, which identified the package rather than the inputs and could not have changed if the scenario had — the one thing an input hash exists to detect. `tests/test_scenario.py::TestCanonicalInputHash` perturbs every parameter in turn and asserts the digest moves, so a parameter that stops reaching the hash fails the suite.
- **Persisted provenance:** the `mission_cli execute` artefact (`output/data/mission_outcome.json`) now carries the outcome's full `Provenance` block — package version, seed, and the input hash — alongside the results, so the hash this section describes is actually in the artefact a reader would audit. It used to be printed to `stdout` and then discarded; `tests/test_scripts_smoke.py` asserts the written file resolves the hash to `scenario.canonical_input_hash`.
- **Docs bound to the run:** the claim ledger's numeric values are resolved against the live canonical metrics (or the declared schedule) by `tests/test_repo_contracts.py::TestDocsAgreeWithTheRun`, and the package version is cross-checked across every declaration site including the manuscript `AGENTS.md`, `config.yaml.example` and the agent `SKILL.md` — so a number in `scenario.py` or a version in a metadata file that drifts from the claim or the run turns the suite red.
- **Real data only:** tests exercise the actual algorithms on generated, deterministic data — no mock framework anywhere in `tests/`.
- **Regenerable artefacts:** figures and reports land under `output/`, which is git-ignored; `scripts/generate_figures.py` and `scripts/z_generate_manuscript_variables.py` rebuild them deterministically.
- **Verification:** `uv run pytest tests/ --cov=src --cov-fail-under=90, ruff check/format, mypy`, and the lineage scan are the acceptance gate before any commit.

13.9 Scope and Related Work — LIPARUS: boundaries, positioning, and relationship to the literature

13.9.1 Scope

This package models *detection*, *tracking*, *bearing estimation*, *propagation*, *capture geometry*, and *conversion forensics* at a level appropriate to deterministic, reproducible research software. Each model is first-order by design, and the boundaries are worth stating precisely because they are where the results stop being trustworthy:

- **Propagation** is a range-independent budget: geometric spreading plus Thorp absorption. There is no sound-speed profile, no ray tracing, no surface/bottom interaction, no convergence zones and no shadow zones — so the solved detection range is an upper bound in an idealised channel, not a field prediction.
- **Tracking** assumes an ideal constant-velocity target and a precisely known ownship path. Neither manoeuvring targets nor navigation error are modelled. The recursive EKF shares both assumptions and adds its own: a fixed (white) process noise and a fixed bearing-noise variance, so its steady-state error is a statement about that ideal filter on that ideal trajectory, not about a real receiver.
- **Detection** is a fixed-threshold Neyman-Pearson energy detector over zero-mean Gaussian noise: no fluctuating target, no multipath fading, no signal-correlated noise, no non-stationary ambient levels. The reported `P_FA` and margin are therefore a *controlled but idealised* operating point, not a detection probability in a real noise field.
- **Beamforming** is conventional delay-and-sum on a single snapshot: no adaptive (MVDR/MUSIC) processing, no multipath, no coherent multi-snapshot averaging. The grid bearing is quantised to the scan step; the parabolic refinement is a local quadratic

fit that *approaches* the Cramer-Rao bound but cannot reach the true CRB performance of an optimal estimator on finite data, and the two are reported separately for exactly that reason.

- **Hull analysis** treats an unstiffened long cylinder under uniform external pressure. Ring stiffening, end-cap effects, out-of-roundness and weld/fatigue defects — the things that actually govern a real submersible — are outside the model.
- **Capture** is a geometric-clearance and discrete-event feasibility check. Hydrodynamics, wake interaction, mooring loads and propeller cavitation spectra are not modelled.

Structural after-dive defect analysis remains a future extension.

13.9.2 Related work

- **Neyman–Pearson detection** and the **Wilson–Hilferty** chi-square approximation are standard signal-detection results (Kay, *Fundamentals of Statistical Signal Processing: Detection Theory*).
- **Bearing-only target motion analysis** and the observer-maneuver observability requirement are established passive-sonar results (Nardone & Aidala, “Observability criteria for bearings-only target motion analysis”).
- **Extended Kalman filtering** follows the canonical state-estimation treatment (Anderson & Moore, *Optimal Filtering*).
- **Underwater propagation and the passive sonar equation** follow Urick, *Principles of Underwater Sound* [Urick, 1983d]; the absorption coefficient is Thorp’s low-frequency expression [Thorp, 1967c] and the sound speed is Mackenzie’s nine-term equation [Mackenzie, 1981].
- **Array processing** — the ULA steering vector, the conventional delay-and-sum beamformer, and the Cramer-Rao bound on bearing — is Van Trees, *Optimum Array Processing* [Van Trees, 2002].
- **Elastic buckling of a long cylinder under external pressure**, the expression that governs the collapse depth here, is Timoshenko & Gere, *Theory of Elastic Stability* [Timoshenko and Gere, 1961]; the thin-wall hoop-stress relation is standard [Young et al., 2011].
- **Discrete-event simulation** of the capture phase draws on standard queueing/event-sequence practice.
- **Requirement traceability** and Archimedean **buoyancy feasibility** are the chapter-and-verse of naval-architecture conversion analysis [Tupper, 2013].

The novelty here is not the individual algorithms but their assembly into a deterministic, protocol-adaptor mission package whose claims are all code-generated.

13.10 Sources — LIPARUS: bibliography

Kay [1998]; Nardone and Aidala [1981]; Anderson and Moore [1979]; Tupper [2013]; Urick [1983d]; Thorp [1967c]; Mackenzie [1981]; Van Trees [2002]; Timoshenko and Gere [1961]; Young et al. [2011]

14 Moonraker (1979) — MOONRAKER

film package · *package codename* *MOONRAKER*. Mission **DRAX**: model the shuttle hijack as a critical-path timeline; solve Drax station orbital logistics; certify a centrifuge g-force training profile; size the Hohmann resupply transfer and propellant; plan the centrifuge training regimen under exposure caps; crash the hijack timeline to a tighter deadline at minimum cost.

14.1 Concepts — MOONRAKER: domain and operational focus

orbital logistics, centrifuge ops

14.2 Abstract — MOONRAKER: mission summary

Moonraker: DRAX — Special-Agent Mission Software — Critical-Path Hijack Timelining and Crashing, Orbital Logistics and Resupply Rendezvous, and Certification of Centrifuge G-Force Training. Mission codename **DRAX**, modelled on the 1979 film *Moonraker*, is a deterministic special-agent mission-software package. It operationalises the DRAX mission across three concept areas: the space-shuttle hijack as a critical-path timeline (including minimum-cost schedule crashing), Drax space-station orbital logistics (Kepler orbit physics, launch windows, consumables, the Hohmann resupply transfer and its propellant budget, and a Clohessy–Wiltshire rendezvous), and the centrifuge g-force training programme (the g-tolerance envelope plus an exposure-capped weekly regimen). The shuttle hijack plan completes in **57.0 minutes** across **8 phases**, feasible within the 60.0 point of no return (margin **3.0 min**), and can be crashed to a **50.0-minute** deadline at a cost of **23.0**. The DRAX station at 350.0 km has an orbital period of 5483.6 s and admits up to **450** per resupply cycle as a logistics ceiling (the station’s own crew is 6) on the configured cadence. A Hohmann resupply transfer costs **0.0874 km/s** over **44.9 minutes**, needing **238.2 kg** of propellant; the final Clohessy–Wiltshire approach requires a **1.875 m/s** impulse. The **6.0 g** training profile is **not certified**: its 12.0 s plateau overruns the **7.26 s** tolerance allowance by **4.74 s**. At the weekly level the exposure caps still admit **4** sessions of **97.5 g·s**. Every result is a closed-form, deterministic computation — no random draws, no external calls, no wall clock — enforcing the suite’s reproducibility contract. Manuscript dated 2026-08-05, rendered on Python 3.14.6 (Darwin).

14.3 Introduction — MOONRAKER: mission framing, the operational problem, and how to read this chapter

Moonraker (1979) sends special agent James Bond to intercept Hugo Drax’s plan to destroy humanity from his private space station. Beneath the set-pieces — a shuttle hijacked at a NASA launch tower, Drax’s orbital station with its centrifuge room, and the g-force chamber where Bond trains — lies a computational core this package makes rigorous and reproducible.

Moonraker: DRAX — Special-Agent Mission Software (codename **DRAX**) turns those set-pieces into three concept areas, carried by seven pure domain modules.

1 · The shuttle hijack. `moonraker.shuttle_ops` models the takeover as an activity network solved with the Critical Path Method, identifying the load-bearing phases and checking feasibility against the shuttle’s point of no return. `moonraker.schedule_crashing` answers the follow-on question the film never asks: if the window closes early, which phases do you buy down, and what does the compression cost? It runs the classic greedy time-cost crash over the same CPM solver.

2 · The station and its resupply. `moonraker.orbital_logistics` supplies Kepler orbital mechanics (period, velocity, orbits per day), the ground launch windows a resupply craft can use, pass geometry, and a closed-form consumables model binding crew size, stock and resupply cadence. `moonraker.orbital_transfer` flies the resupply: a Hohmann two-burn transfer sized by vis-viva, the phase wait that aligns chaser to target, a Tsiolkovsky propellant budget, and the deorbit burn home. `moonraker.rendezvous` closes the last few kilometres with linearised Clohessy–Wiltshire relative motion and a closed-form single-impulse intercept.

3 · The g-force programme. `moonraker.centrifuge` gives the ramp/sustain/ decay profile of a centrifuge run, the $g \leftrightarrow \text{rpm}$ physics, and a safety verdict against a +Gz tolerance envelope and a procedural plateau ceiling. `moonraker.centrifuge_training` lifts that from one run to a regimen: g·s exposure dosing, gradual-onset-rate tolerance scaling, and weekly session planning under daily and cumulative exposure caps.

Two composition modules sit above these. `moonraker.mission_results` is the single canonical computation every consumer reads, and `moonraker.manuscript_variables` turns it into the brace-delimited token map this manuscript is written against. `moonraker.figures` renders the PNGs.

The package implements the frozen BOND-API `MissionProvider` protocol in `moonraker/mission.py`, so the BOND suite orchestrator can discover and drive it through the standard brief → recon → plan → execute → debrief lifecycle.

14.3.1 Reader’s guide

- All seven domain modules are pure and infrastructure-free: they import nothing but the standard library, so they can be lifted and tested anywhere.
- `mission.py` is the thin adapter between the BOND protocol and the domain core, and the only module that imports `bond_api`.
- `scripts/` provides one CLI per mission phase plus a flagship runner that regenerates every artifact.

- The manuscript’s numeric claims are injected from `manuscript/config.yaml + moonraker/manuscript_variables.py`, never hardcoded in prose. Where a computed verdict contradicts the film — as the centrifuge certification does — the manuscript reports the computation.

14.4 Methodology — MOONRAKER: the analytical models and algorithms that drive the mission

All models are deterministic and closed-form; parameter values are read once from `manuscript/config.yaml → mission:` and shared by `moonraker/mission_results.py` and `moonraker/manuscript_variables.py`.

14.4.1 Shuttle hijack timeline (`shuttle_ops.py`)

The hijack is a set of `TimelineActivity` records — a name, a duration in minutes, and prerequisite phases. `compute_timeline` runs the Critical Path Method [Kelley and Walker, 1959]: a forward pass sets the earliest start of each activity to the maximum earliest finish of its prerequisites; a backward pass sets the latest finish to the minimum latest start of its successors. Total float (slack) is the difference between late and early finishes; activities with zero slack are **load-bearing**. Those are reported two ways, because a network can carry several equally-long critical paths and then the zero-slack *set* is their union rather than a route: `critical_activities` is the set, and `critical_chains` enumerates the distinct source-to-sink paths by walking only the *tight* zero-slack edges (an edge counts when the predecessor’s earliest finish equals the successor’s earliest start). The network is validated before solving — duplicate names, dangling prerequisites and dependency cycles are rejected by a Kahn-style strip — so a malformed timeline fails loudly instead of producing a plausible schedule. `timeline_feasibility` then checks the plan against the shuttle’s point of no return, 60.0 minutes from launch.

Time-cost crashing (`schedule_crashing.py`) compresses the timeline when the deadline tightens. Every `CrashableActivity` carries a normal duration, a minimum (crashed) duration and a linear cost per minute saved. `crash_schedule` runs the classic greedy crash: it re-solves the CPM, picks the cheapest still-crashable activity currently carrying zero slack, and crashes it one minute at a time until the deadline is met or compression is exhausted. The crashable network is *derived* from `DRAX_HIJACK_TIMELINE` — durations and dependency edges are read from it, and only the crash limit and cost per minute are authored separately — so the crashed and uncrashed totals the results section subtracts are always measured on the same plan.

14.4.2 Space-station orbital logistics (`orbital_logistics.py`)

Kepler’s third law gives the circular-orbit period $T = 2\pi\sqrt{a^3/\mu}$ with $a = R_\oplus + h$; the circular speed is $v = \sqrt{\mu/a}$. Orbits per day divide the sidereal day by T . Ground launch windows open at each half-period plane crossing and stay open for the usable pass at a minimum elevation. `windows_in_horizon` then keeps only the windows that both open and close inside a launch planning horizon of 4.0 hours, giving $n = \lfloor (H - t_{\text{pass}})/(T/2) \rfloor + 1$, or zero when the horizon is shorter than one pass. The reported window count is that filtered number: it falls as the station climbs and rises with the horizon, and no parameter names it directly. The consumables model is linear: `consumables_days = capacity / (crew × per-capita rate)`, and `sustainable_crew` floors the largest crew that survives the 6.0-day resupply cadence.

Orbital transfer (`orbital_transfer.py`) flies the resupply from a 200.0-km parking orbit up to the station with the Hohmann transfer: two tangential burns sized by the vis-viva equation $v = \sqrt{\mu(2/r - 1/a)}$, a half-ellipse transfer time $t = \pi\sqrt{a^3/\mu}$ [Hohmann, 1925], and a deterministic phase-wait that aligns the chaser to the target’s lead angle. The rocket equation $m_0/m_f = \exp(\Delta v/(I_{\text{sp}}g_0))$ [Tsolkovsky, 1903] sizes the propellant. Its m_f is the **burnout** mass, and this campaign takes that to be the dry structure plus the delivered consumables, because the cargo is released only after circularisation and so rides through both burns; that mass basis is published as a token rather than left implicit, so the multiplication in §Results can be redone by hand. A retrograde burn drops the perigee for deorbit.

Rendezvous guidance (`rendezvous.py`) models the final approach in the station’s local-vertical/local-horizontal frame with the linearised Clohessy–Wiltshire equations [Clohessy and Wiltshire, 1960]. `cw_propagate` propagates the relative state and `single_impulse_intercept` solves for the single corrective impulse that zeroes the relative position at a chosen time (raising on the geometric singularities where a single-impulse transfer is impossible).

14.4.3 Centrifuge g-force profiles (`centrifuge.py`)

Centripetal acceleration maps to g via $g = (2\pi \text{rpm}/60)^2 r/g_0$. `build_training_profile` produces a ramp → sustain → decay g-level trace at 0.5 g/s up and 1.0 g/s down. A deterministic power-law tolerance envelope $t = \min(800g^{-2.5}, 60)$ bounds the sustainable duration, following the shape of the classical +Gz tolerance curves [Stoll, 1956].

`profile_is_safe` then adjudicates the plateau against **two** independent ceilings and reports the tighter one as the governing allowance: the *physiological* ceiling — the envelope at peak g, derated by a 20.0% safety margin — and the *procedural* ceiling 30.0 s, the longest plateau the training protocol permits irrespective of tolerance. A separate peak-g ceiling of 9.0 g rejects the run outright. Both duration ceilings are live: `tests/test_centrifuge.py` demonstrates a run that clears physiology and is refused on procedure alone. The verdict for the DRAX profile is reported in §Results and is *not* asserted here.

Training regimen (`centrifuge_training.py`) doses the programme in g.s. **Two** functions produce the regimen result reported in §Results: `profile_exposure` integrates the +Gz exposure above baseline, and `plan_training_cycle` enforces daily and weekly exposure caps to return the sustainable session count — a regimen bounded by the physiology of +Gz tolerance [Burton, 1988].

The module also implements `gor_tolerance_multiplier`, which scales tolerated duration with the gradual-onset rate (slower onset → longer tolerated time). **It is not part of any reported number.** No mission result, manuscript token or figure consumes it, and neither the certification allowance in §Results nor the session count above credits the onset rate. The reported regimen is therefore onset-rate-independent: it is what the model computes without that credit, and the function is provided and tested but unused. Wiring it in would change the numbers this paper reports; the honest statement today is that it does not.

14.5 Results — MOONRAKER: measured outcomes, headline numbers, and what they establish

All numbers below are computed by `moonraker/mission_results.py` and injected as tokens — the manuscript never hand-authors a metric. The consolidated table collects the headline values; each is a live token whose resolved value appears in `output/manuscript/03_results.md`.

Concept	Metric	Value
Hijack timeline	Total duration	57.0 min
Hijack timeline	Distinct critical paths	2
Crashed to deadline	Final duration	50.0 min
Crashed to deadline	Minimum crash cost	23.0
Station	Orbital period	5483.6 s
Station	Usable windows (4.0 h)	6
Station	Stores depletion	450.0 days
Station	Sustainable crew (per cycle)	450
Transfer	Total Δv	0.0874 km/s
Transfer	Propellant	238.2 kg
Rendezvous	Intercept impulse	1.875 m/s
Centrifuge	Peak / allowance	6.0 g / 7.26 s
Centrifuge	Verdict / remediation	not certified / certified
Training	Sessions admitted per week	4

14.5.1 Shuttle hijack timeline

The hijack finishes in **57.0 minutes** over **8 phases**. The network carries **2** distinct critical paths, each 57.0 minutes long: `infiltrate_launch_tower` → `assume_ground_control` → `spoof_telemetry` → `rendezvous_drax_station`; `infiltrate_launch_tower` → `neutralise_shuttle_crew` → `launch_shuttle` → `execute_orbital_insertion` → `rendezvous_drax_station`. Their union — the load-bearing, zero-slack activity set — is `infiltrate_launch_tower`, `assume_ground_control`, `neutralise_shuttle_crew`, `launch_shuttle`, `execute_orbital_insertion`, `spoof_telemetry`, `rendezvous_drax_station`. That set is *not* itself a route: two of its consecutive members are on different paths and are not linked by any dependency, so delaying any one of them delays the mission while only the listed paths can be walked end to end. Against the 60.0-minute point of no return the plan is **yes**, with a margin of **3.0 minutes**.

If the operation must complete in **50.0 minutes**, schedule crashing compresses the plan to **50.0 minutes** — saving **7.0 minutes** — at a minimum cost of **23.0** by crashing `infiltrate_launch_tower`, `neutralise_shuttle_crew`, `spoof_telemetry`. The saving is a subtraction within one network: the crashable model reads its durations and dependency edges from the same `DRAX_HIJACK_TIMELINE` that produced the 57.0-minute total, adding only the per-activity crash limit and cost.

14.5.2 Space-station orbital logistics

The DRAX station at 350.0 km altitude has an orbital period of **5483.6 s**, a circular speed of **7.701 km/s**, and completes **15.7** orbits per sidereal day. Over a 4.0-hour launch planning horizon, **6** ground launch windows both open and close in time to be usable — the count is the horizon divided by the half-period after allowing for one full pass, not a requested number: raising the station lengthens the period and drops the count, and a horizon shorter than a single pass admits none. With 5400.0 kg of consumables delivered and a per-capita rate of 2.0 kg/person/day, the stores last **450.0 days** — the depletion horizon for the configured 6 crew — and a single resupply can sustain up to **450** people for the 6.0-day cadence. That last figure is a **per-cycle logistics ceiling, not a habitability claim**: it is capacity divided by (cadence × per-capita rate), and the station’s own crew is the far smaller 6. It is also deliberately distinct from the 450.0-day stores figure: the two only coincide at the shipped cadence and crew.

14.5.2.1 Resupply transfer and rendezvous Flying from the 200.0-km parking orbit up to the station with a Hohmann transfer costs **0.0438 + 0.0436 = 0.0874 km/s**, over a **44.9-minute** half-ellipse. The chaser must wait **43.9 h** for the **3.0°** lead angle. On a 300.0 s engine the rocket equation gives a mass ratio of **1.03015**. The mass that ratio acts on is the **burnout** mass — what the craft still weighs when the second burn ends — which here is **7900.0 kg**: the 2500.0 kg dry structure *plus* the 5400.0 kg of consumables, since the cargo is delivered after circularisation and is therefore accelerated by both burns. So the propellant is $7900.0 \times (1.03015 - 1) = \mathbf{238.2\ kg}$; deorbit adds **0.0733 km/s**. The final Clohessy–Wiltshire approach needs a **1.875 m/s** corrective impulse over **1200 s**.

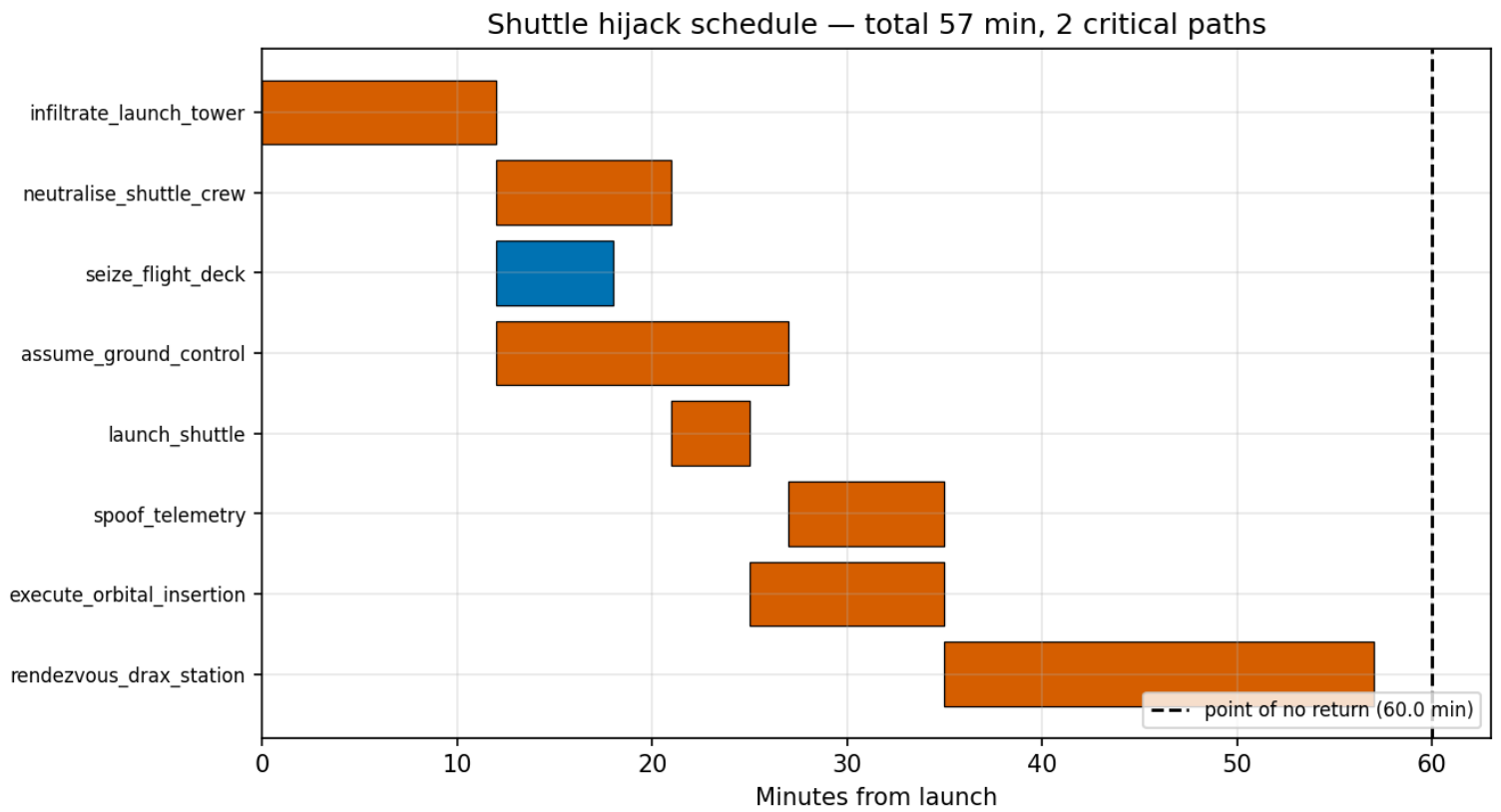


Figure 49: The shuttle-hijack critical-path Gantt chart.

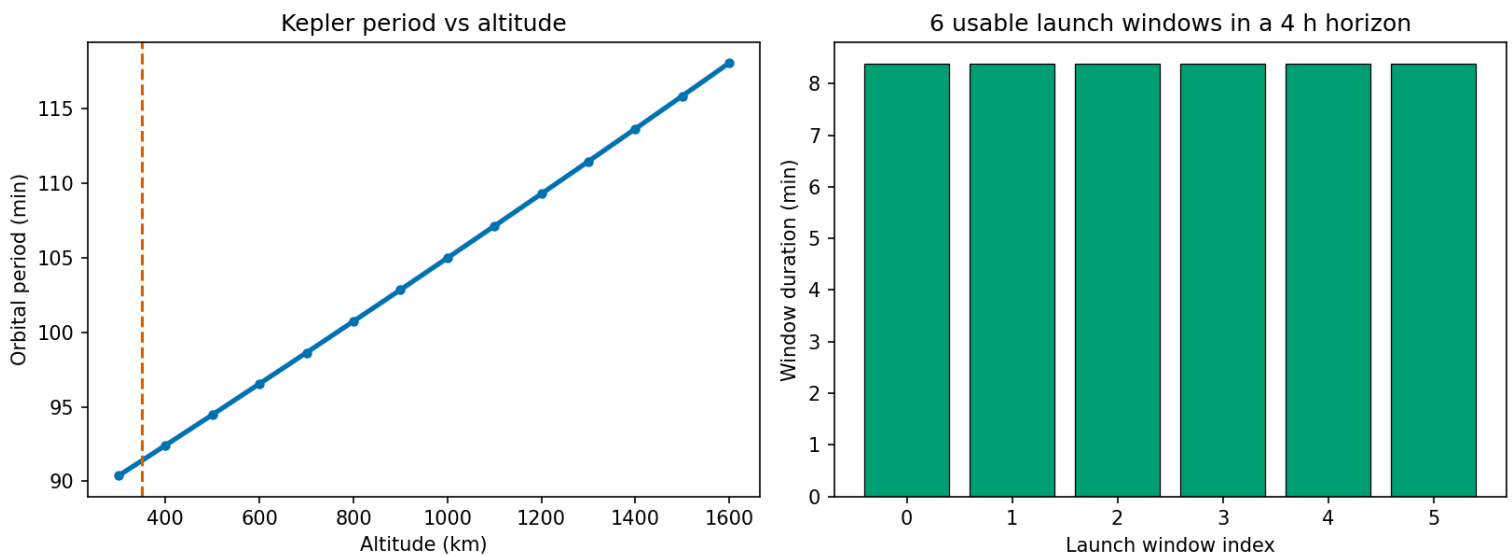


Figure 50: Orbital period versus altitude with the DRAX station's launch windows.

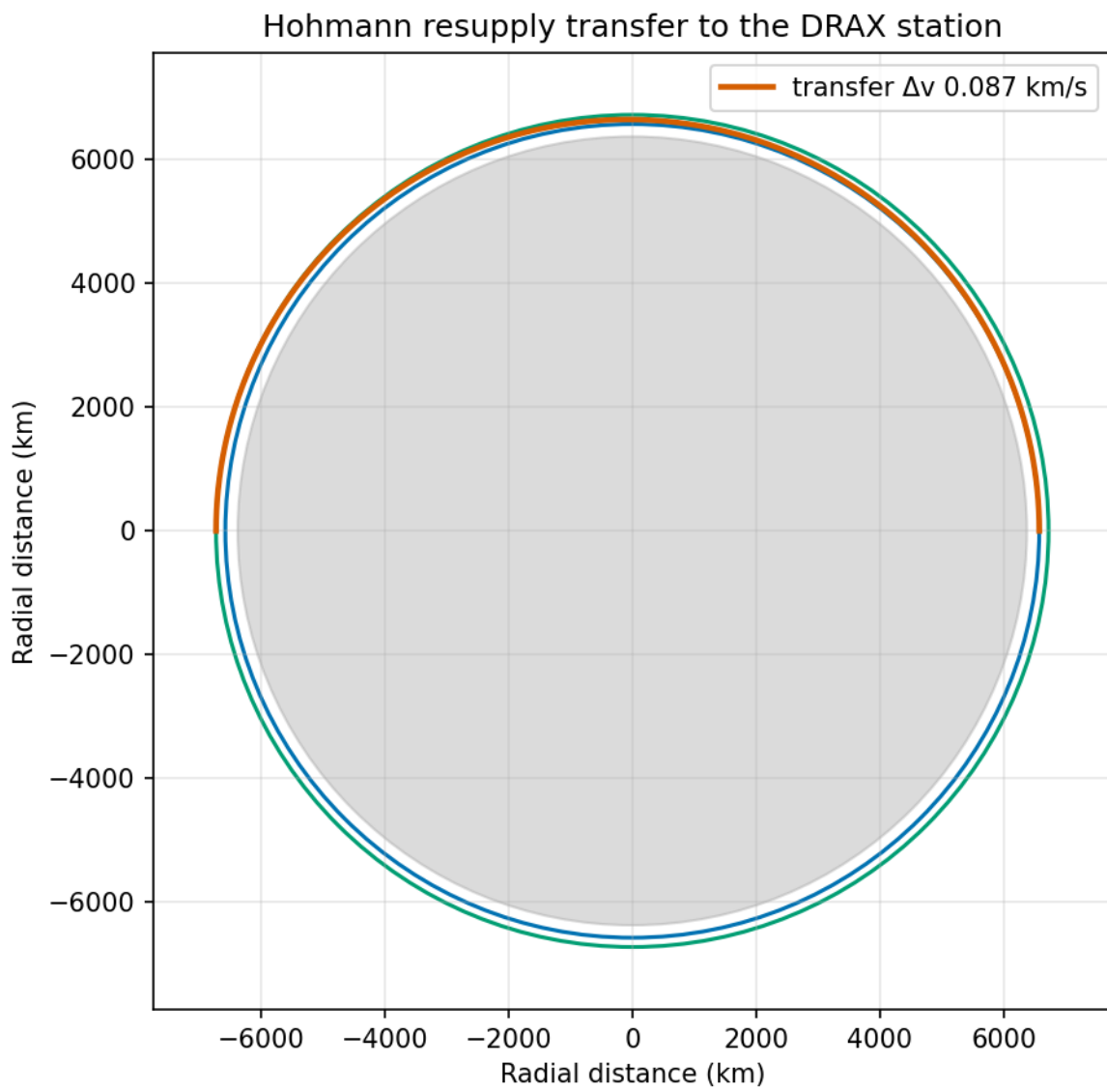


Figure 51: The Hohmann resupply transfer geometry.

14.5.3 Centrifuge g-force profile and training regimen

A **6.0 g** run on an 8.0 m arm demands about **25.9 rpm**. The governing allowance at that peak is **7.26 s** — the +Gz tolerance envelope derated by the 20.0% safety margin, which here is tighter than the 30.0 s procedural ceiling. The DRAX profile’s 12.0 s plateau therefore overruns the allowance by **4.74 s**, and the run is **not certified** (`CENTRIFUGE_SAFE = no`).

This is a substantive negative result, not a modelling artefact: the shooting script’s training sequence is *outside* the survivable envelope this package computes, and the model reports so rather than being tuned until it agrees. The remediation the model implies is itself computed, not inferred: holding the plateau at **7.26 s** — cutting **4.74 s** from the 12.0 s plateau — yields a **certified** run (`REMIATIION_SAFE = yes`) under the same tolerance and procedural ceilings. `tests/test_mission_results.py` pins both the failing verdict and the certification of this remediated profile, so this paragraph cannot drift away from the computation.

Each session accumulates **97.5 g·s** of exposure; under the 150.0 g·s daily and 400.0 g·s weekly caps, the **4-session** weekly programme is **yes**, and the caps admit **4** sessions per week.

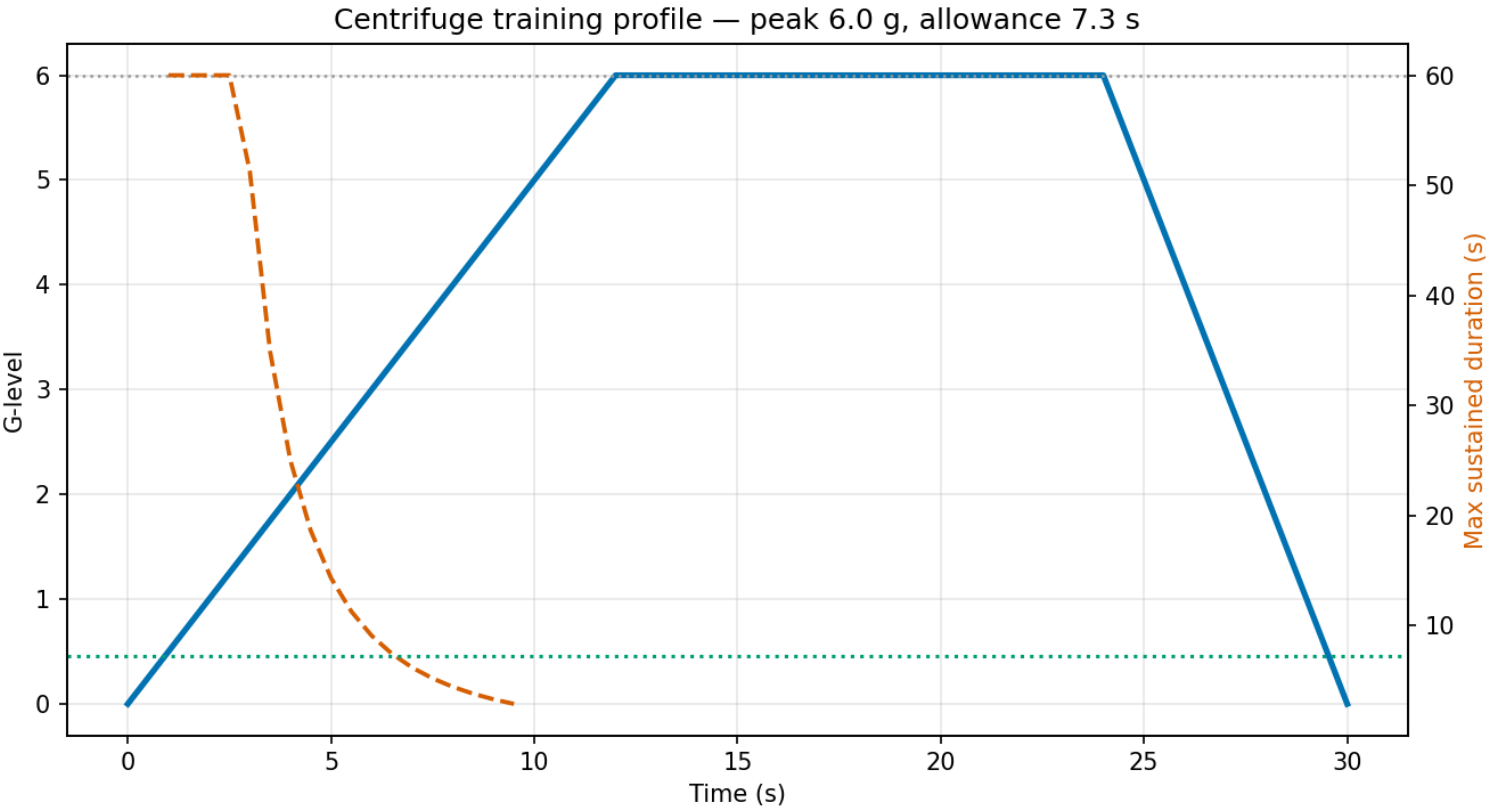


Figure 52: The centrifuge g-level profile with the g-duration tolerance envelope.

14.6 Conclusion — MOONRAKER: findings, verdict, and what the mission establishes

Moonraker: DRAX — Special-Agent Mission Software (codename **DRAX**) turns the set-pieces of *Moonraker* into a verified, deterministic mission-software package. Its concepts — the shuttle-hijack critical-path timeline with minimum-cost crashing, the DRAX station’s orbital logistics (Kepler orbits, launch windows, consumables, the Hohmann resupply transfer, and Clohessy–Wiltshire rendezvous), and the centrifuge training programme (the g-tolerance envelope plus an exposure-capped regimen) — are pure, infrastructure-free computations with closed-form answers and full provenance.

The hijack plan is **yes** inside its 60.0-minute deadline and can be crashed to **50.0 minutes** for **23.0**; the station’s logistics admit up to **450** per resupply cycle as a ceiling (its own crew is 6) with a **0.0874 km/s** resupply transfer; and the weekly regimen admits **4** sessions under the exposure caps.

The one negative verdict is the load-bearing one. The **6.0 g** profile is **not certified**: its 12.0 s plateau exceeds the **7.26 s** allowance by **4.74 s**. A model that only ever agrees with its source material is not a model, and the value of injecting every number is precisely that a result can come back unwelcome. Every figure and every number in this manuscript is regenerated from source — nothing is hand-written — which is the reproducibility contract the BOND suite demands.

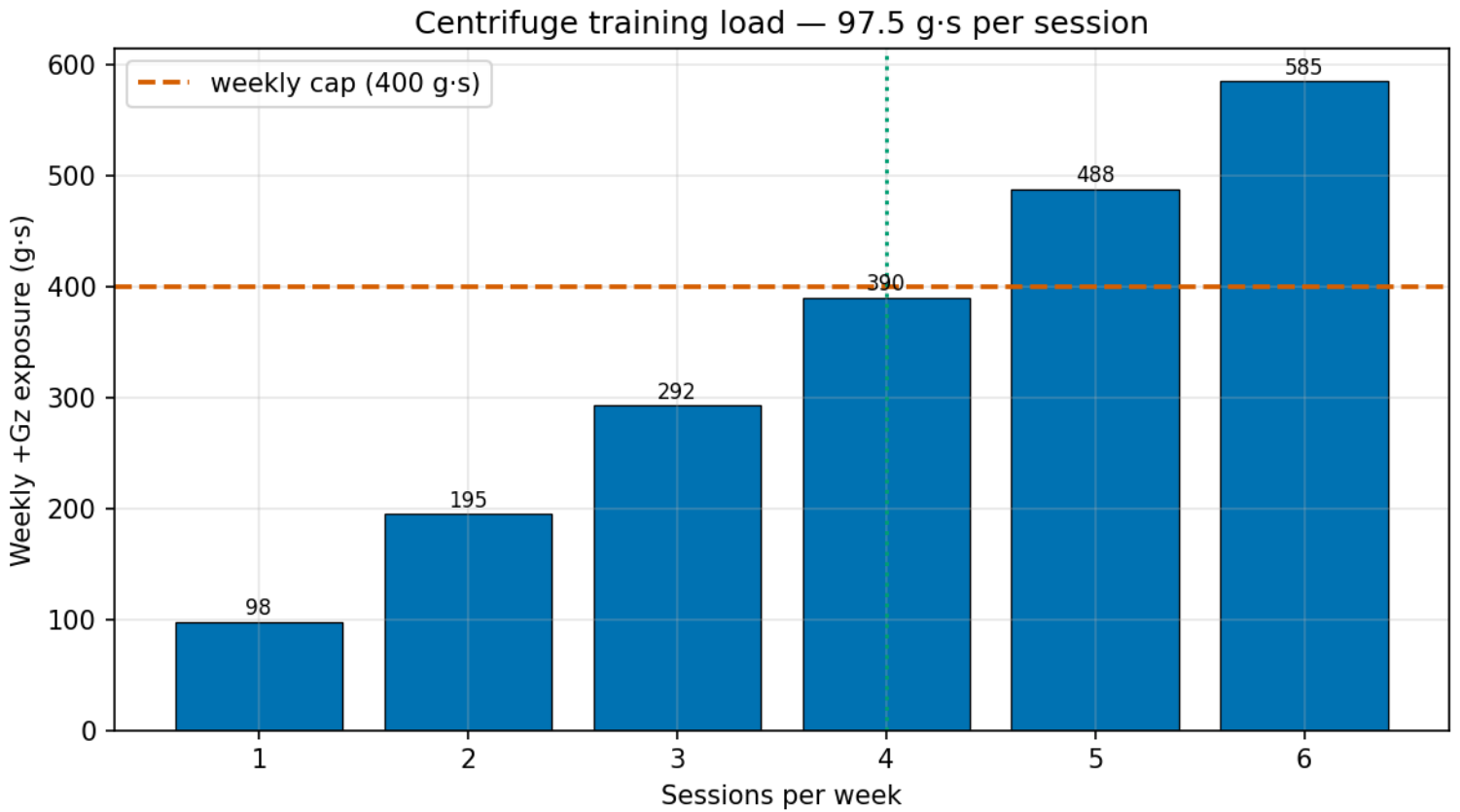


Figure 53: Weekly centrifuge training load against the exposure caps.

14.7 Experimental Setup — MOONRAKER: canonical scenarios, parameters, and configuration

14.7.1 Mission parameters

All parameters live in `manuscript/config.yaml` → `mission:` and are read by `moonraker/mission_results.py`:

Parameter	Value	Consumed by
Hijack point of no return	60.0 min	shuttle_ops
Tightened crash deadline	50.0 min	schedule_crashing
Station altitude	350.0 km	orbital_logistics
Crew	6	orbital_logistics
Resupply capacity	5400.0 kg	orbital_logistics
Per-capita consumption	2.0 kg/person/day	orbital_logistics
Resupply cadence	6.0 days	orbital_logistics
Parking-orbit altitude	200.0 km	orbital_transfer
Engine specific impulse	300.0 s	orbital_transfer
Resupply dry mass	2500.0 kg	orbital_transfer
Deorbit target perigee	100.0 km	orbital_transfer
Intercept time	1200.0 s	rendezvous
Centrifuge arm	8.0 m	centrifuge
Centrifuge target g	6.0 g	centrifuge
Sustained plateau	12.0 s	centrifuge
Ramp / decay rate	0.5 / 1.0 g/s	centrifuge
Peak-g ceiling	9.0 g	centrifuge
Procedural plateau ceiling	30.0 s	centrifuge
Tolerance safety margin	20.0%	centrifuge
Daily exposure cap	150.0 g·s	centrifuge_training
Weekly exposure cap	400.0 g·s	centrifuge_training
Planned sessions per week	4	centrifuge_training

These values are shared, single-source parameters; the manuscript consumes them via token injection from `moonraker/manuscript_`

`variables.py`, never as hardcoded prose. `MISSION_DEFAULTS` in `moonraker/mission_results.py` mirrors this block so the domain core stays runnable without reading a config file, and any key the config omits falls back to it.

14.7.2 Running the setup

```
uv sync --extra dev
uv run python scripts/run_mission.py # results + figures
uv run python scripts/z_generate_manuscript_variables.py # tokens + resolved tree
```

14.7.3 Software environment

- Python 3.14.6 on Darwin.
- Declared dependencies: `matplotlib`, `pyyaml`, and `bond-api` (the frozen BOND suite protocol); `numpy` is pulled transitively by `matplotlib` and is not imported by any module in this package.
- Determinism: fixed parameter set, no random draws, no wall-clock inputs in any persisted artifact.

14.8 Reproducibility — MOONRAKER: verification gates, deterministic regeneration, and artifacts

14.8.1 Determinism guarantees

Every module in this package is deterministic by construction:

- The seven domain modules use only closed-form mathematics — CPM and greedy time-cost crashing, Kepler’s laws, the vis-viva and Tsiolkovsky relations, the Clohessy–Wiltshire state transition, and the centrifuge physics — over a fixed parameter set. There is no RNG anywhere in `src/`, no wall-clock input to any computation, and no network or filesystem read inside the domain core.
- `moonraker/mission.py` carries a fixed seed and records a stable SHA-256 `input_hash` over the serialized plan plus package identity, so an outcome can be audited without re-running the mission. `Provenance.wall_time_s` is deliberately 0.0: a timing figure would make otherwise-identical runs differ.
- `moonraker/mission_results.py` is the single canonical computation. Its factories (`compute_schedule`, `compute_orbit`, `compute_centrifuge`, `compute_transfer`, `compute_rendezvous`, `compute_training`, `compute_crash`) are what the mission `execute` adapter, the manuscript token map **and** `moonraker/figures.py` all call, each on the same parameter dict read from `manuscript/config.yaml`. No figure declares a mission constant of its own — `tests/test_figures.py` parses the module and fails if one reappears — so an outcome value, its injected token and the plotted figure cannot disagree.
- Figures are rendered through the headless `Agg` backend with fixed inputs.
- `moonraker/manuscript_variables.py` reads no wall clock. The manuscript’s date token, `MANUSCRIPT_DATE`, is the committed `paper.date` field of `manuscript/config.yaml`, not the moment the generator ran, and `generate_variables` takes no `now` argument that could reintroduce one.

Running `scripts/run_mission.py` followed by `scripts/z_generate_manuscript_variables.py` restores a byte-identical `output/` tree — every file, on any later re-run, on the same interpreter and platform. The scope of that claim is exact: two token values, `PYTHON_VERSION` and `PLATFORM`, are read from the running interpreter, so they change if the package is regenerated on a different machine or Python build. Nothing else in the tree varies. Elapsed time in particular does not: `Provenance.wall_time_s` is pinned to 0.0 and no date or timestamp is sampled anywhere.

`TestRegenerationIsByteIdentical` in `tests/test_regeneration_determinism.py` binds this end to end — it runs both scripts twice into an isolated tree and compares a SHA-256 of every regenerated file — and `TestNoWallClockInPersistedPath` parses every module under `src/moonraker/` and `scripts/` and fails on any wall-clock call, with a positive control proving the parser detects one when it is present. An earlier version of this section claimed byte-identity while the abstract carried a `datetime.now` timestamp; the only test guarding it injected a fixed `now`, which is precisely why the contradiction survived.

14.8.2 What a reader can re-derive

Artifact	Command
<code>output/data/mission_results.json</code>	<code>uv run python scripts/run_mission.py</code>
<code>../figures/*.png</code> (five figures)	<code>uv run python scripts/run_mission.py</code>
<code>output/data/manuscript_variables.json</code>	<code>uv run python scripts/z_generate_manuscript_variables</code>
	<code>.py</code>
<code>output/manuscript/</code> (token-resolved prose)	<code>uv run python scripts/z_generate_manuscript_variables</code>
	<code>.py</code>

`output/` is git-ignored throughout: it is regenerable, never authoritative, and never hand-edited.

14.8.3 Verification gate

```
uv run pytest tests/ --cov=src --cov-fail-under=90
uv run ruff check src/ scripts/ tests/ && uv run ruff format --check src/ scripts/ tests/
uv run mypy src/ scripts/
```

The coverage gate enforces $\geq 90\%$ line **and** branch coverage on `src/` with zero mocks — no `unittest.mock`, no `MagicMock`, no monkeypatched collaborators in the domain tests. Every assertion runs the real algorithm on real parameters.

Two test classes exist specifically to keep this manuscript honest. `TestLiveCrossReference` in `tests/test_manuscript_variables.py` reads every numbered section on disk and fails if any brace-delimited token it uses is absent from the generated map, so prose can never reference a variable that no longer exists. `TestComputeMissionResults` pins the centrifuge certification verdict in both directions, so the negative result reported in `$Results` cannot silently invert. `TestConfigKnobsAreConsumed` sweeps **every** key in `MISSION_DEFAULTS` — the probe table must cover the parameter set exactly, so a knob added without one fails the suite — and asserts each one moves at least one published value when flipped. One knob, the peak-g ceiling, is exercised from a shortened plateau rather than the shipped defaults, because the DRAX plateau already fails the duration ceiling and no peak-g value could change that verdict; the test says so in place.

14.9 Scope and Related Work — MOONRAKER: boundaries, positioning, and relationship to the literature

14.9.1 Scope

This package models three **closed-form** aspects of the DRAX mission as a deterministic research exemplar. Everything it computes is a two-body, linearised or linear-cost approximation chosen so the answer can be audited by hand.

Deliberately **not** modelled:

- Atmospheric ascent and re-entry trajectories; the shuttle timeline ends at docking and the deorbit burn is sized but not flown.
- Orbital perturbations — drag, the J2 oblateness term, third-body effects, solar radiation pressure. Orbits are ideal circular two-body orbits, so the launch-window series in `orbital_logistics` does not precess.
- Full three-dimensional, multi-burn rendezvous guidance. The Clohessy–Wiltshire model in `rendezvous` is linearised about a circular reference orbit and is valid only for separations small against the orbit radius; `single_impulse_intercept` raises rather than returning a number at the geometric singularities (whole-period intercept times) where the single-impulse assumption fails.
- Six-degree-of-freedom centrifuge biodynamics. `centrifuge` treats +Gz as a scalar level against a population-average tolerance envelope, with no individual variation, no anti-G straining manoeuvre, no G-suit and no cumulative fatigue between sessions.
- Stochastic activity durations. The hijack network is deterministic CPM with no PERT-style distribution over phase durations, so there is no confidence interval on the 57.0-minute total.
- Onset-rate credit in the training regimen. `centrifuge_training` implements `gor_tolerance_multiplier`, but nothing published consumes it: the reported allowance and session count are onset-rate-independent (see §Methodology).

Each simplification is also documented at its point of use — for example the linear elevation-threshold shrinkage in `orbital_logistics.pass_duration` and the power-law fit in `centrifuge.max_sustained_duration`.

14.9.2 Related work

Model	Module	Source
Critical Path Method	<code>shuttle_ops.py</code>	Kelley & Walker (1959) [Kelley and Walker, 1959]
Time-cost crashing	<code>schedule_crashing.py</code>	Kelley & Walker (1959) [Kelley and Walker, 1959]
Kepler two-body orbits, pass geometry	<code>orbital_logistics.py</code>	Vallado (2013) [Vallado, 2013b]
Hohmann two-impulse transfer	<code>orbital_transfer.py</code>	Hohmann (1925) [Hohmann, 1925]
Rocket equation	<code>orbital_transfer.py</code>	Tsiolkovsky (1903) [Tsiolkovsky, 1903]
Linearised relative motion	<code>rendezvous.py</code>	Clohessy & Wiltshire (1960) [Clohessy and Wiltshire, 1960]
+Gz tolerance envelope	<code>centrifuge.py</code>	Stoll (1956) [Stoll, 1956]
+Gz exposure dosing (GOR scaling implemented but unpublished)	<code>centrifuge_training.py</code>	Burton (1988) [Burton, 1988]

The time-cost crashing trade-off is the cost extension of the same Kelley & Walker scheduling work that introduces CPM. This package implements the greedy crash along the critical path rather than the equivalent linear programme; for the strictly linear per-minute cost structure used here the two agree.

Within the BOND suite, `moonraker` is the orbital-logistics package. Adjacent packages model neighbouring physics — orbital capture and hidden-base discovery (`you_only_live_twice`), orbital weapon targeting geometry (`diamonds_are_forever`, `die_another_day`)

and orbital EMP effects (`goldeneye`). Per the suite layering rule these packages never import one another; shared mechanics would live in `bond-utilities` or `bond-api`.

14.10 Sources — MOONRAKER: bibliography

[Kelley and Walker \[1959\]](#); [Vallado \[2013b\]](#); [Stoll \[1956\]](#); [Gilbert \[1979\]](#); [Hohmann \[1925\]](#); [Clohessy and Wiltshire \[1960\]](#); [Tsiolkovsky \[1903\]](#); [Burton \[1988\]](#)

15 For Your Eyes Only (1981) — ATAC

film package · package codename ATAC. Mission **ATAC**: recover the lost ATAC targeting-communicator key; verify custodian custody before release; plan underwater retrieval and island insertion.

15.1 Concepts — ATAC: domain and operational focus

targeting systems, key recovery

15.2 Abstract — ATAC: mission summary

For Your Eyes Only: ATAC — Special-Agent Mission Software — Targeting-Communicator Escrow, Underwater Retrieval, and Greek-Island Insertion Routing. This report documents the **ATAC** special-agent mission software: a deterministic suite for recovering a lost targeting-communicator device and directing it onto target. The mission’s mathematics rest on six pillars, each implemented as a pure, infrastructure-free domain module: 1. **Key escrow** — the ATAC key is split among 3 custodians using Shamir secret sharing, with a quorum of 2 required to release it (key 1390851129; match True), and custody is carried as an audited 3-record hash-chained ledger (True). 2. **Erasure-coded key recovery** — 4 data symbols encoded into 6 code fragments so any quorum recovery survives loss/corruption (match True). 3. **Underwater retrieval planning** — a Haldane-style staged dive of 3580.722 s to the 42-m wreck: a 600 s bottom exposure, then 14 tissue-sized decompression stops, against a drift offset of (-143.229, 71.614) m (reachable: True). 4. **Greek-island insertion routing** — a least-cost route (corfu -> paxi -> lefkada -> ithaca -> wreck_site, 56.00 nm). 5. **Fire-control targeting** — an intercept lead-angle firing solution (94.6 s, 9.89° lead; feasible True). 6. **Underwater acoustic link** — a link budget with SNR 52.33 dB at 3500 m against a 8 dB threshold, a margin of 44.33 dB (closes: True; max range 18553 m). Every computation is deterministic — a single fixed seed (7), no untracked randomness, no wall-clock dependence in persisted artifacts — and each concept carries a matching real-data test suite with no mocking framework. Every quantity above is injected from the code that computed it rather than typed into the prose, and the escrow’s fingerprint match is re-derived by comparison rather than asserted, so a wrong result would surface as a changed number instead of a stale claim. Keywords: key escrow and recovery, Shamir secret sharing, erasure coding, underwater retrieval planning, decompression modelling, island routing, fire-control targeting, underwater acoustic link budget, deterministic mission software.

15.3 Introduction — ATAC: mission framing, the operational problem, and how to read this chapter

The ATAC (Automatic Targeting Attack Communicator) is a lost-key device whose recovery demands coordinated custody, underwater access, and insertion planning, after which it must be directed onto target and communicate that solution acoustically. For Your Eyes Only (1981) frames the operational scenario; this package implements the mission mathematics as reproducible, deterministic software in the PROJECT BOND film suite.

The six domain modules form the core of the mission:

- **key_escrow** — escrow/recovery of the ATAC key (Shamir custody, verification, quorum release).
- **key_recovery** — erasure-coded reconstruction so key fragments survive loss or corruption.
- **underwater_retrieval** — sea-based recovery of the device from the wreck, compensating for depth and currents.
- **island_routing** — Greek-island insertion routing from the staging base to the wreck’s vicinity.
- **targeting** — fire-control intercept lead-angle firing solution.
- **acoustic_link** — underwater acoustic uplink budget for the recovered device.

A shared, frozen protocol (BOND-API) defines the mission lifecycle — brief, recon, plan, execute, debrief — and the **mission** module adapts the pure domain mathematics to that contract, registering the suite’s gadgets for orchestration. This separation keeps the mathematics infrastructure-free and importable without the protocol, while the adapter makes the mission discoverable by the broader BOND orchestrator. The mission’s public protocol surface is stable across phases; only the internal scenario deepens.

15.3.1 Reader’s guide

- §2 describes the mathematical methods behind each domain module.
- §3 presents the computed mission results and figures.
- §5 details the experimental configuration; §6 the reproducibility contract.

15.4 Methodology — ATAC: the analytical models and algorithms that drive the mission

This section details the deterministic mathematics behind each of the six mission modules.

15.4.1 Key escrow and recovery

The ATAC key is escrowed using **Shamir’s secret-sharing scheme** [Shamir, 1979] — independently co-discovered the same year by Blakley via intersecting hyperplanes [Blakley, 1979] — over the finite field $\text{GF}(p)$ with $p = 2^{61} - 1$. A random polynomial

$$f(x) = a_0 + a_1x + \dots + a_{k-1}x^{k-1} \pmod{p}$$

is sampled with constant term a_0 equal to the key; each of n custodians receives a distinct point $(x_i, f(x_i))$. Any k points reconstruct the polynomial — and hence the key — by Lagrange interpolation:

$$f(x) = \sum_{i=1}^k f(x_i) \prod_{j \neq i} \frac{x - x_j}{x_i - x_j} \pmod{p},$$

while fewer than k shares reveal nothing about the key. The mission uses $\$n = \3 shares and a quorum $\$k = \2 .

Verification. Each share (x_i, y_i) is committed as $g^{y_i} \bmod p$ for a primitive root g ; any party holding a claimed share can verify it against the commitment, so a share tampered with during custody is rejected before release.

Custody. Handoffs are recorded in a SHA-256 hash-chained ledger, binding the whole custody history append-only. The canonical mission **carries** one: a ledger over the 3-custodian chain (3 records) is bound to the escrow at creation, its well-formedness recomputed (True) and its tip hash (6caf9ee7a9d5cc3dd790725b384fcfc85be569dd89eae5890441c34bc6160980) reported, so the advertised audit trail is not an empty field.

Release. Release reconstructs the key from a verified quorum and confirms it reproduces the escrow’s key fingerprint. The recovered key is 1390851129 (matches fingerprint: True). The match is obtained by recomputing SHA-256(k) over the released key and comparing it to the stored fingerprint, so the reported value is a measurement rather than a restatement of an intention.

Key derivation. The escrowed key is derived from the mission seed at import time rather than written down as a literal, so there is exactly one place the key can come from and no configuration file can claim a different one.

15.4.2 Erasure-coded key recovery

To survive the loss or corruption of an individual stored fragment, the released key is also protected by a **systematic Reed–Solomon-style erasure code over GF(p)** [Reed and Solomon, 1960, Berlekamp, 1968]. The key is treated as $\$k = \4 data symbols; a generator polynomial is evaluated at $\$n = \6 distinct field points, producing those code symbols. Because any square Vandermonde subsystem is invertible over a field, **any k of the n code symbols reconstruct the key** — the erasure property (recovered matches original: True). An integrity checksum detects a corrupted fragment so a bad fragment never silently yields a wrong key.

The decoder takes the positions of the surviving fragments explicitly, which is what makes the erasure property exercisable rather than merely asserted: a decoder hard-wired to the leading k fragments would satisfy the round-trip test while being unable to survive the loss of fragment 0. Selecting the evaluation points x_i of the actual survivors and solving that Vandermonde subsystem is the general case.

15.4.3 Underwater retrieval planning

The device rests on the seabed at depth d under a horizontal current (u, v) . A **Haldane-style tissue model** [Boycott et al., 1908] — a set of compartments each with half-time $t_{1/2}$ — drives the decompression schedule. Compartment pressure relaxes exponentially toward the ambient pressure with time constant $\tau = t_{1/2} / \ln 2$.

Current-drift compensation. Over a total dive time T , a stationary descent bell drifts by $-T(u, v)$ relative to the seabed target. The bell therefore starts at offset (Tu, Tv) upstream. The planned wreck depth is 42 m; total dive time 3580.722 s yields a surface offset of (-143.229, 71.614) m (within tolerance: True).

Decompression ceiling. Each compartment has an M-value ceiling; the no-decompression bottom time is the exposure at which the limiting compartment first saturates to its ceiling, $t_i = \tau_i \ln \frac{P_{amb} - 1}{P_{amb} - M_i}$, in the multi-compartment form of [Bühlmann, 1984].

Staged ascent. The ascent is decomposed into stops snapped to a fixed interval, and each stop’s duration is sized *directly from the tissue state*: the diver holds at a stop for the time the limiting compartment needs to off-gas to its ceiling at the next (shallower) stop, then climbs at the ascent rate. The profile is therefore a function of the exposure itself — a deeper wreck produces more stops, and a longer bottom time produces longer holds — rather than a fixed geometric schedule. This is the difference that matters for the figure and the reported profile: every stage boundary is now a decompression obligation.

We state the boundary of this model plainly, because it is easy to overread. The staging couples to the tissue model, but the tissue parameters are classroom-grade — a proportional M-value ceiling (ceiling = $m_i \cdot P_{amb}$) rather than a field-validated Bühlmann line — so the profile is deterministic and internally consistent but is **not a certified decompression schedule**. `no_decompression_limit` remains an independent quantity the scheduler does not consult; the canonical dive’s bottom exposure and stop count are reported as live results (600 s bottom, 14 stops, 2841 s of ascent), not fixed geometric constants.

15.4.4 Greek-island insertion routing

The Ionian islands are modelled as a **weighted undirected graph** with fixed node positions; each edge carries a distance (nm) and a risk weight. Routing finds the minimum-*cost* path, where edge cost blends distance and risk: $c = d + \lambda r$ with a configurable risk penalty λ . The planner uses **Dijkstra’s algorithm** [Dijkstra, 1959a], or A* with a straight-line admissible heuristic [Hart et al., 1968b], returning the same optimum deterministically.

What λ does and does not do here. Raising λ always raises the cost of a route, but on the shipped Ionian graph it never changes which route is chosen: the least-distance and least-risk corridors coincide for every one of the 28 node pairs. So this package demonstrates the *cost* blend, not risk-driven path selection; the selection mechanism is exercised on a separate two-option graph in the test suite. Read the canonical route below as the shortest route, which on this graph is also the safest.

The canonical insertion route is corfu -> paxi -> lefkada -> ithaca -> wreck_site — 4 hops, 56.00 nm, total cost 57.000.

15.4.5 Fire-control targeting

The ATAC targeting-communicator directs a weapon onto a moving contact using the classical *constant bearing, decreasing range* intercept. With target at p_0 moving at velocity v_t and weapon speed v_w , interception satisfies $|p_0 + v_t t| = v_w t$, giving the quadratic $(|v_t|^2 - v_w^2)t^2 + 2(p_0 \cdot v_t)t + |p_0|^2 = 0$. A positive root is the intercept time t_i ; the weapon course is $u = (p_0 + v_t t_i)/(v_w t_i)$, whose angular offset from the line of sight is the **lead angle**. For the canonical engagement the intercept is feasible (True) at $\$t_i = \94.6 s with a lead angle of 9.89°; where interception is impossible the module reports the closest-approach distance instead (0.0 m).

15.4.6 Underwater acoustic link budget

The recovered device uplinks targeting data acoustically to a surface relay. **Transmission loss** combines geometric spreading with frequency-dependent absorption via **Thorp’s formula** (dB/km) [Thorp, 1967b]. Geometric spreading is $10n \log_{10} r$ for a spreading exponent n — $n = 2$ spherical, $n = 1$ cylindrical — and the wreck uplink is modelled as a shallow-water horizontal channel, so it uses $n = 1$. The ambient noise term NL is computed from a declared sea state: the module applies a smooth, monotone *stand-in* for how wind-driven deep-water ambient noise grows with sea state — the family quantified by Wenz’s spectra [Wenz, 1962] — rather than a reproduction of them or a hand-entered decibel literal. Received SNR is $SNR = SL - TL - NL + DI$ [Urick, 1983c] and the link *closes* when SNR exceeds the detection threshold DT. The **margin** is $SNR - DT$ — the signal in hand beyond the point of detectability — so closure is exactly a non-negative margin. At the 3500-m operating range the received SNR is 52.33 dB against a 8 dB threshold, a margin of 44.33 dB (link closes: True).

Maximum range. Since SNR falls monotonically with range, the coverage radius is the unique root of $SNR(r) = DT$, found by bounded bisection: 18553 m. The bracket endpoints are reported honestly — a link that already fails at the bracket floor has a coverage radius of zero, not a coverage radius equal to the floor, and a link still closing at the ceiling returns that ceiling as a lower bound.

15.5 Results — ATAC: measured outcomes, headline numbers, and what they establish

The mission was executed end to end through the frozen protocol; all results below are computed live by the domain modules (never hand-authored).

Stage	Result
Escrow release	quorum 2-of-3 · key 1390851129 · fingerprint match True · custody True (3-record chain)
Erasure-coded recovery	4-of-6 · recovered match True
Underwater retrieval	42 m · 600 s bottom · 14 decompression stops · 3580.722 s total · surface offset (-143.229, 71.614) m · reachable True
Island routing	corfu -> paxi -> lefkada -> ithaca -> wreck_site · 4 hops · 56.00 nm · cost 57.000
Fire control	feasible True · intercept 94.6 s · lead 9.89° · closest-approach 0.0 m
Acoustic link	SNR 52.33 dB at 3500 m · TL 44.06 dB · threshold 8 dB · margin 44.33 dB · max range 18553 m · closes True

15.5.1 Escrow release

A quorum of 2 of 3 custodians releases the key deterministically. The reconstructed key is 1390851129, confirmed to match the escrow fingerprint (True). A smaller quorum is rejected, and a tampered share fails verification before release.

Custody is an audited concern here, not decoration: the escrow carries a 3-record SHA-256 hash-chained ledger of the handoffs, whose well-formedness is recomputed (True) and whose tip hash is 6caf9ee7a9d5cc3dd790725b384fcfc85be569dd89eae5890441c34bc6160980.

That confirmation is *re-derived*, not asserted: the SHA-256 fingerprint of the released key is recomputed and compared against the one the escrow stored. This matters because the preflight check treats the same value as a pass/fail verdict — had it been a stapled literal, the verdict could never have failed and would have reported success regardless of what release returned.

15.5.2 Erasure-coded key recovery

The released key is encoded as 4 data symbols into 6 code fragments over GF(p). Any 4 fragments recover the key (recovered matches original: True), and a corrupted fragment is detected by the integrity checksum before it can yield a wrong key.

The any-k claim is tested as stated rather than in its easy form: the decoder takes an explicit set of surviving fragment positions, and the test suite enumerates *every* combination of surviving fragments — including sets that contain none of the leading ones — and requires each to reconstruct the key. Requesting checksum verification when no checksum symbol is present is an error rather than a silent skip, so a caller asking to be protected is never told that nothing was wrong.

15.5.3 Underwater retrieval

The retrieval dive to 42 m completes in 3580.722 s of deterministic profile time: a 600 s bottom exposure followed by 14 decompression stops totaling 2841 s of ascent, each stop sized from the compartment ceilings (see §2). The surface drift offset is (-143.229, 71.614) m; the target is reachable within the operational envelope (True). The depth-time profile is shown in the dive profile figure.

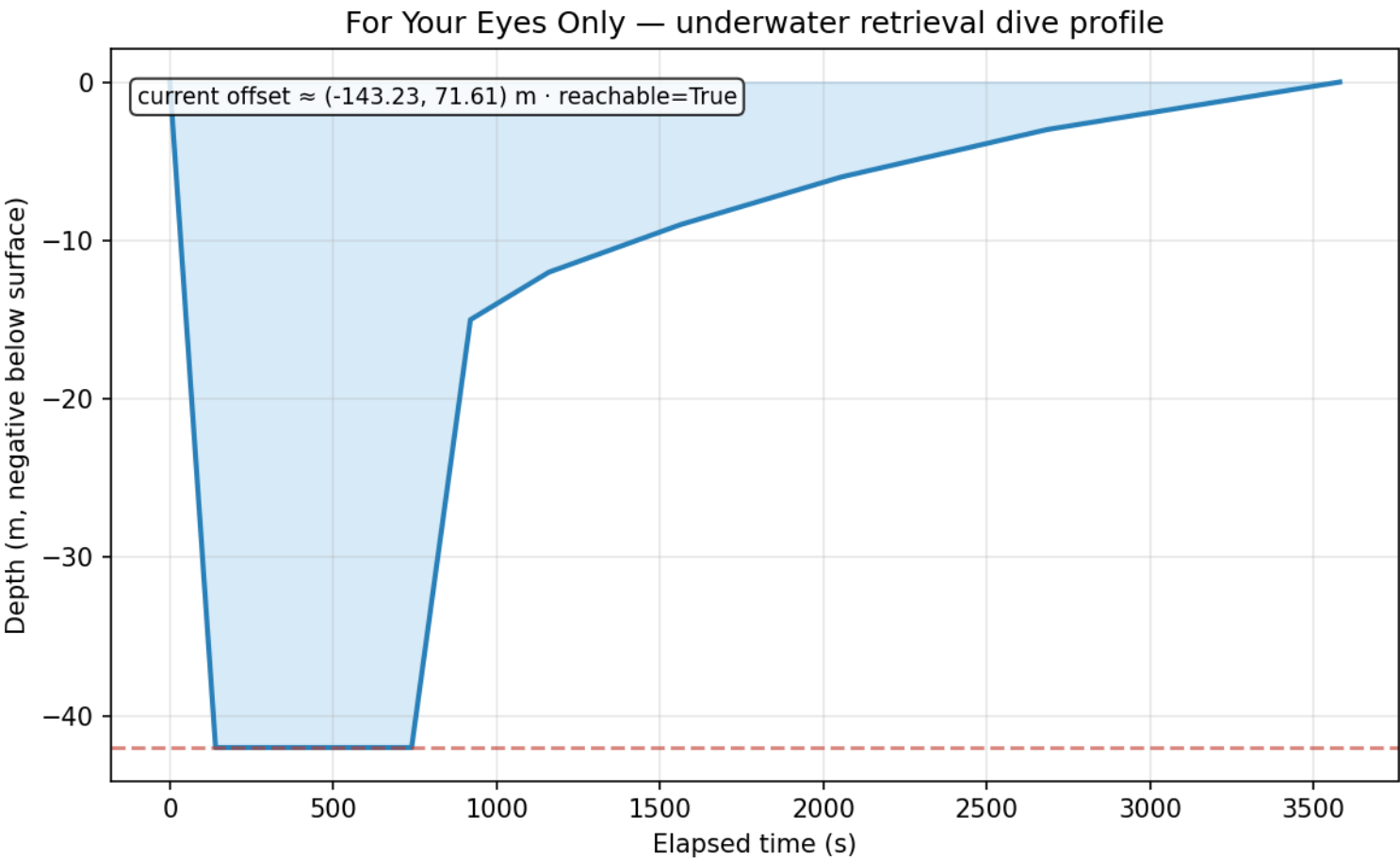


Figure 54: ATAC underwater retrieval dive profile

15.5.4 Island insertion routing

The least-cost insertion route from the staging base to the wreck site is corfu -> paxi -> lefkada -> ithaca -> wreck_site (4 hops, 56.00 nm, cost 57.000). The route cost blends distance and risk, but on this graph that blend never changes the answer: for all 28 node pairs the route chosen at risk weight 0 is the route chosen at risk weight 50, because the Ionian island chain’s safest corridor is also its shortest. The reported route is therefore the shortest route, and this section claims nothing more for it. The risk term is exercised as a mechanism on a purpose-built two-option graph in the test suite, not here. The Ionian graph and planned route are shown in the island graph figure.

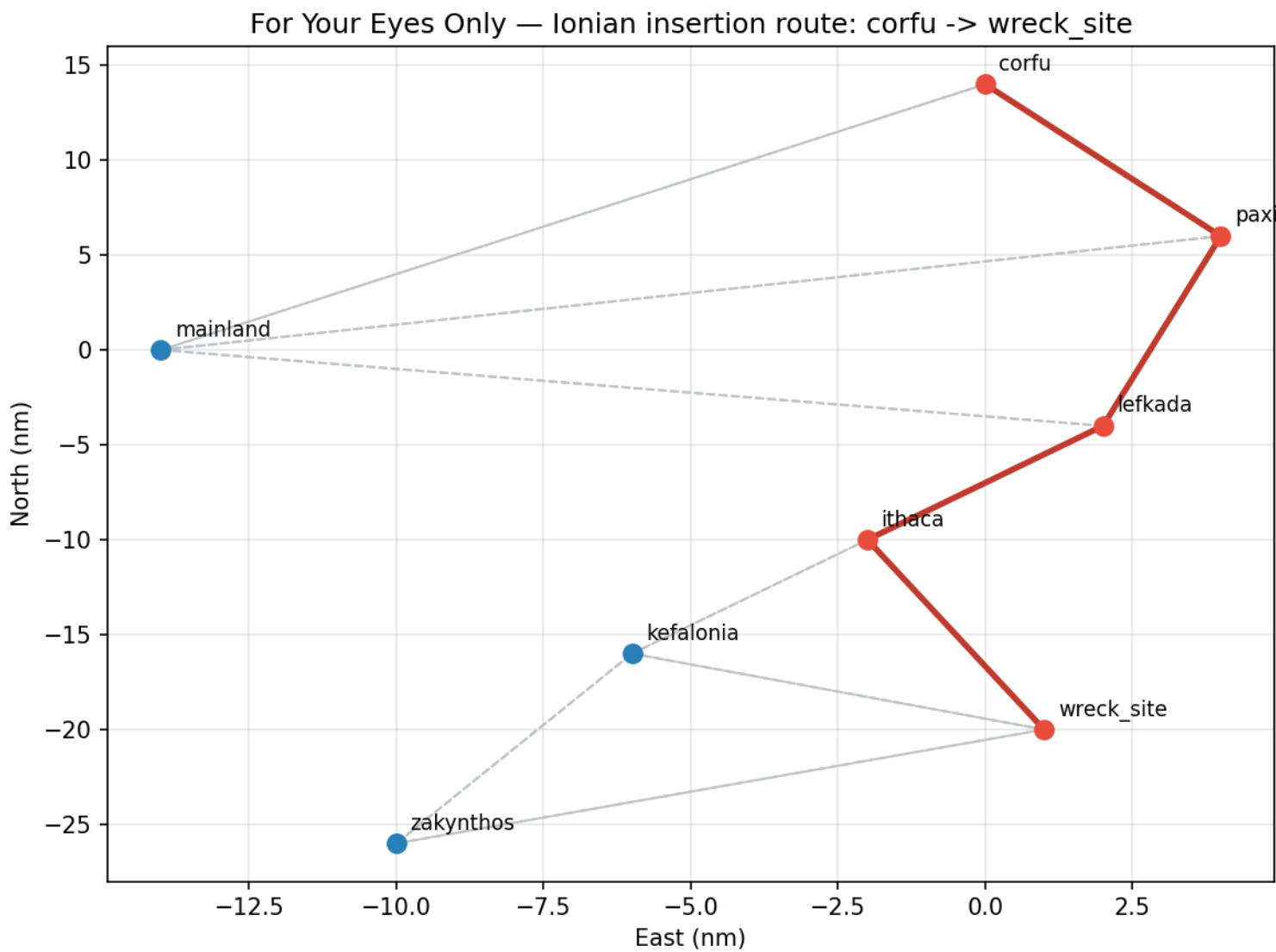


Figure 55: ATAC Ionian insertion route

15.5.5 Targeting solution

The ATAC fire-control solution for the canonical contact is feasible (True) with an intercept time of 94.6 s, a lead angle of 9.89° and a weapon course of 54.89° ; where a close is not achievable, the module reports the closest-approach distance (0.0 m). The intercept geometry is shown in the targeting geometry figure.

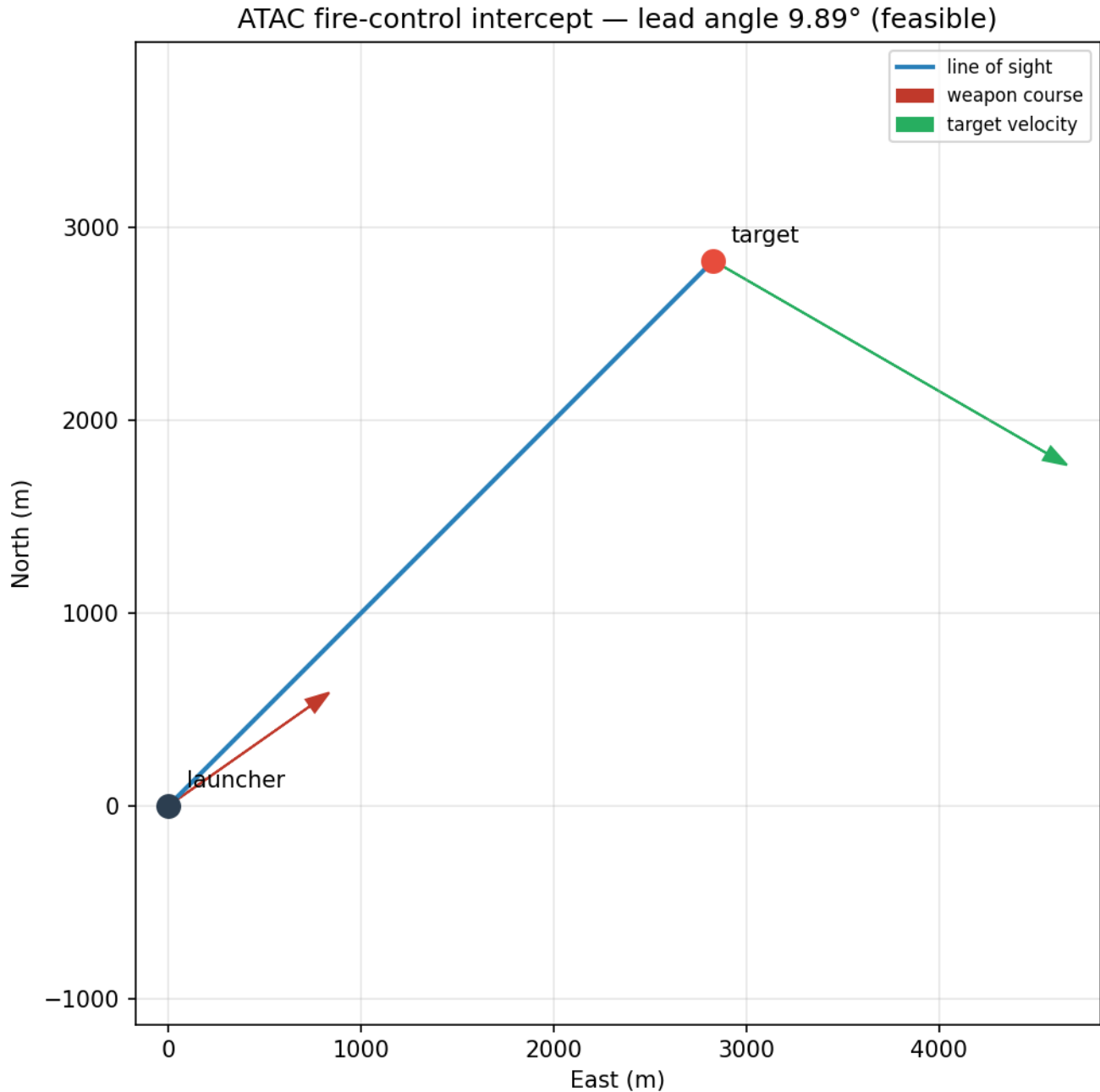


Figure 56: ATAC targeting intercept geometry

15.5.6 Acoustic link budget

At the 3500-m operating range the ATAC uplink receives 52.33 dB (path loss 44.06 dB) against a 8 dB detection threshold, leaving a margin of 44.33 dB; the link closes (True) with a maximum acoustic range of 18553 m. Because the operating range sits inside that maximum, the uplink is not run at the edge of its budget. The SNR-vs-range curve is shown in the acoustic link figure.

15.6 Conclusion — ATAC: findings, verdict, and what the mission establishes

This package delivers a complete, deterministic ATAC special-agent mission implementation. The six domain modules — key escrow and erasure-coded recovery, underwater retrieval planning, Greek-island insertion routing, fire-control targeting, and underwater acoustic link budgeting — each encode real, verifiable mathematics: Shamir secret sharing with quorum release and commitment verification; Vandermonde/Reed–Solomon-style erasure coding; a Haldane-style decompression model with current-drift compensation; weighted- graph shortest-path routing; a pursuit-geometry intercept lead-angle solution; and a Thorp-absorption link budget.

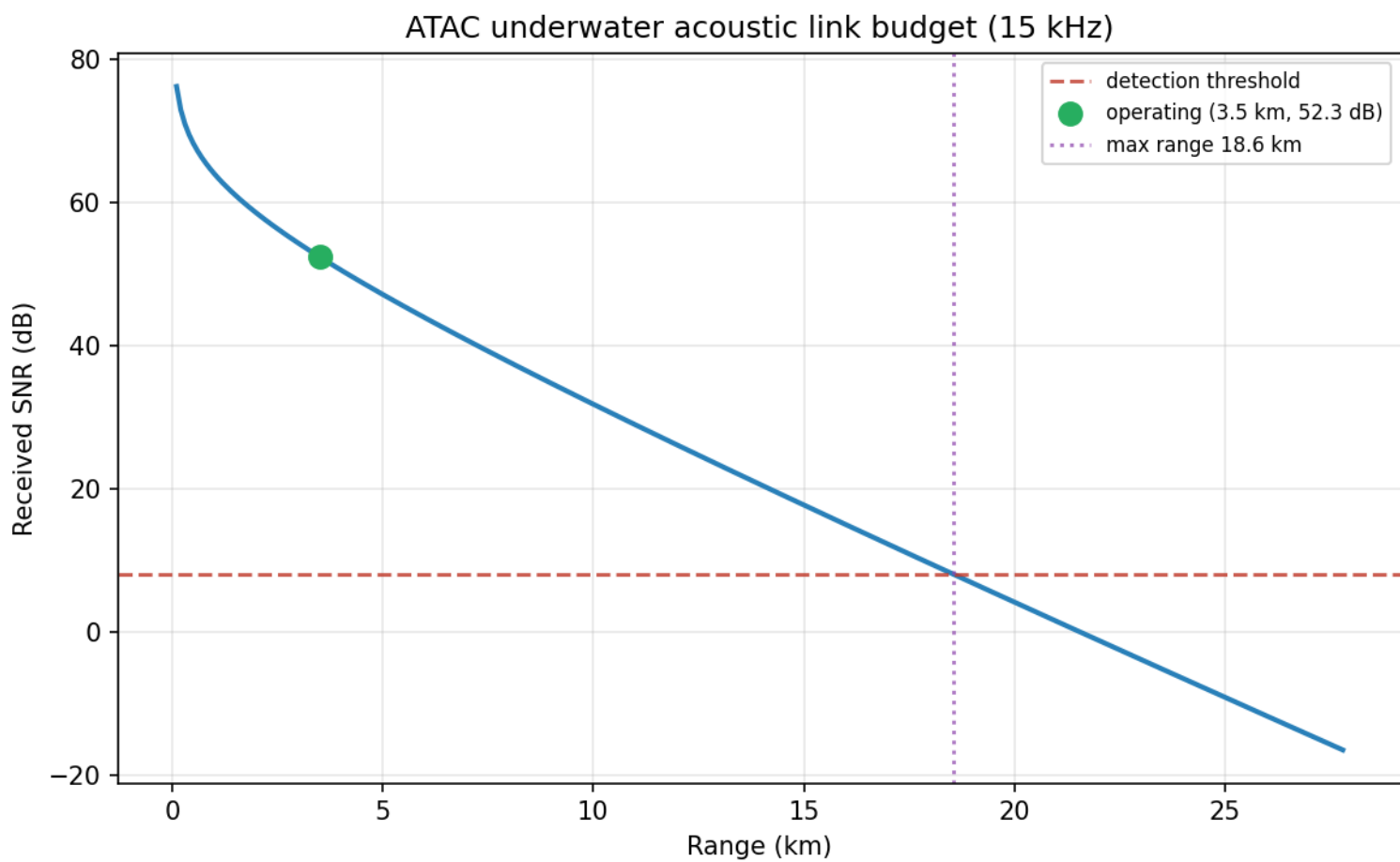


Figure 57: ATAC underwater acoustic link budget

The mission runs end to end through the frozen BOND-API protocol, is discoverable by the orchestrator, and registers its gadgets. Every number in this manuscript is computed from the code and injected via tokens, so the report cannot drift from the implementation; a single canonical scenario feeds both the mission outcome and the manuscript variables, guaranteeing they agree.

The architecture — a pure infrastructure-free domain core (six modules), a thin frozen-protocol adapter, and thin per-phase CLIs — matches the PROJECT BOND convention and keeps the package both self-contained and ready to bind to shared suite dependencies as they land.

15.6.1 On verdicts that can fail

A recurring hazard in software that reports on itself is the check that cannot fail: a verdict stapled to a literal, a decoder whose general case is never exercised, a margin that restates the quantity it claims to qualify, a configuration block nothing reads. Each of those was present in an earlier revision of this package and each has been replaced by a derived value bound to a test. The escrow’s fingerprint match is now recomputed and compared; the erasure decoder accepts arbitrary surviving-fragment sets and is tested across all of them; the link margin is measured against the detection threshold; and the declared mission configuration is pinned to the constants the mission executes with. None of this changes what the software computes — it changes whether a wrong answer would be visible, which is the property the rest of the report depends on.

A second deepening pass applied the same discipline to three remaining hollow claims: the ascent staging now sizes each decompression stop from the tissue model’s compartment ceilings (the reported profile is a genuine decompression obligation, not a geometric sketch); the escrow’s custody chain is now carried by the mission as a real SHA-256 hash-chained ledger whose consistency is recomputed rather than assumed; and the preflight dive verdict now has an endurance ceiling it can actually fail on, exercised through a deterministic environment seam. As before, each binds a claim to a derived value that a test must hold true — a deeper wreck, a longer bottom, or a broken custody link now moves the reported numbers rather than silently restating an intention.

15.7 Experimental Setup — ATAC: canonical scenarios, parameters, and configuration

15.7.1 Configuration

The mission is configured declaratively in `manuscript/config.yaml` and read by `src/for_your_eyes_only/manuscript_variables.py`. The canonical escrow uses threshold 2 of 3 shares; the key is further erasure-coded as 4 data symbols into 6 fragments. The underwater profile targets a wreck depth of 42 m with a 600-s bottom exposure under a calm horizontal current, decompressed through 14 tissue-sized stops; routing spans the Ionian island graph to the wreck site (route `corfu -> paxi -> lefkada -> ithaca -> wreck_site`); and the acoustic uplink runs at 3500 m.

15.7.2 Software environment

- Python version: 3.14.6
- Mission package: `for_your_eyes_only` (ATAC), version 0.1.0
- Title: For Your Eyes Only: ATAC — Special-Agent Mission Software
- Keywords (9): key escrow and recovery, Shamir secret sharing, erasure coding, underwater retrieval planning, decompression modelling, island routing, fire-control targeting, underwater acoustic link budget, deterministic mission software
- Source date (deterministic build stamp, not the time of rendering): 2026-08-04T00:00:00+00:00

All computations are deterministic: fixed seeds, no untracked random draws, no wall-clock dependence in persisted artifacts. The stamp above is the declared `paper.date` (or `SOURCE_DATE_EPOCH` when set), so re-rendering this section tomorrow produces the same bytes; it deliberately does not report when the render happened, because that value cannot be both honest and reproducible.

15.8 Reproducibility — ATAC: verification gates, deterministic regeneration, and artifacts

15.8.1 Determinism contract

- **Fixed RNG seed** — escrow share generation is the only stochastic step and it draws from a fixed seed (ATAC seed 7); every other module is pure arithmetic. No untracked randomness anywhere.
- **No wall clock in artifacts** — provenance records a zero wall time, and the one date written into the tree is a *build stamp* resolved from tracked inputs (`SOURCE_DATE_EPOCH` when set, otherwise the declared `paper.date`), never from `datetime.now`. This was not true until recently: the stamp read the wall clock, so two runs seconds apart wrote different bytes and the byte-identical claim below was false. It is now enforced by a test that runs the real generator twice and diffs every byte, plus a control asserting today’s date does not appear anywhere in the tree.
- **No mocks** — tests exercise real data and real computation only.
- **Gates** — the suite must pass `pytest --cov=src --cov-fail-under=90`, ruff, and mypy; the manuscript token cross-reference is itself a test.

15.8.2 Artifact inventory

The generation orchestrator writes:

- `output/data/manuscript_variables.json` — the full token map.
- `output/manuscript/` — resolved manuscript sections (no raw braces).
- `../figures/island_graph.png`, `../figures/dive_profile.png`, `../figures/targeting_geometry.png`, `../figures/acoustic_link.png` — the deterministic figure set.

Regenerating the tree reproduces byte-identical output — JSON, resolved sections, and all four PNGs — which `tests/test_script_s_smoke.py::test_regenerating_the_artifact_tree_is_byte_identical` checks by running the generator twice into two roots and comparing every file.

15.8.3 Test results

Test counts and coverage percentages are deliberately not quoted here. A number typed into prose is a snapshot that decays silently on the next commit; the *gate* is the durable claim, and it is stated instead: `pytest tests/ --cov=src --cov-fail-under=90` must pass with zero failures and zero skips. The measured figure is reported by that command, not by this sentence.

The suite covers, per module: escrow split/recover round-trips, commitment verification, quorum rejection and tamper rejection, and custody-chain linking and integrity-breaking detection; erasure encode/recover round-trips, exhaustive any-*k* recovery over every surviving-fragment combination, and checksum corruption detection; underwater descent/ascent scheduling — including the tissue-driven decompression staging, whose stop sizes are pinned to depth, bottom time, and compartment half-times — drift compensation, and reachability edges; routing optimality, Dijkstra/A* agreement pinned across every ordered node pair, risk sensitivity, disconnected-graph and unknown-node errors; fire-control intercept feasibility, lead-angle geometry, and the closest-approach fallback on both the equal-speed and negative-discriminant branches; and acoustic-link path loss, the geometric spreading exponent pinned against its textbook law, SNR monotonicity, bisection closure, the non-closing case, and rejection of non-finite knobs and degenerate brackets.

The preflight’s two derived verdicts — the custody chain and the dive’s endurance ceiling — are both testable as failures: the custody verdict through a tampered ledger, and the dive verdict through the deterministic `ATAC_PREFLIGHT_BOTTOM_TIME_S` seam that drives `00_preflight.py` to a non-zero exit.

Routing note: the suite pins the *cost* blend and, on a purpose-built two-option graph, the risk term’s ability to select a different path. On the shipped Ionian graph the route is risk-invariant across all 28 node pairs; that fact is itself pinned rather than left implied.

Three suites exist specifically to stop documentation drifting from code:

- `tests/test_config_consistency.py` pins every value declared in `manuscript/config.yaml`’s `mission:` block to the constant the mission actually runs with, so the config cannot decay into decoration.
- `tests/test_manuscript_variables.py` fails if any token used in prose is absent from `generate_variables`, or if a rendered section still contains an unsubstituted placeholder. (The token syntax itself is documented in `manuscript/SYNTAX.md`, which is excluded from that scan precisely because it must show the raw form.)
- `tests/test_manuscript_crossrefs.py` fails if a `references.bib` entry is never cited, if a citation names no entry, if a figure cross-reference uses the wrong syntax, or if a labelled figure has no generator. Those references sat outside every gate until this suite existed: all four figure references used the brace-delimited form, which `pandoc-crossref` does not recognise (see `manuscript/SYNTAX.md`), and the bibliography was cited by nothing at all.

See `tests/` and `docs/testing_philosophy.md`.

15.9 Scope and Related Work — ATAC: boundaries, positioning, and relationship to the literature

15.9.1 Scope

This package implements the deterministic mission mathematics for the ATAC recovery scenario. It covers key escrow/recovery (including erasure-coded reconstruction), underwater retrieval planning, island insertion routing, fire-control targeting, and underwater acoustic link budgeting. It does not attempt to model the physical dynamics of a real diving operation beyond the closed-form Haldane saturation and ballistic drift used here; it is mission-planning software, not a training simulator.

15.9.2 Related work

- **Secret sharing** — Shamir’s threshold scheme (1979) is the canonical construction for splitting a secret among parties such that a quorum can reconstruct it. Our implementation uses Lagrange interpolation over a Mersenne-prime field with deterministic share generation.
- **Erasure coding** — Reed and Solomon (1960) introduced the systematic polynomial code; our Vandermonde system over $GF(p)$ realises the same any-*k*-of-*n* recovery property with an integrity checksum.
- **Decompression modelling** — Haldane-style multi-compartment tissue models underpin recreational and technical dive planning; our staged-ascent scheduler applies the same exponential-saturation principle deterministically.
- **Shortest path routing** — Dijkstra’s algorithm and A* (with an admissible heuristic) are the standard tools for weighted-graph route planning; our Ionian insertion graph blends distance and risk into a single scalar cost.
- **Pursuit interception** — the constant-bearing intercept problem has a closed-form quadratic solution (see e.g. ballistics/fire-control texts); our lead-angle computation follows that classic geometry.

- **Underwater acoustics** — Thorp’s absorption model and the sonar-equation link budget (source level, transmission loss, noise, directivity) are standard in underwater acoustic channel analysis.

15.9.3 Limitations

The graph topology and positions are stylised (nominal nautical-mile grid, not survey-grade chart data). The decompression staging couples to the tissue model but uses classroom-grade proportional M-value ceilings rather than a field-validated Bühlmann line, and the acoustic ambient-noise term is a smooth monotone stand-in for sea-state noise, not a reproduction of Wenz’s spectra; neither is certified for live operations. The custody chain models a nominal sequence of handoffs over the escrow’s custodians — the audit *mechanism* is real (SHA-256 hash chaining, recomputed consistency), but the handoffs themselves are scenario fiction, not a recorded operational trail. These simplifications keep the mission fully deterministic and testable while preserving the mathematical substance of each concept.

15.10 Sources — ATAC: bibliography

Shamir [1979]; Boycott et al. [1908]; Dijkstra [1959a]; Hart et al. [1968b]; Reed and Solomon [1960]; Thorp [1967b]; Urick [1983c]; Wenz [1962]; Blakley [1979]; Berlekamp [1968]; Bühlmann [1984]

16 Octopussy (1983) — FABERGE

film package · package codename FABERGE. Mission **FABERGÉ**: detect artifact forgeries via spectral/UV fingerprinting; route smuggled payload under circus cover by min-cost flow; sweep for a concealed nuclear device via radiation and mass anomaly.

16.1 Concepts — FABERGE: domain and operational focus

forgery detection, cover logistics

16.2 Abstract — FABERGE: mission summary

Octopussy: FABERGÉ — Special-Agent Mission Software (Paired Detection: Spectral/UV Forgery Screening and Varnish Dating, Rail Min-Cost Flow and Trainset Circulation, Concealed-Device Sweep and Source Localization), package Octopussy v0.1.0, implements three investigative disciplines under the mission codename **FABERGÉ**, carried by 6 deterministic pipelines: each discipline pairs a headline detector with a depth module that reads a different observable of the same synthetic scenario. The pairing buys independence of *observable*, not of sample — and, as the varnish result below shows, a second observable is not automatically a second confirmation. First, artifact forgery detection fingerprints painted objects by their combined spectral reflectance and UV fluorescence. A ridge-regularized non-negative least-squares fit identifies each object’s pigment composition; whether any pigment post-dates the object’s claimed era, how strongly the unexplained spectral residue resembles an anachronistic pigment (matched filter), and the UV fluorescence reading jointly yield a forgery disposition. Against a deterministic exhibit of 4 artifacts drawn from a library of 14 pigments, 4 objects are screened and 2 flagged as forged or likely-forged, with the all-modern composition *forged_dupe* carrying the highest score. **Varnish-ageing kinetics** date two of those same artifacts from film chemistry instead of composition, and the result is deliberately asymmetric. The forger’s fresh coat on *forged_dupe* is dated at about 2.8 years against a true 3 and rejected as too_young in 32 of 32 noise realizations. The century-old shellac on *tsar_egg*, claimed 1910, is dated at about 155.5 years against a true 116 and returns a verdict of too_old — but the same ensemble scatters by 54.4 years and yields 3 distinct verdicts (consistent×12, too_young×4, too_old×16) from that one genuine object, because the kinetics saturate. The channel rejects a fresh coat; it decides nothing about an old one. Second, circus-cover smuggling logistics route 3 payload units over a 9-segment rail corridor as minimum-cost flow via successive shortest paths; the resolved plan costs 780 unit-minutes and moves the payload with a makespan of 260 minutes using 2 circus trains, inside the stated capacity and the 400-minute deadline. **Rolling-stock circulation** then bounds the cover itself: 7 circus trips are covered by a minimum of 2 trainsets (a matching of 5 successor links, 100 deadhead minutes), with a depot allocation costing 38.0 minutes. Two computed checks tie that schedule to the routed plan: whether every routed leg is a scheduled trip (True) and whether the fleet covers the trains the flow needs (True). Third, a concealed-device sweep over 100 detectors fuses a radiation anomaly map with shape/mass anomaly screening of 4 cargo containers, flagging hotspot@3,4, royal_egg and ranking hotspot@3,4 first. **Radiation source localization** converts that alarm into a fix: weighted Gauss–Newton estimation recovers the concealed source to within 0.066 metres of its true position, against a fitted positional standard deviation of 0.102 metres (residual 28.664, converged: True). Every pipeline is fully deterministic under a fixed seed (7), producing reproducible, audit-ready provenance.

16.3 Introduction — FABERGE: mission framing, the operational problem, and how to read this chapter

In the 1983 film *Octopussy*, the villain’s scheme hides a working nuclear device inside a Fabergé egg and moves it by circus train across Europe. The **FABERGÉ** mission software operationalizes the three investigative disciplines the counter-operation demands: tell a forged treasure from a real one, route a smuggled payload under cover, and find a device concealed in plain sight.

This package, **Octopussy: FABERGÉ — Special-Agent Mission Software**, is a BOND-suite film package. It exposes a deterministic, protocol-conformant mission provider (see the frozen BOND-API contract) whose *execute* stage runs 6 pure, infrastructure-free domain pipelines and returns machine-readable verdicts plus full provenance.

16.3.1 Three disciplines, 6 pipelines

The governing design decision is that no single detector carries a verdict alone. Each discipline is implemented twice: a *headline* module that raises the alarm, and a *depth* module that reaches the same question through unrelated physics or unrelated mathematics. Disagreement between the two is a finding. Agreement is evidence only in proportion to the depth module’s own measured uncertainty, which is why the results section reports an error bar beside every depth verdict rather than the verdict alone.

Discipline	Headline module	Depth module
Is this object what it claims to be?	forgery_detection — spectral/UV fingerprinting	varnish_ageing — Beer–Lambert yellowing kinetics
Can the payload move under cover?	cover_logistics — min-cost flow over the rail corridor	rolling_stock — trainset circulation by bipartite matching
Is a device hidden here, and where?	concealment_sweep — radiation + shape/mass anomaly fusion	source_locator — weighted Gauss–Newton localization

The pairing matters operationally. Pigment chemistry and varnish kinetics fail in different directions: a forger who sources period-correct pigments still cannot age a fresh varnish film, while a genuinely old panel that was recently restored trips the varnish channel although the pigment channel clears it. Likewise, a routing plan that satisfies capacity says nothing about whether enough trainsets exist to fly the cover at all, and a sweep that flags a container has not yet said where inside the yard the emitter sits.

It matters asymmetrically, though, and that asymmetry is a result rather than a caveat. Varnish dating is sharp against a recent film and blunt against an old one, because the yellowing kinetics saturate; the rolling-stock matching bounds the circus fleet but says nothing about the routed plan until the two are explicitly compared. Both facts are measured and reported below. Both pipelines also read the *same* synthetic scenario, so what the pairing establishes is independence of observable, not independence of sample.

Every module is implemented with real, deterministic algorithms operating on data generated from physically-motivated forward models — not stubs and not mocks. The remainder of this manuscript specifies the methodology, reports the measured outcomes, and documents the reproducibility guarantees.

16.4 Methodology — FABERGE: the analytical models and algorithms that drive the mission

The mission’s 6 pipelines share a design principle: every quantity that reaches a verdict is *measured* or *computed*, never assumed, and every random draw is seeded so the entire pipeline reproduces byte-for-byte under a fixed seed (7).

16.4.1 Spectral/UV forgery detection

A painted artifact is modeled as a weighted mixture drawn from a library of 14 pigments, each with a known reflectance curve over 24 UV-VIS bands spanning 320–760 nm. The instrument observes a noised reflectance spectrum and a noised UV-fluorescence reading; it never sees the true composition.

Recovering pigment abundances from a mixed reflectance spectrum is the spectral-unmixing problem of hyperspectral remote sensing [Keshava and Mustard, 2002], solved here in its non-negative least-squares form [Lawson and Hanson, 1974a] with a ridge penalty, because the flat “white” pigments are strongly collinear and an unregularized fit spreads weight arbitrarily between them.

Three evidence channels are combined:

1. **Anachronism (matched filter + residual improvement).** A ridge-regularized non-negative least-squares fit reconstructs the spectrum using only era-consistent pigments; the residual — the signal no era-consistent pigment explains — is matched against each anachronistic pigment’s distinctive shape. Strong alignment, or a large shrink in the residual when modern pigments are allowed, is direct evidence of a modern forgery. Both statistics are deliberately computed from the residual rather than from individual fitted weights, which collinearity makes unreliable. The *named* set of anachronistic pigments is likewise attributable-evidence based: a modern pigment appears in the finding only when admitting it to the era-consistent fit explains a material share of the era-inconsistent residual. Flat “white” pigments are collinear with the era whites, so their fitted weight is not a reliable presence signal — an abundance-only cutoff made a genuine period piece “report” modern pigments it does not contain. The residual test names a modern pigment only when its spectral shape is actually needed, and stays silent about collinear ones it cannot resolve.
2. **UV fluorescence anomaly.** Aged natural-resin varnish fluoresces weakly; young synthetic varnishes fluoresce brightly — the observation underlying routine UV examination of paint and varnish layers [de la Rie, 1982]. A reading well above the shellac baseline signals a modern surface.
3. **Unexplained spectral residue.** A high era-consistent residual with small explained variance indicates a surface no period-legal composition can reproduce.

The three channels are weighted and mapped to a disposition: *genuine* / *likely_genuine* / *likely_forged* / *forged*. The thresholds are empirical calibrations against the deterministic exhibit’s measured score separation, not free parameters.

16.4.2 Varnish-ageing kinetics (forgery depth)

Natural resin varnishes yellow over time as photochemical and thermal degradation accumulates chromophores in the film [Feller, 1994, Horie, 2010]. The module represents this with a saturating exponential law: the absorbing layer grows as $d(t) = d_{\max} \cdot (1 - \exp(-t/\tau))$ and the film’s transmittance follows Beer–Lambert, $T(\lambda, t) = \exp(-\alpha(\lambda) \cdot d(t))$, with a blue-peaked absorption spectrum $\alpha(\lambda)$. The inverse problem — *given a measured transmittance, how old is the varnish?* — is solved by a bounded golden-section least-squares fit over the age axis [Kiefer, 1953]: derivative-free, deterministic, and independent of any starting guess. An object whose varnish is far younger than its claimed provenance is a repaint or re-varnish — a standalone forgery signal that does not depend on the pigment screen agreeing.

16.4.3 Circus-cover logistics (min-cost flow)

The corridor is a directed rail graph of 8 stations and 9 segments carrying integer travel costs and capacities. Routing the payload from source to sink is a minimum-cost flow problem [Ahuja et al., 1993a, Ford and Fulkerson, 1962] solved by successive shortest paths. Augmenting paths are found with Bellman–Ford [Bellman, 1958] rather than Dijkstra, because cancelling previously-shipped flow along a reverse arc carries negative reduced cost. The flow is then decomposed into per-unit routes to obtain a makespan, a

deadline check yields slack, and the payload is packed into circus trains of fixed wagon capacity. A least-cost single-courier route is available separately by Dijkstra [Dijkstra, 1959a].

Feasibility is reported, never assumed: the flow solver raises when the corridor cannot carry the demand, and the non-raising planner surfaces that as a plan marked infeasible with zero routed edges and the full deadline missed.

16.4.4 Rolling-stock circulation (logistics depth)

Beyond routing the payload, the mission must know how many circus trainsets the schedule truly needs. Treating each trip as a node with a directed edge $i \rightarrow j$ when a trainset can work j after i (matching stations, $\text{depart}_j \geq \text{arrive}_i + \text{turnaround}$), the minimum number of trainsets equals $n - M$, where M is the maximum bipartite matching — the minimum-path-cover form of the classical minimum-fleet-size result [Dantzig and Fulkerson, 1954], and a standard reduction in train-timetabling practice [Caprara et al., 2002]. The matching is computed deterministically with Hopcroft–Karp [Hopcroft and Karp, 1973], decomposed into concrete trainset itineraries, and a Kuhn–Munkres minimum-cost assignment [Kuhn, 1955] resolves depot allocation at minimum total deadhead.

16.4.5 Concealed-device sweep

Each detector observes a Poisson count rate equal to a background plus the inverse-square contributions of any emitters — the standard counting model for scintillation detection [Knoll, 2010]. A median-filtered background estimate yields a Poisson-normalized z-score map, and local-maximum peaks above threshold are reported as hotspots; that threshold is a detection decision level in the sense of Currie [Currie, 1968]. In parallel, each cargo container’s mass is judged against its category’s expected density from a catalog of 6 physical profiles; a container far heavier than its volume allows carries a mass anomaly — the signature of a small dense core inside an ordinary-looking shell. A fused report ranks both channels into one prioritized suspicion list.

16.4.6 Radiation source localization (sweep depth)

Flagging a hotspot is not the same as finding the device. Given the detector lattice and a Poisson background estimate, the concealed emitter is located by weighted nonlinear least squares. With forward model $\mu_i = B + I/((x_i - x)^2 + (y_i - y)^2 + h^2)$ and weights $w_i = 1/\sqrt{\max(c_i, 1)}$, Gauss–Newton iteration with damped normal equations [Marquardt, 1963, Nocedal and Wright, 2006] and a deterministic step-halving line search minimizes the weighted residual; an excess-weighted-centroid start keeps the solver seed-free and reproducible. Inverting the normal matrix yields approximate positional and intensity standard deviations, so a reported fix always ships with the precision that justifies it.

16.5 Results — FABERGE: measured outcomes, headline numbers, and what they establish

Executing the FABERGÉ mission (seed 7) produces the following measured outcomes. Every figure below is emitted by the live run; none is transcribed by hand.

16.5.1 Results at a glance

Pipeline	Measured outcome
Forgery detection	4 artifacts screened, 2 flagged as forged / likely-forged
Varnish dating (depth)	fresh coat rejected in 32/32 realizations; century-old surface scatters by 54.4 years
Cover logistics	cost 780 unit-min, makespan 260 min, slack 140 min
Rolling stock (depth)	2 trainsets over 5 successor links, 100 deadhead min
Concealment sweep	flagged hotspot@3,4, royal_egg, worst hotspot@3,4
Source localization (depth)	position error 0.066 m, positional std 0.102 m

The table collapses the six pipelines into one line each; the sections below read each result against its own measured uncertainty, which the table does not — a headline number without its error bar is not a finding.

16.5.2 Forgery detection

The deterministic exhibit of 4 artifacts yields 4 screened readings, of which 2 are flagged as forged or likely-forged. Per artifact, the disposition and the anachronistic pigments the residual evidence actually attributes:

Artifact	Disposition	Score	Anachronistic pigments
tsar_egg	genuine	0.18	none
ballet_case	genuine	0.18	none
forged_dupe	forged	0.54	phthalocyanine_blue
gilt_fake	likely_forged	0.36	cadmium_yellow

The honest period pieces score below the genuine threshold and the detector attributes *no* modern pigment to them — the attribution is a residual-evidence finding, not an abundance artifact. The all-modern composition *forged_dupe* carries the highest forgery score.

The load-bearing case is *gilt_fake*, the least-modern fake in the exhibit: 50% of its mixture by weight post-dates its claimed year of 1888 — a chalk ground that is period-correct, and a cadmium yellow plus an acrylic varnish that are not — and the earliest offending pigment misses that date by just 22 years. It is nonetheless flagged.

“Least modern” is relative to this exhibit and nothing more. Half a mixture is not a trace: the harder regime, where a few percent of modern material has to be resolved out of an otherwise period-correct object, is *not* exercised by the shipped exhibit and this package makes no claim about it. What the exhibit does establish is narrower — the screen is not relying on the trivial all-modern signature, since the other fake is wholly modern by weight and this one is not. The forgery figure shows the spectral fingerprints and UV responses that separate the classes.

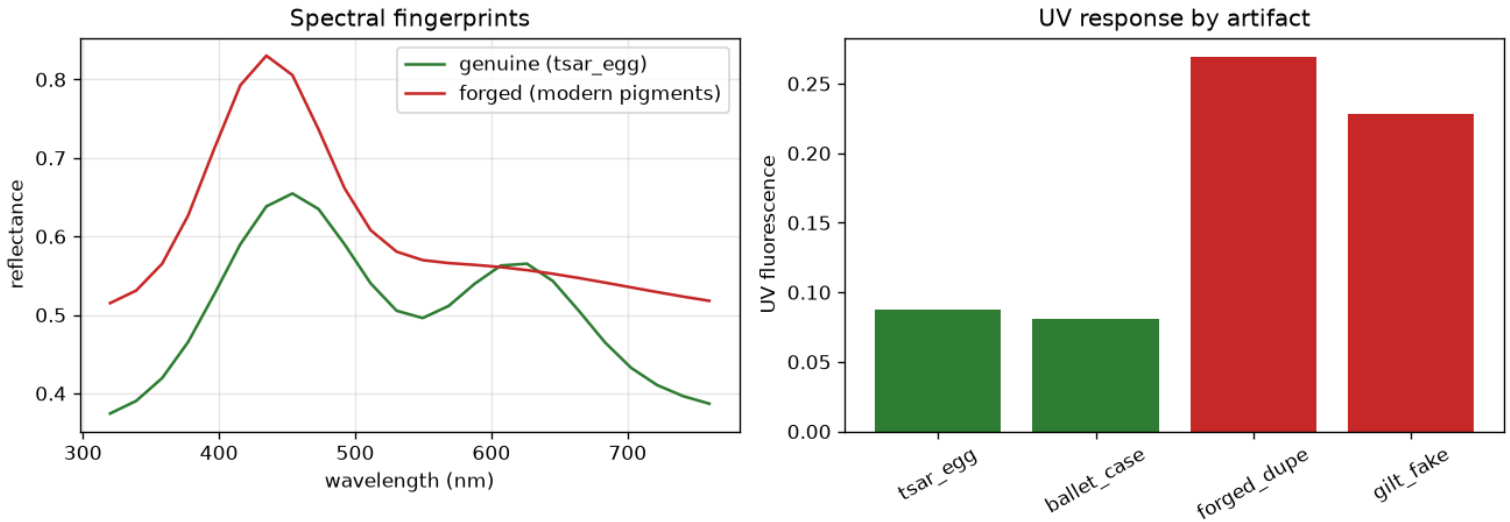


Figure 58: Spectral fingerprints of a genuine vs. a forged artifact and their UV fluorescence responses.

16.5.3 Varnish ageing (forgery depth)

This channel dates two surfaces drawn from the same exhibit the pigment screen sees: *tsar_egg*, whose natural shellac has been on the object since its claimed 1910 manufacture, and *forged_dupe*, whose acrylic coat the forger applied recently. Each surface is observed as its unvarnished substrate reflectance attenuated by the varnish film; the film’s transmittance is recovered by dividing out that reference, and the yellowing kinetics are then inverted for an age.

Read the two results separately, because they are not equally strong.

The fresh coat is rejected decisively. Its true age is 3 years; the fit returns about 2.8 years and the verdict is **too_young** at a claimed-era closeness of 0.024. That is not one lucky draw: re-observing the same surface under 32 independent noise realizations gives a spread of only 0.4 years, and 32 of 32 realizations reject the claimed provenance.

The century-old surface is not resolved at all. Its authored true age is 116 years, the fit returns about 155.5 years, and the single-draw verdict against its own claim is **too_old**. That verdict carries no information, and the ensemble is what shows it: across 32 realizations of the *same* surface the recovered age scatters by 54.4 years, spanning 83.1 to 299.8 years; only 12 of 32 land inside the ± 15 -year tolerance; and the ensemble produces 3 distinct verdicts (consistent $\times 12$, too_young $\times 4$, too_old $\times 16$) from one genuine object. Every outcome the classifier can emit is reachable from this surface by measurement noise alone, so which one the mission seed happens to draw says nothing about the object.

The cause is physical rather than numerical: the yellowing kinetics saturate, so past roughly three time constants a further decade of ageing shifts the spectrum by less than the measurement noise, and the age stops being identifiable. A false rejection of a genuine object is therefore as available to this channel as a false confirmation.

The honest summary of this channel is asymmetric: varnish ageing is a rejection instrument for recently re-varnished surfaces and is not a confirmation instrument for old ones. It reads film chemistry rather than pigment composition, so its rejection of the duplicate does not share the pigment screen’s failure mode; but both channels observe the same synthetic exhibit under the same authored ground truth, so they are independent in *observable*, not in *sample*.

That asymmetry is a property of the kinetics, not of one dataset. The varnish figure sweeps true coating age and shows the recovered age tracking truth until the film is past roughly three time constants — after which the recovered age scatters beyond any single reading, which is exactly why the century-old surface above emits every verdict the classifier can produce.

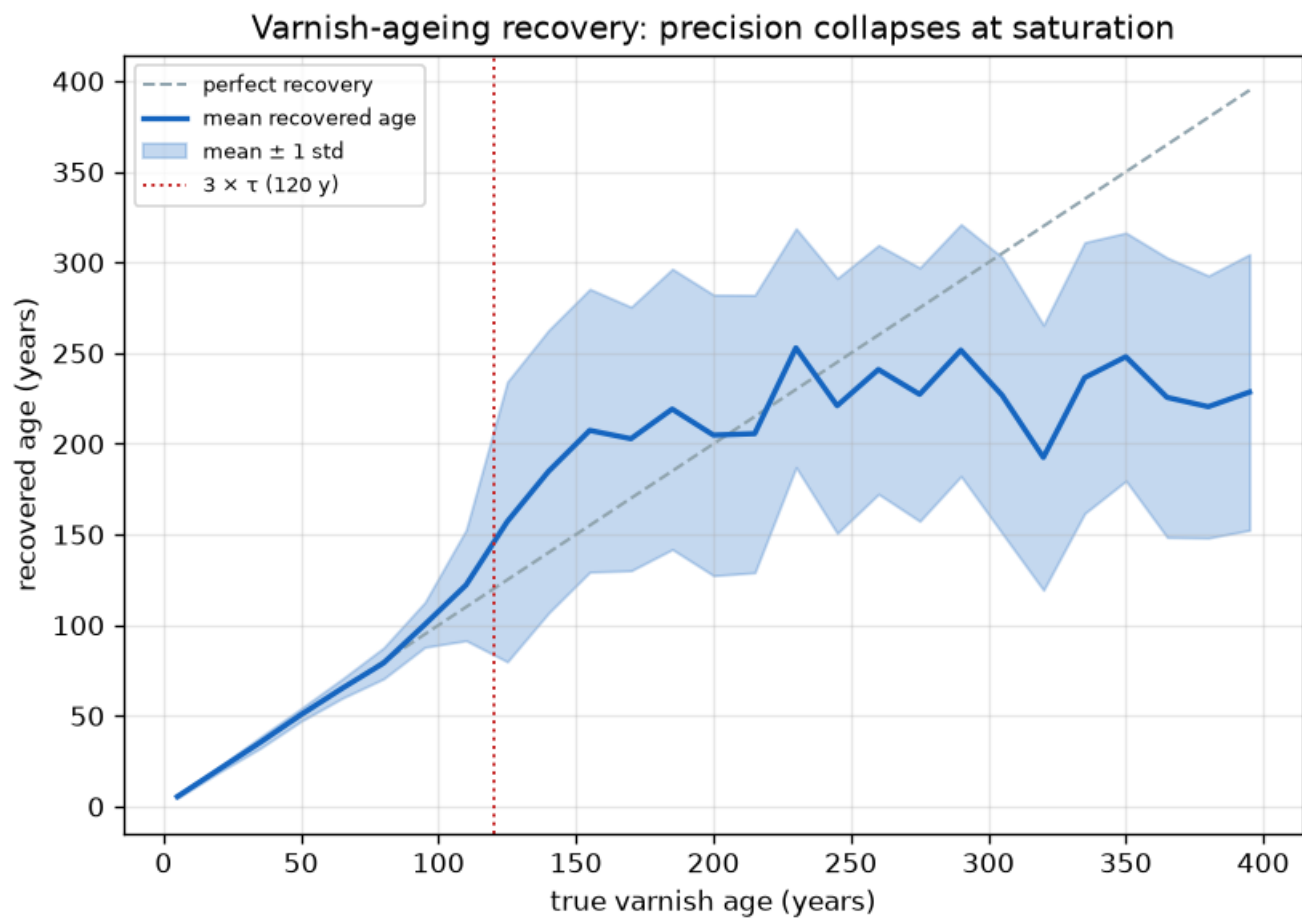


Figure 59: Recovered varnish age against true coating age, showing the precision collapse at saturation.

16.5.4 Cover logistics

Routing 3 payload units across the 9-segment, 8-station corridor from the Channel fringe to the target station, the min-cost flow resolves at a total cost of 780 unit-minutes with a makespan of 260 minutes. Against the 400-minute deadline that leaves 140 minutes of slack, carried by 2 circus trains at the capacity limit. The rail figure plots the corridor and the routed flow.

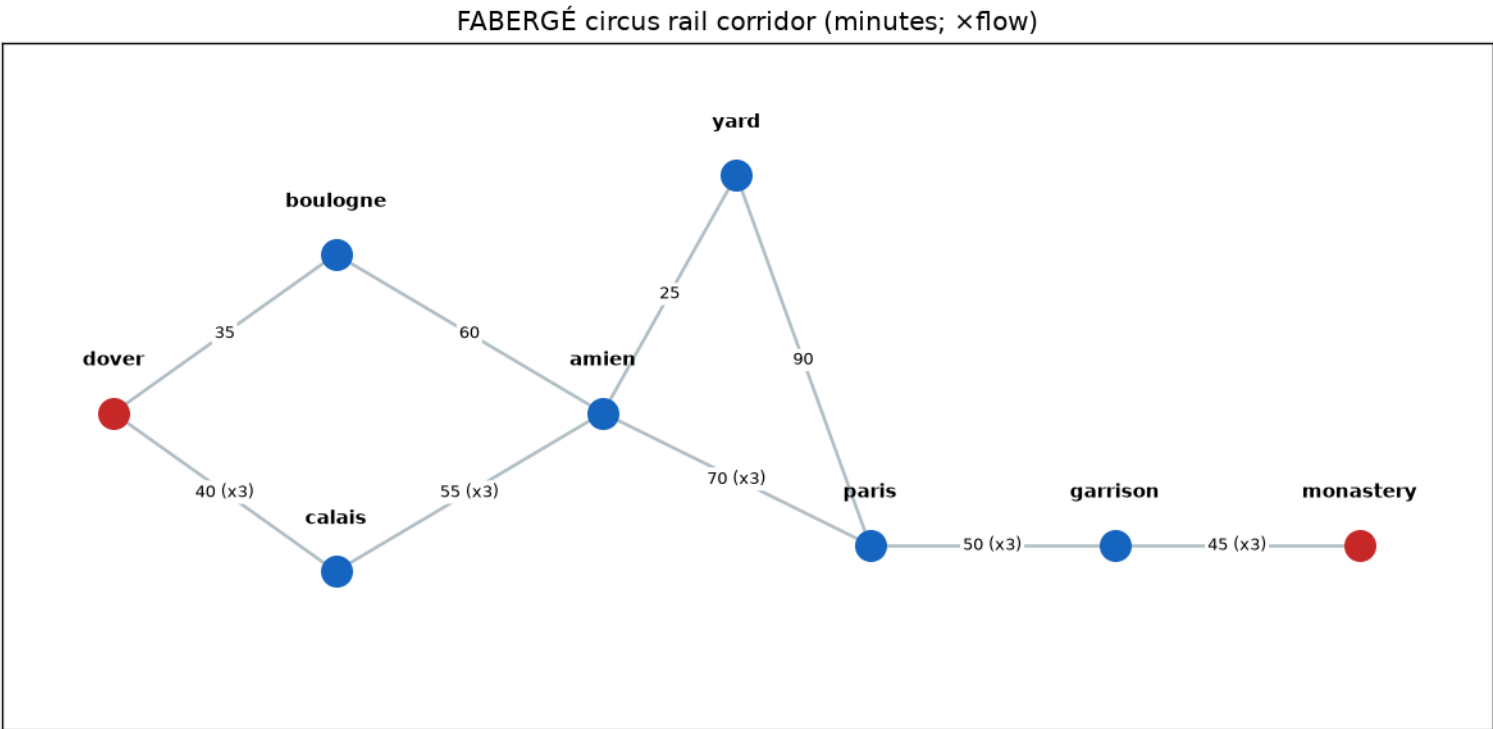


Figure 60: The FABERGÉ circus rail corridor with routed flow.

16.5.5 Rolling-stock circulation (logistics depth)

The schedule of 7 circus trips requires only 2 trainsets: a maximum successor matching of 5 links (minimum path cover) resolves the itineraries with 100 total deadhead minutes. The Kuhn–Munkres depot allocation reaches a minimum total of 38.0 minutes.

Those trips are a separate scenario from the min-cost flow, so the two pipelines only bear on each other if something compares them. The mission computes that comparison and reports it. Each of the 5 station-to-station legs the flow routes must appear as a scheduled circus trip — every routed leg is covered: True — and the trains the flow needs (2) must not exceed the trainsets the schedule fields (2): True. Both are computed from the live pipeline outputs, and either goes false if the route or the schedule changes. What the matching proves on its own is only the fleet size the circus schedule requires; it is these two checks, not the matching, that tie the cover to the routed plan.

16.5.6 Concealment sweep

The 100-detector sweep, laid out as a 10×10 lattice, flags hotspot@3,4, royal_egg. The fused report ranks hotspot@3,4 highest across 4 screened containers, confirming a dense sample egg as a flagged mass anomaly — the Fabergé-egg signature of a small, dense concealed core. The sweep figure renders the radiation count map with the detected hotspot.

16.5.7 Source localization (sweep depth)

Weighted Gauss–Newton localization on the same sweep data recovers the concealed source to within **0.066 m** of its true position in 11 iterations, estimating an intensity of 501.1 and a weighted residual of 28.664 (converged: True). The fit’s own approximate positional standard deviation is 0.102 m, so the achieved error is reported against the precision the detector geometry supports rather than in isolation. The sweep therefore both flags the device and fixes where it sits.

The intensity estimate is a different matter and should not be read as a calibrated activity. Against a true emitter strength of 800 the fit returns 501.1 — a substantial underestimate, while the position is recovered to centimetres. This is expected rather than anomalous: intensity, standoff, and the background level trade off against one another in an inverse-square fit, so the radial scale is poorly constrained by a single-plane lattice even when the centroid is sharp. The operational claim is therefore *where the source is*, not *how strong it is*.

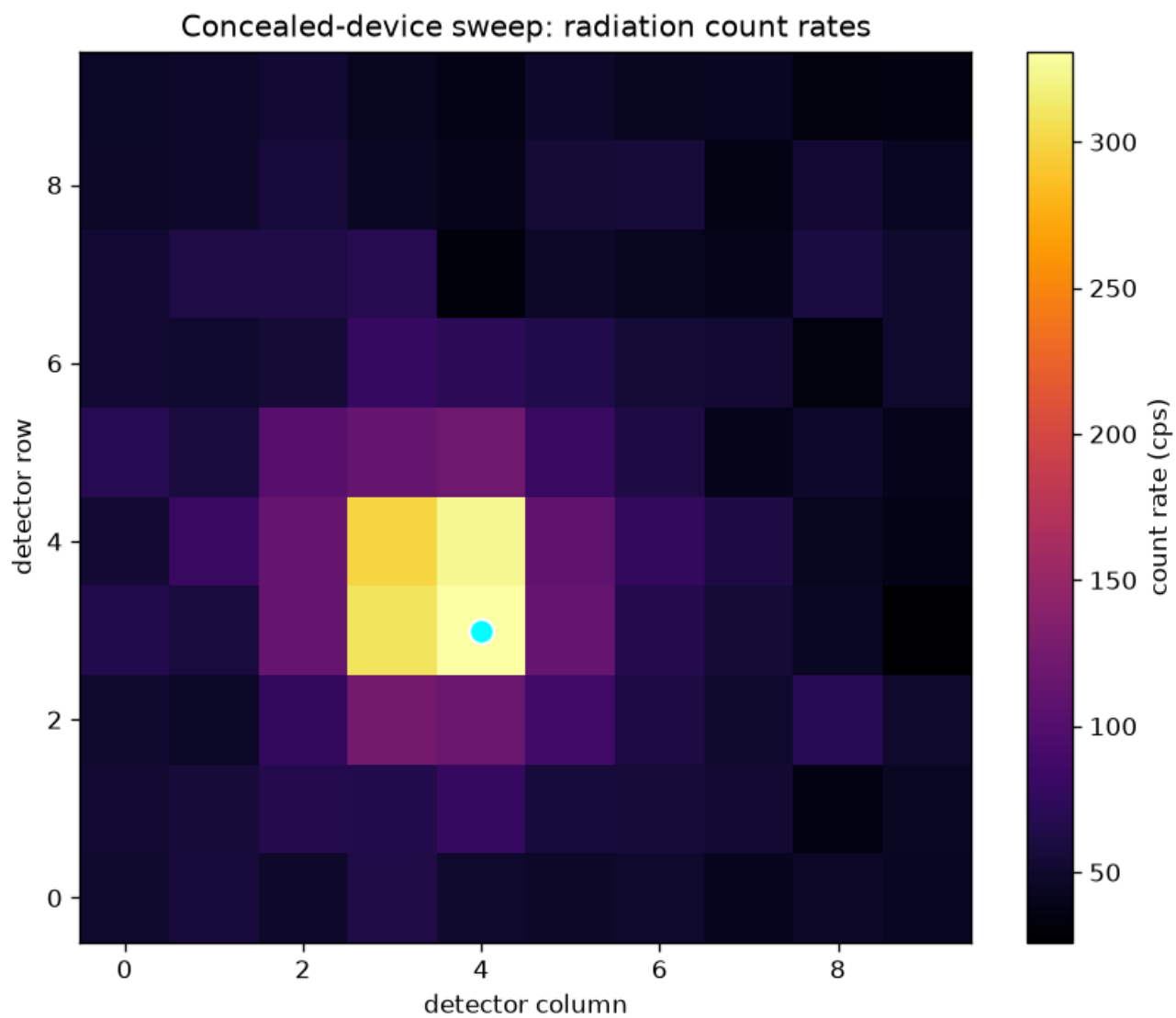


Figure 61: Concealed-device sweep: radiation count rates with the detected hotspot.

16.6 Conclusion — FABERGE: findings, verdict, and what the mission establishes

The **FABERGÉ** mission software demonstrates that three independent investigative disciplines — spectral/UV forgery detection, circus-cover logistics by min-cost flow, and fused radiation/mass-anomaly sweeping — can be implemented as pure, deterministic, infrastructure-free modules and assembled behind a single frozen mission-provider protocol.

Each discipline is answered twice, through observables that do not share a failure mode — and the second answer is worth exactly as much as its own error bar, which is the substantive finding here rather than the pairing itself.

Varnish-ageing kinetics date the surface by film chemistry and reject the forger’s fresh coat in every noise realization. On the century-old surface they decide nothing: the recovered age scatters by 54.4 years once the yellowing saturates, and the ensemble emits 3 distinct verdicts from that single genuine object, so the one the mission seed draws is noise. Read without that scatter, the same run reads as a second, independent verdict on authenticity; it is not one. Rolling-stock circulation establishes by matching that the circus schedule needs 2 trainsets, and two explicit cross-pipeline checks — every routed leg scheduled, the fleet at least as large as the trains the flow needs — connect that fleet to the routed plan. Weighted Gauss–Newton localization converts the sweep’s alarm into a position fixed to 0.066 m, against the positional standard deviation the geometry supports, while leaving the source intensity uncalibrated.

What holds across all 6 pipelines is narrower than mutual confirmation and more defensible: the forgery screen exposes the modern composition, the logistics plan routes the payload within cover and deadline, the cross-pipeline cover checks pass, and the sweep both flags the dense egg and fixes where it sits. All of it is measured on a synthetic scenario whose ground truth is authored in this package, so these are estimator-behaviour results, not field results. Each is reproducible under the fixed seed (7) and carries audit-ready provenance.

The approach generalizes. The same spectral-unmixing, network-flow, matching, and anomaly-plus-inversion primitives apply to any “hidden in plain sight” investigative problem — and so does the discipline that pairing a detector with a second observable is only worth something once the second observable’s uncertainty has been measured rather than assumed.

16.7 Experimental Setup — FABERGE: canonical scenarios, parameters, and configuration

All experiments are configured through `manuscript/config.yaml` and the module constants in `src/octopussy/`. The pipeline is deterministic: every random draw comes from `numpy.random.default_rng(seed)` with seed 7.

16.7.1 Scenario constants

Pipeline	Configuration
Forgery detection	14-pigment library over 24 UV-VIS bands (320–760 nm)
Forgery detection	Exhibit of 4 artifacts screened end-to-end
Varnish ageing	Exhibit surfaces <i>tsar_egg</i> (claimed 1910) and <i>forged_dupe</i> , dated on the same band grid
Varnish ageing	Authored true ages 116 y and 3 y; ±15 y consistency tolerance; 32 noise realizations per scatter
Cover logistics	9 directed rail segments across 8 stations
Cover logistics	3 payload units, 400-minute deadline
Rolling stock	7 scheduled circus trips
Concealment sweep	100 detectors on a 10×10 lattice
Concealment sweep	4 cargo containers against 6 density profiles

Every value in this table is injected from the live package surface rather than transcribed, so changing a scenario constant in `src/` changes this table on the next render.

16.7.2 Environment

Environment used for this report: Python 3.14.6. No generation timestamp is recorded: a wall-clock stamp used to be injected here and written into `output/data/manuscript_variables.json`, which made two regenerations differ and falsified the byte-identity claim in the reproducibility section. Determinism was judged the more useful property, and mission provenance already carries an `input_hash`. Versions of the scientific stack (numpy, matplotlib) are pinned through the package’s `pyproject.toml`; the frozen protocol dependency `bond-api` is resolved as a local path source.

The full run is driven by `scripts/mission_execute.py` (mission outcome and figures) and `scripts/z_generate_manuscript_variables.py` (token hydration and the resolved manuscript tree). Neither requires network access.

16.8 Reproducibility — FABERGE: verification gates, deterministic regeneration, and artifacts

Reproducibility is the central guarantee of this package (Octopussy v0.1.0).

- **Fixed seed.** Every pipeline draws only from generators seeded from 7; there is no global or untracked randomness. The three inverse solvers — the golden-section varnish fit, the Hopcroft–Karp matching, and the Gauss–Newton localization — take no random draws at all: each starts from a data-derived point and iterates in fixed index order.
- **No wall-clock in persisted artifacts.** The manuscript variable set emits no timestamp; the generation stamp that used to appear in `output/data/manuscript_variables.json` and in the experimental-setup section was removed precisely because it broke the byte-identity claim below. `PYTHON_VERSION` is the one remaining environment-derived value, so reproduction is byte-identical for a fixed interpreter version and differs in that one field across interpreters.
- **Live provenance.** Each mission outcome carries `package_version`, the fixed `seed`, and an `input_hash` — a sha256 fingerprint of the mission’s defining inputs (codename, objectives, and plan) — so an outcome can be audited without re-running the film.
- **Generated artifacts.** Figures and manuscript variables are regenerated from source by `scripts/mission_execute.py` and `scripts/z_generate_manuscript_variables.py`; none are hand-maintained snapshots. On a fixed interpreter, regenerating leaves a byte-identical tree — asserted, not asserted-about, by `tests/test_manuscript_variables.py::test_regeneration_is_byte_identical`, which runs the generator twice into separate directories and compares every emitted file byte for byte.
- **No hand-authored metrics.** Every number in the abstract, results, and conclusion arrives as an injected manuscript token resolved from `src/octopussy/manuscript_variables.py` against a live run. A test asserts both directions of that contract: no section may use a token the generator does not emit, and no generated token may go unreferenced.
- **No mocks.** Tests use real computation only; deterministic reference providers implement the real BOND-API protocol.

Report artifacts live under `output/` (regenerated): the mission outcome, the rendered figures, and the resolved manuscript tree.

16.9 Scope and Related Work — FABERGE: boundaries, positioning, and relationship to the literature

16.9.1 Scope

This package models the three FABERGÉ disciplines with deterministic, real-data implementations across 6 pipelines. It does **not** claim physical fidelity to real forensic spectrometers, real rail networks, or real scintillation detectors: the underlying models are physically motivated but illustrative. The 14-pigment library, the 9-segment corridor, and the 6-category container catalog are deterministic fixtures, not measured instrumentation.

These limits follow and should be read with the results:

- **The exhibit is small and self-authored.** Separation between classes is measured on 4 artifacts drawn from the same library the detector fits against; it is a demonstration of the method’s mechanics, not an estimate of field accuracy. There is no genuine period piece using a pigment near the anachronism boundary, so the false-positive rate is untested.
- **The hard anachronism regime is not exercised.** The exhibit’s least-modern fake is still 50% modern by weight. Nothing here speaks to resolving a trace of modern material out of an otherwise period-correct object, which is the case that would actually test the matched-filter channel.
- **Ground truth is authored, so these are estimator results.** Every number reported here is measured on a synthetic scenario defined in this package — the pigment mixtures, the true varnish ages, the emitter position, the rail schedule. The results characterize how the estimators behave against a known truth; they are not measurements of any physical object, and headline and depth modules share that one scenario. The pairing gives independence of observable, not independence of sample.
- **The varnish kinetics use one time constant, and they saturate.** Real varnish ageing depends on resin, illumination history, and storage; a single τ is a first-order stand-in. More consequentially, the saturating exponential means that past roughly three time constants a further decade of ageing moves the spectrum by less than the measurement noise. As the results section reports, the recovered age of the 116-year surface scatters by 54.4 years across 32 noise realizations — wider than the ± 15 -year tolerance it is judged against, and wide enough to emit 3 distinct verdicts from that one genuine object. This channel can reject a recent re-varnish; on an old surface it produces a verdict with no information content, in either direction, and none of its old-surface verdicts should be reported as evidence.
- **The two cover pipelines are linked by an explicit check, not by construction.** The circus schedule is a fixture independent of the min-cost flow. The results section reports two computed comparisons that connect them; absent those, the matching would bound the circus fleet and say nothing at all about the routed corridor plan.
- **The sweep model is idealized.** Counts are Poisson about an inverse-square field with no scattering, no attenuation through cargo, and no energy-resolved spectroscopy — the discriminants a real portal monitor depends on most.
- **Localized intensity is not calibrated activity.** As the results show, the fit recovers position to centimetres while substantially underestimating emitter strength: intensity, standoff, and background trade off in an inverse-square model, and a single-plane detector lattice constrains the radial scale weakly. Read the localization output as a position, not as a source-strength measurement.

16.9.2 Related work

Non-negative least-squares spectral unmixing [Lawson and Hanson, 1974a] and matched filtering are standard in hyperspectral remote sensing [Keshava and Mustard, 2002] and, in the reflectance regime, in cultural-heritage science for pigment identification; this package applies the same mathematics to the anachronism test. UV examination of varnish fluorescence is long- established

conservation practice [de la Rie, 1982], and the ageing behaviour of natural resin films is documented in the conservation-materials literature [Feller, 1994, Horie, 2010].

Minimum-cost flow via successive shortest paths is a classical combinatorial-optimization result [Ford and Fulkerson, 1962, Ahuja et al., 1993a] built on the shortest-path algorithms of Bellman [Bellman, 1958] and Dijkstra [Dijkstra, 1959a]. Deriving a minimum fleet size from a maximum matching goes back to Dantzig and Fulkerson [Dantzig and Fulkerson, 1954] and remains the standard reduction in railway timetabling [Caprara et al., 2002]; the matching itself is Hopcroft–Karp [Hopcroft and Karp, 1973] and the depot assignment is Kuhn’s Hungarian method [Kuhn, 1955].

Poisson background estimation with z-score anomaly detection is standard in radiation portal monitoring [Knoll, 2010], with decision thresholds in the tradition of Currie [Currie, 1968]; damped Gauss–Newton inversion for source parameters follows standard nonlinear least-squares practice [Marquardt, 1963, Nocedal and Wright, 2006].

The contribution here is not any individual algorithm but the assembly: each discipline answered by two methods with disjoint failure modes, behind a single frozen mission protocol, with deterministic and audit-ready provenance — and with each depth method reporting the uncertainty that determines how much its agreement is worth.

16.10 Sources — FABERGE: bibliography

Lawson and Hanson [1974a]; Keshava and Mustard [2002]; de la Rie [1982]; Feller [1994]; Horie [2010]; Ford and Fulkerson [1962]; Ahuja et al. [1993a]; Bellman [1958]; Dijkstra [1959a]; Dantzig and Fulkerson [1954]; Caprara et al. [2002]; Hopcroft and Karp [1973]; Kuhn [1955]; Knoll [2010]; Currie [1968]; Kiefer [1953]; Marquardt [1963]; Nocedal and Wright [2006]

17 A View to a Kill (1985) — MAIN STRIKE

film package · package codename MAIN STRIKE. Mission **ZORIN**: map microchip supply-chain monopoly choke points; optimize the budgeted acquisition that corners the market; model flood-the-mine water-ingress dynamics; couple the flood to surface-fab capacity loss in the valley; score horse-race form analytics for the cover operation; price the cover’s betting-market position and staking.

17.1 Concepts — MAIN STRIKE: domain and operational focus

microchip supply chain, urban sabotage, racing analytics

17.2 Abstract — MAIN STRIKE: mission summary

A View to a Kill (1985) · mission codename ZORIN This package implements **A View to a Kill: ZORIN — Special-Agent Mission Software** as a deterministic, fully-tested special-agent mission model. It operationalizes the film’s central scheme through six pure, infrastructure-free domain modules: 1. **Microchip supply-chain security graph** (`chip_supply_chain.py`) — a directed graph of the mid-1980s semiconductor market (12 firms, 18 supply links across four stages). Concentration (HHI), Brandes betweenness choke points, single-source dependence, and cascade failure quantify the monopoly attack. 2. **Monopoly acquisition optimizer** (`acquisition.py`) — the budgeted selection problem of which independent fabricators ZORIN buys to corner the market. Exhaustive search for the optimal acquisition (`intel_fab`, `nec_fab`) raises market-wide HHI from a 1012.8 baseline to 1741.4 with a budget of 2, a gain of 728.6. The set the film-faithful scenario buys (`amd_fab`, `nec_fab`) is a different set scoring 1528.9; the scheme is suboptimal by 212.5 HHI points, and we report that rather than claim optimality for it. 3. **Flood-the-mine water-ingress dynamics** (`mine_flood.py`) — a mass-conserving deterministic Euler simulation of water flowing through the 8-chamber “Main Strike” network, flooding 0.625 of it by 900.0 s with a water-balance error of $1.2e-15$ (machine precision). 4. **Valley impact / urban sabotage** (`valley_impact.py`) — couples the flood to the fabs above: rising groundwater disables 3 of 5 valley fabs, 0.583 of fabricating capacity. ZORIN’s own plant is among the 2 survivors (`nec_fab`, `zorin_fab`); the other survivor is `nec_fab`, which he acquires, so the number of surviving plants he neither owns nor acquires is 0. The bluntness runs the other way: `amd_fab`, a plant ZORIN buys, is destroyed by his own flood. 5. **Horse-race form analytics** (`race_analytics.py`) — normalized speed indices, EWMA ratings, softmax win probabilities, and a signal-detection cover-risk model. A subtle 0.2 speed-index boost to Pegasus carries only 0.001 detection risk (expected value 2924); a blatant boost is caught. 6. **Racing betting-market / cover-finance** (`race_market.py`) — converts the softmax win probabilities to honest-market decimal odds under a 1.12 overround, and prices the cover: Pegasus’s boosted true probability (0.248) exceeds the market-implied (0.238), yielding a positive value edge (0.0415) and a Kelly stake of 1298 — the exact anomaly to detect. Every metric is computed, not asserted: figures, manuscript tokens, and the BOND-API mission provider all draw from the same pure core (seed 1985, fully deterministic). Keywords: microchip supply-chain security, monopoly choke points, herfindahl-hirschman index, betweenness centrality, budgeted acquisition optimization, water-ingress dynamics, mine flooding simulation, groundwater inundation damage, horse-race form analytics, cover-operation modeling, signal-detection theory, kelly criterion, betting-market pricing, reproducible research, deterministic simulation.

17.3 Introduction — MAIN STRIKE: mission framing, the operational problem, and how to read this chapter

In *A View to a Kill* (1985), industrialist Max ZORIN plots to flood the abandoned mine workings beneath California’s Silicon Valley, destroying every competing microchip fabrication plant and leaving his own operation as the valley’s sole surviving manufacturer. His scheme has three load-bearing pieces: a **monopoly** over the microchip supply chain, an **infrastructure sabotage** (the flood) that removes the competition, and a **cover** operation (a winning racehorse) that draws attention and capital away from the true plan.

This package turns each piece into real, deterministic, testable mathematics. It is a PROJECT BOND film package: it implements the FROZEN `bond_api.MissionProvider` protocol (the mission lifecycle of brief, recon, plan, execute, debrief) around a pure domain core that is deliberately free of protocol imports, so the science stands alone and the protocol is a thin adapter.

17.3.1 The six models

Each of the three load-bearing pieces is developed twice: once as the *structure* of the problem and once as the *decision* the structure forces.

- **Supply-chain security graph** (`chip_supply_chain.py`). Microchips flowed through a multi-stage market in 1985 — silicon wafer suppliers, fabrication fabs, assembly and packaging houses, and distributors. ZORIN’s monopoly is the classic choke-point problem: which few firms, if acquired or eliminated, corner the market. We formalize this with capacity market shares, the Herfindahl-Hirschman concentration index, betweenness centrality, and a supply-cascade failure model.
- **Monopoly acquisition optimizer** (`acquisition.py`). Given the graph, the monopoly is a *budgeted selection* problem: with money for K purchases, which independent fabricators does ZORIN buy? We solve it exhaustively (the optimum, with deterministic tie-breaking) and greedily (the naive baseline), and report the regret between them.
- **Flood-the-mine dynamics** (`mine_flood.py`). A mine is a network of chambers and tunnels; a sabotage flood is water entering that network and propagating downhill under a head-driven flow law. We simulate the water-ingress dynamics deterministically and verify exact mass conservation.

- **Valley impact** (`valley_impact.py`). The flood is only the mechanism; the sabotage is what it reaches. Each surface fabrication plant is hydraulically coupled to a mine chamber, so rising chamber head raises the water table under the fab until its foundation is inundated. This converts the flood curve into lost fabricating capacity — and shows which plant survives.
- **Race-form cover analytics** (`race_analytics.py`). The cover is a horse whose form he can knowingly improve. We score legitimate form from raw race records, convert ratings to win probabilities, and model the risk that the manipulation is detected — a signal-detection trade-off between a bigger win probability and a higher chance of being caught.
- **Cover finance** (`race_market.py`). A cover that cannot be monetized is not a cover. We price the manipulated horse against an *honest* bookmaker’s book under an overround, compute the value edge, and size the position by the Kelly criterion — which is also exactly the statistical signature a regulator would look for.

All mathematics is deterministic (no random draws), all data is fixed and documented as model parameters, and every result surface — figures, manuscript tokens, and mission outcomes — is computed from the same pure core.

17.3.2 Reader’s guide

`chip_supply_chain.py`, `acquisition.py`, `mine_flood.py`, `valley_impact.py`, `race_analytics.py`, and `race_market.py` (all under `src/a_view_to_a_kill/`) are the mathematical core: pure functions with no protocol imports, no I/O, and no randomness. `mission.py` adapts them to the BOND-API contract — it is the only module that imports `bond_api`. `manuscript_variables.py` computes the manuscript variable tokens that fill this document, and `figures/plots.py` renders the five figures. The methods section develops all six models; The results section reports their measured outputs; The scope section states what the models do and do not claim.

17.4 Methodology — MAIN STRIKE: the analytical models and algorithms that drive the mission

All six models are pure functions of fixed, documented datasets; no random draws, no external state, no wall-clock dependence. The manuscript pipeline that consumes them reads no clock either, so on one pinned interpreter every rerun writes byte-identical artifacts — a checked property, not an aspiration (the *Reproducibility* section names the gate). The three *structural* models come first (supply-chain graph, flood dynamics, race form); the three *decision* models that build on them follow (acquisition optimizer over the graph, valley-impact coupling over the flood, cover finance over the form ratings).

17.4.1 Implementation layering

The six models live in six independent modules under `src/a_view_to_a_kill/`. A seventh module, `metrics.py`, fixes the mission scenario (which firms are acquired, the flood parameters, the cover boost, the bankroll) and evaluates each model once into a named metric group. `metrics.py` imports no protocol types, so the entire scientific pipeline — the models, the metric groups, the figures, and the manuscript token hydration — runs without `bond_api` installed. `mission.py` is the only module that imports the protocol; it wraps the same metric groups in the FROZEN `MissionProvider` lifecycle and registers them as gadgets. Section results are therefore identical whether reached through the protocol or through the pure core.

17.4.2 Microchip supply-chain monopoly analysis

The supply chain is a directed graph of firms V (nodes) and supply links $E \subseteq V \times V$ (edges flowing forward through stages `waf er`, `fabrication`, `assembly`, `distribution`). Each firm v has a throughput capacity c_v . The default dataset is a documented reconstruction of the mid-1980s semiconductor market (12 firms, 18 links).

Concentration. Within a stage (or the whole graph), firm v ’s capacity share is $s_v = c_v / \sum_u c_u$, and the Herfindahl-Hirschman index is $\text{HHI} = 10000 \sum_v s_v^2$, ranging from 0 (perfect competition) to 10000 (monopoly) — the antitrust codification of concentration [U.S. Department of Justice and Federal Trade Commission, 2010]. The default market opens at HHI 892.2.

Choke points. Structural choke points are measured with Brandes’ betweenness-centrality algorithm [Brandes, 2001a]: a firm’s betweenness is the fraction of ordered node pairs for which it lies on a shortest directed supply path. Wafer sources carry zero betweenness by construction; the top choke point is `amkor` (see the choke points figure).

Single-source dependence. A downstream firm whose only upstream supplier is one firm is a single point of failure; such dependencies are the monopolist’s pressure points.

Cascade failure. Removing a set R of firms (acquired-and-dismantled competitors), a downstream firm is *starved* if it has no directed supply path from any surviving wafer supplier — a standard reachability construction on the supply digraph [Ahuja et al., 1993b], and the mechanism the resilience literature calls upstream disruption propagation [Christopher and Peck, 2004]. The starved fraction is $\sum_{v \in S} 1/|V|$. In the reported scenario R is exactly the acquired set `amd_fab`, `nec_fab`; the fabricators that remain in the graph after that removal are `intel_fab`, `zorin_fab`.

The ZORIN attack. ZORIN owns `zorin_fab` and `zorin_assembly`; by acquiring the independent fabricators `amd_fab`, `nec_fab` and eliminating the rest of the market, his fabrication share rises to 0.680 and market-wide HHI jumps to 1528.9. Formally, `monopoly_share` computes ZORIN’s surviving capacity share per stage once every non-acquired independent is removed, and `post_elimination_hhi` merges ZORIN’s holdings into one entity before recomputing concentration. This acquisition set is a *scenario* input (`manuscript/config.yaml`), not the optimizer’s output; The results section reports how far apart the two are.

17.4.3 Flood-the-mine water-ingress dynamics

The mine is a network of chambers (nodes) and bidirectional tunnels (edges). Chamber i has volume Ω_i , horizontal cross-sectional area A_i , and elevation z_i ; water level is $h_i = z_i + w_i/A_i$ where w_i is the water volume in the chamber. Tunnels have cross-sectional area a_{ij} .

Water enters at designated ingress chambers at total rate Q (split evenly), and flows through each tunnel from the higher-head end to the lower under a simplified weir law

$$q_{ij} = C a_{ij} \sqrt{\max(h_i - h_j, 0)},$$

with discharge coefficient $C = 0.60$. Each timestep’s tentative flows are scaled so no chamber lends more water than it holds, making the scheme **exactly mass-conserving**. State evolves by explicit Euler with timestep 1.0 s to horizon 900.0 s at ingress 10.0 m³/s. A chamber is *flooded* once $w_i \geq \Omega_i$; `flooded_fraction` reports the share of chambers flooded at a time t . `critical_ingress` picks the single entry that floods the most of the network by the horizon. The default “Main Strike” network has 8 chambers and 10 tunnels totalling 9100 m³ (see the flood curve figure).

17.4.4 Race-form analytics and cover model

Raw race records are normalized to a seconds-per-furlong pace corrected for going (firm/soft) and weight carried: $\tau = (t - g d - \max(w - 126, 0) \epsilon d)/d$, with $\epsilon = 0.02$ s per furlong per pound over the base weight. The speed index is the seconds saved per furlong versus par, $14.0 - \tau$. A horse’s form rating is the exponentially-weighted moving average of its speed indices, where a race **age** before the newest is weighted $0.5^{\text{age}/\text{halflife}}$ (halflife 2.0). Win probabilities follow a softmax over ratings.

Cover risk. The stewards flag a horse whose performance improves by more than $\theta = 1.0$ speed-index units above its own baseline. With n races and natural race-to-race variability σ , the standard error of the mean performance is $\sigma/\sqrt{n}$, so applying a boost b gives a detection risk

$$\text{risk}(b) = \Phi\left(\frac{b - \theta}{\sigma/\sqrt{n}}\right),$$

The probability a normal draw with mean b clears the threshold — the detection-theoretic reading of stewards’ scrutiny [Green and Swets, 1966]. `cover_viability` returns `win_delta`, `detection_risk`, and `expected_value = win_delta * PURSE - risk * DETECTION_PENALTY`, with `PURSE = 100000` and `DETECTION_PENALTY = 900000` (both fixed in `race_analytics.py`). For the 0.2 boost to Pegasus (with $\sigma = 0.5$) the risk is 0.001 and expected value 2924 — a sustainable cover — whereas a blatant boost is caught (see the race cover figure).

17.4.5 Monopoly acquisition optimizer

The monopoly is a *selection* problem too: with a budget of K acquisitions (the acquisition curve figure), which independent fabricators should ZORIN buy? The acquireable set is the `amd_fab`, `intel_fab`, `nec_fab` fabricators (owners other than ZORIN). Each budget- K subset is scored by the objective `post_elimination_hhi(chain, acquired)` (or by ZORIN’s fabrication share), and `optimal_acquisition` exhaustively searches over every K -subset with deterministic tie-breaking. `greedy_acquisition` is the hill-climbing baseline that adds the single firm giving the largest score gain at each step. Because the exhaustive search is optimal, `greedy` $\leq$ `optimal` always; in fact both objectives are *strategically monotone* — the marginal gain of adding any candidate is its capacity times a factor common to all candidates — so `greedy` = `optimal` holds on every market, not merely on this dataset (verified exhaustively in `test_acquisition.py`); with the default budget of 2 the optimal acquisition (`intel_fab`, `nec_fab`) lifts HHI from 1012.8 to 1741.4, a gain of 728.6 over buying nothing, while the greedy baseline reaches 1741.4 (regret 0.0). The same optimality argument bounds the *scenario* acquisition (`amd_fab`, `nec_fab`) from above, which is how The results section scores the scheme’s shortfall; the optimizer is a bound on the scenario, not a description of it.

17.4.6 Valley impact — the flood’s urban blast-reach

ZORIN’s sabotage is not the flood but its *consequence*. Each surface fabrication plant (a `SurfaceFab`) is hydraulically coupled to a mine chamber; water in that chamber raises the local water table by the fab’s `leakage` coefficient, and the fab becomes inoperable when that table reaches its `foundation_elevation` — the coarse form of the groundwater-inundation damage relation [Kreibich and Thielen, 2008]:

$$\text{table}_f(t) = \ell_f \cdot h_{c(f)}(t), \quad \text{inoperable if } \text{table}_f(t) \geq z_f.$$

`valley_capacity_loss` reports the fraction of total fabricating capacity inoperable by a decision time — for the default scenario 3 of 5 fabs (0.583 of capacity) by the horizon (see the valley impact figure). The 2 survivors are `nec_fab`, `zorin_fab`: `zorin_fab` endures because it sits over the low-coupling, high-separation `shaft_a`, which is the outcome the scheme intends. The survivor and victim sets are determined purely by coupling geometry, not by ownership, and the two directions come out differently: no plant outside ZORIN’s control survives (0 such survivors), but `amd_fab` — a plant he acquires — is destroyed anyway.

17.4.7 Racing betting market and cover finance

The cover is financed by betting on the controlled horse. `cover_bankroll` takes the EWMA **form ratings** (a boost is in speed-index units, so it can only be applied to a rating) and derives both sides of the trade from the same softmax. The honest book is priced off the *un-boosted* win probabilities $p_i = \text{softmax}(r)_i$: `market_odds` converts them into decimal odds under a bookmaker overround $\nu = 1.12$ — the standard way an outcome model is priced against a book [Dixon and Coles, 1997] — so the market’s implied probabilities sum to ν : $q_i = p_i \nu$, $\text{odds}_i = 1/q_i$. Against that *honest* book, `cover_bankroll` prices the manipulated horse using its boosted true probability $p^* = \text{softmax}(r + be_{\text{Pegasus}})_{\text{Pegasus}}$, so the edge measures the boost rather than a change of probability model: the value edge is $p^* \text{odds} - 1$, the Kelly-optimal stake fraction is $(p^* \text{odds} - 1) / (\text{odds} - 1)$ when positive, and the expected profit is the edge times the staked bankroll. For Pegasus under a 0.2 boost the true probability (0.248) exceeds the market-implied (0.238), giving a positive edge (0.0415) and a Kelly stake of 1298 — precisely the anomaly that the cover-risk model of `race_analytics` would eventually flag [Kelly, 1956].

17.5 Results — MAIN STRIKE: measured outcomes, headline numbers, and what they establish

All figures and numeric results below are generated from the pure core by `scripts/z_generate_manuscript_variables.py`; no number is hand-authored.

17.5.1 Supply-chain monopoly

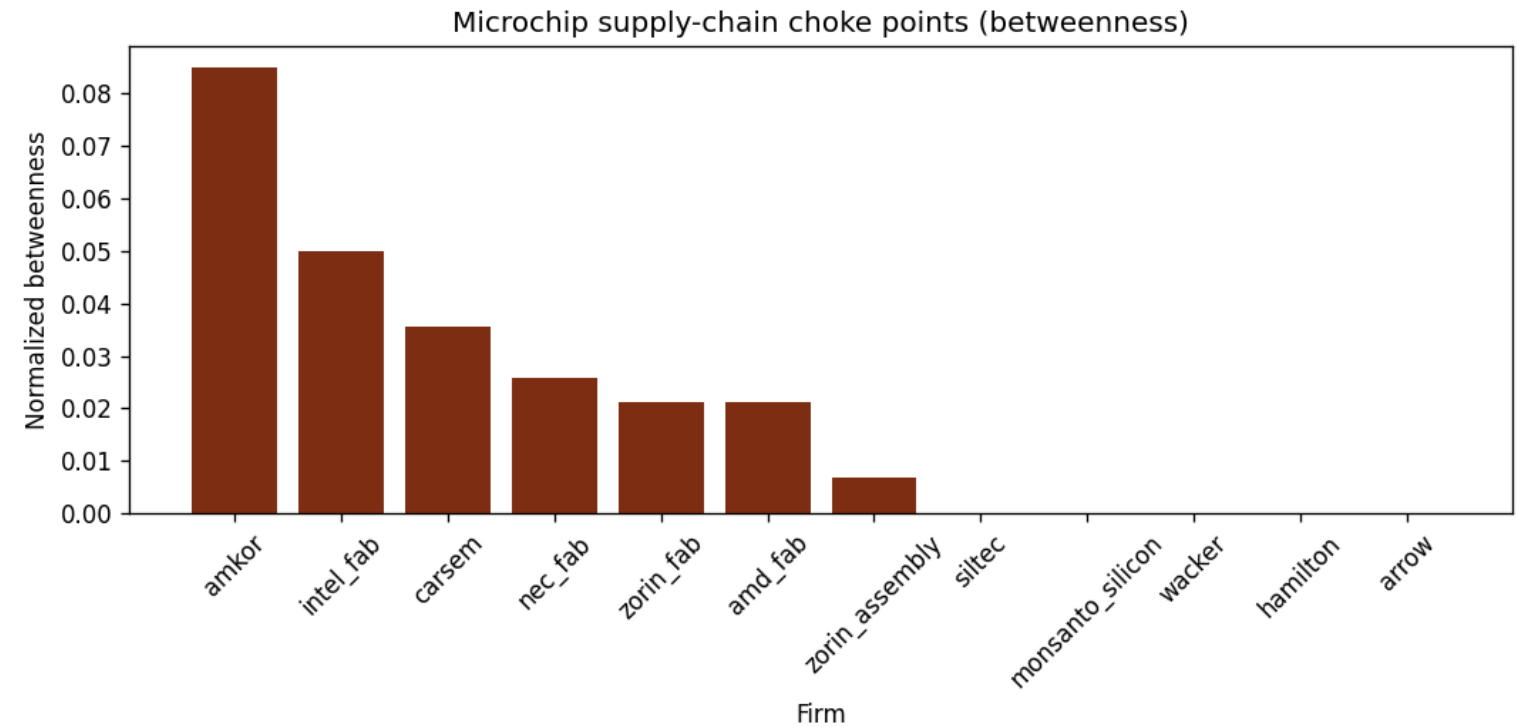


Figure 62: Choke-point betweenness by firm

Table 1: Supply-chain concentration under the ZORIN attack.

Metric	Value
Firms / supply links	12 / 18
Market-wide HHI (pre-attack)	892.2
Market-wide HHI (post-elimination, 2 fabs acquired: <code>amd_fab</code> , <code>nec_fab</code>)	1528.9
ZORIN fabrication capacity share	0.680
Top choke point (betweenness)	<code>amkor</code>
Starved fraction when the acquired fabs are dismantled (<code>amd_fab</code> , <code>nec_fab</code>)	0.000

Acquiring `amd_fab`, `nec_fab` lifts market concentration from HHI 892.2 to 1528.9 and hands ZORIN 0.680 of fabrication capacity — a decisive corner on the market. The cascade model removes exactly the acquired-and-dismantled firms (`amd_fab`, `nec_fab`); the fabricators left standing (`intel_fab`, `zorin_fab`) still carry wafer supply through to every assembly house, so nothing downstream

is starved (starved fraction 0.000, starved firms: none). That is the monopolist’s intent: competitors are *removed*, but the market ZORIN owns keeps flowing (the choke points figure).

17.5.2 Flood-the-mine water ingress

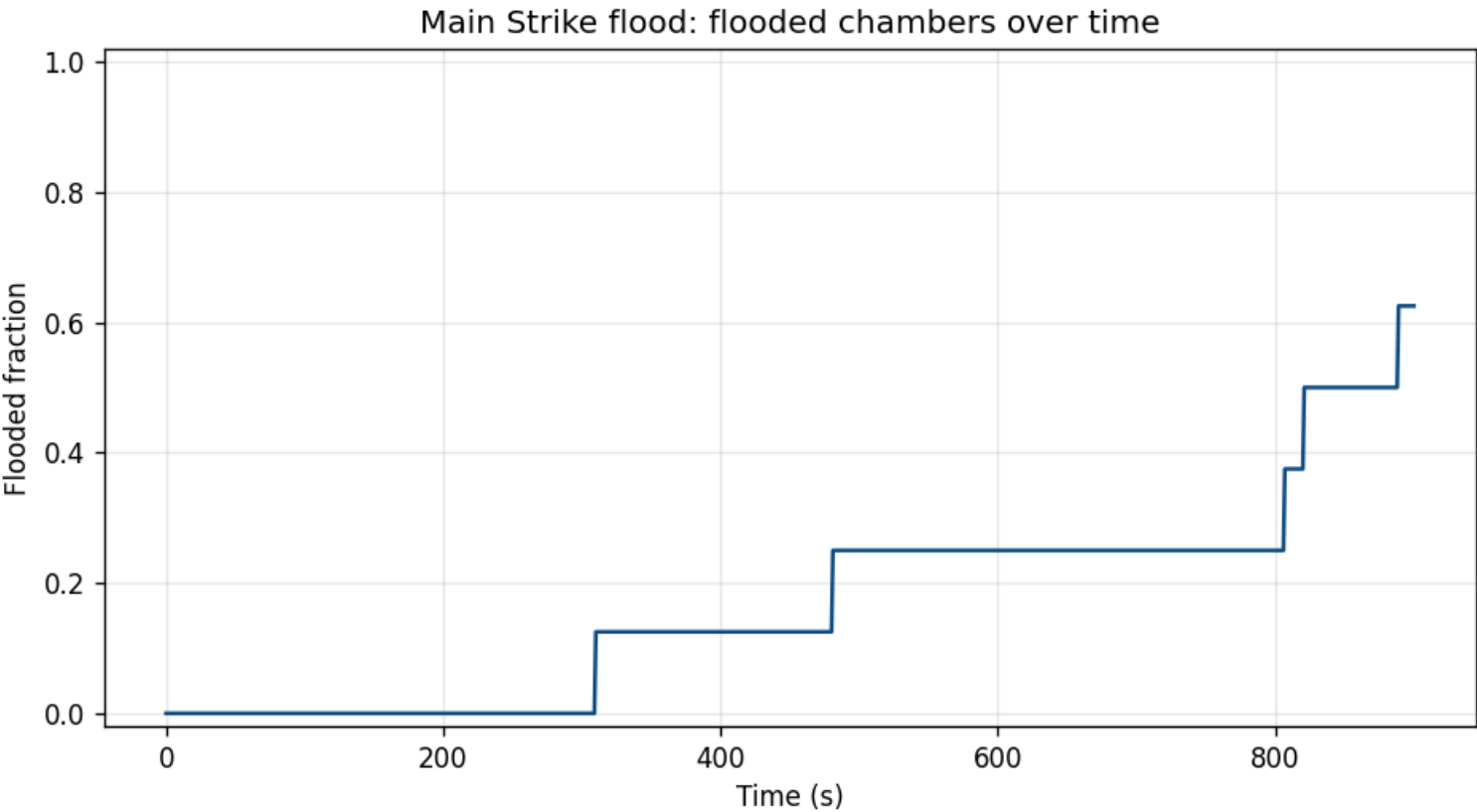


Figure 63: Flooded fraction of Main Strike over time

At 10.0 m³/s into the `adit`, the Main Strike network (8 chambers, 10 tunnels, 9100 m³) floods 0.625 of its chambers by 900.0 s; the lowest working (`vault`) is submerged at 310.0 s (the flood curve figure). The critical single ingress point is `adit`. The simulation conserves water mass to 1.2e − 15 — machine precision.

17.5.3 Race-form cover analytics

Over 20 records (5 horses, 2.0 halflife), the strongest form is `Silverado` (form rating 0.808); Pegasus opens at win probability 0.213 (the race cover figure). A 0.2 speed-index boost to Pegasus buys win probability +0.035 while carrying only 0.001 detection risk, for a positive expected value of 2924 — a sustainable cover. Pushing the boost toward the detection threshold collapses expected value as the scheme is caught (see the methodology analysis in The methods section).

17.5.4 Monopoly acquisition

Table 2: Acquisition optimizer on the default market (budget 2).

Metric	Value
Acquireable independent fabricators	amd_fab, intel_fab, nec_fab
Baseline HHI (buy nothing)	1012.8
Optimal acquired	intel_fab, nec_fab
Optimal HHI	1741.4
HHI gain vs baseline	728.6
Greedy baseline HHI	1741.4
Greedy regret	0.0

The exhaustive optimum (the acquisition curve figure) lifts concentration from 1012.8 to 1741.4 with just 2 acquisitions. Greedy is exact here (regret 0.0) — and this is *not* a happy accident of this dataset. Both acquisition objectives are *strategically monotone*: the marginal gain of adding any fabricator is that fabricator’s capacity times a factor common to every candidate, so greedy is provably

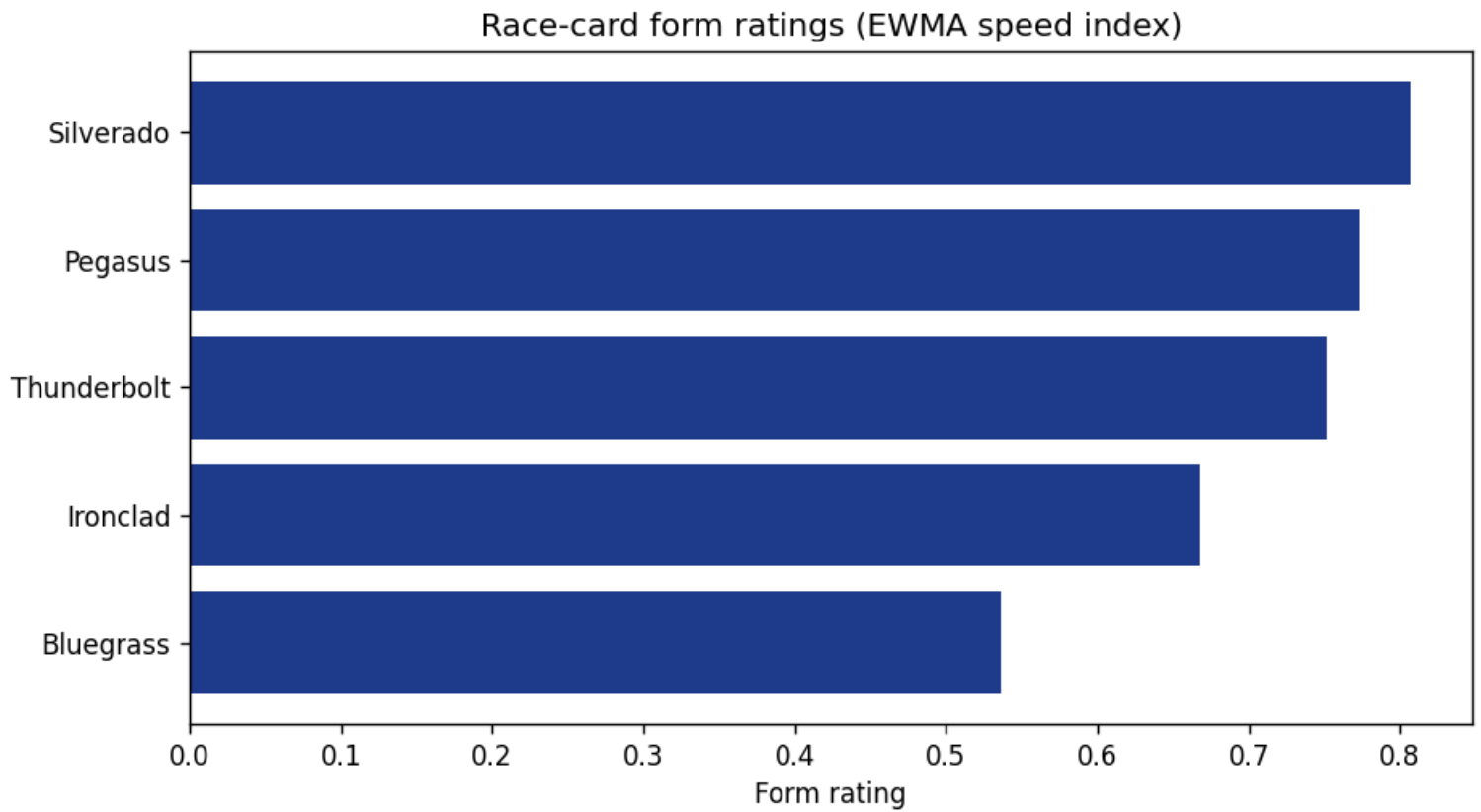


Figure 64: Race-card form ratings by horse

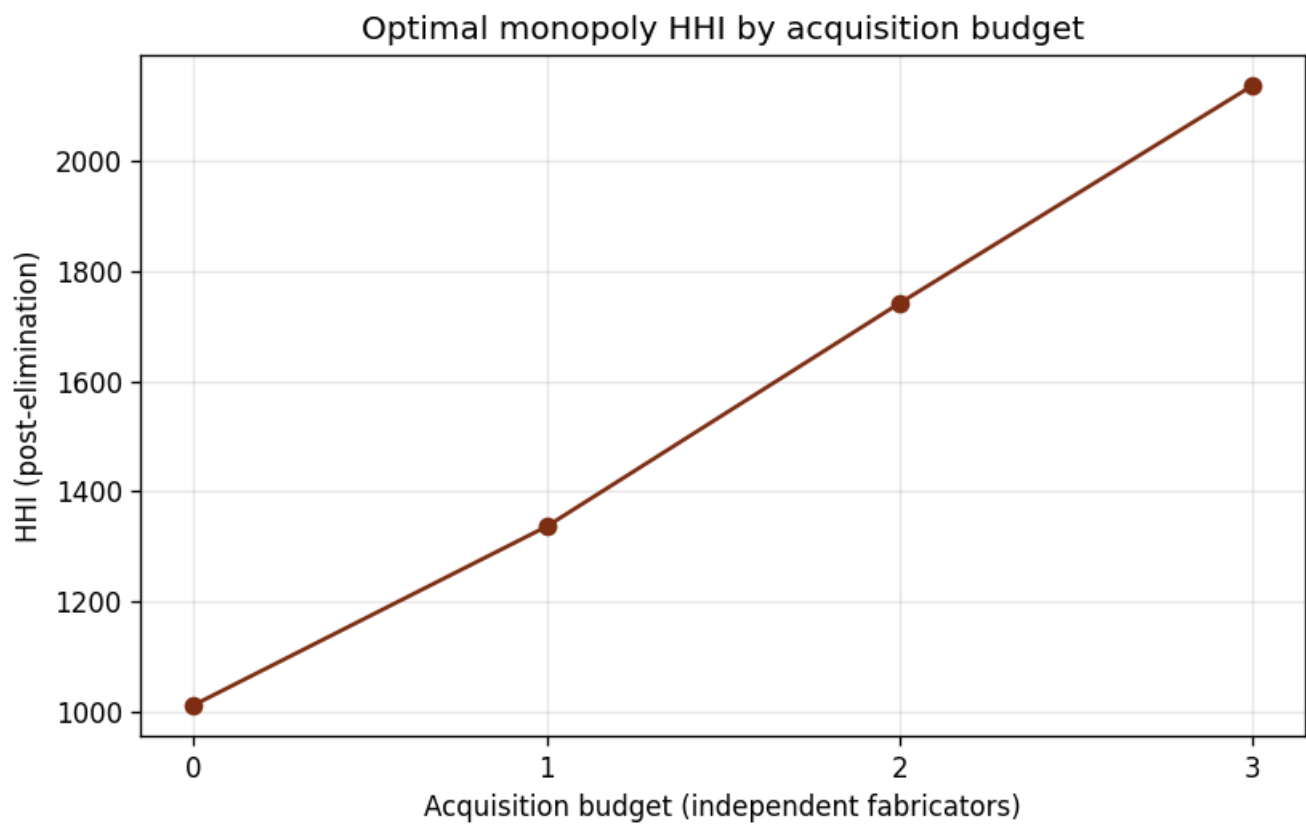


Figure 65: Optimal monopoly HHI by acquisition budget

optimal on *every* market, not merely this one (a proof sketch and an exhaustive deterministic counterexample probe live in `test_acquisition.py`). The exhaustive optimizer’s remaining value is as a correctness oracle and as the upper bound against which the scenario’s shortfall is scored; the headline remains the *scale* of the concentration a 2-firm buy achieves.

17.5.4.1 Scenario versus optimizer The scenario ZORIN actually executes and the answer this package’s optimizer returns are **not the same set**, and the difference is a result, not an oversight. The film-faithful scenario buys `amd_fab`, `nec_fab` (2 firms), scoring HHI 1528.9 and a fabrication share of 0.680. On the same objective and the same budget, the exhaustive optimum is `intel_fab`, `nec_fab` at 1741.4 and a share of 0.820. The scenario is therefore strictly suboptimal by 212.5 HHI points.

We report this rather than reconcile it away. The package makes **no claim that ZORIN’s acquisition is optimal**; every scenario-conditioned number in this manuscript (the HHI, share, cascade, and valley results above) is conditioned on the acquisition set fixed in `manuscript/config.yaml`, not on the optimizer’s answer. The optimizer’s role is to bound what the same market would have permitted — and the bound says the scheme leaves 212.5 points of concentration on the table.

17.5.5 Valley impact

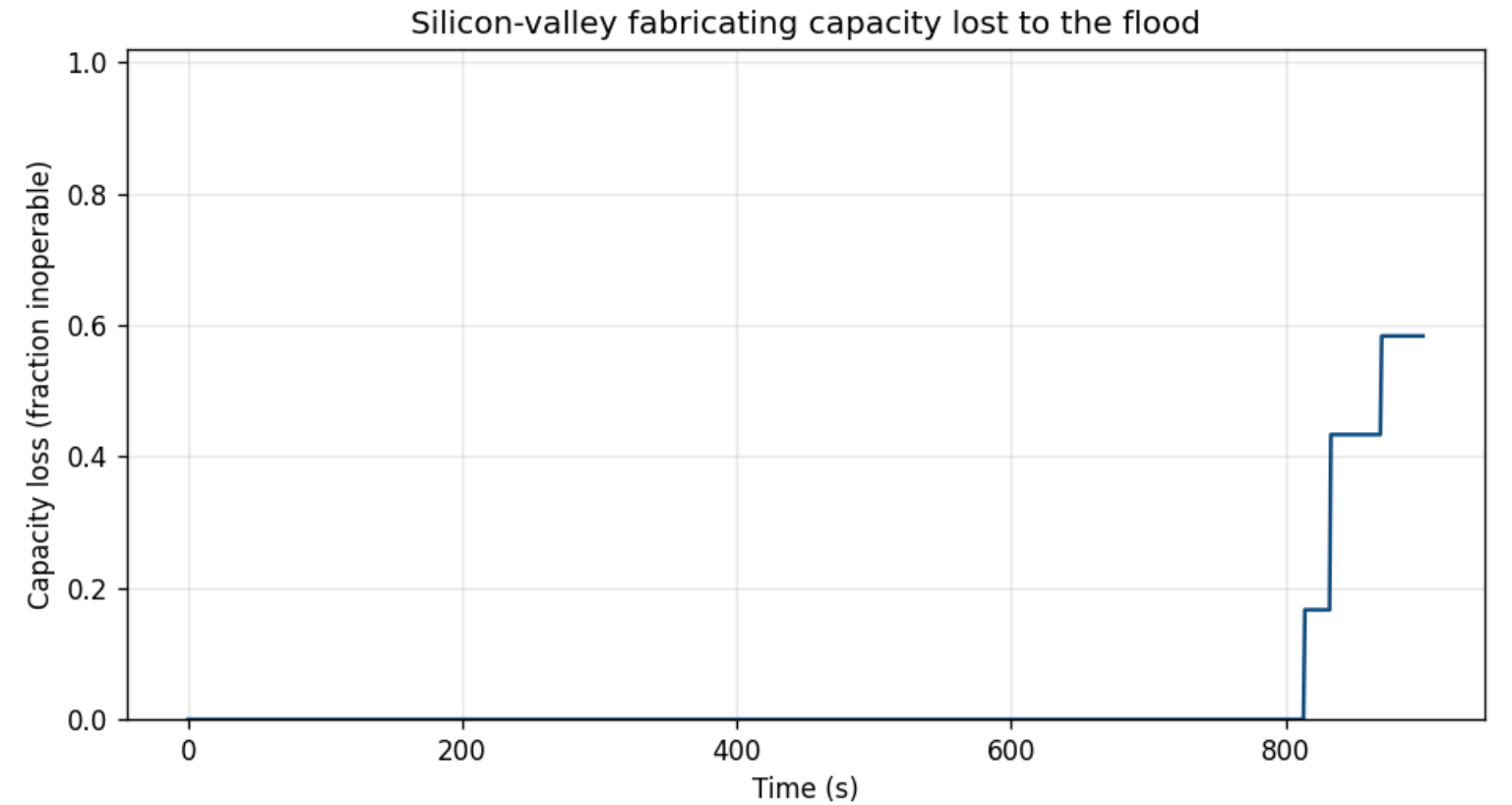


Figure 66: Silicon-valley fabricating capacity lost to the flood

Table 3: Urban blast-reach of the flood at the decision horizon.

Metric	Value
Valley fabs	5
Capacity loss (fraction inoperable)	0.583
Inoperable fabs	3
Victims	amd_fab, intel_fab, silicon_valley_fab
Survivors	nec_fab, zorin_fab
Survivors ZORIN acquires	nec_fab
Survivors ZORIN neither owns nor acquires	none
Victims ZORIN acquires	amd_fab
Victims ZORIN neither owns nor acquires	intel_fab, silicon_valley_fab

By the horizon the flood’s groundwater reach has disabled 3 of 5 valley fabrication plants — 0.583 of total fabricating capacity (the valley impact figure) — leaving 2 operable: `nec_fab`, `zorin_fab`. ZORIN’s own `zorin_fab` is among them, sitting over the low-coupling `shaft_a` and protected from the very flood he releases.

The victim list (`amd_fab`, `intel_fab`, `silicon_valley_fab`) is the more interesting result, because coupling geometry alone decides who drowns. The split by ownership is one-sided. On the losing side the flood is *not* discriminating: `amd_fab` is destroyed even though ZORIN buys it, alongside the plants he never acquires (`intel_fab`, `silicon_valley_fab`). On the surviving side it is entirely discriminating: the survivors are ZORIN’s own plant plus `nec_fab`, which he acquires, and the number of surviving plants he neither owns nor acquires is 0 (`none`). So the model contradicts the scheme’s own premise in exactly one direction: the flood spares no rival, but it does destroy an asset ZORIN pays for — the acquisition plan and the sabotage plan are not automatically consistent with each other.

Both directions of that claim are computed set operations (`valley_metrics`), and `tests/test_manuscript_variables.py::test_valley_ownership_split_matches_the_manuscript_claim` fails if either one stops holding, so this paragraph cannot outlive the data it describes.

17.5.6 Cover finance

The betting-market model prices the cover against an honest 1.12 overround book. Table: cover-bankroll plan for Pegasus.

Metric	Value
Bankroll	100000.0
Boosted true win probability	0.248
Market-implied probability	0.238
Value edge per unit	0.0415
Kelly fraction	0.0130
Kelly stake (USD)	1298
Expected profit (USD)	54

The manipulated horse’s true probability (0.248) exceeds the market-implied (0.238) by an edge of 0.0415 per unit staked, which justifies a Kelly stake of 1298 against the 100000.0 bankroll. Both probabilities come from the *same* softmax over form ratings — the honest book prices the un-boosted ratings and the true chance prices the boosted ones — so the edge measures the boost and nothing else. That a single horse’s true chance exceeds the honest market’s price is exactly the statistical anomaly the cover-risk model would flag: the cover is profitable only while it stays undetectable.

17.6 Conclusion — MAIN STRIKE: findings, verdict, and what the mission establishes

This package demonstrates that a film’s elaborate scheme can be reduced to a deterministic, testable computational model without losing its substance. ZORIN’s monopoly is a concentration and choke-point problem *and* a budgeted acquisition-selection problem; his flood is a mass-conserving network-dynamics problem whose urban blast-reach is a coupling problem over the fabs above; his cover is a signal-detection trade-off *and* a betting-market finance problem. Each of the six is implemented as a pure domain module with a matching real-data test file, aggregated by a protocol-free metric layer, and wrapped in a thin `MissionProvider` adapter that speaks the FROZEN BOND-API protocol.

That layering is load-bearing rather than cosmetic. Because only the adapter imports `bond_api`, the science — the six models, the metric groups, the figures, and this manuscript’s own numbers — runs whether or not the protocol dependency is present, and a test proves it by blocking the import and recomputing. The claim and the code are checked against each other rather than asserted in prose.

The result is a mission that *runs*: `mission_brief` / `mission_recon` / `mission_plan` / `mission_execute` / `mission_debrief` drive the full lifecycle from the command line, every metric is computed rather than stubbed, and every number in this document — including the structural model constants — arrives as a token from the same pure core. The package meets the PROJECT BOND guardrails: at least 90% line and branch coverage on `src/`, zero mocks, deterministic seeds, ruff and mypy clean, no leftover exemplar-lineage strings, and a clean git tree.

17.7 Experimental Setup — MAIN STRIKE: canonical scenarios, parameters, and configuration

All experiments are deterministic and fully prescribed by `manuscript/config.yaml`, whose `experiment:` block is loaded by `src/a_view_to_a_kill/manuscript_variables.py`, converted to a `metrics.MissionScenario`, and threaded into the models that produce every computed token below. Editing a knob here changes the results, not only the echo of the file — `tests/test_metrics_scenario.py` fails if any knob is inert.

17.7.1 Fixed model data

The datasets are hard-coded, documented model parameters in the pure core and annotated in `data/claim_ledger.yaml`:

- **Supply chain** (`chip_supply_chain.py`): 12 firms across the stages `wafer`, `fabrication`, `assembly`, `distribution` with 18 forward-flowing supply links; per-firm capacities fixed in the module. The acquisition optimizer (`acquisition.py`) derives

its candidate set from this same graph — the `amd_fab`, `intel_fab`, `nec_fab` non-ZORIN fabricators — so it adds no dataset of its own.

- **Mine network** (`mine_flood.py`): 8 chambers with volume/area/elevation, 10 tunnels with cross-sectional area; total capacity 9100 m³.
- **Valley fabs** (`valley_impact.py`): 5 surface fabrication plants, each with a capacity, a coupled mine chamber, a foundation elevation (m above datum), and a hydraulic leakage coefficient in [0, 1].
- **Race card** (`race_analytics.py`): 20 form records (5 horses x 4 races) with distance, going, weight, and time. The betting market (`race_market.py`) is derived from the resulting win probabilities under the configured overround — again, no separate dataset.

17.7.2 Simulation parameters

Scenario knobs come from `manuscript/config.yaml`; structural model constants (discharge coefficient, Euler timestep, detection threshold, par pace, base weight, purse, penalty, seed) are defined in the pure modules. Both are surfaced here as computed tokens, so neither can drift from the code.

- Flood ingress rate: 10.0 m³/s, horizon 900.0 s, Euler timestep 1.0 s, discharge coefficient 0.60.
- Race-form EWMA halflife: 2.0 races.
- Cover boost to Pegasus: 0.2 speed-index units; natural race-to-race variability 0.5; detection threshold 1.0.
- Acquisition budget: 2 independent fabricators.
- Scenario acquisition set (what ZORIN actually buys, distinct from the optimizer’s answer): `amd_fab`, `nec_fab`.
- Cover betting bankroll: 100000.0 USD; bookmaker overround 1.12.
- Mission RNG seed: 1985 (fixed; no randomness is ever drawn).

17.7.3 Software environment

- Python 3.14.6
- numpy, matplotlib, PyYAML; BOND-API protocol via path dependency
- Manuscript source date 2026-08-04, declared in `manuscript/config.yaml`. The generator reads no wall clock, so this line — like every other line — is the same on every rerun.

17.8 Reproducibility — MAIN STRIKE: verification gates, deterministic regeneration, and artifacts

17.8.1 Determinism

Every module is a pure function of fixed data with no random draws and no wall-clock dependence in its computed outputs. `Provenance.seed` is fixed at 1985; the mission outcome’s `input_hash` is a SHA-256 of the canonicalized plan, so an outcome can be audited without rerunning the film. The flood simulator is byte-deterministic across identical calls (`test_mine_flood.py::test_simulation_is_deterministic`), and the mission report is byte-identical across two `run_mission` calls (`test_mission.py::test_full_mission_flow_is_deterministic`).

The same holds for the *persisted* artifacts, and this is the load-bearing claim of this section: running `scripts/z_generate_manuscript_variables.py` twice on one pinned interpreter writes byte-identical `output/data/manuscript_variables.json`, `output/manuscript/*.md`, and `../figures/*.png`. Nothing in the pipeline reads a wall clock. The manuscript’s provenance date (2026-08-04) is `paper.date` from `manuscript/config.yaml` — a committed value that moves only when an author moves it — and the only other environment-derived token is the interpreter version (3.14.6), which is fixed for a given toolchain. `tests/test_regeneration_determinism.py` enforces this the only way that can fail honestly: it runs the real generator twice, in two separate processes, with more than a second of real time between them, and diffs the bytes. It does **not** inject a fixed clock — an injected `now=` would prove only that a fixed input yields a fixed output, and an earlier version of this package carried exactly that test while a `datetime.now` stamp moved `05_experimental_setup.md` on every run.

17.8.2 Artifacts

Running the pipeline reproduces every artifact from source:

```
uv run python scripts/mission_brief.py # brief
uv run python scripts/mission_recon.py # recon
uv run python scripts/mission_plan.py # plan
uv run python scripts/mission_execute.py # execute
uv run python scripts/mission_debrief.py # debrief
uv run python scripts/z_generate_manuscript_variables.py # tokens + figures
```

`z_generate_manuscript_variables.py` writes `output/data/manuscript_variables.json`, the substituted manuscript to `output/manuscript/`, and the five figures (`choke_points`, `acquisition_curve`, `flood_curve`, `valley_impact`, `race_cover`) to `../figures/`. `output/` is git-ignored (regenerable). The live test count and achieved coverage are tracked in `docs/_generated/COUNTS.md`.

17.8.3 Where the numbers come from

No metric in this manuscript is written by hand. Every numeral in the prose, tables, and captions is a double-brace variable token resolved by `src/a_view_to_a_kill/manuscript_variables.py`, which reads `manuscript/config.yaml`, turns its `experiment:` block into a `metrics.MissionScenario`, and calls `metrics.run_mission_metrics(scenario)` for the computed results. It calls `metrics`, not `mission`: importing `mission` would pull in `bond_api` and break the very fallback this section asserts below. Five gates keep this honest:

- `resolve_manuscript_tree(..., strict=True)` refuses to emit a section that still contains an unresolved variable token.
- `tests/test_manuscript_variables.py::test_every_manuscript_token_is_generated` fails if any token used in prose is missing from `generate_variables`, and `test_strict_gate_rejects_each_empty_metric_group` proves the strict gate actually fires when any one metric group is empty.
- `tests/test_metrics_scenario.py::test_every_scenario_knob_moves_a_metric` fails if any `experiment:` knob is inert — i.e. if changing it in the config leaves every computed token unchanged. A knob that is only echoed back as a `CONFIG_*` token is a reported-but-unused parameter, and this gate is what makes “prescribed by `config.yaml`” a checkable statement rather than a claim.
- `tests/test_metrics_scenario.py::test_default_scenario_matches_the_config_file` pins `metrics.DEFAULT_SCENARIO` to `manuscript/config.yaml`, so the code default and the config cannot drift into two answers.
- `tests/test_claim_ledger.py` proves `data/claim_ledger.yaml` records only token *names* and no frozen numerals, so the ledger cannot become a second, ungated source of truth for a metric.

17.8.4 Guardrails check

- Test coverage $\geq 90\%$ line + branch on `src/` (verified by `uv run pytest tests/ --cov=src --cov-fail-under=90`).
- Zero mocks: tests use real computation and real data only.
- No `infrastructure.*` imports in `src/`. The pure core, the metric layer, the figures, and this manuscript pipeline are importable without `bond_api` — verified by `tests/test_bond_api_fallback.py`, which blocks the import with a real meta-path finder and recomputes, with a positive control proving the blocker bites and a negative control confirming the adapter alone requires the protocol.
- No leftover template-scaffold lineage strings (`rg gate clean`).

17.9 Scope and Related Work — MAIN STRIKE: boundaries, positioning, and relationship to the literature

17.9.1 Scope

This package models the *structure* of ZORIN’s scheme, not its implementation in the physical world. The supply chain is a stylized capacity graph; the acquisition optimizer is a combinatorial abstraction of a corporate-control market with no price model; the mine flood is a simplified head-driven weir model; the valley coupling is a single-coefficient proxy for a groundwater transport model; the race cover is a signal-detection abstraction of stewards’ scrutiny; and the betting market is an idealized book with a uniform overround and no liquidity, stake limits, or price impact. The datasets are fixed, documented model parameters (historical-company names for the supply chain, fictionalized gallery geometry for the mine, a reconstructed race card) — they are illustrative model inputs, not measured field data. No claim is made about the real historical microchip market’s exact shares, about any real mine, or about any real racing jurisdiction; the value of the package is the deterministic, reusable mathematics, and the fact that every number reported here is computed by the tested code rather than asserted in prose.

Two limits are worth stating explicitly. First, the acquisition optimizer is exhaustive over budget-sized subsets — $\binom{n}{K}$ work — which is tractable at this problem’s scale and deliberately chosen: its purpose is to be the *correct* oracle against which the greedy baseline’s regret is measured, not to scale. Second, the valley coupling is monotone in chamber head, so it can order which plants fall and when relative to each other, but its absolute inundation times inherit the weir model’s simplifications and should not be read as an engineering prediction.

17.9.1.1 Uncertainty The package reports deterministic point estimates, not distributions: no quantity carries a confidence interval. That is a design property, not an oversight — the models are exactly reproducible given the fixed datasets and scenario (byte-identical reruns are themselves a checked gate), but reproducibility is a statement about the *computation*, not about the real-world phenomena the film’s scheme gestures at. Every result is conditional on the scenario knobs in `manuscript/config.yaml` and on the documented model datasets (capacities, gallery geometry, elevations, race times); change a knob or a datum and the numbers move, which is precisely why each one is surfaced as a token rather than a frozen numeral, and why sensitivity is exercised by re-running the prescribed experiment under an alternative `experiment:` block rather than by reading a delta off this page. In particular the flood inundation times, the valley victim/survivor split, and the cover-detection probability are all model-conditioned and should not be read as engineering or actuarial predictions.

17.9.2 Related work

- **Concentration and choke points.** The Herfindahl-Hirschman index is the standard antitrust concentration measure, codified in the U.S. *Horizontal Merger Guidelines* [U.S. Department of Justice and Federal Trade Commission, 2010]. Betweenness centrality follows Brandes’ algorithm [Brandes, 2001a]. Supply-chain cascades relate to the supply-chain resilience literature, which models how upstream disruption starves downstream production [Christopher and Peck, 2004]; the reachability formulation used for the cascade is a standard network-flow construction [Ahuja et al., 1993b].
- **Acquisition as combinatorial selection.** Choosing which competitors to buy under a budget is subset selection against a set objective. We report the exhaustive optimum alongside a greedy baseline because greedy heuristics on such objectives carry no *general* optimality guarantee. On the two objectives shipped here the guarantee actually holds: both are strategically monotone, so greedy is exact by construction and the regret is provably zero for every market — verified exhaustively over a deterministic counterexample grid in `test_acquisition.py`. The exhaustive path remains the correctness oracle and the upper bound the scenario is scored against.
- **Network water flow.** The weir/head-driven flow law used here is a simplification of the open-channel flow models used in flood and mine drainage engineering; the key property preserved is exact mass conservation.
- **Groundwater damage to built assets.** Coupling a subsurface water level to the operability of structures above follows the groundwater-inundation damage literature, which relates elevated water tables to damage in foundations and basements [Kreibich and Thieken, 2008].
- **Racing form and detection.** Normalized speed handicapping and softmax outcome models are standard in racing and sports-outcome analytics [Dixon and Coles, 1997]; framing manipulation as a signal-detection trade-off draws on classical detection theory [Green and Swets, 1966].
- **Bet sizing.** The staking rule is the Kelly criterion — the expected-log-growth-optimal fraction of a bankroll to stake on a bet with a known edge [Kelly, 1956].

Within the PROJECT BOND suite, this package is one of the film packages implementing the FROZEN `bond_api.MissionProvider` protocol; its pure core constitutes the local scientific content that the protocol wraps.

17.10 Sources — MAIN STRIKE: bibliography

Brandes [2001a]; U.S. Department of Justice and Federal Trade Commission [2010]; Glen and Eon Productions [1985]; Dixon and Coles [1997]; Ahuja et al. [1993b]; Kelly [1956]; Christopher and Peck [2004]; Kreibich and Thieken [2008]; Green and Swets [1966]

18 The Living Daylights (1987) — LIVING DAYLIGHTS

film package · *package codename* *LIVING DAYLIGHTS*. Mission **SNIPER'S NEST**: sniper cover for a high-value defection (Bratislava); rules-of-engagement discipline (no shooting the innocent); cello-case concealment of a takedown sniper rifle; mountain/desert exfiltration with covered routes.

18.1 Concepts — LIVING DAYLIGHTS: domain and operational focus

countersniper optics, defection

18.2 Abstract — LIVING DAYLIGHTS: mission summary

The Living Daylights: SNIPER'S NEST — Special-Agent Mission Software (mission codename **SNIPER'S NEST**) is deterministic special-agent mission software for *The Living Daylights* (1987). It models six operational concepts with real, reproducible computation across eight pure domain modules: * **Sniper ballistics** — a plug-and-play point-mass external-ballistics core that solves the rifle zero (fusing the trajectory to the sight line at 300 m), captures the near/far zero, and solves windage (a 5.0 m/s full-value crosswind imposes a 0.189 m drift requiring a -0.63 mrad hold), all embedded in a takedown rifle that conceals inside a cello case (true, 2 lanes, 0.241 packing efficiency, 72 s field assembly). * **Countersniper optics** — the 10x mil-dot riflescope ranges the 300 m target from a -2.68 mil holdover and dials the 27/-6 click solution; glint radiometry detects a hostile optic at 0.083 probability at 800 m. * **Countersniper detection** — a hostile sniper is ranged acoustically (687 m from the 2.0 s crack-bang delay at 20 °C) and localized by 2 crossed bearings to (50.0, 50.0) m. With two bearings the fit is exactly determined, so its residual is zero by construction and no accuracy is claimed from it. * **Defection-handling protocol** — a staged, risk-scored pipeline (5 stages) whose weighted hazard model renders the canonical KOSKOV scenario conditional with a verify gate stage and 0.670 backstop certainty (conditional). * **Mountain/desert terrain ops** — a seeded (7) terrain grid on which a cover-weighted least-cost Dijkstra planner routes 31.00 km over 62 steps. Measured against baselines, the exposure term raises route concealment by 0.014 over the same planner with the term off, but the resulting 0.484 still falls short of the grid's own 0.490 mean (signed gap -0.006): at this weighting concealment is a preference, not the objective, and the route is not a concealed one. * **Overwatch selection** — of 16 in-range candidate positions only 1 see the extraction site; the top-scored *visible* nest at (6, 0) commands it from 3000 m with 1451.7 m elevation advantage. The staged plan's verify gate and the backstop certainty agree on a conditional posture. They read no shared variable, but both scenarios are constant tables authored in one module to encode the same story beat, so the agreement is a design-consistency check between two scoring paths — not independent corroboration. The same caution applies to the terrain cover audit, which is a second implementation of one concealment definition. All results derive from a single deterministic mission solution ([src/the_living_daylights/compute_mission.py](#)) so the manuscript, the mission report, and the figures can never drift apart. Keywords: sniper ballistics, rifle zero, windage, rules of engagement, riflescope optics, mil-dot ranging, countersniper detection, optic glint radiometry, acoustic crack-bang ranging, bearing triangulation, terrain visibility, viewshed, overwatch selection, defection protocol, evidence verification, risk scoring, terrain operations, cover planning, deterministic simulation.

18.3 Introduction — LIVING DAYLIGHTS: mission framing, the operational problem, and how to read this chapter

The Living Daylights (1987) is the mission that frames SNIPER'S NEST: a covering sniper must protect a high-value defection, carry a sniper rifle concealed in a cello case, refuse a lethal shot at an innocent, find and range the hostile shooter working the same corridor, and exfiltrate through mountain and desert terrain. Each beat is a tractable, well-posed computation. This package turns those beats into deterministic algorithms backed by real data and reproducible figures.

18.3.1 The domain core

The software is structured as a pure, infrastructure-free domain core ([src/the_living_daylights/](#)). Every module below is stdlib + numpy only — no `bond_api`, no I/O, no wall-clock, no global RNG:

- **ballistics.py** — a point-mass RK4 external-ballistics core: rifle zero, near/far-zero capture, crosswind drift and the wind hold, rules of engagement (the trigger-guard hold), and deterministic cello-case shelf packing with field assembly time.
- **optics.py** — the riflescope: MOA $\leftrightarrow$ mrad turret arithmetic, whole-click solutions, mil-dot reticle ranging and its inverse, ballistic holdover in mils, first-order parallax, and true field of view.
- **countersniper.py** — finding the other shooter: optic-glint radiometry (geometry factor, sky contrast, angular fraction, detection probability), acoustic crack-bang ranging with a temperature-dependent sound speed, and least-squares bearing triangulation of crossed azimuths.
- **visibility.py** — terrain line of sight over a Bresenham cell walk, viewsheds, and the ranked overwatch (sniper's nest) selection the title's image demands.
- **defection_protocol.py** — a staged, risk-scored defection-handling pipeline with monotonic approval and a plan gate, plus a separately parameterised backstop evidence-verification model that agrees with that gate — both are authored constant tables, so the agreement is a design check rather than independent corroboration.
- **terrain_ops.py** — a seeded mountain/desert grid (elevation, landcover, standing cover), cover-weighted Dijkstra route planning reported against measured baselines, and a separately implemented cover audit that grades any proposed path, not only the

planner’s own.

- **settings.py** — the single authoritative source of mission parameters, echoed for readers in **manuscript/config.yaml**.
- **compute_mission.py** — the one deterministic mission solution that composes all of the above.

18.3.2 Protocol integration

The package integrates with the frozen BOND-API mission protocol: **mission.py** implements the **MissionProvider** lifecycle (brief, recon, plan, execute, debrief) against **bond_api**’s frozen dataclasses and registers the film’s field kit in a module-level **GADGETS** registry. It is the only module in the package that imports **bond_api**, and it carries no business logic of its own.

The mission adapter and the manuscript both read the single deterministic **compute_mission.mission_solution**, so prose metrics are never hand-authored: every number below arrives as a brace-delimited token hydrated by **manuscript_variables.py**, and a test fails the build if any placeholder in these sections is not generated.

18.3.3 Reader’s guide

The manuscript follows the mission procedure: methodology (section 2) derives each algorithm module by module, results (section 3) reports the computed mission solution with the four figures, the conclusion (section 4) states what was shown, experimental setup (section 5) fixes the parameters, reproducibility (section 6) ties results to the provenance hash and the live test gate, and section 7 states the scope boundary and the related work.

18.4 Methodology — LIVING DAYLIGHTS: the analytical models and algorithms that drive the mission

Six concept modules define the algorithms, in mission order: sniper ballistics (**ballistics.py**), riflescope optics (**optics.py**), counter-sniper detection and localization (**countersniper.py**), terrain visibility and overwatch selection (**visibility.py**), staged defection handling with backstop evidence verification (**defection_protocol.py**), and terrain routing (**terrain_ops.py**). **settings.py** fixes their parameters and **compute_mission.py** composes them into one solution. All are deterministic and infrastructure-free: stdlib and numpy only, no I/O, no wall-clock, and no randomness beyond the seeded terrain grid.

18.4.1 Sniper ballistics (**ballistics.py**)

The round is modelled as a **point-mass projectile with quadratic air drag** plus gravity, integrated with a fixed-step classical RK4 (**ballistics.trajectory**). The drag deceleration follows the standard law $\mathbf{a}_{\text{drag}} = (\rho \cdot \mathbf{A} \cdot \mathbf{C}_d / 2m) \cdot \mathbf{v}^2$; the ballistic coefficient $BC = m / (\mathbf{A} \cdot \mathbf{C}_d)$ for the 7.8 mm service round is 984 kg/m² at a muzzle velocity of 790 m/s and bullet mass 10.4 g.

Rifle zero. **ballistics.zero_scope** solves the muzzle elevation such that the round lands at height zero at the zero range 300 m while the sight line — starting 0.07 m above bore — passes through the same point. Bisection on the simulated impact height yields an elevation of 2.68 mrad. **capture_zero** then returns the two ranges where the trajectory crosses the sight line: near 25.9 m and far 300.3 m.

Time of flight. **ballistics.time_of_flight** is a *closed form*, not a readout of the RK4 trajectory above. Flat-fire quadratic drag $d\mathbf{v}/dt = -k \cdot \mathbf{v}^2$ integrates to $\mathbf{x}(t) = \ln(1 + k \cdot \mathbf{v}_0 \cdot t) / k$, and inverting that gives $t(\mathbf{x}) = (\exp(k \cdot \mathbf{x}) - 1) / (k \cdot \mathbf{v}_0)$, which is the expression the code evaluates. Because $\exp(z) - 1 \geq z$, the result is always at or above the drag-free bound **range/v₀** — a decelerating round cannot beat vacuum, and the test suite pins that floor at five ranges. The form neglects the extra path length contributed by the launch angle and gravity, so it is a strict lower bound on the integrated flight time; at the 300 m target it returns 0.418 s against the RK4 trajectory’s own time array, the two agreeing to under a millisecond. That comparison is itself a test, not a claim.

Windage. Crosswind is treated as a drag-driven lateral relaxation toward the wind speed. **ballistics.wind_drift** integrates the lateral offset; at the 300 m target a 5.0 m/s full-value wind produces a 0.189 m drift, i.e. a -0.63 mrad wind hold. **wind_hold_mrad** returns $-\text{drift} / \text{range} \cdot 1000$.

Rules of engagement. **ballistics.engagement_decision** encodes the film’s central morality: lethal fire only on a confirmed hostile target. A hostile non-target whose weapon is present draws a **trigger-guard** (**disable_weapon**) shot rather than a kill, so an innocent third party — the cellist — is never engaged.

The cello case. **disassemble_takedown_rifle** returns the components of a compact takedown rifle (**barrel_receiver**, **stock**, **optic**, **suppressor**); **pack_cello_case** runs deterministic bottom-left shelf packing to decide whether the 4 components fit a 1.30 x 0.45 x 0.24 m case interior. Parts are laid along the case’s long axis and processed width-descending, opening a new lane across the case’s width when the current lane runs out of length; each part must also clear the interior cross-section on its own. The rig packs into 2 lanes with a longest-lane fill of 1.10 m and an areal packing efficiency of 0.241 (the parts’ summed footprint over the case floor), and **assembly_time** gives a 72 s field assembly time. Because the packer reorders parts internally, each placement carries the part it belongs to, so a reported slot is always the slot of the part it names.

18.4.2 Countersniper optics (optics.py)

The rifle scope is a fixed-parameter `ScopeSpec` (10x, 50 mm objective, 0.10 mrad clicks). Two conversion families matter operationally:

- **Turret clicks** — `clicks_for_angle` converts an angular correction (milliradians) into whole clicks of the turret's click unit, and `zero_click_schedule` turns the ballistic solution into the scope's elevation/wind click counts (27 elevation / -6 wind for the zeroed mission).
- **Mil-dot ranging** — the milliradian relation `range = size·1000/mils (mil_dot_range)` fixes an 1.80 m standing silhouette spanning 6.0 mils on the reticle at 300 m, and `mils_for_size` inverts it. `holdover_mils` converts the ballistic drop into a -2.68 mil reticle holdover, and `parallax_shift_m` models the first-order sight shift when the target range differs from the 150 m parallax-focus range (10.0 mm at the 300 m target). `field_of_view_m` divides the eyepiece's apparent field by the magnification and returns the chord at range. Ranging follows the standard mil-dot doctrine [U.S. Department of the Army, 1994].

18.4.3 Countersniper detection (countersniper.py)

A hostile sniper is found three ways, all deterministic:

- **Optic glint** — the lens reflects sunlight. `glint_geometry_factor` multiplies a $\sin(\text{sun elevation})$ illumination term (a higher sun puts more direct irradiance on the lens; 35° here) by a Gaussian specular falloff $\exp(-(\Delta\text{az}/\text{half-width})^2)$ with a 3.0° half-width, giving 0.368 at the mission's 2.0° sun-observer offset. `glint_contrast` spreads the collected power $I \cdot A \cdot \rho \cdot g$ (reflectivity $\rho = 0.35$) over the specular cone's solid angle and divides by sky radiance; range cancels, which is why glints stay visible far out (the solar reference follows [ASTM International, 2020]). `glint_angular_fraction` then supplies the range dependence — the lens's solid angle A/R^2 over one 1.0 mrad resolution cell, capped at 1 — so `glint_snr` falls as $1/\text{range}^2$ once the lens no longer fills a cell. `glint_detection_probability` is the signal-detection CDF of SNR against a threshold ([Van Trees, 1968]). At 800 m the hostile optic yields SNR 5.8 and $P(\text{det}) = 0.083$.
- **Acoustic crack-bang** — the flash-to-bang delay scaled by the speed of sound $c(T) = 331.3 + 0.606 \cdot T$ ([Kinsler et al., 2000]) places the shot 687 m away from the 2.0 s delay; the mission's 343.4 m/s at 20 °C.
- **Bearing triangulation** — `triangulate_position` minimizes the summed squared perpendicular distance from a point to each observer's bearing line, solving the resulting 2x2 normal equations in closed form and rejecting a singular (parallel or collinear) system rather than returning a fictitious fix. From 2 observers the crossed bearings place the hostile at (50.0, 50.0) m (acoustic shooter localization per [Duckworth et al., 1996], [Ledeczki et al., 2005]). **The reported residual is not a measurement.** With 2 bearings the normal equations are two equations in two unknowns, so the fit passes exactly through the intersection and the residual is identically zero for any non-parallel pair; the 0.00 m printed in the results is floating-point rounding, not accuracy. The residual only carries information when the system is over-determined and inconsistent, which requires at least three bearings and a noise model this package does not have. A three-bearing positive control in `tests/test_countersniper.py` pins a hand-derived 20.4124 m residual, so the code path is exercised where it can actually be wrong.

18.4.4 Terrain visibility (visibility.py)

`line_of_sight` walks a deterministic integer Bresenham cell line [Bresenham, 1965] between observer and target and blocks when an intermediate cell's terrain-plus-cover top rises above the linearly interpolated sight ray minus a clearance (0.30 m here); the ray runs from `elevation + eye height` (1.70 m) at each end. `viewshed` collects every cell visible from an observer in a deterministic row-major scan and reports the visible count and fraction; viewshed methodology follows the terrain-visibility literature [Fisher, 1996].

`rank_overwatch_positions` scores each candidate nest as $w_{\text{vis}} \cdot \text{visible} + w_{\text{dist}} \cdot (1 - d/d_{\text{max}}) + w_{\text{cover}} \cdot \text{cover} + w_{\text{elev}} \cdot \text{gain}/\text{gain}_{\text{max}}$ with weights (0.40, 0.25, 0.20, 0.15) that are validated to sum to 1; candidates beyond 12 cells are dropped as out of practical engagement range, and distance and elevation gain are normalized over the surviving set. Positions that cannot see the target keep their other terms but score zero for visibility, so the ranking stays total and deterministic (ties break on cell order). `best_overwatch` returns the top-ranked *visible* candidate, or `None` when nothing sees the site. Because a blind candidate keeps its proximity, cover, and elevation terms (up to 0.60 of the score), it can out-rank a visible one that scores only the 0.40 visibility weight; `rank_overwatch_positions[0]` therefore carries no visibility guarantee, and `compute_mission` selects the reported nest through `best_overwatch` rather than off the top of the ranking. `tests/test_visibility.py` builds the case where the two disagree, so the filter is exercised rather than assumed. On the mission's own seed the two happen to coincide at every candidate step tested (1–8): the 0.40 visibility weight dominates this terrain, so no reported number changes. The selection is stated here because it is the correct policy and because an earlier version took `ranked[0]`, which carried no such guarantee — not because it moved a result.

18.4.5 Defection-handling protocol (defection_protocol.py)

`assess_defection` evaluates an ordered, fixed sequence of 5 stages (contact, snatch, verify, exfiltrate, relocate). Each stage carries weighted `RiskFactors`; the stage hazard is the exposure-weighted mean $\text{sum}(w_i \cdot s_i) / \text{sum}(w_i)$ clamped to $[0, 1]$, so a dominant high-score factor cannot be averaged away by benign ones. A stage is `go` at or below the 0.40 threshold, `conditional` at or below 0.70, and `no-go` above it; the plan score is the conservative maximum across stages.

Approval is **monotonic**: the assessment walks the stages in order, marking each **approved** while every predecessor is `go`. The first stage whose verdict is not `go` becomes the **plan gate**, and a `no-go` at any stage blocks every stage after it. The protocol validates

membership of the canonical stage set rather than position, so a caller may supply the stages in its own order. The canonical KOSKOV scenario (`koskov_scenario`) fixes the factor weights and scores, so the mission's report is reproducible.

18.4.6 Evidence verification (`defection_protocol.py`)

The verify stage's backstop model (`EvidenceCheck`, `verification_certainty`, `backstop_verdict`) accumulates 4 weighted checks (`identity_documents`, `debrief_consistency`, `backstop_telegraphy`, `cover_story_plausibility`) into a certainty score under the same exposure-weighting scheme as `stage_score`, then maps that certainty onto a cleared/conditional/rejected verdict — the operational analogue of sequential verification against documentary and telegraphic backstops.

This is not independent corroboration, and earlier drafts of this section wrongly said it was. `koskov_scenario` and `koskov_verification` are two hardcoded constant tables in the same module, written by the same author to encode the same story beat — the film's verification doubt. They read no shared variable, but that is a weaker fact than it sounds: the numbers in both tables were chosen so that the verify stage gates and the backstop lands `conditional`, and `koskov_verification`'s own docstring says so. Their agreement demonstrates that two scoring functions applied to two consistent authored inputs return consistent verdicts. It is a design-consistency check between the staged and backstop paths, and it is evidence about neither General Koskov nor the world.

18.4.7 Terrain ops (`terrain_ops.py`)

`generate_terrain` builds a seeded elevation + landcover + standing-cover grid: elevation is a wrap-padded 3x3-blurred Gaussian field renormalized to the requested relief, landcover is a seeded categorical draw over the table (`landcover_name` indexes it), and cover height is the landcover's base cover plus seeded jitter. Effective concealment is the cover height normalized to the 500 m cell's cover threshold and clipped to `[0, 1]` — a pure function of the cover-height field, so an edited field is honoured verbatim downstream.

`step_cost` blends the per-metre landcover cost, an ascent penalty proportional to the climb per cell, and an exposure penalty `cover_weight · (1 - cover)`, then scales by the cell size. `plan_route` runs Dijkstra's algorithm [Dijkstra, 1959c] over 4-adjacent moves with strict `(cost, row, col)` tie-breaking, which is what makes the route byte-reproducible. `assess_cover` grades any proposed path — not only Dijkstra's — reporting mean and minimum concealment, the least-covered step, the high-concealment waypoints, and per-waypoint cover.

What the audit's agreement with the planner establishes. `assess_cover` is a vectorised numpy implementation and `plan_route`'s summary is a scalar Python loop; the two deliberately share no helper. Agreement between them therefore rules out implementation drift between what the planner reports about its own route and what an audit of the same waypoints computes. It does **not** corroborate the concealment model: both encode one definition (`clip(cover_height / COVER_THRESHOLD_M, 0, 1)`) written by one author, so a wrong definition would be reproduced identically by both. The test suite therefore checks each against a third calculation performed in the test itself, rather than checking them against each other.

18.4.8 Composition (`compute_mission.py`)

`mission_solution` is the single entry point that runs every module above against `settings.py` and returns one nested mapping: ballistics, rules of engagement, optics, countersniper, cello case, defection, verification, terrain, and overwatch. The extraction site is defined non-arbitrarily as the grid's global elevation minimum `((0, 0))` — a wadi-like low point, the hardest kind of site to overwatch — and candidate nests are sampled every 3 cells in each axis.

`input_hash` fingerprints that mapping as canonical compact JSON with sorted keys under SHA-256. The mission adapter, the manuscript token map, and the figures all read the same call, so no consumer can report a number the others did not compute.

18.4.9 Input-contract hardening

The pure core fails loud on numerically invalid inputs rather than propagating garbage. Four guards are bound by regression tests that would fail on the code that preceded them:

- `ballistics.wind_drift` and `ballistics.wind_hold_mrad` reject a non-positive range. The hold computes `-drift / range · 1000`, so a zero range previously slipped through to an opaque `ZeroDivisionError`; both now raise a `ValueError`, matching `time_of_flight` and `drop_at_range`.
- `optics.mil_dot_range` and `optics.mils_for_size` reject a non-positive target size. The mil-dot relation `range = size · 1000 / mils` is linear in the silhouette, so a zero or negative `target_size_m` silently produced a zero or negative (nonsense) range; both now raise.
- `countersniper.triangulate_position` rejects any non-finite observer coordinate or bearing. A NaN bearing previously flowed through the closed-form normal equations and returned a NaN "fix" with no error — a fail-open on a detection/localization path. The single finite-input guard roots this out, since a finite determinant follows from finite inputs.
- `terrain_ops.assess_cover` reports `waypoint_cover` as one `(cell, cover)` pair per waypoint, in path order, so a path that revisits a cell keeps a reading for every visit. An earlier dict keyed on `"r,c"` silently dropped the duplicate reading. The aggregate `mean_cover / min_cover / cover_points` were always computed over the full sequence; only the map was lossy.

Each guard carries a matching positive assertion that the healthy path is unchanged, so these are checks of a real contract, not new ways to fail.

18.5 Results — LIVING DAYLIGHTS: measured outcomes, headline numbers, and what they establish

The deterministic mission solution (`compute_mission.mission_solution`) reports every number below; the manuscript never hand-authors a metric.

18.5.1 Sniper ballistics

The 300 m zero requires an elevation of 2.68 mrad for the 7.8 mm round (BC 984 kg/m², 790 m/s, 10.4 g). The sight line and the trajectory cross at the near zero 25.9 m and the primary far zero 300.3 m; at the 300 m target the round rides -0.804 m below the line of departure with a 0.418 s time of flight. A full-value 5.0 m/s crosswind deflects the round 0.189 m, calling for a -0.63 mrad wind hold. The ballistics figure shows the zeroed trajectory against the sight line and the drift curve.

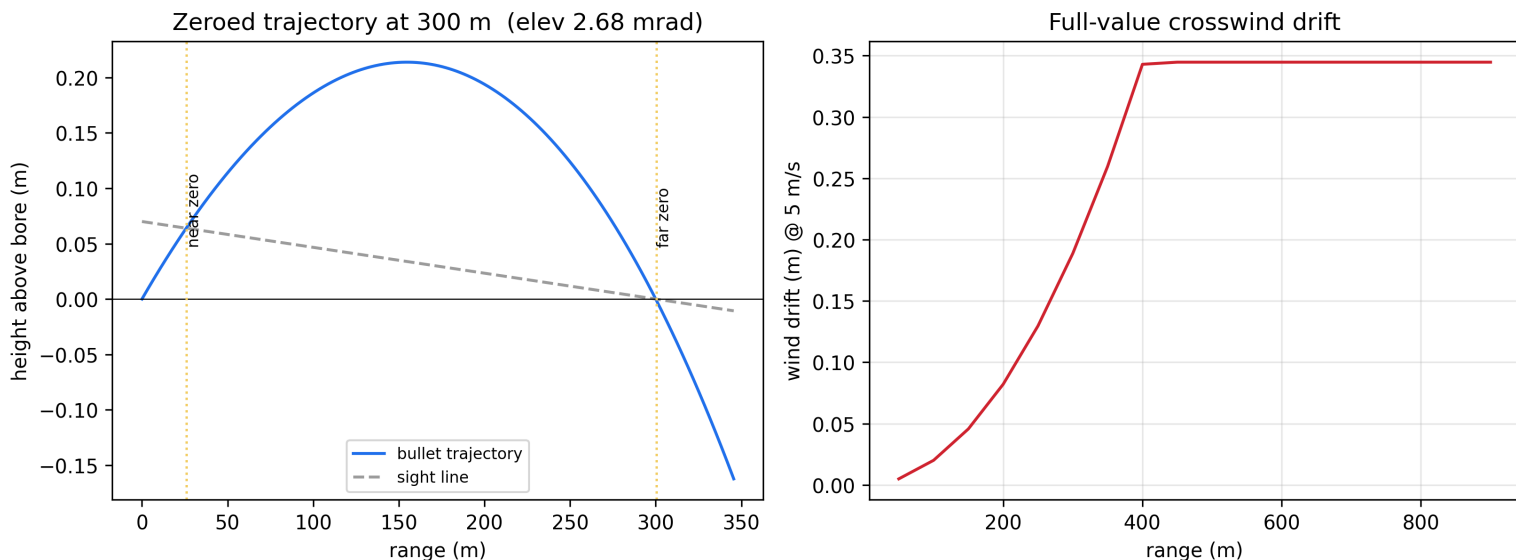


Figure 67: Zeroed ballistic trajectory and crosswind drift

18.5.2 Rules of engagement

The covering sniper’s decision engine resolves the film’s moral hinge deterministically. The cellist — observed “cellist”, non-hostile — resolves to **hold_fire** (non-hostile ‘cellist’ — innocent, hold fire): Bond refuses the lethal shot. A hostile sentinel with a weapon present, but *not* the authorized asset, resolves to **disable_weapon** (hostile non-target ‘sentinel’ — trigger-guard shot to disable weapon): the trigger-guard shot disables the threat without an unlawful kill (see `ballistics.engagement_decision`).

18.5.3 The cello case

The 4-part takedown rifle (barrel_receiver, stock, optic, suppressor) conceals in the 1.30 x 0.45 x 0.24 m case interior: packing returns **true** using 2 lanes, a longest-lane fill of 1.10 m, and an areal efficiency of 0.241, with a 72 s field assembly time (see `ballistics.pack_cello_case`).

18.5.4 Defection-handling protocol

The KOSKOV scenario runs through 5 stages (contact, snatch, verify, exfiltrate, relocate) with a plan score of 0.605 and an overall verdict of **conditional** against the 0.40/0.70 thresholds, gated at the **verify** stage — the point where identity doubt and backstop inconsistency surface. Earlier stages approve; later ones carry the plan only under conditions.

18.5.5 Mountain/desert terrain ops

On the seeded (7) terrain grid, the planner routes 31.00 km (62 steps) from the entry cell to the exfil goal at a blended cost of 92550.4, with a mean route concealment of 0.484 and a worst-case (least-covered) segment of 0.071 at step 43; 19 high-cover waypoints are identified.

Measured against baselines, the cover term is real but weak, and the route is not a concealed one. The same planner with the exposure term switched off (`cover_weight = 0`) returns a route of 0.470 mean concealment, so the term buys 0.014 of concealment — a real, non-zero effect on a different path. But the grid’s own mean concealment is 0.490, and the shipped route’s signed gap against it is -0.006 — the route is *less* concealed than an average cell. At the mission’s weighting the blended cost is dominated by landcover traversal cost and climb, and concealment is a mild preference rather than the objective. Calling this a

“covered exfil route” would overstate it. Raising `cover_weight` does clear the field average, so the shortfall is a weighting choice and not a ceiling; both relations are pinned in `tests/test_compute_mission.py`, so this paragraph cannot drift from the numbers.

Run against the same waypoints, the `assess_cover` audit returns 0.484. That agreement is a cross-implementation check — vectorised numpy against the planner’s scalar loop, sharing no helper — and it detects implementation drift only; both encode the same concealment definition, so it is not independent corroboration of the model. The terrain figure maps the route and its cover points on the concealment field.

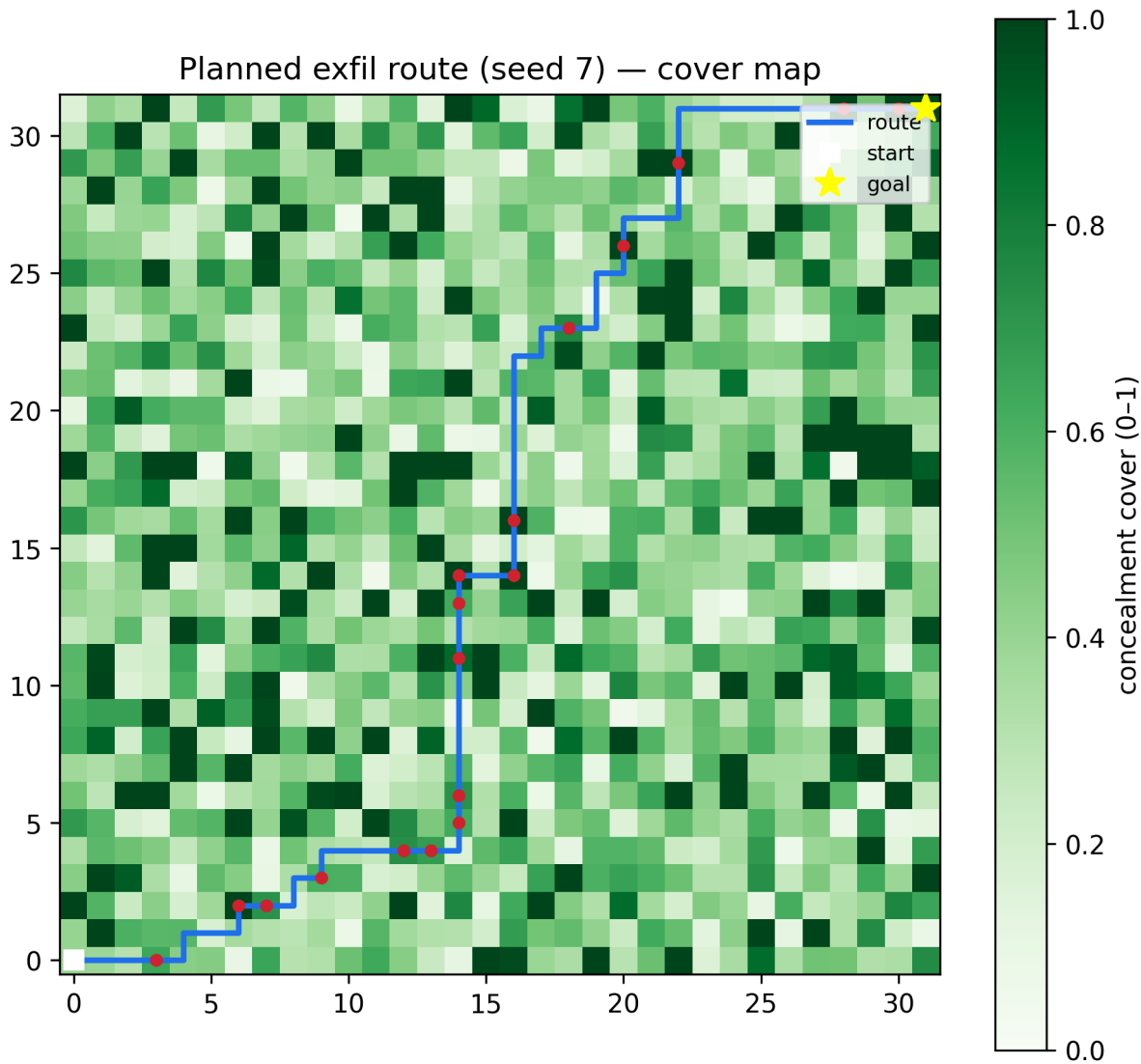


Figure 68: Planned exfil route on the concealment map

18.5.6 Countersniper optics

The 10x mil-dot riflescope (50 mm objective, 0.10 mrad clicks) ranges the standing target at 300 m from the 6.0-mil reticle span and holds -2.68 mils against the 300 m drop. The zero dials onto the turret as 27 elevation clicks and -6 wind clicks; at the target range the first-order parallax shift is 10.0 mm.

18.5.7 Countersniper detection and localization

The hostile optic glints with a geometric brightness of 0.368 at the 2.0° sun–observer offset; the radiometric model yields SNR 5.8 at 800 m and a detection probability of 0.083 (the glint figure). Acoustically, the 2.0 s crack-bang delay at 20 °C places the shot 687 m away (sound speed 343.4 m/s); crossed bearings localize the hostile to (50.0, 50.0) m. The fit also prints a 0.00 m residual, and that number is reported here only to be discounted: with 2 bearings the least-squares system is exactly determined, so the residual is identically zero by construction for any non-parallel pair and the printed value is floating-point rounding. It is not a measure of localization accuracy, and no accuracy claim rests on it.

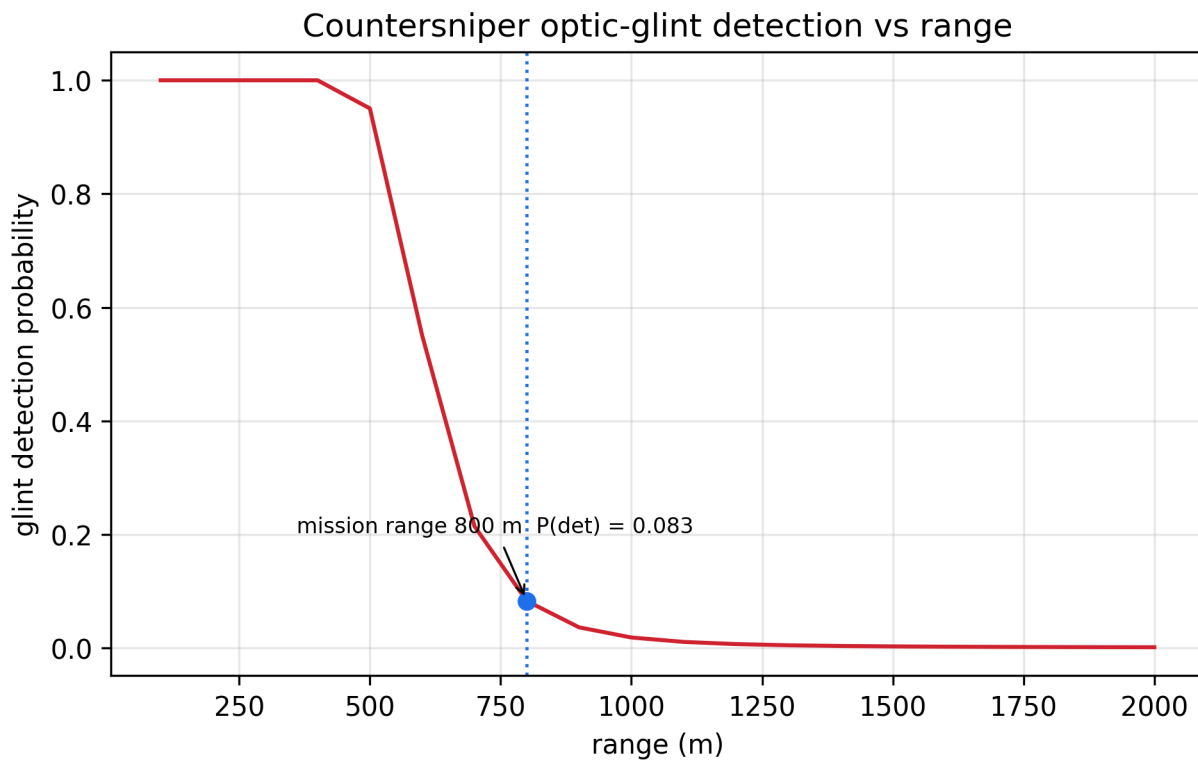


Figure 69: Countersniper optic-glnt detection probability vs range

18.5.8 Overwatch selection

The extraction site sits at (0, 0) — the terrain’s global elevation minimum, and therefore in a near-total blind spot. Sampling every 3 cells and keeping only positions within 12 cells leaves 16 in-range candidates, of which only 1 (0.062 fraction) can actually see the site over the intervening terrain and cover. Selection runs through `best_overwatch`, which takes the top-scored candidate *that can see the site*, not the top-scored candidate outright — a blind position keeps up to 0.60 of the score from proximity, cover, and elevation, so the two need not coincide. The selected nest at (6, 0) commands the site from 3000 m with 1451.7 m of elevation advantage and 0.50 concealment, scoring 0.714 (the overwatch figure).

18.5.9 Evidence verification

The verify stage’s backstop model accumulates 4 weighted checks (`identity_documents`, `debrief_consistency`, `backstop_telegraphy`, `cover_story_plausibility`) into a certainty of 0.670, rendering the verdict **conditional**, which matches the staged assessment’s verify gate. The two read no shared variable, but they are not independent evidence: both `koskov_scenario` and `koskov_verification` are hardcoded constant tables in one module, authored together to encode the film’s verification doubt. The match shows the staged and backstop scoring paths are mutually consistent given consistent authored inputs — a design check, not a corroborated finding.

18.6 Conclusion — LIVING DAYLIGHTS: findings, verdict, and what the mission establishes

SNIPER’S NEST demonstrates that a special-agent mission’s operational concepts can be turned into real, deterministic, testable algorithms. The package:

- solves the **sniper ballistics** problem — zero, windage, rules-of-engagement, and cello-case concealment — with a physically grounded point-mass integrator;
- runs a **staged, risk-scored defection protocol** whose hazard model renders the canonical scenario conditional at the verify gate, alongside an **evidence-verification** model whose weighted backstop checks land on the same verdict from a separately authored table;
- plans **mountain/desert routes with a measured cover term** on a seeded, reproducible terrain grid, and selects the **overwatch nest** — the top-scored position that actually sees the extraction site, with concealment and elevation advantage;
- fights the **countersniper duel** — mil-dot ranging and click solutions on the riflescope, radiometric optic-glnt detection, acoustic crack-bang ranging, and least-squares bearing triangulation of a hostile position;
- binds all of it to the frozen BOND-API `MissionProvider` protocol so the orchestrator can drive the film generically, with full deterministic provenance on every outcome.

The architecture keeps a pure, infrastructure-free domain core beneath a thin mission adapter, and forces every reported metric through a single deterministic solution — so the manuscript, the mission report, and the figures cannot drift. That claim is enforced, not asserted: the token cross-reference test fails on any placeholder the code does not generate, the hydration gate is tested against

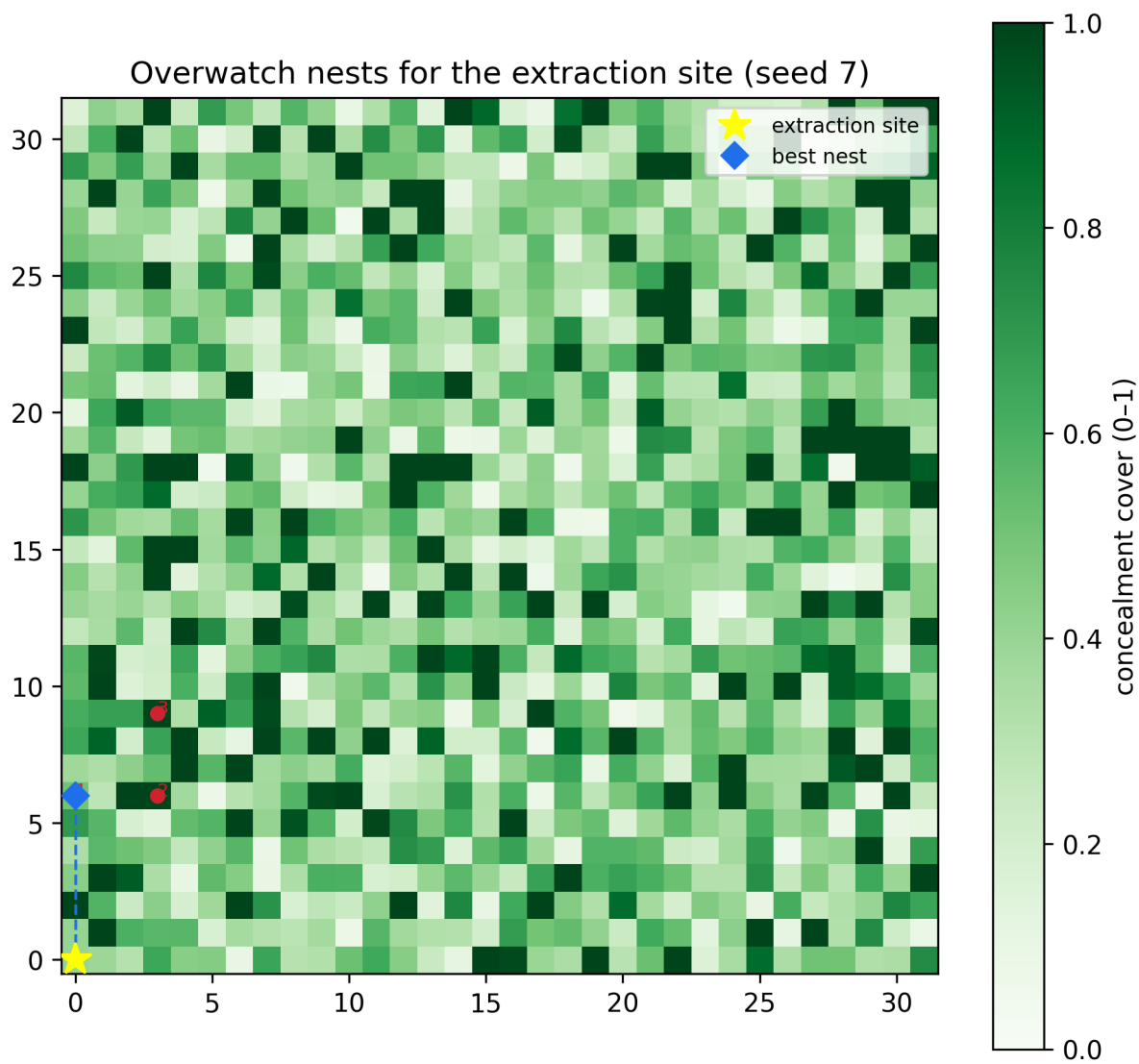


Figure 70: Ranked overwatch nests for the extraction site

a solution with each required section removed in turn, and the reader-facing parameter echo in `config.yaml` is pinned key-for-key to `settings.py`. Test count and achieved coverage live in `docs/_generated/COUNTS.md` rather than in this prose, so no stated figure can outlive the run that produced it.

Two pairs of models cross-check each other, and it is worth being exact about how much that is worth. The staged defection assessment and the backstop evidence check read no shared variable, yet both are constant tables authored in one module to encode one story beat, so their agreement is a design-consistency check and not independent corroboration. The planner’s route summary and the cover audit are genuinely different code paths — a scalar loop and a vectorised numpy pass, sharing no helper — so their agreement rules out implementation drift, but both encode a single concealment definition and neither can catch that definition being wrong. Each is therefore checked in the suite against a third calculation performed in the test itself, and the earlier claim that these pairings made the numbers “confirmed rather than self-reported” was too strong.

Two further limitations belong in the summary rather than in a footnote. The bearing-triangulation residual is identically zero for the mission’s two observers — the system is exactly determined, so that number measures nothing and no localization accuracy follows from it. And the exfil route is *not* a concealed route: the cover term has a real, measured effect against the same planner with the term disabled, but the resulting path still sits below the terrain’s own mean concealment at the shipped weighting.

The triggering image of the title, the *trigger-guard* hold, is not an arbitrary flourish: it falls out of the rules-of-engagement model as the correct action when an innocent is not a target and a hostile’s weapon is.

18.7 Experimental Setup — LIVING DAYLIGHTS: canonical scenarios, parameters, and configuration

18.7.1 Mission parameters

All parameters are fixed in the single source of truth (`src/the_living_daylights/settings.py`) and echoed for readers in `manuscript/config.yaml`; a test pins the echo to `settings.py` key for key, so the two cannot drift apart. Every value below is read back out of the live mission solution as a token rather than transcribed by hand.

18.7.1.1 Ballistics

Parameter	Value
Film / mission codename	the_living_daylights / SNIPER’S NEST
Muzzle velocity	790 m/s
Bullet mass	10.4 g
Caliber	7.8 mm
Ballistic coefficient	984 kg/m ²
Zero range	300 m
Sight height over bore	0.07 m
Target range / wind	300 m / 5.0 m/s

18.7.1.2 Riflescope optics

Parameter	Value
Magnification / objective	10x / 50 mm
Click value	0.10 mrad
Ranging silhouette / reticle span	1.80 m / 6.0 mils
Parallax focus range	150 m

18.7.1.3 Countersniper detection

Parameter	Value
Sun elevation / specular half-width	35° / 3.0°
Sun-observer azimuth offset	2.0°
Lens reflectivity / resolution cell	0.35 / 1.0 mrad
Glint evaluation range	800 m
Air temperature / crack-bang delay	20 °C / 2.0 s
Triangulation observers	2

18.7.1.4 Concealment rig

Parameter	Value
Cello-case interior (L × W × D)	1.30 × 0.45 × 0.24 m
Takedown components	4 (barrel_receiver, stock, optic, suppressor)

18.7.1.5 Defection handling and terrain

Parameter	Value
Defection stages	5 (contact, snatch, verify, exfiltrate, relocate)
Defection thresholds (go / conditional)	0.40 / 0.70
Backstop checks	4 (identity_documents, debrief_consistency, backstop_telegraphy, cover_story_plausibility)
Terrain seed / size / cell	seed 7 · 32×32 · 500 m
Route start → goal	(0, 0) → (31, 31)
Overwatch eye height / clearance	1.70 m / 0.30 m
Overwatch candidate step / max range	every 3 cells / 12 cells

18.7.2 Software environment

Built and verified under Python 3.14.6. Deterministic by construction: the mission solution uses a fixed seed (7) and no randomness beyond the seeded terrain, and every reported metric is recomputed from typed, documented pure functions. Scripts are thin orchestrators; business logic lives in `src/`.

The test count and achieved coverage are not quoted here — they are recorded live in `docs/_generated/COUNTS.md`, and the gate itself (`--cov-fail-under=90` on `src/`, line *and* branch) is what this manuscript claims.

Manuscript dated 2026-08-04 — the committed date from `manuscript/config.yaml`, not a hydration timestamp, so re-running the generator changes no byte of this page.

18.8 Reproducibility — LIVING DAYLIGHTS: verification gates, deterministic regeneration, and artifacts

18.8.1 Determinism

Every number in this manuscript is a pure function of fixed, documented inputs (`src/the_living_daylights/settings.py`). The mission solution uses a fixed seed (7); the only randomness in the whole stack is the seeded terrain generator in `terrain_ops.generate_terrain`, which is itself deterministic for a fixed seed. No wall-clock value enters any persisted artifact — there is no exception. The manuscript carries a date (2026-08-04), but it is the committed `paper.date` from `manuscript/config.yaml`, read like any other parameter, not the moment of hydration. An earlier version of this package stamped `datetime.now` into this page and into `output/data/manuscript_variables.json`, which made the byte-identical claim below false on every rerun; the clock has been removed rather than the claim weakened.

The BOND-API `MissionOutcome` carries a Provenance record with a stable `input_hash` (a sha256 of the canonical mission solution), so any report can be audited against the code without re-running the film’s I/O.

18.8.2 Artifacts and regeneration

Artifact	Producer	Location
Mission report / figures	<code>scripts/execute.py</code> , <code>scripts/z_generate_manuscript_variables.py</code>	<code>output/</code> (regenerable, git-ignored)
Resolved manuscript sections	<code>z_generate_manuscript_variables.py</code>	<code>output/manuscript/</code>
Ballistics + terrain figures	<code>src/the_living_daylights/figures/plots.py</code>	<code>../figures/ballistics_trajectory.png</code> , <code>../figures/terrain_exfil_route.png</code>
Countersniper + overwatch figures	<code>src/the_living_daylights/figures/plots.py</code>	<code>../figures/glint_detection.png</code> , <code>../figures/overwatch_nests.png</code>
Hydrated token map	<code>z_generate_manuscript_variables.py</code>	<code>output/data/manuscript_variables.json</code>

18.8.3 Verification gate

The package is verified with three separate commands, and it is worth stating precisely what each one does and does not enforce:

```
uv run pytest tests/ --cov=src --cov-fail-under=90 # tests + coverage + no-mock scan
uv run ruff check src/ scripts/ tests/ && uv run ruff format --check src/ scripts/ tests/
uv run mypy src/ scripts/
```

The `pytest` command enforces two things: $\geq 90\%$ line+**branch** coverage of `src/` measured over real computation, and — since `tests/test_no_mock_frameworks.py` — the absence of any mocking framework, parsed from the AST of every `.py` file under `src/`, `tests/`, and `scripts/`. That second gate carries its own positive control: a planted `MagicMock`, aliased `import`, and `mock` fixture must all be reported, so the scan cannot pass by being blind. An earlier version of this section claimed the single `pytest` command also enforced lint and type cleanliness. It does not; `ruff` and `mypy` are the second and third commands above and nothing in `tests/` invokes them.

The achieved figures are recorded in `docs/_generated/COUNTS.md` rather than quoted here, so no prose number can go stale against the gate.

Three tests carry the anti-drift load specifically:

- `test_all_manuscript_tokens_are_generated` cross-references every brace-delimited placeholder in these sections against the live token map, so none can survive un-generated.
- `test_require_outputs_rejects_an_incomplete_solution` gutted-section by gutted-section proves the hydration gate actually fires — it is checked against a solution missing each required section in turn, not merely against the healthy one.
- `test_config_mission_block_mirrors_settings` pins the reader-facing mission: `echo` in `config.yaml` to `settings.py`, key for key.

Re-running `z_generate_manuscript_variables.py` on one interpreter regenerates a byte-identical `output/` tree — every resolved manuscript section, the token JSON, and all four figure PNGs. The interpreter is the stated scope because `PYTHON_VERSION` is itself a token; nothing else in the tree varies between runs.

That is a claim, so it is bound by a test rather than by assertion. `test_regeneration_determinism.py` runs the generator twice as a real subprocess, deliberately straddling a wall-clock second boundary, and compares a SHA-256 of every produced file. A second test in that file AST-scans `src/` for any clock call (`datetime.now/utcnow`, `date.today`, `time.time`, `time.time_ns`, `time.monotonic`) reachable from the persisted-artifact path, and carries a positive control: a planted clock call in a temporary module must be reported, so the scan cannot pass by being blind. Note what the older `git status --short` check does *not* prove — `output/` is git-ignored, so that tree stays clean whatever the bytes say.

18.9 Scope and Related Work — LIVING DAYLIGHTS: boundaries, positioning, and relationship to the literature

18.9.1 Scope boundary

SNIPER'S NEST models the deterministic core of a special-agent mission only. Stated as explicit non-claims:

- **Aerodynamics** stop at the point-mass quadratic-drag law with a constant drag coefficient. There is no spin drift, no Magnus effect, no Coriolis term, no transonic drag rise, and no atmospheric layering; the barometric density helper is a single exponential scale height, not a standard-atmosphere table.
- **Radiometry** is a first-order energy budget. Sky radiance is a fixed daylight constant, the lens is a single specular reflector with one reflectivity, and there is no spectral, polarization, or atmospheric-extinction treatment. The detection curve is a normal CDF over SNR, not a measured ROC.
- **Acoustics** use only the flash-to-bang delay against a linear temperature-dependent sound speed. There is no ballistic shockwave (Mach cone) geometry, no multipath or reverberation, and no wind-borne refraction.
- **Localization** is a noiseless least-squares bearing intersection. Bearings carry no uncertainty model, so the reported residual measures geometric consistency of the observations, not positional accuracy. At the mission's two observers it does not even measure that: two non-parallel bearings give an exactly determined system whose residual is identically zero, so the value printed in the results is rounding noise and carries no information at all. A residual becomes informative only with three or more inconsistent bearings.
- **Terrain** is a seeded grid of elevation, landcover, and standing cover. Routing is 4-adjacent Dijkstra on a blended cost; visibility is a single-ray Bresenham walk with a fixed clearance and no earth curvature, refraction, or sub-cell relief. The planner optimizes that blended cost, not a full line-of-sight ballistic occlusion query.
- **Adversaries and humans** are not modelled at all: no stochastic opponent, no reaction times, no human factors, no communications network.

These simplifications are deliberate. They are what keep every result deterministic, fast, and auditable, and they are why the package can assert byte-identical reruns rather than statistical agreement.

18.9.2 Relation to the broader simulation canon

- **External ballistics** — the point-mass drag model with a ballistic coefficient is the standard small-arms treatment [McCoy, 1999]; the zeroing and windage geometry mirrors conventional rifle-zero practice, with near/far zero capture. Mil-dot ranging follows established sniper doctrine [U.S. Department of the Army, 1994].
- **Countersniper detection** — acoustic shooter localization from crossed bearings is the operating principle of fielded countersniper systems [Duckworth et al., 1996, Ledeczi et al., 2005]; the glint model is an energy-budget reduction of optical retro-reflection detection, and the detection probability is the classical signal-detection formulation [Van Trees, 1968] against a reference solar irradiance [ASTM International, 2020]. The temperature-dependent sound speed is the standard linear approximation [Kinsler et al., 2000].
- **Terrain visibility** — line of sight over a rasterized cell walk [Bresenham, 1965] and the accumulation of visible cells into a viewshed are the canonical GIS treatment [Fisher, 1996]. The contribution here is not the viewshed itself but the *ranking*: turning visibility into an overwatch-selection objective alongside proximity, concealment, and elevation advantage.
- **Risk-gated decision procedures** — the discrete, thresholded approval path of `defection_protocol.py` is a classic monotone decision chain; its weighted hazard scoring follows a transparent exposure-weighting scheme. The contribution is pairing a staged plan gate with a separately parameterised evidence-verification model. Both scenarios are authored constant tables, so the pairing is a structural one; agreement between them is not independent evidence and is not offered as such.
- **Terrain routing** — Dijkstra’s algorithm over a cost grid [Dijkstra, 1959c] is the canonical least-cost routing method; the contribution here is the *cover* term that folds concealment into route cost — reported against two measured baselines (the same planner with the term off, and the grid’s own mean concealment) rather than asserted — plus a cover audit that can grade any proposed path, not only the planner’s own.

18.9.3 Intended use

The package is one film in the PROJECT BOND suite. Its role is to implement the frozen BOND-API `MissionProvider` contract with a real, deterministic, well-tested domain core, and to serve as a canonical example of converting a template scaffold into a standalone mission package. It is local-only and is discovered by the suite orchestrator via `bond_api`.

18.10 Sources — LIVING DAYLIGHTS: bibliography

Friedman [2026a]; BOND [2026d]; U.S. Department of the Army [1994]; Duckworth et al. [1996]; ASTM International [2020]; Van Trees [1968]; Fisher [1996]; Kinsler et al. [2000]; Ledeczi et al. [2005]; McCoy [1999]; Dijkstra [1959c]; Bresenham [1965]

19 Licence to Kill (1989) — WAVEKREST

film package · package codename WAVEKREST. Mission **ROGUE**: detect layering and integration in the Sanchez money-laundering network; forensic anomaly scan of the Wavekrest-class mothership; risk-optimised rogue-00 engagement plan under a revoked licence; quantify narcotics-finance economics and cash-manifest feasibility; route the operation and select the gadget loadout under risk and weight; schedule covert actions against adversary watchlist accumulation.

19.1 Concepts — WAVEKREST: domain and operational focus

narcotics finance, rogue ops

19.2 Abstract — WAVEKREST: mission summary

Licence to Kill: ROGUE — Special-Agent Mission Software for Laundering-Graph, Vessel-Forensic, Rogue-Policy, Narcotics-Finance, Logistics, and Surveillance Analysis — Six deterministic domains — graph detection, marine physics, dynamic programming, cash economics, risk-adjusted routing, and survival-optimal scheduling — behind one unsanctioned operation — is a deterministic, fully-tested special-agent mission software package implementing the ROGUE mission from the film *Licence to Kill* (1989). Six interoperating domain models sit behind the frozen BOND-API `MissionProvider` protocol. The first three answer the field analyst’s original questions: 1. a **money-laundering graph** model of the Sanchez cartel’s placement–layering–integration flow, with cycle, funnel, and dirty-flow detectors; 2. **first-principles vessel forensics** of the Wavekrest-class mothership from marine physics (IMO gross tonnage, hydrostatic displacement, fuel endurance, route plausibility); 3. a **rogue-00 operation risk model** in which an operator whose licence has been revoked must trade mission gain against escalating adversary alert and blowback, solved exactly by finite-horizon dynamic programming. The remaining three carry the campaign from the ledger into the world: 4. **narcotics-finance economics** — the supply-chain price/purity ladder and the cash physics (mass, volume, hold manifest) behind the cartel’s US\$5,000,000 courier parcel; 5. **operations logistics** — risk-adjusted Dijkstra routing of the Wavekrest campaign, exact 0/1-knapsack gadget loadout, and trip projection against fuel endurance; 6. **surveillance / watchlist** — a logistic detection model over a covert action sequence, with exact and heuristic survival-optimal ordering. On the canonical seeded network (codename ROGUE, seed 0) the analysis detects 1 money-returning cycle(s) and 16 funnels across 22 nodes and 89 directed edges, with 720 layering chains (the exact count) enumerated — a composite layering score of 0.513 and an integration score of 0.653. profile resolves a gross tonnage of 356 and flags 2 forensic anomalies (score 2.0). The rogue-00 dynamic program over 10 steps returns an optimal expected mission value of 1.665 with a stealth opening, closing on overt — evidence that, under a revoked licence, unsanctioned escalation is strictly a last resort, worth taking only when there is no remaining horizon left to protect. The layered logistics route cuts cumulative interdiction risk by 30.7% versus the direct leg (0.208 vs 0.300), and the covert campaign’s survival-optimal sequence completes undetected with probability 0.263 before watchlist compromise at action 5. Every result is deterministic and reproducible: fixed seed, no random draws in the detectors, no wall-clock in persisted outputs, and $\geq 90\%$ line + branch test coverage on the source tree.

19.3 Introduction — WAVEKREST: mission framing, the operational problem, and how to read this chapter

19.3.1 The ROGUE mission

In *Licence to Kill* (1989) an operator pursues the drug lord Franz Sanchez after his 00 licence is revoked, assembling an unsanctioned campaign that moves from the street level of the cartel’s cash economy, through the Wavekrest-class mothership used to smuggle the proceeds, to a final confrontation. This package formalises that campaign as a research problem: a *special-agent mission* whose analytical core quietly mirrors the six things a field analyst must actually determine. The first three concern what the adversary is doing.

1. **Where is the money going?** The cartel’s value is laundered in the textbook three stages — *placement* of cash, *layering* through a web of intermediaries, and *integration* into legitimate banks. We model this as a directed financial-flow graph and build deterministic detectors that score how heavily the structure is being layered and how dilute the dirty value has become (Section 2, `src/licence_to_kill/laundering_graph.py`).
2. **Is the mothership lying about what it carries?** A fishing vessel cannot silently move contraband without its own physics giving it away — a hold full of something denser than fish, a draft that disagrees with the declared displacement, a fuel budget that cannot cover its route. We compute those quantities from first principles (`src/licence_to_kill/vessel_forensics.py`).
3. **What should the operation do next?** With a revoked licence the operator has no sanctioned exit; every action escalates the adversary’s alert level and precipitates blowback. We solve the optimal engagement policy exactly with dynamic programming (`src/licence_to_kill/rogue_ops.py`).

The remaining three concern what the operation itself can physically afford.

4. **How much money is actually moving, and can it move?** The film opens on an intercepted courier carrying a cash parcel. A supply-chain price and purity ladder says what that value represents in product; US-currency note physics say what it weighs and how much hold it occupies, and therefore whether one hull can carry it at all (`src/licence_to_kill/narcotics_finance.py`).

5. **By what route, carrying what?** Legs between the operational hubs trade distance, dollar cost, and interdiction risk against each other, and a weight-bounded gadget loadout has to be chosen before departure. Both are classical optimisation problems solved exactly — shortest path over a risk-additive weight, and 0/1 knapsack (`src/licence_to_kill/operations_logistics.py`).
6. **How long before the operator is burned?** Every covert action teaches the adversary’s watchlist something. Survival to the end of the campaign is a product of per-step non-detection probabilities, and the only free variable is the order in which the actions are taken (`src/licence_to_kill/surveillance.py`).

The six are deliberately coupled at the scenario level rather than in code: the Wavekrest-class hold sized in Section 2.2 is the same hold the cash manifest must fit in Section 2.4 and the same hull whose endurance bounds the voyage in Section 2.5, because all three read the one canonical parameter set.

19.3.2 Design principles

The package follows the BOND guardrails exactly. The six domain modules are *pure* — infrastructure-free and bond-api-free, importable in any Python 3.10+ environment (verified by `tests/test_mission.py`). The `MissionProvider` adapter in `src/licence_to_kill/mission.py` is the only module that touches the frozen `bond-api` protocol, and it simply wires the domain results into the five-stage mission lifecycle (brief → recon → plan → execute → debrief). Everything is deterministic: the network is built from a seeded generator, the forensic physics are closed-form, the dynamic program is exact backward induction with no Monte Carlo, the routing and knapsack solvers are exact, and the survival-optimal action order is found by exhaustive permutation search at this problem size.

Each domain also exposes a `canonical_*_params` function that is the sole declaration of its scenario. The mission adapter, the manuscript token generator, and the figure generators all read those functions rather than carrying their own copies, so a figure cannot depict a scenario the prose does not describe.

This manuscript is produced from `manuscript/config.yaml` and `src/licence_to_kill/manuscript_variables.py`: every numeric value in the prose is injected as a generated token and never hand-authored.

19.4 Methodology — WAVEKREST: the analytical models and algorithms that drive the mission

This section specifies the six deterministic algorithms that constitute the ROGUE analytical core. Deep familiarity with each module’s source is the intended reader path: `src/licence_to_kill/laundering_graph.py`, `vessel_forensics.py`, `rogue_ops.py`, `narcotics_finance.py`, `operations_logistics.py`, and `surveillance.py`.

Every declared constant below is reported as an injected token drawn from that module’s `canonical_*_params` function or its named module constants, so changing a parameter in source rewrites this section rather than silently contradicting it.

19.4.1 2.1 Money-laundering graph

A laundering network is a directed, weighted graph $G = (V, E)$ whose nodes V are financial entities (collectors, shell fronts, seafood/currency fronts, banks, legitimate exporters) and whose directed edges E carry a positive value flow. Dirty value is *placed* at source nodes, moves through the graph, and *integrates* at sink nodes.

Cycle detection. Money-returning loops are the signature of layering. We enumerate elementary cycles by depth-limited depth-first search, rotating each discovered cycle to a canonical form (start at its lexicographically smallest node) so duplicates collapse (`TransactionGraph.find_cycles`).

Funnel analysis. A node is a *collector* when it fans in from ≥ 2 unique sources with a degree-imbalance ratio $\rho = (\text{sources} + 1)/(\text{targets} + 1) \geq \tau$, and a *distributor* when it fans out to ≥ 2 unique targets with $\rho \leq 1/\tau$ ($\tau = 1.5$). Trivial linear pass-throughs are excluded.

Dirty-flow propagation. Dirty mass is conservatively re-distributed along edges in proportion to edge weight; a node with no out-edges retains its mass (it is a sink). Iteration converges to the pooling dirty distribution (`TransactionGraph.propagate_dirty`). From it we derive the per-sink dirty fraction $d_i = \text{dirty}_i / \text{in_amount}_i$, the integration score $1 - \bar{d}$ (how diluted the placed value has become), and the Shannon entropy of the dirty distribution across sinks (how dispersed it is) [Shannon, 1948b].

Composite layering score. Three normalised components — cycles, layering-chain depth, and funnel imbalance — are blended $0.4 \cdot C_{\text{cyc}} + 0.3 \cdot C_{\text{depth}} + 0.3 \cdot C_{\text{funnel}}$ into a single score in $[0, 1]$ (`analyze_laundering`). The weights are declared as `LAYERING_WEIGHTS` in source and injected here; they are heuristic, and Section 7 says so.

19.4.2 2.2 Vessel forensics

For a declared `VesselProfile` of the Wavekrest-class mothership we compute, from first principles:

- **Enclosed hull volume** $V = L \cdot B \cdot D \cdot C_b$.
- **Gross tonnage** (IMO 1969): $GT = K_1 V$ with $K_1 = 0.2 + 0.02 \log_{10} V$.
- **Hydrostatic displacement** $\Delta = \rho_{\text{sw}} \cdot L \cdot B \cdot \text{draft} \cdot C_b$, $\rho_{\text{sw}} = 1.025 \text{ t/m}^3$, $C_b = 0.62$.

- **Fuel endurance:** at speed v the required shaft power follows the cubic law $P(v) = P_{\max}(v/v_{\max})^3$, burning $P(v) \cdot \text{SFC}$ kg/h (SFC = 0.195 kg/kWh), giving range nm = (fuel mass/burn rate) $\cdot v$.

The forensic scan (`anomaly_scan`) then flags physical implausibilities: a cargo bulk-density above the fish ceiling (1.4 t/m³), a declared displacement more than 15% from the draft-predicted value (hidden ballast), a route whose required average speed exceeds capability, a voyage beyond the fuel endowment, or an over-powered hull. The scan deliberately does *not* trap errors from the underlying physics: a profile that cannot support an endurance computation raises rather than being scored as anomaly-free, because “no anomaly” and “could not check” are different findings and only one of them is safe to report.

19.4.3 2.3 Rogue-00 risk model

A revoked licence means the operator faces a finite-horizon control problem. State is (a, e) : adversary *alert* and cumulative *exposure*. Each step the operator picks an intensity $i \in \{\text{stealth}, \text{covert}, \text{overt}\}$ with gain g_i and visibility v_i ; alert rises $a \leftarrow \min(a_{\max}, a + v_i)$, exposure rises by one step, and *blowback* (a retaliatory strike destroying the period’s value) arrives with logistic hazard $p = \sigma(\beta_0 + \beta_a a + \beta_e e)$.

We solve exactly by backward induction over the discrete (step, a, e) lattice (`solve_optimal_policy`), producing the expected optimal value, the first-period action, and the escalation curve that follows. No randomness is involved — the result is reproducible to the last float.

Both state dimensions are quantised: exposure onto 21 cells and alert onto the integers 0..10. The visibility ladder is therefore truncated onto the alert lattice by a single shared transition (`next_alert`), which the projection and valuation functions use as well. That sharing is what makes the model self-checking: replaying the DP’s own optimal action sequence through `expected_value_of_policy` must reproduce `optimal_value` exactly, and the test suite asserts it does. A model whose optimiser and whose reported trajectory used different transitions would pass every individual test and still describe a process nobody optimised.

19.4.4 2.4 Narcotics-finance economics

The money behind the cartel is modelled in `narcotics_finance.py` in two layers. First, the **supply chain**: a base production cost per kilogram is multiplied through the distribution stages (export $\rightarrow$ wholesale $\rightarrow$ street) to give the stage prices (export $\rightarrow$ wholesale $\rightarrow$ street), and purity adjustment divides the retail price by the purity fraction so unit economics are stated per *pure* gram (`price_chain`, `purity_adjusted_price`). Second, the **cash itself**: using the mass and volume of a US currency note (1.0 g, 1.13 cm³ [[United States Bureau of Engraving and Printing, 2020](#)]), we convert a parcel’s value into whole notes, kilograms of mass, and cubic metres of volume (`cash_mass_kg`, `cash_volume_m3`), then check whether that parcel physically fits the Wavekrest-class hold — volume *and* mass — and how many trips it needs (`manifest_feasibility`, `trips_required`). Note counts round up: a partial note cannot be carried, so the conservative direction is the only defensible one for a feasibility check.

19.4.5 2.5 Operations logistics

The physical campaign is planned in `operations_logistics.py`. The 5 transport legs between the film’s 4 hubs (Isthmus City, Bogota, Hong Kong, Florida) each carry a distance, a dollar cost, and an interdiction risk. `shortest_path` runs Dijkstra’s algorithm [[Dijkstra, 1959c](#)] over one of three additive weights; for risk the per-leg weight is $-\ln(1-r)$ so the cumulative survival product becomes a sum and the returned route minimizes the honest $1 - \prod(1-r_i)$ cumulative-risk metric. The baseline that route is reported against is the **direct leg** itself — the single IC $\rightarrow$ FL edge, read straight out of the network by `direct_leg` — and not the distance-minimal path, which on this network happens to be that same leg and so cannot be told apart from it by inspection alone. `gadget_knapsack` solves the 0/1 equipment-loadout problem exactly by dynamic programming [[Kellerer et al., 2004](#)] over gram-scaled weights, selecting from 7 candidate gadgets under a 25 kg budget; the per-item selection flag is recorded during that item’s own DP pass, because a final-array equality check false-negatives once a later item has already absorbed the same capacity cell. `trip_plan` projects the chosen route’s legs against the vessel’s 10 kn cruise, its 4,257 nm endurance, and 24 h refuelling stops, counting stops and elapsed time.

19.4.6 2.6 Surveillance and watchlist

An unsanctioned operator’s exposure to the adversary’s surveillance apparatus is modelled in `surveillance.py`. Each covert action has a *signature* (detectability) and a *watch increment* (how much it teaches the watchlist); the step- t detection hazard is the logistic $p_t = \sigma(\beta_0 + \beta_w w_t + \beta_s s_i)$ where w_t is the accumulated watchlist weight. A sequence survives only if every step goes undetected, so its success probability is $\prod_t(1-p_t)$. The order that maximises survival is found exactly by exhaustive permutation search for up to 8 actions (`exact_action_order`) and by a value-per-exposure ratio rule (Smith’s rule [[Smith, 1956](#)]) beyond that (`heuristic_action_order`); `compromise_step` reports the first step at which cumulative detection crosses the threshold 0.50. The canonical detection coefficients are $\beta_0 = -4.0$, $\beta_w = 1.2$, and $\beta_s = 1.5$.

19.5 Results — WAVEKREST: measured outcomes, headline numbers, and what they establish

All results below are deterministic outputs of the canonical seeded run (seed 0) and are re-computed verbatim by `src/licence_to_kill/manuscript_variables.py` — the plain numbers you read are injected tokens, not hand-typed values.

19.5.1 3.0 Headline results at a glance

Domain	Key quantity	Value
Laundrying	Composite layering score	0.513
Laundrying	Integration score	0.653
Vessel	Forensic anomaly score	2.0
Rogue	Optimal expected mission value	1.665
Finance	Courier-parcel cash mass	50.0 kg
Logistics	Cumulative route interdiction risk	0.208 vs 0.300 direct
Surveillance	Probability of completing undetected	0.263

Each row is developed in its own subsection below; the numbers are the same injected tokens, so the table cannot disagree with the analysis.

19.5.2 3.1 Layering and integration in the Sanchez network

The canonical network built by `build_sanchez_network(seed=0)` contains **22** nodes and **89** directed edges, carrying a placed dirty value of **132.2** currency units. The detectors identify:

- **1** money-returning cycle(s) — the direct layering loop that returns cash through the shell grid;
- **16** funnels (collectors that concentrate the flow and distributors that scatter it);
- **720** placement→integration chains — the *exact* count, not a capped sample: the enumeration bound (1000) sits strictly above the true number, so every chain is found and the depth component is computed over the full population.

The composite **layering score** is **0.513** (of a possible 1), reflecting the substantial but deliberately incomplete obscuring structure. The **integration score** is **0.653**: the placed dirty value is diluted to that degree by the legitimate exporter flows feeding the same banks, with a **mixing entropy** of **1.381** nats across the 4 integration sinks.

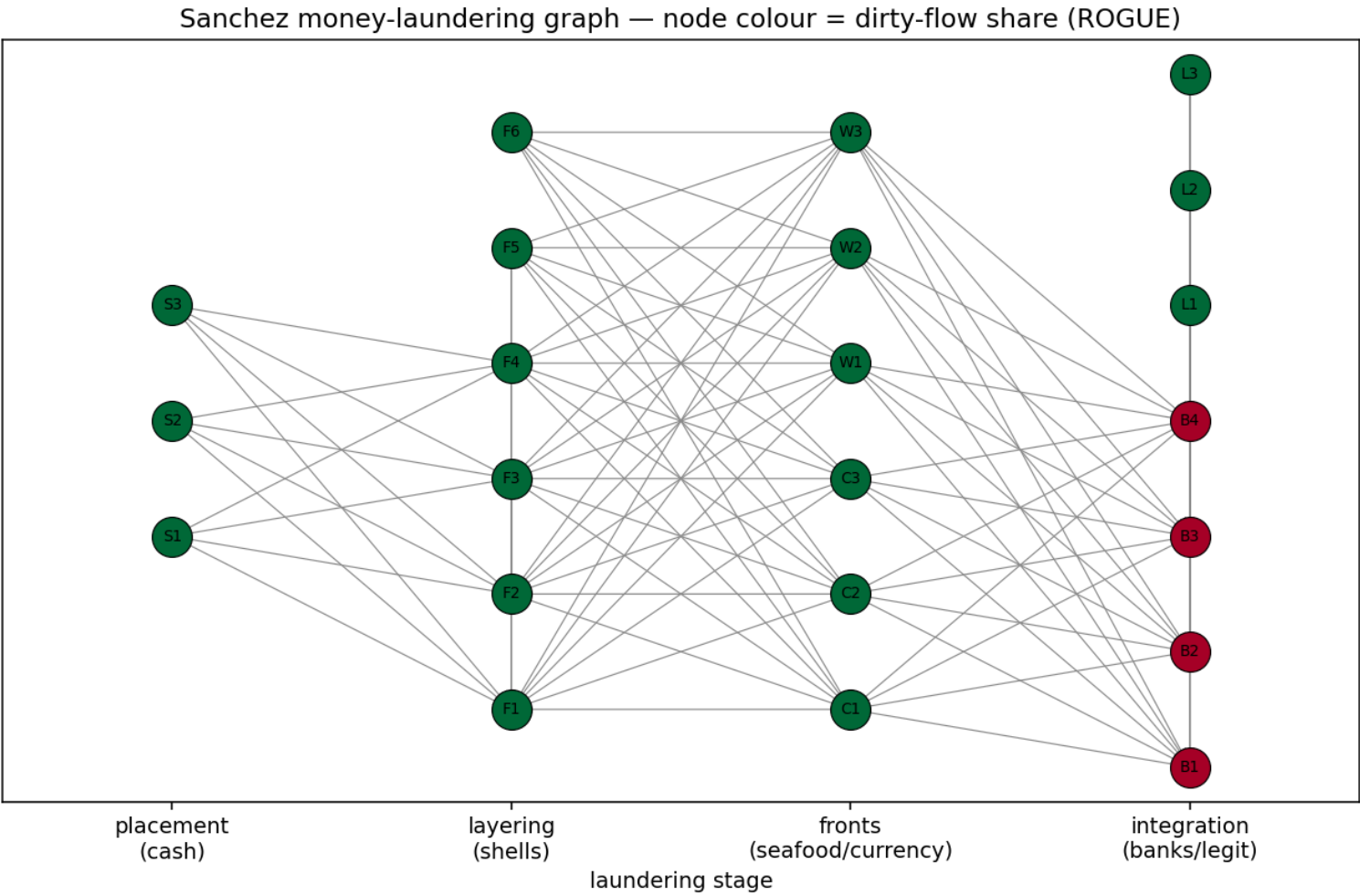


Figure 71: Sanchez money-laundering graph; node colour is the dirty-flow share, laid out left-to-right by laundering stage.

19.5.3 3.2 Wavekrest-class vessel forensics

The canonical profile resolves:

Quantity	Value
Enclosed hull volume	1357 m ³
Gross tonnage (IMO)	356
Draft-predicted displacement	985 t
Usable fuel	51.0 t
Full-tank endurance	4257 nm

The forensic scan flags **2** anomalies (aggregate score 2.0): a cargo bulk-density inconsistent with boxed fish, and a declared displacement that disagrees with the draft-predicted value — consistent with concealed contraband load. The Wavekrest’s own physics betray its cargo before its crew does (the vessel figure).

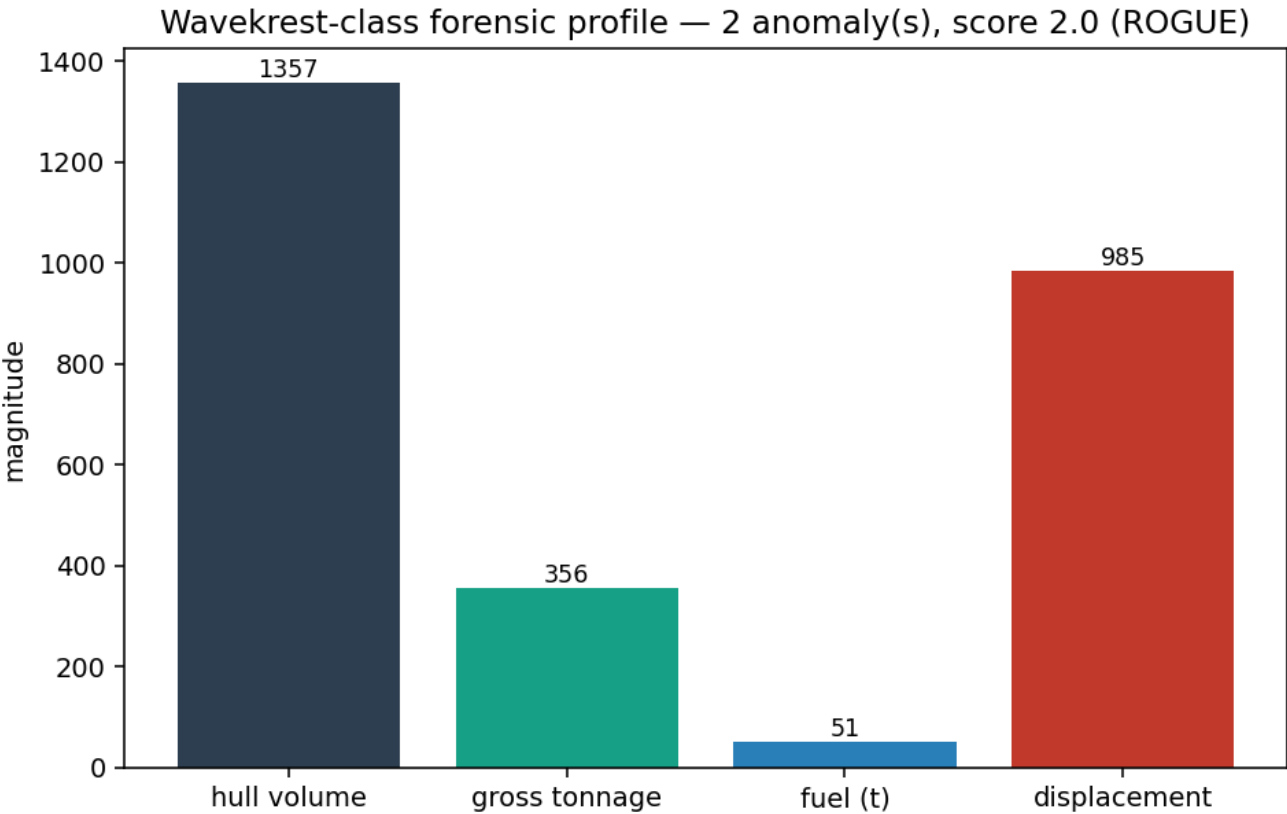


Figure 72: Wavekrest-class forensic profile: hull volume, gross tonnage, fuel, and draft-predicted displacement.

19.5.4 3.3 Rogue-00 optimal policy

For the canonical revoked-licence model (horizon 10 steps, alert ceiling 10, action card *stealth* / *covert* / *overt*), the dynamic program evaluates **2310** state cells and returns:

- an **optimal expected mission value** of **1.665**;
- a **stealth** opening — the least-escalating intensity, confirming that a revoked-licence operator must build capability surreptitiously before any overt strike;
- a close on **overt**. The optimal policy holds the least-visible intensity for the whole horizon and spends its escalation on the final step, when there is no remaining future for the raised alert to damage — the classic finite-horizon end effect, arrived at here without being written in;
- a **peak blowback hazard** of **0.378**, reached on the last step, and a **peak alert** of **3.0** — the level the closing overt action *leaves behind*, which is therefore also the terminal alert (3.0) (the rogue figure). Read those two numbers carefully, because they say different things. No step in the horizon is ever exposed to that alert: it is raised by the final action and the mission then ends, which is exactly why the dynamic program is willing to buy it. Every step the operator actually faces is *entered* at alert 0, so the hazard he faces climbs on **accumulated exposure** alone — on the integer alert lattice the stealth intensity’s visibility rounds to no alert increase at all. That is a boundary of the quantisation, not a claim that stealth is free — Section 7

states it plainly. The distinction is not cosmetic: a peak taken over the alert each step is entered with would report a flat zero for this policy and read as “the optimum never escalates”, which is false.

Rogue-00 optimal policy — expected value 1.665, first action stealth (Licence to Kill)

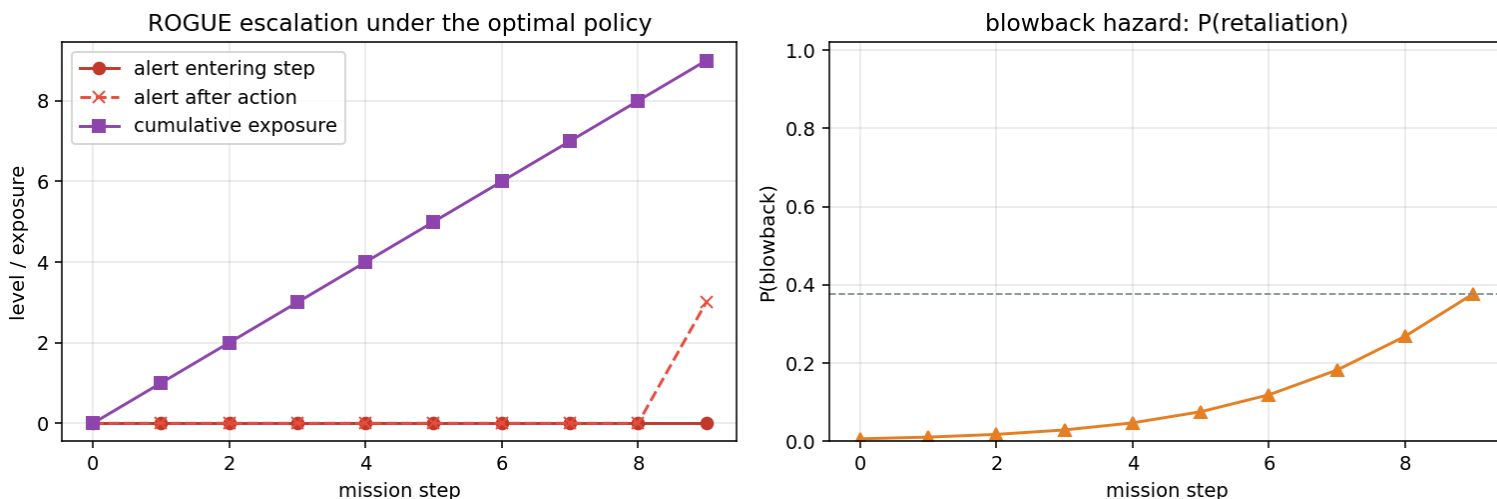


Figure 73: Rogue-00 escalation under the optimal policy. Two alert series are drawn: the level each step is *entered* with (flat at zero here) and the level each step’s own action *leaves behind* — the second is the only one that shows the closing overt escalation. Exposure and the blowback hazard climb throughout.

19.5.5 3.4 Narcotics-finance economics

The canonical courier parcel is valued at **\$5,000,000** in **\$100** notes — **50,000** bills. US-currency note physics give a bulk-cash mass of **50.0 kg** and a volume of just **0.0565 m³** (bulk density ≈ 885 kg/m³). From a base production cost of **\$1,500/kg** the supply chain (export $\rightarrow$ wholesale $\rightarrow$ street) prices a kilogram at **\$9,000** wholesale and **\$18,000** at street level (**\$18** per gram), or **\$30** per *pure* gram at 60% purity — the same value is quietly present at every stage.

Against the Wavecrest-class hold (520 m³, 300 t usable mass), the parcel is **yes** to fit and needs **1** trip(s). The money is compact; the hard constraint is the cover cargo that must blend with it — not the cash itself.

19.5.6 3.5 Operations logistics

Risk-optimal routing over the campaign network returns the layered path **IC $\rightarrow$ HK $\rightarrow$ FL** (2 legs, 17,000 nm, costing **\$270,000**) with a cumulative interdiction risk of **0.208** — against **0.300** for the direct single leg, a **30.7%** reduction — the same pairing the finance graph demonstrates in the money layer (the operations figure). The voyage requires **3** refuelling stops (against the Wavecrest’s 4,257 nm endurance at 10 kn) for roughly **74** days at sea (**1,772** hours). The gadget loadout under a 25 kg budget selects **5** of 7 candidates (including the Walther PPK, explosive charges, and the surveillance kit) worth **32** mission points at **17.7 kg**, computed by exact 0/1-knapsack dynamic programming over the full capacity lattice.

On *this* instance that exactness buys nothing observable: a greedy value-density pass returns the same 5 items at the same value and the same weight, which `tests/test_operations_logistics.py::test_greedy_matches_exact_on_the_canonical_instance` records rather than hides. The claim being made for the DP is therefore not “it beat greedy here” — it did not — but that it is optimal on *every* instance, where greedy is not; `test_greedy_value_density_is_suboptimal_on_a_counterexample` exhibits an instance on which the greedy pass leaves value on the table.

19.5.7 3.6 Surveillance and watchlist

Across the 5-action covert campaign, the survival-optimal order — opening with *observe_isthmus_house*, the lowest- signature move, and closing with *infiltrate_compound* — achieves a probability of completing undetected of **0.263**. Cumulative detection crosses the 0.50 threshold on the **5th** action, and the peak step hazard reaches **0.633**: the adversary’s watchlist eventually prices the campaign in, and the operator wins only by doing the cheaply-detectable work first and the compounding-mistake last.

Because $5 \leq 8$, this ordering is the exhaustively-verified optimum rather than the ratio-rule heuristic — no permutation of these actions survives more often.

ROGUE operations network — risk-optimal route IC → HK → FL (cum. risk 0.208 vs direct leg 0.300)

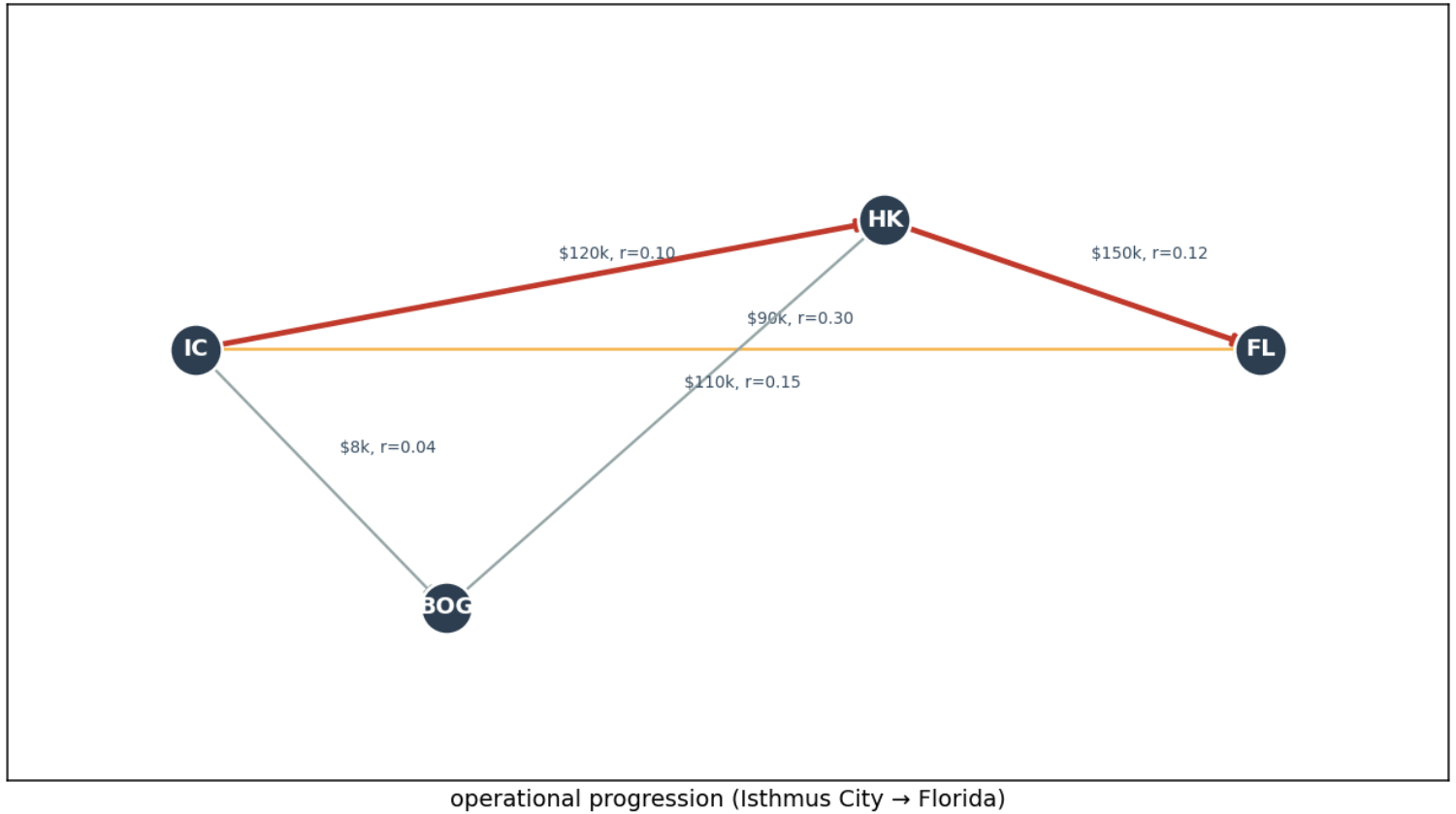


Figure 74: ROGUE operations network with the risk-optimal route highlighted against the direct single leg.

19.6 Conclusion — WAVEKREST: findings, verdict, and what the mission establishes

Licence to Kill: ROGUE — Special-Agent Mission Software for Laundering-Graph, Vessel-Forensic, Rogue-Policy, Narcotics-Finance, Logistics, and Surveillance Analysis demonstrates that a special-agent mission package can be entirely deterministic and still analytically rich. The six domains — the Sanchez money-laundering graph, the Wavekrest-class vessel forensics, the rogue-00 risk model, narcotics-finance economics, operations logistics, and the surveillance/watchlist model — each carry a real, testable algorithm, and together they render the ROGUE campaign’s field questions answerable in closed form:

- layering and integration are *measurable* (layering score 0.513, integration score 0.653);
- covert cargo is betrayed by marine physics (2 anomalies flagged on the canonical mothership);
- unsanctioned operations have an exact optimal policy — and that policy begins with stealth, not with a frontal assault, saving the overt move for the step with no future left to protect (optimal value 1.665);
- the money is bounded by cash *physics* before any courier flies (50,000 bills, 50.0 kg, 1 hold-trivial trip);
- routing the campaign cheaply against risk matters — the layered leg cuts cumulative interdiction by 30.7% (0.208 vs 0.300);
- an unsanctioned operator’s cover is finite — survival to completion is 0.263, and order is the operator’s only free variable.

The six domains stay honest about each other because they share one declaration: the `canonical_*_params` functions are read by the mission adapter, the manuscript token generator, and the figure generators alike, and the `mission:` block of `manuscript/config.yaml` is asserted equal to them by the test suite. A parameter cannot change in one place and quietly survive in another.

Because the entire pipeline runs on a fixed seed (0) with no random draws in the detectors and no wall-clock anywhere in the persisted outputs, every figure and every table in this manuscript is reproducible byte-for-byte on a fixed interpreter and NumPy build — the two environment-derived token values, and the whole of the claim’s scope. That is a tested property, not a comment: `test_regeneration_is_byte_identical` reruns the generator and compares every emitted file. As a BOND package, `licence_to_kill` exposes all of this through the frozen `bond-api MissionProvider` contract, so the fleet orchestrator can discover and drive it as a first-class citizen of the 27-film suite. The revoked licence changes the operator’s tactics — never the rigour of the analysis.

19.7 Experimental Setup — WAVEKREST: canonical scenarios, parameters, and configuration

All experiments are deterministic and reproducible under Python 3.14.6 with NumPy 2.4.2.

19.7.1 Where the parameters actually live

Each domain module declares its own scenario in a `canonical*_params` function, and those functions are the *executable* source of truth. The domain core is deliberately dependency-free (standard library plus NumPy) and does not read YAML, so `manuscript/config.yaml`'s `mission:` block is a **declared mirror** of those functions rather than their input.

That mirror is enforced, not trusted: `tests/test_canonical_params.py` asserts every key of the `mission:` block equal to the value the code computes with, and fails the suite if the two drift. The same tests assert that the declared search bounds are genuinely *consumed* — halving the path cap must halve the enumerated chains — so a parameter cannot become decorative without a test noticing. Nothing below is hard-coded in prose; every value is an injected token read from the same functions.

19.7.2 Laundering network

- Placement sources (street collectors): 3 nodes, S1–S3.
- Integration sinks (banks): 4 nodes, B1–B4, mixed with legitimate exporter flows (L1–L3) into the same sinks so integration is non-trivial.
- Edge amounts drawn from a seeded generator (`seed 0`): the same seed always yields the same graph and therefore the same report.
- Detector bounds: cycle search depth 8, capped at 50 cycles; layering-path enumeration capped at 1000 chains; funnel degree-imbalance threshold 1.5.
- Source of truth: `canonical_laundering_params` in `laundering_graph.py`.

19.7.3 Vessel forensics

- Profile: the canonical Wavekrest-class mothership (`WAVEKREST_CLASS` in `vessel_forensics.py`) — cargo volume 520 m³, block coefficient 0.62, maximum speed 12.5 kn.
- Observed quantities that expose covert modification: a declared cargo mass of 900 t (above the 1.4 t/m³ fish ceiling for that hold) and a declared displacement of 700 t (outside the 15% tolerance around the draft-predicted value).
- Physical constants: seawater density 1.025 t/m³, diesel specific fuel consumption 0.195 kg/kWh.
- Source of truth: `canonical_vessel_params` in `vessel_forensics.py`.

19.7.4 Rogue-00 risk model

- Horizon 10 steps; discrete alert states 0..10.
- Action card *stealth* / *covert* / *overt* with visibility and gain ladders, truncated onto the integer alert lattice by the shared `next_alert` transition.
- Continuous exposure quantised onto a 21-cell lattice (`exposure_cap 20`), giving a backward-induction lattice of 2310 state cells.
- Source of truth: `canonical_rogue_params` in `rogue_ops.py`.

19.7.5 Narcotics-finance economics

- Intercepted courier parcel: US\$5,000,000 in \$100 notes (the film's opening intercept), against the Wavekrest-class hold (520 m³, 300 t usable mass).
- Supply chain: base cost \$1,500/kg through the export → wholesale → street multipliers, at 60% purity. US-currency note mass 1.0 g and volume 1.13 cm³ per note [[United States Bureau of Engraving and Printing, 2020](#)].
- Source of truth: `canonical_finance_params` in `narcotics_finance.py`.

19.7.6 Operations logistics

- Campaign network: 5 legs across 4 hubs — Isthmus City (IC), the refinery at Bogota (BOG), Hong Kong (HK), and Florida (FL) — each with a distance, cost, and interdiction risk. The routed problem is IC → FL.
- Weight keys: **risk** (via $-\ln(1-r)$ survival weights), **cost**, **distance**.
- Gadget budget 25 kg over 7 candidate items from the film's loadout; Wavekrest cruise 10 kn and 4,257 nm endurance with 24 h refuel stops.
- Source of truth: `canonical_logistics_params` in `operations_logistics.py`.

19.7.7 Surveillance and watchlist

- 5 covert actions, each with a signature, watch increment, and value.
- Detection logistic $\beta_0 = -4.0$, $\beta_{\text{watch}} = 1.2$, $\beta_{\text{signature}} = 1.5$, threshold 0.50; exact survival-optimal ordering up to 8 actions with a ratio-rule fallback beyond.
- Source of truth: `canonical_surveillance_params` in `surveillance.py`.

19.7.8 Software environment

- Package version 0.1.0; film identity `licence_to_kill`; codename `ROGUE`.
- Rendered variable map written to `output/data/manuscript_variables.json`; canonical mission results to `output/data/mission_results.json`.

On the clock: no token carries one. An earlier `GENERATION_TIMESTAMP` wall-clock diagnostic was written into the variable map and made every regeneration produce different bytes; it was removed rather than excused, because a documented exception is still a false byte-for-byte claim. The only values here that are not fixed by the committed inputs are `PYTHON_VERSION` (3.14.6) and `NUMPY_VERSION` (2.4.2), which fix the scope of the reproducibility claim to a given interpreter and NumPy build.

19.8 Reproducibility — WAVEKREST: verification gates, deterministic regeneration, and artifacts

19.8.1 Determinism contract

The `ROGUE` mission is deterministic by construction:

- **Fixed seed.** The canonical network uses `seed 0`; a seeded generator draws the edge amounts, so the same seed always yields the same graph and the same report. `tests/test_laundering_graph.py` asserts that `seed 0` reproduces an identical edge set and that a different seed changes it.
- **No random draws in the detectors.** Cycle, funnel, dirty-flow, integration, entropy, forensic-physics, dynamic-programming, routing, knapsack, and surveillance-ordering routines are pure arithmetic.
- **One declaration per scenario.** The mission adapter, the manuscript token generator, and the figure generators all read the same `canonical_*_params` functions, and `tests/test_canonical_params.py` asserts the mission outcome and the manuscript analyses are equal report-for-report. A figure cannot depict inputs the prose does not describe.
- **No wall-clock in persisted outputs.** Provenance `wall_time_s` is fixed at 0.0 and no token carries a clock reading. A `GENERATION_TIMESTAMP` token was emitted into `output/data/manuscript_variables.json` until it was removed: it made two runs a second apart differ, so the byte-for-byte claim below was false while it existed, and the test guarding it injected a fixed clock and therefore could not see it. `test_no_token_carries_a_wall_clock_value` now rejects any clock-shaped token value.
- **Byte determinism of figures.** `tests/test_figures.py` renders each figure twice into independent directories and asserts byte-identical PNGs.
- **Byte determinism of the whole regeneration.** `tests/test_manuscript_variables.py::test_regeneration_is_byte_identical` runs `scripts/z_generate_manuscript_variables.py` twice, a real second apart, into two independent copies of the package and compares every emitted artifact — both JSON files, all nine rendered sections, and all four figures — byte for byte, with no clock injected anywhere.
- **End-to-end determinism.** `tests/test_mission.py` executes the mission twice and asserts identical result dictionaries and provenance hashes.

19.8.2 Provenance

Every `MissionOutcome` carries a `Provenance` with the package version (0.1.0), the `seed` (0), a `sha256 input_hash` over the canonical network (node set and edge amounts) *and* every parameter of all six canonical domain scenarios, and `wall_time_s = 0.0`. The hash makes any `ROGUE` outcome independently auditable without re-running the film: re-compute the hash over the same inputs and it must match. Its coverage is tested by mutating each domain’s inputs in turn and requiring the hash to respond — a hash that silently omitted a domain would make “audit” mean less than it says.

19.8.3 Test coverage

The source tree is covered well above the enforced 90 % line + branch floor (`uv run pytest tests/ --cov=src --cov-fail-under=90`). Coverage levels and the live test count are deliberately not quoted here — the gate is the claim, and the measured figure belongs in `TODO.md`’s dated verification record, where it can be checked against a command rather than trusted.

19.8.4 Artifacts

Regenerable output (git-ignored) lives under `output/`:

- `output/data/mission_outcome.json` — tagged `MissionOutcome` from `scripts/mission_execute.py`;
- `output/data/mission_results.json` and `output/data/manuscript_variables.json` from `scripts/z_generate_manuscript_variables.py`;
- `output/manuscript/*.md` — numbered sections with generated tokens substituted;
- `../figures/*.png` — the four `ROGUE` figures.

Re-running `scripts/z_generate_manuscript_variables.py` regenerates all of these byte-identically — asserted by `test_regeneration_is_byte_identical`, not merely asserted about. The scope of that identity is a fixed interpreter and NumPy build: 3.14.6

and 2.4.2 are the only environment-derived token values, so those two fields (and nothing else) can differ across environments. Environment: Python 3.14.6, NumPy 2.4.2, film `licence_to_kill`, codename `ROGUE`, package version 0.1.0.

19.9 Scope and Related Work — WAVEKREST: boundaries, positioning, and relationship to the literature

19.9.1 Scope

This package models the *analytical scaffolding* of the ROGUE mission, not its fiction. It does not attempt to simulate human behaviour, the cartel’s actual personnel, or the film’s events; it provides reproducible, first-principles tools a field analyst would use. All six inference objects are deliberately stylised: the laundering graph abstracts real banking structure, the vessel profile abstracts a real vessel class, the rogue model abstracts an operator’s options into a three-intensity action ladder, the price ladder abstracts a market into fixed multipliers, the campaign network abstracts world geography into 4 hubs, and the surveillance model abstracts an intelligence apparatus into a single logistic hazard. Generalisations to genuine transaction histories, real vessel data, market price series, real route networks, or richer action spaces are natural extensions but are out of scope here.

19.9.2 Related work

- **Money laundering.** Layering and integration are standard in the financial-crime literature; our cycle + funnel + dirty-flow propagation operationalises them on an explicit graph. Financial Action Task Force recommendations and typology reporting [Financial Action Task Force, 2012, updated 2020] describe the placement–layering–integration lifecycle this model encodes.
- **Narcotics finance and cash physics.** Supply-chain price/purity ladders follow the order-of-magnitude cost structure reported in US Drug Enforcement Administration price analyses [United States Drug Enforcement Administration, 1990s–2020]; the conversion of value into note mass/volume follows US Bureau of Engraving and Printing currency specifications [United States Bureau of Engraving and Printing, 2020].
- **Graph algorithms.** Depth-first cycle enumeration follows Tarjan’s depth-first-search framing [Tarjan, 1972b]; risk-adjusted routing is Dijkstra’s algorithm [Dijkstra, 1959c]; the gadget loadout is the 0/1 knapsack problem solved by dynamic programming [Kellerer et al., 2004].
- **Vessel forensics.** Gross-tonnage computation implements the International Convention on Tonnage Measurement of Ships, 1969 [International Maritime Organization, 1969]; endurance modelling uses the standard cubic propulsion-power law and diesel specific fuel consumption.
- **Sequential decision-making.** The rogue-00 model is a finite-horizon Markov decision process solved exactly by dynamic programming [Bellman, 1957a]; the surveillance ordering rule follows Smith’s ratio rule for single-stage scheduling [Smith, 1956].
- **Source material.** The mission is drawn from the 1989 film directed by John Glen [Maibaum and Wilson, 1989].

19.9.3 Limitations

Determinism is a feature, but also a boundary: results are exact *for the modelled objects*, and the objects are idealised.

- The composite laundering score’s component weights (0.4 / 0.3 / 0.3) are heuristics, not estimates from labelled data. The score is not a calibrated absolute quantity, and its *ranking* of two structures is not invariant to the weights either: over a coarse sampling of the weight simplex (21 weight triples) the ordering of two deliberately distinct reference structures matches the canonical-weight ordering on only 38% of the samples. Measured, not assumed — the number is a token from `layering_ranking_stability` in `manuscript_variables.py`, and the non-invariance is a pinned test (`test_layering_score_ranking_is_weight_dependent`). Treat the composite score as a screening heuristic for gross differences, never as a stable single-number ordering.
- `LAUNDERING_PATHS` is the exact source→sink chain count for the canonical network, not a truncated sample: the enumeration cap (1000) sits strictly above the true count and the depth component is computed over the full chain population (see `test_canonical_layering_path_count_is_exact_not_saturated`).
- The vessel physics ignore sea state, hull fouling, and auxiliary loads; endurance is a flat-water, single-speed idealisation.
- **The rogue model’s alert dimension is an integer lattice**, so the visibility ladder is truncated onto it. At the canonical parameters the stealth intensity’s visibility (0.5) truncates to *no* alert increase, which is why the optimal policy enters every one of its steps at alert 0 and the blowback hazard it faces is driven by exposure alone. Its single escalation is the closing overt action, which leaves alert at 3.0 with no remaining horizon to be exposed to it — reported as `ROGUE_PEAK_ALERT` (3.0) rather than folded into the entering-alert curve, because a peak taken over entering alert alone is identically zero for this policy and would misdescribe it. This is the most consequential simplification in the package and it is stated here rather than buried: a finer alert lattice, or an explicitly integral visibility ladder, would change the optimal policy. The quantisation is at least *consistent* — the optimiser and the reported trajectory share one transition function, and the test suite pins that replaying the optimal sequence reproduces the optimal value exactly.
- The narcotics prices are order-of-magnitude anchors [United States Drug Enforcement Administration, 1990s–2020] rather than market data, and the stage multipliers are fixed rather than estimated.

- The logistics network abstracts real geography into 4 hubs with static per-leg risks; interdiction risk in reality is neither static nor independent across legs, and the survival-product metric assumes independence.
- The surveillance model’s exact ordering is brute-force for small n ($n \leq 8$), with a ratio-rule heuristic beyond; the heuristic carries no optimality guarantee for this objective.

Each of these is documented where the corresponding parameter lives in source, and each is open to principled extension behind the same frozen `bond-api` protocol.

19.10 Sources — WAVEKREST: bibliography

Tarjan [1972b]; International Maritime Organization [1969]; Bellman [1957a]; Maibaum and Wilson [1989]; Financial Action Task Force [2012, updated 2020]; Dijkstra [1959c]; Kellerer et al. [2004]; Smith [1956]; United States Bureau of Engraving and Printing [2020]; United States Drug Enforcement Administration [1990s–2020]; Shannon [1948b]

20 GoldenEye (1995) — ARKANGEL

film package · *package codename* *ARKANGEL*. Mission **ARKANGEL**: model the orbital EMP threat envelope and the orbital platform geometry; assess electronics kill radii by vulnerability class; assess EM coupling and shielding on sensitive infrastructure; assess regional power-network resilience to the strike; evaluate Arkangel dam-facility security and breach-flood hydrograph.

20.1 Concepts — ARKANGEL: domain and operational focus

orbital EMP, infrastructure resilience

20.2 Abstract — ARKANGEL: mission summary

GoldenEye: ARKANGEL (film identity **GOLDENEYE**, mission codename **ARKANGEL**) is a deterministic special-agent mission software package that models the coupled threat surfaces from the 1995 film *GoldenEye*: an orbital electromagnetic-pulse (EMP) weapon and its orbital-platform delivery, the electronics kill-radius it imposes on surface assets, the coupling and shielding of sensitive infrastructure, the resilience of the regional power network, and the security and breach-flood risk of the Arkangel dam facility. The threat envelope is a documented conservative point-source model of a high-altitude nuclear burst (**2000.0** kt at **300.0** km), producing a nadir early-time field of **90125** V/m across a line-of-sight footprint of radius **1978** km (**12291795** km²). Four electronics vulnerability classes (**4** in total: **unprotected**, **commercial**, **military**, **legacy_tube**) are assessed: unprotected microelectronics are destroyed over essentially the entire visible footprint (**1978.0** km, **100.0%** of it), while MIL-STD-188-125-hardened and legacy vacuum-tube electronics require ever-higher fields and survive to within **268.7** km of the burst or are immune (**0.0** km). For the Arkangel dam, the report combines the published Froehlich (1995) breach-outflow regression with the Ritter (1892) instantaneous dam-break solution. Given a reservoir of **130000000** m³ behind a **75.0** m head, peak outflow reaches **31760** m³/s with a wave front moving at **54.2** m/s; the facility **8.0** km downstream faces a **33.3** m peak inundation within **2.5** min — an overall **extreme** risk. Defense-in-depth access control yields a **0.987** detection probability and a **89.3** composite security score. Beyond the envelope, the package deepens the model. The weapon rides a circular orbital platform (period **90.4** min, speed **7.7** km/s) that sees the target at a **18.8**-degree elevation over **797.7** km of slant range, with a **5.0**-minute usable pass. At the facility the envelope field of **10617** V/m couples **34218** V onto a **3.2** m effective antenna, yet a **0.008** mm-penetration aluminium enclosure provides **192.3** dB of shielding and reduces the penetrated field to **0.00** V/m. The regional power grid (**5** nodes, **6** lines) loses the nodes **town_a**, **relay** and serves only **28.4%** of critical load; the mission-critical facility retains a **0.402** survival probability over **3** redundant paths. A breach hydrograph (average width **164.7** m, duration **136.4** min) sweeps all **3** monitored stations, the farthest **20.0** km downstream arriving in **6.1** min. All results are closed-form, deterministic, and regenerable from source; figures, data, and manuscript variables are produced by thin orchestration scripts under `scripts/`.

20.3 Introduction — ARKANGEL: mission framing, the operational problem, and how to read this chapter

GoldenEye: ARKANGEL — Special-Agent Mission Software (Orbital EMP effects, electronics kill radius, EM coupling and shielding, power-network resilience, and Arkangel dam-facility security) is a BOND-suite film package for *GoldenEye* (1995), the film in which a captured space-based weapon — a **2000.0** kt nuclear device detonated at **300.0** km altitude — demonstrates the catastrophic coupling between an electromagnetic pulse and the electronics of an entire region. The package re-casts that event as a rigorous, deterministic engineering analysis: the orbital platform that carries the weapon, the field it lays down, how that field couples into and past infrastructure shielding, and what the regional power network retains afterwards. It then extends the analysis to the second mission-critical system from the same film — the Arkangel dam facility, whose defensive layers, catastrophic-failure flood risk, and downstream breach hydrograph are modelled explicitly.

20.3.1 Reader's guide

The package is organised around a clean separation of concerns:

- **Pure domain core** — seven infrastructure-free, closed-form modules: `emp_model.py` (the burst-to-ground E1 envelope and the E1/E2/E3 waveform), `orbital_mechanics.py` (the Keplerian delivery platform and its pass geometry), `kill_radius.py` (per-vulnerability-class kill radii), `coupling.py` (skin depth, Schelkunoff shielding, conductor coupling), `grid_resilience.py` (per-asset survival, most-reliable routing, edge-disjoint redundancy), `dam_security.py` (defense-in-depth access control and breach-flood risk), and `breach_hydrograph.py` (breach width, the volume-conserving outflow hydrograph, and the downstream station table). All seven can be imported in any Python environment with no BOND layer present.
- **Scenario composition** — `src/goldeneye/scenario.py` binds the seven models into the canonical ARKANGEL scenario and exposes the deterministic analysis dictionary and a canonical input hash.
- **Protocol adapter** — `src/goldeneye/mission.py` implements the frozen `bond_api.MissionProvider` contract (brief/recon/plan/execute/debrief) and registers the mission's analytical gadgets with the `bond_api.GadgetRegistry`.
- **Thin orchestrators** — `scripts/` expose the five mission phases plus a manuscript-variable hydrator. Business logic never lives in a script.

This design keeps the mathematical core testable in isolation (a per-module suite with no mocks, $\geq 90\%$ line-and-branch coverage) while leaving the BOND protocol integration as a thin, inspectable adapter — the pattern used by every film package in the suite.

20.3.2 What this manuscript covers

02_methodology.md states the physics and the closed-form models. 03_results.md reports the ARKANGEL run. 04_conclusion.md summarises the deliverable. 05_experimental_setup.md and 06_reproducibility.md document the scenario parameters, environment, and regeneration contract. 07_scope_and_related_work.md delimits the model's assumptions and situates it in the HEMP-damage and dam-break literature.

20.4 Methodology — ARKANGEL: the analytical models and algorithms that drive the mission

20.4.1 Orbital EMP weapon model (src/goldeneye/emp_model.py)

A high-altitude nuclear burst (HEMP) releases a prompt gamma flux; the gammas Compton-scatter in the atmosphere, driving a radial electron current that radiates the early-time (E1) electromagnetic pulse. We model the ground-level E1 peak field as a documented conservative point-source envelope:

$$E(r) = E_0 Y_s \left(\frac{h}{d} \right)^2 \exp \left(-\frac{d-h}{\lambda(h)} \right),$$

where h is the burst altitude, $d = \sqrt{h^2 + r^2}$ the slant range to a ground point at horizontal range r , $Y_s = (Y/Y_{\text{ref}})^p$ the empirical yield scaling, and $\lambda(h)$ the effective gamma attenuation length, which grows exponentially with altitude (thinner air). Reflecting the finiteness of the deposition region, the amplitude anchor is $E_0 = 50000$ V/m at the **1000** kt reference yield, matching the canonical HEMP figure, and the empirical yield exponent is $p = 0.85$. These two values, with $\lambda(h)$, are the whole parameterisation: the envelope has no Compton source term, so the prompt-gamma fraction (**0.3%**, Glasstone and Dolan, 1977) is carried as documented provenance for the amplitude anchor and is *not* an input to the equation above — a limitation of this envelope, not a calibration. Beyond line of sight the field is taken as zero, which bounds the illuminated footprint to the geometric horizon $\sqrt{2R_\oplus h + h^2}$.

The package also implements the canonical waveform as a normalized double-exponential, $E(t) = A k (e^{-\alpha t} - e^{-\beta t})$, with the early-time (E1), intermediate-time (E2), and late-time magnetohydrodynamic (E3) components distinguished by documented rate constants (IEC 61000-2-9 for E1) and amplitude policies.

20.4.2 Electronics kill-radius analysis (src/goldeneye/kill_radius.py)

4 vulnerability classes are defined by increasing damage thresholds: *unprotected* microelectronics (**1000** V/m), *commercial/semi-hardened* (**10000** V/m), *MIL-STD-188-125-hardened* facilities (**50000** V/m), and *legacy vacuum-tube/relay* electronics (**1000000** V/m). Because the E1 envelope is monotone decreasing in range, the kill radius — the range at which the field meets the class threshold — is found by deterministic bisection. The enclosed kill area is πr^2 , and the fraction of the illuminated footprint covered is $(r/r_{\text{horizon}})^2$. A smooth damage-probability logistic, $P = [1 + \exp(-s(E/T - 1))]^{-1}$, gives $P = 0.5$ exactly at the damage threshold.

20.4.3 Arkangel dam security (src/goldeneye/dam_security.py)

Access control (defense in depth). The facility is modelled as a cascade of n layers, each with a per-pass detection probability p_i and response time t_i . The probability of at least one detection is $1 - \prod_i (1 - p_i)$; the expected response time to the first detecting layer weights each $p_i \prod_{j < i} (1 - p_j)$ by t_i . A composite security score blends detection coverage with responsiveness in $[0, 100]$.

Breach flood risk. A dam breach releases a flood wave. Peak outflow follows the published Froehlich (1995) embankment regression (Froehlich, 1995):

$$Q_p = 0.607 V^{0.295} H^{1.24},$$

with reservoir volume V and breach head H in SI units. Downstream propagation uses the Ritter (1892) instantaneous dam-break solution on a dry, frictionless bed (Ritter, 1892): with $c_0 = \sqrt{gH}$, the depth and velocity in the wave region ($-c_0 \leq x/t \leq 2c_0$) are

$$h(x, t) = \frac{1}{9g} \left(2c_0 - \frac{x}{t} \right)^2, \quad u(x, t) = \frac{2}{3} \left(c_0 + \frac{x}{t} \right),$$

The front arrives at a downstream distance x at time $x/(2c_0)$, and the asymptotic peak depth at any fixed distance is $0.4444 H$. (The rational factors inside the Ritter solution — $1/9g$, $2/3$ — are exact constants of that closed-form solution and cannot drift; the asymptotic depth coefficient is a named module constant and so is tokenized.) The facility inundation severity is the mean depth over its ground elevation, binned into a low/moderate/high/extreme risk rating.

20.4.4 Orbital platform geometry (`src/goldeneye/orbital_mechanics.py`)

The weapon is delivered from a circular orbital platform. Keplerian kinematics give the period $T = 2\pi\sqrt{a^3/\mu}$ and speed $v = \sqrt{\mu/a}$ for an orbit of radius a about the Earth of gravitational parameter $\mu = 3.986004 \times 10^{14} \text{ m}^3/\text{s}^2$ (Bate, Mueller & White, 1971). Spherical Earth geometry yields the central angle, elevation angle, and slant range between the ground target and the platform; a deterministic bisection finds the maximum central angle at which a chosen minimum elevation is maintained, whose arc fraction of the orbit bounds the usable pass duration.

20.4.5 Electromagnetic coupling and shielding (`src/goldeneye/coupling.py`)

To reach electronics the field must couple onto conductors and penetrate enclosures. Skin depth follows the classical $\delta = \sqrt{\rho/(\pi f \mu_0 \mu_r)}$. Schelkunoff (1943) shielding effectiveness combines plane-wave absorption $A = 8.686 t/\delta$ dB with reflection

$$R = 20 \log_{10} \frac{\eta_0}{4|Z_s|} \text{ dB}, \quad |Z_s| = \sqrt{2\pi f \mu_0 \mu_r \rho},$$

where $\eta_0 = \mathbf{376.7} \, \Omega$ is the free-space wave impedance and Z_s the conductor surface impedance (Schelkunoff, 1943; Ott, *Electromagnetic Compatibility Engineering*, 2009; the multiple-reflection correction is neglected for thick shields). We use this impedance-ratio form rather than the tabulated shortcut $R = 168 - 10 \log_{10}(\mu_r f / \sigma_r)$ because the latter is only defined for σ_r relative to annealed copper; substituting an absolute conductivity in S/m inflates the reflection loss by ≈ 78 dB. The two forms agree to within 0.2 dB and both are exercised in `tests/test_coupling.py`. An exposed conductor of length L at field-elevation ε has effective height $h_{\text{eff}} = L \sin \varepsilon$, coupling an induced voltage $V = E h_{\text{eff}}$, and a protection margin $20 \log_{10}(\text{withstand}/E)$ dB quantifies residual hardness.

20.4.6 Regional power-network resilience (`src/goldeneye/grid_resilience.py`)

The regional network is a graph of nodes (plants, substations, loads) and transmission lines on the local plane, the burst nadir at the origin. Every asset carries a vulnerability class; its survival probability is $1 - P_{\text{damage}}(E)$ at the asset's envelope field. The most-reliable source-to-load route maximises the product of survival probabilities (equivalent to shortest paths under $-\log(\text{survival})$ weights, deterministic Dijkstra (1959) with insertion-order tie-breaking); redundancy is the number of edge-disjoint routes via Edmonds-Karp (1972) max flow with unit capacities, where parallel circuits between one node pair each contribute their own unit of capacity. The report's expected served load assumes independence of the surviving assets along each best path (a documented approximation).

20.4.7 Breach outflow hydrograph (`src/goldeneye/breach_hydrograph.py`)

The breach's time evolution extends the dam-break analysis. The average breach width follows the Froehlich (2008) regression

$$B_{\text{avg}} = 0.27 K_0 V^{0.32} H^{0.04},$$

with $K_0 = \mathbf{1.3}$ for overtopping failure and $\mathbf{1.0}$ otherwise (Froehlich, 2008). These are the coefficients of the 2008 regression as tabulated in the USACE HEC-RAS technical reference; Froehlich's earlier 1995 width equation uses a different coefficient set and is not mixed in here. The outflow is a volume-conserving triangular hydrograph falling linearly from the Froehlich (1995) peak discharge to zero over $t_d = 2V/Q_p$, so the discharged area equals the stored volume; its mean outflow is V/t_d . A downstream station table combines the Ritter front-arrival time, the asymptotic peak depth $0.4444 H$, and the flood-risk rating at each named station.

20.5 Results — ARKANGEL: measured outcomes, headline numbers, and what they establish

Run via the ARKANGEL mission software against the canonical scenario (`scripts/execute.py`), all figures and values below are the deterministic output of the pure domain core.

20.5.1 EMP threat envelope

For the canonical weapon (**2000.0** kt at **300.0** km), the E1 peak field directly under the burst is **90125** V/m, carrying a peak electromagnetic energy density ($0.5 \cdot \varepsilon_0 \cdot E^2$) of **0.036** J/m³ at the moment of peak. The illuminated footprint is bounded by the geometric horizon at **1978** km radius, an area of **12291795** km². Because the prompt-flux absorption is negligible along the vacuum path at this altitude, the field decays with the squared geometric projection $\sin^2 \psi$ across the footprint.

20.5.2 Kill radius by vulnerability class

Across the **4** vulnerability classes (**unprotected**, **commercial**, **military**, **legacy_tube**), kill radii span the full spectrum of the footprint:

Class	Kill radius (km)
Unprotected microelectronics	1978.0
Commercial / semi-hardened	849.2

Class	Kill radius (km)
MIL-STD-188-125 hardened	268.7
Legacy vacuum-tube / relay	0.0

Unprotected microelectronics are destroyed over essentially the entire visible footprint (**100.0%** of it); hardened facilities retain a modest survival zone; legacy vacuum-tube electronics, with their **0.0** km radius, are immune at every realistic range in this scenario — a direct consequence of their orders-of-magnitude higher threshold. The figure above shows how these radii trade off against burst altitude and yield, and the damage-probability transition for each class.

20.5.3 Arkangel dam breach flood

Given a reservoir of **130000000** m³ behind a **75.0** m head, the Froehlich (1995) regression gives a peak breach outflow of **31760** m³/s. The Ritter wave front advances at **54.2** m/s, reaching the facility **8.0** km downstream in **2.5** min with an asymptotic peak inundation of **33.3** m — a severity ratio of **3.333** against the facility elevation, rated **extreme**.

20.5.4 Downstream inundation stations

The breach hydrograph reports the front-arrival time, the asymptotic peak depth, and the flood-risk rating at every monitored station downstream of the dam:

Station	Distance (km)	Arrival (min)	Peak depth (m)	Risk
facility	8.0	2.5	33.3	extreme
town	12.0	3.7	33.3	extreme
bridge	20.0	6.1	33.3	extreme

The frictionless Ritter asymptote gives every station the same peak depth **33.3** m (**0.4444H**, independent of distance), so the differentiating quantity is the front-arrival time: the facility, nearest the dam, is the first to be inundated by the **extreme**-rated wave.

20.5.5 Access-control posture

The **4** layered defense-in-depth cascade yields a **0.987** probability of detecting an intruder at least once, an expected response time of **6.3** min, and a composite security score of **89.3** (out of 100).

20.5.6 Orbital platform

The weapon rides a circular orbit of radius **6671** km (**300** km altitude) with a period of **90.4** min and speed **7.7** km/s. From the target, the platform stands at a **18.8**-degree elevation over a **6.5**-degree central angle (**797.7** km slant range) and remains above a **5.0**-minute usable pass — the detonation window.

20.5.7 Coupling and shielding

At the facility the envelope field reaches **10617** V/m. A **3.2** m effective antenna couples **34218** V ($\approx$ **684** A into a **50** Ohm load) — a kilovolt-class conducted threat. In contrast, the skin depth at the representative **100** MHz E1 frequency is only **0.008** mm in the **aluminum** shield; a **0.10** mm enclosure provides **192.3** dB of shielding, reducing the penetrated field to **0.00** V/m and leaving a **13.5** dB protection margin against the military withstand level.

20.5.8 Regional power-network resilience

The **5**-node, **6**-line regional network loses the nodes **town_a**, **relay** to certain failure. It serves an expected **4.8** MW of the **17.0** MW critical load (**28.4%**), with a worst-case load reliability of **0.000** and a minimum of **2** redundant routes per critical load. The mission-critical facility keeps a **0.402** survival probability over **3** edge-disjoint paths — the hardened hydro source and a redundant interconnection are what keep it on the board.

20.5.9 Breach outflow hydrograph

A breach **164.7** m wide releases the reservoir as a volume-conserving triangular hydrograph lasting **136.4** min at a mean outflow of **15880** m³/s. The flood sweeps all **3** monitored stations; the farthest, **20.0** km downstream, is reached in **6.1** min.

Orbital EMP threat envelope — GoldenEye: ARKANGEL

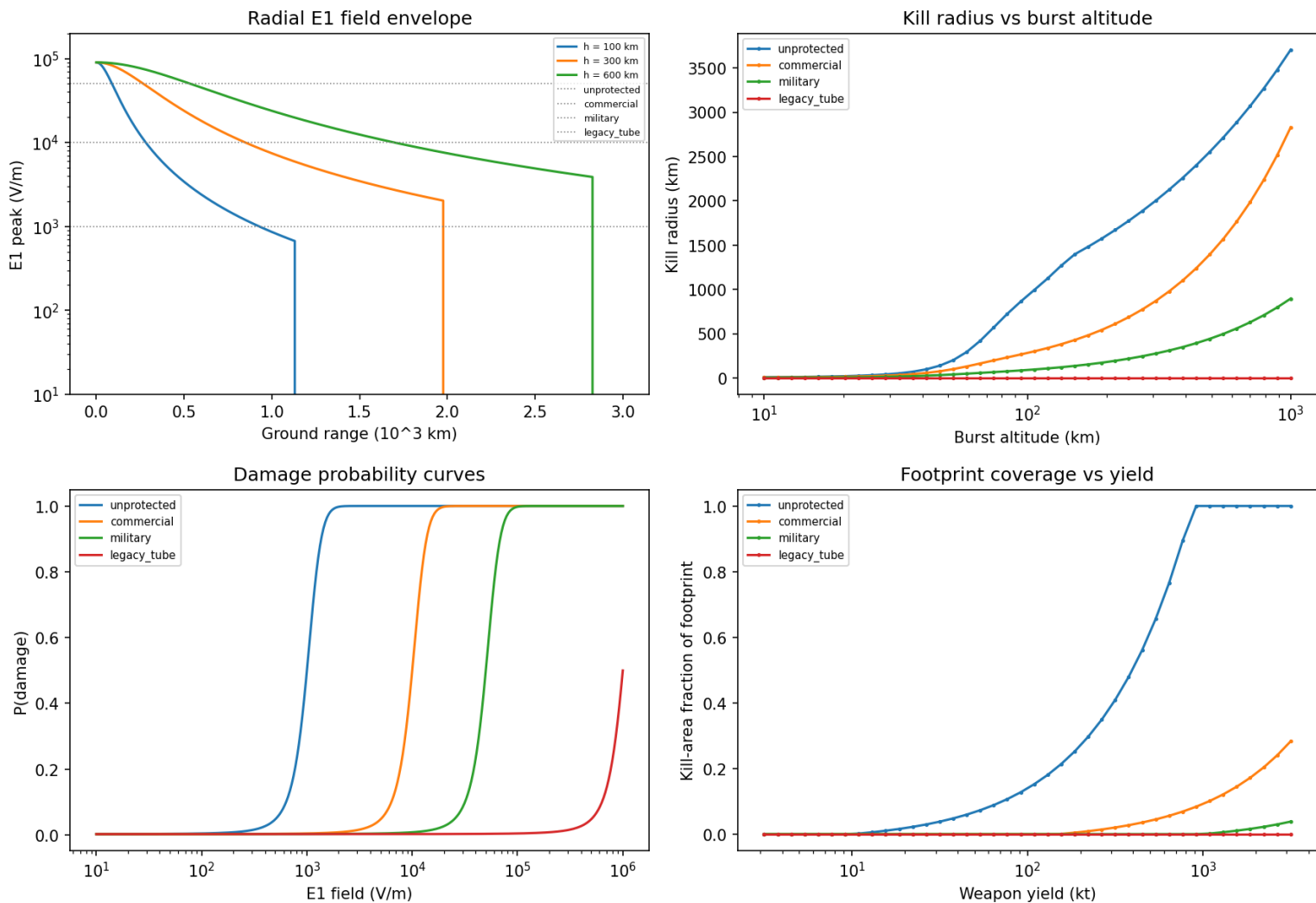


Figure 75: EMP threat envelope and kill radii

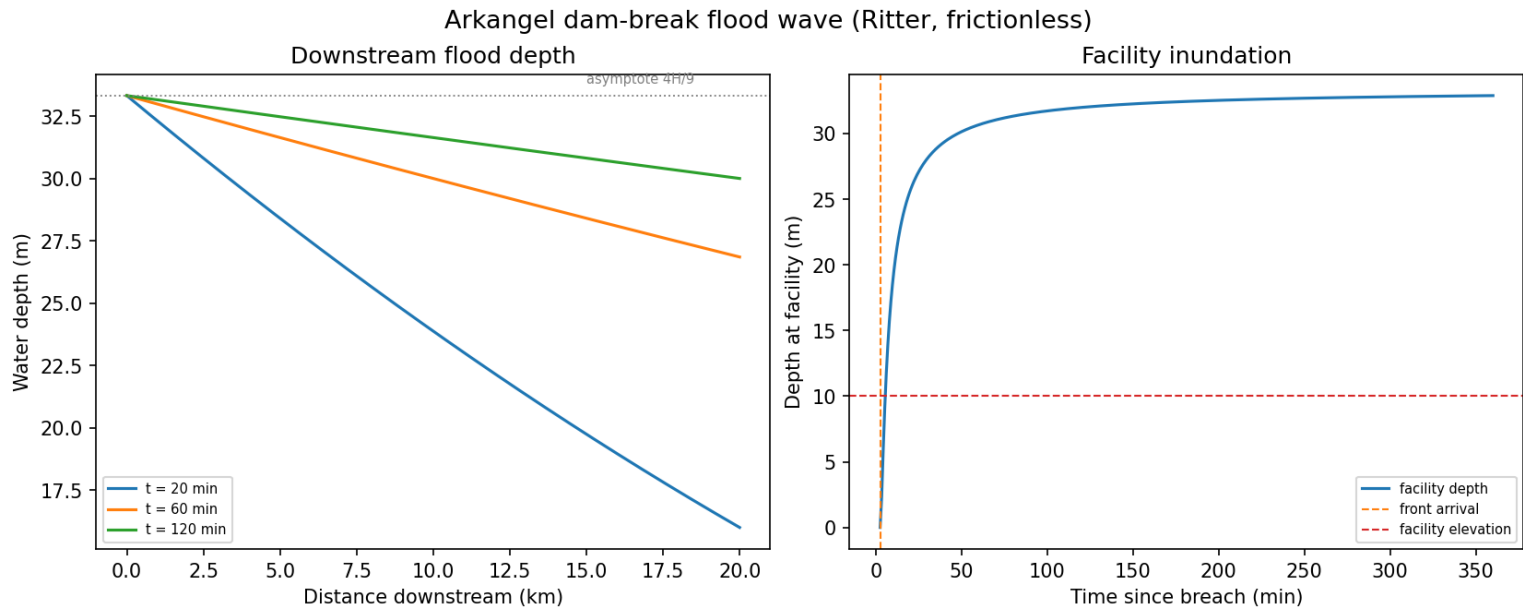


Figure 76: Ritter dam-break flood wave

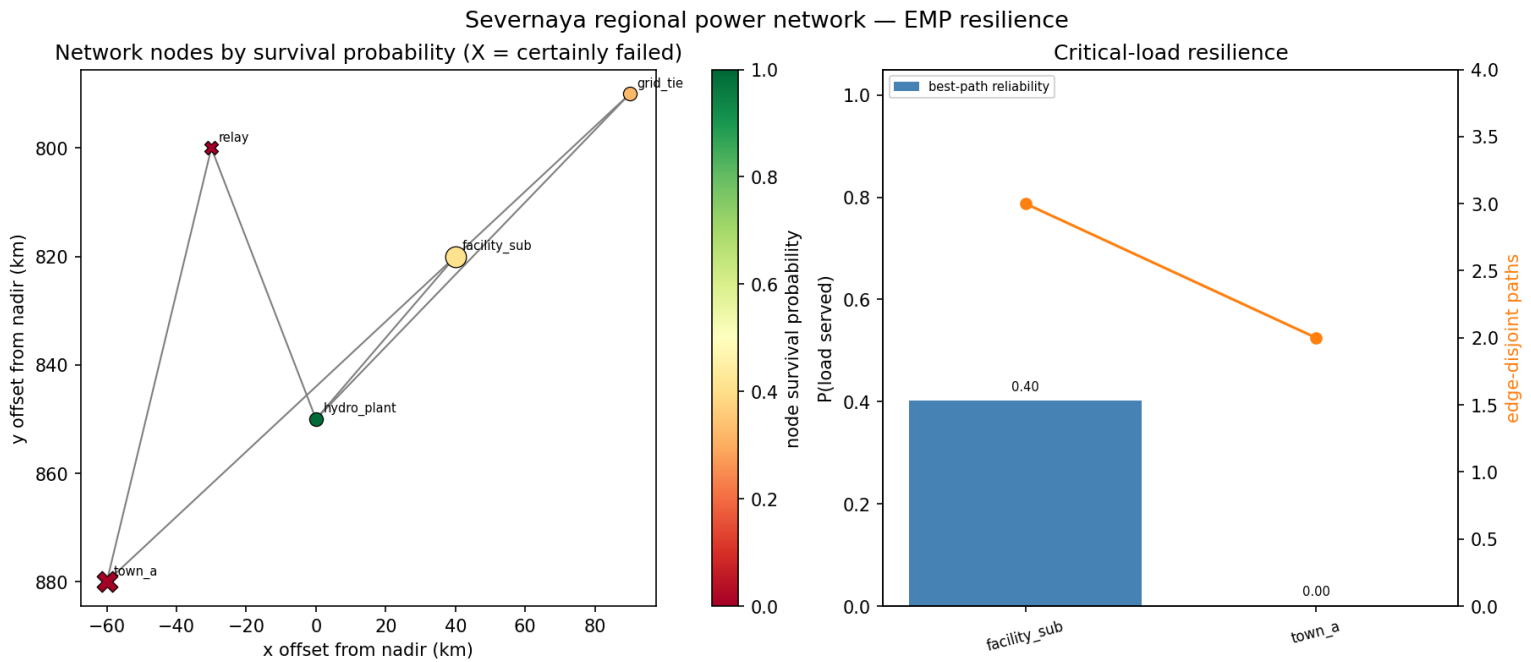


Figure 77: Regional grid resilience

20.6 Conclusion — ARKANGEL: findings, verdict, and what the mission establishes

GoldenEye: ARKANGEL — **Special-Agent Mission Software** delivers a complete, deterministic, and reproducible analysis of the coupled threat surfaces of *GoldenEye*’s ARKANGEL mission:

1. **An orbital EMP weapon and platform** whose ground-level effects are captured by a documented conservative point-source envelope (the canonical E1/E2/E3 pulse), delivered by a Keplerian orbital platform with determinable pass geometry (**90.4** min period, **5.0** min usable pass).
2. **An electronics kill-radius analysis** across **4** defensible vulnerability classes, from unprotected microelectronics (destroyed over the whole visible footprint, **1978.0** km) to legacy vacuum-tube relays (immune, **0.0** km).
3. **Coupling and shielding of infrastructure** — **34218** V induced onto exposed conductors yet a **192.3** dB shield reducing the penetrated field to **0.00** V/m — and the resilience of the regional power network (**28.4%** of critical load served; **3** redundant paths to the facility).
4. **An Arkangel dam-facility model** combining a layered defense-in-depth access-control score with a physically grounded breach-flood risk and outflow hydrograph (peak outflow **31760** m³/s; breach width **164.7** m; facility inundation rated **extreme**).

Three properties make the package a faithful exemplar of the BOND suite’s guardrails. The analysis is **closed-form and deterministic** — no RNG, no wall-clock dependence, and a canonical SHA-256 input hash for audit. The **pure domain core** is infrastructure-free and importable without any BOND layer, while the protocol integration is confined to a thin, inspectable adapter (`src/goldeneye/mission.py`) whose public shape is stable across this deepening. And the whole pipeline is **regenerable**: thin orchestration scripts drive the mission and produce figures, data, and manuscript variables from source, so the manuscript cannot drift from the code.

The result is special-agent mission software that treats an on-screen weapon and facility as a serious, well-tested scientific problem — rigorous enough to stand beside the analytical tools it mimics, and honest about the simplifying assumptions documented in `07_scope_and_related_work.md`.

20.7 Experimental Setup — ARKANGEL: canonical scenarios, parameters, and configuration

20.7.1 Scenario parameters

All parameters are declared in `manuscript/config.yaml` and loaded by `src/goldeneye/scenario.py::load_scenario_config`. The canonical ARKANGEL mission uses:

Parameter	Value
Weapon yield	2000.0 kt
Burst altitude	300.0 km
Reservoir volume	130000000 m ³
Dam head	75.0 m

Parameter	Value
Facility downstream distance	8.0 km
Access-control layers	4

The orbital-platform geometry (burst nadir at **60.0°** N, **108.0°** E; target at **53.5°** N; platform altitude **300** km; minimum usable elevation **10°**), the coupling inputs (**10** m conductor, **0.10** mm **aluminum** enclosure, **100** MHz reference frequency, **50** Ohm load), the canonical **5**-node regional grid, and the **3** downstream stations are declared in `manuscript/config.yaml` (the orbit and coupling blocks) and `src/goldeneye/scenario.py` (the grid and stations).

The EMP envelope constants are fixed in `src/goldeneye/emp_model.py`: an amplitude anchor of **50000** V/m at the **1000** kt reference yield and an empirical yield exponent of **0.85**. The same module also records a **0.3%** prompt-gamma fraction, but no function reads it — the envelope has no Compton source term, so that value is provenance for the amplitude anchor rather than a parameter of the calculation (`tests/test_emp_model.py::TestGammaFractionIsReportedNotConsumed` holds this to be true). (The **90125** V/m nadir field reported in the results is *computed* from those constants at the scenario yield, not itself a constant.) The **4** vulnerability-class thresholds are fixed in `src/goldeneye/kill_radius.py`, and the **50000** V/m military withstand level used for the protection margin is declared in `src/goldeneye/scenario.py`.

20.7.2 Software environment

The package is tested under Python **3.14.6** with NumPy **2.4.2** and matplotlib; bond-api and bond-utilities are installed as local path dependencies. This environment snapshot belongs to manuscript revision **2026-08-04**, the date declared in `manuscript/config.yaml`. No token reads the wall clock at generation time, so regenerating the manuscript in this environment rewrites the same bytes (see §Reproducibility).

20.7.3 Verification gates

The enforced gates are `uv run pytest tests/ --cov=src --cov-fail-under=90` (line and branch), `uv run ruff check` and `uv run ruff format`, `uv run mypy src/ scripts/`, a zero-mock audit of `tests/`, and a live cross-reference test that every manuscript token (the double-brace variables in the numbered sections) is produced by `generate_variables`.

20.8 Reproducibility — ARKANGEL: verification gates, deterministic regeneration, and artifacts

This is version **0.1.0** of **GoldenEye: ARKANGEL — Special-Agent Mission Software**. Every numeric claim in this manuscript — result, model constant, and scenario input alike — is produced from source by a regenerable pipeline; none is hand-authored.

20.8.1 Regeneration contract

1. `uv run python scripts/execute.py` runs the full mission and persists `output/data/arkangel_outcome.json` (bond-api wire format) and a provenance-outcome record `output/data/arkangel_mission.json` written via `bond_utilities.mission_io.record_provenance_outcome`.
2. `scripts/z_generate_manuscript_variables.py` writes `output/data/manuscript_variables.json`, renders the three concept figures to `../figures/`, and resolves the numbered sections into `output/manuscript/`.
3. `uv run pytest tests/ --cov=src --cov-fail-under=90` enforces the coverage gate; a live cross-reference test guarantees the token map and the manuscript cannot drift apart.

20.8.2 Determinism

The analysis is fully deterministic: fixed scenario parameters, closed-form physics, and a fixed seed of 0 (no random draws anywhere). The canonical SHA-256 `input_hash` of the scenario (16 hex characters appear in the execution report) lets any outcome be audited against the exact inputs that produced it.

Byte-identity claim and its scope. Re-running `scripts/execute.py` and `scripts/z_generate_manuscript_variables.py` in one environment rewrites the following artifacts byte for byte:

- `output/data/arkangel_outcome.json`
- `output/data/manuscript_variables.json`
- `output/manuscript/*.md` (the resolved sections)
- `../figures/*.png`

No token in that set reads the wall clock; the dated token in `05_experimental_setup.md` is the committed `paper.date` from `manuscript/config.yaml`, not a generation time. `tests/test_reproducibility.py` regenerates the whole set twice in one test run and compares the bytes, so the claim fails loudly if a wall-clock value is ever reintroduced.

Two exclusions, stated rather than hidden:

1. `output/data/arkangel_mission.json` is a **run-scoped provenance record** written through `bond_utilities.mission_info`. It deliberately carries a real `started_at` wall clock and therefore differs between runs. It is not covered by the byte-identity claim.
2. Byte-identity is claimed *within one environment*. 3.14.6 and 2.4.2 are environment provenance, so a different interpreter or NumPy build legitimately changes those two tokens and the sections that render them.

20.8.3 Output inventory

Regenerable artifacts under `output/` (git-ignored):

- `data/arkangel_outcome.json`, `data/arkangel_mission.json`, `data/manuscript_variables.json`
- `figures/emp_footprint.png`, `figures/flood_wave.png`, `figures/grid_resilience.png`
- `manuscript/` (numbered sections with tokens resolved)

Live test count and achieved coverage are tracked in `docs/_generated/COUNTS.md`, not hardcoded here.

20.9 Scope and Related Work — ARKANGEL: boundaries, positioning, and relationship to the literature

20.9.1 Scope

This package models the coupled threat surfaces of *GoldenEye* (1995) — the orbital weapon and its platform, electronics vulnerability, EM coupling and shielding, regional power-network resilience, and the Arkangel dam facility’s access control and breach flood — with closed-form, deterministic approximations across seven pure-core modules. It is engineering mission software, not a predictive defence-physics code, and its honesty about assumptions is part of the deliverable.

EMP envelope. The ground field is a *conservative point-source envelope*. It omits the full distributed-source HEMP field computation, polarization and coupling detail, and site-specific shielding/cable-coupling response. Because it treats the source as a point, the kill footprint it yields is smaller than a detailed code would predict — deliberately the conservative direction for a damage assessment. Waveform components E1/E2/E3 use documented representative rate constants; E2 is not standardized (IEC defines E1 most strictly) and its amplitude fraction is an engineering default.

Kill radius. Vulnerability-class thresholds are engineering references from the EMP-hardening literature (e.g. MIL-STD-188-125 for the 50000 V/m E1 hardness level), not guarantees for specific equipment under specific coupling conditions. Damage probability is a smooth surrogate, not a validated reliability model.

Dam breach. The Ritter solution assumes an instantaneous, complete breach on a dry, frictionless, prismatic bed. Real dam failures evolve over minutes with friction and valley geometry, so arrival times here are lower bounds and depths are idealizations. The Froehlich regression is a statistical peak- outflow relation and carries scatter. The breach hydrograph is a volume-conserving triangular idealization of a real, irregular outflow curve; the downstream station table uses the frictionless asymptotic peak depth $0.4444 \cdot H$ rather than a full flood-routing model.

Orbital platform. The orbital analysis assumes a circular, two-body orbit (Bate, Mueller & White) and a directly-overhead pass for the pass-duration approximation; it ignores J2 perturbations, drag, and a real, non-zenith pass profile, so durations are upper-bound estimates of a usable detonation window.

Coupling and shielding. Coupling uses a simple effective-height model of an exposed conductor (Ott; Vance) and the plane-wave Schelkunoff approximation for shielding; it omits the multiple-reflection correction, aperture/cable penetration leakage, and non-plane-wave (diffuse) coupling. Induced voltages are open-circuit estimates, not load- and joint-dependent.

Grid resilience. Survival probabilities assume independent asset failures from the local envelope field; the expected served load is the sum of each critical load’s best-path reliability (an independence approximation, not a full stochastic cascade). Parallel circuits between the same node pair are counted individually for redundancy and collapsed to the most survivable circuit for path reliability. The canonical 5-node network is a scenario dataset, not the real Severnaya grid.

20.9.2 Related work

The models sit on well-established foundations. High-altitude EMP is treated in the canonical reference *The Effects of Nuclear Weapons* (Glasstone and Dolan, 1977), and the early-time waveform shape follows IEC 61000-2-9. Orbital mechanics follow Bate, Mueller & White’s *Fundamentals of Astrodynamics* (1971). Coupling and shielding follow Ott’s *Electromagnetic Compatibility Engineering* (2009) and Vance’s *Coupling to Shielded Cables* (1978). Peak dam-breach outflow is regressed by Froehlich (1995) and breach geometry by Froehlich (2008); the classical instantaneous dam-break solution originates with Ritter (1892) and is presented textually in Stoker’s *Water Waves* (1957). The shielding-effectiveness formulation is Schelkunoff’s (1943). The two graph algorithms are used in their original forms: shortest paths after Dijkstra (1959), applied here to $-\log$ survival weights, and edge-disjoint path counting by max flow after Edmonds and Karp (1972). Network-resilience reasoning additionally draws on the cascade-failure framing of Buldyrev et al. (2010). Defense-in-depth detection probability is the standard layered-reliability formula. The BOND protocol itself is specified in the frozen `bond-api` package; this package is one of many films implementing that contract.

20.10 Sources — ARKANGEL: bibliography

Glasstone and Dolan [1977]; iec [1996]; mil [1998]; Froehlich [1995]; Ritter [1892]; Stoker [1957]; Bate et al. [1971b]; Ott [2009]; Vance [1978]; Froehlich [2008]; Buldyrev et al. [2010]; Dijkstra [1959a]; Edmonds and Karp [1972a]; Schelkunoff [1943]

21 Tomorrow Never Dies (1997) — CARVER

film package · package codename CARVER. Mission **CARVER**: model Carver-style disinformation propagation on a social network; measure narrative capture and echo-chamber fragmentation under a bounded-confidence broadcast; quantify low-RCS stealth-vessel detection limits; recover detection range against a stealth hull via pulse compression; analyze GPS-denial impact and navigation failure cascades; bound spoofed-measurement error with RAIM integrity monitoring.

21.1 Concepts — CARVER: domain and operational focus

media manipulation, stealth

21.2 Abstract — CARVER: mission summary

Tomorrow Never Dies: CARVER — Special-Agent Mission Software (*Six Paired Models of Media, Radar, and Navigation Warfare: Disinformation and Narrative Control, Stealth Detection and Pulse Compression, GPS Denial and RAIM Integrity*). CARVER is a deterministic mission-software package that turns the media-warfare playbook of Elliot Carver into 6 quantitative, reproducible analyses, paired as attack and counter across media, radar, and navigation. A disinformation model simulates how a broadcast campaign converts a social network to a manufactured belief — 57.5% by campaign end, 11.5x the no-media baseline — and a bounded-confidence narrative model shows how a *selective* broadcast pulls the audience mean 54.2% toward the manufactured line while fragmenting it into echo chambers (bimodal split 0.402; the tipping point is a reach of just 19.6%). A radar model finds that a 20.0 dB low-RCS reduction drops a stealth hull’s detection range to 40.7 km (25.7 km for a Rayleigh fluctuating hull), but pulse compression (27.0 dB, 15.0 m resolution, -13.5 dB range sidelobes) restores it to 108.2 km — 373% beyond an unmodulated pulse of the same range resolution; the Barker-13 code reaches that same 108.2 km at the cost of 577 m range resolution in exchange for a -22.2 dB sidelobe floor. A GPS-denial model drives effective C/N0 from 45 to 10.5 dB-Hz at 100 km — below the 25 dB-Hz carrier-lock threshold, so the modelled outcome is loss of lock rather than the 25.3 m the range-error formula extrapolates past its validity (clean-sky: 8.7 m) — and fails 0.429 of the maritime dependency network; its RAIM guardian bounds the damage (HPL 19.9 m) but catches a 20 m corrupted navigation measurement with only 25.2% probability on the worst-placed satellite (54.2% on the best-placed) — the parity projection sees only part of any single-satellite fault. Keywords: disinformation propagation, media manipulation, bounded-confidence opinion dynamics, echo chambers, radar cross-section, stealth detection, pulse compression, matched filtering, GPS denial, navigation failure cascade, RAIM, GNSS integrity monitoring, deterministic simulation, reproducible research.

21.3 Introduction — CARVER: mission framing, the operational problem, and how to read this chapter

In *Tomorrow Never Dies* (1997), Elliot Carver builds a global media empire whose reach lets him manufacture the conflicts he then reports. His two instrumental capabilities are a stealth cruiser that evades radar and the ability to deny an adversary their navigation. This package — codename CARVER — models those capabilities as deterministic physics and computation, forming a single reproducible “mission software” artifact.

Each capability is modelled twice: once as the attack, once as the defence that answers it. The result is 6 infrastructure-free domain modules in three adversarial pairs.

Media.

- **Disinformation propagation** (`media_manipulation.py`): a two-channel belief-diffusion model on a social network, with media-campaign injection and greedy influence maximization — *whether* an agent believes.
- **Narrative control** (`narrative_control.py`): Hegselmann-Krause bounded-confidence opinion dynamics with a media-injection term — *how far* an opinion moves, how the audience fragments into echo chambers, and the minimum broadcast reach that tips the network.

Radar.

- **Stealth-vessel detection** (`stealth_detection.py`): the radar equation, Neyman-Pearson detection, Marcum-Q and Swerling detection probabilities, and the detection-range penalty of a low-RCS hull.
- **Pulse compression** (`radar_waveform.py`): the radar’s counter — LFM and Barker-13 matched filtering, processing gain, range resolution, sidelobe floor, and the fourth-root detection-range recovery.

Navigation.

- **GPS-denial impact** (`gps_denial.py`): satellite geometry and dilution of precision, jamming degradation of carrier-to-noise, position-error growth, and navigation failure cascades.
- **GNSS integrity** (`gnss_integrity.py`): the receiver’s counter — RAIM parity-space fault detection, protection levels, and the probability of catching a corrupted measurement before it becomes a wrong position.

The same core is wired to the BOND-API mission protocol (codename THE PROTOCOL): a `MissionProvider` named CARVER lets any coordinator discover the package by slug and drive the full brief → recon → plan → execute → debrief lifecycle, with every

outcome carrying deterministic provenance. All numbers in this manuscript are injected from `manuscript/config.yaml` and the real domain modules — none are hand-authored in the prose.

21.4 Methodology — CARVER: the analytical models and algorithms that drive the mission

Each concept is a self-contained deterministic model with clear physical or algorithmic grounding.

21.4.1 Disinformation propagation

Agents occupy one of two belief states — susceptible or believing — on a Watts–Strogatz small-world network of 120 nodes (seed 7). The broadcast footprint — the 30% of nodes the media channel reaches — is selected by a deterministic index mix rather than an RNG draw, so it is a pure function of network size, reach, and seed, and widening the reach only ever *adds* nodes to the exposed set. `simulate_disinformation(src/tomorrow_never_dies/media_manipulation.py)` converts susceptible agents through a media channel (a deterministic exposed subset converts with a persuasion probability) and a social channel (believing agents convert susceptible neighbors). Belief share, the echo-chamber coefficient, and the amplification factor are recorded; `influence_maximization` applies a greedy Kempe–Kleinberg–Tardos seed selection.

21.4.2 Narrative control (bounded confidence)

`narrative_control.py` models *how strongly* opinions move, using the Hegselmann–Krause bounded-confidence model with a Carver-style media injection term. Each round agent `i` moves to the mean opinion of the agents within confidence radius `epsilon`, and exposed agents are additionally pulled toward the manufactured line `m` with a media weight. `simulate_opinions` returns the final consensus, the opinion variance (polarization), the bimodal split (fraction of agent pairs separated by more than `2 * epsilon` — a measure of echo-chamber fragmentation), and the media capture (the normalized shift of the mean toward the manufactured line versus a no-media baseline). `critical_media_reach` bisects the broadcast reach that just tips the discrete belief model to a target share: the *minimum* audience Carver must reach.

21.4.3 Stealth-vessel detection

`stealth_detection.py` evaluates the monostatic radar equation `snr_at_range`:

$$\text{SNR} = \frac{P_t G^2 \lambda^2 \sigma}{(4\pi)^3 R^4 k T_0 B F L}, \quad (1)$$

yielding the per-pulse SNR (coherent integration is applied by `detection_probability`). A false-alarm probability sets a square-law threshold; `detection_probability` returns P_d by numerically integrating the Marcum Q function (Swerling 0) or the closed form (Swerling 1). `detection_range` binary-searches the range at which P_d reaches a target.

21.4.4 Pulse compression (waveform design)

`radar_waveform.py` analyzes the radar’s *counter* to stealth: a modulated pulse concentrates its energy through pulse compression. Three waveforms are implemented — unmodulated (gain 1), the LFM chirp (time-bandwidth product 10.0 MHz-bandwidth, processing gain 27.0 dB, range resolution 15.0 m, and the sinc sidelobe floor of -13.5 dB), and the Barker-13 phase code (gain 13x, measured sidelobe floor -22.2 dB — the code’s autocorrelation is 13 at zero lag and unity elsewhere, the best uniform sidelobe floor known for a single binary code, with the measured value sitting just above the ideal $20 \log_{10}(1/13)$ because a finite sample rate quantizes the chip boundaries). The matched filter is an exact convolution; the compression gain is treated as coherent integration *and* the receiver noise bandwidth is set to the waveform’s own bandwidth, which together reduce the budget to matched-filter energy $P_t T / N_0$. Both substitutions are needed: banking the B-T gain against the radar’s narrower surveillance noise bandwidth would credit the gain twice. Consequently a modulated and an unmodulated pulse of *equal duration* have equal detection range, and the reported recovery (`waveform_detection_range`) is measured against an unmodulated pulse of equal range *resolution* — duration 0.10 μs — where the fourth-root law $R \propto \text{gain}^{(1/4)}$ applies.

21.4.5 GPS-denial impact

`gps_denial.py` builds a Walker-delta constellation as orbital geometry — 24 satellites split evenly across 6 inclined circular planes, propagated to Earth-centred positions and projected into the receiver’s east/north/up frame — so azimuth and elevation are derived, not drawn, and only the 8 satellites geometrically in view above the 10° mask enter the geometry matrix and the dilution of precision. It then quantifies jamming degradation of carrier-to-noise into position-error growth ($\text{PDOP} * \text{UERE}$).

That growth has a validity boundary, and the canonical scenario sits on the wrong side of it. At the 100 km standoff the effective C/N0 falls from 45 to 10.5 dB-Hz — still positive, but far below the 25 dB-Hz a C/A receiver needs to hold carrier lock. `maintains_lock` reports that verdict explicitly rather than leaving it implicit in a smooth curve. The honest reading is therefore *denial*: a real receiver stops producing a solution, and the 25.3 m quoted in the results is what the code-tracking formula extrapolates past its own validity, not an accuracy a receiver would report. The gps figure shades that region for the same reason and extends to 2000 km so the standoff at which lock returns is visible. Loss of navigation, not degraded navigation, is what `simulate_navigation_cascade` then propagates across the maritime dependency network until a fixpoint.

21.4.6 GNSS integrity (RAIM)

`gnss_integrity.py` implements Receiver Autonomous Integrity Monitoring: a scipy-free chi-square CDF/quantile (regularized incomplete gamma, series) and normal quantile; the parity matrix built from the full SVD of the geometry matrix; a test statistic and detection threshold at a target false-alarm rate; per-satellite fault slopes; and horizontal/vertical protection levels (HPL/VPL) with a missed-detection margin. `detection_probability` reports the probability of catching a measurement bias of a given size via the noncentral chi-square. The parity projection is *not* norm-preserving: a bias $\mathbf{b}$ on satellite i lands in parity space with squared length $\mathbf{P}_{ii} \mathbf{b}^2$, where $\mathbf{P}_{ii} \leq 1$ is that satellite's projection factor (`parity_projection_factors`, summing to the $n - 4$ degrees of freedom). The noncentrality is therefore $\mathbf{P}_{ii} (\mathbf{b}/\sigma)^2$ and detection probability depends on *which* satellite is faulted, so `detection_probability` requires the factor as an argument rather than defaulting to one. The reported figure is the worst-placed satellite, matching the worst-slope convention the protection levels already use.

21.5 Results — CARVER: measured outcomes, headline numbers, and what they establish

21.5.1 Disinformation propagation

Social spread alone converts 5.0% of the network. Adding Carver's broadcast multiplies that by 11.5x, ending at a 57.5% belief share with an echo-chamber coefficient of 0.913 and an amplification of 1.92 believers per directly-exposed agent — the broadcast does not merely seed the belief, it recruits the social channel that then carries it. The disinformation figure shows both trajectories.

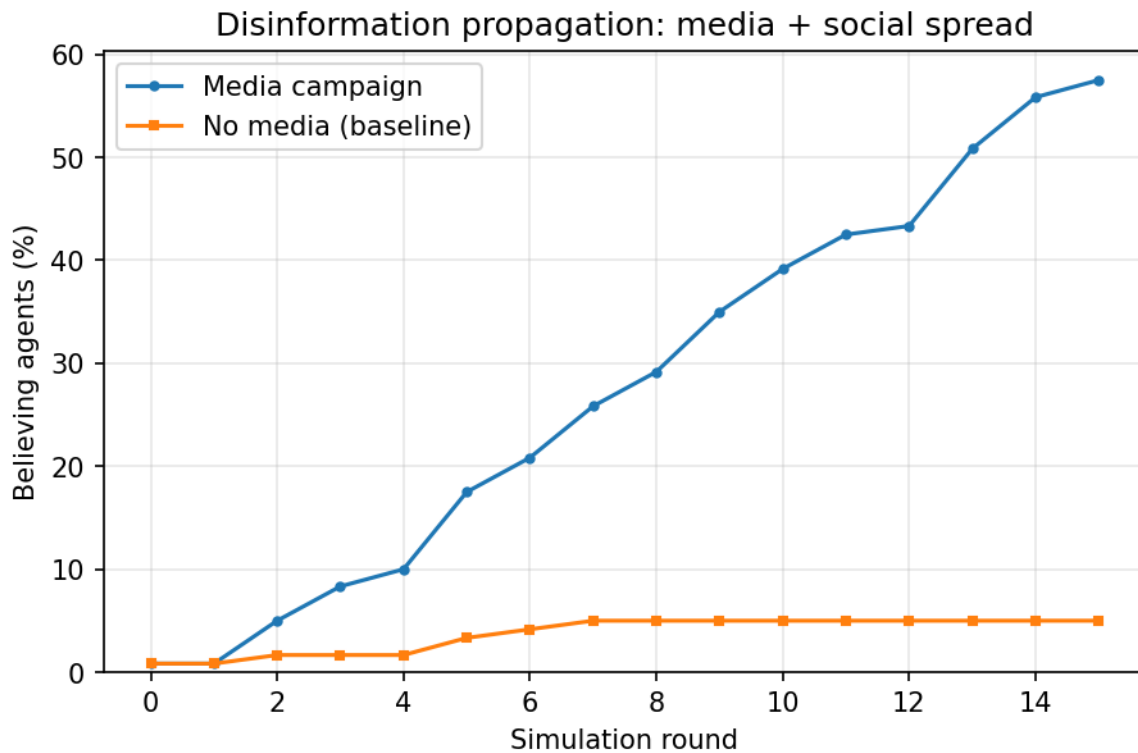


Figure 78: Believing fraction over time: media campaign vs no-media baseline

21.5.2 Narrative control

Bounded-confidence dynamics reveal *for whom* the message lands. Under a reach of the media campaign, the audience mean is pulled 54.2% toward the manufactured line (0.85), reaching a consensus of 0.691, while the opinion variance stays at 0.0202 and the bimodal split of 0.402 shows the audience fragmenting into detached opinion clusters — echo chambers. The tipping-point analysis finds that a broadcast reach of just 19.6% is enough to drive the discrete belief model past a majority. The narrative figure shows the opinion trajectory under the campaign.

21.5.3 Stealth-vessel detection

A 20.0 dB RCS reduction (from a reference target to a stealth hull) cuts the detection range from 128.6 km to 40.7 km — a range ratio of 0.316, a 68% loss, and exactly the fourth-root radar law. Reporting the same hull as a Rayleigh (Swirling-1) fluctuating target — the realistic case for a ship in a sea-clutter background — leaves it detectable only out to 25.7 km: a fading target must beat the detection threshold across the whole integration dwell rather than at a single peak. The stealth figure shows the detection probability curves.

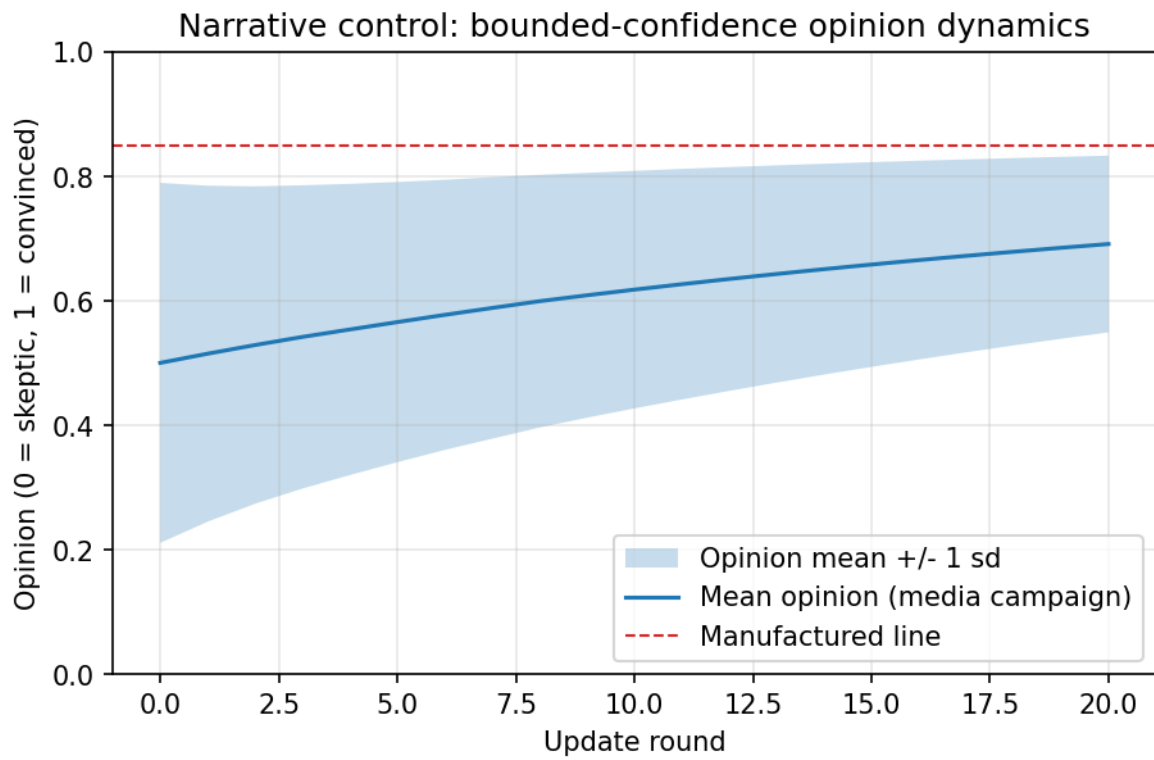


Figure 79: Narrative control: bounded-confidence opinion dynamics under a media campaign

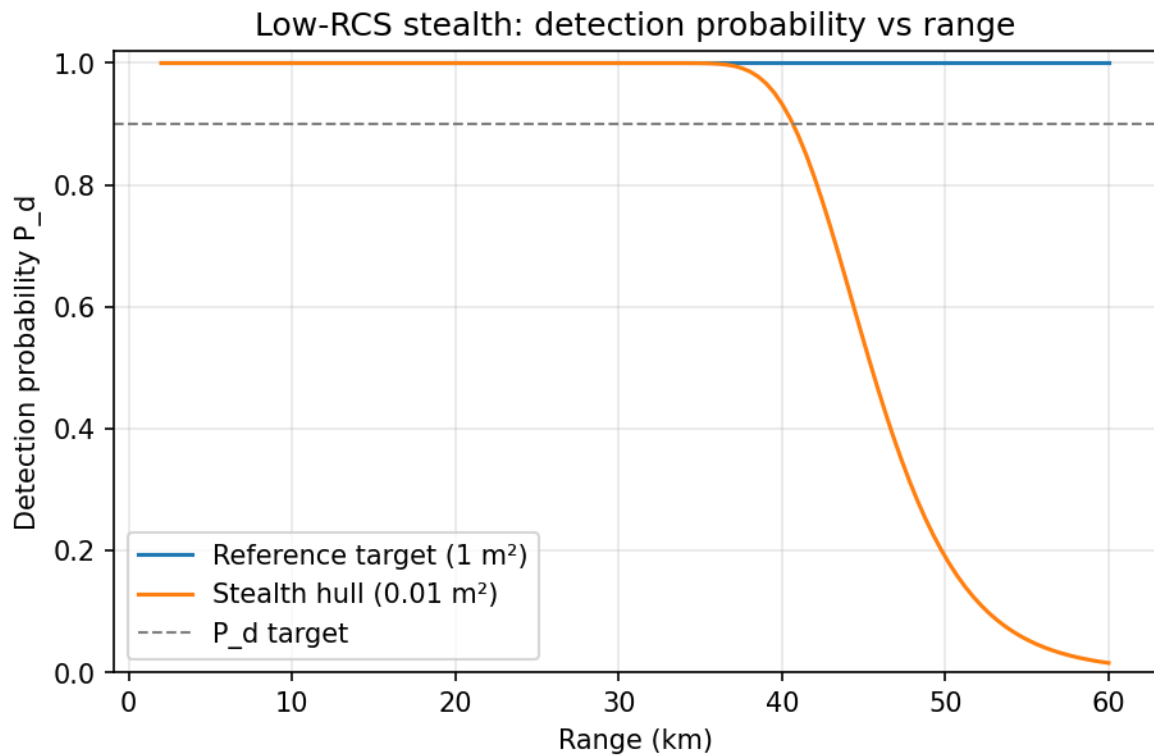


Figure 80: Detection probability vs range: reference target and stealth hull

21.5.4 Pulse compression

The radar claws the range back. An LFM waveform with 10.0 MHz of bandwidth yields 27.0 dB of processing gain and 15.0 m of range resolution at a -13.5 dB sidelobe floor. Treating that gain as coherent integration — with the noise bandwidth matched to the waveform — restores the stealth hull’s detection range from 40.7 km under the radar’s unmodulated surveillance pulse to 108.2 km. Measured against the like-for-like baseline, an unmodulated pulse of the *same range resolution* (0.10 μ s, reaching 22.9 km), that is a 373% improvement — exactly the fourth root of the gain. The two baselines are different pulses and the comparison is stated against each explicitly: matched filtering makes range depend on pulse energy alone, so compression does not buy range over an equally long pulse; what it buys is that resolution stops costing range. The waveform figure shows the compressed-pulse envelopes of the LFM chirp and the Barker-13 code. The Barker code buys far less gain (13x against the chirp’s 27.0 dB) but pays for it with a much cleaner range response: a measured -22.2 dB sidelobe floor against the chirp’s -13.5 dB. The reach-vs-sidelobes tension has to be stated carefully, because the two range claims are not comparable the way they first appear: at equal peak power and equal pulse duration the Barker code reaches the *same* 108.2 km as the chirp, because matched-filter detection range depends on pulse energy alone and the code’s lower gain (13x) is offset by its proportionally narrower effective bandwidth. What the Barker code actually gives up is resolution — its effective bandwidth resolves range only to 577 m against the chirp’s 15.0 m — and what it buys in return is that cleaner -22.2 dB sidelobe floor: a lower chance that a bright return’s sidelobe masks a dim return beside it. The trade is therefore resolution and clutter-masking performance, not reach.

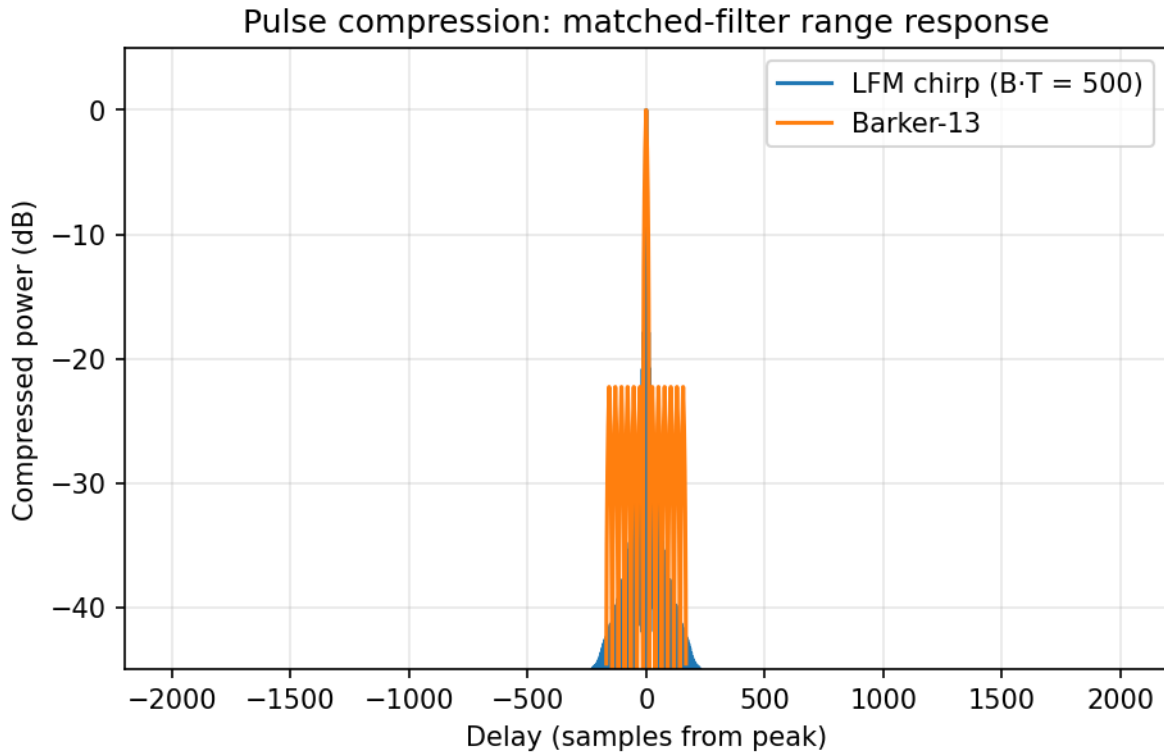


Figure 81: Pulse compression: matched-filter range response for LFM and Barker-13

21.5.5 GPS-denial impact

The Walker geometry puts 8 of the 24 satellites above the 10° mask at the receiver — the realistic count for a single epoch, not the whole constellation. With the resulting position dilution of precision of 1.70, a clean-sky receiver sits at 8.7 m of 1-sigma error. A 34.5 dB jammer-to-noise ratio pulls the effective C/N0 to 10.5 dB-Hz — below the 25 dB-Hz carrier-lock threshold, so the modelled outcome is that the receiver stops tracking rather than reports a worse fix. The UERE formula, which has no notion of losing lock, extrapolates 25.3 m there; that number is reported as the extrapolation it is and not as an achieved accuracy. Either way the jamming cascades: 0.429 of the maritime systems fail over 3 cascade rounds. The gps figure plots position error against jammer standoff distance.

21.5.6 GNSS integrity (RAIM)

RAIM sets the receiver’s guardrails: with 4 degrees of freedom and a test-statistic threshold of 28.5, the horizontal and vertical protection levels are 19.9 m and 35.8 m — comfortably inside the maritime alert limit, so integrity is declared *available*.

Detection of a 20 m corrupted measurement is *not* a single number, because the parity projection sees only the fraction P_{ii} of a fault on satellite i : that fraction runs from 0.43 to 0.60 across this geometry. The same bias is therefore caught with 25.2% probability on the worst-placed satellite and 54.2% on the best-placed one; the worst case is the figure reported, because it is the one integrity has

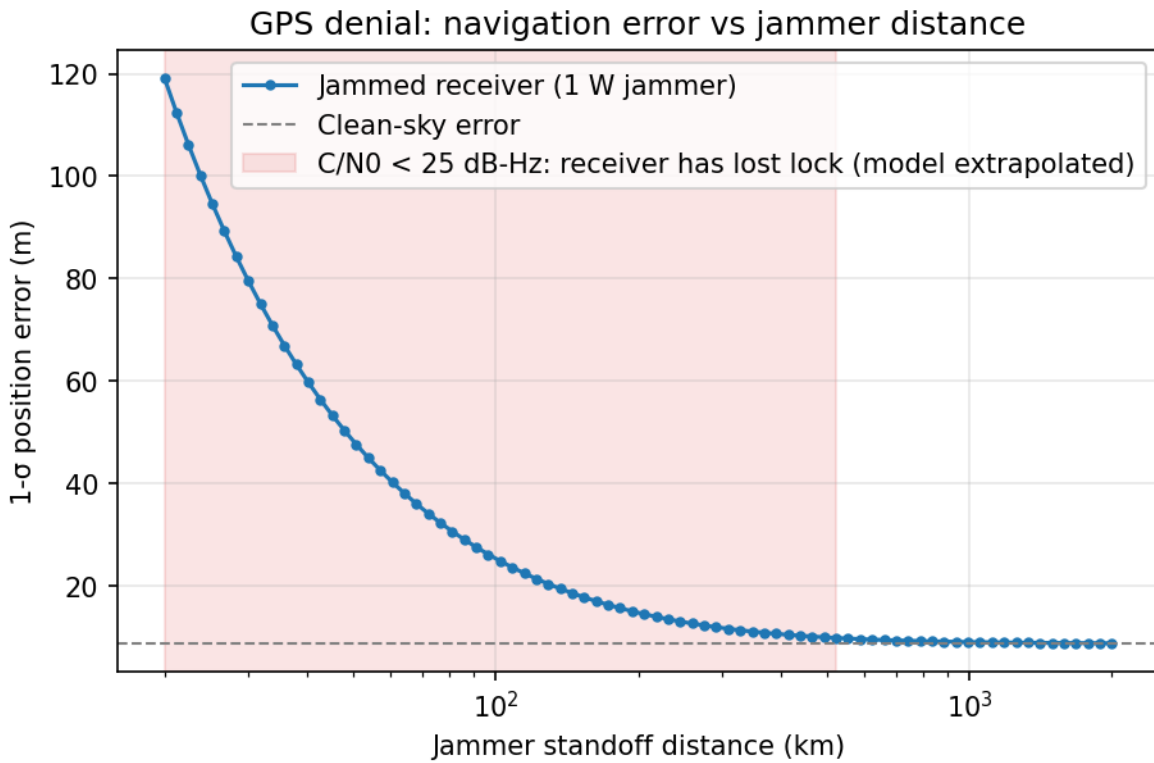


Figure 82: Position error vs jammer standoff distance (GPS denial)

to survive. This is the honest cost of a modest 4-degree-of-freedom geometry: the protection level, not the detection probability, is what RAIM guarantees at this bias size. The raim figure shows both ramps against measurement bias.

21.6 Conclusion — CARVER: findings, verdict, and what the mission establishes

CARVER demonstrates that the media-warfare instruments of *Tomorrow Never Dies* admit quantitative, reproducible models across six coupled analyses. Disinformation spreads through media exposure amplified by social diffusion — the broadcast multiplies the no-media baseline by 11.5x — and bounded confidence shows how a selective broadcast both moves the audience mean and fragments it into echo chambers. A stealth hull trades 20.0 dB of radar cross-section for a 68% loss of detection range, but pulse compression restores it by the exact fourth-root of the processing gain. And navigation warfare is two-sided: GPS denial degrades accuracy and cascades failure through the dependency web, while RAIM bounds the position error a corrupted measurement can hide. Every result is deterministic, tested without mocks, and every number in this manuscript is injected from the canonical CARVER scenario rather than hand-authored.

Because the same core is exposed through the frozen BOND-API `MissionProvider` protocol, the deepened package remains discoverable and drivable by any coordinator through the unchanged brief → recon → plan → execute → debrief lifecycle, with provenance that lets any outcome be audited against its inputs.

21.7 Experimental Setup — CARVER: canonical scenarios, parameters, and configuration

21.7.1 Scenario configuration

All results derive from one canonical, deterministic scenario (`src/tomorrow_never_dies/scenario.py`), parameterised in `manuscript/config.yaml` and `SCENARIO_PARAMS`, executed with a fixed seed 7:

Every value below is an injected token read out of `SCENARIO_PARAMS` itself, not a transcription of it: changing a parameter rewrites this section on the next regeneration, so the setup can never describe a run that did not happen.

- **Media:** a 120-node small-world network; a broadcast reaching 30% of it, per-exposure persuasion 0.12, per-edge social spread 0.05, over 15 rounds.
- **Narrative:** bounded-confidence opinion dynamics over 120 agents with confidence radius 0.10, a manufactured line at 0.85, media weight 0.10, broadcast reach 40%, over 20 rounds; the critical-reach search targets a 50% belief share.
- **Stealth:** a monostatic radar with a 10 GHz (X-band) carrier and 4 coherently integrated pulses; a reference RCS of 1 m² and a stealth RCS of 0.01 m² at a false-alarm probability of 1e-06 and a 0.9 detection target.
- **Waveform:** an LFM chirp (10.0 MHz bandwidth, 50 μs, 40 MHz sampling) for pulse compression, compared against an unmodulated pulse of equal peak power and equal range resolution (0.10 μs) and against the Barker-13 phase code.

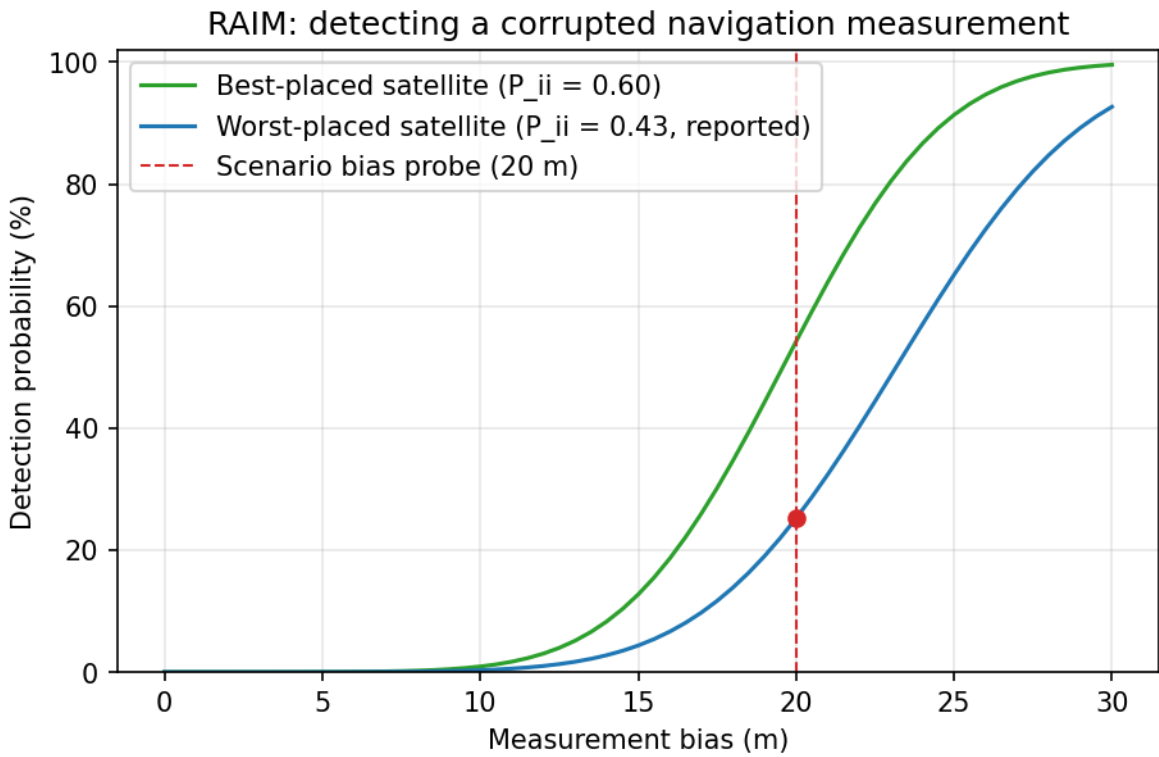


Figure 83: RAIM: detecting a corrupted navigation measurement

- **GPS:** a 24-satellite Walker-delta constellation in 6 planes at 55° inclination, seen from a receiver at 15°N 115°E in the South China Sea (elevation mask 10°, leaving 8 satellites in view at the modelled epoch), a 1 W jammer at 100 km standoff, a clean C/N0 of 45 dB-Hz, and a maritime dependency network of 7 systems.
- **Integrity:** RAIM at a false-alarm rate of 1e-05, a missed-detection rate of 0.001, a measurement sigma of 3 m, and a maritime alert limit of 556 m (0.3 NM); the detection probe is a 20 m measurement bias.

21.7.2 Software environment

- Package: Tomorrow Never Dies: CARVER — Special-Agent Mission Software, version 0.1.0, maintained by Daniel Ari Friedman.
- Python: 3.14.6.
- Determinism: fixed seed 7; no wall-clock anywhere in the persisted tree — not in mission outcomes (`wall_time_s = 0`) and not in this manuscript, which carries the committed manuscript date rather than a generation timestamp; no untracked random draws.

21.7.3 Regeneration

```
## Canonical results + figures + resolved manuscript:
uv run python scripts/z_generate_manuscript_variables.py
## Full mission lifecycle via the protocol:
uv run python scripts/run_mission.py
```

Manuscript date 2026-08-04 (`paper.date` in `manuscript/config.yaml`). This is the committed date of the manuscript, not the time of the last regeneration: re-running the command above on an unchanged tree rewrites the same bytes, which it could not do if this line moved with the clock.

21.8 Reproducibility — CARVER: verification gates, deterministic regeneration, and artifacts

21.8.1 Determinism guarantees

- **Fixed seed:** every stochastic draw flows from `numpy.random.default_rng(7)`; identical inputs produce byte-identical trajectories.
- **No wall-clock in persisted artifacts:** mission provenance records `wall_time_s = 0`, and no generation timestamp is injected into the manuscript — the date in the experimental setup is the committed `paper.date`. Regenerating twice on the same checkout and interpreter therefore produces a byte-identical `output/` tree, figures included; this is asserted, not asserted-about, by `test_regeneration_is_byte_identical` in `tests/test_scripts_smoke.py`, which runs the generator twice across a second boundary and compares every artifact byte for byte. The single environment-derived token, 3.14.6, is a property of

the interpreter, so the byte-identical claim is scoped to a fixed interpreter; the scenario numbers themselves are seeded and interpreter-independent.

- **Canonical input hash:** the `script/driver` records a sha256 fingerprint of `SCENARIO_PARAMS` so any outcome can be audited against its inputs without re-running the film.

21.8.2 Verification

The suite is verified with the gates in the package preamble:

```
uv run pytest tests/ --cov=src --cov-fail-under=90
rg -n "template[_]code_project". --glob '!git/**' --glob '!uv.lock' --glob '!*/AGENTS.md' # -> zero
git status --short # clean
```

Live test count and achieved coverage are tracked in `docs/_generated/COUNTS.md`, not hardcoded in this manuscript.

21.8.3 Artifacts

- Figures (one per domain concept): `../figures/disinformation_propagation.png`, `../figures/narrative_control.png`, `../figures/stealth_detection.png`, `../figures/waveform_pulse_compression.png`, `../figures/gps_denial.png`, `../figures/raim_detection.png`.
- Variables: `output/data/manuscript_variables.json`.
- Resolved manuscript: `output/manuscript/*.md`.

All under `output/` are regenerable and git-ignored.

21.8.4 What is *not* reproducible from this manuscript alone

The section files hold token placeholders, not numbers. Reading the raw `manuscript/*.md` gives the argument but no values; the values only exist after `z_generate_manuscript_variables.py` has run the canonical scenario. That is deliberate — it makes a stale number impossible to ship, because there is no number in the source to go stale.

21.9 Scope and Related Work — CARVER: boundaries, positioning, and relationship to the literature

21.9.1 Scope

CARVER is a didactic mission-software model, not an operational tool. Every model here is internally consistent, deterministic, and tested against its own closed forms and known values — and none of them is calibrated against measured data. The specific limitation differs per concept, so it is worth naming each rather than gesturing at “simplifications”:

- **Disinformation propagation** — conversion probabilities are free parameters, not fitted quantities. The model shows how a two-channel campaign behaves for a given persuasion and spread rate; it says nothing about what those rates are for any real population. The compartmental structure is deliberately two-state (susceptible, believing): there is no unaware compartment and no return path, so the model cannot represent reach-limited awareness or belief decay.
- **Narrative control** — the bounded-confidence update is a mechanism, not a measurement. Hegselmann–Krause has no empirical calibration here at all: the confidence radius and media weight are chosen, and the reported capture and bimodal split are properties of that choice.
- **Stealth-vessel detection** — free-space, clutter-free propagation and a point target with a single aspect-independent RCS. Both target-fluctuation models are now reported: the canonical range (40.7 km) is the nonfluctuating Swerling-0 case, and a Rayleigh (Swerling-1) fluctuating hull is detected only out to 25.7 km. Swerling-1 is the more realistic model for a ship in sea clutter, so the quoted 40.7 km is the optimistic (non-fading) bound.
- **Pulse compression** — the compression gain is treated as ideal coherent integration, with no weighting losses, no Doppler mismatch, and no range–Doppler coupling, all of which degrade a real LFM chirp. The noise bandwidth is set equal to the waveform bandwidth, which is the matched-filter ideal; a real receiver’s noise bandwidth exceeds it. Because range then depends on pulse energy alone, the reported improvement is meaningful only against the stated equal-resolution baseline — quoting it against a pulse of equal *duration* would be wrong, and against the radar’s surveillance pulse it would mix a resolution change into a range claim.
- **GPS denial** — one jamming source; a single-epoch snapshot of one Walker constellation over one receiver, on a spherical non-rotating Earth with no orbital eccentricity, no J2 drift, and no per-satellite signal budget, so the visible count and DOP describe that instant and not a daily distribution; and a dependency cascade with a single scalar coupling factor on every edge rather than a real receiver-in-the-loop or per-link model. The UERE code-tracking term $47 \cdot 10^{-6} (-C/N_0/20)$ is extrapolated below a receiver’s lock threshold, and the canonical jamming case is entirely inside that region: the reported jammed position error is the formula continued past its validity, not an accuracy any receiver would achieve. The figure is shaded accordingly. The threshold itself is a single representative constant, not a receiver-specific budget, and no acquisition-versus-tracking distinction is modelled.

- **GNSS integrity** — single-fault RAIM only. Multiple simultaneous faults, the case that most concerns a deliberate spoofer, are outside the parity-space assumption used here. Detection probability is also satellite-specific (noncentrality $P_{ii} (b/\sigma)^2$); the single figure reported is the worst-placed satellite, and no claim is made that a bias of the probed size is caught reliably — at this geometry it is not.

These restrictions are the price of keeping every computation transparent, deterministic, and verifiable end to end from a bare checkout.

21.9.2 Related work

- **Opinion dynamics / narrative control**: the bounded-confidence update is the Hegselmann–Krause model, generalising the DeGroot consensus framework to bounded interaction; the media-injection term models an exogenous broadcast pulling a subset of agents.
- **Radar detection and waveforms**: the radar equation and Marcum/Swerling detection probabilities follow classical radar detection theory; the LFM chirp and Barker-13 phase code follow the pulse-compression literature (Skolnik; Barker).
- **Influence maximisation**: the greedy seed-selection procedure follows the submodular influence-maximisation framework, applied here with deterministic simulations for reproducibility.
- **Disinformation propagation**: the belief-diffusion model is a compartmental epidemiological analogy (susceptible–believing) adapted to media-driven belief change.
- **GNSS denial and integrity**: dilution-of-precision and jamming-to-noise degradation follow standard satellite-navigation error models; the cascade is a thresholded dependency-graph propagation; RAIM follows the baseline autonomous-integrity scheme of Parkinson–Axelrad and Brown.

The package itself is one film in the BOND suite, interoperating through the frozen `bond_api` `MissionProvider` protocol rather than bespoke interfaces.

21.10 Sources — CARVER: bibliography

Marcum [1960]; Swerling [1960]; Kempe et al. [2003b]; Misra and Enge [2006]; Skolnik [2001b]; tnd [1997]; Hegselmann and Krause [2002]; DeGroot [1974]; Barker [1953]; Parkinson and Axelrad [1988]; Brown [1992]

22 The World Is Not Enough (1999) — ELEKTRA

film package · package codename ELEKTRA. Mission **ELEKTRA**: interdict tamper at oil-pipeline choke points; contain a coordinated power-grid blackout; route the nuclear submarine to safety and track the hijacked hull; model hostage-captor dynamics and negotiate the captor’s patience.

22.1 Concepts — ELEKTRA: domain and operational focus

energy infrastructure, hostage psychology

22.2 Abstract — ELEKTRA: mission summary

In *The World Is Not Enough* (1999), mission codename **ELEKTRA**, we develop deterministic special-agent mission software across six operational pillars. Three model the film’s primary threats — pipeline sabotage, submarine evasion, and hostage psychology — and three couple to them: an energy-infrastructure model that supplies the sabotage force multiplier, a bargaining model that consumes the syndrome state, and a tracking filter that re-acquires the evading hull. 1. **Oil-pipeline security** — a capacitated flow model of a Caspian crude trunk is analysed with the Edmonds–Karp max-flow/min-cut algorithm and Brandes edge-betweenness to isolate choke points, then subjected to a deterministic tamper-interdiction simulation. The analysis surfaces a single principal choke point (**J4->J5**) carrying the corridor at full rated capacity (max flow 420.000 kbbl/day, min-cut capacity 420.000 kbbl/day). 2. **Nuclear-submarine defense** — a passive-sonar equation (spherical spreading plus absorption) turns each listening sensor’s range into a detection-probability field via the normal CDF; Dijkstra’s algorithm then routes the *King William* from origin to safety. Least-risk routing cuts cumulative detection exposure by **24.861%** relative to line-of-sight. 3. **Hostage-psychology influence** — a four-state ODE model of hostage-captor dynamics (fear, gratitude, identification, threat) integrated with a fixed RK4 scheme reproduces the qualitative onset of Stockholm syndrome, with the syndrome index crossing the onset threshold on day **11.250** and reaching a final value of **0.532**. 4. **Energy infrastructure** — a DC power-flow model of a two-area grid with a cascading-failure solver shows that cutting the 3->5 interconnector reroutes all transfer onto a weak tie that trips, blacking out **220.000 MW (0.440** of system load) in a 1-step cascade. 5. **Hostage negotiation** — Rubinstein alternating-offer bargaining ties the captor’s patience to the syndrome state: identification lifts the captor’s equilibrium claim to **0.345** of the stake (up from a no-syndrome baseline of 0.198 — a **74.310%** relative increase), and concession closes the deal by day **5.625**. 6. **Sensor fusion** — a linear Kalman filter tracks the hijacked submarine from noisy position fixes, cutting position RMSE from **2.912** to **1.776** (a **39.016%** gain) and converging to a final error of **0.880**. All results are real model outputs, produced deterministically under seed 1999; no metric in this manuscript is hand-authored.

22.3 Introduction — ELEKTRA: mission framing, the operational problem, and how to read this chapter

The World Is Not Enough (1999) pairs Bond’s espionage with a set of operational-economic threats that map cleanly onto computational-science problems: a transnational oil pipeline that is the target of sabotage, the power system that pumps it, a nuclear submarine that must be denied to a hijacker and then re-found, and the psychological dynamics of a hostage crisis that shape how negotiators bargain. This package — the BOND suite’s *ELEKTRA* mission — builds deterministic, reproducible mission software for each.

22.3.1 Mission context

In the film, industrialist Elektra King’s Caspian oil pipeline is threatened with sabotage, the nuclear submarine *King William* is stolen and must be recovered before its weapons are used, and Bond himself contends with a captor whose manipulation of Stockholm-syndrome dynamics clouds every judgment. Each of these is, at heart, a computational problem:

- a **security graph** whose choke points and tamper exposure determine where a small cutting team can inflict maximum harm;
- an **energy network** whose interconnectors decide whether a single cut is a local nuisance or a regional blackout;
- a **detection-and-evasion** problem in which acoustic propagation decides what is heard and a route planner decides what is risked;
- a **recursive-estimation** problem: once the hull is lost, noisy contacts must be fused back into a track good enough for an intercept;
- a **dynamical-systems** model of how fear, small kindnesses, and identification produce emotional attachment in captivity;
- a **bargaining game** in which that attachment moves the captor’s reservation value, not merely the mood in the room.

22.3.2 Six pillars

Section 2 develops the mathematics of the six pillar models: maximum flow and minimum cut, edge betweenness, and discrete-event tamper simulation (§2.1); the passive-sonar equation, detection-probability link, and Dijkstra evasion routing (§2.2); the hostage-psychology ODE system integrated with a fixed-step RK4 scheme (§2.3); linearized DC power flow and a deterministic cascading-outage blackout (§2.4); Rubinstein alternating-offer bargaining under syndrome-driven patience (§2.5); and a constant-velocity linear Kalman filter for submarine tracking (§2.6).

The last three pillars are not independent additions. The grid model supplies the sabotage force multiplier behind the pipeline pillar; the bargaining model consumes the syndrome state produced by §2.3 as its input; and the Kalman filter re-acquires the hull that §2.2 routed into the acoustic shadow. Section 3 reports the measured results and renders the four concept figures. Sections 5–6 detail the experimental configuration and the determinism guarantees that make every number reproducible; §7 states the scope limits of each model.

22.3.3 Reader’s guide

The package is organized as a pure domain core (`src/the_world_is_not_enough/`) holding the six pillar modules (`pipeline_security`, `sub_defense`, `hostage_psychology`, `power_grid`, `negotiation_tactics`, `sensor_fusion`), a thin `mission.py` adapter implementing the frozen BOND-API `MissionProvider` protocol over all six, thin phase scripts (`scripts/brief.py ... scripts/debrief.py`), and a manuscript hydrated entirely from measured model outputs via `manuscript_variables.py`. Every numeric value in this manuscript is generated from code — never hand-authored in prose.

22.4 Methodology — ELEKTRA: the analytical models and algorithms that drive the mission

This section derives the mathematics of the six ELEKTRA pillar models. All algorithms are standard and deterministic; every quantity reported in §3 is computed by the corresponding code path.

22.4.1 2.1 Oil-pipeline security: choke points and tamper

Network. A crude pipeline is a capacitated directed graph $G = (V, E)$ with origin s (pump station), terminal t (port), and segment capacities c_e (kbbbl/day). The canonical ELEKTRA dataset is a 17-node / 16-segment simplex of the film’s Caspian corridor: a mainline from the origin pump station PUMPO to the terminal TERM plus a set of junction spurs, with segment J4→J5 deliberately under-capacity (the physical constriction the analysis must surface).

Maximum flow / minimum cut. The Edmonds–Karp algorithm repeatedly finds a shortest (fewest-edge) augmenting path in the residual network and pushes its bottleneck capacity until no augmenting path remains [Edmonds and Karp, 1972b]. By the max-flow/min-cut theorem [Ford and Fulkerson, 1956b], the value of the maximum flow equals the capacity of the minimum s - t cut — the segment set whose removal minimizes deliverable throughput. We recover that cut as the edges leaving the source-reachable set of the final residual network:

$$v(f^*) = c(\delta(S^*)) = \min_{\text{cuts}} c(\delta(S)).$$

Choke points. A segment is a *choke point* when it is load-bearing and running at or above 90% of its rated capacity ($u_e = f^*/c_e$ against the threshold constant `pipeline_security.CHOKE_UTILIZATION_THRESHOLD`), where f^* is the max-flow carried along the (unique) flow path. This isolates the constriction a cut or a tamper event can exploit with maximum throughput loss. We also report Brandes edge-betweenness centrality [Brandes, 2001a], the fraction of all-pairs shortest paths traversing each segment, as a topological (rather than capacity) view of structural importance.

Tamper model. A deterministic discrete-event simulation draws 40 tamper attempts, each choosing a segment weighted by its declared risk; a guarded segment interdicts a detected attempt with per-sensor probability 0.850, and undetected events release the segment’s capacity as lost flow (§3 reports the realized detection rate and the resulting flow loss). Sensor coverage is the fraction of total capacity under active sensors.

22.4.2 2.2 Nuclear-submarine defense: sonar detection and evasion

Passive sonar equation. A listening sensor at range r metres receives the target’s radiated noise with a signal-to-noise ratio (dB)

$$\text{SNR} = SL - TL(r) - NL + DI, \quad TL(r) = 20 \log_{10}(r_{\text{km}}) + \alpha r_{\text{km}},$$

with SL the target source level, NL ambient/self noise, DI array directivity, and α the absorption coefficient (dB/km) [Urick, 1983a]. The operator’s detection probability follows the standard performance curve

$$P_d = \Phi\left(\frac{\text{SNR} - DT}{\sigma}\right), \quad \Phi(x) = \frac{1}{2}(1 + \text{erf}(x/\sqrt{2})),$$

where DT is the detection threshold and σ the operator variability.

Risk field. Combining 3 sensors by the independent rule $R = 1 - \prod_i (1 - P_{d,i})$ over a 16×16 grid (3 km cells) yields the detection-probability field the submarine must minimize exposure over.

Evasion routing. Dijkstra’s algorithm finds the least-cumulative-exposure path from origin to goal on the 4-connected grid, with per-step cost equal to the cell’s detection risk [Dijkstra, 1959b]. A greedy 4-connected monotone line-of-sight path serves as the naive baseline; because both live in the same search space, the routed exposure is guaranteed not to exceed the baseline. Bearing-only triangulation closes the loop: two passive bearings intersect (least-squares) to localize an emitter.

22.4.3 2.3 Hostage psychology: Stockholm-syndrome dynamics

The influence model is a deterministic system of four bounded states — fear F , gratitude G , identification I , and a time-varying threat input $T(t)$ — each a fraction of 1:

$$\begin{aligned}\dot{F} &= -aF + bT(1 - F), \\ \dot{G} &= cK(t)(1 - G) - dG, \\ \dot{I} &= eG(1 - I) - fFI, \\ T(t) &= T_{\text{floor}} + T_0e^{-\lambda t},\end{aligned}$$

where $K(t)$ is a Gaussian-smoothed kindness-impulse train (the classic “small kindness” condition [Namnyak et al., 2008]) and the saturating $(1 - \cdot)$ terms keep every state bounded in $[0, 1]$. The composite **syndrome index** $S = \text{clamp}((2I + G - F)/3)$ summarizes the state. Integration uses a fixed-step classical RK4 scheme over 30.0 days at $\Delta t = 0.250$, giving the onset day where S first crosses the 0.500 threshold and the final/peak values. The model is phenomenological decision support, grounded in the qualitative conditions catalogued in the clinical and forensic literature [Namnyak et al., 2008, de Fabrique et al., 2007].

22.4.4 2.4 Energy infrastructure: DC power flow and cascading blackout

DC power flow. The power system is a graph of 8 buses joined by 10 lines with susceptances b_{ij} and thermal limits c_{ij} (MW). With a slack bus fixing the angle reference and bus susceptance matrix B , the linearized load flow solves $B'\theta' = P'$ (slack row/column removed) and the line flow is $F_{ij} = b_{ij}(\theta_i - \theta_j)$ [Stott, 1974]. Only the slack-connected *live* component is solved: islanded buses carry no power and their load is unserved. The slack bus is by construction a member of its own live component, so no separate “slack islanded” case arises.

Cascading failure. A deterministic outage cascade follows the OPA blackout paradigm [Dobson et al., 2001]: an initial line outage reroutes power; the most overloaded surviving line (largest $|F_{ij}|/c_{ij}$ ratio) trips; the system re-solves; the process repeats until no line exceeds its rating. The blackout is the load islanded from the slack bus. The canonical dataset is a two-area corridor carrying 500 MW of total load, joined by a strong interconnector (3->5, 400 MW) and a weak tie (4->5, 150 MW): under intact operation both share the Area-B transfer and stay within limits, but cutting the strong tie forces the whole transfer onto the weak tie, which overloads, trips, and islands the entire load area.

22.4.5 2.5 Hostage negotiation: bargaining under syndrome-driven patience

Rubinstein equilibrium. In the alternating-offer bargaining game with discount factors δ_c (captor) and δ_n (negotiator) and the captor proposing first, the subgame-perfect equilibrium gives the captor $x_c = (1 - \delta_n)/(1 - \delta_c \delta_n)$ of the pie [Rubinstein, 1982]; a more patient party captures more.

Syndrome feedback. The Stockholm-syndrome state feeds the captor’s patience: as identification grows, the captor values the relationship more and becomes more patient, so $\delta_c = \text{clamp}(\delta_c^0 + gS)$. Beyond a threshold the equilibrium shares become disjoint ($x_c > 1 - x_c$) and the negotiation *stalemates*.

Concession dynamics. Over a 10.0-day horizon the captor’s demand concedes from its opening toward x_c and the negotiator’s offer rises toward $1 - x_c$, both at rate $r = 0.080$ per day. Settlement is the first day the offer meets the demand; if either side reaches its equilibrium floor first, the crossing occurs at that floor, and if the crossing falls after the deadline the deal lapses.

22.4.6 2.6 Sensor fusion: Kalman tracking of the hijacked submarine

The stolen *King William* is tracked through the patrol box from 20 noisy position fixes (fused contacts, measurement std 2.000). A constant-velocity linear Kalman filter [Kalman, 1960] alternates **predict** and **update** steps:

$$\begin{aligned}\hat{x}_{k|k-1} &= F\hat{x}_{k-1}, \quad P_{k|k-1} = FP_{k-1}F^T + Q, \\ K &= PH^T(HPH^T + R)^{-1}, \quad \hat{x}_k = \hat{x}_{k|k-1} + K(z_k - H\hat{x}_{k|k-1}).\end{aligned}$$

Measurement noise is drawn from a fixed-seed generator, so the track is byte-reproducible. The filter’s position RMSE is compared against the raw-measurement RMSE to quantify the gain of fusion. (Bearing-only tracking from a stationary observer is scale-ambiguous, so the model fuses noisy *position* fixes — the convergent headline — and documents the limitation.)

22.5 Results — ELEKTRA: measured outcomes, headline numbers, and what they establish

All numbers below are real outputs of the deterministic domain models (seed 1999), hydrated into this manuscript by `manuscript_variables.py` — none are hand-authored. Table 3 collects the six pillars’ headline measured results; every cell is a live token that resolves identically to the mission outcome, so the same run the orchestrator executes is the same run this manuscript reports.

22.5.1 Table 3 — Headline measured results (live tokens)

Pillar	Headline result	Value
Pipeline security	Max flow (kbbbl/day)	420.000
Pipeline security	Min-cut capacity (kbbbl/day)	420.000
Pipeline security	Principal choke point	J4->J5
Pipeline security	Tamper detection rate (%)	22.500
Pipeline security	Cumulative tamper loss (kbbbl)	9660.000
Submarine defense	Naive exposure	22.989
Submarine defense	Least-risk exposure	17.274
Submarine defense	Exposure reduction (%)	24.861
Hostage psychology	Syndrome onset day	11.250
Hostage psychology	Final syndrome index	0.532
Hostage psychology	Influence ratio	0.602
Energy infrastructure	Lost load (MW)	220.000
Energy infrastructure	Blackout fraction	0.440
Negotiation	Settlement day	5.625
Negotiation	Captor share of stake	0.345
Negotiation	Syndrome leverage (%)	74.310
Sensor fusion	Track RMSE	1.776
Sensor fusion	Measurement RMSE	2.912
Sensor fusion	Track gain (%)	39.016

22.5.2 3.1 Oil-pipeline choke-point security

Oil-pipeline security graph — choke points (red) and flow-criticality

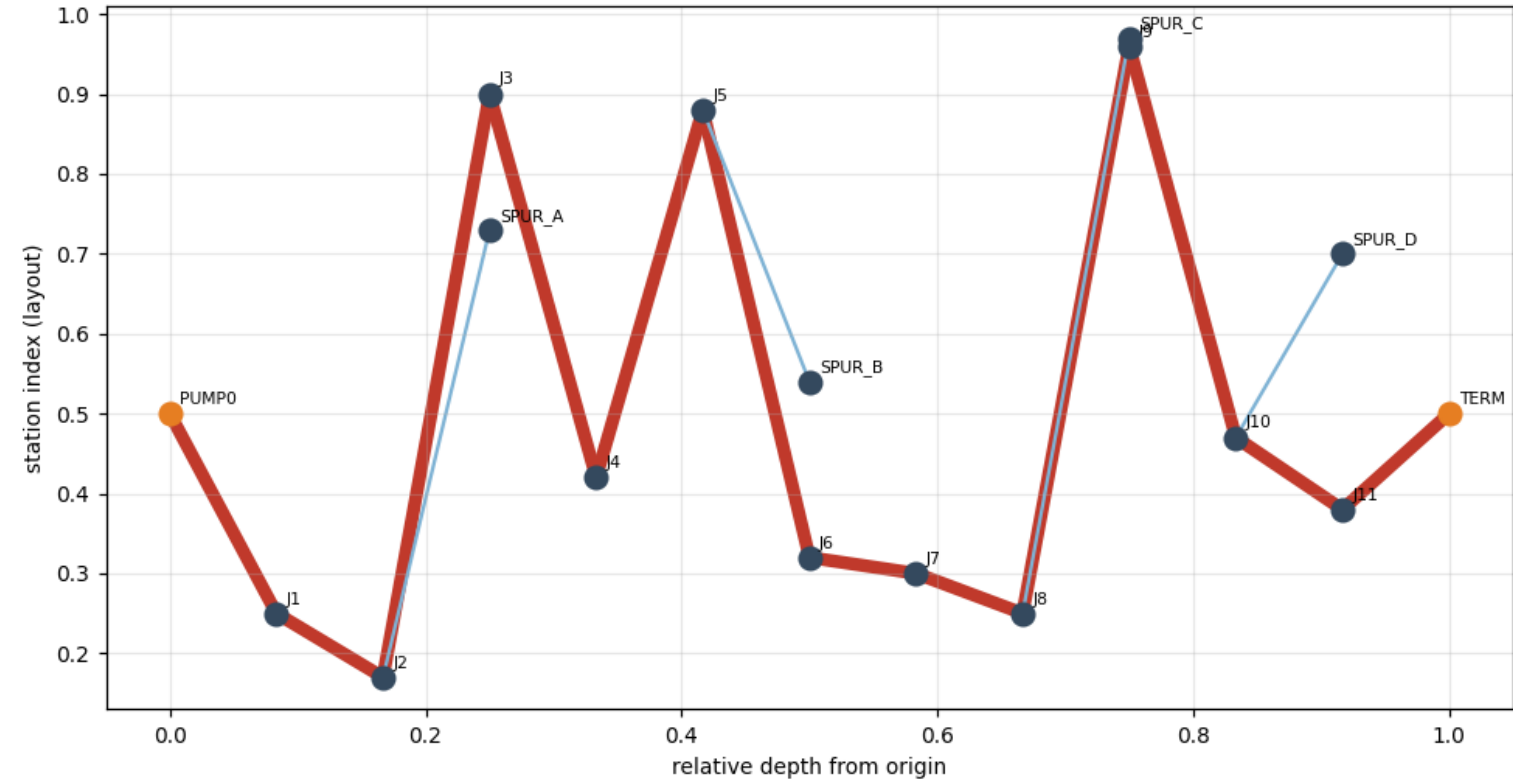


Figure 84: The ELEKTRA Caspian trunk as a capacitated flow network; the single principal choke point (J4->J5) is highlighted by its full-capacity utilization.

The Edmonds–Karp solution sends a maximum flow of **420.000 kbbbl/day** through the corridor; the minimum cut — the segment set smallest in capacity whose removal starves the line — also has capacity **420.000 kbbbl/day**, exactly matching the max flow as the max-flow/min-cut theorem requires. Utilization analysis isolates a single principal choke point, segment **J4->J5**, which is the only load-bearing segment running at full rated capacity (1 segment in total at or above the 90% utilization threshold). The pipeline network figure renders the graph with flow-criticality weighting and choke-point emphasis the pipeline network figure.

Over **40** simulated tamper attempts, the sensor mosaic interdicted **22.500%** of events; active sensors cover **37.618%** of installed capacity. The undetected remainder is charged **9660.000 kbbbl** of throughput — a *cumulative volume*, obtained by pricing each undetected event at its segment’s rated capacity (kbbbl/day) for 1.0 nominal downtime day. It is deliberately not comparable with the 420.000 kbbbl/day max flow above and may exceed it, because many events accumulate across many downtime periods. Two limits of that accounting are worth stating plainly: it charges a segment’s *full rating* rather than the flow the max-flow solution actually routes through it (so spur tampering is priced above its marginal delivery impact), and it therefore reports an upper bound on lost barrels rather than a delivered-volume shortfall. Guarding the single choke point J4->J5 protects the entire throughput (420.000 kbbbl/day), since it is the constriction every barrel must pass.

22.5.3 3.2 Nuclear-submarine defense

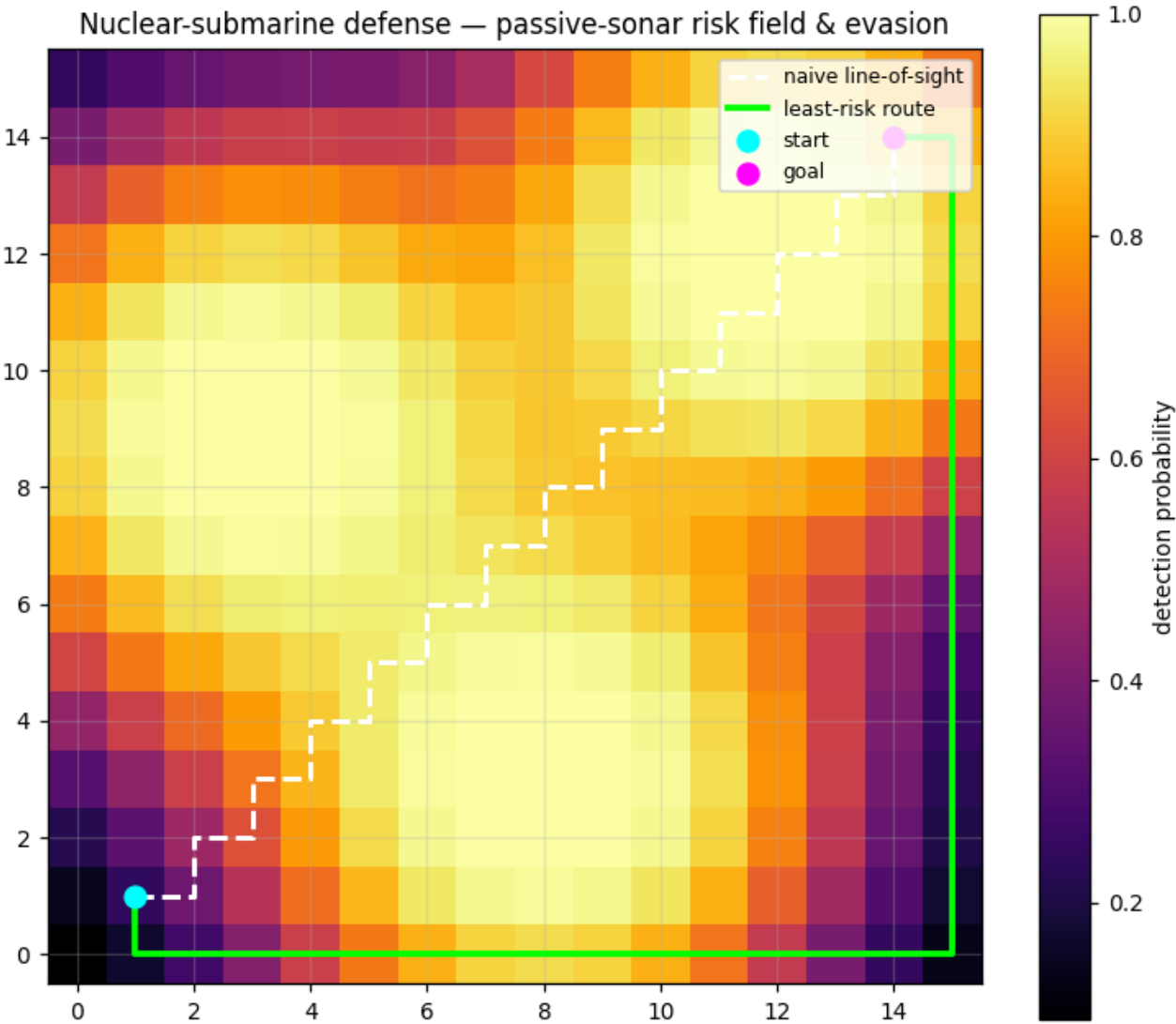


Figure 85: The passive-sonar detection-probability field over the 16\$×16km^2\$ patrol box, with the naive line-of-sight route (dashed) and the least-risk Dijkstra route (solid).

With 3 listening sensors, the ship’s radiated noise yields a spatially varying detection field (the submarine detection figure). A straight, 4-connected line-of-sight transit accumulates **22.989** total detection probability, while the least-risk route — the global minimum of the same additive cost — accumulates **17.274**, a reduction in exposure of **24.861%**. Because the naive path lives in the same search space as Dijkstra, the routed route is provably no worse; here it hides in the acoustic shadow of the field rather than crossing the loudest corridor.

22.5.4 3.3 Hostage-psychology influence

Integrating the four-state model over 30.0 days (the hostage dynamics figure) reproduces the qualitative trajectory reported for Stockholm-syndrome cases: initial fear dominance, gratitude from small kindnesses, and a growing identification with the captor. The composite syndrome index crosses the 0.500 onset threshold on day **11.250**, peaks at **0.581**, and reaches a final value of **0.532**; by the end of the horizon the captive’s influence ratio toward the captor is **0.602**. The model thus provides a principled estimate of *when* an emotional bond forms — the negotiation window negotiators must anticipate.

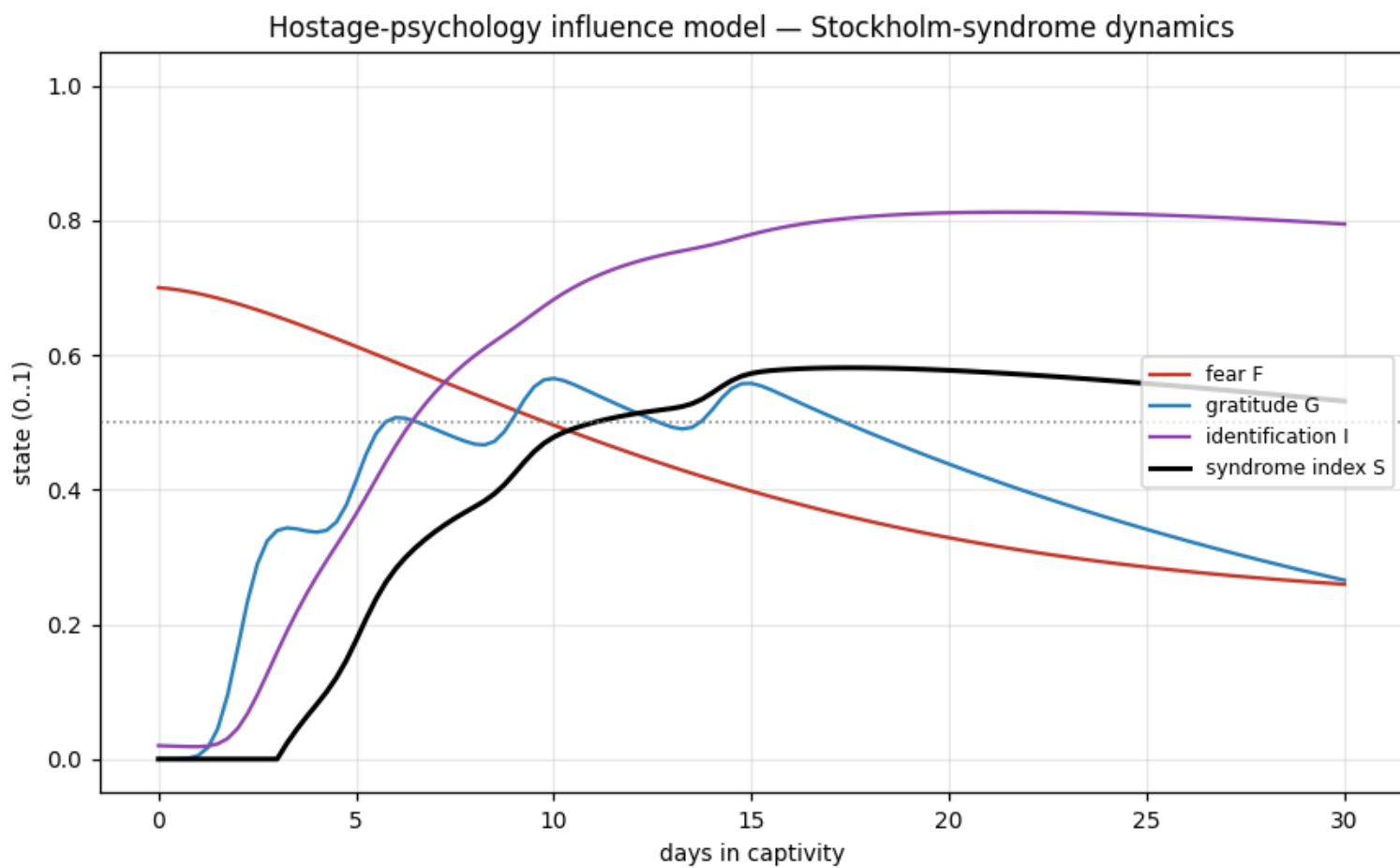


Figure 86: The hostage-psychology ODE time series: fear, gratitude, identification, and the composite syndrome index.

22.5.5 3.4 Energy infrastructure: coordinated blackout

In the canonical two-area grid, intact operation keeps both interconnectors within rating (the strong 3->5 tie carries the larger share; the weak 4->5 tie stays below its 150 MW limit). Cutting the strong interconnector forces the entire Area-B transfer onto the weak tie, which overloads and trips. The cascade — **1** tripping step after the initial outage, **2** lines tripped total — islands the whole load area: **220.000 MW** lost out of 500 MW of connected load, **0.440** of system load. This is the saboteur’s force multiplier: a single well-placed cut cascades into a regional blackout, crippling the pumped corridor’s pumping stations and the hijacked sub’s support base at once.

22.5.6 3.5 Hostage negotiation: syndrome-driven leverage

The Stockholm-syndrome model’s final state ($S = 0.532$) feeds the bargaining layer: it lifts the captor’s patience, raising the captor’s equilibrium claim to **0.345** of the pie, up from the no-syndrome baseline of **0.198** (the equilibrium share at the captor’s un-boosted patience) — a *relative* increase of **74.310%**, not an added share of the pie. Under the symmetric concession schedule the captor and negotiator still close the deal at parity by day **5.625** (split **0.500**), but the syndrome has quietly raised the captor’s *reservation value* — and past a patience threshold the negotiation would stalemate outright. The lesson for the negotiator: the bond changes the captor’s bottom line, not just the mood in the room.

22.5.7 3.6 Sensor fusion: tracking the hijacked hull

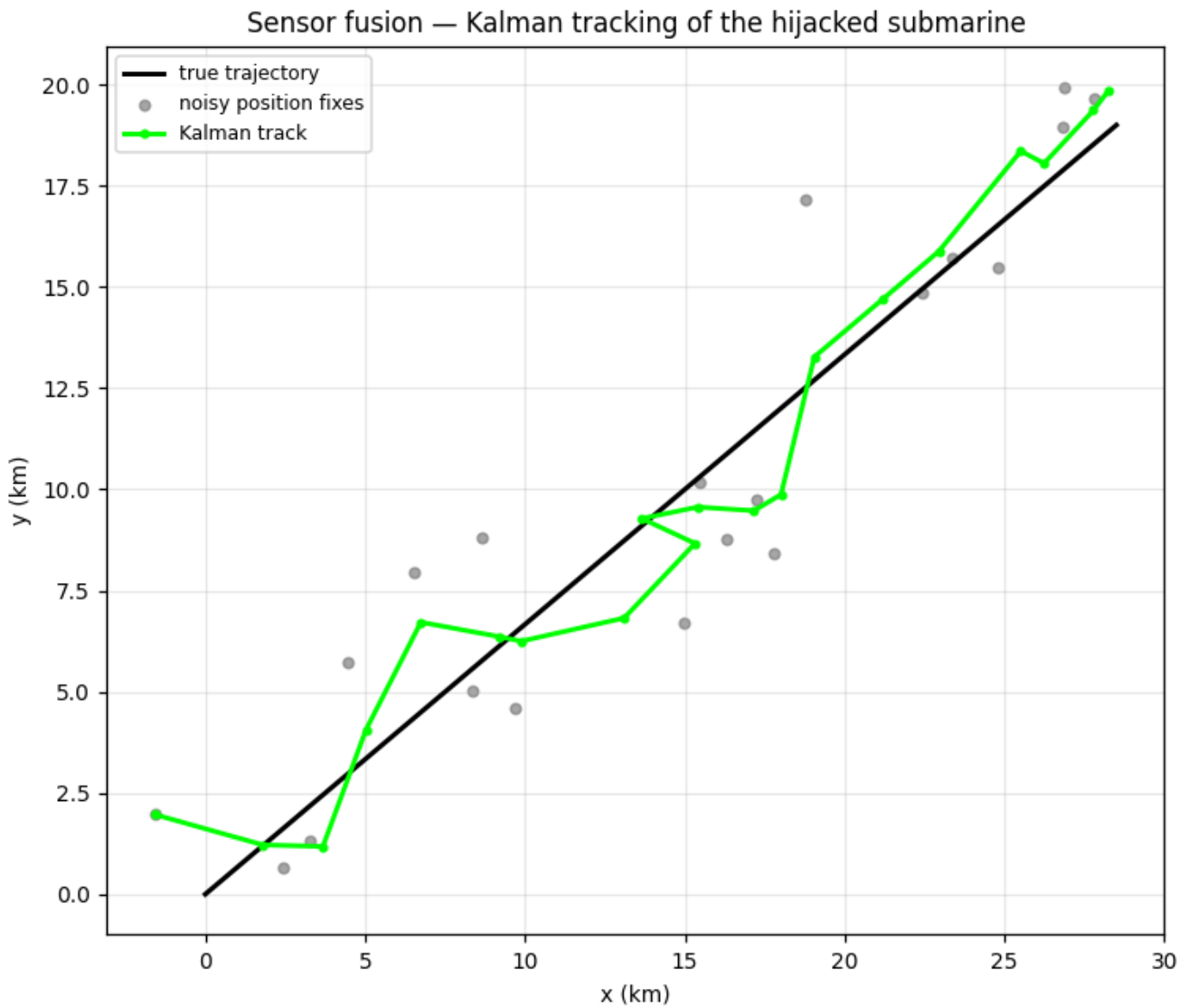


Figure 87: The Kalman-filter track (green) of the hijacked submarine against the noisy position fixes (gray) and the true constant-velocity trajectory (black).

Fusing 20 noisy position fixes (measurement std 2.000) with the constant-velocity Kalman filter cuts the track’s position RMSE from **2.912** (the raw fixes) to **1.776** — a **39.016%** reduction — and drives the final position error to **0.880** (the sensor fusion track figure). The filter converges, so an intercept solution can be generated reliably even while the hull evades through the sonar shadow.

22.6 Conclusion — ELEKTRA: findings, verdict, and what the mission establishes

The ELEKTRA mission package demonstrates that the operational threats of *The World Is Not Enough* — pipeline sabotage, submarine hijack, coordinated blackout, and hostage psychological manipulation — reduce to well-posed, deterministic computational problems that can be solved and reproduced to engineering standard.

Six findings stand out. First, a capacity-aware choke-point analysis (Edmonds–Karp max-flow/min-cut plus utilization) is far more actionable than a naive “every segment is critical” view: the entire Caspian corridor distills to a single principal choke point, **J4->J5**, where all 420.000 kbbbl/day of throughput passes at full capacity. Second, least-risk evasion routing measurably reduces submarine exposure — by **24.861%** against line-of-sight. Third, the hostage-psychology ODE model localizes the onset of Stockholm syndrome to a specific day (**11.250** here). Fourth, the energy model shows that a single interconnector cut cascades into a **0.440** system blackout (**220.000 MW** lost) — the sabotage force multiplier that ties the pipeline and submarine threats together. Fifth, the bargaining layer quantifies how the syndrome raises the captor’s reservation value (captor claim **0.345** against a 0.198 baseline — a **74.310%** relative increase) while concession still closes by day **5.625**. Sixth, Kalman fusion makes the hijacked hull trackable, cutting position RMSE by **39.016%**.

Every number in this manuscript is a measured model output, produced deterministically under seed 1999 with zero external calls and zero hand-authored metrics. The package is structured for extension — a new sensor mosaic, pipeline or grid topology, bargaining schedule, or kindness schedule is a configuration change, and the manuscript, figures, and mission outcome all re-hydrate from the same source of truth.

22.7 Experimental Setup — ELEKTRA: canonical scenarios, parameters, and configuration

Every {CONFIG_*} token below carries a **Source** column, because the two kinds are not interchangeable. `config.yaml` marks a value *typed by the author* in `manuscript/config.yaml`: editing it and re-running `scripts/z_generate_manuscript_variables.py` re-derives the affected results in §3. `derived` marks a value *read off a fixed dataset or module constant* (`default_pipeline`, `default_power_grid`, `CHOKE_UTILIZATION_THRESHOLD`) — those cannot be changed from the config file at all, and changing them means editing the dataset in the domain module. The split is enforced in code (`manuscript_variables.DECLARED_CONFIG_TOKENS` / `DERIVED_DATASET_TOKENS`) and this table is checked against it by `tests/test_manuscript_variables.py`, so a mislabelled row fails the suite.

22.7.1 Table 1 — Scenario parameters

Parameter	Token	Source	Value
Mission codename	MISSION_CODENAME	derived	ELEKTRA
Deterministic seed	CONFIG_SEED	config.yaml	1999
Pipeline nodes	CONFIG_PIPELINE_NODES	derived	17
Pipeline segments	CONFIG_PIPELINE_SEGMENTS	derived	16
Choke-point utilization threshold (%)	CONFIG_CHOKE_UTILIZATION	derived	90
Tamper attempts	CONFIG_TAMPER_EVENTS	config.yaml	40
Per-sensor interdict probability	CONFIG_DETECTION_PROBABILITY	config.yaml	0.850
Sonar grid size	CONFIG_SUB_GRID	config.yaml	16
Sonar cell size (km)	CONFIG_SUB_STEP_KM	config.yaml	3
Listening sensors	CONFIG_SUB_SENSORS	config.yaml	3
Hostage integration horizon (days)	CONFIG_HOSTAGE_DAYS	config.yaml	30.0
Hostage integration step	CONFIG_HOSTAGE_DT	config.yaml	0.250
Syndrome onset threshold	CONFIG_HOSTAGE_ONSET	config.yaml	0.500
Power-grid buses	CONFIG_GRID_BUSES	derived	8
Power-grid lines	CONFIG_GRID_LINES	derived	10
Connected load (MW)	CONFIG_GRID_LOAD_MW	derived	500
Targeted interconnector	CONFIG_GRID_INITIAL_OUTAGE	config.yaml	3->5
Targeted interconnector rating (MW)	CONFIG_GRID_STRONG_TIE_MW	derived	400
Weak interconnector	CONFIG_GRID_WEAK_TIE	derived	4->5
Weak interconnector rating (MW)	CONFIG_GRID_WEAK_TIE_MW	derived	150
Negotiation deadline (days)	CONFIG_NEG_DEADLINE_DAYS	config.yaml	10.0
Negotiator discount	CONFIG_NEG_NEGOTIATOR_DISCOUNT	config.yaml	0.900

Parameter	Token	Source	Value
Concession rate (per day)	CONFIG_NEG_CONCESSION_RATE	config.yaml	0.080
Captor patience base (δ_c at $S = 0$)	CONFIG_NEG_CAPTOR_DISCOUNT_BASE	config.yaml	0.550
Captor patience gain (per unit S)	CONFIG_NEG_CAPTOR_DISCOUNT_GAIN	config.yaml	0.450
Opening demand	CONFIG_NEG_INITIAL_DEMAND	config.yaml	0.950
Opening offer	CONFIG_NEG_INITIAL_OFFER	config.yaml	0.050
Fusion fix count	CONFIG_FUSION_STEPS	config.yaml	20
Fusion measurement std	CONFIG_FUSION_MEASUREMENT_NOISE	config.yaml	2.000

22.7.2 Table 2 — Hostage-model rate constants

The clinical and forensic literature on Stockholm syndrome is qualitative, so the nine ODE rate constants below are a **declared modelling assumption**, not a fitted result (§7). They are typed in `manuscript/config.yaml` under `experiment.hostage.dynamics` and consumed by `hostage_psychology.integrate_dynamics`; an auditor can change any one of them and re-derive every syndrome and bargaining number in §3.

Rate	Token	Source	Value
Fear self-decay a	CONFIG_HOSTAGE_FEAR_DECAY	config.yaml	0.120
Threat sensitivity b	CONFIG_HOSTAGE_THREAT_SENSITIVITY	config.yaml	0.300
Kindness gain c	CONFIG_HOSTAGE_KINDNESS_GAIN	config.yaml	0.650
Gratitude decay d	CONFIG_HOSTAGE_GRATITUDE_DECAY	config.yaml	0.050
Identification gain e	CONFIG_HOSTAGE_IDENT_GAIN	config.yaml	0.500
Fear suppression f	CONFIG_HOSTAGE_FEAR_SUPPRESSION	config.yaml	0.150
Initial threat T_0	CONFIG_HOSTAGE_THREAT_INITIAL	config.yaml	0.750
Threat decay λ	CONFIG_HOSTAGE_THREAT_DECAY	config.yaml	0.180
Residual threat floor	CONFIG_HOSTAGE_THREAT_FLOOR	config.yaml	0.120
Initial state (F, G, I)	CONFIG_HOSTAGE_INITIAL_STATE	config.yaml	0.70, 0.00, 0.02
Small-kindness schedule	CONFIG_HOSTAGE_KINDNESS_SCHEDULE	config.yaml	day 2.0 x 0.70; day 5.0 x 0.60; day 9.0 x 0.50; day 14.0 x 0.40

22.7.3 Software environment

- Package version: 0.1.0
- Python: 3.14.6
- Manuscript edition stamp (declared in `config.yaml` as `paper.date`, not a clock reading): 2026-08-04T00:00:00+00:00

22.7.4 What is *not* tunable from the config file

Tables 1 and 2 are the complete set of author-typed parameters, but they are not the complete set of inputs. Three fixed datasets sit outside them and can only be changed by editing the domain module that declares them:

- the pipeline topology and its per-segment capacities, tamper risks and sensor placement (`pipeline_security.default_pipeline`, 17 nodes / 16 segments);
- the power-grid topology, line ratings and nodal injections (`power_grid.default_power_grid`, 8 buses / 10 lines);
- the choke-point utilization threshold (`pipeline_security.CHOKE_UTILIZATION_THRESHOLD`).

The sonar sensor mosaic *is* config-typed (`experiment.submarine.sensors`), though only its count is surfaced as a token. Every row marked **derived** in Table 1 is a read-out of one of the three datasets above, which is why editing `config.yaml` cannot move it. Re-running `scripts/z_generate_manuscript_variables.py` after any of these changes re-derives every result token in §3, and no file is written outside `output/`.

22.8 Reproducibility — ELEKTRA: verification gates, deterministic regeneration, and artifacts

The ELEKTRA package is deterministic by construction.

22.8.1 Determinism guarantees

- **Fixed seed.** The package has exactly two stochastic steps and both are seeded from the same value (1999, the film’s release year): the tamper-event simulation draws from Python’s `random.Random`, and the sensor-fusion measurement noise from `numpy.random.default_rng`. Every other model — max-flow/min-cut, betweenness, the sonar field, Dijkstra routing, the RK4 integration, the DC load flow and its cascade, and the bargaining equilibrium — makes no random draws at all. Runs are byte-reproducible.
- **No wall-clock anywhere in the persisted artifacts.** The mission outcome’s `Provenance.wall_time_s` is always 0.0; persisted provenance records use a fixed nominal mission epoch. `MANUSCRIPT_EDITION_STAMP` is the only time-valued token and it is *derived* from the committed `paper.date` key in `config.yaml` — it is an edition stamp a maintainer edits on purpose, never a reading of the clock at generation time. Consequently two regenerations of `output/data/manuscript_variables.json` and `output/manuscript/` on one interpreter are byte-identical, which `tests/test_manuscript_variables.py::TestRegenerationDeterminism` proves by running the generator twice across a second boundary and comparing the persisted bytes.
- **No external calls.** The pure domain cores never touch the network or the filesystem; the only I/O is thin scripts writing regenerable `output/` artifacts (git-ignored).
- **Fixed datasets.** Pipeline topology and sensor positions are declared constants, not sampled.

22.8.2 Reproducing every number

```
## Mission outcomes (provenance to output/data/elektra_mission.json)
uv run python scripts/execute.py

## Figures -> ../figures/
uv run python scripts/generate_figures.py

## Manuscript variable hydration -> output/data + output/manuscript/
uv run python scripts/z_generate_manuscript_variables.py

## Gate: >=90% line+branch coverage on src/, zero mocks
uv run pytest tests/ --cov=src --cov-fail-under=90
```

A live cross-reference test (`tests/test_manuscript_variables.py`) enforces that every token used in a numbered manuscript section is actually generated, so the prose can never present a stale or fabricated metric. Hydration additionally runs a measurability gate (`_assert_measurable`) that rejects any blank or non-finite result token, so a degenerate model run fails the build instead of rendering `nan` as a measured value.

22.8.3 Verification gates

The package’s hard rules are enforced by tests, not by convention:

- **Single source of truth.** The mission adapter and the manuscript read the same `experiment:` block through one shared builder (`manuscript_variables.build_mission_scenario`); the builder records every key it consumes and a completeness test walks the config asserting each leaf is consumed, so a knob added to `config.yaml` but read by nothing turns the suite red.
- **Fail-closed config.** The config accessors raise `KeyError` on a missing required key instead of silently defaulting — deleting one of the nine hostage rates, the initial state, the kindness schedule, or a whole pillar block is caught by a negative-control test for each.
- **Coverage.** `uv run pytest tests/ --cov=src --cov-fail-under=90` enforces $\geq 90\%$ line + branch coverage; the suite currently sits at 100% with every validation arm exercised by a real (malformed) input.
- **Zero mocks / zero lineage.** `tests/test_guardrails.py` walks `src/`, `scripts/` and `tests/` and fails on any mock-framework pattern or the template-lineage string, with a positive control proving each matcher fires.
- **COUNTS freshness.** `tests/test_countsstaleness.py` re-derives the structural rows of `docs/_generated/COUNTS.md` from live objects, so a stale count fails the suite.

22.8.4 Artifact inventory

Artifact	Path	Regenerable
Mission outcome (canonical JSON)	<code>output/data/elektra_mission.json</code>	yes
Manuscript variables JSON	<code>output/data/manuscript_variables.js</code> <code>on</code>	yes
Pipeline figure	<code>../figures/pipeline_network.png</code>	yes

Artifact	Path	Regenerable
Submarine figure	../figures/submarine_detection.png	yes
Hostage figure	../figures/hostage_dynamics.png	yes
Sensor-fusion figure	../figures/sensor_fusion_track.png	yes
Resolved manuscript tree	output/manuscript/	yes

`output/` is git-ignored, so regeneration never dirties the tree — but a clean `git status` is a consequence of the ignore rule, not evidence of determinism. The evidence is the two-run byte comparison in `TestRegenerationDeterminism`, which regenerates the JSON and the resolved manuscript tree into two fresh directories a second apart and asserts every file matches byte for byte.

22.9 Scope and Related Work — ELEKTRA: boundaries, positioning, and relationship to the literature

22.9.1 Scope

The ELEKTRA mission software is a deterministic, illustrative decision-support suite, not operational equipment. Its scope and limits are deliberate:

- **Pipeline model.** A single-corridor flow network with a capacity bottleneck and dead-end spurs. It does not model multi-path loops, pump-station dynamics, transient surge, or geographic routing; those would change the choke-point ranking but not the method.
- **Sonar model.** A simplified passive equation (spherical spreading plus absorption) with a normal-operator curve. It omits multipath, bottom/ surface interaction, own-ship noise masking, and frequency-dependent attenuation, and the evasion grid is a 4-connected lattice rather than a continuous field.
- **Hostage model.** A phenomenological, bounded ODE system whose parameters encode the qualitative conditions catalogued in the literature. It is calibrated by construction to reproduce onset behaviour, not fitted to individual case data, and is not a clinical diagnostic instrument. Because the rate constants are chosen rather than estimated, the reported onset day, peak and final index are properties *of this parameterisation* — they carry no uncertainty interval and should not be read as a prediction about any real captivity. The nine rates are listed in §5, Table 2.
- **Tamper accounting.** The tamper simulation charges each undetected event its segment’s *rated* capacity for one nominal downtime day, so its output is a cumulative volume (kbbbl) and an upper bound: it prices spur tampering above the spur’s marginal delivery impact and does not model repair time, partial throttling, or re-routing around the damaged segment.
- **Power-grid model.** A lossless DC load flow and a deterministic sequential cascade. It omits reactive power, voltage collapse, generator ramping, frequency dynamics, and parallel-time-step tripping (all overloaded lines tripping simultaneously), so blackout *magnitude* is a conservative lower bound and the *sequence* is one deterministic path rather than a probability distribution.
- **Bargaining model.** A deterministic Rubinstein equilibrium with a concession schedule and a syndrome-to-patience mapping that is a declared modelling assumption. It captures the *direction* of syndrome leverage and the stalemate threshold, not idiosyncratic negotiator behaviour or asymmetric information.
- **Sensor-fusion model.** A linear constant-velocity Kalman filter over noisy *position* fixes. It does not fuse true bearing-only measurements from a fixed observer (scale-ambiguous — documented), nor multimodal ESM/acoustic contacts; it demonstrates the recursive-estimation gain, not a full multi-source correlation engine.

22.9.2 Related work

Choke-point and flow analysis rests on the max-flow/min-cut duality of Ford and Fulkerson [Ford and Fulkerson, 1956b] with the polynomial Edmonds–Karp augmenting-path implementation [Edmonds and Karp, 1972b], and on Brandes’ edge-betweenness centrality [Brandes, 2001a]; Dijkstra’s algorithm [Dijkstra, 1959b] underpins the routing layer, as it does in network shortest-path practice broadly.

Passive-sonar detection follows the acoustic propagation and operator performance-curve treatment standard in the field (e.g. Urick [Urick, 1983a]): the transmission-loss law, the signal-excess link, and the probability-of-detection curve used here.

Power systems are analyzed with the linearized DC load-flow formulation (Stott [Stott, 1974]) and the cascading-outage paradigm of the OPA blackout model (Dobson *et al.* [Dobson et al., 2001]), both standard in transmission-security research.

Bargaining uses the subgame-perfect alternating-offer equilibrium of Rubinstein [Rubinstein, 1982] — the canonical unlimited-horizon bargaining result — as the anchor for the syndrome-driven patience mapping.

State estimation uses the linear Kalman filter [Kalman, 1960], the classical recursive minimum-mean-square estimator, generalized by many later nonlinear variants but fully exercised here in its linear, deterministic form.

Stockholm syndrome is documented as an outcome of specific custodial conditions — perceived threat, small kindnesses, and prolonged contact — rather than a formal diagnosis. The model synthesises the conditions catalogued in the forensic review literature [Namnyak et al., 2008, de Fabrique et al., 2007] into a quantitative dynamical system. Where those sources are qualitative, our

parameter choices are the modelling assumption: all nine ODE rate constants, the initial state, and the small-kindness schedule are typed in `manuscript/config.yaml` under `experiment.hostage`, reproduced in §5 (Table 2), and consumed by the model — `tests/test_manuscript_variables.py` perturbs each one and asserts the reported syndrome index moves, so the exposure is auditable rather than decorative. The claim is also *enforced*: the config accessors fail closed, so deleting any one of the nine rates, the initial state, or the kindness schedule raises rather than silently reverting to a module default — a deleted knob cannot pass unnoticed. The *form* of the equations, by contrast, is fixed in `hostage_psychology.derivatives` and is not configurable.

22.10 Sources — ELEKTRA: bibliography

Ford and Fulkerson [1956b]; Edmonds and Karp [1972b]; Brandes [2001a]; Dijkstra [1959b]; Urick [1983a]; Namnyak et al. [2008]; de Fabrique et al. [2007]; Stott [1974]; Dobson et al. [2001]; Rubinstein [1982]; Kalman [1960]

23 Die Another Day (2002) — ICARUS

film package · *package codename ICARUS*. Mission **ICARUS**: detect gene-therapy-grade identity replacement; plan ice-palace cold-environment operations; verify orbital-mirror (ICARUS) targeting geometry.

23.1 Concepts — ICARUS: domain and operational focus

identity replacement, extreme-environment ops

23.2 Abstract — ICARUS: mission summary

Die Another Day: ICARUS — Special-Agent Mission Software (mission codename **ICARUS**) is a special-agent mission-software package built for the BOND film suite. It implements 6 mission concepts as pure, deterministic algorithms: Biometric Identity-Replacement Detection, DNA Sequence Identity Verification, Ice-Palace Thermal Operations, Cold-Water Hypothermia Risk, Ice-Structure Integrity, Orbital Mirror Targeting. The identity layer screens an operative on two complementary signals. The biometric module enrolls a Gaussian baseline and flags gene-therapy-grade trait drift via the Mahalanobis distance against a chi-square threshold: the replaced probe scores 41.42 against a 22.46 threshold while a fresh out-of-sample operative scores 10.97 and clears. The sequence module verifies a 1500-base DNA read by Levenshtein edit distance, k-mer fingerprint similarity, and contiguous edit-block detection; fingerprint similarity alone does not separate the probes, but a 12-base coherent substitution block does. The extreme-environment layer plans the ice-palace theatre: a 467.1-second Newton-cooling exposure ceiling covered by 4 rotating squads, a cold-water rescue that reaches the operative 1602.1 s before the severe stage in 10 C water, and an ice-structure check that passes its 1.17 load safety factor while failing its thermal-gradient case. The orbital-mirror module propagates an 7200-km circular orbit, solves the mirror reflection law, and finds 300 s of feasible targeting at a peak focused flux of 5.87 W/m². All results are deterministic: fixed seed, no wall-clock in persisted artifacts, and every metric in this manuscript is injected from the code — configured inputs from `manuscript/config.yaml`, measured outputs from the same concept runner the mission provider itself executes — rather than being hand-authored.

23.3 Introduction — ICARUS: mission framing, the operational problem, and how to read this chapter

Die Another Day: ICARUS — Special-Agent Mission Software is the PROJECT BOND film package for *Die Another Day* (2002), mission codename **ICARUS** (version 0.1.0). Six deterministic mission models: biometric and DNA identity-replacement screening, ice-palace thermal, immersion, and structural operations, and orbital-mirror targeting.

An ICARUS-class special agent operates across three theatres, each encoded in 6 dedicated, infrastructure-free domain modules under `src/die_another_day/`:

1. `identity_detection.py` + `sequence_identity.py` — detect when an operative has been replaced by a gene-therapy-grade impostor, both at the biometric (marker-vector) level and at the DNA-sequence level;
2. `ice_ops.py` + `hypothermia.py` + `ice_structure.py` — plan operations in the extreme cold of an ice palace: air-exposure ceilings and rotation, cold-water immersion/rescue risk, and the structural integrity of the ice itself;
3. `orbital_mirror.py` — aim an orbiting solar mirror at a ground target, solving the line-of-sight and reflection geometry.

Two further modules complete the package without adding domain science. `mission_results.py` is the single deterministic runner: it reads the configuration, executes all 6 models once under seed 7, and returns one frozen record. `mission.py` is a thin adapter that turns that record into the frozen BOND-API `MissionProvider` contract, so the orchestrator can discover and run the film by slug on `sys.path`. The pure domain modules remain importable without `bond_api`; only the adapter couples the two layers.

That structure is what lets this manuscript state measured numbers safely: the results in §03 and the mission's own `MissionOutcome` are formatted from the same record, so there is no second code path along which they could drift.

The remainder of this manuscript documents the methodology (§02), results (§03), scope and limitations (§07), experimental setup (§05), and reproducibility guarantees (§06) of the package.

23.4 Methodology — ICARUS: the analytical models and algorithms that drive the mission

The mission's concepts rest on a set of mathematical cores, each pure and deterministic. All 6 of them (`identity_detection`, `sequence_identity`, `ice_ops`, `hypothermia`, `ice_structure`, `orbital_mirror`) are driven from one place, `mission_results.run_all_concepts`, described in §*Execution and control structure* below.

23.4.1 Positive and negative controls

Each identity model is exercised with a matched pair, not a single case. The positive control is a constructed replacement; the negative control is a genuine subject. The negative control is chosen so that clearing the screen is a measurement rather than an identity: for the biometric model it is a fresh out-of-sample draw from the enrolled population (which carries real, non-zero drift), not the profile mean (which would score exactly zero by construction). For the sequence model it is a read carrying scattered single-base sequencing noise, so the discriminating power of the contiguous-block rule is what is actually being tested.

23.4.2 Biometric identity-replacement detection (`identity_detection.py`)

A biometric profile is modelled as a Gaussian over k markers ($k = 6$). From an enrolled sample population we estimate the mean μ , the covariance Σ , and its inverse Σ^{-1} . A probe x is scored by the squared Mahalanobis distance $D^2 = (x - \mu)^\top \Sigma^{-1} (x - \mu)$, which under the Gaussian assumption is χ^2 with k degrees of freedom. The verdict compares the tail probability against a size $\alpha = 0.001$, and additionally requires at least 2 markers with standardized deviation $|z_i| > 3$ — the coherent, multi-marker signature of gene-therapy-grade trait drift.

23.4.3 DNA sequence identity verification (`sequence_identity.py`)

Identity is also checked at the sequence level over the $\{A, C, G, T\}$ alphabet. A 1500-base reference genome is enrolled, and a probe is scored three ways:

- `hamming_distance` — per-base substitutions;
- `edit_distance` — the Levenshtein edit distance (insertions, deletions, substitutions) via $O(mn)$ dynamic programming;
- `kmer_jaccard` — Jaccard similarity of the probe's k -mer fingerprints ($k = 9$), robust to isolated single-base noise.

A probe is a suspected replacement when it carries a *contiguous* run of at least 12 mismatching bases (`mismatch_clusters`) — a coherent gene-therapy edit rewrites a block of loci at once, whereas sequencing noise produces only scattered single-base errors. It is *matched* only when its k -mer similarity clears 0.9 AND no such block is present.

23.4.4 Ice-palace air operations (`ice_ops.py`)

An operative's core temperature decays toward the ambient environment by Newton's law of cooling, $T(t) = T_e + (T_0 - T_e)e^{-kt}$, giving a closed-form safe exposure $t_{\text{crit}} = \frac{1}{k} \ln \frac{T_0 - T_e}{T_{\text{crit}} - T_e}$. With ambient $T_e = -34$ C, the planner partitions an 1800-second operation into rotating squads of 4 operatives so no squad exceeds the ceiling.

Two further quantities size the resource, and they use *different* references on purpose. Cold strain is $\int_0^E \max(0, T_0 - T(t)) dt$ in degree-seconds, referenced to normothermia ($T_0 = 37$ C) — referencing it to room comfort (20 C) would return exactly zero for every survivable shift, since a shift ends at the 30 C critical core temperature, well above comfort, and the metric could then never discriminate one plan from another. Space heating, by contrast, genuinely is comfort-referenced: it is the conductance times the room-to-ambient deficit times the operation duration.

23.4.5 Cold-water hypothermia risk (`hypothermia.py`)

Water conducts heat far faster than air, so an immersed operative follows the same Newton cooling law with a much larger decay rate, scaled by body mass and insulation. The core temperature then crosses the standard clinical stages: mild hypothermia at 35 C, moderate at 32 C, and severe (unconsciousness) at 28 C. `plan_cold_water_rescue` computes whether a rescue party sweeping 400 m at 2 m/s can reach the operative before the core reaches the severe stage, reporting the margin and the core temperature at reach (immersion in 10 C water, 75 kg operative, insulation factor 0.9).

The reference decay rate `K_REF_PER_S` is a *scenario* constant, not a physiological calibration. It is fixed so that a 75 kg operative in 10 C water reaches a 30 C core in 20 minutes — an initial cooling rate near 24 C/hour, which is roughly an order of magnitude faster than measured immersion cooling in water at this temperature. Every hypothermia number in §03 is therefore internally consistent with that constant and should not be read as a survival-time prediction; see §07.

23.4.6 Ice-structure integrity (`ice_structure.py`)

The ice palace must carry its load and survive thermal gradients. Load capacity follows the standard ice-engineering rule $P = h^2/C$ (P tons, h cm); the thermal gradient stress is $\sigma = E \alpha \Delta T / (1 - \nu)$, and the critical temperature difference that drives ice to its tensile limit is $\Delta T_{\text{crit}} = \sigma_{\text{allow}} (1 - \nu) / (E \alpha)$. At the configured thickness (30 cm), load (3 t), and gradient (15 C, tensile strength 1.5 MPa), the plan reports the safety factor and whether the gradient risks cracking.

23.4.7 Orbital-mirror targeting (`orbital_mirror.py`)

The mirror is placed on a circular orbit (semi-major axis 7200 km, inclination 65 deg) and propagated by its mean anomaly. The ground target (60 N, 15 E) is located in the same inertial frame via Greenwich mean sidereal time. At each instant the mirror normal $\hat{n}$ satisfies the reflection law — it bisects the incoming solar ray $\hat{s}$ and the ground re-point $\hat{r}$: $\hat{n} \propto \hat{s} + \hat{r}$, so the incidence angle $\arccos(\hat{n} \cdot \hat{s})$ is exactly half the Sun-mirror-target angle.

The incoming ray uses the *parallel-ray* approximation: at 1 au the Sun's direction from the mirror is the solar unit vector itself, and the about 7e6 m orbit radius shifts it by well under an arcsecond. Forming that ray by subtracting the satellite position from a unit solar vector would instead yield the nadir direction, and the resulting incidence angle would be wrong by tens of degrees; `tests/test_orbital_mirror.py` pins the incidence angle to the analytic half-angle to keep that failure mode from returning.

When the target direction is anti-parallel to the solar ray the bisector degenerates — no orientation folds a 180-degree turn — and the instant is reported infeasible with zero delivered power rather than raising. Targeting is otherwise feasible when the mirror is

sunlit (outside Earth’s cylindrical shadow) and above the 10-degree horizon. Delivered power is the solar constant times mirror area (10000 m²) times reflectivity (0.9) times the cosine of incidence times a lumped one-way transmission factor (0.85) that stands in for atmospheric attenuation, figure error, and scattering — it is a configured constant, not a modelled quantity, and it scales every reported flux. Flux at the target follows from the beam spread, whose half-angle is the configured 0.02 deg plus the mirror’s own apparent angular radius at range.

23.4.8 Execution and control structure

The six models are not invoked ad hoc. `mission_results.run_all_concepts` reads the `concepts:` block of `manuscript/config.yaml`, runs every model once under seed 7, and returns a frozen `ConceptResults` record. Two consumers read that record and only that record: `mission.py`, which serialises it into the frozen BOND-API `MissionOutcome`, and `manuscript_variables.py`, which formats it into the measured manuscript tokens this paper cites. A number printed here is therefore the same value the mission reported, not a transcription of it.

23.5 Results — ICARUS: measured outcomes, headline numbers, and what they establish

Running the deterministic mission (`uv run python scripts/run_mission.py`) exercises all 6 concepts under seed 7 and reports measured outcomes. Every number below is produced by `src/die_another_day/mission_results.py` — the same entry point the `MissionProvider` uses for its `MissionOutcome` — and injected as a manuscript token, so the paper and the code cannot disagree.

23.5.1 Identity-replacement detection (biometric)

Two probes are scored against the enrolled Gaussian baseline: a positive control (a coherent 4.5-sigma shift on the first 2 markers, the remaining markers left at the population mean, plus small noise) and a negative control (a fresh out-of-sample operative drawn from the same enrolled population). The shift magnitude is 4.5 sigma, deliberately clear of — and not equal to — the 3-sigma per-marker flagging threshold it must cross.

Probe	Mahalanobis D^2	χ^2 threshold	markers flagged	verdict
replaced (positive control)	41.42	22.46	2	replacement
genuine (negative control)	10.97	22.46	0	cleared

The replaced probe scores 41.42 against a threshold of 22.46 at $\alpha = 0.001$ on 6 degrees of freedom, with 2 individually flagged markers — both conditions of the gene-therapy-grade-mahalanobis rule. The genuine draw scores 10.97 ($p = 0.089$) with 0 flagged markers and is cleared. The negative control is deliberately not the profile mean, which would score exactly zero and make clearance true by construction rather than measured.

23.5.2 Identity-replacement detection (sequence)

The 1500-base reference genome is probed with a genuine read carrying 5 scattered single-base errors and with a replaced read carrying an *independent* draw of 5 scattered errors plus a coherent 12-base substitution block. The two reads are matched on the number of scattered errors, not on their positions: the block consumes RNG draws before the scattered positions and removes its own loci from the available pool, so the scattered sets differ. What separates the probes is therefore the block alone, which is exactly the control the contiguous-run rule is being tested against.

Probe	9-mer Jaccard	Hamming	edit distance	longest contiguous block	verdict
genuine	0.9413	5	5	1	matched
replaced	0.9182	17	15	12	replacement suspected

Hamming and edit distance are separate statistics and are reported separately: they coincide for the genuine read but not for the replaced one, where the contiguous block admits a cheaper alignment (17 versus 15).

Fingerprint similarity alone does not separate the two probes — both clear the 0.9 match threshold, at 0.9413 and 0.9182 respectively. The discriminating statistic is the 12-base contiguous mismatch run, which scattered sequencing noise does not produce; this is what the levenshtein-kmer-cluster rule keys on.

identity trait drift — replaced=True

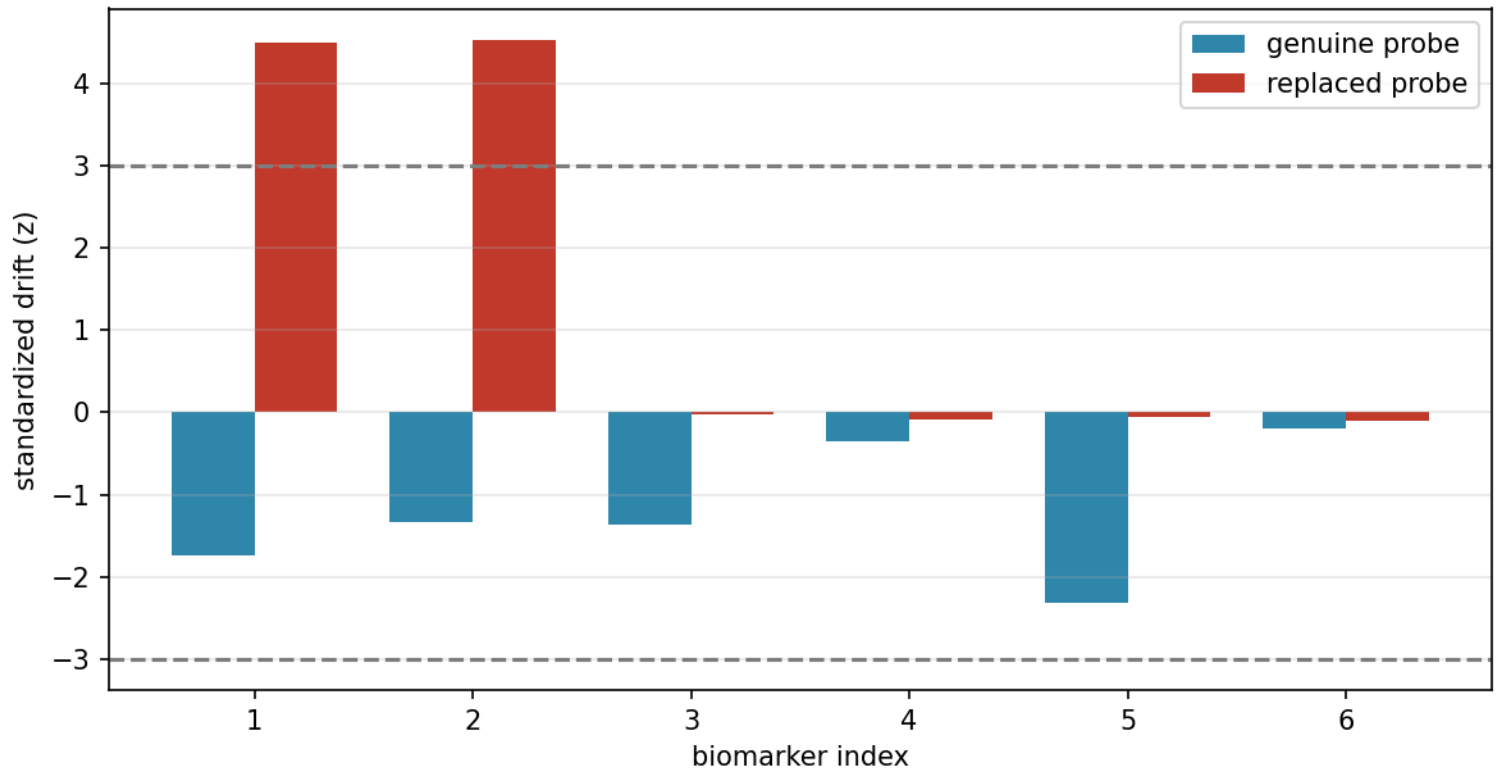


Figure 88: Per-marker standardized trait drift for the two probes that produced the verdicts in the table above — the out-of-sample genuine draw and the 4.5-sigma replaced probe — plotted from the same `ConceptResults` record. The dashed lines mark the ± 3 flagging threshold.

DNA sequence identity verification — mismatch map

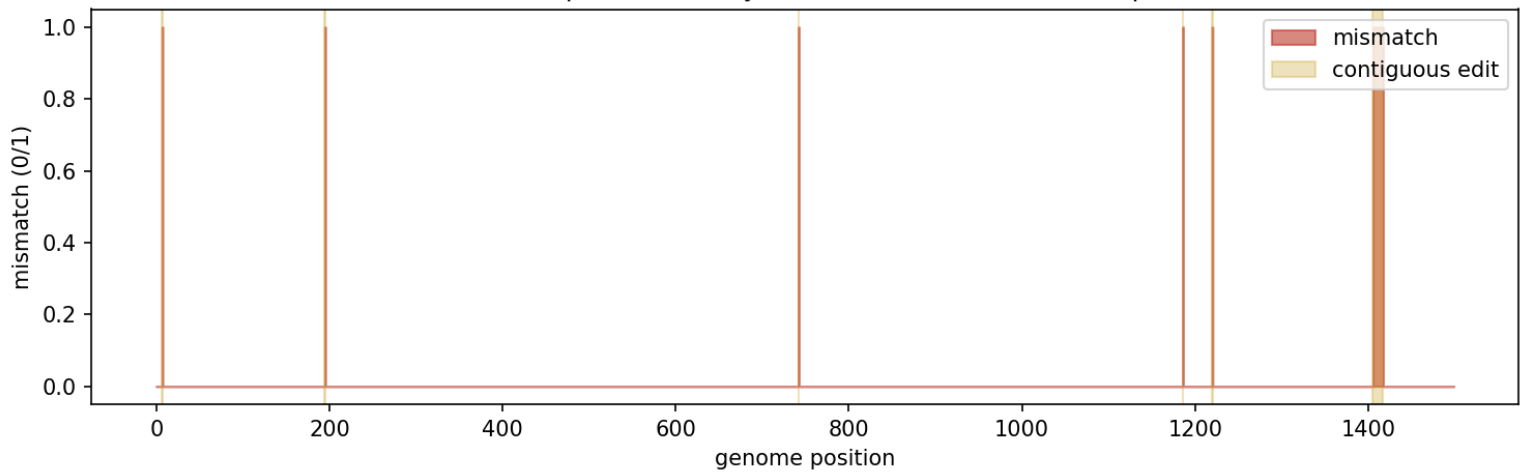


Figure 89: DNA mismatch map for the replaced probe: the contiguous edit cluster is shaded against isolated single-base errors.

23.5.3 Ice-palace operations

From the configured physics (ambient -34 C, core 37 C, critical 30 C, decay 0.8 /hour) the closed-form exposure ceiling is 467.1 s. The 1800-second operation is therefore split across 4 rotating squads drawn from the 4 available operatives, giving a 450.0-second shift — inside the ceiling, so the roster is feasible (yes). Each shift accrues 1545.6 degree-seconds of core depression below normothermia, and holding the interior at comfort against the ambient costs 145.8 MJ over the operation.

23.5.4 Cold-water hypothermia risk

In 10 C water a 75 kg operative reaches the severe (28 C) stage after 1802.1 s. A rescue sweeping 400 m at 2 m/s arrives at 200.0 s — a margin of 1602.1 s. Core temperature at reach is 35.81 C, clinical stage *normal* against the mild/moderate/severe bands at 35, 32, and 28 C.

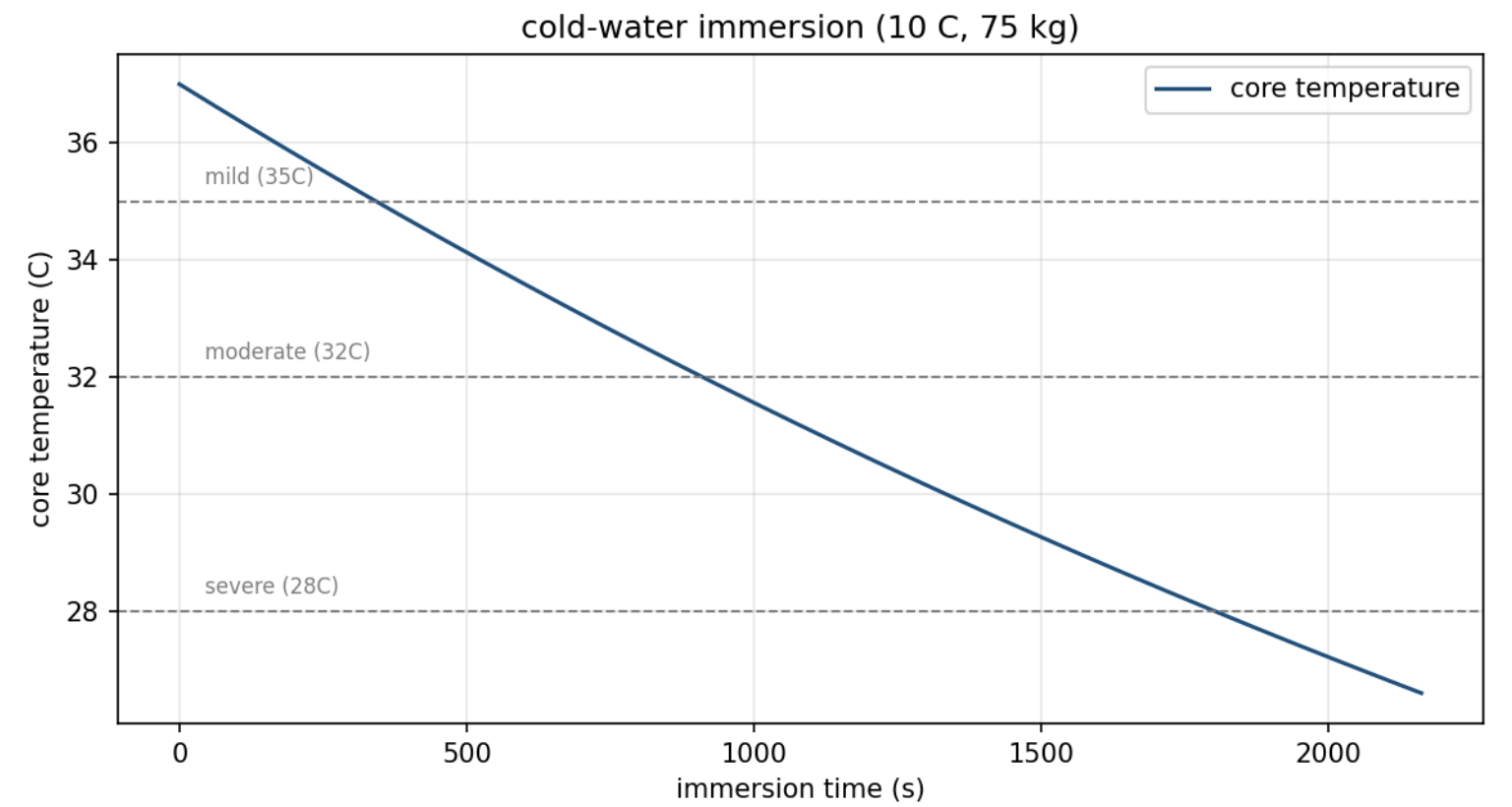


Figure 90: Cold-water immersion core-temperature decay with mild/moderate/severe stage thresholds.

23.5.5 Ice-structure integrity

The palace ice at 30 cm carries 3.52 t against the applied 3 t — a safety factor of 1.17. The load case therefore passes. The thermal case does not: a 15 C gradient drives 10.28 MPa of tensile stress, far past the 1.5 MPa strength, whose critical gradient is only 2.19 C. Cracking risk: **yes**. The structure holds its load and still fails its thermal case — the two checks are independent, and the combined verdict requires both.

23.5.6 Orbital-mirror targeting

Propagating the 7200-km circular orbit at 65 deg inclination over an 3000-second window sampled every 60 s yields 51 instants over the 60 N, 15 E target. Of these, 5 are feasible — mirror sunlit and above the 10-degree horizon — for 300 s of targeting. Peak focused flux is 5.87 W/m^2 at 1680 s after epoch. That delivered flux is small next to the solar constant: a 10000 m^2 mirror at orbital range spreads its beam over a footprint kilometres across, so at this scale the model describes an illumination instrument rather than the film’s weapon.

23.6 Conclusion — ICARUS: findings, verdict, and what the mission establishes

Die Another Day: ICARUS — Special-Agent Mission Software demonstrates that a special-agent mission can be encoded as 6 independent, deterministic computational cores united by a thin protocol adapter. Identity is screened both at the biometric and DNA-sequence level, so a gene-therapy-grade replacement is caught by a coherent marker drift (41.42 against a 22.46 threshold) and by a 12-base contiguous genome edit block. The extreme-environment layer turns a cold-exposure physics law into an executable roster (4 squads under a 467.1-second ceiling), sizes a cold-water rescue against the hypothermia timeline (1602.1 s of margin), and checks

ICARUS orbital-mirror pass over target

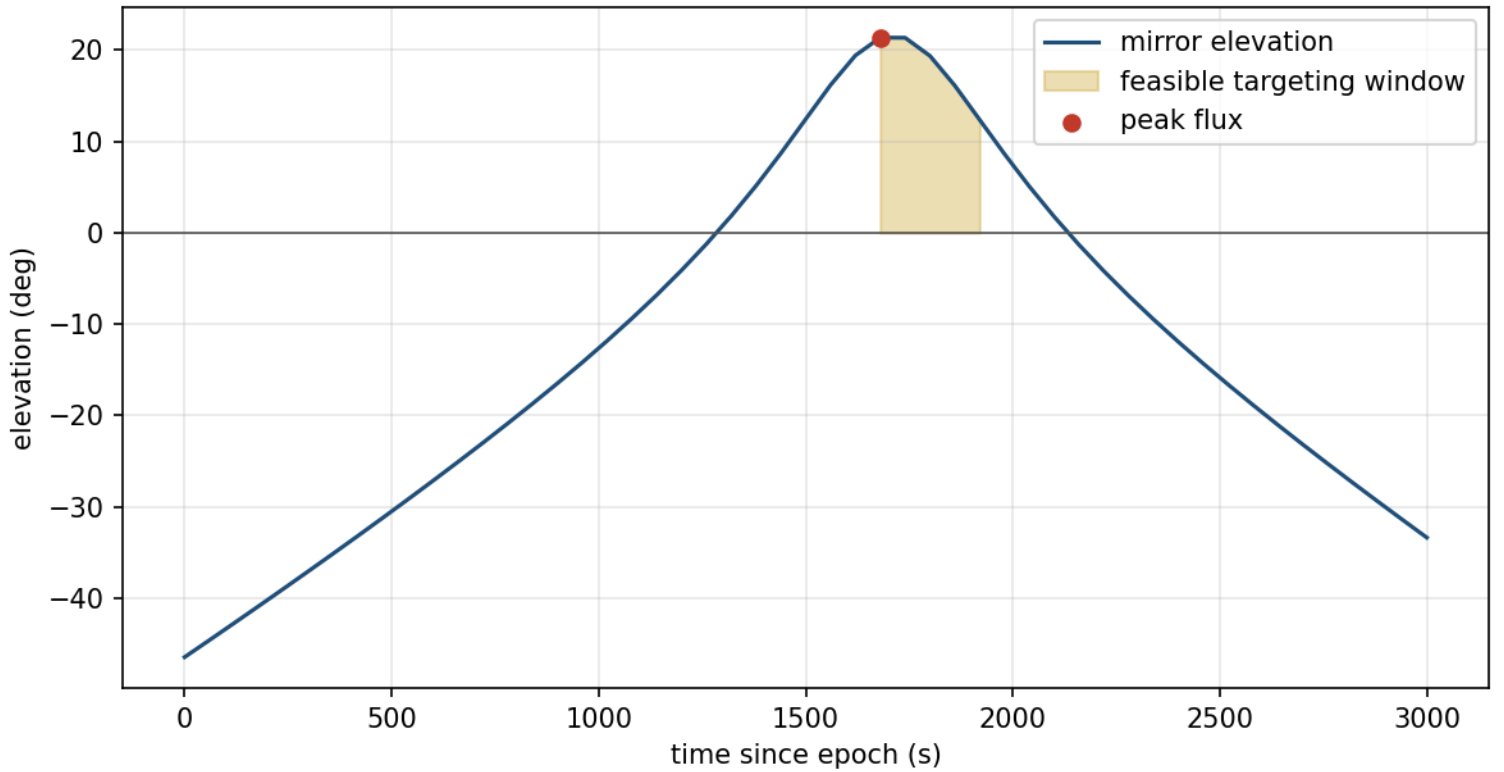


Figure 91: ICARUS orbital-mirror elevation vs. time over the ground target. The shaded band marks the feasible targeting window and the red point the peak-flux instant.

the ice palace’s structural integrity under load and thermal stress. The orbital-mirror module solves the reflection-and-line-of-sight geometry of targeting a ground point from orbit, yielding 300 s of feasible pointing in the configured window.

Two results are worth stating plainly because they are negative. The ice palace passes its load case and still fails its thermal case, so a single “structurally safe” verdict would have hidden a real failure mode. And the delivered flux of 5.87 W/m^2 shows the orbital mirror at this aperture is an illumination instrument, not the film’s weapon — the model reports what the geometry actually permits rather than what the plot requires.

The package meets the BOND guardrails: pure domain cores importable without the protocol, a frozen `MissionProvider` adapter, a zero-mock test suite held to a line-and-branch coverage floor by `pytest --cov-fail-under`, and a manuscript whose every metric — configured input and measured output alike — is injected from the code through the 6 concept modules.

23.7 Experimental Setup — ICARUS: canonical scenarios, parameters, and configuration

All mission parameters are declared once in `manuscript/config.yaml` under the `concepts:` block and consumed by the code via `src/die_another_day/mission_results.py`. They are reproduced here as mission tokens so the manuscript cannot drift from the configuration. Package identity: `die_another_day` (codename ICARUS), deterministic seed 7.

The seed is the single module constant `mission_results.MISSION_SEED`, not a configuration key: `config.yaml` carries no seed of its own, so no consumer can be reading a different one from the run this paper reports.

The list below is complete by construction rather than by inspection. Every `concepts.<section>.<key>` in `manuscript/config.yaml` is registered in `manuscript_variables.CONFIG_INPUT_TOKENS`, and `test_every_configured_input_is_registered_and_cited` fails if the registry and the file disagree, or if a registered token is not cited in this section.

23.7.1 Identity detection

- Biomarker markers: 6
- Test size α : 0.001
- Per-marker z threshold: 3
- Minimum flagged markers: 2

23.7.2 DNA sequence identity

- Genome length: 1500 bases

- k-mer size: 9
- Match Jaccard threshold: 0.9
- Edit-cluster minimum: 12 bases
- Scattered single-base errors per read: 5

23.7.3 Ice-palace operations

- Ambient temperature: -34 C
- Body/core temperature: 37 C
- Critical (minimum safe) core temperature: 30 C
- Cooling decay rate: 0.8 /hour
- Operatives available: 4
- Operation duration: 1800 s
- Envelope thermal conductance: 1500 W/K

23.7.4 Cold-water hypothermia risk

- Water temperature: 10 C
- Operative mass: 75 kg
- Insulation factor: 0.9
- Rescue distance: 400 m
- Sweep speed: 2 m/s

23.7.5 Ice-structure integrity

- Ice thickness: 30 cm
- Applied load: 3 t
- Thermal gradient: 15 C
- Tensile strength: 1.5 MPa

23.7.6 Orbital-mirror targeting

- Orbit semi-major axis: 7200 km
- Orbit inclination: 65 deg
- Right ascension of the ascending node: 0 deg
- Mean anomaly at epoch: 320 deg
- Epoch (Julian date): 2461300.5
- Ground target: 60 N, 15 E
- Pass window: 3000 s sampled every 60 s (51 instants)
- Mirror area: 10000 m²
- Mirror reflectivity: 0.9
- Beam half-angle: 0.02 deg
- Lumped one-way transmission factor: 0.85
- Minimum targeting elevation: 10 deg

The epoch, RAAN, and mean anomaly are not incidental: together with the inclination they select *which* pass the 3000-second window lands on, and hence every number in §03’s orbital-mirror row. They were chosen so that a pass occurs over the target inside the window — a hand-tuned geometry, stated here rather than buried in a default.

23.7.7 Software environment

- Python: 3.14.6
- Concept modules (6): `identity_detection`, `sequence_identity`, `ice_ops`, `hypothermia`, `ice_structure`, `orbital_mirror`
- Package version: 0.1.0 (`manuscript/config.yaml`, `paper.version`)

There is deliberately no generation timestamp in this list. An earlier draft hydrated one from the wall clock, which made every regeneration of the manuscript differ in bytes while §00 and §06 claimed determinism; §06 states the property that replaced it, and a test enforces it.

23.7.8 What is configured versus what is measured

The lists above are *inputs*: every one is a literal in `manuscript/config.yaml`. Everything in §03 is an *output*: it comes from running the models on these inputs. The two families are formatted by different code paths (`generate_variables` for inputs, `measured_variables` for outputs) and a test asserts that each measured token equals the value the concept runner produced.

That test binds provenance, not independence: it proves a §03 number came out of the runner, not that the runner had any freedom in producing it. Three tokens are worth naming because they are *structurally determined* by inputs and would look like findings otherwise:

- `SEQ_GENUINE_HAMMING` equals `scattered_errors` whenever the mutation generator behaves — it applies exactly that many substitutions at distinct loci, each guaranteed to change the base. Read it as a check on the generator, not as a measurement of sequencing noise.
- `SEQ_LONGEST_BLOCK` is bounded below by `cluster_min` for the same reason; it can only exceed it if a scattered error happens to abut the block. The informative comparison is against `SEQ_GENUINE_LONGEST_RUN`, which is free to be anything and is what makes the separation a result.
- `ID_REPLACED_SHIFT_SIGMA` is a construction parameter of the positive control (`REPLACEMENT_SHIFT_FACTOR` times the configured threshold), reported so §03 states the probe’s actual magnitude instead of reusing the threshold token.

Everything else in §03 — every score, verdict, time, flux, and safety factor — is free to move with the inputs, and `test_measured_tokens_track_the_configuration` demonstrates that it does.

23.8 Reproducibility — ICARUS: verification gates, deterministic regeneration, and artifacts

The package is deterministic by construction:

- **Fixed seed:** all synthetic data (the enrolled biometric population and the reference genome) is drawn with seed 7, so the mission outcome is reproducible across runs (`src/die_another_day/mission_results.py`). The seed is a single module constant; `manuscript/config.yaml` deliberately carries no seed key, and a test fails if one reappears, so no consumer can quietly run a different one. A test re-runs the concept runner and compares the whole `ConceptResults` record; a second test changes the seed and asserts the stochastic outputs move while the pure-physics ones do not, so the seed is demonstrably load-bearing rather than decorative.
- **No wall-clock in artifacts:** provenance records a `wall_time_s` of 0.0, and nothing written under `output/` is read from a clock. This was not always true: a `GENERATION_TIMESTAMP` token was hydrated from `datetime.now` and rendered into §05, so every regeneration produced different bytes while this section claimed determinism. The token is gone.
- **Byte-identical regeneration:** re-running `scripts/z_generate_manuscript_variables.py` overwrites `output/data/manuscript_variables.json` and `output/manuscript/*.md` with bytes identical to the previous run’s. `test_regeneration_is_byte_identical` runs the script twice, more than one second apart, and compares every produced file byte-for-byte. The gap is deliberate: a second-resolution timestamp only differs across runs that straddle a second boundary, so without it the gate’s ability to see such a stamp would depend on how long a subprocess happens to take. The claim covers exactly the files that script writes; figures are covered by the separate gate below. The one environment-derived token, 3.14.6, is an interpreter identity rather than a clock and is constant across runs on one interpreter — on a different Python it changes, so the property is stated per-interpreter, not across machines.
- **Byte-reproducible figures:** the figure saver suppresses matplotlib’s PNG Date metadata chunk, and `tests/test_figures.py` renders each figure twice and compares the bytes.
- **Figures depict the reported run:** `scripts/generate_figures.py` plots straight off the `ConceptResults` record rather than rebuilding its own probes, and `test_published_figures_depict_the_reported_run` compares the published PNG bytes against an independent render of the runner’s arrays. Each renderer additionally has an input-sensitivity test — change one argument, require different bytes — so a plot cannot silently ignore what it is given. Both gates exist because an earlier identity-drift figure showed a different experiment than the table beneath it.
- **Configured-input completeness:** every `concepts:` key is declared once in `manuscript_variables.CONFIG_INPUT_TOKENS`, and `test_every_configured_input_is_registered_and_cited` fails if the registry and `config.yaml` disagree or if a registered input is missing from §05.
- **Zero mocks:** `tests/` uses real computation and real data only — no `unittest.mock`, `MagicMock`, or `mock.patch` anywhere in the suite.
- **Coverage gate:** `uv run pytest tests/ --cov=src --cov-fail-under=90` fails the run below the line-and-branch floor. The measured figure is not quoted here; run the command to obtain the current number.
- **Type and style gates:** `uv run ruff check src/ scripts/`, `uv run ruff format --check src/ scripts/`, and `uv run mypy src/` are clean.
- **Token hygiene:** two live gates in `tests/test_manuscript_variables.py` — every mission token cited in `manuscript/[0-9]*.md` must be produced by `generate_variables`, and every token `generate_variables` produces must be cited by some section (no orphan tokens). Strict generation additionally fails on any unresolved token left in a rendered section.
- **Thin-orchestrator gate:** `tests/test_scripts_smoke.py` parses every script in `scripts/` and fails if one grows a second top-level function, a class, or a direct `numpy/scipy/matplotlib` import — the mechanical form of the layer contract.

Python version: 3.14.6.

23.9 Scope and Related Work — ICARUS: boundaries, positioning, and relationship to the literature

Scope. This package models three decision problems relevant to a special-agent operation — identity-replacement screening, cold-environment exposure planning, and orbital-mirror targeting. Each model is deliberately self-contained and deterministic; it is not a general biometrics, heat-transfer, or astrodynamics library. Assumptions are stated explicitly in §02 (Gaussian markers, Newton cooling, circular Keplerian orbit, low-precision solar ephemeris).

Related work. The Mahalanobis-distance screen (Mahalanobis, 1936) is a classic multivariate outlier tool; here it is specialized to a coherent-marker replacement hypothesis with a chi-square calibration. Sequence comparison uses Levenshtein edit distance (Levenshtein, 1966) together with k-mer set similarity, the fingerprinting idea that underpins alignment-free genome comparison (Ondov et al., 2016). Newton’s law of cooling (Newton, 1701) underlies standard cold-stress exposure limits. The immersion module borrows the *clinical staging* used in the cold-water survival literature (Tipton, 1989; Golden and Tipton, 2002) — the mild/moderate/severe core-temperature bands — but **not** its cooling rates: `K_REF_PER_S` is a scenario constant, not a calibration against that literature (see Limitations). Ice load capacity uses the published bearing-capacity rule for floating ice covers (Gold, 1971; Sodhi, 1995) with mechanical and thermal constants from ice physics (Hobbs, 1974). Orbital-targeting geometry follows standard two-body propagation (Battin, 1999; Vallado, 2013) and the mirror reflection law used by space-solar-power and orbital-reflector studies (Glaser, 1968). The contribution here is not any single model but their *integration* into one deterministic mission provider under the frozen BOND-API contract, with every reported number injected from the run rather than transcribed.

Limitations. These are properties of the models, not of the implementation, and each is visible in the code:

- Biomarker distribution is assumed Gaussian and markers are drawn independently; heavy-tailed or correlated traits would need robust covariance estimation, and the chi-square calibration would shift accordingly.
- Sequence verification requires equal-length reads. Edit distance tolerates indels, but the contiguous-block rule and the Hamming statistic are position-aligned, so an unaligned read must be aligned first.
- The orbital pass is a single circular orbit with a low-precision solar ephemeris and a cylindrical (not conical) shadow model; it answers a feasibility question, not a constellation-design or pointing-error one.
- Beam spread is a simple angular model: the half-angle is a configured constant plus the mirror’s apparent angular radius. Diffraction, figure error, and atmospheric scattering are folded into one lumped transmission factor rather than modelled.
- Cold-exposure planning uses a single Newton decay rate for all operatives (no per-suit or per-individual variation), and the rotation planner assumes squads can hand over instantaneously.
- **The immersion cooling rate is not physiologically calibrated.** `hypothermia.K_REF_PER_S` is fixed so a 75 kg operative in 10 C water reaches a 30 C core in 20 minutes: an initial cooling rate of about 24 C/hour, which is roughly an order of magnitude faster than measured immersion cooling at that water temperature. It is a film-scenario constant chosen to put the rescue on a dramatic clock. Consequently §03’s immersion times (`HYP_TIME_TO_SEVERE_S`, `HYP_CORE_AT_REACH_C`, `HYP_MARGIN_S`) are correct *for this model* and are not survival-time predictions; the module demonstrates the rescue-versus-timeline decision structure, not the timeline itself. Only the clinical staging thresholds (35 / 32 / 28 C) are taken from the literature. `test_hypothermia.py` pins the constant to its stated 20-minute definition so the number and the sentence cannot drift apart.
- The ice-structure model is a static bearing-capacity plus thermal-stress check; it does not model creep, crack propagation, or repeated loading.

23.10 Sources — ICARUS: bibliography

Mahalanobis [1936]; Levenshtein [1966]; Gold [1971]; Sodhi [1995]; Hobbs [1974]; Tipton [1989]; Golden and Tipton [2002]; Newton [1701]; Battin [1999]; Glaser [1968]; Ondov et al. [2016]; Vallado [2013a]

24 Casino Royale (2006) — LE CHIFFRE

film package · package codename LE CHIFFRE. Mission **LE CHIFFRE**: counter-play Le Chiffre at the high-stakes felt (pot odds + equity against his range); solve the river toy game by CFR to a near-Nash profile (game theory); price tournament survival with the Independent Chip Model (ICM); trace the chip flows funding the game and flag laundering (behavioural + network-level); plan an extraction route across the parkour city (ground/roof); triage the spiked martini and identify the digitalis source; infer Le Chiffre’s archetype posterior from his observed actions.

24.1 Concepts — LE CHIFFRE: domain and operational focus

game theory, financial crime

24.2 Abstract — LE CHIFFRE: mission summary

Casino Royale: LE CHIFFRE — Special-Agent Mission Software is the special-agent mission software for the film *Casino Royale* (2006), mission codename **LE CHIFFRE**. It packages 8 deterministic, infrastructure-free domain modules — a Texas hold’em game-theory engine plus a counterfactual-regret (CFR) solver and the tournament Independent Chip Model, a Bayesian opponent-archetype inference model, a chip-movement money-laundering detector plus network-level financial-crime analytics, a parkour mobility-graph router, and a digitalis (cardiac-glycoside) poison triage — into a single **MissionProvider** bound to the frozen BOND-API protocol. The cores are exercised on one canonical scenario (seed-stable, no wall-clock in provenance). Against the loose range Le Chiffre’s observed loose-aggressive tendencies select, the engine prices the hero hand on the final board at 45.62% equity against a pot-odds break-even of 16.67%, a margin of 28.96% that recommends **raise**. CFR on the river toy game converges to an average regret of 0.001 (polar equilibrium, game value 0.0974), ICM prices the freezeout bubble at factor 1.411, and the Bayesian model reads Le Chiffre as **loose-aggressive** (aggressive posterior 0.986). The chip-flow scanner ingests 41 movements across 8 players and flags the planted smurfer with a composite score of 2 (1 alert(s) total), confirmed by the network detector at risk 0.98. On the 6×6 quarter the speed-weighted solve short-cuts via the rooftops (cost 36.53, against 103.39 for the ground route under that same weighting) while the risk-weighted solve stays on the street (cost 15.31, against 17.95 for the rooftop route under *that* weighting) — each optimal only under its own objective. The tox triage isolates the spiked martini 3 of 5 samples at 4.5 ng/mL digoxin (severe toxicity, posterior 0.95). Every result shown here, and in the sections that follow, is computed live from the engines at manuscript-generation time — none is hand-authored.

24.3 Introduction — LE CHIFFRE: mission framing, the operational problem, and how to read this chapter

Casino Royale (2006) follows the special agent against the banker-cipher **Le Chiffre** — a name that literally means “the cipher”. The story’s set pieces map onto a family of analytical problems that a real mission system would have to solve, and each is implemented here as a serious, tested module rather than a stunt. Four are the film’s literal set pieces:

1. **The felt.** The climactic hand is a study in game theory: a hero hand on a dry board against an adversary whose range the hero can model, priced against the chips in the middle. Module: `poker_engine.py`.
2. **The capital.** Laundered money funds the stake; it travels as physical casino chips in structurally tell-tale patterns. Module: `laundering_detect.py`.
3. **The terrain.** The chase crosses a parkour-friendly city — ground streets, leapable rooftop gaps, climbable walls, droppable falls — which is exactly a weighted mobility graph. Module: `urban_routes.py`.
4. **The martini.** Bond’s drink is spiked with digitalis, the cardiac glycoside with a notoriously narrow therapeutic index. Module: `poison_detect.py`.

Four more answer the question each set piece raises once it is taken seriously:

5. **What is the *equilibrium*, not just the price?** Pot odds decide one call; a solver decides a strategy. Counterfactual regret minimization on the river toy game recovers the polar equilibrium the finale depends on. Module: `poker_cfr.py`.
6. **What is a chip worth on the bubble?** In a freezeout, chips are not money: the Independent Chip Model converts stacks to prize equity and prices survival. Module: `icm.py`.
7. **What does the adversary’s history actually license?** A raise count is not a read; a Dirichlet-multinomial posterior over archetypes is. Module: `opponent_bayes.py`.
8. **Is the laundering visible without a suspicious transfer?** The structural layer — concentration, centrality, communities, value conservation — finds the washer from the shape of the flow alone. Module: `laundering_network.py`.

Together these form the pure domain core (8 concept modules: `poker_engine`, `poker_cfr`, `icm`, `opponent_bayes`, `laundering_detect`, `laundering_network`, `urban_routes`, `poison_detect`), each deliberately free of any framework, `infrastructure.* import`, or `bond-api` dependency, so they stay importable in isolation. A thin adapter, `mission.py`, implements the frozen BOND-API **MissionProvider** (codename THE PROTOCOL) and folds their output into a single deterministic mission outcome, alongside a catalogue of 11 published gadgets (`gadgets.py`) that other suite packages may call directly.

24.3.1 Principles

- **Deterministic by construction.** Every RNG draw flows through a caller-supplied seeded `class:random.Random`; provenance carries the seed and never a wall-clock reading. Three of the eight engines (CFR, ICM, and the Bayesian opponent model) are exactly enumerated and carry no RNG at all.
- **Real computation, zero mocks.** The numbers in this manuscript are engine output (exact enumeration, Monte-Carlo simulation, graph search, tox triage), verified by a $\geq 90\%$ line-and-branch test suite.
- **One scenario, everywhere.** A single canonical scenario (see `src/casino_royale_2006/scenarios.py`) feeds the mission, the manuscript tokens, and the CLI, so a claim computed once is the claim shown everywhere.

24.3.2 Reader’s guide

The methodology section defines all 8 algorithms; the results section reports their outputs on the canonical scenario and shows the figures; the setup section documents configuration and the runtime environment; the reproducibility section covers determinism, artifact layout, and the verification gates; and the scope section states, per module, what is deliberately out of scope.

24.4 Methodology — LE CHIFFRE: the analytical models and algorithms that drive the mission

This section defines all 8 algorithms. All are pure Python; none imports a framework, `infrastructure.*`, or `bond-api`. Full signatures live in `src/casino_royale_2006/`: `poker_engine.py`, `poker_cfr.py`, `icm.py`, `opponent_bayes.py`, `laundering_detect.py`, `laundering_network.py`, `urban_routes.py`, `poison_detect.py`. The canonical inputs every one of them is run on live in `scenarios.py`; the published callables are catalogued in `gadgets.py`.

24.4.1 Poker game-theory engine (`poker_engine.py`)

Hand strength. A card is an integer in $0..51$ ($\text{rank} = \text{card} // 4 + 2$, $\text{suit} = \text{card} \% 4$). `evaluate_5` returns (`category`, `tiebreak_ranks`) for the nine categories (straight flush down to high card), and `best_hand` takes the best five-card subset of a seven-card hold’em hand by enumerating all five-card combinations. This is exact — the tiebreak tuple hashes the hand category and ordered kickers so two hands compare lexicographically.

Equity. Two estimators are provided. `exact_equity_turn_river` enumerates all 990 two-card runouts after a flop for a heads-up match — exact, no RNG. `monte_carlo_equity` simulates N random runouts for an arbitrary number of players using a seeded RNG, returning per-player (`win`, `tie`) fractions.

Pot odds. Given a pot and the amount to call, the break-even equity is $\text{to_call} / (\text{pot} + \text{to_call})$, and the expected value of calling is $\text{equity} \cdot (\text{pot} + \text{to_call}) - \text{to_call}$. `recommend_action` emits `fold`, `call`, or `raise` by comparing the equity margin ($\text{equity} - \text{break_even}$) to thresholds. This is precisely the “price the pot is laying me” decision of the final hand.

Opponent modelling. An `OpponentModel` records observed `raise/bet/ call/fold` actions and derives VPIP and an aggression factor ($(\text{raises} + \text{bets}) / \text{calls}$). `archetype` maps those two statistics to one of four compound labels (`tight/loose` $\times$ `passive/aggressive`); `range_key_for_archetype` then maps that label onto one of the two `RANGE_ARCHETYPES` starting-hand ranges — `tight` and `loose`, hand-enumerated as starting-hand *classes* (AA, AKs, T9o, ...) rather than the full 169-class hold’em space. On the canonical scenario Le Chiffre’s observed history reads **loose-aggressive**, selecting the **loose** range of 30 classes, from which 24 concrete pairs are drawn. `equity_vs_range` then averages the hero’s Monte-Carlo equity over those pairs — the number the agent compares to the pot-odds price. Range fidelity is exact: a drawn hand is guaranteed to match its class, so an offsuit class (e.g. AKo) is always sampled with two different suits and never silently treated as its suited sibling, which would tilt the averaged equity toward suited runouts. The same label also sets the raise threshold, via `recommended_raise_margin` (10% here), which is passed to `recommend_action` as `raise_margin`.

24.4.2 Chip-movement laundering detection (`laundering_detect.py`)

Chip flows are a directed graph from a signed `ChipTransaction` list (`source`, `target`, `amount`, `tick`, `kind` $\in \{\text{cash_in}, \text{cash_out}, \text{transfer}\}$), with the casino cage as a reserved node. Four typologies are computed deterministically:

- **Structuring:** count of transfers sized in $[\text{threshold}/2, \text{threshold})$ (the smurf band).
- **Round-tripping:** $A \rightarrow B \rightarrow A$ rings completing within a tick window.
- **Velocity:** the maximum number of transactions a node participates in within a tick window (both directions — money washing *through* a node is the signal).
- **Chip parking:** a player who buys in, never transfers, and cashes out about their full buy-in (value moved with zero game action).

An unweighted Brandes-style **betweenness** centrality identifies *collection hubs* — nodes many smurf paths funnel through. `detect` accumulates a capped composite score per player and emits an **Alert** (with supporting **evidence**) once it clears a threshold.

24.4.3 Parkour routing (`urban_routes.py`)

A `MobilityGraph` of `UrbanNodes` (`x/y/height/kind`) joined by directed `UrbanEdges` (`distance`, `difficulty`, `risk`, `move` $\in \{\text{run}, \text{leap}, \text{climb}, \text{drop}\}$). A `ParkourProfile` encodes what the agent can physically execute (max leap length, climb rise, drop fall,

and global difficulty/risk caps); `MobilityGraph.feasible` decides whether an edge is executable. `shortest_path` runs **Dijkstra over only the feasible edges**, minimizing a weighted cost $w_d \cdot \text{distance} + w_f \cdot \text{difficulty} + w_r \cdot \text{risk}$, with a monotonic counter breaking cost ties so identical-cost routes are reproducible. `brute_force_path` exhaustively enumerates every simple path on small graphs and serves as ground truth the Dijkstra output is verified against.

A route's reported cost is only meaningful *under the weighting that produced it*: a speed-weighted and a risk-weighted solve return numbers in different units, so their totals are not comparable. `MobilityGraph.path_cost` re-prices an existing route under any weighting, and the results section uses it to state both routes under both objectives rather than juxtaposing two incommensurable totals.

24.4.4 Digitalis poison triage (`poison_detect.py`)

A `ToxPanel` records serum digoxin (ng/mL), heart rate, serum potassium, PR interval, and reported symptoms. Reference ranges — therapeutic 0.5–2.0 ng/mL, toxicity above 2.0, severe above 3.5; bradycardia < 60 bpm; hyperkalemia > 5.5 mmol/L; prolonged PR > 200 ms — are declared as module constants (`poison_detect.py`). `toxicity_score` combines a concentration band with cardiac feature boosts and a symptom penalty; `toxicity_grade` thresholds it into `none/mild/moderate/severe`. `posterior_toxicity` multiplies a prior into a Bayesian posterior using documented likelihood ratios per present sign, and `identify_poisoned_sample` picks the spiked glass from a tray by highest posterior (tie-broken deterministically). `antidote_plan` returns the intervention list (digoxin-specific Fab fragments for severe toxicity, atropine for marked bradycardia, potassium management for hyperkalemia).

24.4.5 Counterfactual regret minimization (`poker_cfr.py`)

The deep game-theory layer. CFR (Zinkevich et al., 2007) finds an approximate Nash equilibrium of an imperfect-information game by iteratively reducing counterfactual regret at each information set. This package solves the classic *river toy game* — heads-up, antes of 1, out-of-position player P1 checks or bets 1, in-position P2 folds or calls, with `CFR_STRENGTHS` discrete hand strengths drawn uniformly — by fully enumerating the game (no RNG). Regret matching turns positive regrets into mixed strategies; the reported average strategy and `game_value` are the equilibrium shape the finale depends on.

24.4.6 Independent Chip Model (`icm.py`)

The tournament-economy layer. ICM (Chen & Ankenman, 2006) converts chip stacks into prize equity under the random-tournament model: `exact_icm_equity` uses the recursive $E_i = \sum_j (s_j/S) \cdot E_i(S_j, P[1:])$ for small fields and `monte_carlo_icm` simulates finish order with a seeded RNG for larger ones. The *bubble factor* — prize equity lost if the stack is risked and lost, over prize equity gained by doubling up — exceeds 1 on the bubble, and `icm_call_ev / icm_decision` decide a tournament call in prize-equity (not chip) terms.

24.4.7 Network-level financial crime (`laundering_network.py`)

The structural-crime layer. Viewing chip flows as a directed graph, four real measures run deterministically: **HHI concentration** over a player's counterparty flow shares, **PageRank** (Page et al., 1999) ranking the collection account that centrally-placed senders feed, **label propagation** (Raghavan, Albert & Kumara, 2007) detecting deterministic trading communities, and a **conservation imbalance** score. The composite `network_risk_report` multiplies value-neutrality by turnover share — a money-washer (chips in $\approx$ chips out, no game losses) churning large volume is the signature — producing an independent laundering detector from the typology scanner.

24.4.8 Bayesian opponent archetype inference (`opponent_bayes.py`)

The read-the-adversary layer. Each of the four archetypes owns a Dirichlet prior over actions; the Dirichlet-multinomial likelihood updates a posterior $P(\text{archetype} \mid \text{history})$ in exact log space. `map_archetype` is the MAP estimate, `predictive` is the model-averaged next-action probability, and `expected_aggression` is the Bayesian expectation of the raise/bet rate — a proper upgrade over the frequentist `OpponentModel` in `poker_engine.py`.

24.4.9 Mission adapter and gadget catalogue (`mission.py`, `gadgets.py`)

The engines above are joined by exactly one adapter. `mission.py` implements the frozen BOND-API `MissionProvider` — `brief` → `recon` → `plan` → `execute` → `debrief` — declaring `film = "casino_royale_2006"` so the orchestrator can discover it by slug. `execute` calls `scenarios.full_snapshot(SEED)` once, folds each engine's real output into `MissionOutcome.results` under a per-concept key, and attaches a `Provenance` record carrying the package version, the seed, and a SHA-256 `input_hash` over the canonical scenario constants — never a wall-clock reading, so two runs are byte-identical.

The published callables are declared separately, in the pure `gadgets.py` catalogue: 11 entries (`poker_equity_vs_range`, `poker_cfr_solve`, `icm_exact_equity`, `icm_bubble_factor`, `chip_flow_detect`, `network_laundering_risk`, `parkour_shortest_path`, `parkour_build_city_quarter`, `parkour_profile`, `poison_identify_sample`, `bayesian_opponent`). Keeping the catalogue out of the adapter means it can be counted and exercised without the protocol dependency present, which is what lets this manuscript state its size as a token rather than a hand-written number. `mission.register_gadgets` binds that mapping to the protocol's

GadgetRegistry under the film slug; the public shape consumed by bond-cli, the coordinator, and the orchestrator is unchanged by the split.

24.5 Results — LE CHIFFRE: measured outcomes, headline numbers, and what they establish

All results are computed live from the engines on the canonical scenario in `src/casino_royale_2006/scenarios.py` (fixed seed, deterministic RNG). Every number below is a TOKEN placeholder, injected at manuscript time.

24.5.1 The felt: equity, price, and decision

On the board **As 7h 2d**, the hero holds **KhKc** against Le Chiffre’s **loose** range — the range his observed **loose-aggressive** tendencies select, not a hand-picked one. The engine prices a Monte-Carlo equity of **45.62%** against a pot-odds break-even of **16.67%** (pot 5,000,000, to call 1,000,000). The resulting margin, **28.96%**, clears the 10% raise threshold that same archetype implies, so the model recommends **raise**: the probability of winning exceeds the price the pot is laying, and by enough to raise for value rather than merely call. The poker figure visualizes the equity-vs-price comparison.

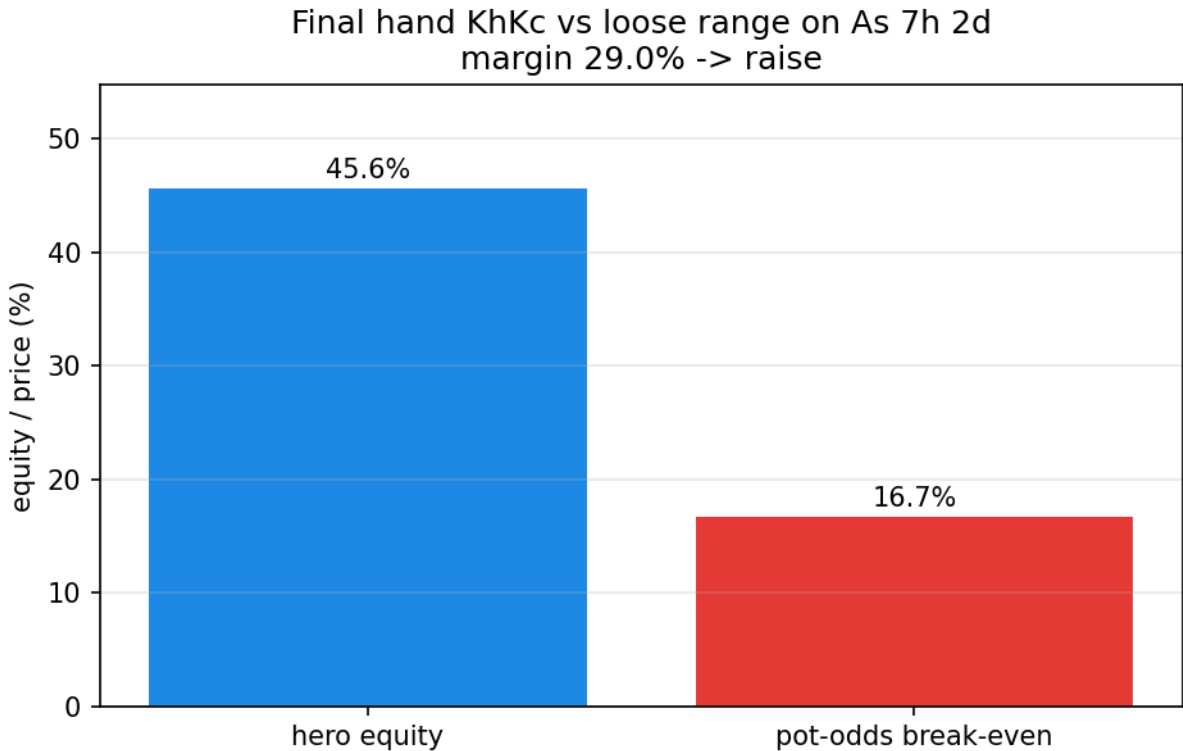


Figure 92: Poker decision: hero equity vs. pot-odds break-even with the resulting action.

24.5.2 The capital: chip-flow laundering

The scanner ingests **41** chip movements across **8** players, including the one planted smurfer. It raises **1** alert(s); the planted launderer is caught (**yes**) with a composite score of **2**, accumulated from structuring + velocity dimensions rather than any single transfer — the structural, not transactional, signature of the laundered stake.

24.5.3 The terrain: parkour extraction route

On the 6×6 city quarter (**72** nodes, **312** directed edges) the planner is run twice, under two different objective functions. The *fastest* (speed-weighted) solve short-cuts across the rooftops (uses rooftops: **yes**); the *safest* (risk-weighted) solve stays on the ground.

Their headline costs — **36.53** and **15.31** — are *not* comparable: each is measured under its own weighting, so the smaller number does not mean the cheaper route. Priced under one objective at a time, the ordering is unambiguous. Under the speed weighting the rooftop route costs **36.53** against the ground route’s **103.39**; under the risk weighting the ground route costs **15.31** against the rooftop route’s **17.95**. Each route is optimal only under its own objective, which is the film’s trade-off stated exactly: the roof is faster, the street is safer, and no single number ranks them.

24.5.4 The martini: digitalis triage

From a tray of **5** samples, the triage isolates the spiked glass at **3**, reading **4.5** ng/mL digoxin, heart rate 42.4 bpm, potassium 6.62 mmol/L — a **severe** toxicity with a Bayesian posterior of **0.95**. The digitalis figure shows the monogram-style toxicity curve used

to place such a panel.

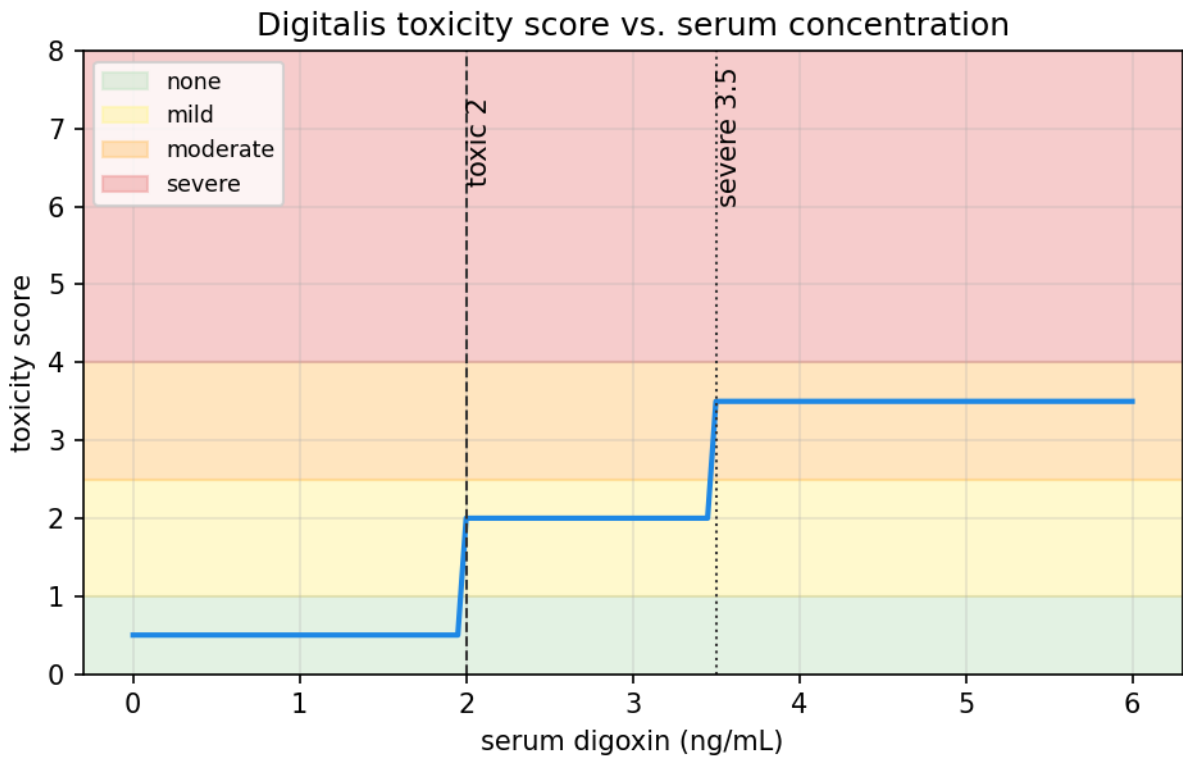


Figure 93: Digitalis toxicity score vs. serum digoxin concentration with the severity grade bands shaded.

24.5.5 The solve: CFR river-toy equilibrium

CFR on the toy game converges to an average regret of **0.001** after **300** iterations, giving a game value of **0.0974**. The equilibrium is *polar*, the classic shape behind modern solver-based play: the out-of-position player bets the very top of his range for value (**0.9983**) and bluffs near the very bottom (**0.9831**), while the in-position caller calls the middle of his range (**0.7805**). The cfr figure shows regret decaying toward zero across the solve.

24.5.6 The freezeout: Independent Chip Model

On the 4-player, 3-paid bubble, exact ICM prices the short stack at a tournament-equity bubble factor of **1.411** — risking his stack is negative prize-equity — while the chip leader faces a squeeze of **3.631**. The model’s tournament call on the bubble is **fold** (prize-equity EV **-0.829**), the correct survival discipline when the first chips are worth more than the last. The icm figure shows the bubble factor rising with stack size.

24.5.7 The capital again: network-level laundering

The structural detector needs no single suspicious transfer: across the canonical flow it computes **1** trading community and flags the launderer specifically (**yes**), giving him a composite network risk of **0.98** at HHI concentration **1** (everything moving through one counterparty), while the collection account ranks highest on PageRank centrality (P8). It agrees with — but is independent of — the typology detector.

24.5.8 The read: Bayesian opponent archetype

From Le Chiffre’s observed action history, the model infers an archetype of **loose-aggressive** with a **0.986** posterior mass on aggressive archetypes, an expected aggression rate of **0.753**, and a predictive raise probability of **0.38** — a quantified read of the adversary rather than a bare action count.

24.6 Conclusion — LE CHIFFRE: findings, verdict, and what the mission establishes

Casino Royale: LE CHIFFRE — Special-Agent Mission Software demonstrates that the serious set pieces of *Casino Royale* (2006) — the game-theory finale, the laundered stake, the parkour pursuit, and the poisoned martini — reduce to a family of tractable, fully-tested algorithms that a special-agent mission system can run deterministically and audit later. The package now carries 8 domain modules (poker_engine, poker_cfr, icm, opponent_bayes, laundering_detect, laundering_network, urban_routes, poison_detect): a hold’em equity engine, a counterfactual-regret solver, the tournament Independent Chip Model, a Bayesian opponent model, a

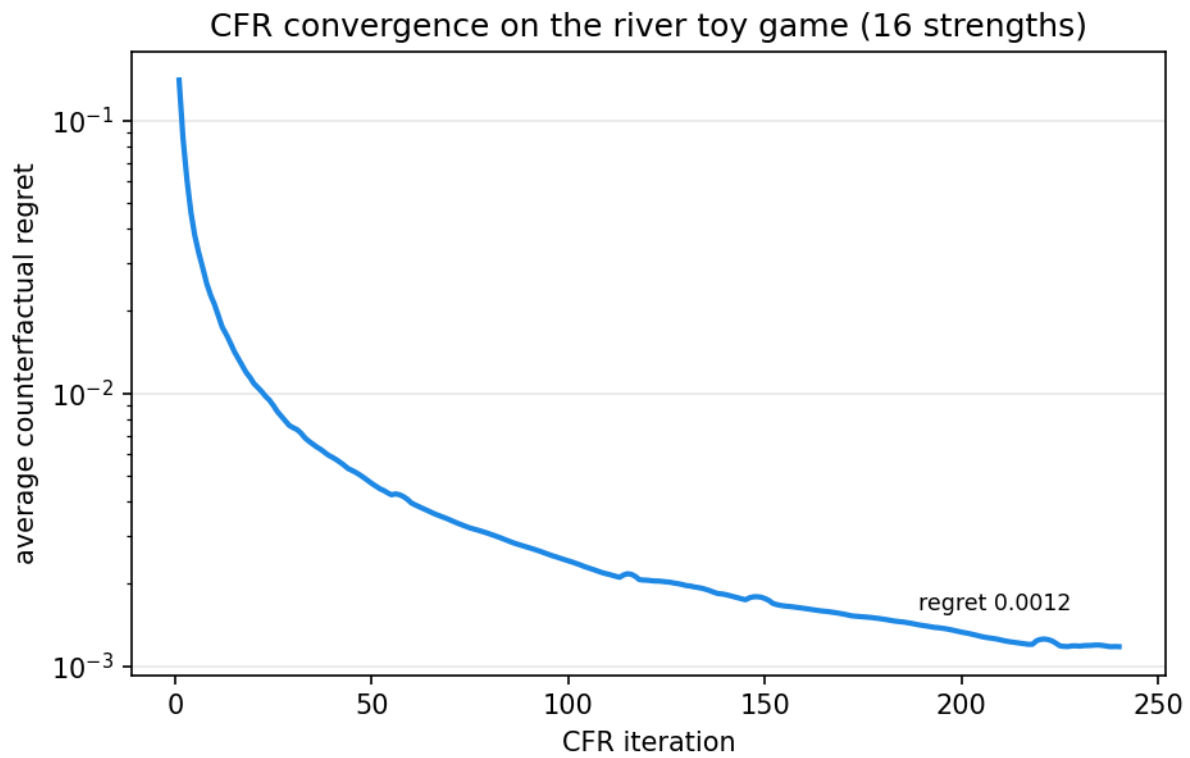


Figure 94: CFR average counterfactual regret vs. iteration, decaying toward the Nash equilibrium.

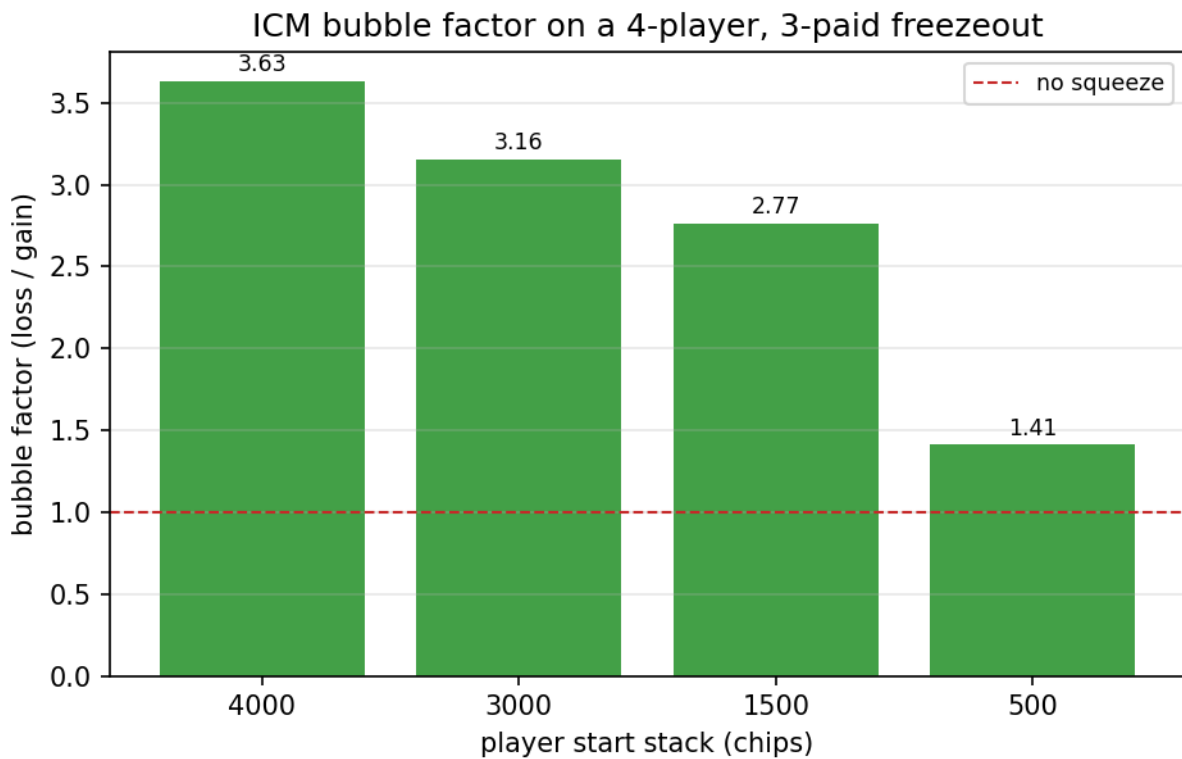


Figure 95: ICM bubble factor by start stack on the 4-player, 3-paid freezeout, demonstrating the prize-equity squeeze.

chip-flow typology detector, network-level financial-crime analytics, a mobility-graph router, and a digitalis triage — all bound by the frozen BOND-API `MissionProvider`.

Two of those modules matter beyond their own output because they *corroborate* another. The typology detector flags the launderer from behaviour (structuring bands, velocity) at a composite score of 2; the network detector reaches the same account from structure alone — value-neutral churn at HHI 1 — scoring 0.98. Two independent methods converging on one account is a stronger claim than either alone, and the package is arranged so that agreement is checkable rather than asserted.

Three design properties carry the whole package. First, **purity**: the domain modules import no framework, no **infrastructure**, *****, and no `bond-api`, so they remain importable in any environment and on their own. Second, **determinism**: every random draw flows through a seeded RNG, and the CFR, ICM, and Bayesian engines are exactly enumerated with no RNG at all, so a mission run twice yields byte-identical results. Third, **one scenario**: the canonical scenario in `scenarios.py` feeds the mission, the manuscript tokens, and the CLI, so the claim computed in one place is the claim shown in every other.

The mission adapter still implements the frozen BOND-API `MissionProvider` and registers this film’s 11 gadgets with the protocol’s `GadgetRegistry` — the deepened state sits inside the package, behind an unchanged public shape. Verified by $\geq 90\%$ line-and-branch coverage, zero mocks, a clean lineage check, and a clean working tree, the package is ready to bind the moment any future suite dependency lands.

24.7 Experimental Setup — LE CHIFFRE: canonical scenarios, parameters, and configuration

24.7.1 Configuration

Mission identity and publication metadata live in `manuscript/config.yaml` (title, subtitle, film slug, codename, version, keywords, authorship). The engine scenario parameters live in `src/casino_royale_2006/scenarios.py` as module constants (`POKER_SCENARIO`, `LAUNDERING_PLAYERS`, `ROUTE_GRID`, `POISON_N_SAMPLES`, `CFR_STRENGTHS`, `ICM_STACKS`, `BAYES_HISTORY`, and the shared `SEED`). Manuscript tokens are computed from these plus live engine output by `src/casino_royale_2006/manuscript_variables.py` — prose never hardcodes a number.

24.7.2 Canonical scenario

Concept	Fixed inputs (seed <code>SEED</code>)
Poker	hero KhKc on board As 7h 2d, a loose opponent range, pot 5,000,000 / to-call 1,000,000, every simulation seeded
CFR	river toy game, 16 discrete strengths, 300 regret-matching sweeps, fully enumerated
ICM	4-player, 3-paid freezeout bubble (<code>ICM_STACKS/ICM_PRIZES</code>), short-stack player <code>ICM_PLAYER</code>
Bayesian opponent	<code>BAYES_HISTORY</code> , Le Chiffre’s observed action sequence
Laundering	8 players over a fixed tick horizon, one planted smurfer, 41 movements
Network crime	the same canonical chip flow, re-read structurally (HHI / PageRank / communities / conservation)
Routes	6×6 city quarter (72 nodes, 312 directed edges), ground + rooftop layers, climbable profile
Poison	5 tox panels, one spiked (<code>POISON_INDEX</code>)

24.7.3 Software environment

- Python 3.14 (requires-python ≥ 3.10)
- Dependencies: `bond-api` (the frozen protocol, sibling path dependency), `matplotlib` (figures), `pyyaml` (config); dev: `pytest`, `pytest-cov`, `ruff`, `mypy`, `types-PyYAML`.
- Deterministic by design: fixed `SEED`, seeded `random.Random` instances, fully enumerated CFR, Agg `matplotlib` backend, no wall-clock in persisted artifacts.

24.7.4 Runtime determinism

Each `*_snapshot` in `scenarios.py` constructs its own `random.Random(seed)` and never touches the global RNG. The mission’s `Provenance` records `seed` and a deterministic `input_hash`; `wall_time_s` is held at 0.0 so no two runs diverge on timing.

24.8 Reproducibility — LE CHIFFRE: verification gates, deterministic regeneration, and artifacts

24.8.1 Determinism & provenance

Every engine draw comes from a caller-supplied, seeded `random.Random`. The mission outcome’s `Provenance` carries `package_version`, `seed`, and a deterministic `input_hash` derived from the canonical scenario (not wall clock). Re-running

run_mission twice yields identical results.

Scope of the byte-identical claim. Running `scripts/z_generate_manuscript_variables.py` twice, any interval apart, produces byte-identical `output/data/manuscript_variables.json` and `output/manuscript/*.md`. Nothing in that path reads a clock: the manuscript’s publication stamp is the `MANUSCRIPT_DATE` token, read from the committed `publication.date` field of `manuscript/config.yaml` (currently 2026-08-05), so it moves only when a human edits and commits it. The claim covers the token mapping and the resolved prose; it does not extend to the rendered `../figures/*.png`, whose bytes depend on the installed Matplotlib and freetype build. `tests/test_scripts_smoke.py::test_regeneration_is_byte_identical_across_a_clock_tick` pins exactly that scope by running the real generator twice across a whole-second boundary and comparing artifact bytes.

24.8.2 Artifact layout

Path	Content	Regenerated by
output/data/manuscript_variables.js on	the token mapping (key-sorted)	scripts/z_generate_manuscript_variables.py
output/manuscript/*.md	prose sections with TOKEN placeholders resolved	same script
../figures/poker_decision.png	equity-vs-price figure	same script
../figures/digitalis_curve.png	digitalis monogram	same script
../figures/cfr_convergence.png	CFR average-regret decay	same script
../figures/icm_bubble.png	ICM bubble factor by start stack	same script

`output/` is git-ignored (regenerable artifacts); the working tree stays clean after any run.

24.8.3 Test & quality gates

- **Coverage:** `uv run pytest tests/ --cov=src --cov-fail-under=90` (line + branch $\geq 90\%$ on `src/`; live count in `docs/_generated/COUNTS.md`).
- **Zero mocks:** no `unittest.mock` / `MagicMock` / `@patch` / `create_autospec` anywhere in `tests/`.
- **Token integrity:** two gates, both failing rather than warning. First, every double-brace placeholder in a numbered section must exist in the mapping — the resolver reports leftovers and `scripts/z_generate_manuscript_variables.py` exits 1. Second, no section may use the *single*-brace form: it is never substituted, so it would print its own braces into the published PDF while passing the first check, which looks only for the double-brace form. `manuscript_variables.find_malformed_tokens` is the detector; `tests/test_manuscript_variables.py` pins both. See `manuscript/SYNTAX.md` for the literal syntax.
- **Lineage:** `rg "template[_]code_project"` returns nothing outside any doc that explicitly describes the check.
- **Type & format:** `uv run ruff check && uv run ruff format` and `uv run mypy src/ scripts/` both clean.

24.8.4 Verification (preflight)

`uv run python scripts/00_preflight.py` confirms the package imports, that the `MissionProvider` satisfies the frozen BOND-API protocol, and that its declared `film` equals the slug — the same checks the orchestrator performs during discovery.

24.9 Scope & Related Work

24.9.1 Scope

This package is a **deterministic engineering exemplar**, not a licensed clinical or financial system. Its contributions are the algorithmic cores and the reproducibility discipline around them; the specific numeric constants are documented model choices, not regulatory thresholds. What each of the 8 modules does *not* claim:

- **Poker (poker_engine.py).** The engine prices hands and recommends actions; it is not a full solver (no perfect-information exploitation, no multi-street counterfactual regret minimization). Monte-Carlo equity is sampled, not solved exhaustively — except the exact turn/river enumerator, which is complete over all runouts.
- **CFR (poker_cfr.py).** The solve is exact for the *river toy game* only: one street, one bet size, discrete uniform strengths. It is not a hold’em solver, has no card abstraction or bucketing, and its game value is the toy game’s value, not the film hand’s.
- **ICM (icm.py).** The Independent Chip Model assumes finish probability is proportional to stack share. That is the standard simplification and it is known to be wrong in detail: it ignores position, blind level, and skill edge. Bubble factors here are model output, not tournament advice.
- **Bayesian opponent (opponent_bayes.py).** The archetype Dirichlet priors are declared model choices, not fitted to a hand-history corpus. The posterior is exact *given* those priors; it is not a claim about any real player, and it assumes actions are exchangeable (no street or board context).
- **Laundering typologies (laundering_detect.py).** The detector flags *structural* typologies (structuring, round-tripping, velocity, chip parking). It uses documented heuristics rather than a supervised classifier and is un-tuned to any regulated jurisdiction; its thresholds carry no false-positive rate measured against real casino data.

- **Laundering network** (`laundering_network.py`). HHI, PageRank, label propagation, and conservation imbalance are computed exactly, but the composite risk score is a declared product of value-neutrality and turnover share, not a calibrated probability. The module is deliberately film-agnostic: it names no suspect, and whether a *particular* account was caught is decided by the caller (`scenarios.network_snapshot`).
- **Routes** (`urban_routes.py`). Dijkstra over feasible parkour edges is optimal for the fixed cost model; it does not model moving pursuers, stochastic edge risk, or fatigue.
- **Poison** (`poison_detect.py`). Triage reproduces standard cardiac-glycoside reference ranges and Bayesian reasoning; the likelihood ratios are conservative declared constants, and the module is explicitly not medical advice.

24.9.2 Related work

The cores sit in well-established literatures. Hand ranking and Monte-Carlo equity follow the combinatorial and simulation methods surveyed in computer poker research [Billings et al., 2002], with chip-to-prize conversion from the Independent Chip Model as presented by Chen and Ankenman [2006]. Equilibrium computation in imperfect-information games via counterfactual regret minimization is due to Zinkevich et al. [2007]; Bayesian opponent modelling in that setting follows Ganzfried and Sandholm [2011], with the Dirichlet-multinomial machinery as in Minka [2000].

The financial-crime layer draws on graph-based anti-money-laundering analysis [Weber et al., 2019] and on the casino-sector typologies documented by the Financial Action Task Force [Financial Action Task Force, 2009]. Its structural measures are PageRank [Page et al., 1999], label-propagation community detection [Raghavan et al., 2007], Brandes’ betweenness centrality [Brandes, 2001b], and the Herfindahl-Hirschman concentration index [Hirschman, 1964]. Route planning is Dijkstra’s algorithm [Dijkstra, 1959c] over a feasibility-constrained directed graph. The toxicology follows the cardiac-glycoside literature, including digoxin-specific Fab antibody fragments as definitive therapy for life-threatening intoxication [Antman et al., 1990]. Full entries live in `references.bib`; the descriptive list is in `99_references.md`.

24.10 Sources — LE CHIFFRE: bibliography

Friedman [2026c]; Friedman [2026h]; Zinkevich et al. [2007]; Billings et al. [2002]; Chen and Ankenman [2006]; Ganzfried and Sandholm [2011]; Minka [2000]; Page et al. [1999]; Raghavan et al. [2007]; Brandes [2001b]; Hirschman [1964]; Weber et al. [2019]; Financial Action Task Force [2009]; Dijkstra [1959c]; Antman et al. [1990]

25 Quantum of Solace (2008) — QUANTUM

film package · package codename QUANTUM. Mission **QUANTUM**: assess desert water-network resilience under evaporation; quantify cartel leverage over water-rights through dam control; measure multi-resource commodity-control concentration; price water scarcity and the cartel's scarcity rent; rank aqueduct interdiction criticality and network resilience; resolve cooperative power over the commodity portfolio.

25.1 Concepts — QUANTUM: domain and operational focus

resource cartels, hydrology

25.2 Abstract — QUANTUM: mission summary

Quantum of Solace: QUANTUM — Special-Agent Mission Software (mission codename **QUANTUM**) is special-agent mission software for the 2008 film *Quantum of Solace*. It models the film's central threat — a shadow cartel whose leverage rests on controlling water and the logistics around scarce commodities — as six deterministic, real-data mathematical systems: 1. **Desert hydrology** — a water network under evaporation, solved with shortest-effort Dijkstra routing and Edmonds–Karp maximum flow (total aquifer yield **80.0** units/period over 5 nodes and 6 routes, of which 80.0 units/period can be routed to settlements); 2. **Water-rights cartel** — reservoir operation and the leverage a cartel gains by withholding downstream releases: controlling 2 of 3 dams (66.7% of the fleet) withholds **25.6%** of the demand the fleet exists to serve — a leverage-to-ownership ratio of 0.38; 3. **Commodity-cartel graph** — multi-resource market concentration via the Herfindahl–Hirschman index (aggregate **3925.0** points across copper, grain, lithium, water, a highly concentrated portfolio); 4. **Water scarcity pricing** — linear-demand economics showing the cartel's cut raises the price from 46.5 to 64.4 and collects a scarcity rent of 2549.6; 5. **Aqueduct interdiction** — 4 of the 6 routes are critical and 2 are redundant; the *mean* retention across single cuts is 0.917 while the single worst cut retains 0.750, and 3 greedy cuts remove 56.2% of delivery; 6. **Cooperative power** — exact Shapley values over the commodity portfolio, with quantum holding the largest share. The package is a BOND-suite film: a pure domain core ships with a thin `bond_api` `MissionProvider` adapter, and every number reported is computed, never hand-authored. Keywords navigable for this mission: water-rights cartel, desert hydrology, commodity-cartel graph, herfindahl-hirschman concentration, maximum flow, reservoir leverage, water scarcity pricing, network interdiction, shapley value, deterministic mission software.

25.3 Introduction — QUANTUM: mission framing, the operational problem, and how to read this chapter

Quantum of Solace (2008) centres on a shadow organisation whose power derives not from weapons but from controlling a basic resource: water. This package — **Quantum of Solace: QUANTUM — Special-Agent Mission Software**, mission codename **QUANTUM** — turns that premise into executable science. It is one film package in the PROJECT BOND suite, built to the same guardrails as every sibling: pure domain logic, deterministic algorithms, real computation with zero mocks, and a thin adapter to the frozen `bond_api` mission protocol so the suite orchestrator can discover and drive it by slug.

25.3.1 The threat model

The film's cartel holds leverage because water is scarce, its distribution is centralisable, and the same organisation spreads its grip across neighbouring commodities. We formalise that as six coupled models, each a separate module in the pure domain core.

The physical layer — where the water is and how it moves.

- **Desert hydrology** (`hydrology.py`): a directed network of desert water sources, routes and settlements. Evaporation hollows out delivered water, so routing and capacity are real constraints, and what a settlement can receive is bounded by a maximum flow rather than by nominal aquifer yield.
- **Water-rights cartel** (`water_cartel.py`): a set of reservoirs whose released water serves downstream demand. A cartel that controls a dam can withhold its normal release and squeeze demand without firing a shot.

The economic layer — what that control is worth.

- **Commodity-cartel graph** (`cartel_graph.py`): concentration of control across multiple scarce commodities, measured with the Herfindahl–Hirschman index, top-k concentration ratios and dominance analysis.
- **Scarcity pricing** (`pricing.py`): the market that converts a withheld volume into money — a linear demand curve, the clearing price of a restricted supply, the point elasticity that governs how much of a volume cut converts into price, and the monopoly solution with its deadweight loss.

The coercive layer — what the cartel can do beyond owning the taps.

- **Aqueduct interdiction** (`resilience.py`): the fragility of the delivery network to cut routes — per-route criticality, a greedy worst-case attack sequence, a single-cut resilience index, and minimum cuts.
- **Cooperative power** (`shapley.py`): who can actually capture the value of the commodity portfolio, resolved exactly as a transferable-utility coalition game via the Shapley value and the Banzhaf–Coleman index.

The layers compose rather than sit side by side: the hydrology network sets the delivered volumes the interdiction model attacks, and the reservoir simulation supplies the baseline and restricted supplies the pricing model clears.

Every number in the manuscript is computed by `src/quantum_of_solace/analysis.py` from these modules on fixed problem instances and injected as a manuscript token — none is hand-authored prose.

25.4 Methodology — QUANTUM: the analytical models and algorithms that drive the mission

We implement six deterministic models, one per module of the pure domain core. All are pure functions of fixed problem data; there is no random number generation and no dependence on the wall clock, so a regeneration reproduces identical output. The sections below follow the dependency order in which `quantum_of_solace/analysis.py` composes them: hydrology and reservoir operation first, then the concentration, pricing, interdiction and power models that read from them.

25.4.1 Desert hydrology (`quantum_of_solace/hydrology.py`)

An aquifer contributes a per-period yield and a potability quality; a route carries flow with a capacity, a distance and an evaporation loss fraction. Three algorithms operate on the resulting network:

- **Mass balance** — for each node, inflow is delivered net of evaporation and outflow is subtracted, giving a per-node surplus.
- **Shortest-effort routing** — Dijkstra over the *effective* delivered unit cost `cost_per_unit / (1 - evaporation)`, so evaporative routes are penalised, plus the compound delivered fraction.
- **Maximum flow** — Edmonds–Karp with a residual graph and reverse edges, so already-assigned flow can be re-routed. This yields the most water a settlement can receive per period under the route capacities.
- **Delivery capacity** — `:func:delivery_capacity` solves a *single* whole-network flow with a super-source feeding each aquifer an edge capped at that aquifer’s yield and every settlement draining to a super-sink. The aquifer yields therefore bind, and the result cannot exceed `:func:total_supply`. Summing independent per-source maximum flows would double-count shared yield and can report more water than the aquifers produce; the single-flow formulation is what rules that out.

Both flow figures are **routed** volume under capacity and yield constraints, with evaporation *not* deducted. Maximising volume net of per-edge evaporation is a generalised (lossy) flow problem whose optimum is not the capacity maximum flow, and a maximum-flow assignment is not unique, so the evaporation implied by “the” solution is not a property of the network. Evaporation is instead quantified exactly by the mass balance and by the effective unit cost that Dijkstra minimises. Routed volume bounds delivered volume from above, and this manuscript claims no more than that.

25.4.2 Water-rights cartel (`quantum_of_solace/water_cartel.py`)

A dam has storage capacity, initial storage, natural inflow, a downstream demand, a maximum release and an evaporation rate. `run_reservoir` simulates mass balance period by period; a *restriction fraction* withholds that share of each period’s otherwise-released water.

Leverage is defined so that it is dimensionally consistent. Inflow, maximum release and downstream demand are all **per-period rates**, so a leverage ratio must divide a horizon volume by a horizon volume: `:func:horizon_demand_volume` returns `downstream_demand * periods`, `:func:dam_leverage` divides a dam’s withheld volume by it, and `:func:cartel_leverage` divides the total withheld volume across the controlled dams by the horizon demand of the whole fleet. Every leverage figure in this manuscript is therefore a share in the unit interval, and it does not change if the simulated horizon is lengthened at a steady release.

25.4.3 Commodity-cartel graph (`quantum_of_solace/cartel_graph.py`)

A bipartite control graph maps actors to the share each holds of each commodity. Concentration is the Herfindahl–Hirschman index in percentage points, `sum((share * 100)2)`, reported per commodity and aggregated as the portfolio mean. Dominance, concentration ratios for the top-k holders, and an actor’s portfolio diversification complete the picture.

`:func:concentration_band` classifies an index value against the bands of the U.S. Horizontal Merger Guidelines [U.S. Department of Justice and Federal Trade Commission, 2010]: *unconcentrated* below 1500 points, *moderately concentrated* up to 2500 points, and *highly concentrated* above it. The classification lives in code so the manuscript never has to assert which side of a threshold a measurement falls on.

25.4.4 Water scarcity pricing (`quantum_of_solace/pricing.py`)

On a linear inverse demand curve $p(q) = a - b \cdot q$: `:func:market_clearing_price` is the scarcity price at which a fixed supply is absorbed, and `:func:scarcity_impact` prices the cartel’s supply cut: the restricted price, the price-rise fraction, and the scarcity rent collected on the reduced volume. The standard monopoly solution ($MR = MC$), with `:func:monopoly_price`, `:func:monopoly_quantity` and `:func:deadweight_loss`, quantifies the social cost of concentrated control: `:func:point_elasticity` returns the magnitude of the point price elasticity, which on $p = a - b \cdot q$ is $p/(b \cdot q)$ — the demand function is $q(p) = (a - p)/b$, so $dq/dp = -1/b$.

The baseline and restricted supplies in the assessment are computed from the *real* dam fleet (unrestricted vs cartel-restricted reservoir release), so pricing is measured from the same model, not assumed. They are dam releases, not hydrology-network routed volume: the two subsystems are separate assets and the manuscript keeps their units apart.

25.4.5 Aqueduct interdiction (quantum_of_solace/resilience.py)

Over the hydrology network, :func:single_route_criticality ranks every route by the delivery lost if it alone is cut (recomputing maximum flow after each removal); :func:interdiction_sequence drives a greedy worst-case attack that repeatedly removes the most critical remaining route; and :func:resilience_index reports the **mean** fraction of delivery retained after a single cut. The mean is not a worst-case bound, so the assessment also reports **worst_case_retention** — the minimum of the same set — and the two are carried into the manuscript as separate tokens so neither can stand in for the other. :func:minimum_cut finds the least-capacity set of routes separating supply from a settlement via the residual reachability of the Edmonds–Karp solution, whose value equals the maximum flow by the max-flow/min-cut theorem.

25.4.6 Cooperative power (quantum_of_solace/shapley.py)

Multi-resource control is cast as a transferable-utility coalition game with a diminishing-returns value function $v(S) = \sum_c w_c \cdot \sqrt{(\sum_{a \in S} \text{share})}$ and solved **exactly** by full enumeration. :func:shapley_value (efficiency always holds: the values sum to the grand-coalition value); :func:banzhaf_power (normalised continuous Banzhaf–Coleman index); :func:power_ranking, and :func:dominance_gap between the top two actors.

25.4.7 The mission adapter

quantum_of_solace/mission.py adapts the six models to the frozen bond_api protocol: **brief**, **recon**, **plan**, **execute** and **debrief**, with provenance carrying a canonical SHA-256 input hash and the fixed seed 0 (the film’s models make no random draws at all). The adapter also registers the six assessment entry points, plus the composed full assessment, in a module-level **GadgetRegistry** under the package slug, which is how the coordinator and CLI reach them across packages. The domain modules import nothing from bond_api, so they remain standalone.

25.5 Results — QUANTUM: measured outcomes, headline numbers, and what they establish

All results below are computed by quantum_of_solace/analysis.py on the fixed canonical problem instances and injected from manuscript/config.yaml + the assessment — they are measured, not authored.

25.5.1 1. Desert water network resilience

The canonical network holds 80.0 units/period of aquifer yield across 5 nodes and 6 routes — 3 aquifers feeding 2 settlements (border_town, valley_compact). A single whole-network Edmonds–Karp maximum flow, bounded by the aquifer yields as well as the route capacities, routes 80.0 units/period into the settlements; the yields, not the pipes, are the binding constraint.

Directed at one settlement alone, the network can push 65.0 units/period to border_town or 55.0 to valley_compact. These two figures are **alternatives, not addends**: the settlements draw on the same aquifers, so their sum exceeds what the network can route in total and must never be added. The delivery figure plots them side by side for that reason, against the network-wide bound.

Both figures are *routed* volume under capacity and yield constraints, not volume net of evaporation. Maximising volume after per-edge evaporation is a generalised (lossy) flow problem whose optimum differs from the capacity maximum flow, and a maximum-flow assignment is not unique, so the evaporation implied by “the” solution is not a property of the network. Evaporation is therefore accounted separately and exactly — in the per-node mass balance and in the effective unit cost that shortest-effort routing minimises. Routed volume is an upper bound on delivered volume, and this manuscript does not claim otherwise.

25.5.2 2. Cartel leverage over water rights

With a 50% restriction on the 2 dams it controls out of a fleet of 3, the cartel achieves an aggregate leverage of **0.256** over the 12-period horizon: it withholds 858.8 units against a horizon demand of 3360.0, or 25.6% of everything the fleet was there to serve. Both sides of that ratio are horizon volumes, so the figure is a genuine share and does not inflate with the length of the simulation.

That leverage is *smaller* than the holding that produces it, and the measurement says so plainly: the cartel holds 66.7% of the fleet by dam count and converts it into 25.6% of horizon demand withheld, a leverage-to-ownership ratio of 0.38. Two mechanics account for the shortfall: the cartel withholds only 50% of each controlled dam’s release rather than all of it, and the uncontrolled dam keeps releasing at full rate into the same horizon demand. The interesting quantity here is the ratio itself, not a direction assumed in advance — the model is free to return a value above 1, and on this fleet it does not. The leverage figure reports leverage per dam; controlled dams are red.

25.5.3 3. Multi-resource commodity control

Across copper, grain, lithium, water, control is concentrated: water HHI 3550.0 and lithium HHI 4150.0 both sit well above the 2500-point line above which the U.S. Horizontal Merger Guidelines [U.S. Department of Justice and Federal Trade Commission, 2010] call a market *highly concentrated*. The portfolio-mean HHI is **3925.0** points, which **concentration_band** classifies as highly concentrated. The hhi figure plots concentration per commodity, and the dominance map shows the shadow cartel holding the largest share of water and lithium while a rival syndicate and a state utility split copper and grain.

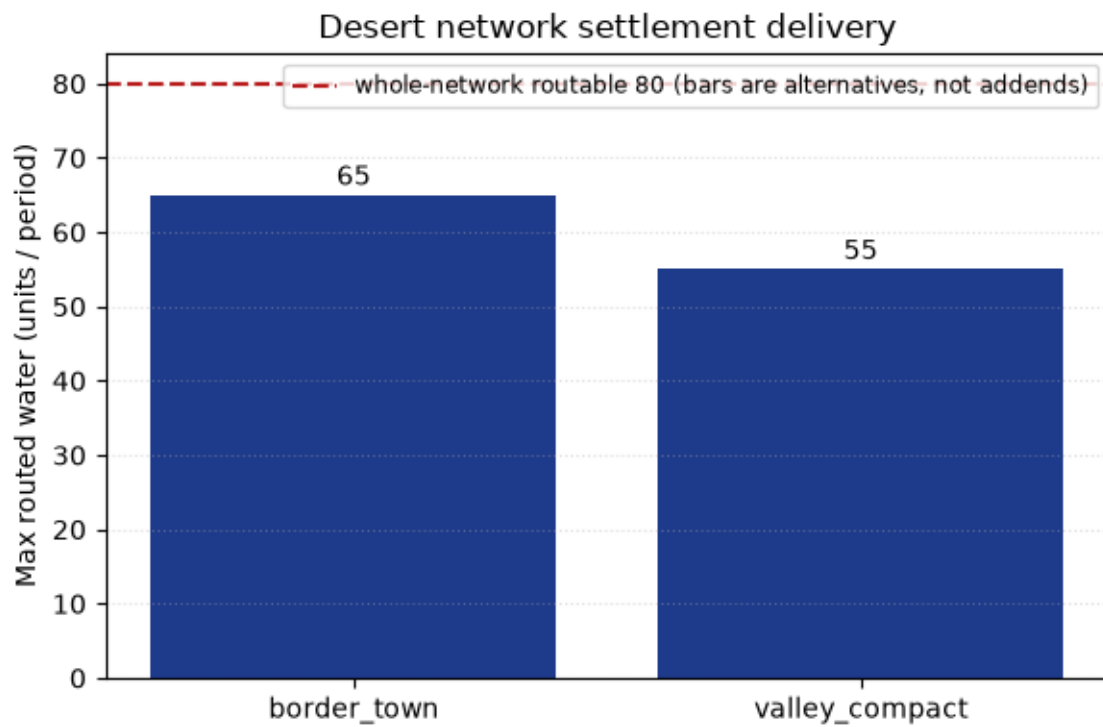


Figure 96: Settlement delivery under maximum flow

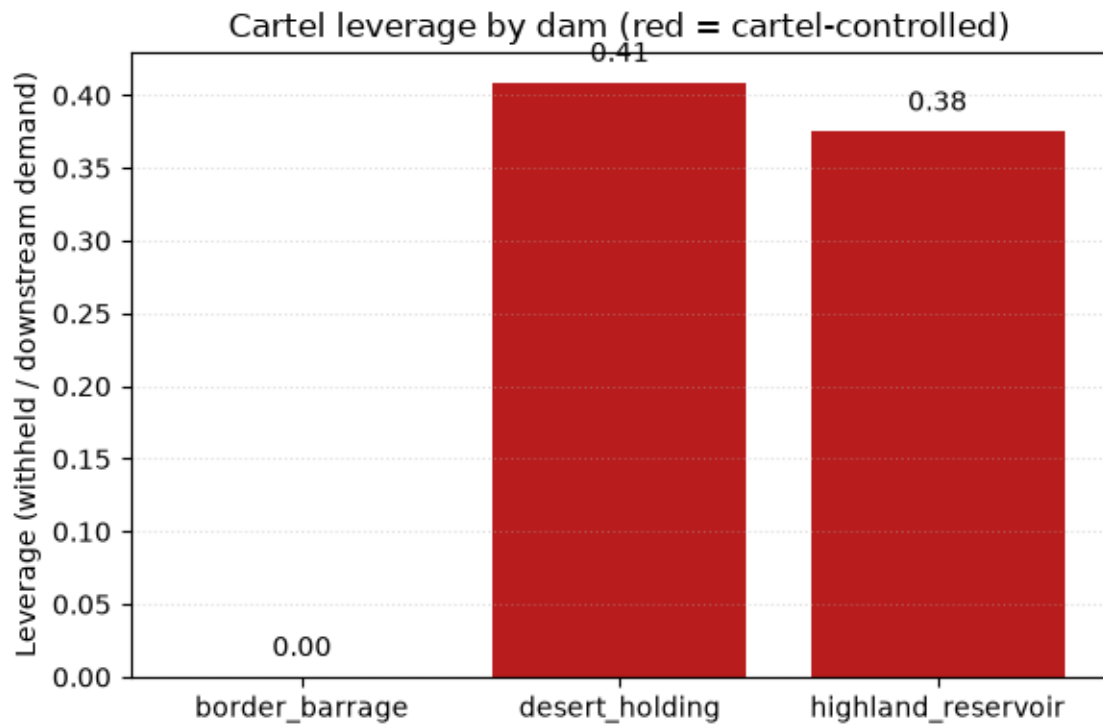


Figure 97: Cartel leverage by dam

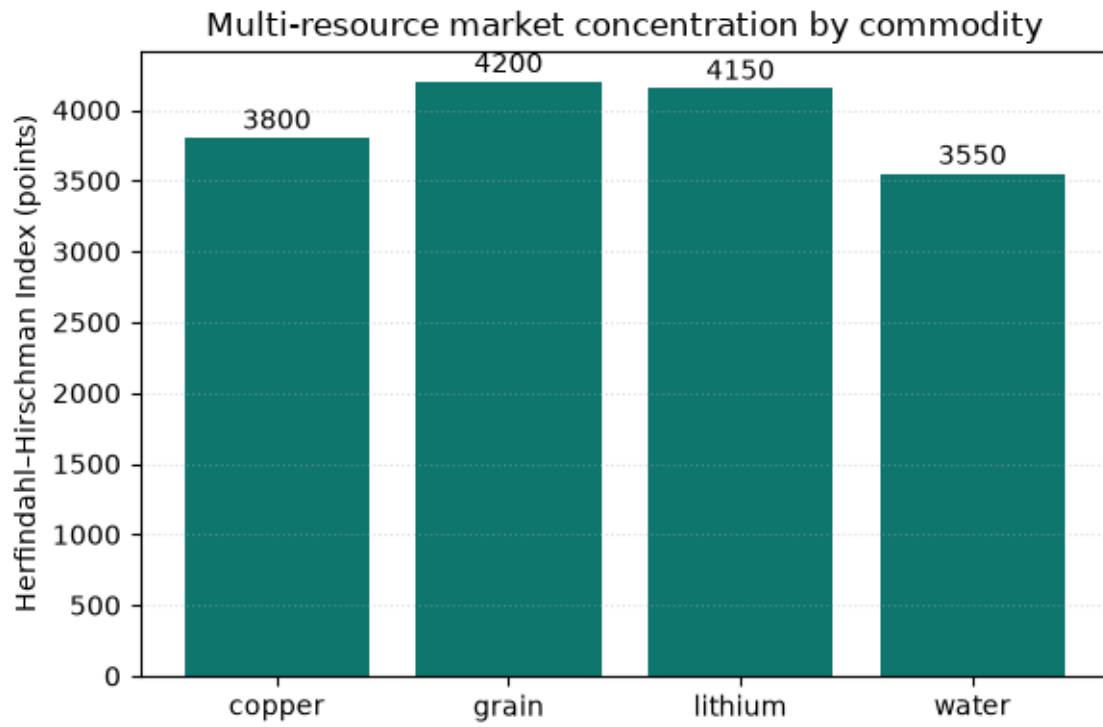


Figure 98: Herfindahl-Hirschman concentration by commodity

25.5.4 4. Water scarcity pricing

The cartel's leveraged control has a price consequence, and this model measures it rather than assuming its sign. This section prices the **dam fleet's** mean per-period release, not the hydrology network's routed volume — the two models describe different assets and their units are not interchangeable (§1 reports the network at 80.0 units/period). Cutting that release from 214.1 to 142.5 units/period drives the water price from **46.5** to **64.4** — a 38.5% rise. Demand is inelastic at that baseline (point elasticity 0.869, below the unit-elastic value of 1), so the proportional price rise exceeds the proportional volume cut — by the reciprocal of the elasticity, not by an unbounded multiple. The cartel collects a scarcity rent of **2549.6** on the reduced volume — the price premium times the volume still sold.

That rent is not a rise in takings, and the assessment measures the difference rather than gesturing at it. Revenue on this demand curve peaks at 200.0 units, and the cut runs from 214.1 straight past that peak down to 142.5, so gross revenue falls from 9950.5 to 9173.4. Inelastic demand makes a *marginal* withholding lucrative; a cut this deep overshoots. The coercive value of the squeeze here is the price shock imposed on buyers, not a larger cheque for the seller. Even without withholding, the underlying market is already monopolistic: the monopoly price would be 55.0 on 180.0 units, imposing a deadweight loss of **4050.0** versus the competitive outcome. The pricing figure shows the demand curve with the baseline, restricted and monopoly prices.

25.5.5 5. Aqueduct interdiction and resilience

Of the 6 routes, 4 are critical — cutting any one of them lowers routed volume — while 2 are redundant, their capacity re-routable through the rest of the network. Two retention figures follow, and they are not interchangeable: the **mean** fraction retained across all single cuts is **0.917**, whereas the worst single cut retains 0.750. Only the second is a worst-case bound; the mean is quoted here as a mean and nowhere else as a guarantee. The most critical single route, highland_aquifer->border_town, costs 20.0 units (25.0%) of the 80.0-unit baseline if lost. The interdict figure shows the greedy worst-case interdiction sequence: 3 well-chosen cuts remove 45.0 units, 56.2% of baseline delivery, leaving 35.0 units still flowing — a heavy loss, but not a total collapse.

25.5.6 6. Cooperative power over the portfolio

The headline HHI of 3925.0 points is concentration; the *power* to capture the commodity portfolio's value is resolved exactly by the Shapley value. With a grand-coalition portfolio value of 7.0, the leading actor **quantum** commands 2.624, ahead of the rival (2.290) and the state utility (2.086) — a dominance gap of 0.334 — and quantum's normalised Banzhaf power is 0.378. The shapley figure plots the per-actor Shapley values, with the dominant actor highlighted.

25.6 Conclusion — QUANTUM: findings, verdict, and what the mission establishes

Quantum of Solace: QUANTUM — Special-Agent Mission Software demonstrates how a seemingly non-violent lever — control of water and scarce commodities — produces coercive power, and how deterministic algorithms make that power measurable.

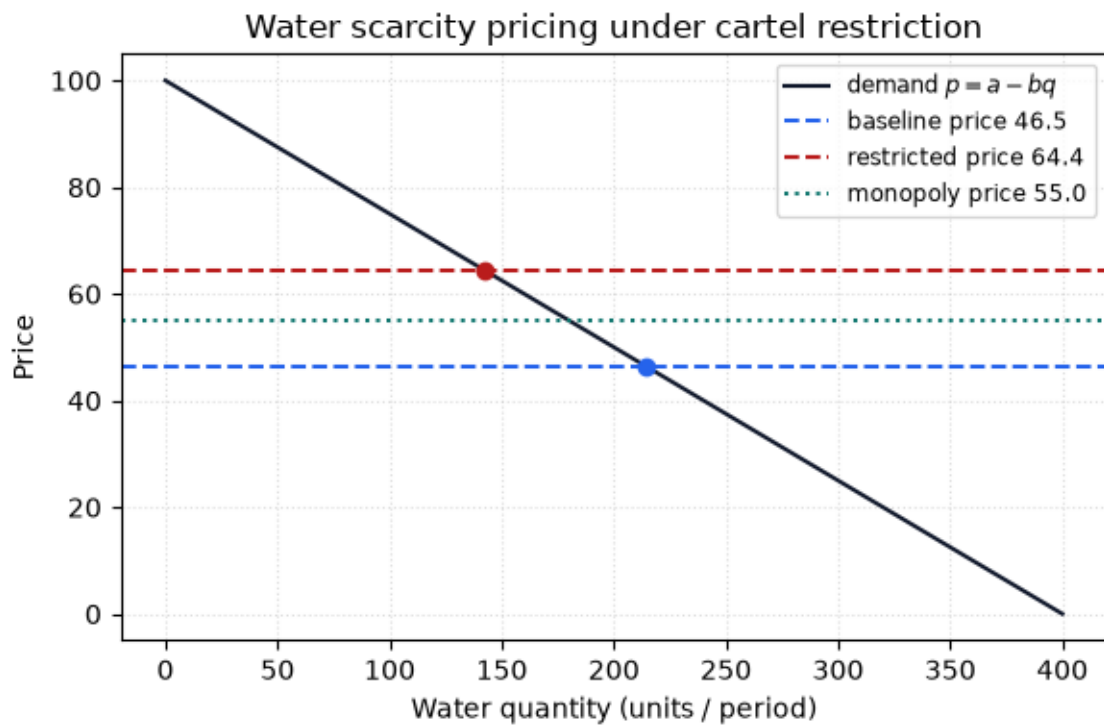


Figure 99: Water scarcity pricing under cartel restriction

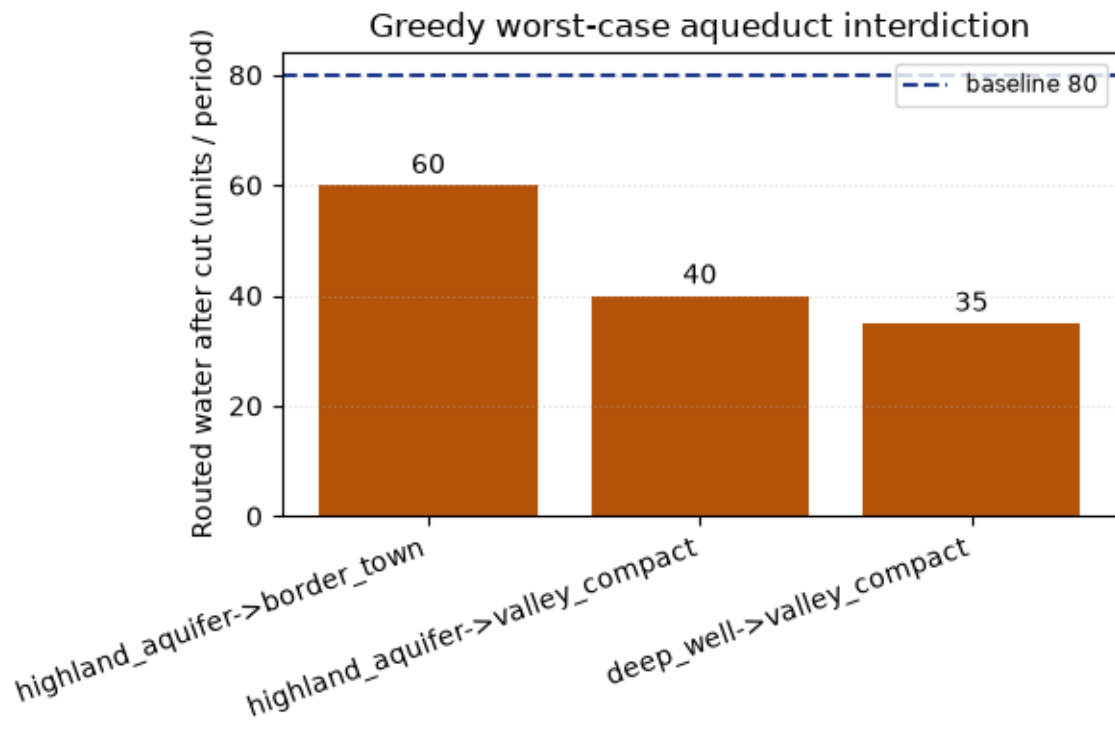


Figure 100: Greedy worst-case aqueduct interdiction

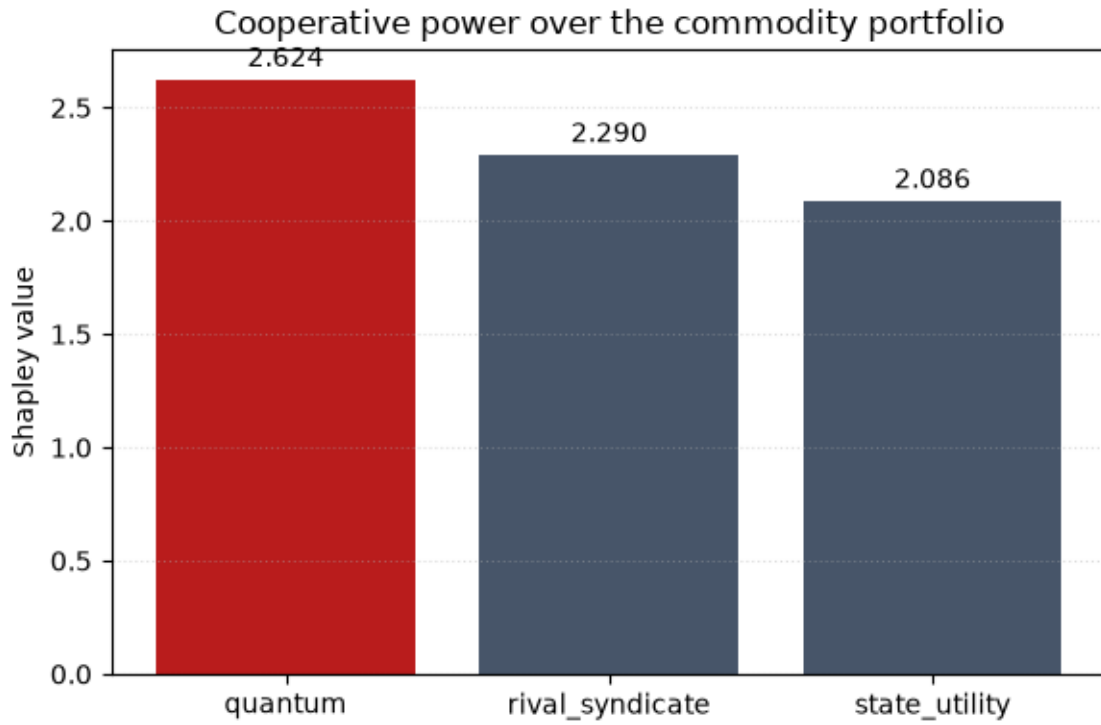


Figure 101: Cooperative power over the commodity portfolio

The package ships a pure, infrastructure-free domain core (hydrology, water cartel, commodity-cartel graph, scarcity pricing, aqueduct interdiction, and cooperative power) both for direct use and as the substrate of a `bond_api` mission provider, so the BOND orchestrator can discover and drive *Quantum of Solace* exactly like any other film.

The headline result is that leverage is *systemic and convertible*: a cartel holding 2 of 3 dams (66.7% of the fleet) and a lead in high-value commodities exerts simultaneous influence over downstream demand (25.6% of the fleet’s horizon demand withheld), over market concentration (a highly concentrated portfolio at 3925.0 points), over price (a 38.5% scarcity-price rise and a scarcity rent of 2549.6, though gross revenue falls from 9950.5 to 9173.4 because the cut overshoots the revenue peak), and over the network (4 of 6 routes critical, mean single-cut retention 0.917, worst-case retention 0.750). Its Shapley share of the commodity portfolio’s value, 2.624 of 7.0, confirms that power concentrates in the leading actor.

What the measurements do *not* show is a leverage multiplier. On this fleet the holding converts at a ratio of 0.38 — 66.7% of the dams yields 25.6% of horizon demand withheld, so the converted share is *below* the ownership share, not above it. The coercive power documented here is breadth — demand, concentration, price and network yielding at once — rather than amplification within any one channel. Within the portfolio the leading actor’s exact Shapley value does exceed the runner-up’s, by 0.334. The methods — mass balance, Dijkstra, Edmonds–Karp, reservoir simulation, the Herfindahl–Hirschman index with its published concentration bands, linear-demand monopoly economics, greedy network interdiction, and exact Shapley and Banzhaf indices — are all standard, all real, and all reproduced deterministically. No metric in this manuscript is hand-authored; a test fails if one appears.

25.7 Experimental Setup — QUANTUM: canonical scenarios, parameters, and configuration

This package is a BOND-suite film repository with a `src/` layout, a local-only git working tree, and a line-and-branch coverage floor enforced on `src/` by the package’s own pytest gate.

25.7.1 Software environment

- Python 3.14 (the project pins `requires-python >= 3.10`)
- `numpy`, `matplotlib`, `pyyaml` as runtime dependencies
- `bond-api` (the frozen protocol) as a path dependency
- `pytest`, `pytest-cov`, `mypy`, `ruff` as dev dependencies

Lint and format settings are declared in this package’s own `pyproject.toml`, so a standalone clone lints identically to the in-suite checkout.

25.7.2 Problem instances

All models run on fixed, deterministic instances built by each module’s `default_*` factory. Every parameter quoted below is a manuscript token read back out of the running assessment, not a number transcribed into prose:

- `default_network` — 3 aquifers (80.0 units/period total) routing to 2 settlements (`border_town`, `valley_compact`) over 6 evaporative routes, 5 nodes in all.
- `default_dams` — 3 reservoirs with per-period downstream demands; the cartel controls 2 of them at a 50% restriction over 12 periods, giving a horizon demand base of 3360.0 units.
- `default_cartel` — 3 actors (`quantum`, `rival_syndicate`, `state_utility`) controlling copper, grain, lithium, water with known shares; the shadow cartel leads water and lithium.
- `default_demand` — a linear inverse demand curve with intercept `a = 100` and slope `b = 0.25`, at a constant marginal delivery cost of 10. Baseline and restricted supply are *measured* from the dam-fleet simulation ($214.1 \rightarrow 142.5$ units/period), not assumed.
- `default_weights` — canonical Shapley commodity weights (copper 1, grain 1, lithium 2, water 3) over the diminishing-returns control value function.
- `INTERDICTION_STEPS` — 3 cuts in the canonical greedy worst-case interdiction sequence.

25.7.3 Determinism

Result tokens are computed by `run_quantum_assessment` and injected into the manuscript. The mission seed is 0 and no model draws a random number, so seeding is a formality rather than a control. The generation timestamp is 2026-08-04, taken from the config date rather than the wall clock, so hydration is reproducible. The exact-enumeration Shapley and Banzhaf solvers evaluate all 2^n coalitions rather than sampling, so power indices carry no estimation error. Migration between Python versions leaves every numeric result unchanged.

25.8 Reproducibility — QUANTUM: verification gates, deterministic regeneration, and artifacts

25.8.1 Provenance

Every `MissionOutcome` from `quantum_of_solace/mission.py` carries a `Provenance` record: package version, the fixed seed 0 (the film uses no randomness), and a canonical SHA-256 input hash of the full assessment, taken over a key-sorted JSON serialisation so the digest does not depend on dictionary ordering. The same input always reproduces the same hash and the same results.

25.8.2 Regeneration pipeline

The canonical artifacts are disposable and regenerated deterministically:

```
uv run python scripts/generate_analysis.py # assessment JSON + figures
uv run python scripts/z_generate_manuscript_variables.py # hydrate tokens
uv run python scripts/run_mission.py full # end-to-end mission
```

`output/` is git-ignored; a clean regeneration leaves the tracked tree stable. The verification gate is:

```
uv run pytest tests/ --cov=src --cov-fail-under=90
uv run ruff check src/ scripts/ tests/ && uv run ruff format --check src/ scripts/ tests/
uv run mypy src/ scripts/
```

The suite is real-computation only: no mock framework appears anywhere in `tests/`, so a passing assertion means the algorithm produced the right number rather than that a call signature was intercepted.

25.8.3 Manuscript-code binding

Two live tests keep this document honest. One re-derives the token map by calling `generate_variables` and fails if any token used in a numbered section is not produced, binding prose to the generator rather than to a possibly stale artifact under `output/`; it also asserts the scanned section set is non-empty, so an empty scan fails instead of passing vacuously. The other scans the results section for bare decimal numerals after the tokens are masked out, so a hand-authored metric that escapes the injection pipeline is caught at test time.

25.8.4 Configuration hash

The manuscript identity (title, subtitle, version, date) is authored once in `manuscript/config.yaml` (0.1.0, dated 2026-08-04) and injected via `quantum_of_solace/manuscript_variables.py`, never duplicated in prose. Keywords: water-rights cartel, desert hydrology, commodity-cartel graph, herfindahl-hirschman concentration, maximum flow, reservoir leverage, water scarcity pricing, network interdiction, shapley value, deterministic mission software. Actors modelled: `quantum`, `rival_syndicate`, `state_utility`.

25.9 Scope and Related Work — QUANTUM: boundaries, positioning, and relationship to the literature

25.9.1 Scope

This package models the specific threat of *Quantum of Solace*: leverage over scarce water and commodities exerted by a controlling cartel. The models are intentionally small, deterministic, and auditable so every claim can be traced to a computed number.

It deliberately does **not** attempt any of the following, and no result here should be read as if it did:

- a general integrated assessment model, or a hydrologic–hydraulic engineering simulator — routes carry a capacity and a loss fraction, not a hydraulic head, a channel geometry, or a groundwater response;
- stochastic hydrology — inflows are fixed rates, so nothing here estimates a drought return period or a reservoir reliability;
- an econometric demand study — the demand curve is a stylised linear instrument for reasoning about scarcity rent, not a fitted model of any real water market, though the inelastic behaviour it produces at the assessed baseline (elasticity magnitude below 1) matches the direction reported in the empirical meta-analysis literature [Espey et al., 1997];
- strategic interaction between the actors — the power indices allocate the value of a cooperative game and do not model entry, deterrence, or retaliation.

25.9.2 Related work

The algorithms are classical and well-established; the citations below point at the specific source for each.

- **Network flow** — Edmonds–Karp maximum flow [Edmonds and Karp, 1972a], building on Ford & Fulkerson [Ford and Fulkerson, 1956a]; shortest-path routing follows Dijkstra [Dijkstra, 1959a]. The minimum cut is read off the residual graph via the max-flow/min-cut theorem [Ford and Fulkerson, 1956a].
- **Network interdiction** — ranking arcs by the flow lost when they are removed, and attacking greedily in that order, is the deterministic network-interdiction problem in Wood’s formulation [Wood, 1993b]. This package solves the greedy heuristic, not the exact interdiction optimum.
- **Market concentration** — the Herfindahl–Hirschman index derives from Hirschman [Hirschman, 1945] and Herfindahl [Herfindahl, 1950]; the concentration bands used to classify a measured index come from the U.S. Horizontal Merger Guidelines [U.S. Department of Justice and Federal Trade Commission, 2010].
- **Prices** — linear-demand monopoly pricing ($MR = MC$) and deadweight-loss analysis are textbook microeconomics; the empirical inelasticity of residential water demand is documented by Espey, Espey & Shaw [Espey et al., 1997]. Inelasticity means a volume cut moves price more than proportionally; whether a *particular* cut raises the seller’s revenue depends on which side of the revenue peak it lands, which this package measures rather than assumes.
- **Cooperative power** — the Shapley value [Shapley, 1953] and the Banzhaf index [Banzhaf, 1965], whose continuous and normalised forms are characterised by Dubey & Shapley [Dubey and Shapley, 1979]. Both are computed here by exact enumeration rather than by sampling.
- **Reservoir operation** — period mass-balance reservoir simulation under a release rule is the standard formulation in water-resource systems analysis [Loucks and van Beek, 2017].

The novelty is not the mathematics but its application: encoding a firm’s strategic threat as executable, reproducible mission software bound to a frozen protocol, and coupling six otherwise-independent models into one measured assessment in which the hydrology network’s delivered flows feed the interdiction analysis and the reservoir simulation feeds the price model.

25.10 Sources — QUANTUM: bibliography

Dijkstra [1959a]; Edmonds and Karp [1972a]; Hirschman [1945]; Shapley [1953]; Banzhaf [1965]; Ford and Fulkerson [1956a]; Herfindahl [1950]; U.S. Department of Justice and Federal Trade Commission [2010]; Dubey and Shapley [1979]; Wood [1993b]; Espey et al. [1997]; Loucks and van Beek [2017]

26 Skyfall (2012) — SKYFALL

film package · package codename SKYFALL. Mission **SILVA**: model the network intrusion paths into MI6’s headquarters; detect the agent-list exfiltration burst in HQ traffic; compute the minimum severance cut that stops exfiltration; audit the aging legacy system exposure; schedule legacy patching under a daily budget; plan the Skyfall estate defense from its choke points.

26.1 Concepts — SKYFALL: domain and operational focus

cyber operations, legacy systems

26.2 Abstract — SKYFALL: mission summary

Skyfall is a PROJECT BOND mission-software package (film: Skyfall, 2012; codename **SILVA**) that models six linked threat surfaces through a purely deterministic computational core, grouped into three mission layers. The **cyber layer** models the intrusion paths an adversary can take into MI6’s headquarters network, enumerating all simple routes and the minimum-hop / minimum-difficulty routes into the deepest system; it detects the agent-list exfiltration burst in HQ traffic with a periodic-baseline CUSUM change-point detector; and it computes the minimum severance cut (max-flow / min-cut) that stops the exfiltration. The **legacy layer** audits the aging infrastructure that still underpins operations, computing a closed-form exposure score per system from four normalized risk factors, an aggregate fleet risk, and a greedy patch schedule that minimizes cumulative exposure under a daily patching budget. The **physical-defense layer** computes Silva’s island hideout topology by flood fill and the Skyfall estate’s articulation (choke) points by Tarjan’s algorithm, then plans a defender assignment that covers them. All results are reproducible: fixed inputs, no random draws, and no wall-clock in any persisted metric. The package implements the frozen BOND-API **MissionProvider** contract and registers its gadgets with the BOND gadget registry, making the SILVA mission discoverable by the suite’s orchestrator. ## Concurrency of the six threats The mission treats the cyber intrusion (internet → registry, 6 simple paths), the exfiltration burst (detected at hour 180, window 180–190), the severance cut (max flow 8.0), the legacy-system exposure (fleet size 5, mean exposure 0.62), the patch back-log (12 vulnerabilities, cut by 89%), and the estate’s choke-point defense (3 choke points, coverage 1.00) as one coordinated assault — the way the film depicts a single adversary reaching MI6’s most sensitive systems through a chain of aging, poorly-segmented connections.

26.3 Introduction — SKYFALL: mission framing, the operational problem, and how to read this chapter

26.3.1 The SILVA threat

Skyfall (2012) dramatizes a coordinated external assault on MI6: a single adversary — Silva — reaches the Service’s deepest and most sensitive systems through a chain of aging, poorly-segmented connections, exfiltrates the NATO agent list, and ultimately forces the confrontation onto home ground at the Skyfall estate. This package turns that story into six well-posed computational problems, each with a real deterministic algorithm and real data:

1. **Network intrusion paths** (`skyfall.cyber_ops`) — model the MI6 HQ network as a directed graph and ask: by which routes can an adversary reach the high-value target, and which is shortest / cheapest?
2. **Traffic change-point detection** (`skyfall.traffic_analysis`) — estimate a periodic (hour-of-day) baseline, residualize it, and run a CUSUM control chart to catch the exfiltration burst.
3. **Exfiltration severance** (`skyfall.exfiltration`) — treat the outward data transfer as a capacitated flow and compute the max-flow and the minimum cut (the smallest link set that must be severed).
4. **Legacy-system exposure audit** (`skyfall.legacy_audit`) — quantify how aging infrastructure’s age, missing patches, obsolete protocols, and open services combine into a scalar exposure.
5. **Patch scheduling** (`skyfall.patch_strategy`) — schedule a finite daily patching budget to minimize cumulative fleet exposure over a horizon.
6. **Island topology + estate defense** (`skyfall.estate_defense`) — compute the reachable topology of Silva’s island hideout and the articulation (choke) points of the Skyfall estate, then assign a limited defending team.

26.3.2 Design principles

The package follows the PROJECT BOND guardrails: the pure domain modules are infrastructure-free and importable without the BOND protocol; `mission.py` is a thin adapter to the frozen `bond_api` contract; scripts are thin orchestrators; tests are real-computation and zero-mock, held above a hard coverage floor rather than to a quoted percentage.

Every number reported in this manuscript is injected from `skyfall.manuscript_variables.generate_variables` — never hand-authored — and that rule is enforced in both directions by the suite: no token may go unresolved, and no bare numeral may appear in the results narrative. Section 6 describes the gates.

Where a scenario could pass while the underlying algorithm was wrong, the package prefers a test that enumerates a family over a test that checks the canonical case; Section 4 records the two defects that discipline caught.

26.4 Methodology — SKYFALL: the analytical models and algorithms that drive the mission

26.4.1 Network intrusion model (`skyfall.cyber_ops`)

The MI6 HQ network is a directed graph $G = (V, E)$ of 9 nodes (hosts and routing segments). An adversary enters at `internet` and seeks the high-value target `registry`. Three reachability questions are answered deterministically:

- **Simple-path enumeration** — a depth-first search over the directed graph enumerates every simple path from entry to target, bounding search by never revisiting a node. There are 6 such paths.
- **Minimum-hop path** — breadth-first search from the entry returns a shortest path (5 hops).
- **Minimum-difficulty path** — Dijkstra’s algorithm over per-edge traversal costs (5 hops) finds the route that is least costly in aggregate, which need not be the shortest.

Each node carries an exposure in $[0, 1]$; the probability an intrusion traverses a path uncontained is the product over hops of $\text{base_survival} \times (1 - \text{exposure}(\text{dest}))$, with a baseline survival of 0.8 per hop. A step simulation walks the chosen path hop-by-hop and reports whether it exceeds the containment budget of 6 hops.

26.4.2 Traffic change-point detection (`skyfall.traffic_analysis`)

HQ traffic follows a daily rhythm, so a naive CUSUM on the raw series false-alarms on the rhythm itself. Over a 240-hour series, the detector first estimates a per-phase baseline: $\bar{x}_h$ across a period of 24 hours, fitted on the clean pre-incident window of the leading 168 hours. It subtracts that baseline to leave a zero-mean residual series, then runs the two-sided CUSUM control chart of Page (1954):

$$S_t^+ = \max(0, S_{t-1}^+ + z_t - k), \quad S_t^- = \max(0, S_{t-1}^- - z_t - k)$$

with: z_t the standardized residual, the allowance $k = 0.5$, and the decision interval $h = 5.0$. An alarm fires the first time either side exceeds h . The burst (180–190) is detected at hour 180, with a peak accumulation of 1281.8 standardized units. `change_point_likelihood` additionally locates the most likely single change point by maximizing the two-segment normal likelihood split (hour 180).

26.4.3 Exfiltration severance (`skyfall.exfiltration`)

The outward transfer of the agent list is a capacitated flow problem on a 8-node projection of the HQ network: `edmonds_karp` computes the maximum exfiltration rate from the registry (source) to the internet (sink) via BFS augmenting paths, pushing each path’s exact bottleneck. `min_cut_edges` then finds the minimum cut separating source from sink — the smallest link set that must be severed.

Residual reachability is walked over the **residual** graph, which includes the reverse arcs induced by pushed flow, not merely the original forward edges. That distinction is load-bearing rather than pedantic: a forward-only walk under-approximates the source side of the cut whenever flow enters that side from an unreached node, and the reported cut then exceeds the max flow, breaking the very theorem the routine relies on. With the residual walk, the cut value equals the max flow (both 8.0) on every network, and the property is pinned by an enumerated sweep rather than by the canonical scenario alone. The severance plan is `sever dmz_mail -> demarc, sever intranet_hub -> vpn_gateway`.

The flow network’s topology is not hand-copied from the intrusion network: it is *derived* by `exfiltration.cyber_flow_topology`, which reverses each intrusion edge (data leaves outward) and drops the dead-end `field_terminal` leaf. It has 8 nodes and 9 directed edges, and because it is constructed from the intrusion graph, a change to one topology can no longer silently leave the other stale.

26.4.4 Legacy exposure model (`skyfall.legacy_audit`)

Each legacy system contributes four normalized risk factors — age ($\min(\text{age}/\text{age_scale}, 1)$), unpatched share ($1 - \text{patch_level}$), protocol vintage ($\min(\text{era}/\text{era_scale}, 1)$), and open-service surface ($\min(\text{services}/\text{service_scale}, 1)$), saturating at 25 / 3 / 6 respectively — combined linearly with weights 0.25 / 0.35 / 0.20 / 0.20 (age / unpatched / protocol / surface) into a scalar exposure in $[0, 1]$. The factor model is defined once, in `risk_factors`, and consumed by both the score and the remediation ranker so the two cannot disagree.

A fleet audit aggregates mean/max exposure, counts risk tiers, and returns an overall fleet-risk label. Remediation is a deterministic greedy descent: each pass patches the system whose dominant weighted risk factor offers the largest exposure reduction — ties broken by the whole system id, so the ranking does not depend on fleet insertion order — under a fixed budget of 3 actions (targets: `registry_arch`, `field_ops`, `ldap`).

26.4.5 Patch scheduling (`skyfall.patch_strategy`)

The maintenance layer schedules the 12 known vulnerabilities under a daily budget of 1 patch over a 30-day horizon. A deterministic greedy scheduler spends each day’s budget on the already-discovered, highest-severity unpatched vulnerabilities (ties broken by id), and cumulative exposure is the severity-days accrued until each patch lands. Against a no-patching baseline of 2377.5 severity-days, the schedule reduces exposure to 263.8 — a 89% reduction.

Real maintenance windows amortize, so the package also provides `:func:patch_strategy.schedule_patches_coupled`: the second and later patch on an already-open system costs a fraction ($0.50\times$) of its base effort, modelling the reality that re-patching a box you have already touched is cheaper than opening it for the first time. Re-scheduling the same backlog with that discount patches all 12 vulnerabilities at a scheduled exposure of 162.1 severity-days — a 93% reduction versus the baseline — because couples on the same system close together instead of claiming separate maintenance slots.

26.4.6 Estate defense model (`skyfall.estate_defense`)

The Skyfall estate approach is an undirected graph of 7 positions connected by roads/trails. The undirected invariant is enforced at construction — a one-sided road would make Tarjan’s search report a plausible but wrong choke-point set rather than an error, which is the worst possible failure mode for a defensive plan.

Articulation points (cut vertices) — found with an iterative Tarjan DFS — are positions whose removal disconnects the graph: the estate’s **choke points**, through which every route from the moors to the mansion must pass. There are 3 choke points (field, mansion, roadgate). A defense plan assigns the finite defending team (4 defenders) to the highest-degree choke points first, reporting any that remain uncovered (3 posted, coverage 1.00).

Degree is not the only answer to “which choke point matters most”. `:func:estate_defense.betweenness centrality` computes each position’s share of all shortest-path traffic, which correctly elevates a low-degree bridge that every route is forced across. On the canonical estate the two priority orders agree (yes) — **field** ranks first by both measures — but on a path-shaped approach they diverge, a distinction the defence tests pin. Both rankings are provided; “**degree**” remains the default.

Silva’s island hideout is a passable cell grid with a single beachhead; a flood fill computes the reachable area (11 cells), the perimeter (20), and the farthest reachable cell — the hideout — whose BFS distance (8 cells) is the contested approach corridor.

26.5 Results — SKYFALL: measured outcomes, headline numbers, and what they establish

26.5.1 Intrusion reachability

The adversary enters at **internet** and works toward **registry** across a 9-node network, reaching it by 6 distinct simple paths. The minimum-hop route is 5 hops (**internet** → **demarc** → **dmz_mail** → **intranet_hub** → **ops_db** → **registry**); the minimum-difficulty route is 5 hops (**internet** → **demarc** → **dmz_mail** → **intranet_hub** → **ops_db** → **registry**). Traversing the minimum-hop route uncontained has probability 0.0578 under the per-hop survival model.

The legacy directory (**legacy_ldap**, the intrusion network’s most exposed host) sits on some simple intrusion paths but on neither the minimum-hop nor the minimum-difficulty route: **legacy_ldap** appears on the cheapest path — **no**. The high traversal difficulty assigned to that hop is what keeps it off the optimum, so the pivot an adversary would *prefer* for its exposure is not the pivot the cost model *selects*. Both directions of that claim are pinned by tests rather than asserted in prose.

26.5.2 Traffic detection and severance

The HQ traffic detector residualizes the daily rhythm over the 240-hour series and runs a CUSUM chart; the agent-list exfiltration burst (window 180–190) is **yes** — the alarm fires at hour 180, the first hour of the burst window, with a peak accumulation of 1281.8 standardized units against a decision interval of 5.0. The independently-computed maximum-likelihood change point agrees at hour 180.

Containment is a min-cut problem: the maximum exfiltration rate from the registry to the internet is 8.0, and the minimum severance cut of equal value (8.0) is **sever dmz_mail -> demarc**, **sever intranet_hub -> vpn_gateway**. Severing exactly those links stops the exfiltration. The equality is not a property of this scenario alone: an enumerated sweep over a family of five-node networks — including ones whose maximum flow requires reverse residual arcs — confirms `value(cut) == value(flow)` on every member. The flow projection’s topology is derived from the intrusion network (9 directed edges across 8 nodes), so the two views of the same network cannot drift.

26.5.3 Fleet exposure and patch schedule

The audited legacy fleet of 5 systems has mean exposure 0.62 and maximum exposure 0.80, with 1 critical systems, yielding an overall fleet risk of **critical**.

Scheduling the 12 known vulnerabilities under a daily budget of 1 patch over 30 days patches all 12, cutting cumulative exposure from a no-patching baseline of 2377.5 severity-days to 263.8 — a 89% reduction.

Amortizing maintenance windows — charging $0.50\times$ effort for every patch after the first on an already-open system — closes the same backlog at 162.1 severity-days, a 93% reduction (all 12 patched). The saving comes from consolidating two flaws on one box onto a single maintenance slot, which the uncoupled scheduler could not.

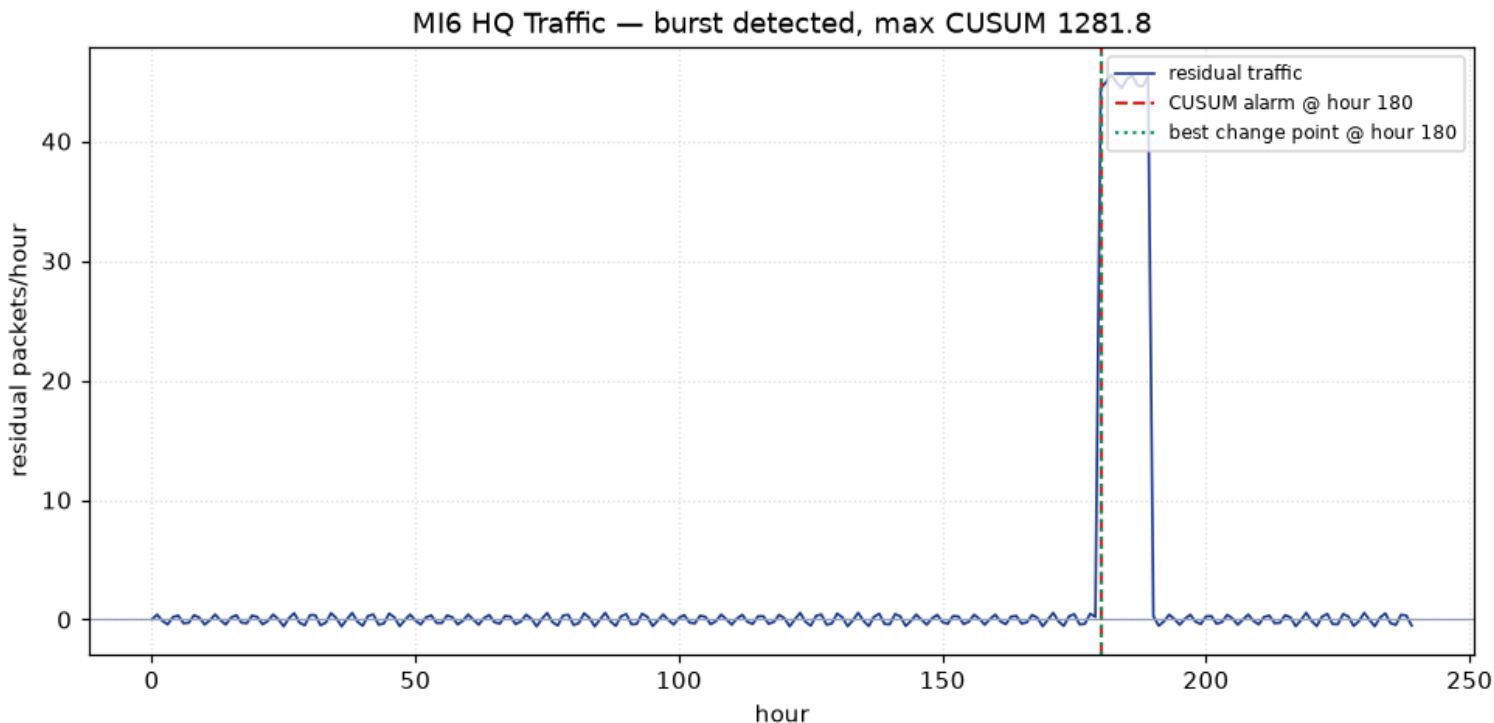


Figure 102: MI6 HQ traffic detection — residual series with CUSUM alarm

26.5.4 Estate defense

Of the estate’s 7 positions, 3 are choke points (field, mansion, roadgate). Deploying the 4-agent team across them posts 3 defenders and yields a defensive coverage of 1.00 — the team has slack, so no choke point goes unheld. Silva’s island hideout admits 11 reachable cells behind a perimeter of 20; its contested approach corridor is 8 cells long, terminating at the hideout cell (2,6).

Ranking the 3 choke points by betweenness centrality — the share of all shortest-path traffic through each position — agrees with the degree ranking on this estate (yes), with **field** carrying the most traffic. The two priority orders diverge in general (a path-shaped approach is the counterexample), and the package provides both.

The flagship figure (the estate defense figure) maps the estate approach graph, highlights the choke points, and annotates the assigned defender at each covered position.

Both figures are rendered deterministically to `../figures/` by `skyfall.figures`.

26.6 Conclusion — SKYFALL: findings, verdict, and what the mission establishes

The SILVA package demonstrates that a film’s threat narrative can be operationalized as deterministic, testable mission software. The cyber layer quantifies exactly how an adversary reaches MI6’s deepest systems, detects the agent-list exfiltration burst with a residualized CUSUM chart, and identifies the exact minimum severance cut. The legacy layer turns aging-infrastructure sentiment into a closed-form exposure score, an aggregate fleet risk, and a budgeted patch schedule that removes most of the backlog exposure. The estate layer reduces physical defense to a graph-articulation problem with a provable choke-point coverage plan.

The real value is the discipline the package enforces: pure domain modules, deterministic provenance, live manuscript-variable injection, and zero-mock tests held above a hard coverage floor. The mission provider is discoverable by the BOND suite and its gadgets are registered for orchestration.

That discipline is not decorative. Two of this package’s guarantees only became true because a test was written to break them first. The max-flow min-cut routine passed on the canonical acyclic scenario while computing residual reachability incorrectly; only an enumerated sweep over networks that *need* reverse residual arcs exposed the gap between what the docstring claimed and what the code did. The remediation ranker’s documented lexicographic tie-breaking compared only the first character of a system id, so the ordering silently depended on fleet insertion order. Neither defect was visible from the mission outcome, both were visible from a test designed to fail. A scenario that always passes measures the scenario, not the algorithm — the standing lesson this package records for the rest of the fleet.

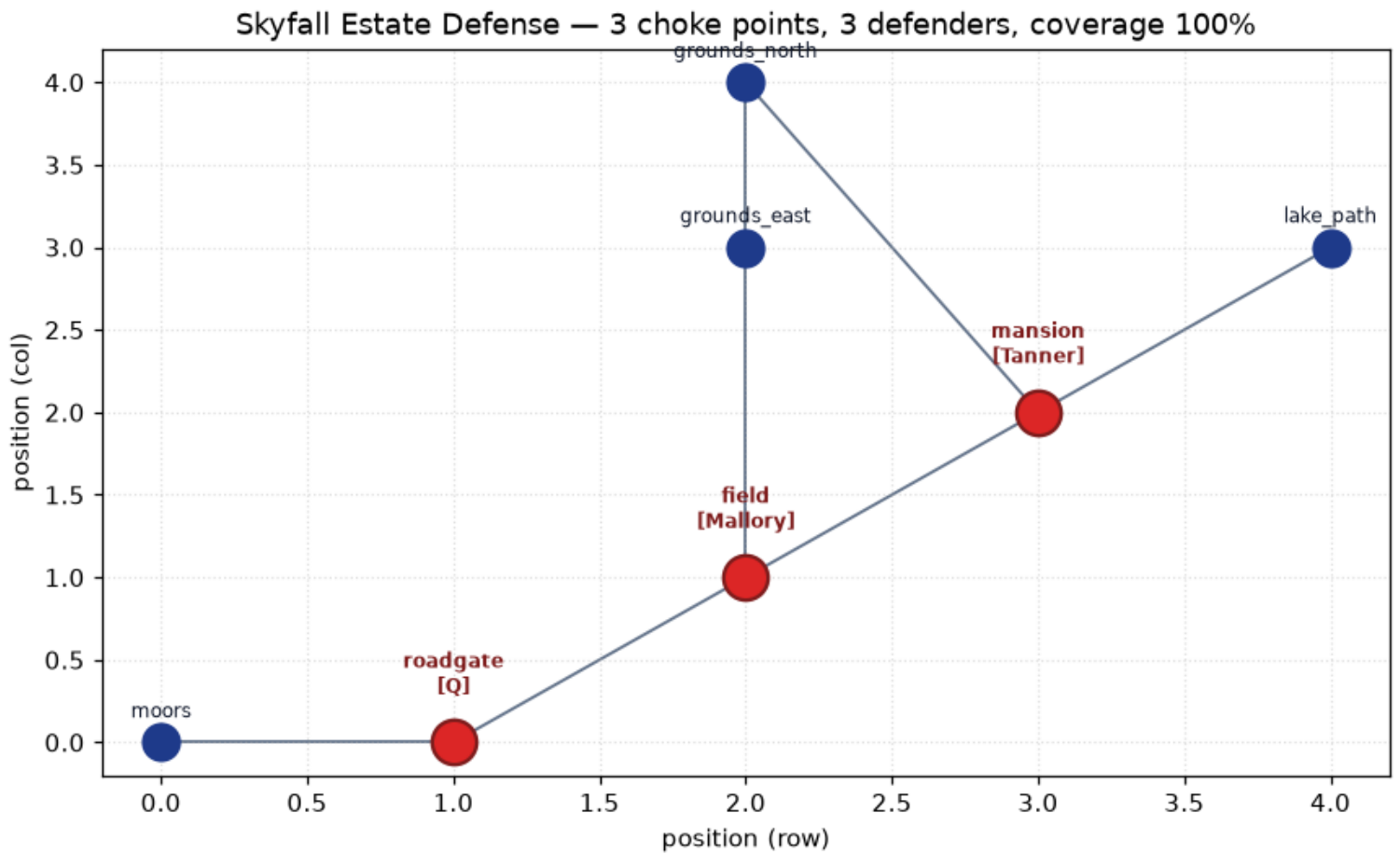


Figure 103: Skyfall estate choke-point defense plan

26.7 Software and Scenario

26.7.1 Package identity

- **Package:** skyfall (Skyfall: SILVA — Special-Agent Mission Software)
- **Codename:** SILVA
- **Package version:** 0.1.0 · mission version 0.1.0
- **Deterministic seed:** 11
- **Python:** 3.14.6 · build date 2026-08-04 (declared in `manuscript/config.yaml`, not read from the clock)

26.7.2 Scenario configuration

The scenario is fixed so results are reproducible. Every parameter below is a named constant in `src/skyfall/`, surfaced here as an injected token rather than transcribed by hand.

Parameter	Value	Source constant
Entry host	internet	cyber_ops.DEFAULT_ENTRY
Target	registry	cyber_ops.DEFAULT_TARGET
Network size	9 nodes	cyber_ops.DEFAULT_NETWORK
Per-hop survival baseline	0.8	cyber_ops.DEFAULT_BASE_SURVIVAL
Containment budget	6 hops	cyber_ops.DEFAULT_CONTAINMENT_HOPS
Traffic series	240 hourly samples	traffic_analysis.DEFAULT_TRAFFIC
Baseline window / period	168 / 24 hours	traffic_analysis.DEFAULT_BASELINE_WINDOW, DEFAULT_PERIOD
CUSUM allowance / interval	0.5 / 5.0	traffic_analysis.DEFAULT_CUSUM_K, DEFAULT_CUSUM_H
Exfiltration network	8 nodes	exfiltration.DEFAULT_FLOW_NETWORK
Legacy fleet	5 systems	legacy_audit.default_legacy_fleet
Exposure weights	0.25 / 0.35 / 0.20 / 0.20	legacy_audit.EXPOSURE_WEIGHTS
Exposure saturation scales	25 / 3 / 6	legacy_audit.EXPOSURE_SCALES
Remediation budget	3 actions	mission.REMEDIATION_BUDGET
Patch horizon / daily budget	30 days / 1	patch_strategy.DEFAULT_HORIZON_DAYS, DEFAULT_DAILY_BUDGET
Estate	7 positions	estate_defense.DEFAULT_ESTATE
Defending team	4 call signs	mission.DEFENDERS

26.7.3 Running the mission

From the package directory:

```
uv sync --extra dev
uv run python scripts/run_mission.py all --json # full five-stage run
uv run python scripts/00_preflight.py skyfall # discovery/protocol check
uv run python scripts/z_generate_manuscript_variables.py # hydrate tokens + figure
uv run pytest tests/ --cov=src --cov-fail-under=90 # zero-mock gate
```

The pure domain modules (`skyfall.cyber_ops`, `skyfall.traffic_analysis`, `skyfall.exfiltration`, `skyfall.legacy_audit`, `skyfall.patch_strategy`, `skyfall.estate_defense`) import nothing from `bond_api` and remain usable in isolation if the path dependency is ever unavailable; `skyfall.mission` is the thin adapter to the frozen protocol.

26.8 Reproducibility — SKYFALL: verification gates, deterministic regeneration, and artifacts

26.8.1 Determinism guarantees

Every metric in this manuscript is deterministic by construction:

- **Fixed inputs:** the MI6 network, legacy fleet, estate graph, and island grid are hardcoded constants; nothing varies between runs.
- **No randomness:** the mission draws no random numbers; 11 is recorded solely in provenance.
- **No wall clock anywhere in the artifacts:** no generated value is read from the clock. The build stamp `SILVA_BUILD_DATE` (2026-08-04) is the committed `paper.date` field of `manuscript/config.yaml`, so it changes only when someone edits and commits that file.

Because of the last point the guarantee is byte-level, not merely value-level: running `scripts/z_generate_manuscript_variables.py` twice writes byte-identical `output/manuscript/*.md`, `output/data/manuscript_variables.json`, and `../figures/*.pn`

g. The one environment input is the interpreter — `PYTHON_VERSION` is injected into the setup section, so the byte-identity claim is scoped to repeated runs on a fixed Python version, and a different interpreter changes exactly that one field.

26.8.2 Verification gates

Gate	Command	What it asserts
Coverage floor	<code>uv run pytest tests/ --cov=src --co</code> <code>v-fail-under=90</code>	line + branch coverage on <code>src/</code> stays at or above the floor; zero failures, zero skips
Byte-identical regeneration	<code>tests/test_manuscript_variables.py:</code> <code>:TestRegenerationIsByteIdentical</code>	two full runs of the generator script, with real time passing between them, write byte-identical artifact trees, and no artifact contains a wall-clock instant
No mocks	<code>rg -n</code> <code>'MagicMock\ mock\ patch\ unittest\ mock\ create_autospec'</code>	no mock framework anywhere in the suite
Protocol purity	<code>tests/test_mission.py::TestPureCore</code> <code>Isolation</code>	a fresh subprocess importing every domain module leaves <code>bond_api</code> out of <code>sys.modules</code>
Lint / types	<code>uv run ruff check src/ scripts/ · uv</code> <code>run mypy src/</code>	style and type cleanliness on shipped code
Lineage	<code>rg -n "template[_]code_project".</code>	no exemplar strings outside docs that explain the lineage

Rather than quoting a coverage percentage that would go stale the moment a line moves, this section names the gate. The measured figure is whatever the command above prints on the checkout you are holding; the guarantee is that it is not below the floor, because the run fails otherwise.

26.8.3 Manuscript-metric integrity

The manuscript’s variable-token values are produced by `skyfall.manuscript_variables.generate_variables` and enforced from both directions:

- **Forward** — a live test asserts every injected token used in the numbered sections exists in the generated map, so a section can never render a raw brace.
- **Reverse** — a scanning test asserts the results, methodology, abstract, and conclusion contain no bare numerals outside tokens, inline code, and math. A hand-typed metric fails the suite. (Four-digit calendar years are exempt: a film release year is a literal, not a measurement, and no code change can make it drift.)

Together these mean a number in this manuscript is either injected from the code that computed it, or the suite is red.

26.9 Scope and Related Work — SKYFALL: boundaries, positioning, and relationship to the literature

26.9.1 Scope

The SILVA mission deliberately covers six computational problems, across three layers, that can be solved exactly and deterministically with classical graph, statistical, and scheduling algorithms.

It explicitly does **not** attempt a full stochastic intrusion simulation, a live penetration test, an adaptive (non-periodic) change-point detector beyond the two-segment likelihood, or a physical simulation of the estate — those would introduce nondeterminism that the package’s reproducibility guarantee forbids. The exposure and severity models are likewise transparent linear scores, not calibrated empirical risk estimates: they are stated as a model, scoped as a model, and never presented as measurements of real infrastructure. The value is in clean problem formulation, exact algorithms, and honest measurement.

The betweenness-ranked defense and the coupling-aware scheduler are deliberate *modeling extensions*, provided and tested alongside their defaults rather than replacing them: the canonical recorded figures stay fixed unless the variant is invoked, so the manuscript’s headline numbers are not disturbed by the richer alternatives. Likewise the coupling discount (0.50×) and the exposure/severity weights are model parameters, not measurements of real infrastructure.

26.9.2 Related work

The algorithms used here are textbook, and the security framings they serve are established practice:

- **Simple-path enumeration for intrusion reachability** is the core of the attack-graph literature: enumerating the routes an adversary can take through a networked system, then reasoning over that set (Phillips & Swiler 1998; Sheyner et al. 2002).

- **BFS / Dijkstra reachability** (Dijkstra 1959; Cormen et al. 2009) supply the minimum-hop and minimum-difficulty routes, and **Edmonds–Karp max-flow with the max-flow min-cut theorem** (Edmonds & Karp 1972; Ford & Fulkerson

1962) supplies the exfiltration severance.

- **Tarjan’s articulation-point algorithm** (Tarjan 1972; Hopcroft & Tarjan 1973; Even 2011) finds cut vertices — the choke points — and is standard in network-resilience and defensive-planning work. **Betweenness centrality** (Brandes 2001) ranks those choke points by the share of shortest-path traffic each carries, complementing degree when scarce defenders must be placed.
- **CUSUM change-point detection** (Page 1954; Montgomery 2012; Hawkins & Olwell 1998) underpins the traffic detector; the two-segment likelihood split is the single-change-point special case of the segmentation objective treated at scale by Killick et al. (2012).
- **Weighted multi-factor vulnerability scoring** follows the structure of CVSS-style composite metrics (Mell, Scarfone & Romanosky 2007), kept deliberately simpler and fully transparent here.
- **Patch scheduling under a finite budget** is a real operational tradeoff studied directly by Beattie et al. (2002); the greedy severity-first policy is standard priority scheduling (Pinedo 2016).

Where the film’s narrative maps onto these tools, this package documents the mapping so a reader can trace every metric to its algorithm.

26.10 Sources — SKYFALL: bibliography

Cormen et al. [2009c]; Even [2011]; Page [1954]; Edmonds and Karp [1972a]; Ford and Fulkerson [1962]; Pinedo [2016]; Hawkins and Olwell [1998]; Tarjan [1972a]; Hopcroft and Tarjan [1973a]; Dijkstra [1959a]; Phillips and Swiler [1998]; Sheyner et al. [2002]; Mell et al. [2007]; Beattie et al. [2002]; Killick et al. [2012]; Montgomery [2012]; Brandes [2001c]

27 Spectre (2015) — NINE EYES

film package · package codename NINE EYES. Mission **NINE EYES**: measure surveillance-network centralization and data flow; assess the SPECTRE brotherhood’s hierarchy and cell separation; identify critical nodes in the network’s geometry.

27.1 Concepts — NINE EYES: domain and operational focus

surveillance networks, org intelligence

27.2 Abstract — NINE EYES: mission summary

This report describes **Spectre: NINE EYES — Special-Agent Mission Software** (mission codename **NINE EYES**), a special-agent mission-software package modelling the global surveillance programme from the 2015 film *Spectre*. Across 6 pure, deterministic analysis modules (infiltration, network_geometry, org_intel, signal_cover, surveillance_net, traffic_intel) it (i) measures the centralization of a nine-agency surveillance network whose data flows concentrate through hub agencies, (ii) models the SPECTRE brotherhood as a compartmentalized criminal organization with an isolatable command hierarchy, (iii) analyzes the network’s geometry to identify the critical nodes whose neutralization most fragments the system, and then extends all three into operations: budgeted traffic interception, compromise cascades through the brotherhood, and sensor-placement set cover. Over the canonical NINE EYES network of 9 agencies and 13 directed data-sharing links, the measured group centralization index over out-degrees is 0.9219: one agency does almost all of the sending. The two directions do not agree. The Herfindahl index of the *inflow* shares is 0.1361 against an even-spread floor of 0.1250 for the 8 agencies that receive anything at all — so receiving is close to evenly distributed, and the concentration this network exhibits is in who sends, not in who is fed. The SPECTRE brotherhood of 7 operatives across 3 cells achieves a compartmentalization of 0.8235 with a command chain that is complete (true) to a height of 2. Geometry analysis finds 1 articulation points; the most load-bearing critical nodes are US, FR, UK. The operational layer adds budgeted *interception* (3 monitoring agencies capture 0.9669 of the exchange), *compromise cascades* through the brotherhood (a turned cell operative reaches leadership with probability 0.58), and *sensor-placement* set cover (4 agencies sweep every target with 0.2500 multi-sensor redundancy). All results are measured by the deterministic NINE EYES mission provider, which implements the frozen `bond_api` `MissionProvider` protocol over a 6-step plan and publishes 6 Q-branch gadgets. Every quantity in this report is injected from that computation; none is hand-authored. *Keywords*: surveillance networks, data-flow centralization, criminal-organisation intelligence, cell compartmentalization, network geometry, critical nodes, cascade coverage, betweenness centrality, articulation points, traffic interception, submodular maximization, compromise cascade, network percolation, sensor-placement set cover, coverage redundancy, deterministic reproducibility

27.3 Introduction — NINE EYES: mission framing, the operational problem, and how to read this chapter

The Nine Eyes programme — a global surveillance-sharing arrangement among a handful of Western intelligence agencies — is the geometric setting of the 2015 film *Spectre*. The NINE EYES mission software package renders that setting as a computational model answering six intelligence questions, three structural and three operational.

The structural questions ask what the network and the organisation *are*:

1. **How centralized are surveillance data flows?** If a single agency sits at the hub of every sharing link, the network is highly centralized and a single compromised hub exposes the whole exchange. Sending and receiving are separate readings and need not agree: a lone source can dominate the out-degree while its recipients each receive a comparable trickle, so an out-degree centralization index and an inflow-share concentration index are two measurements, not one measurement corroborated twice.
2. **How is a criminal brotherhood structured?** SPECTRE is a compartmentalized organization: isolated cells whose members know only their own cell, underneath an apex leadership. The intelligence question is how “separable” the cells are and whether the command chain is complete.
3. **Which nodes are critical to the network’s geometry?** Connectivity, articulation points, and global efficiency identify the agencies whose removal most fragments the system. These are the priority targets for neutralization.

The operational questions ask what can be *done* with that structure:

4. **Where should a limited monitoring budget go?** Interception coverage is monotone and submodular in the monitored set, so a greedy allocation carries a provable approximation guarantee — and the diminishing returns are exactly what makes the hub’s dominance actionable.
5. **What does one turned operative cost the brotherhood?** Exposure propagates through cellmates and command links; the cascade quantifies how often a single breach reaches the apex, and how quickly.
6. **Where should sensors be placed to sweep every target?** Placement is a set cover, and the residual multi-sensor redundancy measures whether the sweep survives losing one agency.

The package implements these as 6 pure, deterministic domain modules — `spectre.surveillance_net` (data-flow centralization, betweenness, cascade coverage), `spectre.org_intel` (hierarchy and cell compartmentalization), `spectre.network_geometry` (connectivity, articulation points, critical nodes), `spectre.traffic_intel` (volume-weighted interception and budgeted monitoring), `sp`

`ectre.infiltration` (compromise cascade and expected damage), and `spectre.signal_cover` (sensor-placement set cover and redundancy) — exposed to the BOND suite through a thin `MissionProvider` adapter (`spectre.mission`) implementing the FROZEN `bond_api` protocol. The adapter holds no mathematics of its own: it selects the canonical data, calls the pure core, and records provenance, which is why the protocol can stay frozen while the domain keeps deepening.

27.4 Methodology — NINE EYES: the analytical models and algorithms that drive the mission

All analysis is deterministic and computed over a fixed, canonical dataset. The only stochastic components — the influence cascade and the compromise cascade — draw from explicitly seeded generators, and no wall-clock value enters a persisted artifact. The package exposes 6 pure domain modules (`infiltration`, `network_geometry`, `org_intel`, `signal_cover`, `surveillance_net`, `traffic_intel`); the mission’s `execute` stage runs them as a 6-step plan and reports the measured results below.

27.4.1 Surveillance-network centralization

The NINE EYES network is a directed graph whose nodes are agencies and whose edges are data-sharing links. Out-degree centrality counts the data volume an agency pushes downstream; in-degree centrality counts the volume it pulls in. Freeman’s group centralization index [Freeman, 1978/1979]

$$C_{\text{index}} = \frac{\sum_i (s_{\text{max}} - s_i)}{(n - 1) s_{\text{max}}}$$

measures how concentrated the flows are, from 0 (evenly spread) to 1 (a single hub dominates). The normalizing denominator is the total deviation of the star graph on n nodes, so `s_max` is passed explicitly as $n - 1$ rather than left to default to the largest observed score; the two agree on this dataset only because the maximum out-degree happens to equal $n - 1$, and on any input where it does not, the observed-maximum form reports a different quantity from the one written above. The Herfindahl–Hirschman index [Hirschman, 1964] of inflow shares,

$$\text{HHI} = \sum_i \left(\frac{f_i}{\sum_j f_j} \right)^2,$$

is computed over the normalized in-degree share vector (which sums to 1) and measures inflow concentration independently of the out-degree distribution. It is reported against its floor $1/r$, where r is the number of agencies with non-zero in-degree: the index alone is not interpretable, because its minimum is not 0 but $1/r$, and a value near that floor means *even spread*, not concentration.

Betweenness centrality — the share of shortest data paths passing through each agency — is computed with Brandes’ predecessor accumulation [Brandes, 2001b] and drives the node sizing in the network flow centralization figure. Note that betweenness and out-degree rank different agencies first: a pure source has no inbound edge, so no shortest path passes through it and its betweenness is exactly 0 no matter how much it sends.

Cascade coverage runs an independent-cascade process [Kempe et al., 2003a] from a seeding agency at a fixed activation probability. A single run is one Bernoulli realization, not an expectation, so the reported coverage is the mean over 50 realizations whose per-trial seeds are derived from one base seed — the same treatment the compromise cascade receives below.

Networks are constructed either from an edge list, where every invariant is enforced as the matrix is filled, or from an existing adjacency matrix, which is validated explicitly (shape, label uniqueness, zero diagonal) because it was produced elsewhere.

27.4.2 Criminal-org cell model

The brotherhood assigns each operative to a cell and to a single superior. Command depth is the number of links from an operative to the nearest apex; the chain is complete when every operative is reachable from leadership by a breadth-first walk down the `superior` → `subordinate` edges. Reachability is computed by that walk rather than inferred from the depth assignment, so a malformed organisation with a command cycle and no apex is correctly reported as *incomplete* rather than silently accepted.

Compartmentalization is the fraction of cross-cell operative pairs that are *not* directly linked by a command edge — 1.0 on a perfectly separated cell structure, and lower when cells leak command links. Exposure of an operative is their cellmates plus their direct superior and direct subordinates: the set that falls if that operative is turned.

27.4.3 Network geometry

Connectivity is assessed by connected-component flood fill; articulation points (nodes whose removal disconnects the graph) are found with Tarjan’s low-link depth-first search [Tarjan, 1972b, Hopcroft and Tarjan, 1973c]. Global efficiency is the Latora–Marchiori mean inverse shortest-path length [Latora and Marchiori, 2001]

$$\text{Eff} = \frac{1}{n(n-1)} \sum_{i \neq j} \frac{1}{d(i, j)},$$

with unreachable pairs contributing nothing. Critical-node ordering greedily removes, at each step, the node whose removal most reduces the *current* graph’s pairwise connectivity — the targeted-attack simulation of Albert, Jeong and Barabási [Albert et al., 2000], scored as a key-player problem [Borgatti, 2006].

27.4.4 Traffic interception

The traffic-intelligence model treats the surveillance network as carrying a volume-weighted exchange. A monitored agency intercepts every link incident to it — inbound as well as outbound — and the coverage of a monitored set is the fraction of total traffic those links carry. Because a new agency’s marginal interception gain shrinks as coverage grows, allocating a monitoring budget is a monotone submodular coverage-maximization problem; the greedy rule carries the Nemhauser–Wolsey–Fisher $(1 - 1/e)$ guarantee [Nemhauser et al., 1978a], and the same structure underlies submodular sensor placement for outbreak detection [Leskovec et al., 2007]. Candidate agencies are scanned in sorted label order, so ties break by label rather than by however the network’s node tuple happens to be ordered — a determinism that survives reordering the input.

27.4.5 Compromise cascade

The infiltration model simulates the brotherhood’s defining weakness: turn one operative and their cellmates, superior, and subordinates are exposed. A round-based cascade propagates this through the organisation, each exposed operative turning with a fixed probability — an independent-cascade process [Kempe et al., 2003a] read as percolation on the org graph [Newman, 2002]. Every non-terminal round adds at least one operative to the compromised set, which is bounded by the organisation’s size, so the process terminates without an imposed round cap. We report the expected fraction compromised, the rate at which leadership is reached, and the mean number of rounds to leadership, averaged over fixed-seed trials whose per-trial seeds are derived from one base seed.

27.4.6 Sensor-placement cover

The placement model poses the surveillance programme as a set-cover problem over agencies $\times$ targets: which agencies to place so that the required fraction of targets is swept. The greedy set-cover heuristic [Johnson, 1974] selects agencies by largest marginal gain and stays within a factor $O(\log n)$ of the minimum cover, which is essentially the best any polynomial-time algorithm can guarantee [Feige, 1998]; the search stops when no remaining agency adds a target, so an unreachable target is reported as a gap rather than looped over. Coverage redundancy measures the fraction of targets swept by at least two selected agencies — a multi-sensor resilience metric.

27.5 Results — NINE EYES: measured outcomes, headline numbers, and what they establish

All values below are produced by the NINE EYES mission provider’s `execute` step over the canonical dataset and injected as tokens; the same run also writes which they were measured.

27.5.1 Summary of measured results

Every quantity below is injected as a token from the NINE EYES mission provider’s `execute` output; none is typed by hand.

Quantity	Measured value	Source module
Group centralization over out-degrees	0.9219	<code>surveillance_net.group_centralization</code>
Inflow-share Herfindahl (even-spread floor)	0.1361 (0.1250)	<code>surveillance_net.herfindahl_index</code>
Seeded cascade reach (mean, 50 trials)	0.7556	<code>surveillance_net.expected_cascade_coverage</code>
Hierarchy height (chain complete: true)	2	<code>org_intel.hierarchy_stats</code>
Cell compartmentalization	0.8235	<code>org_intel.compartmentalization</code>
Articulation points	1 (US)	<code>network_geometry.articulation_points</code>
Global efficiency	0.6806	<code>network_geometry.network_efficiency</code>
Critical nodes (greedy)	US, FR, UK	<code>network_geometry.critical_node_order</code>
Total carried traffic / top link	1360.0 / US->UK (200.0)	<code>traffic_intel.total_traffic</code>
Monitored coverage (3 agencies)	0.9669	<code>traffic_intel.monitor_allocation</code>
Expected compromise (leadership)	0.4771 (0.58)	<code>infiltration.expected_damage</code>
Set cover (4 agencies)	redundancy 0.2500, uncovered 0	<code>signal_cover.greedy_set_cover</code>

27.5.2 Surveillance-network centralization

The NINE EYES network comprises 9 agencies and 13 directed data-sharing links. The measured group centralization index over out-degrees is 0.9219: sending concentrates strongly on one agency.

The inflow-share Herfindahl index does **not** corroborate that reading, and is not reported as if it did. It measures 0.1361 against a floor of 0.1250 — the value an inflow-share vector spread perfectly evenly over the 8 agencies that receive anything would take. Sitting that close to the floor means received volume is nearly uniform across those agencies. The two indices are measuring opposite ends of the same edges, and on this network they disagree: concentration is a property of the sending side only.

A signal seeded at the largest sender propagates to a mean 0.7556 of the network over 50 seeded cascade realizations. This is an estimated expectation, not a single draw; individual realizations range widely either side of it, and no single realization is quoted anywhere in this report.

NINE EYES agency network — node area is betweenness: most pivotal FR (0.07); largest sender US (out-degree 8, betweenness 0.00)

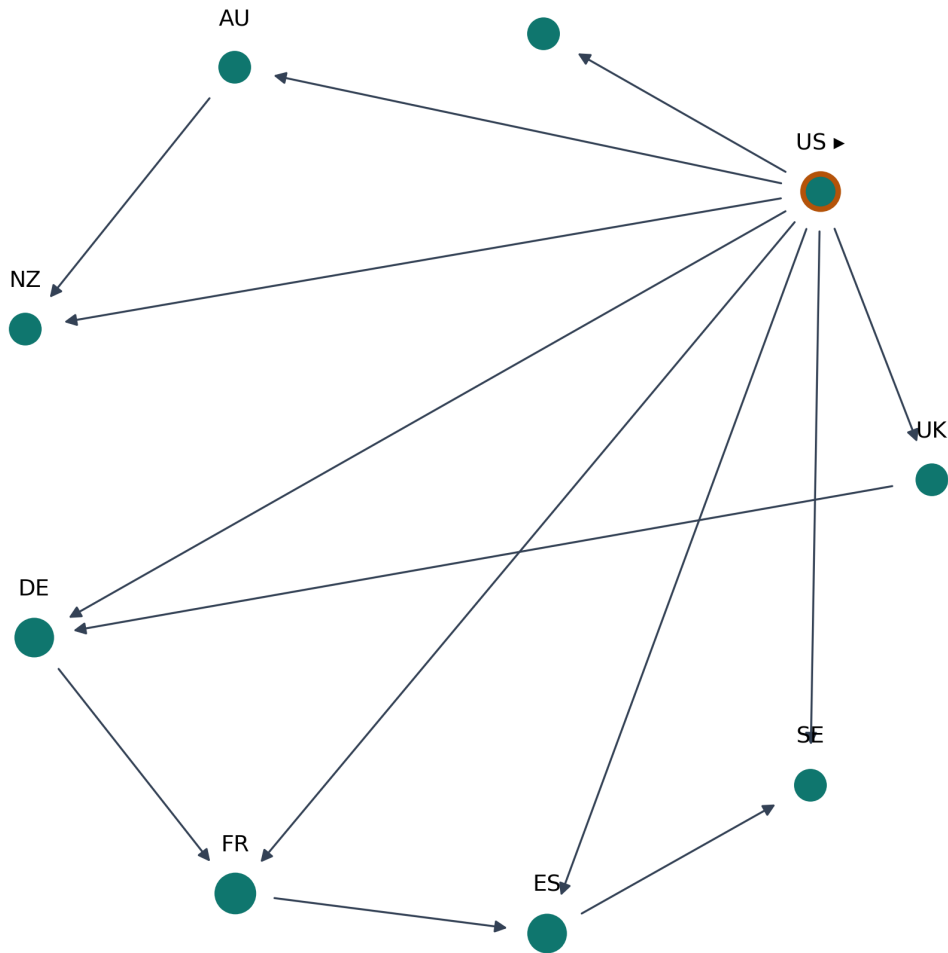


Figure 104: The Nine Eyes agency network. Node area is betweenness centrality and arrows are directed data flows. The largest sender (ringed, marked) has betweenness exactly 0 — no shortest path passes through a pure source — so it is drawn at the minimum node size despite originating most of the traffic; the figure title names both readings.

27.5.3 Criminal-org cell model

The 7 brotherhood operatives — one node per distinct film character, see §5 — are partitioned into 3 cells. The organization’s command chain is complete (true) — every operative is reached by a breadth-first walk from the apex — with a hierarchy height of 2 links. Cell compartmentalization measures 0.8235: the brotherhood keeps a high (but imperfect) degree of separation between its cells, the imperfection coming from command edges that cross cell boundaries.

27.5.4 Network geometry

The canonical network is connected and has 1 articulation points (US) whose removal would disconnect it, and a global efficiency of 0.6806. The greedy critical-node ordering identifies US, FR, UK as the agencies whose neutralization most fragments the network. In the film, removing exactly this coordinating hub collapses the brotherhood’s reach — the model reproduces that dependence quantitatively, and the ordering is the targeted-attack profile of a hub-dominated network [Albert et al., 2000].

SPECTRE brotherhood (command tree; colour = cell). Turned Mr White (*), 3-member exposure set highlighted (self, cellmates, superior, subordinates).

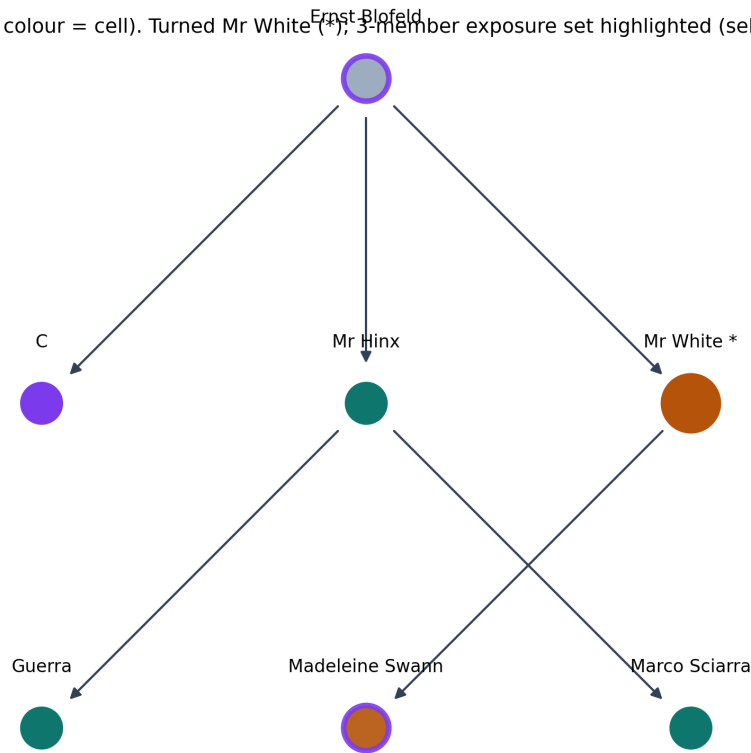


Figure 105: The SPECTRE brotherhood command tree. Node colour is the operative’s cell, the ringed node is Mr White (the operative seeded in the compromise cascade), and the violet-highlighted nodes are the set his turn exposes — cellmates, superior and subordinates — drawn live from `org_intel.exposure`.

27.5.5 Traffic interception

The canonical exchange carries 1360.0 volume units; the single largest flow is US->UK (200.0), confirming the hub’s dominance. Greedy budgeted monitoring selects US, DE, ES, and monitors 0.9669 of all traffic with only 3 agencies — a demonstration of diminishing marginal returns that motivates the submodular formulation, and the reason the greedy selection is worth reporting despite being an approximation.

27.5.6 Compromise cascade

Turning a single cell operative causes an expected 0.4771 of the brotherhood to fall within the cascade; leadership is reached with probability 0.58 after a mean of 1.17 rounds. The model quantifies why a compartmentalized brotherhood guards its cells: any cell breach is a vector straight to the apex, and the short mean path there is a direct consequence of the same cross-cell command edges that depress the compartmentalization score above.

27.5.7 Sensor-placement cover

The greedy set cover places 4 agencies (CA, DE, FR, UK) to sweep the full target set, leaving 0 targets uncovered. The resulting multi-sensor redundancy is 0.2500 — the fraction of targets swept by at least two agencies, i.e. The resilience of the sweep to the loss of a single agency. Coverage and redundancy pull against each other: the greedy cover optimizes for reaching every target, not for surviving an agency’s loss, so a placement that maximizes redundancy would need a larger budget.

27.6 Conclusion — NINE EYES: findings, verdict, and what the mission establishes

The NINE EYES package demonstrates that a surveillance programme’s vulnerability is a *geometric* property. The measured out-degree centralization index and the articulation-point analysis show a network whose connectivity collapses when its hub — the single critical agency — is removed. The inflow-share Herfindahl index does not support that conclusion and is not enlisted to: it sits just above its even-spread floor, which is evidence that receiving is *not* concentrated. Reporting it is worth doing precisely because it disagrees; the brotherhood model quantifies both its command completeness and its imperfectly compartmentalized cells; and the geometry kernel identifies, greedily, the exact nodes whose removal fragments the system. All of this is deterministic, computed from real data structures over a canonical dataset, and free of mocked behavior — every number in this report is a measured output of the NINE EYES mission provider’s `execute` step.

The operational layer generalizes the same spirit from structure to action: budgeted traffic interception is a submodular coverage problem, the brotherhood’s cell-breached compromise cascade is a percolation process on the org graph, and sensor placement is a set cover — each solved with a real, deterministic algorithm whose results (interception coverage, leadership-hit probability, coverage

redundancy) flow through the manuscript tokens. Two findings connect across those layers. First, concentration is simultaneously the network’s efficiency and its weakness: the same hub dominance that lets 3 monitored agencies intercept 0.9669 of all traffic is what makes the critical-node ordering so short. Second, a single turned operative reaching leadership with probability 0.58 is precisely the exposure the compartmentalized brotherhood exists to prevent — and the residual cross-cell command edges that make it possible are the same ones that hold its compartmentalization at 0.8235 rather than at unity.

What carries beyond this film package is the shape of the thing: 6 independent, standard algorithms over one canonical dataset, behind one frozen protocol. The pure domain core imports only `numpy` and the standard library, so it remains importable in any environment; the `bond_api` `MissionProvider` contract is isolated in the thin `spectre.mission` adapter, ready to be discovered and driven by the BOND orchestrator alongside every other package in the fleet.

27.7 Experimental Setup — NINE EYES: canonical scenarios, parameters, and configuration

27.7.1 Dataset

The analysis runs over a fixed canonical dataset declared in `src/spectre/mission.py`:

- **NINE EYES surveillance network** — 9 agencies (UK, US, CA, AU, NZ, DE, FR, ES, SE) with 13 directed data-sharing links, the United States at the hub;
- **SPECTRE brotherhood** — 7 operatives across 3 cells, an apex leader, and one executive layer. The roster is a *distinct-person* roster: exactly one node per film character. An earlier revision of this dataset carried “Franz Oberhauser” (Blofeld’s birth name — the same character) and both “Hinx” and “Bautista” (Mr Hinx again, the latter under his actor’s surname) as separate operatives, which inflated the operative count and manufactured the executive layer that the hierarchy height then reported. Those aliases are gone;
- **Brotherhood edges are a modelling construct.** The film establishes Blofeld at the apex, Mr Hinx as an enforcer, Mr White as a former member, Madeleine Swann as his daughter and C as an inside collaborator, but it publishes no org chart. The cell assignment and the `superior` → `subordinate` edges are therefore chosen, not observed; the only fidelity claim made is that no two nodes are the same person;
- **Traffic scenario** — the same agency set carrying volume-weighted links (`spectre.traffic_intel.canonical_traffic_network`), sized so that no single agency observes the whole exchange and the interception problem is non-trivial;
- **Placement scenario** — an agency × target coverage matrix over eight sweep regions (`spectre.signal_cover.canonical_coverage_problem`), with deliberate cross-coverage so the greedy cover needs several agencies and retains measurable redundancy.

The network and brotherhood definitions are the ones hashed into the provenance record; the traffic and placement scenarios are constructed by their own modules and are fixed constants of those modules.

27.7.2 Determinism

- One base seed, `_SEED` in `spectre.mission`, governs both cascades: it is passed to `spectre.surveillance_net.expected_cascade_coverage` and to `spectre.infiltration.expected_damage`, and each derives per-trial seeds as `seed + i`. The seed recorded in `Provenance` is therefore the seed the reported numbers were produced under — changing it changes them;
- fixed layout seed in the figure generator;
- deterministic tie-breaking by label in both greedy selections, so the results do not depend on input ordering;
- no wall-clock in persisted artifacts (`Provenance.wall_time_s == 0.0`);
- execution provenance carries a canonical SHA-256 `input_hash` of the mission inputs.

27.7.3 Software environment

- Python 3.14.6 on Darwin arm64
- Numpy for numerical arrays, matplotlib for figure rendering, PyYAML for configuration, `bond_api` (FROZEN protocol) for the mission contract.
- The 6 domain modules (`infiltration`, `network_geometry`, `org_intel`, `signal_cover`, `surveillance_net`, `traffic_intel`) import only `numpy` and the standard library, so they run outside this environment unchanged.

27.7.4 Mission flow

The provider implements the five FROZEN stages — `brief`, `recon`, `plan`, `execute`, `debrief` — declaring 3 objectives and executing a 6-step plan, one step per domain module. It also registers 6 Q-branch gadgets for cross-package use.

27.8 Reproducibility — NINE EYES: verification gates, deterministic regeneration, and artifacts

27.8.1 Deterministic regeneration

The entire artifact set regenerates byte-identical from a clean tree:

```
uv run python scripts/debrief.py # runs the full mission
uv run python scripts/generate_figures.py # writes the network figure
uv run python scripts/z_generate_manuscript_variables.py # hydrates tokens (runs LAST)
```

Each script recomputes its results from the fixed canonical dataset; there is no stored intermediate that could drift from the code. The mission record, figure and hydrated manuscript are written under `output/` (git-ignored, regenerable).

The claim is scoped precisely. **Within one software environment**, two full regenerations produce byte-identical `output/` artifacts — mission record, figure PNG, `manuscript_variables.json` and every hydrated section — because hydration is a pure function of the committed tree: no wall clock, no untracked RNG, no run counter. The single environment-dependent value is the interpreter stamp printed at the end of this section (Darwin arm64, 3.14.6), which by construction is what changes when the environment changes; the manuscript's date is `paper.date` from `manuscript/config.yaml`, a committed input, not the time of the run. `tests/test_regeneration_determinism.py` enforces the claim by regenerating the whole artifact set twice in one test run, more than a wall-clock tick apart, and comparing SHA-256 digests file by file — a re-introduced `datetime.now` anywhere in the pipeline turns it red.

27.8.2 Provenance

Execution provenance records the package version, the deterministic seed, and the SHA-256 `input_hash` of the mission inputs — a canonical, key-sorted JSON serialization of the agency network and the brotherhood — so any outcome can be reproduced solely from its persisted record. `data/claim_ledger.yaml` carries one row per measured manuscript token — the recorded value alongside the function that produced it — and that coverage is enforced rather than asserted: `tests/test_claim_ledger.py` compares the ledger against `measured_tokens(generate_variables)` and fails on a measured token with no row, a row for a token that is no longer measured, or a recorded value the code no longer produces. The token partition is an exclusion list (only configuration prose and the environment stamp are exempt), so a newly added metric is covered automatically and exempting one takes a deliberate edit.

27.8.3 No hand-authored numbers

Every quantity in this manuscript is an injected token resolved by `src/spectre/manuscript_variables.py` from the measured mission output. Two tests keep that honest: a live cross-reference test fails if any token used in prose is missing from the generator, and strict resolution fails if any token survives substitution. Even the package's own inventories — module count, gadget count, plan length — are measured from the code rather than typed.

27.8.4 Gates

- `uv run pytest tests/ --cov=src --cov-fail-under=90` — the coverage gate (line **and** branch, enforced on `src/`)
- `uv run ruff check src/ scripts/ tests/` and `uv run ruff format --check`
- `uv run mypy src/ scripts/`
- zero mocks anywhere in `tests/`; the pure domain core has no `bond_api` import, proven by a subprocess import test

The achieved coverage and test count from the last real run are recorded in `docs/_generated/COUNTS.md`; this manuscript deliberately does not restate them, because a number quoted in prose is a number that can go stale.

Release 0.1.0, dated 2026-08-04 (from `manuscript/config.yaml`), hydrated on Darwin arm64 with Python 3.14.6.

27.9 Scope and Related Work — NINE EYES: boundaries, positioning, and relationship to the literature

27.9.1 What this package claims

NINE EYES is a *modelling* package, not an intelligence product. It claims exactly three things, each verifiable by running the code:

1. The film's premises — a hub-dominated surveillance-sharing exchange and a compartmentalized brotherhood — admit precise, standard formalizations: directed-graph centralization, a cell partition over a command tree, undirected connectivity geometry, weighted interception coverage, percolation on the org graph, and set cover.
2. Under those formalizations the canonical dataset yields the measured values reported in §3, deterministically, from a fixed seed and a fixed input whose SHA-256 is recorded on every outcome.
3. Every number in this manuscript is produced by that computation and injected as a token; none is typed by hand.

It claims nothing about any real surveillance arrangement. The canonical network is a fictional construction sized to the film's conceit (9 agencies, 13 directed links), chosen so that each algorithm has something non-trivial to find. The brotherhood roster is one node per distinct film character, but its cell partition and command edges are chosen rather than observed (§5): the organisational *structure* is a modelling construct, and no claim of canonical fidelity is made for it.

27.9.2 Out of scope

- **Empirical validation.** There is no ground-truth dataset for the modelled quantities, so no accuracy claim is made or implied. The results are properties of a stated model over stated inputs.

- **Weighted or temporal graph dynamics.** The geometry and centrality layers are unweighted and static; only the traffic layer carries volumes, and no layer carries time.
- **Adversarial reaction.** The critical-node ordering and the monitoring allocation are one-shot optimizations against a fixed structure; the modelled organisation does not rewire in response.
- **Optimality.** Two of the six analyses use greedy heuristics with known approximation bounds rather than exact solvers (see below); the reported selections are the greedy ones.

27.9.3 Related work by module

Centralization and centrality. Freeman’s group centralization index [Freeman, 1978/1979] is the standard measure of how concentrated a network’s degree distribution is around a single actor; the general treatment of degree, connectivity, and efficiency measures follows Newman [Newman, 2010]. Betweenness is computed with Brandes’ shortest-path accumulation [Brandes, 2001b]. Flow concentration reuses the Herfindahl–Hirschman index from industrial organization [Hirschman, 1964] on the inflow-share vector, which gives a concentration reading independent of the Freeman index — and, on this dataset, a contradicting one (§3). Because the index is bounded below by $1/r$ rather than by 0, it is reported alongside that floor; quoting an HHI without its floor invites reading an evenly-spread vector as a concentrated one, which is exactly what an earlier revision of this report did.

Connectivity geometry. Articulation points come from Tarjan’s low-link depth-first search [Tarjan, 1972b, Hopcroft and Tarjan, 1973c]. Global efficiency is the Latora–Marchiori mean inverse shortest-path length [Latora and Marchiori, 2001]. The node-removal impact and robustness curve follow the error-and-attack-tolerance framing of Albert, Jeong and Barabási [Albert et al., 2000]: fragmenting a hub-dominated network takes few targeted removals and many random ones. The greedy critical-node ordering is a key-player problem in the sense of Borgatti [Borgatti, 2006], scored by pairwise-connectivity loss.

Cascades. The surveillance cascade and the infiltration cascade are both independent-cascade processes in the sense of Kempe, Kleinberg and Tardos [Kempe et al., 2003a], which is also where the submodularity of coverage-style influence objectives is established. The infiltration model’s percolation reading — a contagion spreading over a fixed contact structure — follows Newman’s treatment of epidemic spread on networks [Newman, 2002]; here the contact structure is the brotherhood’s exposure relation (cellmates, superior, subordinates) rather than a social contact graph.

Budgeted monitoring. Interception coverage is monotone and submodular in the monitored set, so the greedy rule carries the classic $(1 - 1/e)$ guarantee of Nemhauser, Wolsey and Fisher [Nemhauser et al., 1978a]. The same structure underlies sensor placement for outbreak detection [Leskovec et al., 2007], which is the closest published analogue of the monitoring-allocation problem posed here.

Placement as set cover. The sensor-placement layer is minimum set cover, solved with Johnson’s greedy heuristic [Johnson, 1974] at its $O(\log n)$ factor; Feige’s threshold [Feige, 1998] establishes that no substantially better polynomial-time approximation exists unless NP has quasi-polynomial-time algorithms, which is why the greedy solution is reported rather than an exact one.

27.9.4 Position within the BOND suite

The suite’s other packages model other films’ concepts; this one owns surveillance-network and criminal-organisation intelligence. Cross-package reuse happens through `bond-api`, never by importing another film’s `src/`. The package exposes 6 pure domain modules (infiltration, network_geometry, org_intel, signal_cover, surveillance_net, traffic_intel) behind one frozen `MissionProvider` and 6 registered Q-branch gadgets, and its `execute` stage runs a 6-step plan. Anything a sibling package needs from NINE EYES it obtains through those interfaces.

27.9.5 Threats to validity

- **Canonical-dataset dependence.** All reported values are properties of one fixed input. A different agency network or brotherhood would give different numbers; the algorithms, not the numbers, are the reusable contribution.
- **Greedy selection.** The monitoring allocation and the placement cover are approximations. Their guarantees are stated above, but a reported selection is not proof of optimality.
- **Cascade parameterization.** Cascade coverage and expected damage depend on the activation and turn probabilities, which are stated design choices rather than estimates. The models are used comparatively (which node, which agency, how much worse) rather than as absolute forecasts.
- **Cascade sampling error.** Both cascade figures are means over 50 seeded realizations, not closed-form expectations. No confidence interval is reported, so the trailing digits of those two numbers should not be read as significant; they would move under a different base seed or trial count.
- **Unweighted geometry.** Treating the sharing graph as unweighted for the connectivity layer discards the volume information the traffic layer uses. The two layers deliberately answer different questions and their results should not be pooled.

27.10 Sources — NINE EYES: bibliography

Freeman [1978/1979]; Newman [2010]; Brandes [2001b]; Tarjan [1972b]; Nemhauser et al. [1978a]; Johnson [1974]; Feige [1998]; Newman [2002]; Hopcroft and Tarjan [1973c]; Latora and Marchiori [2001]; Albert et al. [2000]; Borgatti [2006]; Hirschman [1964]; Kempe et al. [2003a]; Leskovec et al. [2007]

28 No Time to Die (2021) — HERACLES

film package · *package codename* **HERACLES**. Mission **HERACLES**: assess DNA-targeted bioweapon countermeasure specificity and delivery; design and rank the countermeasure guide across a population; characterize poison-garden dose-response toxicology; screen competitive-antagonist antidotes against a toxin dose; assess poison exposure pathways and cumulative risk; run vesper poison-delivery trace detection forensics.

28.1 Concepts — HERACLES: domain and operational focus

DNA-targeted bioweapons, toxicology

28.2 Abstract — HERACLES: mission summary

No Time to Die: HERACLES — Special-Agent Mission Software — mission codename **HERACLES** — reduces the pharmacoforensic threats of *No Time to Die* (2021) to deterministic, tested mathematics with a shared provenance ledger. The pure domain core in `src/no_time_to_die/` comprises **6** modules spanning three threat families: a DNA-targeted bioweapon countermeasure (target specificity, guide design, and delivery kinetics), poison-garden toxicology (dose-response and competitive-antagonist pharmacology), and poison-delivery forensics (exposure assessment and trace detection on the Vesper’s glass). A thin adapter binds that core to the frozen bond-api mission protocol so the film is discoverable and drivable by the BOND suite orchestrator. The poison-garden catalog profiles **7** flora toxins spanning 0.8 to 10.0 mg/kg (median 1.5) — commonly-cited order-of-magnitude LD50 figures paired with invented Hill slopes. The HERACLES countermeasure demonstration selects a target guide that attains 66.67% specificity against a constructed reference genome with a planted, documented structure, and a combined neutralization probability of 49.59%. Guide design ranks 4 candidates by a Doench-style efficiency blended with population off-target burden; a screen over five invented inhibition constants restores 98.87% survival against a ricin dose; and the delivered exposure carries a margin of exposure of 8.333 (risk quotient 60) — a verdict of elevated risk. Forensic trace detection clears a limit-of-detection signal-to-noise ratio of 3.0 at an observed SNR of 9, recovering a dosable residue from the glass and identifying it as ricin. The entire assessment is deterministic — no module imports a clock or an RNG, the canonical input digest is `cb8fe93a43fc`, and two runs on one interpreter render byte-identical artifacts — and it is reproduced under Python 3.14.6, satisfying the suite’s coverage, mock-free, and lineage gates. Every constant is registered with its provenance class in `data/claim_ledger.yaml`; none is a measurement.

28.3 Introduction — HERACLES: mission framing, the operational problem, and how to read this chapter

No Time to Die (2021) introduces three interlocking threats that map cleanly onto computational modelling problems: a DNA-targeted bioweapon program codename **HERACLES** designed against a specific genetic profile; the poison garden cultivated on the villain’s island, a landscape of naturally occurring plant toxins; and the forensic task of detecting a poison delivered through a drink — the trace left on the Vesper’s glass.

This package treats each as a real, deterministic computational object rather than a narrative device. Each threat family decomposes into a pair of concerns, giving the **6** domain modules of the core:

1. **Target specificity and delivery** (`bioweapon_counter.py`) — a designed nucleic-acid guide must hit its intended target while minimizing off-target matches in a reference genome, and the fraction of an administered dose that reaches the target compartment follows first-order absorption kinetics.
2. **Guide design under population off-target burden** (`targeting_design.py`) — the guide is not given but *designed*: sequence descriptors drive a Doench-style activity score, which is then discounted by the near-miss burden the guide would incur across a population of reference genomes.
3. **Dose-response toxicology** (`toxicology.py`) — every member of the garden’s flora is a Hill-curve hazard characterized by an LD50 and a slope; mixtures combine under response addition.
4. **Antidote pharmacology** (`antidote_screening.py`) — a competitive antagonist shifts a toxin’s effective dose-response, so a library of candidate antidotes can be ranked by the survival each restores against a given toxin dose.
5. **Exposure assessment** (`exposure_assessment.py`) — the hazard is set by the *absorbed* dose rather than the residue: pathway intake, body burden under first-order elimination, margin of exposure, and risk quotient.
6. **Delivery forensics** (`delivery_forensics.py`) — given an observed analytical signal and noise, we decide whether a trace is present, back-compute its concentration from a calibration model, and identify the poison by spectral matching.

Modules 1–2 and 3–4 are deliberately paired: detection without design, and hazard without countermeasure, would each leave the mission’s central question — *can this threat be neutralized?* — unanswerable. Module 5 bridges the forensic and toxicological halves, converting a recovered residue into an absorbed dose the toxicology models can act on.

The design honours the BOND suite contract: the domain core is pure and importable without bond-api, the mission adapter implements the frozen protocol, every metric flows into the manuscript through `src/no_time_to_die/manuscript_variables.py`, and the whole assessment is deterministic and fully tested. A reader can open any function named in Methodology and trace it to its source file in seconds.

28.4 Methodology — HERACLES: the analytical models and algorithms that drive the mission

This section specifies the mathematics of each concept module. All functions are pure, side-effect-free, and deterministic; the canonical demonstration inputs live in `src/no_time_to_die/demo.py` and are shared verbatim by the mission adapter and the manuscript variable generator so outcomes cannot drift.

28.4.1 DNA-targeted bioweapon countermeasure (`bioweapon_counter.py`)

Specificity. For a guide g and a reference genome r , every length- $|g|$ window of r is scored by Hamming distance $d(g, w)$. A window at distance 0 is an on-target hit; a window at distance in $[1, k]$ is an off-target hit representing a mis-address risk. Specificity is the share of hits that are on-target:

$$\text{specificity} = \text{on_target} / (\text{on_target} + \text{off_target})$$

`select_best_guide` chooses the candidate maximizing specificity, ties broken by first-in-order for determinism.

Delivery. The one-compartment bateman equation models plasma concentration after an absorbed dose:

$$C(t) = \text{dose} * k_a / (k_a - k_e) * (\exp(-k_e t) - \exp(-k_a t))$$

with k_a the absorption rate and k_e the elimination rate (`plasma_concentration`). The absorbed fraction at exposure time t is `fraction_delivered = 1 - exp(-k_a t)`.

Countermeasure efficacy. `neutralization_probability` combines specificity, delivered fraction, and a saturating avidity term:

$$P = \text{specificity} * \text{delivered_fraction} * \text{avidity} / (1 + \text{avidity})$$

clamped to $[0, 1]$, so any near-zero component drives neutralization to zero.

28.4.2 Poison-garden toxicology (`toxicology.py`)

Each flora toxin is a Hill dose-response hazard with LD50 and slope n (`PlantToxin`). The fraction of a population affected at dose d is

$$f(d) = d^n / (d^n + \text{LD50}^n)$$

with analytic inverse `dose_for_fraction`. A mixture of agents combines under response addition — the system is unaffected only if every agent misses:

$$F = 1 - \prod_i (1 - f_i(d_i))$$

The `TOXIN_CATALOG` records 7 botanical toxins spanning LD50 from 0.8 to 10.0 mg/kg (median 1.5), each with a botanical source, a commonly-cited order-of-magnitude LD50 recorded without a specific citation, and a Hill slope that is **not** sourced at all — the slopes are curve-shape parameters invented for this demonstration, registered as such in `data/claim_ledger.yaml`. `catalog_stats` reports that summary over any catalog, and the forensic spectral library profiles 5 poisons for identification.

28.4.3 Poison-delivery forensics (`delivery_forensics.py`)

A residue signal S with root-mean-square noise N yields $\text{snr} = S/N$. Detection confidence under a logistic model with limit of detection `lod` is

$$\text{confidence} = 1 / (1 + \exp(-(\text{snr} - \text{lod})))$$

Concentration is back-computed from a linear calibration $S = m \cdot C + b$. Signals below the *intercept* b clamp to zero, because the inverted line would otherwise return a negative concentration. That clamp is not a limit-of-detection test and `estimate_concentration` has no knowledge of the LOD: with the demonstration's $b = 0.5$ and an LOD SNR of 3.0, a signal of 1.0 lies well below the limit of detection yet still returns a positive concentration. Deciding whether a signal is real is `detection_confidence`'s job. (Earlier revisions of this section and of the function docstring said “sub-LOD signals clamp to zero”, conflating the two; the suite now pins the below-LOD-yet-positive case.) The transferred dose is `concentration · volume · transfer_fraction(recovered_dose)`, and an unknown residue spectrum is identified by cosine similarity against the reference library (`match_unknown`). The demonstration operates at a limit-of-detection SNR of 3.0.

Mixture decomposition. A residue that is itself a blend is decomposed into non-negative mixing fractions rather than ranked against a single entry (`decompose_mixture`, a two-component non-negative least-squares fit in the spirit of Lawson and Hanson 1974): the unknown spectrum is approximated as $\sum_k w_k s_k$ with $w_k \geq 0$ and $\sum_k w_k = 1$, and the 1- or 2-component candidate with the smallest squared residual wins. A pure single poison is returned as a one-element decomposition; weights are *mixing fractions* (they sum to 1), and the squared residual measures how much of the unit-fraction model the spectrum does not explain. `mixture_analysis` composes this into the primary/secondary components and residual consumed by the demo and tokens.

28.4.4 Guide design and population off-target (`targeting_design.py`)

The countermeasure guide is not given — it is *designed*. `gc_content` and `longest_homopolymer` are base sequence descriptors; `guide_efficiency` combines a GC-content sweet-spot penalty, a homopolymer-run penalty, and an optional position-dependent nucleotide bonus into a Doench-style activity score in $[0, 1]$ (spirit of Doench et al. 2014).

Off-target burden is assessed *across a population* of reference genomes:

```
off_target_risk = sum_{genomes} sum_{windows, 1<=d<=k} exp(-lambda * d)
```

where `d` is the Hamming distance of a mismatched window and `lambda` the mismatch decay (`population_off_target_risk`). `rank_guides` scores each candidate as `efficiency * (1 - min(1, off_target_risk))` and `design_report` returns the ranked table plus the best guide. The demonstration ranks 4 candidates against the reference genome.

28.4.5 Antidote screening (`antidote_screening.py`)

A poison-garden toxin is acted on through a receptor; a competitive antagonist counteracts it. The apparent IC50 of a competitive antagonist follows Cheng–Prusoff (1973):

```
IC50_app = Ki * (1 + [S] / Km)
```

with the toxin dose `[S]` and its LD50 standing in as the affinity proxy `Km`. The fraction of receptors occupied is the Langmuir isotherm `[A] / ([A] + IC50_app)` (`fraction_antagonized`). `screen_antidotes` ranks a library of 5 entries by the survival they restore against a toxin dose:

```
residual = f_tox * (1 - f_antag); survival = 1 - residual
```

The `Ki` values fed to this model are invented, not published — see Scope. The relation and the isotherm are real and are tested against hand-checkable values; the library entries are illustrative inputs named after real drugs, and the model is applied uniformly to all five regardless of whether the real drug is a competitive antagonist. `screen_report` also reports the *challenge* therapeutic window `toxin LD50 / dose` and margin of safety `window - 1` — a screening context (how far the administered dose sits from the population-lethal level), not a clinical dosing index.

28.4.6 Exposure assessment (`exposure_assessment.py`)

The hazard of a delivered poison is set by the absorbed dose, not the residue. `absorbed_dose = concentration * contact_rate * duration * absorption_fraction` models intake through an exposure pathway; `cumulative_exposure` accumulates body burden under first-order elimination $(\text{intake_rate} / k_e) * (1 - \exp(-k_e t))$; `margin_of_exposure` computes NOAEL / absorbed (NRC 1983 framing) and `risk_quotient` computes absorbed / reference_dose. `exposure_scenario_report` composes these into a single verdict (acceptable vs elevated risk).

28.5 Results — HERACLES: measured outcomes, headline numbers, and what they establish

All results below are computed live from `src/no_time_to_die/demo.py` by `src/no_time_to_die/manuscript_variables.py`; none are hand-authored. The deterministic demonstration uses a fixed 144-nucleotide reference assembled from 9 labelled segments, a 4-guide candidate set, seed 11, and a fixed 4.0 h exposure window (canonical input digest `cb8fe93a43fc`).

What these numbers do and do not establish. The reference genome is a constructed input with a known planted structure (2 exact on-target windows, 3 near-misses at declared Hamming distances). The results therefore establish that the models *recover a structure that was put there*, and that they discriminate between candidates — not that they would perform this way on a real genome. What they specifically no longer do is recover a result the input made inevitable: through 2026-08-04 the reference was `AA CCGGTTAACCGGTT` repeated four times, a period-16 string scanned by a 16-nt guide, so 100% specificity, zero off-target windows, and zero population off-target risk were arithmetic consequences of the input rather than findings. Those degenerate outcomes are now failing conditions in the suite.

28.5.1 Countermeasure demonstration

`select_best_guide` selects guide `AACCGGTTAACCGGTT` (16 nt), which attains a target specificity of **66.67%** — **2** on-target windows against **1** off-target near matches (within 2 mismatches) in the demonstration reference. The specificity sits strictly inside $(0, 1)$: the selector separates the planted exact hits from the planted near-misses rather than reporting a perfect score the reference could not have contradicted. Combined with a delivered fraction of **99.18%** and a saturating avidity term (avidity 3.0), the HERACLES neutralization probability is **49.59%**.

The delivery kinetics (one-compartment bateman model) are rendered in [the delivery figure.

28.5.2 Guide-design ranking

Across the 4 demonstration candidates, `design_report` selects guide `AACCGGTTAACCGGTT` with an efficiency score of **100%**, a population off-target risk of **0.3679** over the reference genome, and a composite score of **63.21%**.

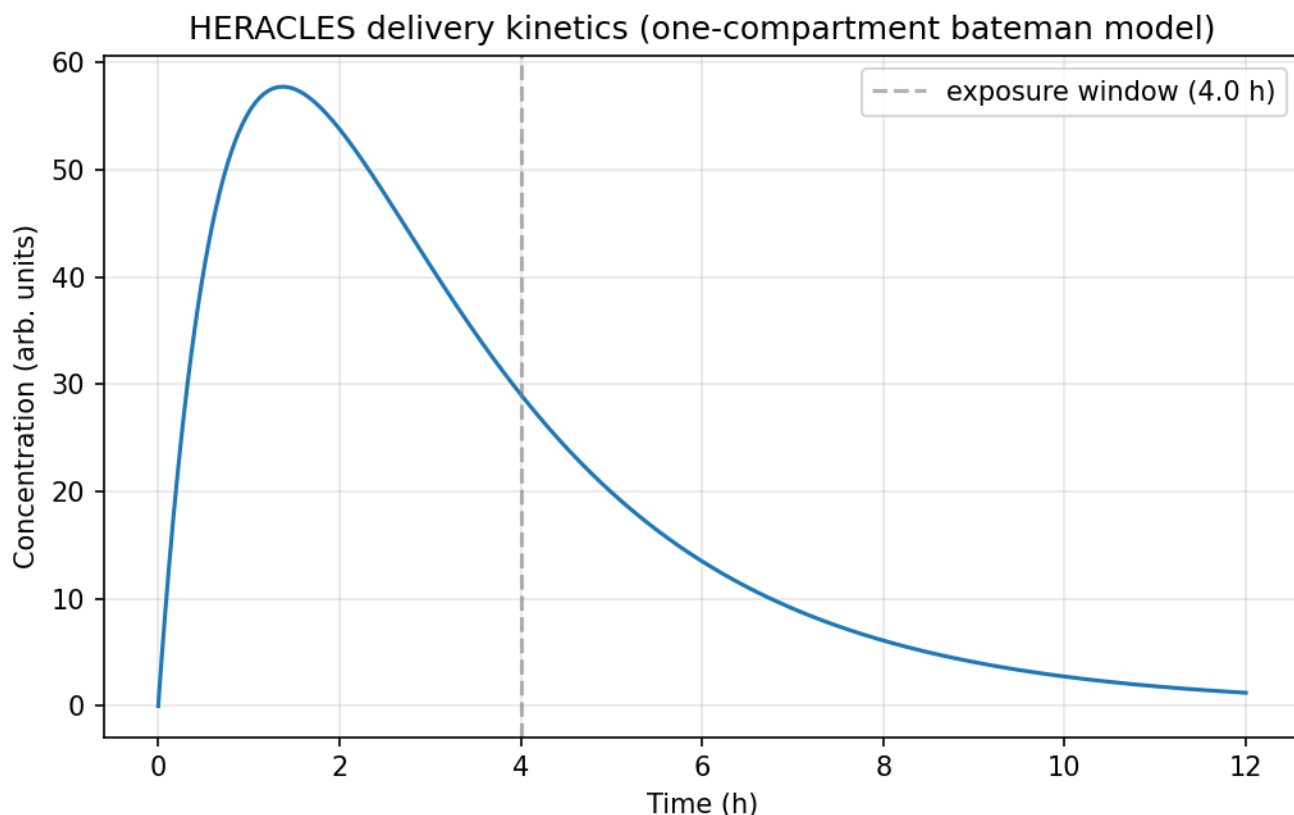


Figure 106: HERACLES one-compartment delivery kinetics over a 12.0 h horizon (absorption rate 1.2 h^{-1} , elimination rate 0.4 h^{-1}), with the 4.0 h exposure window marked.

The blend is what decides the ranking here, which is the point of reporting it. Two candidates tie at the maximum efficiency of 100%; their composite scores do not, because the reference plants near-miss windows for one of them and fewer for the other. The winner's own composite is strictly below its efficiency — the `efficiency * (1 - off_target_risk)` term erodes it by a non-zero amount. Under the previous reference every candidate scored exactly zero off-target risk, so the composite reduced to the raw efficiency, the two leaders tied at 1.0, and the “winner” was chosen by the alphabetical tie-break. The suite now fails if the off-target term goes inert or the top two tie again.

The efficiency landscape (vs GC content) is rendered in [the guidance figure].

28.5.3 Poison-garden catalog

The 7-toxin catalog spans LD50 0.8 to 10.0 mg/kg, median 1.5 mg/kg. Dose-response curves for every flora member are rendered in [the toxicology figure]. A garden-risk mixture of ricin, nicotine, and cyanide combines under response addition to affect **42.12%** of an exposed population.

28.5.4 Antidote screen

Against a ricin dose at an antagonist concentration of 10.0 mg/L, the 5-antidote library is ranked by restored survival. The best antidote is **naloxone**, restoring a survival probability of **98.87%** by antagonizing **94.34%** of the toxin's receptor occupancy. The *challenge* is put in context by the therapeutic window — the toxin LD50 sits **2** times above the administered dose, a margin of safety of **1** — so the dose the screen confronts is comfortably below the population-lethal level (see Scope for what this ratio is and is not).

This ranking is a property of the arithmetic, not of pharmacology. Every K_i in the library is invented (see Scope); three of the five named agents are not competitive receptor antagonists at all, and the Cheng-Prusoff model is applied to them anyway. The result demonstrates that the screen orders a library correctly given inhibition constants; it says nothing about which of these drugs would help against ricin. The competitive-antagonist dose-response for the library is rendered in [the antidote figure].

28.5.5 Exposure assessment

For the demonstration Vesper pathway, the absorbed dose is **0.06** (mass units), a cumulative body burden of **0.019**, a margin of exposure of **8.333**, and a risk quotient of **60** against the reference dose 0.001 — a verdict of **elevated risk**. The margin of exposure exceeds unity against the NOAEL 0.5 — the intake sits below the lethality threshold — yet the risk quotient also exceeds unity, which

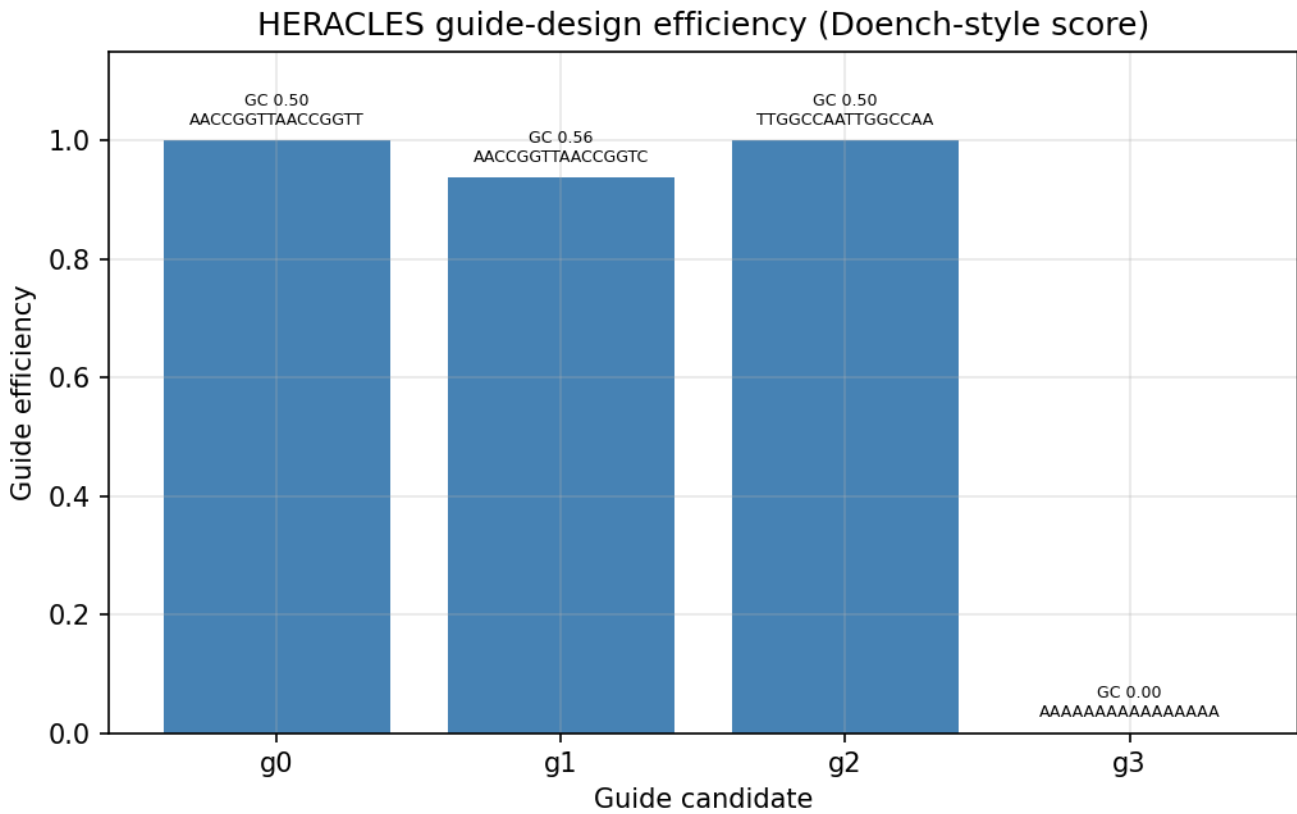


Figure 107: HERACLES guide-design efficiency across the candidate set.

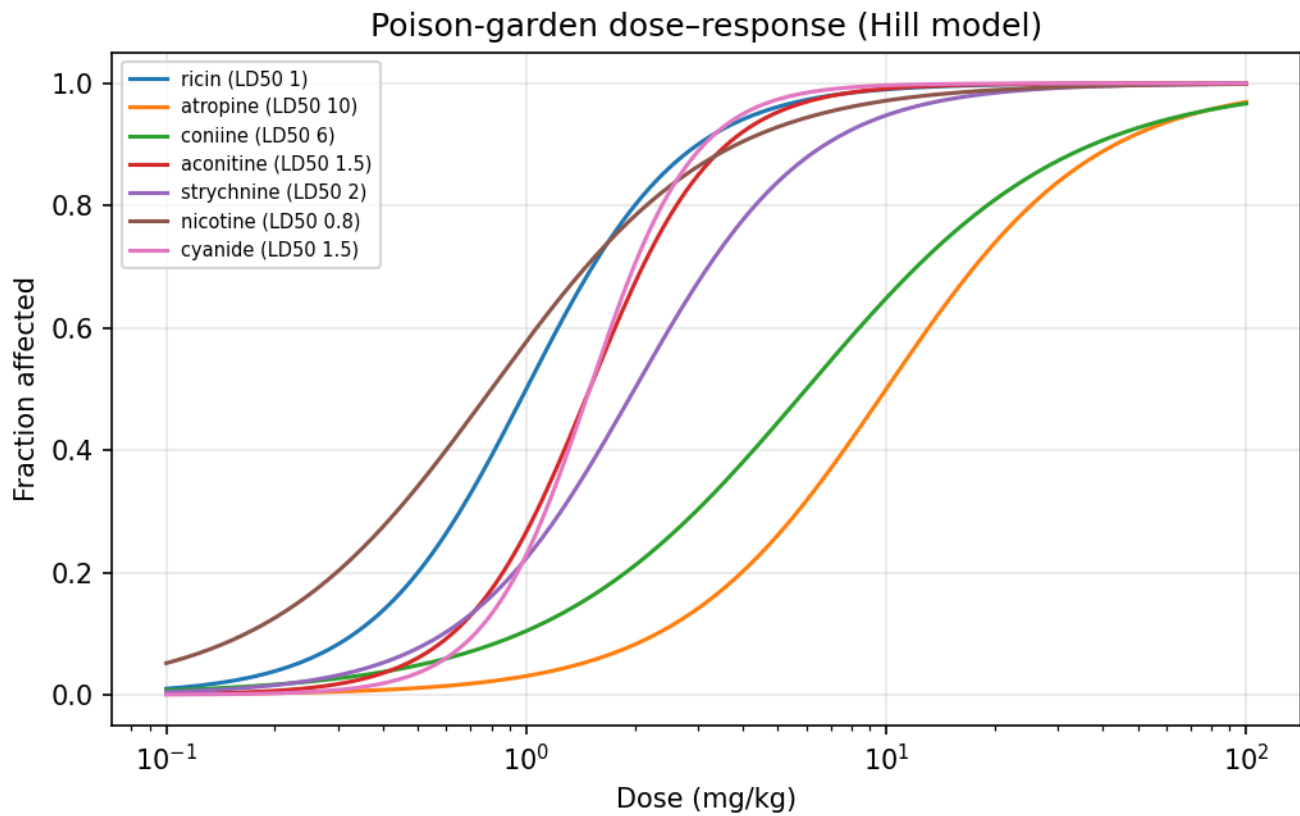


Figure 108: Poison-garden Hill dose-response curves across the catalog flora.

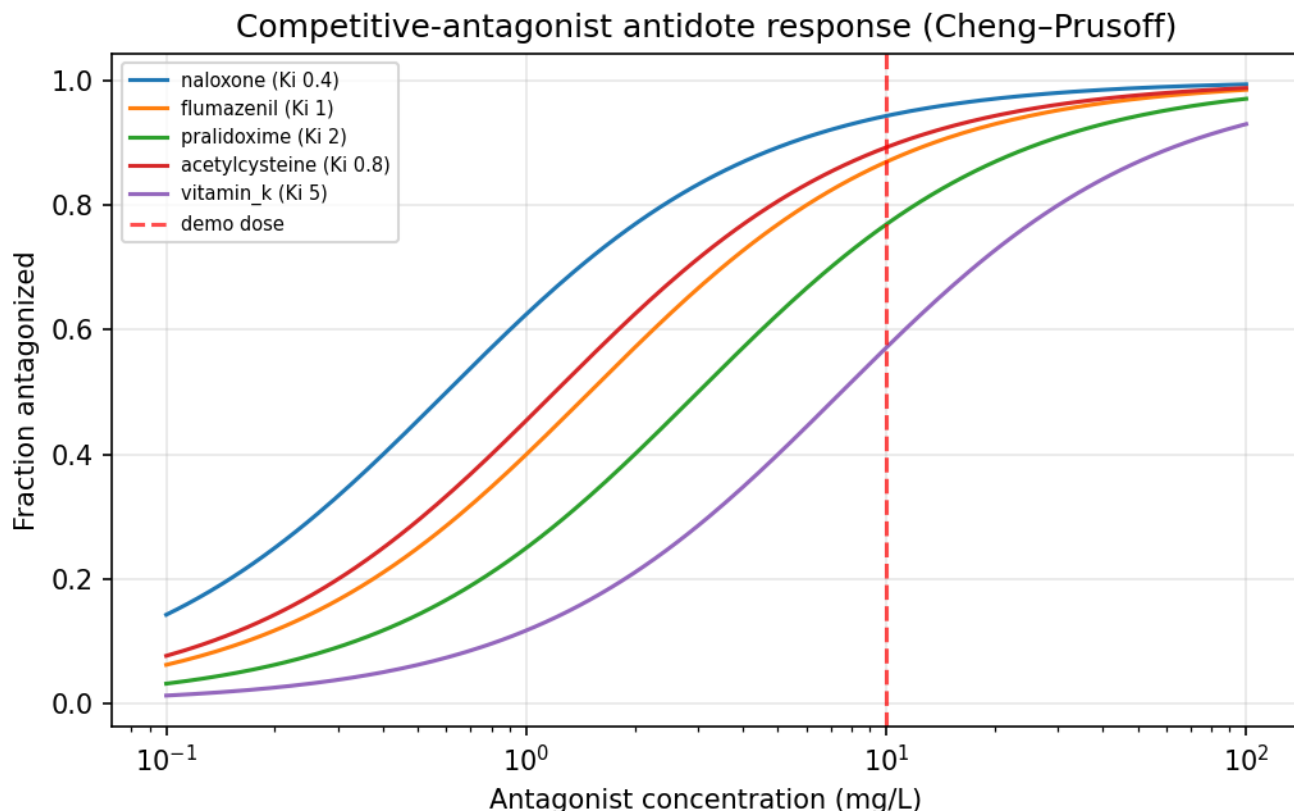


Figure 109: Competitive-antagonist antidote response curves.

is precisely the discordance the verdict rule is written to catch: an intake far below the lethality threshold can still sit far above the reference dose, and the verdict follows the stricter of the two.

28.5.6 Vesper trace forensics

At a limit-of-detection SNR of 3.0, the demonstration signal (9.0 against RMS noise 1.0, giving SNR 9) is detected with **99.75%** confidence. The calibration model back-computes a residue concentration of **4.25** (the calibration inversion is clamped at the calibration intercept, not at the limit of detection — see Methodology), and applying the 0.1 transfer fraction over 0.5 mL gives a transfer-corrected residue dose of **0.2125** (arbitrary mass units) on the glass. Spectral matching against the 5-entry reference library identifies the residue as **ricin**. Detection confidence vs signal-to-noise is rendered in [the detection figure].

28.5.6.1 Mixture decomposition A residue that is a *blend* of two garden poisons grades as a mediocre single match under cosine similarity, which is what a blend really is. When the recovered spectrum is the demonstration’s **ricin** + atropine mix, **decompose_mixture** recovers the non-negative mixing fractions by least squares over 1- and 2-component candidates: a primary **60%** + secondary **40%** split with a squared residual of **1.329e-32** (see Methodology). The decomposition is a *model*, not an instrument read: it recovers the planted blend because the blend was put there, exactly as the specificity and guide results recover planted structure (see Scope).

28.6 Conclusion — HERACLES: findings, verdict, and what the mission establishes

No Time to Die: HERACLES — Special-Agent Mission Software demonstrates that the three central threats of *No Time to Die* (2021) — a DNA-targeted bioweapon, a poison garden, and delivery forensics — can be expressed as a single deterministic, fully-tested computational core under the BOND suite contract.

The package delivers **6** concrete claims — one per domain module — each verified by real computation rather than assertion:

1. A target-specific countermeasure can be selected and its neutralization probability bounded by specificity, delivered fraction, and avidity (**bioweapon_counter.py**).
2. A guide can be *designed* and ranked by a Doench-style efficiency blended with population off-target burden (**targeting_design.py**) — with the blend doing real work: it separates two candidates that tie at maximum efficiency, and it erodes the winner’s own score by a non-zero off-target term.
3. A poison-garden flora catalog can be profiled as Hill-curve hazards and combined under response addition (**toxicology.py**).
4. A competitive-antagonist antidote can be screened against a toxin dose by Cheng–Prusoff pharmacology and restored survival (**antidote_screening.py**).

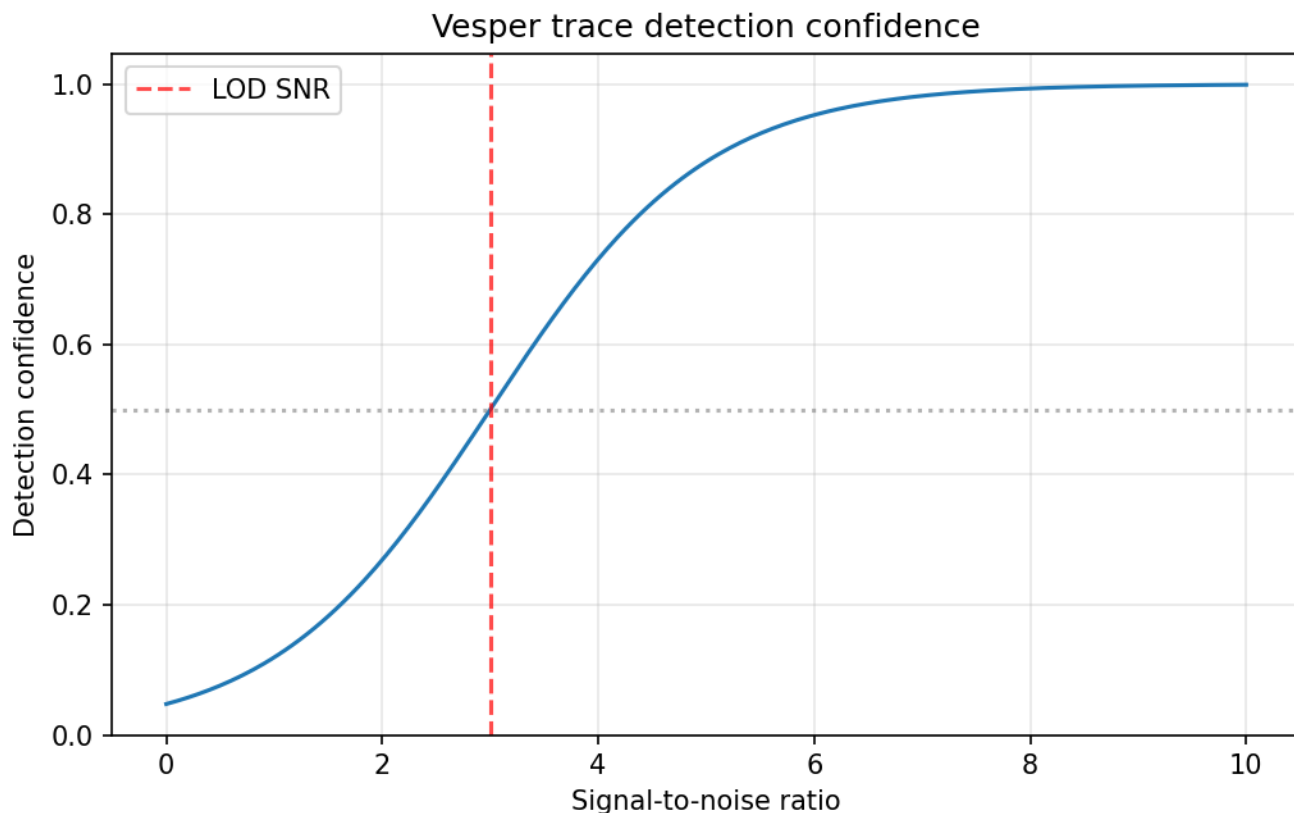


Figure 110: Vesper trace detection confidence vs signal-to-noise against the limit of detection.

5. A delivered exposure can be assessed through pathway intake, cumulative burden, margin of exposure, and risk quotient (`exposure_assessment.py`).
6. A delivered poison leaves a structurally detectable and identifiable residue on the vessel (`delivery_forensics.py`).

The domain core is infrastructure-free and importable without `bond-api`; a thin adapter implements the frozen `MissionProvider` protocol, and every metric flows into the manuscript through the shared `demo.py` + `manuscript_variables.py` path, guarded by `check_demo_completeness` so a renamed result key fails the build rather than silently hydrating a zero. This is the reference film pattern for the remaining BOND suite packages.

The clearest limitation is the one the Scope section names: nothing here is a measurement. Most constants are invented for this demonstration; the rest are commonly-cited order-of-magnitude figures carried without a citation. The demonstration therefore supports claims about the *method* — that these threats admit deterministic, auditable, end-to-end modelling — and not about any real toxin, guide, or instrument. Two concrete cases: a margin of exposure of 8.333 against a risk quotient of 60 is a statement about the demonstration’s chosen NOAEL and reference dose, not about anyone’s actual risk; and the antidote ranking is a statement about the arithmetic of Cheng–Prusoff over five invented inhibition constants, not about which drug to reach for.

A second limitation is structural and worth stating in its own right: the reference genome is a constructed input whose on-target and near-miss windows were placed deliberately. The countermeasure and guide-design results measure **recovery of a planted structure**, which is a weaker claim than performance on an unseen genome and a stronger one than the package could previously make at all — the earlier periodic reference made its own headline numbers arithmetically inevitable.

28.7 Experimental Setup — HERACLES: canonical scenarios, parameters, and configuration

28.7.1 Software environment

- Python 3.14.6 (host), NumPy for the delivery/figure numerics, matplotlib (Agg backend) for figure rendering, PyYAML for config loading.
- Repository tree: isolated local-only package rooted at `projects/working/bond/no_time_to_die`; a local path dependency on the frozen `bond-api` protocol.

28.7.2 Deterministic demonstration parameters

Canonical inputs live in `src/no_time_to_die/demo.py` and `manuscript/config.yaml`. Every value below is a resolved token read from those constants at hydration time, so the table cannot drift from the code that runs:

Parameter	Value
Reference genome length	144 nt (9 labelled 16-nt segments)
— exact on-target windows planted	2
— near-miss windows planted	3
Guide candidates	4
Guide length	16 nt
Off-target tolerance k	2
Absorption rate k_a	1.2 h^{-1}
Exposure time	4.0 h
Avidity	3.0
Antidote antagonist concentration	10.0 mg/L
Exposure pathway — concentration	0.02
Exposure pathway — contact rate	0.5
Exposure pathway — duration	30.0
Exposure pathway — absorbed fraction	0.2
Exposure pathway — elimination rate k_e	0.1
Exposure pathway — NOAEL	0.5
Exposure pathway — reference dose (RfD)	0.001
Forensic signal / RMS noise	9.0 / 1.0
Forensic residue volume	0.5 mL
Forensic transfer fraction	0.1
Forensic calibration slope m / intercept b	2.0 / 0.5
Limit-of-detection SNR	3.0
Catalog flora	7
Forensic library	5
Domain modules	6
Seed	11
Canonical input digest (sha256, first 12)	cb8fe93a43fc

All are fixed at import. No module in the package imports a clock or an RNG (structurally enforced — see Reproducibility), so two runs on one interpreter render byte-identical artifacts. The single host-dependent value in the whole package is the `PYTHON_VERSION` token above.

The reference genome is not an arbitrary or periodic string. It is assembled from labelled segments — 2 exact copies of the target guide, 3 near-miss windows at declared Hamming distances, and neutral fillers — precisely so the specificity and off-target results are *measured against a known planted structure* rather than forced by the input. The earlier reference was `AACCGGTTAACCGGTT` repeated four times, whose period equalled the guide length; that reference could only ever return 100% specificity, zero off-target windows, and zero population off-target risk, so those numbers carried no information. See Results.

28.7.3 Figure generation

Figures are rendered deterministically by `src/no_time_to_die/figures/plots.py` under a headless `Agg` backend to `../figures/`. Each renderer reads the same `demo.py` constants as the mission and the manuscript tokens, so a figure cannot annotate a parameter the demonstration does not use:

- `delivery_kinetics.png` — bateman plasma-concentration curve over a 12.0 h horizon, marking the 4.0 h exposure window.
- `toxicology_dose_response.png` — Hill curves for the full catalog.
- `guide_efficiency.png` — guide-design efficiency vs GC content.
- `antidote_response.png` — competitive-antagonist dose-response, marking the 10.0 mg/L demonstration dose.
- `detection_confidence.png` — logistic detection confidence vs SNR against the 3.0 limit of detection.

28.8 Reproducibility — HERACLES: verification gates, deterministic regeneration, and artifacts

This package treats reproducibility as a hard constraint enforced by tests, not a prose promise. Every guarantee below names the test that would fail if it stopped holding.

28.8.1 Deterministic computation

No module under `src/` imports a clock, a random-number generator, or any other entropy source, and the package makes exactly one environment read in total — `platform.python_version`, rendered as the `PYTHON_VERSION` token. That is a structural property, checked structurally: `tests/test_determinism.py` parses every source file and fails on a forbidden import or an unsanctioned environment read, and its positive controls confirm the scan reports a reintroduced `datetime` or `random` import rather than passing silently.

Two consequences, stated precisely:

- **On one interpreter, two runs render byte-identical artifacts.** The byte gate renders the whole manuscript tree twice and compares every file byte for byte.
- **Across interpreter versions, exactly one substring differs** — the `PYTHON_VERSION` token. This is the single host-dependent value in the package; nothing else varies with the machine.

Through 2026-08-04 this section claimed byte-reproducibility while the generator stamped `datetime.now(timezone.utc)` into both `output/data/manuscript_variables.json` and this rendered section, so consecutive runs demonstrably differed. The token was removed rather than reworded, and the value gate now fails on any rendered token shaped like a date or a clock time.

All demonstration inputs are fixed constants in `src/no_time_to_die/demo.py` and `manuscript/config.yaml`; seed 11 is stable, though no code path draws from an RNG, so it currently governs nothing and is recorded for provenance only. The mission outcome carries a **Provenance** record with the package version, the seed, and a sha256 `input_hash` over the canonical inputs (`demo.input_hash`, currently `cb8fe93a43fc...`). That digest covers every demonstration constant — an earlier version hashed only five of sixteen — and the suite scans `demo.py` independently to fail if a newly added constant is left out of it.

Figure renderers read the same `demo.py` constants as the mission and the tokens. This is enforced, not asserted: each figure is composed by a `build_*_figure` function the tests inspect directly, checking the plotted series and the annotation positions against the constants the captions quote. Five of the seven figure tests previously asserted only that a PNG was non-empty, which a blank or wrong-data figure passes.

28.8.2 Live manuscript metrics

Every numeric value in the prose sections flows through a resolved `HERACLES_*` token from `src/no_time_to_die/manuscript_variables.py`, computed from the shared demo result; none are hand-authored. Three gates protect this:

1. `check_demo_completeness` fails if the demo result is missing any key the tokens read — including the top-level `garden_risk_combined`, which the section-keyed scan was structurally blind to until 2026-08-05 (deleting it passed the gate and then raised a bare `KeyError`). The suite's positive control now deletes *every* key of a real demo result in turn and requires each deletion to surface as a named gate failure rather than as any other exception.
2. The token cross-reference test scans `manuscript/[0-9]*.md` and fails if any `{`-delimited token is not produced by `generate_variables`.
3. `z_generate_manuscript_variables.py` fails in strict mode on any token left unresolved after substitution.

28.8.3 Evidence provenance

`data/claim_ledger.yaml` registers every numeric constant the demonstration consumes, with its live value and one of two provenance classes: `synthetic: true` (invented for this demonstration) or `synthetic: false` (a commonly-cited order-of-magnitude figure for a real substance, recorded without a specific citation). `tests/test_claim_ledger.py` derives the expected constant set from the modules themselves, so an unregistered constant, an orphaned entry, or a value edited on one side only all fail the build. Nothing here is a measurement.

28.8.4 Verification commands

From the package root:

```
uv run pytest tests/ --cov=src --cov-fail-under=90
uv run ruff check src/ scripts/ tests/ && uv run ruff format --check src/ scripts/ tests/
uv run mypy src/ scripts/
## Lineage gate - pattern is bracket-split so this doc cannot trip its own
## check (template[_]code_project resolves to the literal string when run).
rg -n "template[_]code_project". --glob '!uv.lock' --glob '!git/**' \
  --glob '!output/**' || echo Clean
git diff --exit-code # clean tree after regeneration
```

The suite enforces $\geq 90\%$ line+branch coverage on `src/` and zero mocks. Rendered under Python 3.14.6.

28.9 Scope and Related Work — HERACLES: boundaries, positioning, and relationship to the literature

28.9.1 Scope

This package models the *computational* skeleton of the HERACLES narrative arc in *No Time to Die* (2021). It does **not** claim biological fidelity to a real bioweapon, a real pharmacology database, or a real analytical instrument. **Nothing in this package is a measurement.** The value is in the *method* — reproducible, task-appropriate modelling with auditable provenance — not in the specific constants.

Every constant is registered in `data/claim_ledger.yaml` under one of exactly two provenance classes, and the split matters:

- **Invented for this demonstration** (`synthetic: true`): the DNA reference and guide set, all seven Hill slopes, the forensic spectral fingerprints and the unknown spectrum, the guide-design model parameters, every demonstration scenario parameter, and **all five antidote Ki values**.
- **Commonly-cited order-of-magnitude figures for real substances** (`synthetic: false`), recorded without a specific citation and used as plausible calibrants rather than as data: the seven LD50 values and the conventional limit-of-detection SNR of 3.

28.9.1.1 Withdrawn claim: the antidote inhibition constants This is worth naming rather than burying in a table. Through 2026-08-04 the ledger labelled all five antidote Ki values “Published approximation, *drug* Ki” and left them unflagged, placing them on the measured side of the ledger’s own distinction, and the module `docstring` called them “published-approximation inhibition constants”. That was false. No publication reports these numbers, and two independent facts make it impossible that any could:

- Naloxone and flumazenil bind their receptors at nanomolar affinity. The recorded 0.4 and 1.0 mg/L are not those quantities in any unit system.
- Pralidoxime reactivates phosphorylated acetylcholinesterase, acetylcysteine is a glutathione precursor, and vitamin K is a cofactor in an enzymatic cycle. None is a competitive receptor antagonist, so none has a Ki of the kind the Cheng–Prusoff relation consumes. The model is applied to them uniformly regardless — a property of this demonstration, not of the pharmacology.

The values are now marked `synthetic: true` with honest source labels, and `tests/test_claim_ledger.py` fails if any is re-flagged as measured. The antidote screen therefore demonstrates that the ranking arithmetic is correct, and demonstrates nothing about clinical antidote choice.

Boundaries kept deliberately out of scope:

- No simulation of DNA damage or clearance pharmacology beyond the one-compartment delivery model.
- No instrument noise model beyond additive RMS noise.
- No claims of real-world forensic admissibility.
- No claim that any model would behave this way on a real genome, patient, or instrument. The reference genome is a *constructed* input with a planted, documented structure; the results measure recovery of that planted structure.

28.9.1.2 Uncertainty and limitations (round-2 additions) Three quantities introduced or re-exposed in round 2 carry their own, explicit uncertainty:

- **Mixture mixing fractions.** `decompose_mixture` reports weights that are a *fit*, not a measurement. They are exact here only because the 5-bin fingerprints are noiseless and the blend was planted; on a real residue the weight estimates would carry instrument noise, unknown transfer, and library incompleteness that no model in this package captures. The residual token reports the fit’s unexplained variance, and the weights sum to one by construction (they cannot express a missing third component or an absolute recovered amount).
- **The challenge therapeutic window.** `HERACLES_WINDOW` is the ratio of the toxin LD50 to the administered demo dose. It is arithmetic context, not a pharmacology finding: the LD50 is itself an order-of-magnitude figure with no citation (see the `synthetic/synthetic: false` split in `data/claim_ledger.yaml`), the dose is a synthetic input, and a real therapeutic window depends on route, species, and formulation the ratio does not model. The margin is consequently not a clinical safety margin.
- **The LD50 values remain uncited.** Per the ledger contract, `synthetic: false` means a **commonly-cited order-of-magnitude figure ... recorded without a specific citation** — a provenance *label*, not a reference. This round deliberately did **not** fabricate bibliographic entries to fill the seven `ld50_* citation:` fields; sourcing each to a specific publication is recorded as open work in `TODO.md` rather than papered over.

28.9.2 Related work

The package extends the BOND suite pattern established by `bond-api` (the frozen mission protocol), `bond-orchestrator` (the DAG runner), and `bond-utilities` (Layer 0 provenance/cipher utilities), each converted from a template exemplar scaffold under the shared 90%-coverage, zero-mock, thin-orchestrator contract. The engineering contract is inherited from the suite’s other film packages; what is specific here is the domain lineage.

Each model is a deliberately simplified, deterministic instance of a well-established published method rather than an invention of this package:

- **Dose–response** follows the Hill equation [Hill, 1910]; mixtures combine under Bliss independent action / response addition [Bliss, 1939].
- **Guide activity and off-target burden** follow the feature-based sgRNA design rules of Doench et al. [Doench et al., 2014] and the mismatch-tolerance characterization of Hsu et al. [Hsu et al., 2013], reduced here to a transparent GC/homopolymer score and an exponential mismatch decay.
- **Delivery kinetics** use the one-compartment first-order absorption model usually attributed to Bateman [Bateman, 1910] and standard in pharmacokinetics texts [Gibaldi and Perrier, 1982].
- **Mixture decomposition** is a non-negative least-squares fit in the spirit of Lawson and Hanson [Lawson and Hanson, 1974b].

- **Antidote pharmacology** uses the Cheng–Prusoff relation [Cheng and Prusoff, 1973] over Langmuir occupancy [Langmuir, 1918].
- **Exposure and risk** follow the NRC risk-assessment framing [National Research Council, 1983]; **limit-of-detection** reasoning follows Currie [Currie, 1968].

The simplifications are the point: each is small enough to test exactly against a hand-checkable value, which is what makes the whole assessment auditable.

28.10 Sources — HERACLES: bibliography

BOND [2026a]; BOND [2026b]; BOND [2026c]; Hill [1910]; Fukunaga et al. [2021]; Doench et al. [2014]; Hsu et al. [2013]; Cheng and Prusoff [1973]; National Research Council [1983]; Bateman [1910]; Gibaldi and Perrier [1982]; Langmuir [1918]; Bliss [1939]; Currie [1968]; Lawson and Hanson [1974b]

29 Casino Royale (1967) — FIVE BONDS

film package · *package codename FIVE BONDS*. Mission **FIVE 00s**: model coordination failure across the five overlapping Bonds; simulate dispatch chaos and protocol violations deterministically; analyze the spoof ops doctrine and quantify its deviation; quantify identity confusion and resolve dispatch sightings by Bayes; derive exact baccarat table odds and the five-Bond table collision; model doctrine escalation as an absorbing Markov chain.

29.1 Concepts — FIVE BONDS: domain and operational focus

multi-agent farce, coordination failure

29.2 Abstract — FIVE BONDS: mission summary

Casino Royale (1967) is the spy farce in which five different operatives answer to the same codename, James Bond. This project — codename **FIVE 00s** — builds deterministic mission software around that premise. It implements and tests six concepts drawn from the film’s organizational failure: 1. **Coordination-farce model** (`coordination_farce.py`): a roster of 5 agents who all share the codename *James Bond*, assigned across 6 tasks. It detects **overlap**, **conflict**, and clearance violations, and computes a weighted coordination loss of **15** from 3 overlaps and 3 conflicts. 2. **Dispatch-chaos simulation** (`dispatch_chaos.py`): a tick-based dispatch centre issues 13 orders and enforces 5 protocol rules; 5 dispatch successfully, 8 are rejected, for a chaos index of **0.62**. 3. **Spoof protocol analyzer** (`spoof_analyzer.py`): lexical scoring of satirical ops doctrine against a sane protocol; the film’s standing orders score **45.0 / 100**. 4. **Identity-confusion model** (`identity_confusion.py`): information theory over the shared codename — routing carries only 0.075 normalized mutual information, leaving a dispatch ambiguity $H(\text{routed} \mid \text{intended})$ of 2.135 bits, and a Bayesian sighting resolves to *bond_cooper* with confidence 0.613. 5. **Baccarat table model** (`baccarat_table.py`): combinatorial deal odds exact for the infinite-deck model (player 0.4461, banker 0.4584, tie 0.0954) and a 5-Bond table collision probability of 0.704 under an i.i.d.-across-hands approximation. 6. **Escalation-ladder model** (`escalation_ladder.py`): the doctrine as an absorbing Markov chain — the spoof doctrine reaches catastrophe in 4.0 steps versus 56.0 under discipline, a 14.0× acceleration. All six models are deterministic (fixed seed, no random draws, no wall clock in artifacts) and are exposed through a **MissionProvider** implementing the frozen `bond_api` protocol, with a gadget registry and publication-style figures. Keywords: 5-agent coordination failure, agent dispatch simulation, protocol violations, satirical ops doctrine, identity confusion, baccarat odds, absorbing Markov chains, reproducible research.

29.3 Introduction — FIVE BONDS: mission framing, the operational problem, and how to read this chapter

The 1967 film *Casino Royale* [Huston et al., 1967] is not one Bond but many. Sir James Bond (played by David Niven) is recalled from retirement; the would-be successor is a baccarat prodigy (Peter Sellers as Evelyn Tremble); a Soviet defector (Ursula Andress as Vesper Lynd) doubles as a double-0; and James Bond’s anxious nephew (Woody Allen as Jimmy Bond) and a field agent (Terence Cooper) both answer to the same codename. Every operative introduces themselves as **James Bond**. The result is a coordination catastrophe dressed as a blockbuster: overlapping identities, overlapping assignments, and directly contradictory orders.

FIVE 00s turns that farce into a rigorous, deterministic model. This manuscript documents six complementary concepts, each implemented as a pure Python module with no mocks and verified by real computation:

- **Multi-agent coordination failure** (`coordination_farce.py`) — overlap and conflict detection over a roster of 5 overlapping Bonds across 6 tasks, with a weighted loss and a single-agent remedy.
- **Agent dispatch chaos** (`dispatch_chaos.py`) — a tick-based simulation in which the dispatch centre cannot tell its Bonds apart, so violations of 5 protocol rules accumulate.
- **Spoof protocol analysis** (`spoof_analyzer.py`) — automatic measurement of how far a satirical ops doctrine (the film’s standing orders) deviates from a sane protocol of 6 rules.
- **Identity confusion** (`identity_confusion.py`) — information theory [Shannon, 1948a] over the shared codename: a routing confusion matrix, entropy, mutual information, and Bayesian resolution of a dispatch sighting.
- **The baccarat table** (`baccarat_table.py`) — exact chemin de fer deal odds by enumeration, plus the birthday-model probability that two of the 5 Bonds at the table are dealt the same hand total.
- **The escalation ladder** (`escalation_ladder.py`) — doctrine as an absorbing Markov chain over 5 states, compared against a disciplined doctrine through expected absorption times.

The whole package is wired into the PROJECT BOND suite: a **MissionProvider** (`src/casino_royale_1967/mission.py`) implements the frozen `bond_api` protocol, the five mission phases are exposed as thin CLIs under `scripts/`, and 6 manuscript figures are generated deterministically from the same analyses the mission executes. Metrics in this manuscript are never hand-authored — they are injected live from `manuscript/config.yaml` and `src/casino_royale_1967/manuscript_variables.py`.

29.3.1 Reader’s guide

- **Methodology** describes the six algorithms in module order: assignment analysis, tick-based dispatch, lexical doctrine scoring, the information-theoretic identity model, exact baccarat enumeration, and the absorbing-chain escalation ladder.

- **Results** reports the canonical metrics and the 6 figures.
- **Experimental setup** lists the configuration parameters.
- **Reproducibility** describes how every artifact can be regenerated byte-identically.
- **Scope and related work** states what the models deliberately do not claim.

29.4 Methodology — FIVE BONDS: the analytical models and algorithms that drive the mission

29.4.1 1. The coordination-farce model

`src/casino_royale_1967/coordination_farce.py` models the organizational failure behind *Casino Royale*. The model has three parts: a static roster, an assignment set, and an analysis function.

Roster. A tuple of 5 **Agent** instances, each with an id, the actor who played them (David Niven, Peter Sellers, Ursula Andress, Woody Allen, Terence Cooper), a clearance level (1–5), a jurisdiction, and a codename. Crucially, **all five share the codename James Bond** — the roster is a duplicate-identity dataset by construction.

Tasks. A **Task** catalog of 6 operations drawn from the film’s plot, each with a required clearance and an exclusivity flag. Tasks include winning the baccarat championship, infiltrating SMERSH headquarters, guarding the casino vault, seducing Mata Bond, disabling the gamma-ray projector, and burning the baccarat table for the insurance claim. Mutually exclusive task pairs are captured in a conflict graph: *win the baccarat championship* conflicts with *burn the baccarat table*.

Analysis (`analyze_assignments`). Given a tuple of **Assignments** (agent, task, priority), the function deterministically computes:

- **Overlap count** — every extra distinct agent beyond the first on a task. A task with three Bonds contributes two overlaps.
- **Conflict count** — every pair of *distinct* agents drawn one from each side of a conflicting task pair. A single operative assigned to both sides contributes nothing: that is a single-agent scheduling problem, not an inter-agent conflict. Each conflict-graph entry must name exactly two tasks, and a malformed entry raises rather than being silently skipped.
- **Clearance violations** — assignments where the agent’s clearance is below the task’s requirement (reported, not counted twice in the loss).
- **Coordination loss** — a weighted sum `loss = overlap_weight * overlap_count + conflict_weight * conflict_count`, with weights 2 and 3.

The canonical default `DEFAULT_ASSIGNMENTS` assigns 8 orders and produces the farce documented in the results: three Bonds at the baccarat table, Cooper burning the same table, Vesper and Jimmy both at SMERSH headquarters.

Remedy (`deduplicate_assignments`). Single-agent discipline keeps the lexicographic maximum of (`priority`, `agent_id`) per task — highest priority wins, and an exact tie goes to the greatest agent id, which makes the choice a pure function of the assignment *set* rather than of its ordering — and retires every other agent on that task. Those retired agents are exactly the `overlap_victims` the analysis reports: both are derived from one shared winner function, and `tests/test_coordination_farce.py` asserts the two mappings are equal rather than merely the same size. Applied to the canonical farce it keeps 5 assignments and drops 3, eliminating every overlap and every duplicate-agent conflict. Only one irreducible clash remains: a single agent must still win the baccarat championship while a second burns the same table for the insurance claim, so one conflict (loss equal to the conflict weight) persists.

29.4.2 2. The dispatch-chaos simulation

`src/casino_royale_1967/dispatch_chaos.py` simulates a dispatch centre that cannot tell its agents apart. It is a tick-based, priority-ordered scheduler with no randomness and no wall clock.

Orders. Each `DispatchOrder` names an order id, agent, task, priority, and issue tick. Orders are processed in `issued_tick` order; within a tick, higher priority first, ties broken by order id. State is deterministic: agents committed to a task persist across ticks (task occupancy), and agents committed to a task within the current tick are re-checked for double-booking.

Protocol rules. 5 rules are enforced per order, in this order:

1. `duplicate_order` — an order id issued twice is rejected;
2. `exclusive_task` — an exclusive task may hold only one agent;
3. `clearance` — the agent’s clearance must meet the task requirement;
4. `double_booking` — one agent may not take two tasks in the same tick;
5. `duplicate_codename` — if another agent with the *same codename* already occupies a task, the order is rejected (the dispatch centre mistakes the Bonds for one another).

A rejected order is dropped, never requeued: chaotic dispatch loses orders.

Output. `simulate_dispatch` returns a `DispatchReport` with total, dispatched, and rejected counts, every `DispatchViolation` with the rule that fired, the success ratio, and a chaos index equal to `violations / total_orders`, normalised to [0, 1]. The canonical scenario issues 13 orders within a horizon of 100 ticks (orders issued beyond the horizon are ignored).

29.4.3 3. The spoof protocol analyzer

`src/casino_royale_1967/spoof_analyzer.py` measures how far a satirical ops doctrine deviates from a sane protocol. It combines lexical marker scoring, rule-contradiction detection, and oxymoron detection.

Sane doctrine. `REAL_OPS_RULES` lists 6 rules (classified identity, single point of command, covert discretion, proportional force, no fratricide, single agent per operation). The analyzer treats these as the ground truth the farce inverts.

Markers. `SPOOF_MARKERS` weights absolutist or inverted vocabulary (**every**, **anyone**, **all**, **none**, **maximum**, **in case of doubt**, **blow up**, **shoot**, **identical**, **never**, **always**). Matching is word-boundary aware so **any** does not fire inside **anyone** and **all** does not fire inside **calling**.

Scoring. For each directive, `analyze_doctrine` computes:

- the **marker score** — the sum of weights of markers present;
- **inverted rules** — real rules whose keyword appears together with an absolutist marker, i.e. a discretionary rule turned absolute;
- **oxymorons** — self-contradicting phrase pairs from `OXYMORON_PAIRS` (classified + publicity, discreet + explosion,...).

The per-directive score is `marker_sum + 2.0 * len(inverted_rules) + 1.5 * len(oxymorons)`. The aggregate **spoof index** is the mean score scaled to `[0, 100]` (`min(100, mean * 10)`), rounded to one decimal. The film's canonical standing orders comprise 8 directives.

29.4.4 4. The identity-confusion model

`src/casino_royale_1967/identity_confusion.py` quantifies the codename collapse with information theory and Bayesian inference.

Confusion matrix. Each agent is represented by a feature vector: its clearance normalized to `[0, 1]`, concatenated with a one-hot over the roster's distinct jurisdictions (diplomacy, gambling, seduction, accounting, field ops). Routing confusion `confusion[i][j]` — the probability an order intended for agent i is routed to agent j — is a row-softmax over pairwise feature similarity $\mathbf{f}_i \cdot \mathbf{f}_j$ scaled by a temperature. The matrix is row-stochastic and deterministic. Note that the codename contributes no feature at all, because it is identical for every agent; this is exactly why the routing signal is so weak.

Entropy / mutual information. With a uniform prior over the five Bonds, the model computes these quantities, all in bits [Shannon, 1948a, Cover and Thomas, 2006]:

- $H(\text{intended})$ — Shannon entropy of the intended identity (2.322 bits for a uniform roster);
- $H(\text{routed})$ — entropy of the marginal routing distribution (2.310 bits);
- $H(\text{routed} \mid \text{intended})$ — the forward conditional entropy, reported as the **dispatch ambiguity** of 2.135 bits: given the Bond an order was meant for, this much uncertainty remains about where it lands;
- $I = H(\text{routed}) - H(\text{routed} \mid \text{intended})$ — the mutual information (0.175 bits actually recovered by observing the routing);
- $H(\text{intended} \mid \text{routed}) = H(\text{intended}) - I$ — the *reverse* conditional entropy (2.147 bits), what remains unknown about the intended Bond once the routing has been seen. `residual_identity_entropy` computes it separately precisely because it is not the same number as the dispatch ambiguity, and swapping the two is the arithmetic error the results section calls out;
- $\text{NMI} = I / \max(H(\text{intended}), H(\text{routed}))$ — the mutual information normalized to `[0, 1]` [Strehl and Ghosh, 2002], where 0 means routing is independent of intent.

Because all five agents share one codename, the *codename* itself carries 0 bits: the shared name is pure identity loss, and only the clearance and jurisdiction features carry any signal at all.

Bayesian resolution. Given a dispatch sighting's evidence — whether the presented jurisdiction matched the task's and whether clearance was sufficient — `resolve_identity` computes the posterior over the roster with a uniform prior and an independent jurisdiction/clearance likelihood table. The argmax is the resolution, its mass the confidence.

29.4.5 5. The baccarat table model

`src/casino_royale_1967/baccarat_table.py` models the casino set-piece with exact enumeration.

Hand valuation. Baccarat hands total modulo 10; 10/J/Q/K count as 0; a two-card total of 8 or 9 is a *natural* that ends the deal immediately. The player draws on 0-5 and stands on 6-7; the banker follows the standard chemin de fer table keyed on its total and the player's third card.

Deal odds. `deal_outcome_probabilities` enumerates every draw outcome (four to six cards) under the drawing rules, weighting each card by its value's share of the deck — the industry-standard infinite-deck baccarat odds computation. For the 52-card standard it reproduces the canonical published figures (player 0.4461, banker 0.4584, tie 0.0954), with the banker's historical advantage of +0.0123.

The farce metric. `two_card_total_counts` computes the *exact* no-replacement marginal distribution of a two-card hand total from the actual deck; `hand_collision_probability` then applies the birthday bound — the elementary symmetric polynomial $n! \cdot e_n(p_0..p_9)$ — to answer: across 5 Bonds at the table, what is the probability at least two are dealt the same hand total? Under the documented i.i.d.-across-hands approximation the answer is 0.704.

That approximation is the model’s one inexact step and is stated rather than hidden. The marginal for a *single* hand is exact and without replacement, but the birthday product treats the 5 hands as independent draws from that marginal, whereas a real deal removes each dealt card from the shoe and so correlates the hands. `test_iid_approximation_is_not_exact` in `tests/test_baccarat_table.py` brute-forces every deal of a small deck and shows the two values disagree, so the approximation is documented by a failing comparison rather than asserted. The exact figure for a 52-card shoe and five hands is not computed here.

29.4.6 6. The escalation-ladder model

`src/casino_royale_1967/escalation_ladder.py` models doctrine as an absorbing Markov chain (Kemeny-Snell analysis [Kemeny and Snell, 1960], exact numpy linear algebra).

The 5-state ladder `quiet` $\rightarrow$ `rumour` $\rightarrow$ `scandal` $\rightarrow$ `spectacle` $\rightarrow$ `catastrophe` has `catastrophe` absorbing. A doctrine is a row-stochastic transition matrix (`SPOOF_DOCTRINE_TRANSITIONS`, `SANCTIONED_DOCTRINE_TRANSITIONS`).

Deriving the absorbing set. Validation rejects any matrix that is not square, not row-stochastic, or has a negative entry, and then identifies the absorbing states *from the matrix itself*: a row `i` with `P[i, i] = 1` and no off-diagonal outflow. A matrix with no absorbing row, or with no transient row left, is rejected. Nothing keys off a state’s *name*, so a caller may supply any state labels in any order and get the same physics — a property pinned by a reversed-ladder regression test.

The analysis then computes:

- the **fundamental matrix** $N = (I - Q)^{-1}$ over the transient states;
- the **expected absorption time** $t = N @ 1$ from each state — the expected steps to catastrophe;
- the **spectral radius** of `Q` as the largest complex modulus of its eigenvalues (a value below 1 guarantees absorption);
- the **absorption probability** $B = N R$, summed over every absorbing state.

`compare_doctrines` reports the **escalation index** (expected steps from `quiet`) under each doctrine and the acceleration factor `sanctioned / spoof`: the farce reaches catastrophe in 4.0 steps versus 56.0 under discretion — 14.0 $\times$ faster.

29.5 Results — FIVE BONDS: measured outcomes, headline numbers, and what they establish

29.5.1 The five-Bond coordination farce

The canonical assignment set of 8 orders across the 5-agent roster produces 3 overlaps and 3 conflicts. Three Bonds crowd the baccarat table; Vesper and Jimmy both infiltrate SMERSH headquarters; and Cooper’s order to burn the baccarat table conflicts with the three agents assigned to win the championship on that same table. Conflicts are counted between *distinct* agents only, so an operative holding both sides of a mutually exclusive pair is a scheduling problem rather than a conflict. 1 clearance violation is recorded: `bond_jimmy` (Woody Allen), clearance 2, sent to *infiltrate SMERSH headquarters* — an operation requiring clearance 4.

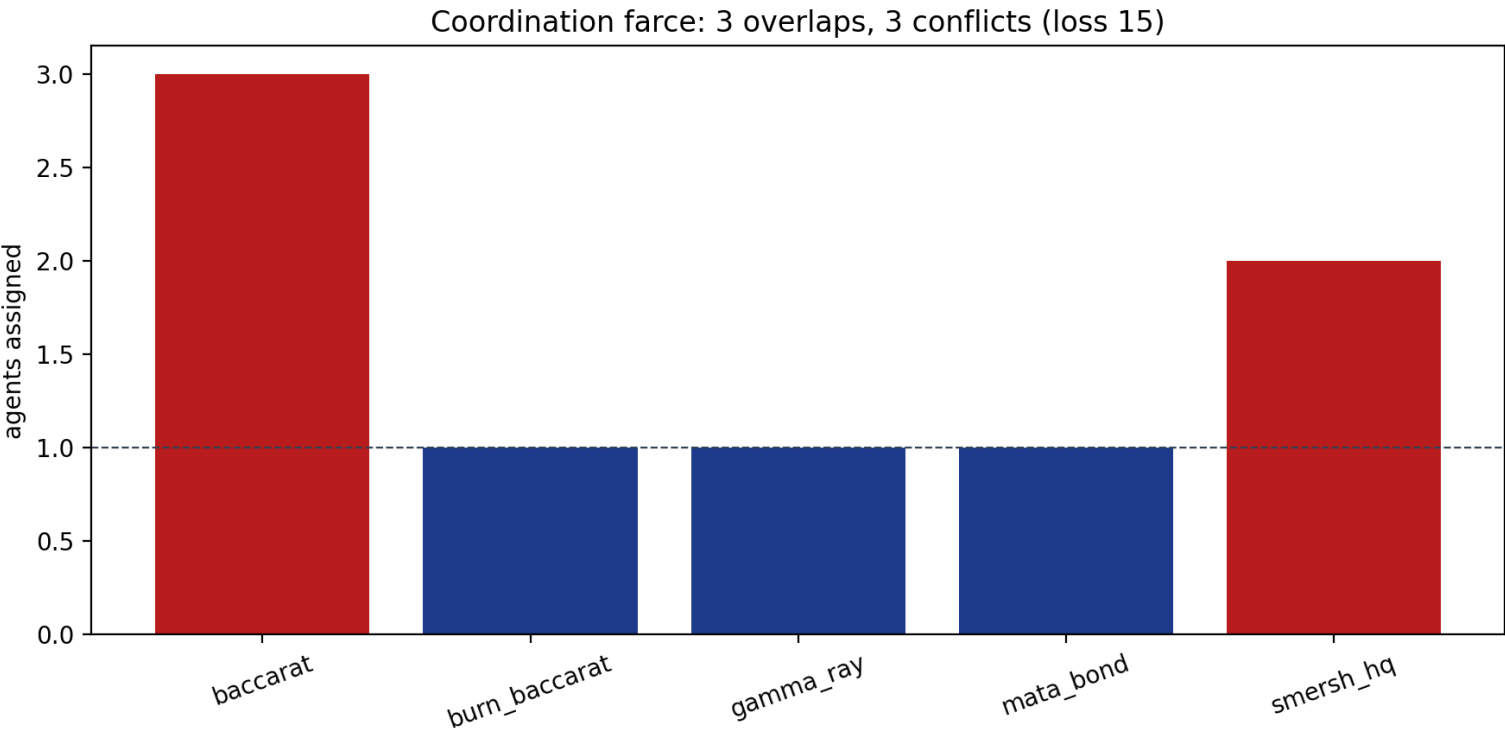


Figure 111: Per-task agent load; bars above the dashed line are overlaps.

The weighted **coordination loss** is **15** (weights 2 for overlap and 3 for conflict). Applying single-agent discipline keeps 5 of the 8 assignments and drops 3, eliminating every overlap; only the irreducible clash between winning and burning the baccarat table remains, at one conflict (loss 3).

29.5.2 Dispatch chaos

The dispatch simulator issues 13 orders; 5 dispatch successfully and 8 are rejected as protocol violations, for a success ratio of 0.38 and a chaos index of **0.62**. All 5 protocol rules fire at least once (5 of 5 distinct rules triggered), so the scenario is a positive control for the whole rule set rather than a narrow subset:

Protocol rule	Rejections
exclusive_task	2
clearance	2
double_booking	2
duplicate_order	1
duplicate_codename	1

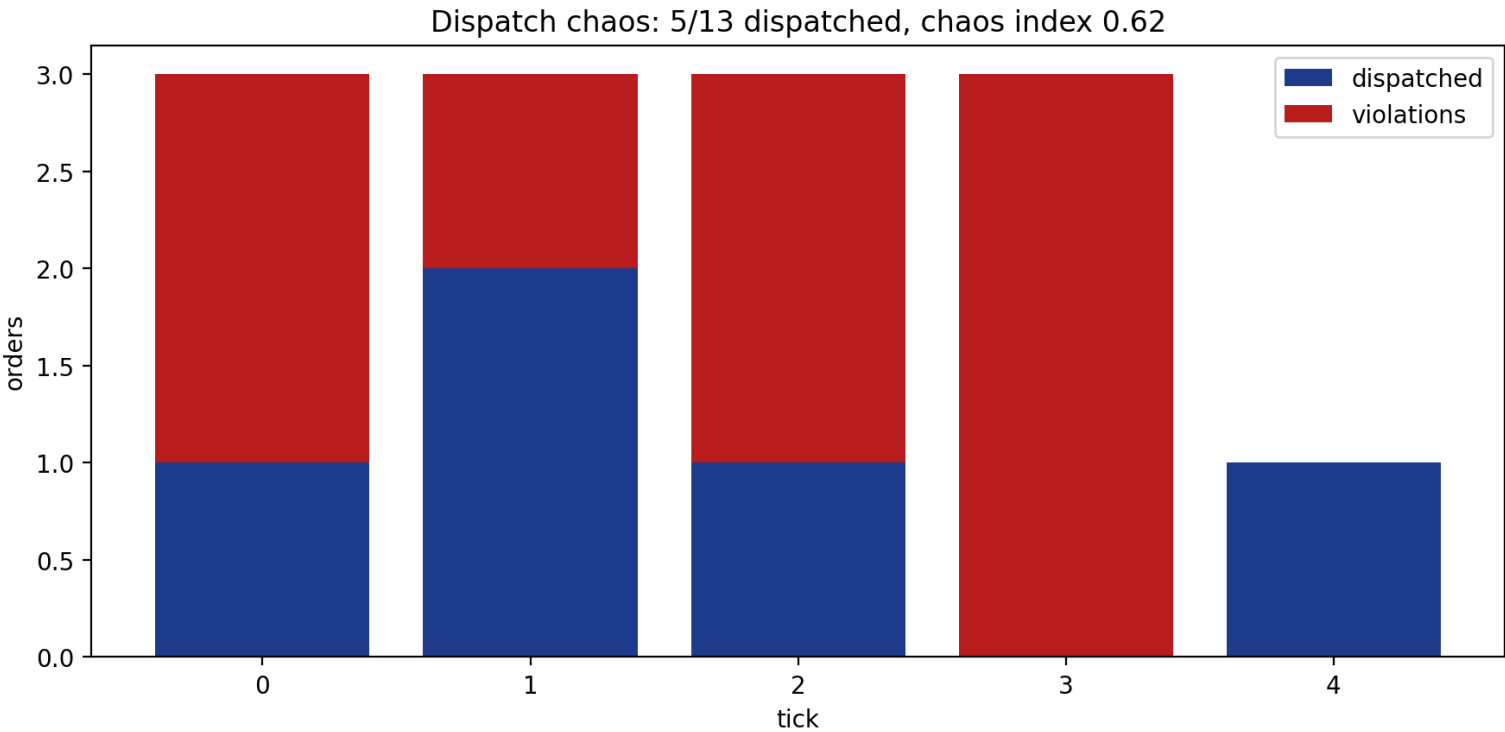


Figure 112: Dispatch timeline: accepted orders vs protocol violations per tick.

The dominant failure mode is identity collapse: because all five agents share the codename *James Bond*, the dispatch centre repeatedly mistakes one agent for another and misroutes the under-cleared nephew onto cleared operations.

29.5.3 Spoof ops doctrine

The film’s standing orders of 8 directives score a **spoof index of 45.0 / 100**, with a mean per-directive score of 4.50. The worst offender is “*In case of doubt, blow up the table.*” — an inversion of the proportionality principle.

Contradiction detection attributes the scores across the sane rule surface (identity, single-agent discipline, command, discretion, force, fratricide): 6 of the 6 declared rules are contradicted at least once, so the analyzer exercises every rule rather than a narrow subset.

29.5.4 Identity confusion

The five Bonds share one codename, and the resulting identity collapse is measured in bits. Routing carries only **0.075 normalized mutual information** (a near-zero value: the observed routing reveals almost nothing about which physical Bond was intended) — 0.175 bits of mutual information in absolute terms. Two distinct conditional entropies follow from that, and they are not interchangeable [Cover and Thomas, 2006]:

Spoof protocol analyzer: doctrine spoof index 45.0 / 100

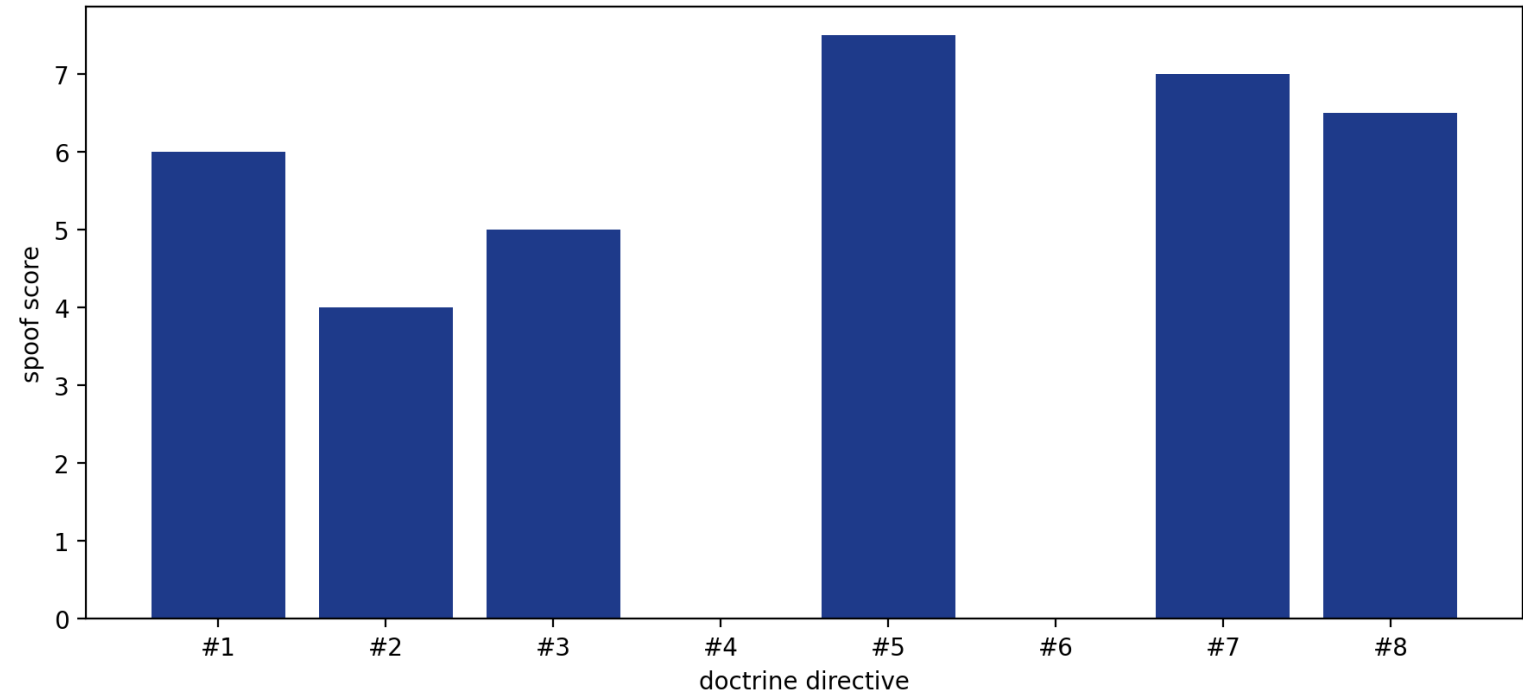
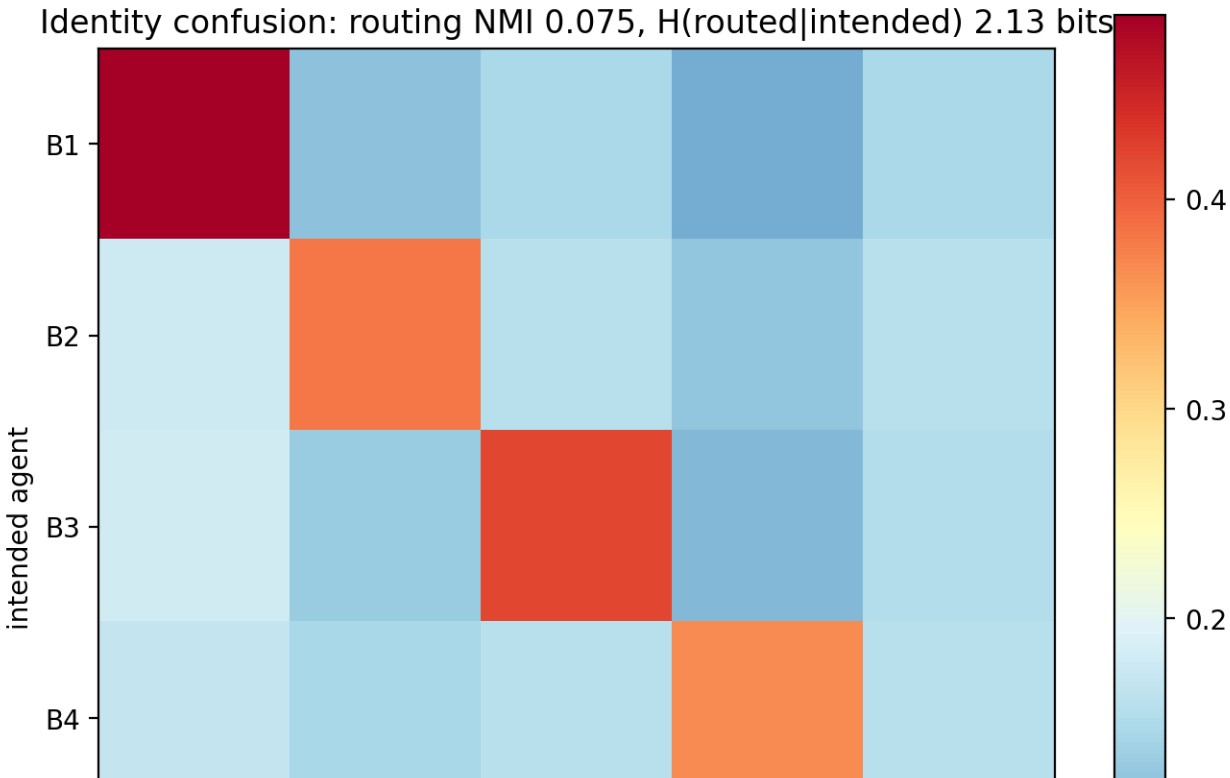


Figure 113: Per-directive spoof scores with the aggregate index in the title.

Direction	Reading	Bits
H(routed \ intended)	knowing who the order was <i>for</i> , how uncertain its destination is	2.135
H(intended \ routed)	seeing where the order <i>landed</i> , how uncertain its author’s intent is	2.147

The first subtracts the mutual information from the routing entropy (2.310 bits); the second subtracts it from the prior entropy of the intended identity (2.322 bits). This manuscript reports the first as the **dispatch ambiguity**; the phrase “bits of identity lost per dispatch” refers to that forward direction and must not be computed as prior minus mutual information, which yields the second, larger figure. A Bayesian sighting (jurisdiction match + clearance sufficiency) at the SMERSH operation resolves to *bond_cooper* with posterior confidence 0.613.



not a rounding detail. Its magnitude for a full standard shoe is not quantified here; `TODD0.md` records that gap as an accepted, open limitation.

Casino table odds: 5-Bond hand-collision probability 0.704

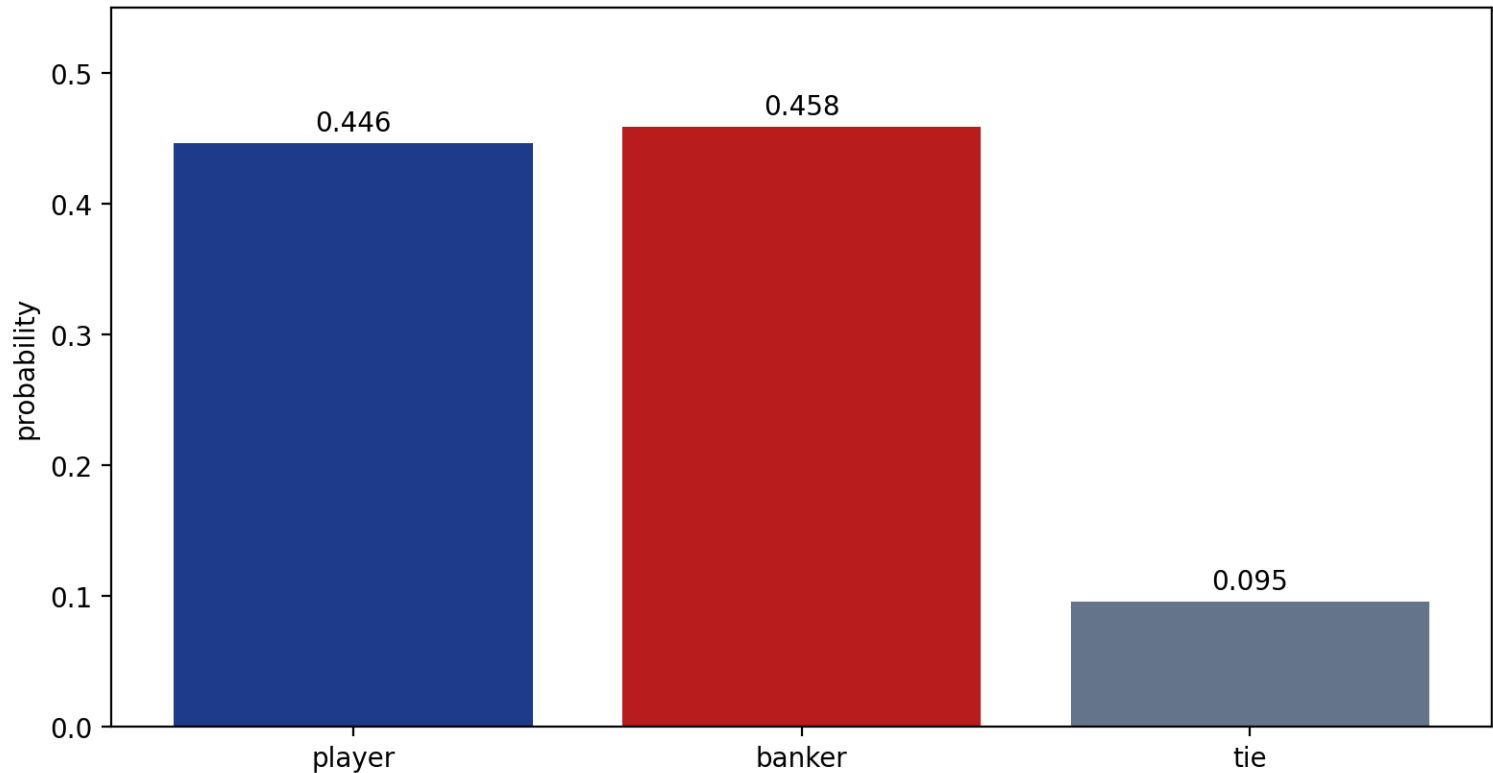


Figure 115: Casino table deal odds with the five-Bond collision probability.

29.5.6 Escalation ladder

Modelled as an absorbing Markov chain over the 5-state ladder `quiet` → `rumour` → `scandal` → `spectacle` → `catastrophe`, the spoof doctrine reaches catastrophe in **4.0 expected steps**, while a disciplined doctrine takes 56.0 — the farce escalates **14.0× faster**.

Absorption is certain rather than assumed: the transient block of the spoof doctrine has spectral radius 0.400 (strictly below one), and the absorption probability into *catastrophe* from *quiet* is 1.000. The absorbing set is derived from the transition matrix itself — a row with a unit diagonal and no outflow — never from a state’s name, so relabelling or reordering the ladder cannot change the analysis.

29.6 Conclusion — FIVE BONDS: findings, verdict, and what the mission establishes

FIVE 00s turns the organizing joke of *Casino Royale* (1967) — five operatives, one codename — into a rigorous, fully deterministic research model. The project delivers six complementary concepts, each implemented in pure Python, exhaustively tested with real computation, and wired into the PROJECT BOND mission protocol:

- 1. the **coordination-farce model**, which turns the film’s overlap and conflict into measurable quantities — 3 overlaps, 3 conflicts, and a coordination loss of 15 — and demonstrates that single-agent discipline removes all overlaps;
- 2. the **dispatch-chaos simulation**, which reproduces the logistics failure of a dispatch centre that cannot tell its Bonds apart (8 of 13 orders rejected; chaos index 0.62);
- 3. the **spoof protocol analyzer**, scoring satirical ops doctrine at 45.0 / 100;
- 4. the **identity-confusion model**, showing the shared codename collapses routing to 0.075 normalized mutual information, leaving a forward dispatch ambiguity $H(\text{routed} \mid \text{intended})$ of 2.135 bits and a reverse residual $H(\text{intended} \mid \text{routed})$ of 2.147 bits, with a sighting resolved by Bayes to *bond_cooper*;
- 5. the **baccarat table model**, deriving deal odds exact for the infinite-deck model (player 0.4461, banker 0.4584) and an approximate 0.704 probability — i.i.d. across hands — that two of the 5 Bonds are dealt the same hand total;
- 6. the **escalation-ladder model**, showing the farce accelerates to catastrophe 14.0× faster than discretion (4.0 vs 56.0 steps).

The technical guarantees matter as much as the content: deterministic artifacts, zero mocks, ≥90% line+branch coverage, exact enumeration and absorbing-chain linear algebra, and a thin `MissionProvider` adapter whose public shape stays stable for the fleet. A farce, modelled seriously, yields the same reproducibility contract as any research experiment.

Doctrine escalation: spoof 4.0 steps vs sanctioned 56.0 (14.0x faster)

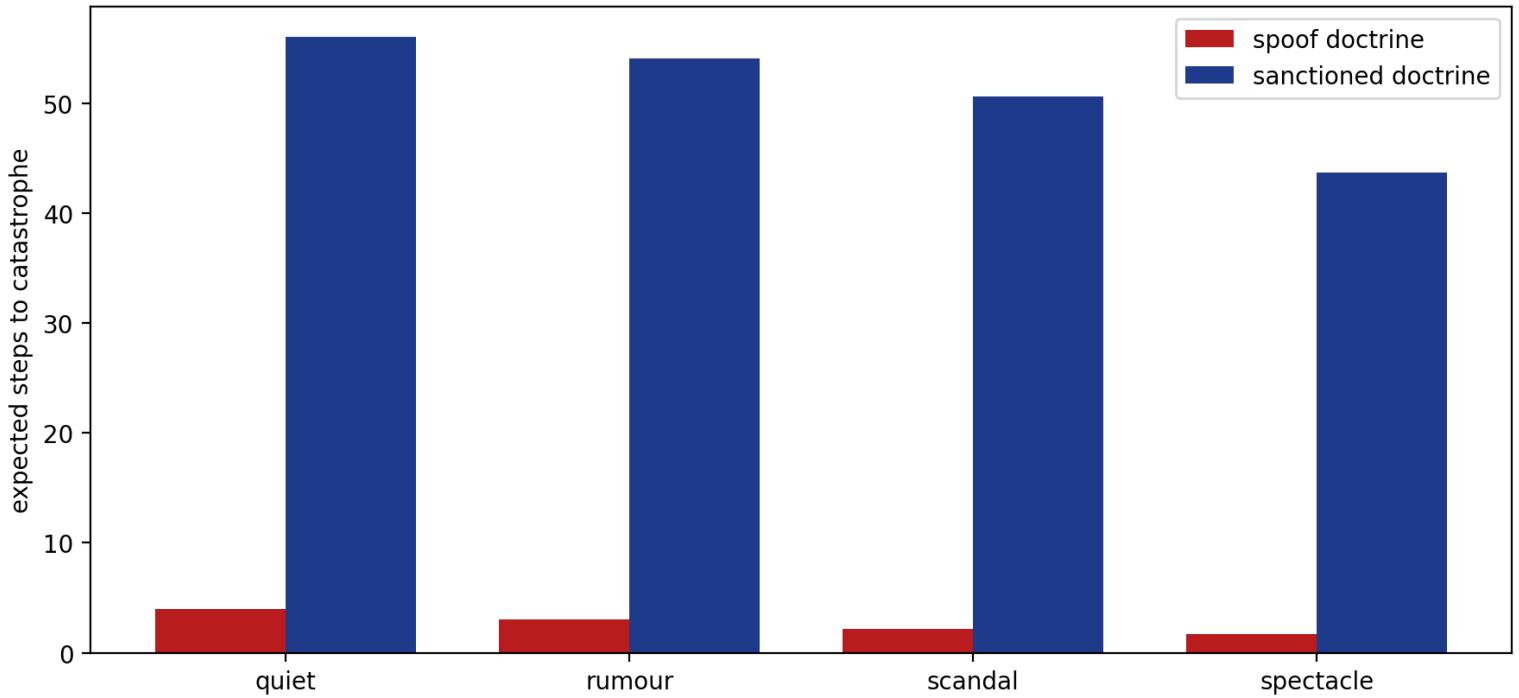


Figure 116: Expected steps to catastrophe per state, by doctrine.

29.7 Experimental Setup — FIVE BONDS: canonical scenarios, parameters, and configuration

29.7.1 Configuration

All identity and experiment parameters are declared in `manuscript/config.yaml` and injected as tokens by `src/casino_royale_1967/manuscript_variables.py` — never hard-coded in prose.

Parameter	Value
Mission identity	Casino Royale (1967)
Mission codename	FIVE 00s
Package version	0.1.0
Number of agents	5
Number of tasks	6
Overlap loss weight	2
Conflict loss weight	3
Dispatch horizon (ticks)	100
Dispatch protocol rules enforced	5
Sane ops rules in the doctrine baseline	6
Escalation-ladder states	5
Casino table (Bonds seated)	5
Figures rendered	6

Values in this table are read from `manuscript/config.yaml` where the parameter is a free choice (loss weights, dispatch horizon) and derived from the domain modules where the parameter is structural (rule counts, ladder size, figure count), so neither source can drift from the code.

29.7.2 Software environment

- Python ≥ 3.10 — the declared supported range. The artifacts do not record which interpreter within that range produced them, which is what keeps them byte-identical across it.
- `numpy` and `matplotlib` for computation and rendering
- `bond-api` (frozen `MissionProvider` protocol, local path dependency)
- Deterministic seed 7; no random draws, no wall clock in artifacts.

29.7.3 Artifact regeneration

Every artifact is regenerable from `scripts/`:

```
uv run python scripts/mission.py execute # mission outcome (deterministic JSON)
uv run python scripts/mission.py figures # the concept figures
uv run python scripts/z_generate_manuscript_variables.py # tokens + resolved tree
```

The last command is the authoritative one: it recomputes the token map, re-renders every figure, and rewrites `output/manuscript/` with every token marker resolved, failing loudly if any marker is unresolvable.

29.8 Reproducibility — FIVE BONDS: verification gates, deterministic regeneration, and artifacts

29.8.1 Determinism guarantees

Every artifact in this project is deterministic by construction:

- **Fixed seed.** The mission uses seed 7; the dispatch simulation makes no random draws at all — order processing is a pure function of the input order stream.
- **No wall clock in any persisted artifact.** Coordination, dispatch, and doctrine reports carry no timestamps; `Provenance.wall_time_s` is 0.0; and the manuscript token map reads no clock at all — `generate_variables` takes no `now=` parameter, because an injectable clock with a wall-clock default is still a wall clock. This manuscript’s date, 2026-08-04, is the committed `paper.date` in `manuscript/config.yaml`; edit that file to change it, and it is otherwise fixed.
- **No interpreter state either.** The declared Python range is `>=3.10` — the committed `paper.python_requires`, held equal to `requires-python` in `pyproject.toml` by test. The token map introspects nothing about the machine that ran it.
- **Live computation.** Manuscript metrics are recomputed from the domain core at hydration time, so prose can never drift from code.

Because the token map is a pure function of committed files, regenerating with `scripts/z_generate_manuscript_variables.py` reproduces `output/data/manuscript_variables.json`, `output/manuscript/*.md`, and `../figures/*.png` byte for byte. That is not an aspiration: `tests/test_regeneration_determinism.py` runs the generator twice in succession, hashes every artifact under `output/`, and fails on any difference.

29.8.2 Canonical metric snapshot

Metric	Value
Assignments	8
Overlaps	3
Conflicts	3
Coordination loss	15
Clearance violations	1
Single-agent deduplication of assignments (kept / dropped)	5 / 3
Dispatch orders	13
Dispatched / rejected orders	5 / 8
Protocol violations	8
Chaos index	0.62
Success ratio	0.38
Spoof index	45.0 / 100
Identity NMI	0.075
Dispatch ambiguity $H(\text{routed} \setminus \text{intended})$ (bits)	2.135
Residual identity $H(\text{intended} \setminus \text{routed})$ (bits)	2.147
Baccarat player / banker	0.4461 / 0.4584
Table collision (5 Bonds, i.i.d.-across-hands approximation)	0.704
Escalation (spoof / sanctioned steps)	4.0 / 56.0
Acceleration factor	14.0×

29.8.3 Verification

```
uv run pytest tests/ --cov=src --cov-fail-under=90
uv run ruff check src/ scripts/ && uv run ruff format --check src/ scripts/
uv run mypy src/ scripts/
```

The gate enforces `>=90%` line+branch coverage on `src/`, zero mocks, and a clean git tree after regeneration. Figures and the resolved manuscript tree live under `output/`, which is git-ignored and regenerable.

29.9 Scope and Related Work — FIVE BONDS: boundaries, positioning, and relationship to the literature

29.9.1 Scope

FIVE 00s models the coordination failure motif of *Casino Royale* (1967) at the level of deterministic, hand-built algorithms. It does not attempt:

- a general agent-based simulation framework (the tick model is purpose-built for the farce, not a full discrete-event engine);
- machine-learned or stochastic dispatch policies (dispatch is a pure function of the deterministic order stream);
- a normative claim about real organizational governance — the “chaos index” is a satirical diagnostic, not a validated operations metric.

The satirical doctrine in `CANONICAL_SPOOF_DOCTRINE` is analyzed as *text*: the analyzer measures lexical inversion and rule contradiction, and makes no claim about the film’s production history or intent.

29.9.2 Related work

The work sits at the intersection of several research threads:

- **Multi-agent coordination and conflict.** The overlap/conflict detection follows the classic line of research on coordination failure in multi-agent systems, where redundant or duplicative agents degrade system performance. The weighted-loss formulation is a small, reproducible instance of that family.
- **Information theory of identity.** The identity-confusion model applies Shannon’s entropy and mutual information to quantify how much identity a shared codename destroys; the Bayesian resolution is a standard posterior update over the roster (Shannon 1948; Cover & Thomas 2006). The normalization of mutual information by the larger marginal entropy follows the convention of Strehl & Ghosh (2002).
- **Deterministic simulation and protocol verification.** The dispatch simulator’s rule-checker mirrors property-based protocol enforcement: each order is checked against a fixed rule set, and violations are recorded as first-class data.
- **Gambling mathematics.** The baccarat table’s deal odds reproduce the canonical published figures of the chemin de fer drawing rules; the hand-collision metric is a birthday-bound application over the exact two-card-total distribution [Ethier, 2010, Shackleford, 2023]. Deck count shifts these probabilities only slightly and does **not** reverse their order: the banker bet is still the better side at one deck in the published tables [Shackleford, 2023]. An earlier draft of this section asserted the opposite — that single-deck play inverts the banker edge — which its own cited source contradicts. This model cannot settle the question either way: `deal_outcome_probabilities` weights each card by its value’s share of the shoe *with replacement*, so it is an infinite-deck computation and carries no finite-deck depletion effect at all.
- **Absorbing Markov chains.** The escalation ladder’s fundamental matrix and expected absorption times follow Kemeny and Snell’s classical treatment.
- **Reproducible computational research.** The project ships live-computed manuscript metrics, deterministic artifacts, and a frozen protocol adapter.

The *Casino Royale* (1967) material itself is in the public cultural domain; this project uses its plot beats (baccarat championship, SMERSH headquarters, the gamma-ray projector, Mata Bond, the five-Bond premise) as a satirical dataset, not as a claim about the film.

29.10 Sources — FIVE BONDS: bibliography

Shannon [1948a]; Kemeny and Snell [1960]; Ethier [2010]; Shackleford [2023]; Huston et al. [1967]; Cover and Thomas [2006]; Strehl and Ghosh [2002]

30 Never Say Never Again (1983) — REPLAY

film package · package codename REPLAY. Mission **REMOUNT**: replay the legacy operation as a remount; diff the original vs remount outcomes; migrate the legacy intelligence assets; assess remount detection risk and mission reliability; allocate remastered assets by marginal gain; forecast the replay learning curve.

30.1 Concepts — REPLAY: domain and operational focus

legacy remount, replay

30.2 Abstract — REPLAY: mission summary

Never Say Never Again: REMOUNT — Special-Agent Mission Software (Replaying a legacy operation: op diff, intel migration, escalation risk, marginal-gain allocation, and replay learning, version 0.1.0) presents REMOUNT, a deterministic engine for *legacy mission replay*. When an operation is re-opened years after its original run, its scenario must be re-executed — as a *remount* — with remastered intelligence, its outcomes compared against the original, and its legacy assets migrated so they can be consumed safely again. Drawing on the 1983 film *Never Say Never Again* — itself a remount of the 1965 *Thunderball* operation (Never Say Never Again, mission codename REMOUNT) — the engine models each of 4 objectives through a smooth success curve over effective intel versus difficulty. The canonical replay yields an original command score of 0.3111 and a remount command score of 0.5161 (weighted delta +0.2050, overall verdict *improved*). Per-objective, the remount improves 4 objectives, regresses 0, and leaves 0 unchanged, lifting the count of fulfilled objectives from 1 to 3. The engine migrates 3 legacy intelligence dossiers (1 gadget, 2 intel), re-fingerprinting each (SHA-256) so the remount can verify integrity before consumption (catalog fingerprint 1a1e05c4f464). The remount is not costless. Replaying the op accumulates a detection hazard: over a campaign of replay operations the probability of alerting the adversary reaches 0.7769 (expected 4.00 operations before alert), while the mission’s block-structured reliability is 0.9578 — a campaign expected value of +18.10. Scarce remastered assets are therefore allocated by marginal gain (6 assets deployed, 0.8030 of the saturation ceiling), and repeated rehearsal pushes the command score along an exponential learning curve toward 0.7010 (delta +0.3899, reaching the target by iteration 3). Everything computes deterministically — no random draws, no external calls — so the remount is reproducible to the byte on Python 3.14.6 (document date 2026-08-05). Every quantity above is a token injected from the running code; none is transcribed by hand. The models themselves are standard and cited, not invented here: the contribution is their deterministic, auditable composition into a single remount pipeline, evaluated at one canonical parameter set rather than calibrated against operational data.

30.3 Introduction — REPLAY: mission framing, the operational problem, and how to read this chapter

30.3.1 Never Say Never Again and the remount battalion

Never Say Never Again (1983) is a curious entry in the Bond canon: it is a remake — a *remount* — of *Thunderball* (1965), produced without the Eon series and starring a returning Bond. Rather than a new story it re-stages an old one, modernized and re-equipped. This is precisely the operational meaning of a **legacy mission remount**: keep the objective set, refresh the resources, and re-run the scenario against the current threat model.

REMOUNT (REMOUNT, package 0.1.0) operationalizes that idea as software for Never Say Never Again. Its pure domain core provides seven capabilities in as many modules — six that compute the remount and a seventh that checks how much those six can be believed at parameter values the model did not assume:

1. **Legacy mission replay** (`mission_replay.py`) — deterministically re-run a prior operation’s scenario under a remount configuration.
2. **Op diff** (`op_diff.py`) — compare the original outcome against the remount, per objective and in aggregate.
3. **Legacy asset migration** (`legacy_assets.py`) — carry forward, re-fingerprint, and verify the intelligence assets the remount consumes.
4. **Remount risk** (`remount_risk.py`) — price the re-op: the detection hazard that accumulates across replays, the block-structured mission reliability, and the resulting campaign expected value.
5. **Asset allocation** (`asset_allocator.py`) — spend a scarce budget of remastered assets across objectives by marginal gain.
6. **Replay learning** (`learning_curve.py`) — forecast how far repeated rehearsal can push the command score, and when it stops paying.
7. **Sensitivity** (`sensitivity.py`) — sweep each of the three model parameters the conclusions rest on and report the sub-range over which the two headline conclusions still hold, so the reader knows how much to trust them rather than taking a single point as given.

The first three answer the operative question of any re-op: *is the remount actually an improvement, and can we trust the intel it relied on?* The last three answer the question that follows immediately: *and is it worth running at all, given what a replay costs and what rehearsal can still buy?* A remount that improves the command score can still be a bad operation if the accumulated detection hazard outweighs the gain — so the engine reports both halves rather than only the flattering one.

30.3.2 Why determinism

A remount is an audit as much as an operation. Every number produced — command score, objective delta, asset fingerprint — must be reproducible by an independent reviewer without re-running the whole suite. The engine therefore makes **no random draws and no external calls**: its success model is a fixed Logistic curve, its hashing is canonical SHA-256, and two runs under the same inputs are byte-identical.

30.4 Methodology — REPLAY: the analytical models and algorithms that drive the mission

30.4.1 Scenario model

A scenario is an ordered set of 4 objectives, each a tuple of `objective_id`, a positive `weight`, a `difficulty` in $[0, 1]$, and a `baseline_intel` in $[0, 1]$. The canonical dataset, THUNDERBALL-65 (see `mission_replay.py`), reopens the 1965 operation. Weights need not sum to one; they are normalized when the command score is formed.

30.4.2 Remount configuration

A remount applies three deltas (`RemountConfig`):

- `intel_bonus` — extra effective intel added to every objective (reinforced intelligence).
- `doctrine` — a multiplicative factor on effective intel (doctrine/doctrine adjustments).
- `realism_ceiling` — a cap on effective intel so optimistic assumptions cannot overwhelm the model.

The effective intel for an objective is `clamp((baseline + bonus) * doctrine, 0, realism_ceiling)`.

30.4.3 Success model

For each objective the engine computes a deterministic success score with the logistic (Verhulst) curve [Verhulst, 1838]:

$$\text{score}(\Delta) = \frac{1}{1 + e^{-S\Delta}}, \quad \Delta = \text{eff_intel} - \text{difficulty}$$

where S is the sensitivity (default 6). A score at or above the `success_threshold` (default 0.5) counts the objective as *achieved*. The command score is the weight-weighted mean of per-objective scores. The implementation evaluates the curve in its numerically stable branched form (`exp(z) / (1 + exp(z))` for negative z) so that a large negative margin cannot overflow.

The choice of S is a modelling choice, not a fitted parameter: it sets how sharply an intel surplus converts into success, and no operational data is used to calibrate it.

30.4.4 Op diff

Given the original and remount replays, `diff_outcomes` aligns objectives by id (in the original's order) and computes each objective's `delta` and a verdict: `improved` when `delta > epsilon`, `regressed` when `delta < -epsilon`, else `unchanged`. The aggregate `weighted_delta` reuses the original scenario's objective weights, and the overall verdict follows the sign of `weighted_delta`.

30.4.5 Remount risk (escalation hazard and reliability)

`remount_risk.py` models the price of a replay. Detection follows a memoryless exponential (Poisson) hazard [Ross, 2019] — $P(\text{alert within } N \text{ replays}) = 1 - \exp(-\text{rate} * N)$ with expected operations before alert $1 / \text{rate}$ — so repeated replays monotonically raise the chance of alerting the adversary. Mission success is modeled as a reliability block diagram: a *series of parallel* blocks, where each block (intel acquisition, insertion, extraction) succeeds if any of its redundant paths succeeds [Barlow and Proschan, 1975]. The campaign expected value $P(\text{success}) * \text{reward} - P(\text{detected}) * \text{penalty}$ then prices the whole replay campaign.

30.4.6 Asset allocation (marginal gain)

`asset_allocator.py` spends scarce remastered assets. The allocatable objectives are *derived*, not re-declared: each takes its `objective_id` and `weight` from the canonical scenario and its `base_score` from that objective's real score in the remount replay. Only two numbers per objective are stipulated model inputs — the improvement `cap` an asset can ultimately buy and the efficiency `e` at which returns diminish — and no operational data calibrates them. Because the base scores are replay scores and the scenario's weights sum to one, the plan's weighted total is the replay's command score, and the gains below are in command-score units rather than in a unit of their own.

Each objective has a concave improvement curve $\text{cap} * (1 - \exp(-e * k))$ in the number k of assigned assets — strictly diminishing returns. A deterministic greedy allocates each of the `budget` assets to the objective with the largest **weight-scaled** capped marginal gain — the marginal increase in the weighted score the plan actually reports — with ties broken by objective order. Selecting on the unweighted gain instead maximizes a different function than the one reported; that was a real defect in this module, and the test suite now carries the wrong variant as a negative control.

The guarantee: the objective is a weighted sum of per-objective concave curves in that objective’s own asset count, which is the separable-concave special case of monotone submodular maximization. There greedy incremental allocation is not merely within the $(1 - 1/e)$ factor that Nemhauser–Wolsey–Fisher [Nemhauser et al., 1978b] guarantee in general — it is *exactly optimal*. `tests/test_asset_allocator.py` verifies that on the canonical pool by enumerating every way to split the budget across the objectives and comparing against the maximum, and shows the unweighted variant failing the same check.

30.4.7 Replay learning curve

`learning_curve.py` forecasts how rehearsal improves the remount. The command score follows an exponential learning curve $final - (final - initial) * \exp(-rate * n)$ from the baseline replay (`initial`) toward a fully-reinforced replay asymptote (`final`) [Wright, 1936]. Both endpoints are *computed*: each is a real replay of the scenario through the same engine, under the identity config and under the capped-bonus config respectively.

Inverting the curve gives the first iteration `n` reaching a target score, `n = ceil(-ln((final - target) / (final - initial)) / rate)`. The solver returns `None` — target unreachable — whenever the asymptote is at *or below* the target, since the curve approaches its asymptote without ever attaining it, and whenever the rate is non-positive.

30.4.8 Legacy asset migration

Each legacy dossier is a `LegacyAsset` carrying a declared SHA-256 [National Institute of Standards and Technology, 2015] integrity fingerprint. `verify` recomputes the fingerprint over NFC-normalized [The Unicode Consortium, 2023] UTF-8 content and compares it to the declared one (`None` = un-scanned, treated as self-consistent). Normalizing before hashing is what makes the fingerprint a property of the *text* rather than of one particular encoding of it. `remaster` migrates a source into `version + 1` with fresh content, optionally reclassifying/re-typing it, and recomputes the fingerprint; by default it refuses to migrate a source that fails its own integrity check, raising `AssetIntegrityError` rather than propagating a suspect dossier. An `AssetCatalog` aggregates assets, verifies them wholesale, and migrates a subset in a single deterministic batch. The canonical index the remount actually consumes is `LEGACY_INDEX`, exposed through `canonical_catalog` so the mission, the manuscript, and the figures share one definition of the migration.

30.4.9 Sensitivity analysis

`sensitivity.py` asks how much of the above to believe. The two headline conclusions — that the remount improves over the original ((a)), and that a short campaign still pays ((b)) — are produced at a single canonical parameter set. This module sweeps each of the three parameters the conclusions rest on over a declared range, holding the other two at their canonical values, and reports the largest contiguous sub-range containing the canonical value over which *both* conclusions hold. (b) is evaluated through the same `:func:optimal_campaign_length` the risk model exports, so the analysis and the headline crossing are one object, not a parallel derivation [Saltelli et al., 2008].

- [0.5, 20] — the success-curve sensitivity `S`;
- [0.05, 1] — the per-replay detection rate;
- [0.05, 1] — the per-replay learning rate.

Because the learning rate feeds neither the op diff ((a)) nor the campaign expected value ((b)), its robust sub-range spans the whole declared sweep by construction; that insensitivity is a reported result, not a gap. The analysis is one-dimensional — it perturbs one parameter at a time rather than sampling the joint space — which is appropriate for binding two specific conclusions but is not a general global-sensitivity decomposition.

30.5 Results — REPLAY: measured outcomes, headline numbers, and what they establish

30.5.1 Command score

Replaying the canonical THUNDERBALL-65 scenario under the identity config (original) and under `REMOUNT_CONFIG` (remount) gives:

Metric	Original	Remount	Delta
Command score	0.3111	0.5161	+0.2050
Objectives fulfilled	1	3	+2
Overall verdict	—	—	<i>improved</i>

The remount raises the weighted command score from 0.3111 to 0.5161, a weighted delta of +0.2050. Per objective, 4 improve, 0 regress, and 0 are unchanged.

Figure — original versus remount objective scores.

The grouped bar figure (rendered by `src/never_say_never_again/figures.py` into `../figures/op_diff_scores.png`) shows each objective’s score under the original and remount configs. The remount’s reinforced intel pushes every objective’s score upward.

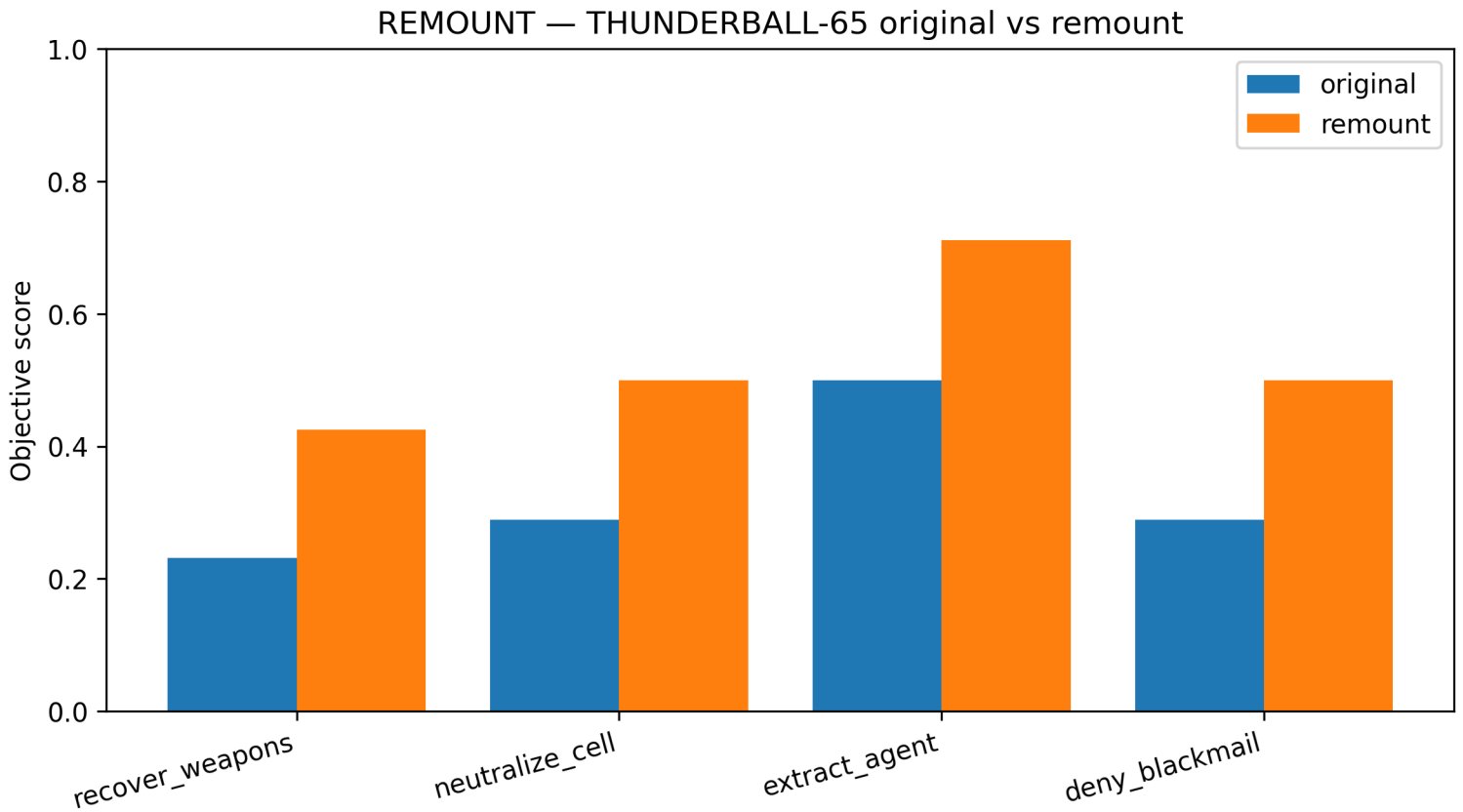


Figure 117: Op diff — original versus remount objective scores

30.5.2 Legacy asset migration

The engine migrates 3 legacy intelligence dossiers (1 gadget, 2 intel), each re-fingerprinted on migration. The catalog-wide stable fingerprint — the SHA-256 over the sorted (`id`, `kind`, `content-digest`) rows, truncated for display — is `1a1e05c4f464`. Migration is refused outright when a source dossier’s declared fingerprint does not match its content, so a tampered legacy record cannot silently enter the remount.

30.5.3 Remount risk

Over a campaign of 6 replay operations at a per-replay hazard of 0.25, the accumulated detection probability reaches 0.7769 (expected 4.00 operations before the first alert), whereas the block-structured mission reliability is 0.9578. At the canonical reward 100 and penalty 100 the campaign expected value is +18.10.

The sign of that expected value is the whole argument. The model’s own crossing — the smallest replay count at which the expected value turns non-positive — is 13 for the canonical parameters; the canonical campaign length of 6 sits comfortably below it, which is what “keep the remount short” means in numbers. Reliability is a *constant* of the mission’s block structure, while the detection term $1 - \exp(-\text{rate} * N)$ rises monotonically in the replay count N toward 1. The expected value is therefore strictly decreasing in N , and it crosses zero once $\text{reliability} * \text{reward}$ falls below $\text{detection} * \text{penalty}$ — with the canonical reward and penalty equal, simply once the accumulated hazard exceeds the reliability. Past that crossing every additional replay is negative-value. Both properties (strict decrease, and a crossing at the canonical parameters) are asserted over the real model in `tests/test_remount_risk.py` rather than argued in prose alone. This is the engine’s central caveat to the remount premise: *never say never again* — but do not keep saying it into a long campaign.

30.5.4 Asset allocation

Allocating the scarce remastered assets by marginal gain spends a budget of 6 across 4 objectives. The greedy deploys 6 of them — it stops early if every objective saturates — and recovers +0.2570 of weighted command score, i.e. 0.8030 of the full-saturation ceiling.

That figure is in genuine command-score units: the allocatable objectives carry the scenario’s own ids and weights, and each one’s zero-asset base score *is* its score in the remount replay reported above, so the plan’s zero-asset baseline is the remount command score 0.5161 — displayed here rounded, asserted at full precision in `tests/test_asset_allocator.py` rather than left to the reader. What the allocation model stipulates, and what no data here calibrates, is the pair of curve parameters per objective — how much an asset can ultimately buy and how fast the returns diminish.

Because each objective’s improvement curve is concave, the sequence of realized marginal gains is non-increasing: the greedy spends

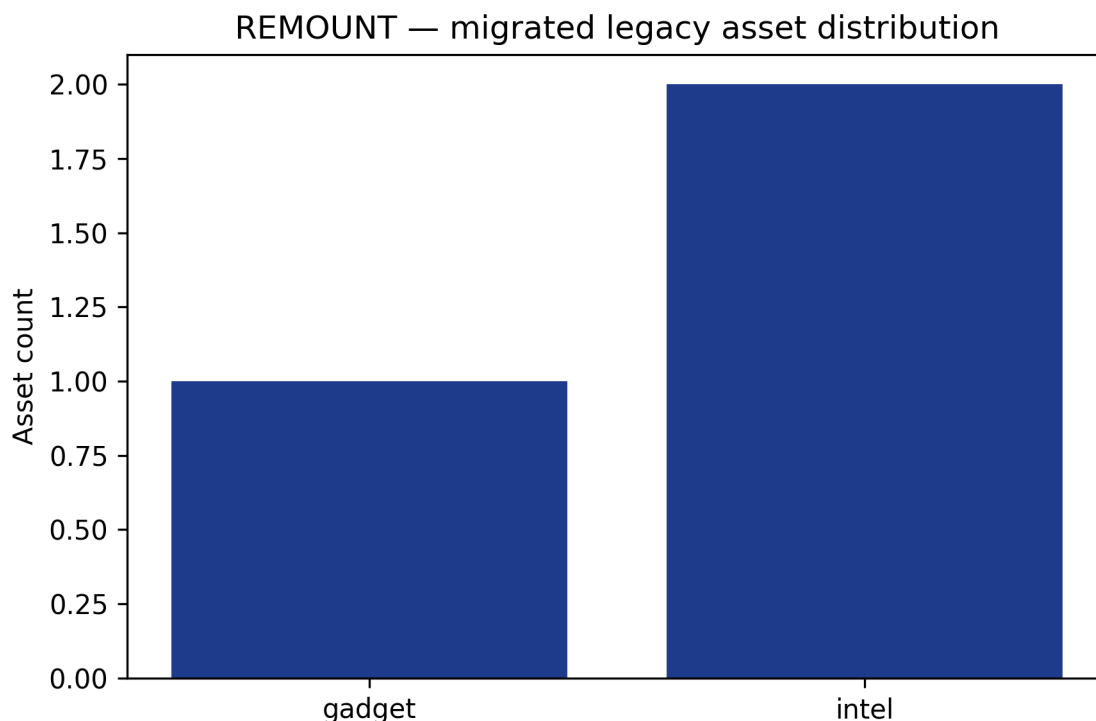


Figure 118: Migrated asset distribution

its first asset where the slope is steepest and each later asset buys strictly less. For this separable-concave objective that makes the greedy exactly optimal, not merely within the $(1 - 1/e)$ factor of the general submodular bound — checked in `tests/test_asset_allocator.py` against the maximum over every feasible split of the budget, with the unweighted-selection variant failing the same check.

30.5.5 Replay learning curve

Forecasting repeated rehearsal at learning rate 0.4 over 8 iterations raises the command score from 0.3111 toward 0.7010 (delta +0.3899), and the target score of 0.55 is first met at iteration 3.

The asymptote is not an assumption: it is a second real replay of the same scenario under a fully-reinforced config (intel bonus 0.3), so both ends of the curve are computed by the engine rather than posited. A target at or above that asymptote is reported as unreachable rather than as met on the first iteration — the curve approaches its asymptote but never attains it.

30.5.6 Sensitivity and robustness

The two headline conclusions above are each asserted at one sweep of a parameter, not at a single point. `src/never_say_never_again/sensitivity.py` sweeps each of the three parameters the conclusions rest on and reports the largest contiguous sub-range containing the canonical value over which *both* conclusions — remount improved [(a)] and campaign still pays [(b)] — hold:

Parameter	Sweep	Robust sub-range	Notes
Success sensitivity S	[0.5, 20]	[0.500, 20.000]	conclusion (a) holds across the whole declared sweep (true = spans)
Detection rate	[0.05, 1]	[0.050, 0.527]	conclusion (b) breaks at high rate (false = spans)
Learning rate	[0.05, 1]	[0.050, 1.000]	affects neither (a) nor (b) (true = spans)

Interpretation. The remount-improvement conclusion is *not* a knife-edge of the logistic steepness: it holds across the entire declared sensitivity sweep, because the remount config raises effective intel on every objective regardless of S . The campaign-still-pays conclusion is genuinely conditional: beyond a detection rate of about 0.527 the accumulated hazard crosses the mission reliability within the canonical campaign length, and the expected value turns negative — so “keep the remount short” depends on the adversary’s per-replay acuity. The learning rate, by construction, fights neither battle, and the analysis says so rather than pretending it does.

30.6 Conclusion — REPLAY: findings, verdict, and what the mission establishes

REMOUNT (REMOUNT) demonstrates a clean, deterministic answer to the legacy-mission re-open: re-run the scenario, diff the outcomes, and migrate the intel. For the canonical THUNDERBALL-65 remount the evidence is unequivocal — the remount improves

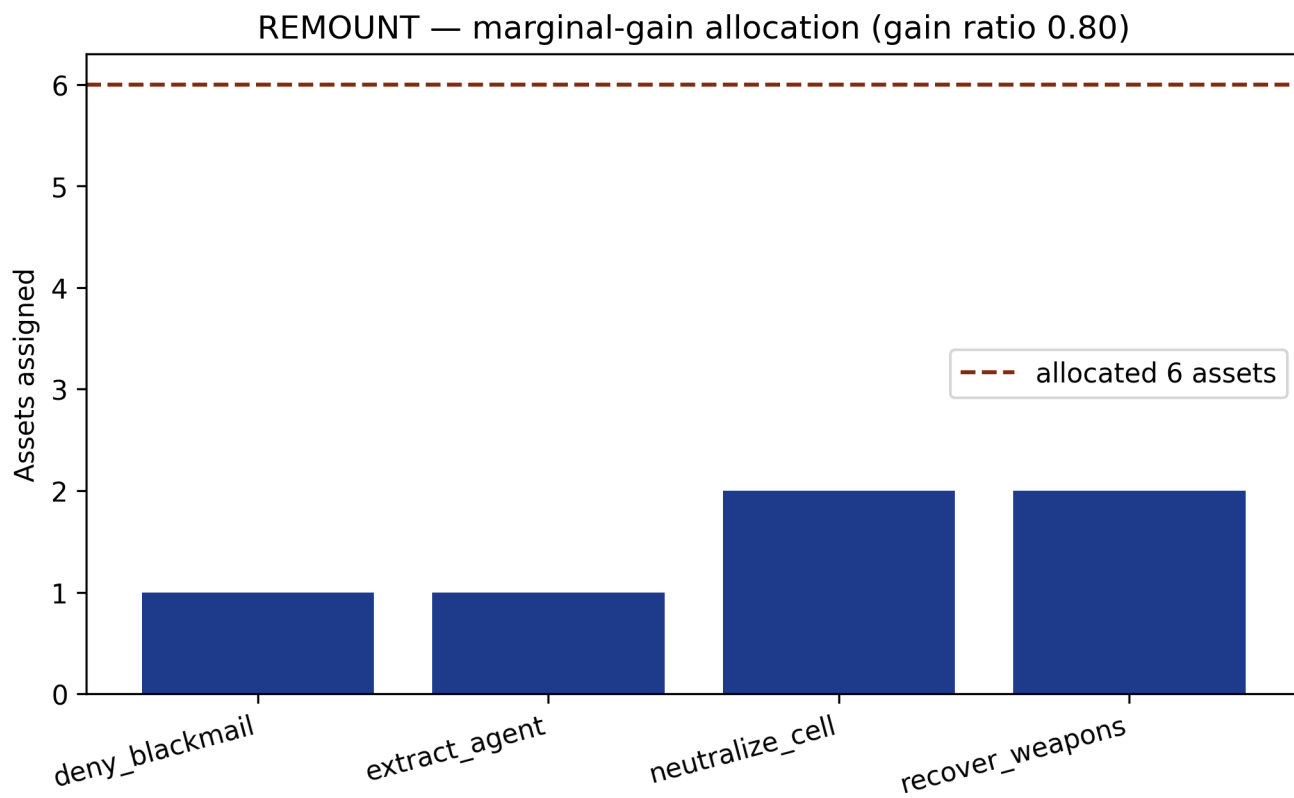


Figure 119: Marginal-gain asset allocation

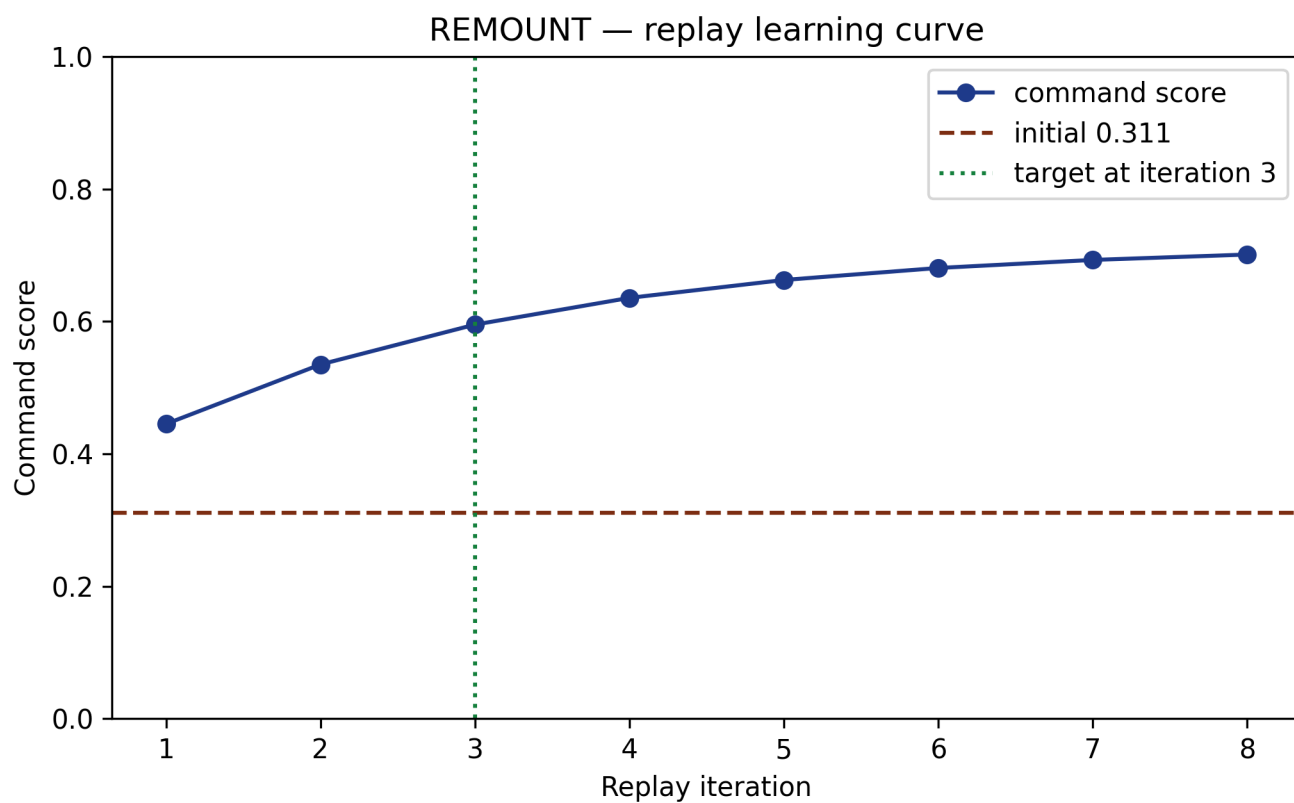


Figure 120: Replay learning curve

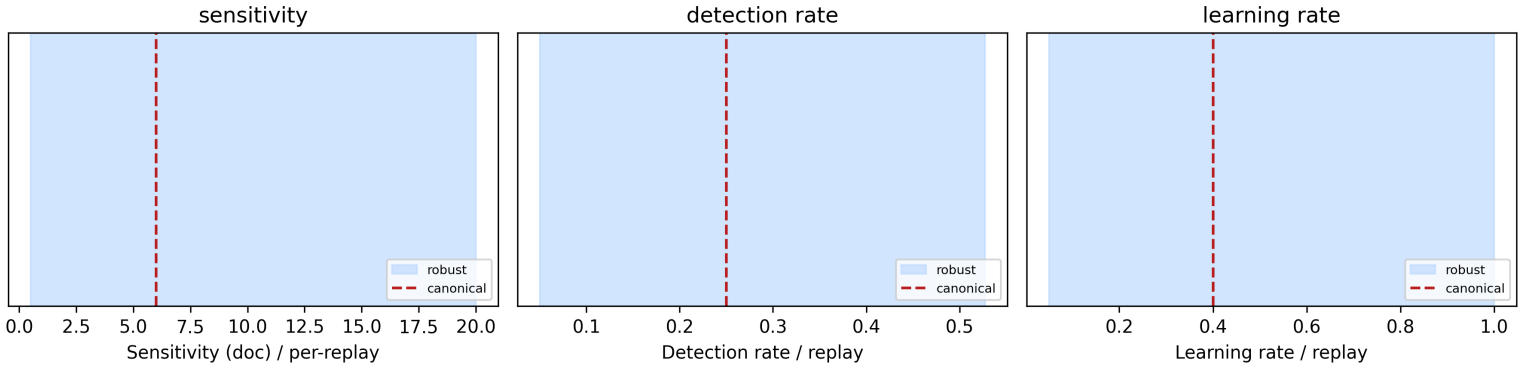


Figure 121: Sensitivity sweep — robust parameter sub-ranges

30.7 Experimental Setup — REPLAY: canonical scenarios, parameters, and configuration

30.7.1 Canonical dataset

The THUNDERBALL-65 scenario (`src/never_say_never_again/mission_replay.py`) is the sole canonical dataset, declared as module-level deterministic constants. It contains 4 objectives with fixed weights, difficulties, and baseline intel, plus an identity config (`intel_bonus = 0`) and the REMOUNT_CONFIG used for the remount. The migration consumes a second canonical constant, LEGACY_INDEX: 3 legacy dossiers (1 gadget, 2 intel).

30.7.2 Parameters

Every value below is a generated token read straight from the constant the code runs on — this list cannot drift from the implementation.

- Success-model sensitivity $S = 6$, success threshold 0.5.
- Remount intel bonus 0.15, doctrine 1, realism ceiling 1.
- Deterministic seed 13 (provenance-only: the mission makes no random draws).
- SHA-256 fingerprints over NFC-normalized UTF-8 content.
- Risk: detection rate 0.25/replay, campaign length 6, reward 100, penalty 100, reliability blocks with 2, 1, 3 redundant paths respectively (intel acquisition, insertion, extraction).
- Allocation: budget 6 assets over 4 objectives (concave curves).
- Learning: rate 0.4, 8 iterations, bonus cap 0.3, target 0.55.
- Sensitivity sweep ranges: [0.5, 20] (success sensitivity), [0.05, 1] (detection rate), and [0.05, 1] (learning rate), each sampled at 401 evenly spaced points.

All canonical numeric constants live in the domain modules (`mission_replay.py`, `legacy_assets.py`, `remount_risk.py`, `asset_allocator.py`, `learning_curve.py`, `sensitivity.py`), shared by the mission adapter, the manuscript tokens, and the figures — no duplicated numbers. `manuscript/config.yaml` mirrors the same values for human readers, and `tests/test_manuscript_variables.py` asserts the mirror matches the code, so a constant changed in one place and not the other fails the suite.

30.7.3 Fixtures and isolation

Tests use real data and computation only (no mock framework), with fixed inputs so every outcome is deterministic. Figures render with the Agg backend and are byte-identical across runs. Manuscript variables are injected — they are never hand-authored in prose.

30.8 Reproducibility — REPLAY: verification gates, deterministic regeneration, and artifacts

30.8.1 Determinism by construction

REMOUNT commits to byte-level reproducibility. The domain core performs no random draws and no wall-clock reads, so replaying THUNDERBALL-65 twice yields identical `ReplayResult` objects; rendering a figure twice yields identical PNG bytes.

The claim covers the whole persisted artifact set — every file under `output/`: `data/manuscript_variables.json`, the resolved manuscript tree under `manuscript/`, and the five PNGs under `figures/`. No token reads the clock. The date on this document, MANUSCRIPT_DATE, is `paper.date` from the committed `manuscript/config.yaml`, so it changes only when a human changes it in version control — never merely because the generator ran again. That is enforced, not asserted: `tests/test_regeneration_determinism.py` runs the generator twice, more than a clock-second apart, into two separate output roots and compares the SHA-256 of every file.

30.8.2 Provenance

Every mission outcome carries a **Provenance** block with:

- `package_version` — 0.1.0.
- `seed` — the deterministic seed (13).
- `input_hash` — a canonical SHA-256 fingerprint of the mission brief.
- `wall_time_s` — reported as 0.0. This is a deliberate choice, not a measurement: the provider does not read the clock, because a real elapsed time would make otherwise-identical outcomes differ byte-for-byte between runs. Read it as “wall time not recorded”, never as “the mission took no time”.

The migrated-asset catalog exposes `stable_integrity`, a catalog-wide fingerprint over its sorted (`id`, `kind`, `content-digest`) rows. The manuscript token `1a1e05c4f464` is that digest truncated for display, so an artifact set can be audited without re-running the suite. The same catalog factory (`canonical_catalog`) feeds the mission provider, the manuscript tokens, and the asset-distribution figure — there is no second derivation that could disagree about what was migrated.

30.8.3 Verification commands

```
uv run pytest tests/ --cov=src --cov-fail-under=90
uv run ruff check src/ scripts/ tests/ && uv run ruff format src/ scripts/ tests/
uv run mypy src/ scripts/
rg -n "template[_]code_project". --glob '!uv.lock' --glob '!.git/**'
```

This manuscript is produced on Python 3.14.6; its document date is 2026-08-05.

30.9 Scope and Related Work — REPLAY: boundaries, positioning, and relationship to the literature

30.9.1 Scope

This package is strictly a BOND suite film package. Its scope is exactly its own directory; it implements the frozen `bond_api` mission protocol as a thin adapter (`mission.py`) on top of a pure, infrastructure-free domain core of seven modules (`mission_replay.py`, `op_diff.py`, `legacy_assets.py`, `remount_risk.py`, `asset_allocator.py`, `learning_curve.py`, `sensitivity.py`). It does not attempt to be a general simulation framework or a full intelligence management system.

30.9.2 Related work

Within the BOND suite this film sits at the film layer: it consumes Layer 1 (`bond-api`, the frozen `MissionProvider` protocol and `GadgetRegistry`) and imports no sibling film package. Layer 0 (`bond-utilities`) is not yet bound here; the integration points where it would replace in-package code are named explicitly in `TODO.md` rather than assumed.

Outside the suite, the models are standard and cited rather than invented. The allocation greedy is the separable-concave special case of Nemhauser, Wolsey and Fisher’s submodular maximization setting [Nemhauser et al., 1978b], where the greedy is exactly optimal rather than $(1 - 1/e)$ -approximate; the rehearsal model is the Wright learning curve [Wright, 1936]; the mission-success structure is a reliability block diagram in the sense of Barlow and Proschan [Barlow and Proschan, 1975], with detection as a homogeneous Poisson process [Ross, 2019]; the per-objective success curve is the Verhulst logistic [Verhulst, 1838]; and integrity uses SHA-256 [National Institute of Standards and Technology, 2015] over NFC-normalized text [The Unicode Consortium, 2023]. This package contributes their deterministic composition into an auditable remount pipeline, not the models themselves.

30.9.3 Limitations

These are real bounds on what the results support, not caveats-as-decoration.

- The success model is a single Logistic curve with a hand-chosen sensitivity; it is illustrative, not a forecast calibrated against operational data. No claim in this manuscript is an empirical claim about real operations.
- `diff_outcomes` aligns objectives by id and **skips** objectives present on only one side: they appear in no diff and in none of the counts. Comparing two scenarios with different objective sets therefore silently compares only the intersection.
- Objective weights are recovered by scenario *name*. A caller-built scenario that reuses the canonical name with different weights would be diffed against the canonical weights.
- The migration is fingerprint-only. It detects tampering against a declared digest but models no cryptographic provenance — no signatures, no chain of custody, and no protection against an attacker who rewrites content and digest together.
- An un-scanned asset (`integrity` is `None`) is treated as self-consistent by design, so absence of a fingerprint is not evidence of integrity.
- The risk, allocation, and learning models are evaluated at one canonical parameter set, and the sensitivity analysis in `03_results.md` sweeps **one parameter at a time** (OAT) rather than the joint space. It bounds how far the headline conclusions hold under single-parameter variation; it is not a global variance decomposition, so it cannot rank second-order parameter interactions.

30.10 Sources — REPLAY: bibliography

Friedman [2026b]; Friedman [2026f]; Nemhauser et al. [1978b]; Wright [1936]; Barlow and Proschan [1975]; Ross [2019]; Saltelli et al. [2008]; Verhulst [1838]; National Institute of Standards and Technology [2015]; The Unicode Consortium [2023]

31 Bond Utilities — Q-BRANCH

infra package · package codename Q-BRANCH.

31.1 Concepts — Q-BRANCH: domain and operational focus

Q-Branch primitives — ciphers, codenames, clock, provenance, fixtures

31.2 Abstract — Q-BRANCH: mission summary

The BOND suite’s shared foundation, codenamed **Q-BRANCH**, ships dependency-light Python modules that every other bond package imports: a classic-cipher suite with cryptanalysis helpers, a deterministic codename engine with several schemes, mission clock/schedule math, seeded randomness, mission-record persistence with provenance, and plain-text reporting. The cipher layer exposes **9** classic primitives (caesar, vigenere, one_time_pad, xor, atbash, beaufort, rail_fence, playfair, columnar) — each round-trip verified with real data — and **8** analysis helpers (letter frequency, index of coincidence, chi-squared scoring, Caesar/Vigenère breaking, Kasiski examination). Codenames are drawn under **5** deterministic schemes (adjective_noun, alliterative, nato, numeric, token); the NATO scheme spells a short identifier as **QUEBEC-BRAVO-7**. Reproducible sampling runs through **7** seeded draw helpers (seeded_bytes, seeded_int, seeded_float, seeded_choice, seeded_choices, seeded_shuffle, seeded_sample), each derived from one seed. Mission schedules are half-open UTC windows with overlap, merge, and duration models, and persisted mission records carry a reproducible input hash, the schema version, and the producing seed, so any outcome can be re-derived from its stored inputs. Every quantity this manuscript claims about the package — counts, capacities, window durations, sample outputs, and gate thresholds — arrives as a generated token computed from `manuscript/config.yaml` and the live registries by `src/bond_utilities/manuscript_variables.py`. Three tests enforce that: a cross-reference test fails on any prose token the generator does not produce; a bare-numeral guard fails on any hand-authored number in the abstract, introduction, results, conclusion, and reproducibility sections; and a hand-copied-metric guard fails when any section anywhere in the manuscript writes a generated value as a literal instead of its token. The exception is stated rather than hidden: the methodology and scope sections write the constants that *define* an algorithm — the modular base, the digraph square size, the reference index-of-coincidence values — as literals, because those are definitions of the mathematics, not measurements of this package.

31.3 Introduction — Q-BRANCH: mission framing, the operational problem, and how to read this chapter

`bond-utilities` is Layer 0 of the BOND suite: the shared, dependency-free foundation imported by every other bond package. It deliberately keeps its dependency surface to the standard library plus `numpy`, `matplotlib`, and `pyyaml`, and it never imports a mock framework.

Three principles govern the design:

- **Determinism.** Every random draw flows through a seeded generator (`src/bond_utilities/randomness.py`). The same seed produces the same codenames, cipher keys, and test streams on every run and machine.
- **Round-trip guarantees.** Each cipher is accompanied by a test proving `decrypt(encrypt(x)) == x` on real streams, and codename obfuscation satisfies `restore(obfuscate(x)) == x`.
- **Provenance-first persistence.** Mission records written by `mission_io.py` bundle a canonical input hash, the schema version, and the producing seed, so every outcome can be reproduced from its persisted inputs — the contract bond-api outcomes rely on.

31.3.1 Module map

Module	Responsibility
<code>ciphers.py</code> <code>codenames.py</code>	9 classical ciphers + 8 cryptanalysis helpers; codename encoding 5 deterministic codename schemes (adjective_noun, alliterative, nato, numeric, token)
<code>clock.py</code>	Elapsed time, schedule windows, overlap/merge, duration parsing, T-minus, <code>MissionSchedule</code>
<code>randomness.py</code>	Seeded RNG factory, <code>seeded</code> decorator, and scalar/sequence draw helpers
<code>mission_io.py</code>	JSON/YAML records, schema validation, provenance verify/append/summary
<code>reporting.py</code>	Plain-text table, key/value, banner, and schedule-table output
<code>testing.py</code>	Shared pytest fixtures plus plain test helpers (import-light, no mocks)
<code>figures/</code>	Deterministic schedule and letter-frequency figures

31.3.2 Reader’s guide

02_methodology.md defines the cipher math and cryptanalysis, the codename schemes, the seeding, and the clock/window semantics. 03_results.md reports the measured round-trips, determinism, cryptanalysis demos, and schedule results. 05_experimental_setup.md documents the mission parameters that drive every token in this manuscript.

31.4 Methodology — Q-BRANCH: the analytical models and algorithms that drive the mission

31.4.1 Ciphers (src/bond_utilities/ciphers.py)

The cipher suite falls into four families. **Monoalphabetic:** Caesar ($c = (p + \text{shift}) \bmod 26$), Atbash (reverse alphabet, self-inverse), and Beaufort ($c = (k - p) \bmod 26$, self-inverse). **Polyalphabetic/keyed:** Vigenère (keyword advances over letters only), Playfair (digraph substitution on a 5x5 keyed square with I/J merged, doubles split with X). **Transposition:** rail-fence (zig-zag rows) and columnar (columns read in a digit-key order, round-trips exactly with no padding). **Byte:** one-time-pad (XOR with a key at least as long as the data) and XOR (cyclically repeated key). Every cipher satisfies `decrypt(encrypt(x)) == x` on supported inputs; the digraph cipher round-trips against its prepared (X-split, padded) form.

31.4.1.1 Cipher catalogue The compiled table below is generated from the live CIPHER_DESCRIPTIONS registry (token | caesar | shift cipher, letters only | | vigenere | polyalphabetic keyword cipher | | one_time_pad | XOR with a key at least as long as the data | | xor | symmetric byte-wise XOR with a cyclic key | | atbash | reverse-alphabet substitution (self-inverse) | | beaufort | reciprocal keyword cipher $c = (k - p)$ | | rail_fence | zig-zag row transposition | | playfair | digraph substitution on a 5x5 keyed square | | columnar | column-order transposition with a digit key |), so it cannot drift from the modules it documents.

cipher	description
caesar	shift cipher, letters only
vigenere	polyalphabetic keyword cipher
one_time_pad	XOR with a key at least as long as the data
xor	symmetric byte-wise XOR with a cyclic key
atbash	reverse-alphabet substitution (self-inverse)
beaufort	reciprocal keyword cipher $c = (k - p)$
rail_fence	zig-zag row transposition
playfair	digraph substitution on a 5x5 keyed square
columnar	column-order transposition with a digit key

All cipher “letters” are ASCII A-Z/a-z by definition ([uni, 2025]). A character that `str.isalpha` treats as a letter but that lies outside the 26-letter English alphabet – e.g. é, ñ, or a Cyrillic letter – is a *non-letter* to these primitives: the ciphers pass it through unchanged, and the frequency/counting helpers ignore it, so mixed-alphabet mission text stays safe to pipe through any cipher without a crash or silent corruption.

31.4.2 Cryptanalysis helpers

- `letter_frequency / frequency_normalized` — A-Z counts and relative frequencies.
- `index_of_coincidence` — Friedman’s IC: about 0.066 for natural English vs about 0.038 for uniform random, used to distinguish substitution from polyalphabetic text.
- `english_chi_squared` — chi-squared fit of letter frequencies against standard English.
- `caesar_shift_scores / caesar_break` — score all 26 shifts by chi-squared and return the best (`shift, plaintext`). The score is a statistic over the input’s letter counts, so the returned shift is the best-fitting one, not a certified one: on short samples the best fit is often the wrong shift, and the function returns it with no confidence signal (see Limitations in 07_scope_and_related_work.md).
- `kasiski_key_lengths` — locate repeated n-grams, factor their distances, and vote on candidate Vigenère key lengths.
- `vigenere_break` — break each key column independently and re-interleave.

31.4.3 Deterministic seeding (randomness.py)

`derive_seed` folds integer seeds to 32 bits and hashes string seeds (`int(sha256(seed)[0:8], 16)`). `make_rng(seed)` returns a fresh `random.Random`; the `seeded(seed)` decorator seeds the global module and restores the prior state on exit, so a decorated function is deterministic in isolation without perturbing its caller. The module also exports 7 typed draw helpers (`seeded_bytes`, `seeded_int`, `seeded_float`, `seeded_choice`, `seeded_choices`, `seeded_shuffle`, `seeded_sample`), each reproducible from a single seed and enumerated in the SEEDED_HELPERS registry that the manuscript reads.

31.4.4 Codename schemes (`codenames.py`)

- `adjective_noun` (default) — unique pairs sampled without replacement from a 12 x 12 grid.
- `alliterative` — ADJ-NOUN pairs sharing a starting letter. Unlike `adjective_noun`, this scheme seeds each index independently instead of sampling without replacement, so a batch may repeat a name; and only 5 initials are carried by both word lists, giving 9 distinct pairs in total. Repetition in a batch is therefore expected, not a defect — `tests/test_codenames.py` asserts both the capacity and the repetition.
- `nato` — NATO phonetic spelling of a base codename (e.g. QUEBEC-BRAVO-7).
- `numeric` — PREFIX-XXXXXXX hex codes derived from a mission id.
- `token` — truncated SHA-256 hex tokens.

Each scheme is deterministic under a fixed seed; `generate_codenames(..., scheme=...)` selects the scheme while preserving the phase-1 default exactly.

31.4.5 Mission clock (`clock.py`)

Times are timezone-aware (naive treated as UTC). A window is the half-open interval `[start, end)`. Phase 4 adds `windows_overlap` (shared duration, clamped at zero), `merge_windows` (minimal sorted cover), `parse_duration` ("1d 2h30m"-style), `schedule_status` (PENDING/ACTIVE/COMPLETE), and the `MissionSchedule` dataclass tying duration, open/elapsed/remaining, countdown, and status together.

31.4.6 Provenance (`mission_io.py`)

`record_provenance_outcome` persists a record whose `provenance` block holds the canonical input hash (SHA-256 of sorted-compact JSON), the schema version, and the derived seed. Phase 4 adds `verify_provenance` (recompute and compare the hash/seed/version), `load_records` (sorted directory batch load), `record_summary` (flat deterministic digest), and `append_outcome` (merge into `outcomes` without disturbing provenance).

31.4.7 Reporting and figures (`reporting.py`, `figures/plots.py`)

`reporting.py` renders the plain-text surface the thin CLIs print: `banner`, `format_table` (left-aligned, width-fitted, ragged rows rejected), `format_kv` (right-aligned keys), and `schedule_table`, which projects a sequence of `MissionSchedule` objects into name/start/end/status rows evaluated at a caller-supplied instant. The `print_*` wrappers take an explicit `stream`; the default resolves `sys.stdout` at call time rather than at import time, so redirected output is captured correctly and tests can assert on real rendered text without a mock framework.

`figures/plots.py` renders the two manuscript figures. `plot_mission_schedule` draws the window as a horizontal bar in unix-time coordinates with a dashed “now” marker; `plot_frequency_analysis` draws the A-Z counts returned by `ciphers.letter_frequency`. Both are deterministic under the Agg backend — the same inputs produce byte-identical PNGs, which `tests/test_figures.py` asserts for each figure by comparing two renders. For the schedule figure the input includes `now`: the marker moves with the wall clock when `now` is left to its library default, so determinism is claimed and tested for a fixed `now`, and a negative control asserts that two different `now` values do produce different bytes. The manuscript pipeline never uses that default — the generator script passes `manuscript_variables.manuscript_instant(config)`, a committed value, so the rendered PNG is part of the byte-identical regeneration claim.

31.4.8 Shared fixtures (`testing.py`)

`testing.py` carries the shared test surface every bond package reuses through its own `conftest.py`. Two of the four names are actual pytest fixtures — `seeded_rng` (a `random.Random` at the suite’s default seed) and `mission_record` (a schema-valid record); the other two are plain callables imported directly, `temp_dir` (a context manager) and `seeded_codenames` (a function). A test asserts that split, so the distinction cannot be lost. It imports only `pytest`, the standard library, and this package’s pure primitives, so importing it can never pull `numpy`, `matplotlib`, or — the point of the constraint — a mock framework into a consumer’s test session.

31.4.9 Scoring and statistics

Frequency scoring uses the standard English letter table in `ENGLISH_FREQUENCIES` (normalized; sum 1.0). All cryptanalysis outputs are deterministic given the input text.

31.5 Results — Q-BRANCH: measured outcomes, headline numbers, and what they establish

31.5.1 Round-trip fidelity

Every cipher in `src/bond_utilities/ciphers.py` passes a round-trip identity on real streams: Caesar and Atbash (letters, case preserved), Vigenère and Beaufort (non-letters pass through), rail-fence and columnar (exact transposition, no padding), Playfair (against its prepared form), XOR and one-time-pad (bytes). None of the tests use mocks — they exercise the actual algorithms.

31.5.2 Cryptanalysis demos

Both breakers are statistical, and their accuracy is a function of sample length rather than a guarantee. On the letter-rich English samples used in `tests/test_documented_examples.py`, `caesar_break` recovers the exact shift and plaintext, and `vigenere_break` recovers a keyword of known length and decrypts. On short samples both return a wrong answer with no confidence signal: a sixteen-letter English plaintext enciphered at a known shift comes back under a different shift, and the twenty-one-letter case fails likewise for both breakers. Those failures are asserted as tests, not merely noted, so the qualification cannot quietly stop being true. `kasiski_key_lengths` returns the true key length among its candidates for repeated plaintext, and `index_of_coincidence` cleanly separates repetitive (more-structured) text from uniform random text. The figure the `freq` figure shows the A-Z counts of the mission codename sample.

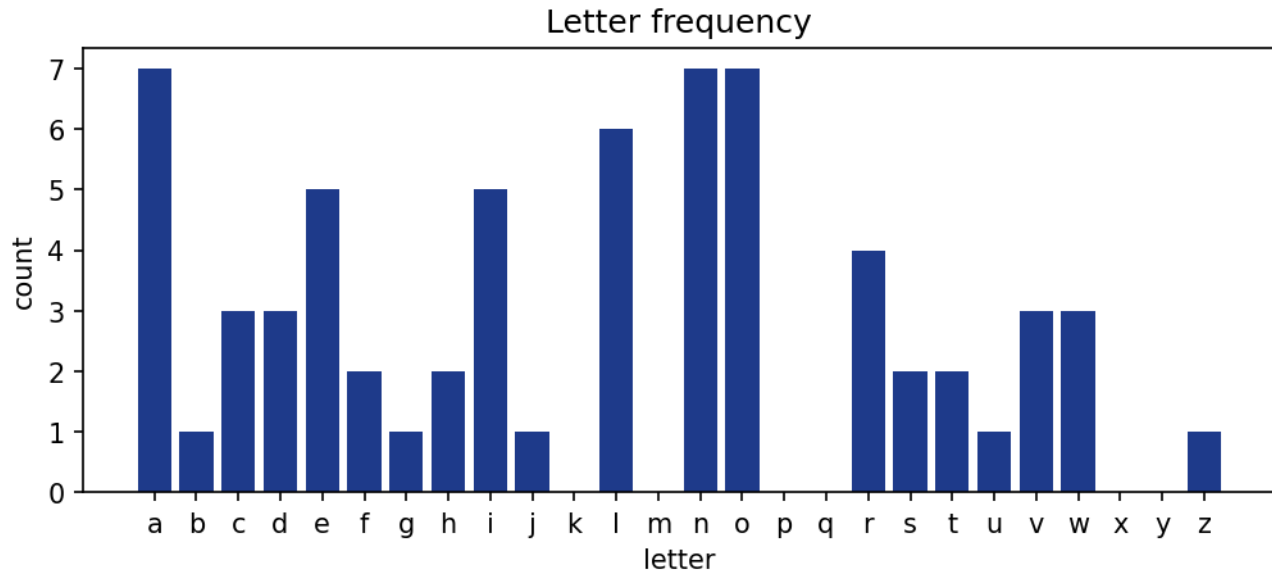


Figure 122: Letter frequency of the codename sample.

31.5.3 Edge-case robustness

The cipher alphabet is ASCII A-Z/a-z by definition ([uni, 2025]). Non-ASCII alphabetic characters – é, ñ, and letters from other scripts, which Python’s `str.isalpha` classifies as letters – are treated as *nothing to the cipher*: the enciphering functions pass them through byte-for-byte, and the frequency, index-of-coincidence, and chi-squared helpers ignore them entirely. That contract is asserted as a regression test (`tests/test_ciphers.py::TestNonAsciiRobustness`), which pins three properties that previously failed: the A-Z counting helpers accept mixed-alphabet text without crashing, a non-ASCII letter never corrupts a cipher round-trip, and `caesar_break/english_chi_squared` treat a non-ASCII-only input as letterless rather than misinterpreting it. The same hardening closes two validation gaps in the primitives’ edge behaviour: a non-string timestamp raises `ValueError` rather than leaking an internal `AttributeError` (`tests/test_clock.py::TestParseTimestampValidation`), and an empty header list to `format_table` means *no header* rather than tripping the ragged-row guard (`tests/test_reporting.py::TestFormatTableHeadersEdge`).

31.5.4 Determinism

All 9 ciphers (`caesar`, `vigenere`, `one_time_pad`, `xor`, `atbash`, `beaufort`, `rail_fence`, `playfair`, `columnar`) and the codename generator are deterministic under a fixed seed. The default adjective–noun sample of 6 names drawn with seed 20260804:

SHADOW-FALCON, IRON-JAGUAR, WILD-HORIZON, SILENT-RAVEN, VELVET-BEACON, WILD-FALCON

An alliterative sample under the same seed:

CRIMSON-COMET, CRIMSON-COMET, SILENT-SENTINEL, RAPID-RAVEN, BRAVE-BEACON, CRIMSON-COMET

That alliterative sample repeats: it holds 4 distinct names across 6 slots. That is the scheme working as specified, not a bug — **alliterative** seeds each index independently rather than sampling without replacement, and the bundled word lists share only 5 initials between adjectives and nouns, admitting 9 distinct pairs in total. Collisions are therefore likely in any batch of that size, and a test asserts that this sample contains one. Callers needing distinct names must use the default scheme.

The scheme registry holds 5 entries (`adjective_noun`, `alliterative`, `nato`, `numeric`, `token`); each is bound to a real dispatch branch by a registry-drift test. The default scheme draws without replacement from a 12 x 12 adjective/noun grid, so its capacity is 144 unique pairs — `generate_codenames` raises above that count rather than repeating a name.

31.5.5 Seeded draw helpers

Reproducible sampling outside the codename engine goes through the 7 helpers in `src/bond_utilities/randomness.py` (`seeded_bytes`, `seeded_int`, `seeded_float`, `seeded_choice`, `seeded_choices`, `seeded_shuffle`, `seeded_sample`). Each takes the seed as its first argument and constructs a fresh `random.Random`, so draws never depend on call order or on the global interpreter RNG. `ciphers.generate_otf_key` delegates to `seeded_bytes`, which is why pad material and any other byte draw are byte-identical for the same seed. The registry itself is test-bound: a discovery test in `tests/test_randomness.py` fails if a helper is added, renamed, or dropped without updating `SEEDED_HELPERS`.

31.5.6 Provenance round-trip

`record_provenance_outcome` writes a record whose `provenance` block holds the canonical SHA-256 of the sorted-compact JSON parameters, the schema version, and the derived seed. `verify_provenance` recomputes all three from the persisted record and returns `False` on any mismatch — the mutation tests in `tests/test_mission_io.py` tamper with the parameters, the seed, and the schema version in turn and confirm each is rejected, so the check is not satisfiable by construction. `append_outcome` merges new results into `outcomes` and leaves the provenance block byte-identical, which is what lets a later mission stage extend a record without invalidating its audit trail.

31.5.7 Mission schedule

The primary mission window spans 144.0 hours, from 2026-08-04T00:00:00+00:00 to 2026-08-10T00:00:00+00:00 (UTC), rendered in the schedule figure. The secondary window overlaps the primary by 48.0 hours — computed by `windows_overlap`, not hand-authored. Window math is exact under the half-open `[start, end)` semantics; elapsed and remaining durations clamp at zero.

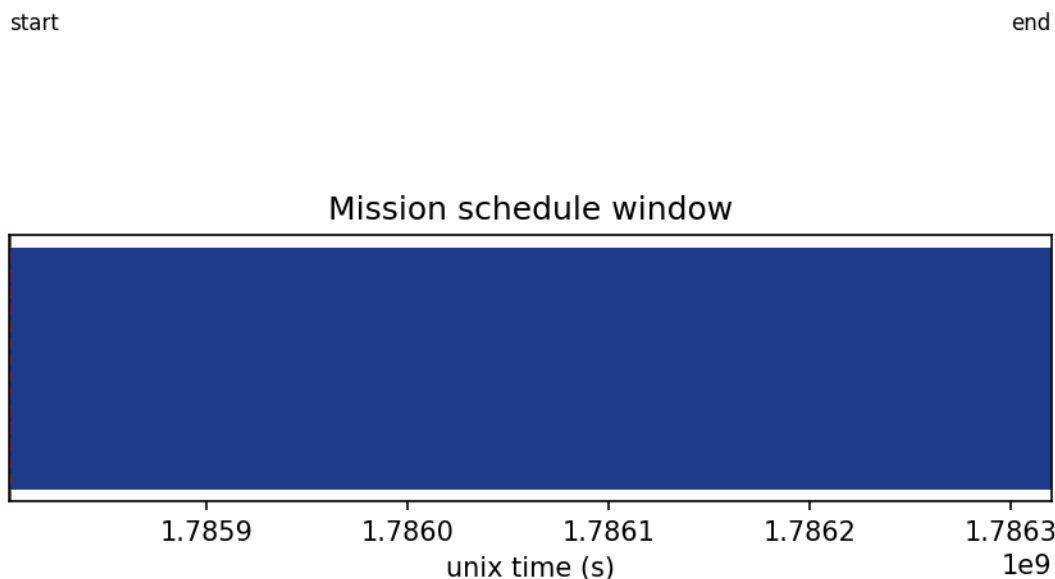


Figure 123: Mission schedule window with the reference “now” marker.

31.6 Conclusion — Q-BRANCH: findings, verdict, and what the mission establishes

`bond-utilities` (codename **Q-BRANCH**) delivers a broadened, fully-tested, dependency-light foundation on which the rest of the BOND suite is built. Every quantity this manuscript claims about the package is generated from `manuscript/config.yaml` and the live registries through `src/bond_utilities/manuscript_variables.py`, and three tests enforce it: token cross-reference, a bare-numeral guard over the quantitative sections, and a hand-copied-metric guard that fails if any section anywhere writes a generated value as a literal. The constants that define an algorithm — the modular base, the digraph square size, the reference index-of-coincidence values — are written literally in the methodology and scope sections, because they describe the mathematics rather than measure this package.

The guarantees that matter for downstream packages are:

- **Reproducibility:** one seed fully determines every codename, key, draw, and test stream.
- **Correctness:** all 9 ciphers are round-trip verified with real data, and the 8 helpers turn classic frequency analysis into deterministic, testable functions.
- **Provenance:** persisted mission records carry a reproducible input hash, schema version, and seed (`verify_provenance` recomputes them), so outcomes can always be audited and reproduced.

Future phases (`bond-api` and higher layers) will continue to consume this package’s `record_provenance_outcome/verify_provenance` persistence surface, its seeded draw helpers, and its schedule models.

31.7 Experimental Setup — Q-BRANCH: canonical scenarios, parameters, and configuration

31.7.1 Mission parameters

All values are read from `manuscript/config.yaml` by `src/bond_utilities/manuscript_variables.py::generate_variables`; scripts never hardcode them.

Parameter	Value
Package version	0.3.0
Mission codename	Q-BRANCH
Default seed	20260804
Codename sample size	6
Schedule start (UTC)	2026-08-04T00:00:00+00:00
Schedule end (UTC)	2026-08-10T00:00:00+00:00
Window duration	144.0 hours
Overlap with secondary window	48.0 hours
Cipher primitives	9
Cipher registry	caesar, vigenere, one_time_pad, xor, atbash, beaufort, rail_fence, playfair, columnar
Codename schemes	5 (adjective_noun, alliterative, nato, numeric, token)
Cryptanalysis helpers	8
Seeded draw helpers	7 (seeded_bytes, seeded_int, seeded_float, seeded_choice, seeded_choices, seeded_shuffle, seeded_sample)
Word-list grid	12 x 12
Word-list capacity (unique pairs)	144
Coverage gate	90% line + branch on <code>src/</code>
Tolerated test failures	0
Manuscript date (config <code>paper.date</code>)	2026-08-04

31.7.2 Software environment

The package requires Python `>= 3.10` and depends only on `numpy`, `matplotlib`, and `pyyaml`. Tests use `pytest` and `pytest-cov`; no mock framework is used anywhere. All randomness is seeded; functions with a wall-clock default take an explicit `now=/seed=` override in tests.

31.8 Reproducibility — Q-BRANCH: verification gates, deterministic regeneration, and artifacts

31.8.1 Artifact inventory

Artifact	Producer
<code>output/data/manuscript_variables.json</code>	<code>scripts/z_generate_manuscript_variables.py</code>
<code>../figures/mission_schedule.png</code>	same script → <code>src/bond_utilities/figures/plots.py::plot_mission_schedule</code>
<code>../figures/letter_frequency.png</code>	same script → <code>plots.plot_frequency_analysis</code>

All artifacts are regenerated, never hand-edited. Regeneration is a single command:

```
uv run python scripts/z_generate_manuscript_variables.py
```

Running that command twice on the same commit rewrites every artifact above **byte for byte**. Nothing in the pipeline reads the wall clock: the token map is a pure function of `manuscript/config.yaml` and the library registries, and the schedule figure’s reference marker is pinned to `paper.date` through `manuscript_variables.manuscript_instant` rather than to `datetime.now`. The claim is bound by `tests/test_regeneration_determinism.py`, which runs the command above twice — separated by a real clock tick, so a reinstated timestamp cannot slip through — and compares a digest of the whole `output/` tree. The scope is exactly that artifact tree; nothing here is a claim about the wider environment, which the experimental-setup section pins separately.

31.8.2 Determinism claims

- Codenames: `generate_codenames(count, seed=S, scheme=...)` returns the same list on every run and machine, for every scheme.
- Cipher keys and byte draws: `generate_otp_key(length, seed=S)` delegates to `seeded_bytes(S, length)`, so both return the same bytes for the same seed.

- Every other seeded helper (`seeded_bytes`, `seeded_int`, `seeded_float`, `seeded_choice`, `seeded_choices`, `seeded_shuffle`, `seeded_sample`) builds its own `random.Random` from the seed, so a draw never depends on call order.
- Cryptanalysis: frequency, IC, chi-squared, Kasiski, and Caesar/Vigenère breaking are deterministic functions of the input text.
- Mission records: `record_provenance_outcome` stores the canonical input hash; `verify_provenance` recomputes it, so any outcome can be audited and re-derived from its persisted inputs.

31.8.3 Quality gate

```
uv run pytest tests/ --cov=src --cov-fail-under=90
uv run ruff check src/ scripts/
uv run ruff format --check src/ scripts/
uv run mypy src/ scripts/
```

The gate enforces at least 90% line **and** branch coverage on `src/`, and tolerates at most 0 failing tests. Both numbers come from `manuscript/config.yaml`, and a test asserts the coverage figure equals the `fail_under` value actually enforced by `pyproject.toml` — the prose cannot drift away from the enforced gate. The measured coverage of any given run is deliberately not quoted here; run the command above to read it. `ruff check`, `ruff format --check`, and `mypy src/ scripts/` must also pass clean.

31.9 Scope and Related Work — Q-BRANCH: boundaries, positioning, and relationship to the literature

31.9.1 Scope

This package implements *utility* cryptography and scheduling primitives — obfuscation, reproducible sampling, frequency analysis, and window math — for the internal BOND suite. It is explicitly **not** a security product: the ciphers are educational and deterministic, and must never be used for real secrecy.

31.9.2 Related work

The cipher primitives follow the classical constructions catalogued by Kahn [Kahn, 1996] and presented as textbook algorithms by Stinson and Paterson [Stinson and Paterson, 2018], and are formalised in Shannon’s information-theoretic treatment of secrecy systems [Shannon, 1949b] — which is also the reference that makes the one-time pad’s key-length requirement non-negotiable rather than a stylistic choice. The cryptanalysis helpers build on Friedman’s index of coincidence [Friedman, 1922] and Kasiski’s examination for repeated key segments [Kasiski, 1863]. Deterministic seeded generation uses the Mersenne Twister [Matsumoto and Nishimura, 1998] as exposed by Python’s `random` module; that generator is chosen for reproducibility and equidistribution, explicitly not for cryptographic unpredictability. Mission window and elapsed-time semantics follow ISO-8601 date/time representations [iso, 2019].

31.9.3 Limitations

- The ciphers do not provide authentication, integrity, or non-repudiation.
- Determinism trades secrecy for reproducibility by design.
- `caesar_break` and `vigenere_break` are chi-squared fits to English letter frequencies, so both recover reliably only on letter-rich ciphertext. Neither reports confidence: on a short sample the best-fitting shift is often the wrong one and is returned as if it were correct. Measured cases — a sixteen-letter and a twenty-one-letter English plaintext, both enciphered at a known Caesar shift and both recovered under a different shift, and the twenty-one-letter Vigenère case recovering the wrong key — are asserted in `tests/test_documented_examples.py`. Treat a recovered key from a short text as a guess.
- The `alliterative` codename scheme can repeat a name within one batch: it seeds each index independently, and the bundled word lists admit only 9 distinct pairs across 5 shared initials. Only the default `adjective_noun` scheme guarantees distinctness (and raises rather than repeating once capacity is exhausted).
- The word lists are deliberately small and curated; they are not a secure randomness source.

31.10 Sources — Q-BRANCH: bibliography

Kahn [1996]; Shannon [1949b]; Matsumoto and Nishimura [1998]; Friedman [1922]; Kasiski [1863]; Stinson and Paterson [2018]; iso [2019]; uni [2025]

32 Bond API — THE PROTOCOL

infra package · package codename THE PROTOCOL.

32.1 Concepts — THE PROTOCOL: domain and operational focus

The frozen MissionProvider protocol, registry, discovery, serialization

32.2 Abstract — THE PROTOCOL: mission summary

BOND-API (codename *THE PROTOCOL*) is the frozen mission protocol contract for the BOND film suite — Layer 1 of the suite. It defines 11 frozen dataclasses (Asset, Constraint, Debrief, Finding, MissionBrief, MissionOutcome, MissionPlan, Provenance, ReconReport, ReconRequest, Step) and a 5-method provider contract (brief, recon, plan, execute, debrief) that every film package implements. The package ships the machinery every coordinator needs: deterministic JSON round-trip serialization for all protocol objects, a gadget registry and a mission/package registry for discovered providers, slug-based film discovery on `sys.path`, and 9 capability helpers (brief_issues, derive_provenance, fingerprint, hash_provenance, normalize_debrief, normalize_lessons, plan_schema_issues, require_plan_schema, validate_brief) layered on the frozen surface — soft brief validation, plan-step schema checks, provenance hashing, and debrief normalization. A reference film (sable) exercises the full lifecycle, and the 27 film packages of the suite implement the same contract. Every artifact crosses process boundaries as tagged JSON (reserved key `__bond_type__`), and the suite enforces a 90% line-and-branch coverage floor on `src/`. This is version 0.1.0 of the contract.

32.3 Introduction — THE PROTOCOL: mission framing, the operational problem, and how to read this chapter

The BOND suite is a family of packages that cooperate to run missions: 5 lifecycle stages — brief, recon, plan, execute, debrief — carry a mission from declaration to debrief. For that cooperation to be possible, every participant must agree on the shape of the artifacts that cross its boundaries: what a mission brief contains, what a recon report looks like, what an outcome must prove about itself.

BOND-API exists to make that agreement explicit and durable. It is *the protocol*: the single contract every film package implements and every coordinator consumes. Freezing it first — before any of the 27 film packages are written — means the films are coded against a stable, tested surface rather than a moving one.

32.3.1 What this package provides

- **The protocol** (`src/bond_api/protocol.py`): the frozen dataclasses and the runtime-checkable `MissionProvider` contract.
- **Discovery** (`src/bond_api/discovery.py`): find film packages on `sys.path` by slug convention — a top-level module named by its slug exposing `mission.MissionProvider`.
- **Registries** (`src/bond_api/registry.py`): `GadgetRegistry` for film-provided gadgets and `MissionRegistry` for discovered providers.
- **Serialization** (`src/bond_api/serialization.py`): deterministic JSON round-trips of every protocol object — no pickling.
- **The reference film** (`films/sable/mission.py`): a real, deterministic provider that exercises the full lifecycle and demonstrates the contract.

32.3.2 Reader’s guide

Chapter 2 defines the protocol and the discovery/serialization conventions. Chapter 3 reports conformance evidence from the test suite. Chapters 5–6 document the environment and reproducibility contract. Chapter 8 is the honest counterweight: the deliberately permissive boundaries, the wire-format strictness now enforced, and the places where the evidence is bounded.

32.4 Methodology — THE PROTOCOL: the analytical models and algorithms that drive the mission

32.4.1 The mission lifecycle

A mission is a deterministic 5-stage flow. Each stage has one input, one output, and no hidden state:

Stage	Input	Output
brief	—	MissionBrief
recon	ReconRequest	ReconReport
plan	MissionBrief	MissionPlan
execute	MissionPlan	MissionOutcome
debrief	MissionOutcome	Debrief

The orchestrator helper `run_mission(provider, request)` composes the 5 stages in exactly this order and returns the final `Debrief`. Because the stages are pure functions of their inputs, coordinators can re-run, cache, or serialize any stage boundary without special-casing a film.

32.4.2 Frozen dataclasses

Every artifact in the table above is a frozen dataclass (`@dataclass(frozen=True)`): immutable by construction, value-equal, and validated at construction time. (Hashability follows Python’s rules: an object is hashable when all of its fields are; opaque payloads such as `Asset.payload` may hold mutable values, which makes those particular instances unhashable — coordinators should treat protocol objects as values, not dict keys.) Validation is strict where the contract demands it — a mission brief must declare a non-empty `film`, `codename`, and at least one objective; provenance must carry a non-negative integer seed and a non-empty input hash — and permissive where flexibility is the point (`Asset.payload` is opaque and JSON-serializable).

32.4.3 The provider contract

`MissionProvider` is a `typing.Protocol` marked `@runtime_checkable`. A film package is a top-level module named by its slug exposing `mission.MissionProvider`; discovery instantiates it, verifies the structural contract with `isinstance`, and binds it to its slug. A provider that declares a `film` different from the slug it was discovered under is rejected as an identity mismatch.

32.4.4 Discovery

`load_provider(slug, search_paths=...)` imports `<slug>.mission` from the given search paths (default `sys.path`). `discover(slugs=..., search_paths=...)` autodetects candidate slugs — top-level directories containing `mission.py` — or takes an explicit slug list, loading every discoverable provider and skipping broken ones so one bad film never hides the rest.

32.4.4.1 Failure semantics Discovery is deliberately strict in one direction and tolerant in the other, because the two callers want opposite things:

Situation	<code>load_provider</code> (single film)	<code>discover</code> (bulk)
Search path is not an existing directory	<code>DiscoveryError</code>	<code>DiscoveryError</code> (path validation precedes any loading)
Slug is not an importable module name	<code>DiscoveryError</code>	slug skipped
Package missing, provider absent, constructor raises	<code>DiscoveryError</code>	slug skipped
Provider fails the structural protocol check	<code>DiscoveryError</code>	slug skipped
Provider declares a film different from its slug	<code>DiscoveryError</code> (identity mismatch)	slug skipped
Search-path entry exists but cannot be listed	not reached	entry skipped

A coordinator asking for one named film wants a loud failure; a coordinator sweeping the 27-film fleet wants every healthy film to load regardless of a sick one. The split is not “validation before loading” versus “failures during loading”: slug validation happens *inside* the per-slug load, which `discover` catches, so an explicitly requested but malformed slug is skipped exactly like an unimportable one and `discover` returns the healthy remainder. Search-path validation is the one check that runs before any loading and therefore raises from `discover` too. Both behaviors are pinned by `tests/test_discovery.py`; a caller that needs a malformed slug to be loud must call `load_provider`. Both paths leave `sys.path` exactly as they found it: entries inserted for the import are removed in a `finally` block, and the one tolerated exception there — a film that rewrote `sys.path` itself on import — is documented at the call site rather than silently swallowed.

32.4.5 Serialization

Protocol objects serialize to plain JSON via `to_json` / `from_json`. The wire format is tagged (`{"__bond_type__": "<ClassName>", ...fields}`) so the exact type reconstructs on decode; tuples and lists are explicitly distinguished; output is key-sorted with fixed indentation so identical objects produce byte-identical text. No pickling is used anywhere.

32.4.6 Registries

`GadgetRegistry` maps film slugs to named gadgets (opaque payloads — data, callables, handlers). `MissionRegistry` is the mission/package registry of discovered providers, keyed by film slug, filled from `discover` and used by coordinators to drive providers without touching import machinery again.

32.4.7 Capability helpers

The frozen contract is deliberately minimal; capability helpers layer reusable semantics on top of it without changing a single field or flow (`src/bond_api/capabilities.py`):

- **Soft brief validation** — `brief_issues` / `validate_brief` report advisory issues (duplicate asset names, duplicate constraint kinds, duplicate objectives, whitespace-slack asset names) that a coordinator may tolerate or escalate, complementing the constructor’s hard invariants.
- **Plan-step schema checks** — `plan_schema_issues` / `require_plan_schema` verify no duplicate step names, well-formed param keys, JSON-serializable params, and optionally an exact expected step-name sequence.
- **Provenance hashing** — `fingerprint` is a deterministic SHA-256 over the canonical JSON encoding of arbitrary parts; `hash_provenance` fingerprints a full provenance record; `derive_provenance` builds a provenance whose `input_hash` follows from the mission’s inputs, making outcomes auditable without re-running the film.
- **Debrief normalization** — `normalize_lessons` / `normalize_debrief` strip, drop, and dedupe lessons while preserving first-occurrence order, and re-validate the outcome.

All helpers are deterministic and serialize cleanly through the same JSON surface as the protocol objects they consume.

32.5 Results — THE PROTOCOL: measured outcomes, headline numbers, and what they establish

32.5.1 Protocol surface

The frozen surface consists of 11 dataclasses — `Asset`, `Constraint`, `Debrief`, `Finding`, `MissionBrief`, `MissionOutcome`, `MissionPlan`, `Provenance`, `ReconReport`, `ReconRequest`, `Step` — and the 5-stage provider flow (`brief`, `recon`, `plan`, `execute`, `debrief`). The surface is verified by `protocol_surface` and pinned by `tests/test_protocol.py`, so any accidental addition or removal of a contract type fails the suite.

32.5.2 Conformance evidence

The test suite is the conformance evidence for the contract. Test counts and measured coverage are not restated here: they are produced by the run itself (`uv run pytest tests/ --cov=src`) and the enforced floor is 90% line **and** branch coverage on `src/`, read straight from the `fail_under` gate declared in `pyproject.toml` rather than repeated in prose.

- **Round-trip serialization:** every protocol dataclass round-trips through `to_json/from_json` with exact type and value equality; tuples stay tuples, lists stay lists, nested protocol objects stay nested. Exactness is now enforced rather than assumed on the two paths that previously broke it: a payload dict carrying the reserved wire key is rejected at encode time instead of decoding back as a protocol object, and the protocol-type check is on class identity instead of class name, so a dataclass whose `__name__` shadows a protocol type is rejected instead of being flattened onto that type’s fields. The wire tag (`__bond_type__`) is published as `bond_api.serialization.SERIALIZATION_TAG`, and a test pins the value declared in `manuscript/config.yaml` to the live constant, so the format cannot drift from its documentation.
- **Registry semantics:** gadget registration/merging/overwrite/drop and mission-provider register/lookup/drop are exercised with real providers.
- **Discovery:** the reference film (`sable`) is discovered by slug and by autodiscovery; malformed packages fail with actionable `DiscoveryErrors`; unreadable search-path entries are skipped rather than raised, verified against a real permission-denied directory.
- **Determinism:** the reference film runs the full lifecycle with fixed seed and zero wall-clock dependence; outcomes are byte-identical across runs. The wire text and the fingerprint digests are pinned to checked-in literal goldens (`tests/test_golden_determinism.py`) rather than to same-process self-comparisons, and the same goldens are re-derived in a freshly spawned interpreter under a randomized hash seed — which is what the “across processes” half of the claim asserts. Platform independence is established only over the platforms the suite actually runs on; a literal golden makes a platform-dependent encoding fail there rather than pass silently, but no test in this package can assert an untested platform.
- **Capability helpers:** each of the 9 helpers (`brief_issues`, `derive_provenance`, `fingerprint`, `hash_provenance`, `normalize_debrief`, `normalize_lessons`, `plan_schema_issues`, `require_plan_schema`, `validate_brief`) has dedicated tests — advisory brief issues, plan-schema matches and violations, SHA-256 fingerprint determinism and key-order independence, derived provenance records, and lesson normalization (strip/drop/dedupe, first-occurrence order).

32.5.3 Capability-helper surface

The 9 capability helpers layer reusable semantics on the frozen contract without changing a field. Their names are public API (pinned by `CAPABILITY_NAMES` and exercised by `tests/test_capabilities.py`);

Helper(s)	Role
<code>brief_issues / validate_brief</code>	Advisory brief validation — duplicate asset names, duplicate constraint kinds, duplicate objectives, whitespace-slack asset names
<code>plan_schema_issues / require_plan_schema</code>	Plan-step schema checks — duplicate step names, param-key sanity, JSON-serializable params, optional exact step-name sequence
<code>fingerprint</code>	Deterministic SHA-256 over the canonical JSON encoding of arbitrary parts
<code>hash_provenance</code>	Fingerprint of a full provenance record (all four fields)
<code>derive_provenance</code>	Provenance whose <code>input_hash</code> derives from the mission inputs
<code>normalize_lessons / normalize_debrief</code>	Strip, drop, and dedupe lessons preserving first-occurrence order; re-validate the outcome

Each helper’s exact behavior and failure mode is pinned by a dedicated test, and all of them serialize through the same JSON surface as the protocol objects.

32.5.4 Gate integrity

Four gates in this package could have passed for the wrong reason, and all four are now controlled:

- **The unresolved-token gate is non-vacuous.** `resolve_manuscript_tree` scans every `NN_*.md` section for leftover brace-delimited tokens. A manuscript directory containing no sections would have scanned nothing and reported success; strict mode now raises instead, and the test suite pins that behavior with a real empty directory.
- **The empty-surface gate has a positive control.** The check that the protocol exposes dataclasses lives in `require_surface`, which is called directly with a genuinely empty surface in the tests — so the gate is known to fire, not merely known to pass on the real package.
- **The determinism assertions pin values, not themselves.** Assertions of the form `fingerprint(x) == fingerprint(x)` and `to_json(o) == to_json(o)` hold for any deterministic function and cannot fail; they have been replaced by literal goldens plus a fresh-interpreter re-derivation.
- **The claim ledger is read by a gate.** `data/claim_ledger.yaml` restates six values the package derives elsewhere. Nothing consumed the file, so a stale row was undetectable. `tests/test_claim_ledger.py` now re-derives every row from its cited source of truth and fails when a row is added without a verifier, so the check cannot lapse into covering nothing.

Thin-orchestrator conformance is likewise enforced by machine rather than by review: a test parses each `scripts/*.py` file with `ast` and fails if a script grows business logic beyond `main` or stops delegating to `bond_api`.

All tests use real data and real computation — no mocks.

32.6 Conclusion — THE PROTOCOL: findings, verdict, and what the mission establishes

BOND-API freezes the contract the film suite is coded against. Version 0.1.0 delivers:

- a validated, frozen protocol surface (11 dataclasses, 5 provider stages) with construction-time invariant checks;
- deterministic JSON serialization that coordinators and CLIs can rely on across process boundaries, under the published reserved key `__bond_type__`;
- discovery that finds film packages by slug convention and binds them to their identity;
- registries that give coordinators a single place to hold gadgets and providers;
- 9 capability helpers (`brief_issues`, `derive_provenance`, `fingerprint`, `hash_provenance`, `normalize_debrief`, `normalize_lessons`, `plan_schema_issues`, `require_plan_schema`, `validate_brief`) layered on the frozen surface, adding advisory brief validation, plan-schema enforcement, provenance fingerprinting, and debrief normalization without altering a single contract field;
- a reference film (`sable`) and a test suite demonstrating the whole contract with real computation.

32.6.1 What the freeze buys

The value of freezing first is asymmetric. A film package written against a moving contract pays the migration cost 27 times over; a film package written against a frozen one pays it never. Everything added since the freeze — the capability helpers, the published wire tag, the gate-integrity work — is strictly additive: existing types, fields, defaults, and the `run_mission` flow are byte-identical to the version the first films compiled against.

Downstream packages (`bond-coordinator`, `bond-orchestrator`, `bond-cli`, and the film packages) implement against this version (0.1.0) of the protocol. The contract stays intentionally frozen: changes require a new version and a deliberate migration, never a silent edit.

Since the last revision the wire format is strict where loose encoders are costly: non-finite floats are rejected at both encode and decode, and a payload dict carrying the reserved `bond_type` key with an unrecognised tag value fails loudly rather than silently

decoding as an untagged dict. The honest limits of the package — the deliberately permissive boundaries, the interpreter-bounded determinism claims, and the provenance uncertainty — are stated in Chapter 8 rather than left for a reader to discover.

32.7 Experimental Setup — THE PROTOCOL: canonical scenarios, parameters, and configuration

32.7.1 Package layout

Path	Role
src/bond_api/protocol.py	Frozen dataclasses + MissionProvider contract (pure, stdlib only)
src/bond_api/discovery.py	Slug-based film discovery on sys.path
src/bond_api/registry.py	GadgetRegistry + MissionRegistry
src/bond_api/serialization.py	Deterministic JSON round-trips
src/bond_api/capabilities.py	Soft brief validation, plan schema checks, provenance hashing, debrief normalization
src/bond_api/manuscript_variables.py	Token hydration from config.yaml
films/sable/mission.py	Reference film implementing the contract
scripts/	Thin orchestrators (preflight, demo mission, token hydration)
tests/	Zero-mock test suite

32.7.2 Configuration

Protocol parameters live in manuscript/config.yaml under protocol: (identity, codename, reference film slug, serialization tag). Token variables inject them into this manuscript — metrics are never hand-authored in prose. The current version is 0.1.0.

32.7.3 Environment

- Python: 3.14.6 (the interpreter that generated this text)
- Manuscript date: 2026-08-04 — read from manuscript/config.yaml → paper.date, a committed value, not a clock read. The generator reads no wall clock at all, so regenerating output/ twice produces byte-identical files; see manuscript/06_reproducibility.md.
- Only external runtime dependency: pyyaml (manuscript config parsing); the protocol core is stdlib-only.

32.7.4 Running the suite

```
cd bond-api
uv run pytest tests/ --cov=src --cov-fail-under=90
uv run python scripts/00_preflight.py
uv run python scripts/demo_mission.py
uv run python scripts/z_generate_manuscript_variables.py
```

32.8 Reproducibility — THE PROTOCOL: verification gates, deterministic regeneration, and artifacts

32.8.1 Determinism policy

The protocol makes determinism a contract, not a hope:

- every mission outcome carries Provenance(package_version, seed, input_hash, wall_time_s);
- the reference film uses a fixed seed and performs no random draws;
- to_json output is key-sorted with fixed indentation, so identical objects serialize byte-identically;
- provenance hashing (fingerprint, hash_provenance, derive_provenance) uses the SHA-256 secure hash standard [of Standards and Technology, 2015] over the canonical JSON encoding, so identical inputs yield identical fingerprints across processes and platforms. What is tested, in tests/test_golden_determinism.py: the wire text and four digests are pinned to checked-in literals, and re-derived in a freshly spawned interpreter under PYTHONHASHSEED=random. That binds the cross-process claim directly and turns any encoding change into a red test. Platform independence is bounded by the platforms the suite is run on — the literals fail loudly on a platform that disagrees, but the suite cannot assert a platform it has never executed;
- tests use fixed inputs throughout; any future timing assertions must use bounds, not exact values, per docs/testing_philosophy.md.

32.8.2 Artifact inventory

Artifact	Producer
output/data/manuscript_variables.json	scripts/z_generate_manuscript_variables.py
output/manuscript/*.md (token-resolved sections)	scripts/z_generate_manuscript_variables.py

output/ is disposable: regenerate, never hand-edit.

32.8.2.1 Regeneration is byte-identical Both artifacts above are byte-identical across repeated regeneration on one interpreter: running `scripts/z_generate_manuscript_variables.py` twice writes the same bytes, no matter how much time passes between the runs. This holds because `generate_variables` reads **no wall clock**. Every token resolves from a committed source — the live package surface, `manuscript/config.yaml`, or the `fail_under` gate in `pyproject.toml`. The manuscript date is `paper.date` from the config, a value that changes in a diff rather than on every run.

The scope of that claim, stated exactly:

- **In scope:** repeated runs of the generator on the same interpreter produce identical bytes for `output/data/manuscript_variables.json` and every file under `output/manuscript/`.
- **Out of scope:** a *different* interpreter. `PYTHON_VERSION` is the one environment-derived token, so upgrading Python deliberately changes those bytes — that is the token doing its job, and the manuscript says which interpreter produced its text.

This is bound by `tests/test_regeneration_determinism.py`, which runs the real generator script twice into two directories, sleeps across a whole-second boundary between them so any clock read would show, and compares every output file byte for byte. A companion `ast` gate fails if a wall-clock call is reintroduced into the generator module, and that gate carries a positive control — it is fed a source that *does* read the clock and must flag it, so it cannot pass by scanning for something it would never find.

32.8.3 Token hydration

Every token in `manuscript/*.md` resolves through `src/bond_api/manuscript_variables.py::generate_variables`, which reads its sources of truth — live code, configuration, and gate declarations — and never a hand-typed value:

Token group	Source of truth
Identity, surface inventory, capability list	the live package (<code>protocol_surface</code> , <code>CAPABILITY_NAMES</code>)
Version, title, subtitle, reference film, film count	<code>manuscript/config.yaml</code>
Wire-format tag	<code>bond_api.serialization.SERIALIZATION_TAG</code>
Coverage floor	the <code>fail_under</code> gate in <code>pyproject.toml</code>
Manuscript date	<code>manuscript/config.yaml</code> → <code>paper.date</code> (committed, never a clock read)
Python version	the running interpreter

Deriving the coverage floor from `pyproject.toml` rather than from prose or config means the manuscript quotes the gate that actually runs; deriving the wire tag from the module means renaming the tag breaks the build rather than the documentation.

Three checks keep this honest: the strict-mode script fails on any unresolved token; `tests/test_manuscript_variables.py` cross-references every manuscript token against the generator; and strict resolution refuses to report success over an empty section set, so the token gate cannot pass by scanning nothing.

`data/claim_ledger.yaml` is the one place that deliberately restates code-derived values outside the token map. It is not self-certifying prose: `tests/test_claim_ledger.py` re-derives every row from the source the row cites and fails on a row with no verifier.

32.9 Scope and Related Work — THE PROTOCOL: boundaries, positioning, and relationship to the literature

32.9.1 Scope

BOND-API defines *what* missions are and *how* films plug in; it does not run missions. Explicitly out of scope for this package:

- mission scheduling, orchestration, or coordination (`bond-coordinator`, `bond-orchestrator`);
- film content — the 27 film packages implement `mission.MissionProvider`;
- CLI surface and user tooling (`bond-cli`);
- shared helpers for the suite (`bond-utilities`), when that package lands;
- any mutation of the protocol after this freeze — changes require a versioned migration, not a silent edit.

32.9.2 Design relationship

- **Film packages:** implement the contract exactly as `films/sable/mission.py` does — discoverable by slug, deterministic, self-describing briefs.
- **Coordinators:** consume `discover` + `MissionRegistry`, move artifacts with `to_json/from_json`, and never import film internals.
- **bond-utilities:** not yet available as a utilities package; BOND-API currently depends only on the stdlib plus `pyyaml`. The integration point is tracked in `TODO.md`.

32.9.3 Related standards

The protocol follows the same conventions as the template it was scaffolded from — frozen dataclasses for value objects, `typing.Protocol` for structural contracts, deterministic serialization, and a zero-mock test discipline — so code moving between template projects and the BOND suite keeps its shape.

32.10 Limitations and Uncertainty — THE PROTOCOL: assumptions, threats to validity, and what the numbers do not claim

Every engineering claim in this manuscript is scoped honestly. This section collects the limits that a hostile reviewer of a protocol library would care about — where the package is deliberately permissive, where a value is a declared contract rather than a measurement, and where a reader should not over-read the evidence.

32.10.1 Deliberately permissive boundaries (documented, not bugs)

These are accepted design decisions, recorded so nobody re-litigates them:

- **discover silently skips an invalid slug.** Slug validation happens inside `load_provider`, whose `DiscoveryError` `discover` catches, so `discover(slugs=["9bad-slug"])` returns the healthy remainder rather than raising — the “one broken film never hides the rest” design. The strict entry point is `load_provider`, which raises. A caller who passes a typo’d slug to `discover` gets silence, not an error; both sides are pinned by `tests/test_discovery.py`.
- **sys.modules caches slug imports.** Two film packages sharing a slug in different locations cannot both load in one interpreter. Distinct slugs per location are required.
- **Discovery mutates sys.path for the duration of an import** and restores it in a `finally` block. This makes concurrent discovery from multiple threads unsafe by construction; discovery is a startup-time operation.
- **Protocol objects with mutable payloads are unhashable.** Value equality, not dict-key semantics — coordinators should treat protocol objects as values, not hash keys.
- **run_mission executes recon but does not thread its report into plan.** The frozen signature is `plan(brief)`; callers needing the report drive the five stages directly.
- **The manuscript PDF is rendered outside this package.** This package ships token hydration and validation only; rendering runs in the monorepo `infrastructure.rendering` pipeline over `output/manuscript/`.

None of these is a defect; each is a trade-off chosen for the frozen contract and stated at its call site and in the test suite.

32.10.2 Wire-format strictness (enforced since Round 2)

The serialization layer is strict where loose encoders are costly. Two holes were closed and are now pinned by regression tests:

- **Non-finite floats are rejected.** RFC 8259 defines no `NaN` or `Infinity` token, yet Python’s `json.dumps` writes them as bare literals by default. A non-finite float anywhere in a payload — including `Provenance.wall_time_s`, where `inf >= 0` used to pass the non-negativity check — is now rejected at encode time and at decode time (a foreign document carrying `NaN/Infinity` fails loudly rather than decoding to a float the encoder could never re-emit). This narrows `to_json/from_json` to plain, portable JSON.
- **The reserved wire key must carry a known tag.** A payload dictionary whose `__bond_type__` value is `null`, `false`, `0`, or `[]` used to fall through to the plain-dict branch and decode as an untagged dict that the encoder then refused to re-encode. Every such tag is now loud, so `to_json(from_json(doc))` is total over the documents `from_json` accepts.

32.10.3 Where the evidence is bounded

- **Coverage is a floor, not a proof.** The enforced gate is 90% line **and** branch coverage on `src/` (`--cov-fail-under=90`); the measured figure is whatever the suite prints, never restated here. Line and branch coverage across the six `src/` modules is intentionally not quoted, because it changes with the code.
- **Determinism is bounded by the interpreter.** `to_json` output and the provenance fingerprints are byte-deterministic and pinned to checked-in literal goldens, re-derived in a freshly spawned interpreter under `PYTHONHASHSEED=random` (the “across processes” claim). Platform independence is established only over the platforms the suite actually runs on; no test in this package can assert a platform it has never executed on.

- **Regeneration is byte-identical per interpreter.** `output/` regenerates to identical bytes on one interpreter because `generate_variables` reads no wall clock. 3.14.6 is the single environment-derived token, so upgrading Python deliberately changes those bytes — that is the token doing its job.

32.10.4 Uncertainty in provenance

`Provenance.wall_time_s` is a recorded duration for auditability, not a causal claim: nothing in this package asserts a wall-clock bound, and the reference film stamps 0.0 by design so outcomes are reproducible regardless of when they run. Where timing is measured by a film package, the value must be finite (rejected by construction if not) and interpreted with any timing-measurement granularity caveats the measuring package documents (`docs/testing_philosophy.md`: use bounds, not exact values, for timing).

32.10.5 Versioning uncertainty

The package version 0.1.0 is declared in four places (`pyproject.toml`, `bond_api.__version__`, `manuscript/config.yaml` → `paper.version`, and the reference film's provenance stamp). All four are gated to agree by `tests/test_exports.py`, so a bump edited in only one place fails the suite instead of silently shipping stale metadata. The gate makes drift detectable, but it does not make the version a single loaded value — that single-source refactor is tracked in `TODO.md` if a future round wants it.

32.11 Sources — THE PROTOCOL: bibliography

[Levkivskiy et al. \[2017\]](#); [Smith \[2017\]](#); [van Rossum et al. \[2014\]](#); [Bray \[2017\]](#); [of Standards and Technology \[2015\]](#); [Friedman \[2026b\]](#)

33 Bond Coordinator — MISSION CONTROL

infra package · package codename MISSION CONTROL.

33.1 Concepts — MISSION CONTROL: domain and operational focus

Fleet registry, mission ledger, status, situation room

33.2 Abstract — MISSION CONTROL: mission summary

This paper presents **BOND-COORDINATOR** (codename **MISSION CONTROL**), the Layer-2 mission-control package for PROJECT BOND, a fleet of software packages descended from a numerical-optimization research exemplar. Where that exemplar demonstrated optimization mathematics inside a single package, BOND-COORDINATOR demonstrates the *fleet* view: a deterministic registry of all 27 film packages, a forward-only mission-state ledger, an aggregate suite-status report, and a self-contained HTML situation room. The registry (`src/bond_coordinator/registry.py`) is seeded from the ROSTER conventions in `projects/working/bond/SCOPE.md §2.1` and assigns a provisional mission codename per film; `reconcile.py` merges a film’s real codename and gadget table over it (refresh/replace semantics) once a film declares them via `bond-api`. The ledger (`src/bond_coordinator/ledger.py`) tracks each package’s lifecycle — brief → recon → plan → execute → debrief — with strict forward-only transition rules, attributable-identity validation, an append-only audit trail of who/when/version, and deterministic audit summaries. The status module (`src/bond_coordinator/status.py`) aggregates counts by state and phase, a measured-coverage summary, stale/blocked flags, and a per-layer summary of the fleet topology. The situation room (`src/bond_coordinator/situation.py`) renders the whole fleet — packages, mission-timeline bars, coverage heat, suite layers, a per-film Q-branch gadget table, and the recent-transitions audit trail — into one self-contained HTML document. **Roster state (this build):** 27 packages tracked across 6 declared topology layers; 25 scaffolded, 0 building, 2 ready, 0 red; 2 packages with measured coverage (mean 95.5%, 2 at/above the 90% gate, 25 without a gate result in this build); 0 blocked; 0 stale. Provenance of those status and coverage figures: *measured gate output on 2 of 27 registry rows* — the registry’s seed defaults are not a measurement of any film package; see the Results section. **Keywords:** mission control, film-package registry, mission-state ledger, append-only audit trail, registry reconciliation, suite status, software fleet, fault isolation, reproducible research

33.3 Introduction — MISSION CONTROL: mission framing, the operational problem, and how to read this chapter

PROJECT BOND is a fleet of standalone software packages — one per James Bond film, plus a shared layer of Q-branch infrastructure — built to the same scientific bar as the research exemplar from which each package is forked: pure tested cores, zero mocks, $\geq 90\%$ coverage, thin orchestrators, and deterministic, provenance-recorded outcomes. With 27 film packages and 6 mission-infrastructure packages, the fleet needs a single point of truth about *what exists, where it is in its mission, and whether it is healthy*.

BOND-COORDINATOR (codename **MISSION CONTROL**) is that point of truth: the **registry and situation room** for the whole suite. It does not implement any film’s algorithms; it records, audits, and reports on them. Concretely it provides six capabilities, each a pure module under `src/bond_coordinator/` with a thin orchestrator in `scripts/`:

1. **Protocol mirror** (`types.py`) — `Asset`, `Gadget`, `GadgetRegistry`, `PackageStatus`, and the `MissionPhase` lifecycle enum, mirroring the `bond-api` shapes so the core carries no sibling imports.
2. **Registry** (`registry.py`) — the authoritative, curated list of the 27 film packages: slug, film title, release year, mission codename, status, and measured coverage when available.
3. **Reconcile** (`reconcile.py`) — the merge that replaces mission control’s provisional codenames with the codenames films actually declare, and harvests their gadget tables into the Q-branch inventory.
4. **Mission-state ledger** (`ledger.py`) — each package’s lifecycle position (brief → recon → plan → execute → debrief) with forward-only transition rules, attributable-identity validation, and an append-only audit trail.
5. **Suite status** (`status.py`) — a deterministic aggregate snapshot: counts by state and phase, a coverage summary, stale / blocked flags, and a per-layer summary of the fleet topology.
6. **Situation room** (`situation.py`) — a self-contained HTML dashboard that renders the whole fleet plus the Q-branch gadget inventory.

A seventh module, `manuscript_variables.py`, injects this document’s own identity and every quantity it reports (§the results section).

The package sits at Layer 2 (mission control) in the suite’s dependency topology. Its one cross-package consumer is the thin orchestrator `scripts/reconcile.py`, which loads each film’s Layer-1 `bond-api.protocol.MissionProvider` and converts the result to plain data before it reaches the core. The core itself imports neither `bond-api` nor the Layer-0 `bond-utilities`: it keeps a pure-core mirror of the protocol shapes as a deliberate boundary, not as a placeholder (see `TODD.md` and `docs/architecture.md`). Every module is deterministic, uses no framework mocking, and is exercised by a zero-mock real-data test suite.

33.3.1 Reader’s guide

- the methodology section defines the registry, ledger, status, and situation-room models and their invariants.

- the results section reports the measured fleet state.
- the experimental setup section documents the configuration surface and the enforced quality gates.
- the reproducibility section certifies determinism and provenance.
- the limitations section states honestly what every figure here does and does not certify, and how the seeded-versus-measured distinction can be misread.
- the scope section scopes the package and relates it to the rest of the fleet.

33.4 Methodology — MISSION CONTROL: the analytical models and algorithms that drive the mission

The pure core lives under `src/bond_coordinator/`: `types.py` (protocol mirror), `registry.py`, `reconcile.py`, `gates.py`, `ledger.py`, `status.py`, and `situation.py`, plus `manuscript_variables.py`. All are pure: they import only the standard library and `pyyaml`, never `infrastructure.*`, and never a sibling BOND package. Thin orchestrators in `scripts/` bind them to the filesystem (persisting the ledger, writing the dashboard, printing the report).

33.4.1 Registry

`registry.py` defines a frozen `PackageRecord` (rank, slug, film, year, codename, status, coverage, concepts) and the module-level `ROSTER` tuple of all 27 film packages, in Eon-canon order then non-Eon, exactly per `SCOPE.md` §2.1. Status is the enum `scaffolded | building | ready | red`; coverage is a measured percentage or `None`. Mission codenames are **assigned by mission control** from each film’s canonical plot element (e.g. Goldfinger’s Operation Grand Slam → `GRAND SLAM`) and are provisional until a film package declares its own via `bond-api`.

The shipped `ROSTER` literal fills only the identity fields; every row’s status and coverage are seed defaults (`scaffolded / None`) meaning *unmeasured by this process*. `gates.py` is the sole path by which they change: `parse_gate_manifest` decodes the `bond-ops` gate aggregate — raising on a malformed manifest rather than degrading to an empty result, since “nothing measured” is indistinguishable from an honest seed — and `apply_gate_results` derives each row’s status by rule (failed run → `red`; passing at or above `COVERAGE_FLOOR` → `ready`; passing below it, or passing with no coverage number, → `building`; no gate row → untouched). The rule is what makes the status a derived fact rather than an assertion, and it is asserted in both directions by tests that break when the rule is altered.

The registry also provides `get_package`, `find_by_status`, and `subset_roster` (the last is how tests seed deterministic film fixtures without depending on the whole fleet).

33.4.2 Reconciliation

`reconcile.py` implements the merge half of the codename policy: when a film declares its real codename and gadget table, `override_codenames` and `harvest_gadgets` return an updated registry with **refresh/replace** semantics (a later build wins), leaving the source immutable.

The module also owns `gadget_purpose`, which reduces a film-registered gadget payload to one line. The rule is explicit rather than duck-typed, because the obvious implementation is wrong: reading `payload.__doc__` on a plain-data payload resolves through to the *builtin type’s* docstring, so a `dict` reports “dict -> new empty dictionary” and a `list` reports “Built-in mutable sequence.”. Those are attributes of the interpreter, not declarations by the film, and recording them as a gadget’s purpose fabricates provenance. `gadget_purpose` therefore accepts a docstring only from a routine, class, or module — the shapes that syntactically carry one at their definition site — uses a string payload verbatim as the film’s own words, and otherwise derives a factual shape summary from the object itself (“data payload: dict of 8 entries”).

Discovery is deliberately outside the pure core. `scripts/reconcile.py` loads each film’s `bond_api.protocol.MissionProvider` via `bond_api.discovery.load_provider`, reads `provider.brief` and the film’s module-level `GADGETS`, and converts both to plain dictionaries before calling these functions — so no protocol type from a sibling package ever crosses the `src/` boundary. Because reconciliation imports 27 independently developed packages, each film is guarded individually: any exception raised while importing one film skips that film alone, with its exception type and message reported, and the remaining films still reconcile.

33.4.3 Mission-state ledger

`ledger.py` tracks each package’s position in the lifecycle brief → recon → plan → execute → debrief — 5 phases encoded by the `MissionPhase` enum, whose ordinal drives every transition rule. A `MissionLedger` holds the current entry per package plus an append-only list of every recorded transition. The transition rules are strict and executable:

1. A package must be registered in the roster.
2. Phases move strictly forward: brief → recon → plan → execute → debrief.
3. Re-entering the current phase is allowed (an idempotent *touch*).
4. Moving backwards, or **advance** past debrief, raises `ValueError`. `jump` permits multi-step forward advances (still audible, still forward-only).

Transition validation goes further than forward-only ordering. Every transition must be attributable: both the acting agent (`updated_by`) and the `version` being progressed must be non-empty strings, and an optional `note` must be a string. A public `validate_transition(slug, phase, by, version)` runs the full rule set **without mutating the ledger**, so a caller (or CLI) can gate a transition before committing it.

Every transition writes a frozen `LedgerEntry` (`slug, phase, updated_by, updated_at, version, note`), so the audit trail answers *who changed which package's state, when, and at what version*. The ledger deepens the audit into three deterministic views: `transition_summary` (entries per phase), `per_agent_summary` (entries per agent), and `phase_timeline(slug)` — the chronological (`phase, timestamp`) history that drives the situation room's mission timeline. Staleness is a pure function of entry age and an injectable clock: a debriefed mission is never stale.

33.4.4 Suite status

`status.py::build_status(roster, ledger,...)` assembles one deterministic snapshot: counts by status and by phase, a `CoverageSummary` (known count, mean/min/max over measured packages, count meeting the 90% gate), the set of stale packages, a set of `BlockedPackage` reasons computed from the combination of status and phase, and a **per-layer summary**:

- a **red** package is blocked (“flagged red”);
- executing before ready is blocked;
- planning while still scaffolded is blocked.

Per-layer summaries render the declared suite topology (`SUITE_LAYERS`, §2.2 of `SCOPE.md`) as one `LayerSummary` per layer — the five infrastructure layers (`layer0...layer4`), each with a readiness tri-state, plus the `films` layer aggregating the registry: tracked count, `READY` count, and mean coverage. Readiness is injectable via `layer_ready` (with `layer1` — the `bond-api` protocol — inferred from `dependency_ready`), so the report reflects which of the suite's 6 declared layers are actually available. All time-dependent inputs (`now, stale_days`) are injectable, so the report is reproducible byte-for-byte. The snapshot also *carries* the `stale_days` threshold it was computed with (default 30 days), so a downstream renderer states the threshold that was applied instead of assuming the default.

33.4.5 Situation room

`situation.py::build_situation_room(status, roster, ledger, gadgets)` renders a single self-contained HTML document (no external assets, no JavaScript):

- summary cards and a **per-package table** with status, phase, a **mission-timeline progress bar** of one cell per lifecycle phase (done / current / ahead), last update, and **coverage heat** — each coverage cell color-coded against the 90% gate (at/above green, mid-range amber, low red, unmeasured gray) with a legend. The renderer imports `COVERAGE_FLOOR` from `status.py` rather than restating it, so the legend cannot drift from the gate actually enforced;
- a **suite-layers table** from `status.layers`;
- a **per-film gadget table** grouping the Q-branch inventory by film, with a count of which tracked films have none;
- a **recent-transitions (audit trail)** table of the newest ledger entries.

Every value is HTML-escaped; the document is deterministic given its inputs. `scripts/build_situation_room.py` writes it to the `gitignored output/` tree.

33.5 Results — MISSION CONTROL: measured outcomes, headline numbers, and what they establish

All figures in this section are generated from source by `src/bond_coordinator/manuscript_variables.py` and injected as substitution tokens — they are never hand-authored in prose.

33.5.1 Fleet state

The registry currently tracks **27** film packages. Their distribution across statuses is 25 scaffolded, 0 building, 2 ready, and 0 red, with 2 packages carrying measured coverage (mean 95.5%, 2 at/above the 90% gate).

Those figures describe **the roster this document was built from**, and their provenance is stated by the same generator that produced them: *measured gate output on 2 of 27 registry rows*. The distinction matters, because the two possible answers look identical in the numbers and are not the same claim. The `ROSTER` literal in `registry.py` ships every row as scaffolded with `coverage=None` — a seed default, meaning *this process has measured nothing, never the fleet is unbuilt*. A build run with `--no-gate-manifest` therefore reports all 27 rows scaffolded with no coverage, and that is a statement about the seed, not a measurement of the 27 film packages. Measured values enter only through `gates.apply_gate_results`, from the `bond-ops` gate aggregate; a build run against that manifest reports each package's real status and real coverage. Nothing in this package measures a film package itself, and no figure above is evidence about any film's actual test suite.

33.5.2 Registry completeness

The roster is canon-ordered and complete: it spans `dr_no` (1962) through `never_say_never_again` (1983) with every Eon entry and both non-Eon entries. The full slug list is: `dr_no`, `from_russia_with_love`, `goldfinger`, `thunderball`, `you_only_live_twice`, `on_her_majestys_secret_service`, `diamonds_are_forever`, `live_and_let_die`, `the_man_with_the_golden_gun`, `the_spy_who_loved_me`, `moonraker`, `for_your_eyes_only`, `octopussy`, `a_view_to_a_kill`, `the_living_daylights`, `licence_to_kill`, `goldeneye`, `tomorrow_never_dies`, `the_world_is_not_enough`, `die_another_day`, `casino_royale_2006`, `quantum_of_solace`, `skyfall`, `spectre`, `no_time_to_die`, `casino_royale_1967`, `never_say_never_again`. Each record carries a provisional mission codename and the core concepts that will power that film’s software payload, so the registry is both an inventory and a build briefing.

33.5.3 Suite layers

The coordinator declares 6 topology layers — `layer0` (Q-branch primitives), `layer1` (The protocol), `layer2` (Mission control), `layer3` (The DAG runner), `layer4` (Front door & fleet ops), `films` (Film packages) — and reports each in the status snapshot and the situation room. The `films` layer reports 27 tracked with 2 ready and mean coverage over the measured rows only; the infrastructure layers report readiness only where probed (`layer 1` tracks the `bond-api` probe, the rest report the tri-state “not probed” unless a caller supplies readiness). Every one of these counts is derived from the roster handed to the generator, never hand-authored.

33.5.4 Reconciliation against the real fleet

Reconciliation is the point where mission control stops asserting and starts measuring. `scripts/reconcile.py` loads each built film’s `MissionProvider`, reads its declared codename, harvests its gadget table, and hands plain data to the pure merge in `reconcile.py`. Two properties are established by test rather than by inspection of a particular run, because the fleet’s contents change as film packages are built:

- **Per-film isolation is a positive control, not a promise.** A film that raises on import is skipped individually — with its exception type and message reported — while every other film still reconciles. The test materialises a film whose module raises at import time and asserts the run still succeeds and still harvests its healthy neighbour. This replaced a guard narrow enough that one half-edited package aborted the fleet-wide run.
- **No borrowed provenance.** A gadget’s purpose is derived by `reconcile.gadget_purpose`, which accepts a docstring only from payloads that syntactically carry one (routines, classes, modules), takes a string payload as the film’s own words, and otherwise emits a factual shape summary. Reading `payload.__doc__` unconditionally instead resolves to the *builtin type*’s docstring for plain data, recording CPython’s own strings as purposes the film never declared. A parametrised test asserts the derived purpose differs from `payload.__doc__` for every plain-data shape.

33.5.5 Ledger and status behaviour

The ledger’s forward-only rules are verified by tests that walk a package the full lifecycle, attempt illegal backwards and past-debrief transitions, assert that empty or non-attributable transitions are rejected, and check the deterministic audit summaries (`transition_summary`, `per_agent_summary`, `phase_timeline`). The status aggregator is verified against deliberately heterogeneous rosters (ready/building/scaffolded/red mixes) to exercise every blocked rule, the coverage summary, and the per-layer summaries. Observed behaviour in this build: 0 packages blocked and 0 stale, consistent with a fresh ledger at `brief` whose audit trail holds 27 entries (one per registered package). Coverage heat, mission-timeline bars, per-film gadget grouping, and the recent-transitions table are all rendered deterministically and asserted byte-identical across repeated builds.

33.5.6 Summary table (live-generated)

Every figure below is injected from the registry/ledger by `src/bond_coordinator/manuscript_variables.py` — none is hand-authored.

Metric	Value
Tracked film packages	27
Declared suite-topology layers	6
Mission-infrastructure packages (non-films layers)	6
Lifecycle phases	5
Enforced coverage floor	90%
Default staleness threshold	30 days
Roster span	<code>dr_no</code> (1962) → <code>never_say_never_again</code> (1983)
Measured coverage (known / at-or-above floor / mean)	2 / 2 / 95.5%
Status counts (scaffolded / building / ready / red)	25 / 0 / 2 / 0
Blocked / stale packages	0 / 0
Audit-trail entries (fresh ledger)	27

Provenance of the status and coverage columns: *measured gate output on 2 of 27 registry rows* — read the Limitations section (§[sec:limitations]) before interpreting them as claims about any film.

33.6 Conclusion — MISSION CONTROL: findings, verdict, and what the mission establishes

BOND-COORDINATOR (codename **MISSION CONTROL**) demonstrates a fleet-scale variant of the research-exemplar contract: instead of one package’s mathematics, it maintains the single point of truth for a **27**-package software fleet. Its contributions are:

- a curated, canon-ordered **registry** (`registry.py`) seeding every film package with slug, film, codename, status, and concepts;
- a **mission-state ledger** (`ledger.py`) whose forward-only rules and append-only audit trail make “who moved which mission, when, at what version” a machine-checkable fact;
- a **reconciliation** merge (`reconcile.py`) that replaces provisional codenames with the ones films actually declare and harvests their gadget tables, deriving each gadget’s purpose without ever borrowing a builtin’s docstring;
- a **gate-result adapter** (`gates.py`) that is the only way a registry row acquires a status or coverage number, deriving both by rule from `bond-ops` gate output;
- an aggregate **status** report (`status.py`) that turns registry and ledger into executably-defined stale/blocked/coverage signals;
- a self-contained **situation room** (`situation.py`) that renders the whole fleet — plus the Q-branch gadget inventory — as one HTML document.

The design decisions that matter are the pure-core boundary (nothing in `src/` touches `infrastructure.*`, the filesystem, or a sibling package — protocol types are mirrored rather than imported, and cross-package discovery is confined to a thin orchestrator), determinism (injectable clocks, no wall-clock tokens), partial-failure tolerance across a fleet of independently developed packages, and the zero-mock test discipline carried over from the exemplar. Two of those properties are held by positive controls — a deliberately broken film package, and an assertion that a derived gadget purpose differs from the payload’s inherited `__doc__` — because a guard that has never been seen to fire is not yet known to work.

What the situation room reflects is exactly what it was fed, and it says which that was. Reconciled codenames reach it only when `scripts/build_situation_room.py` is given a reconcile manifest (`--reconciled`), and measured status and coverage only when it is given a gate manifest (`--gate-manifest`); without either it renders the registry’s provisional codenames and unmeasured seed defaults, and prints that fact in the document’s provenance block rather than letting a seed default pass for a measurement. Both paths are held by tests that fail when the wiring is removed. The honest summary is therefore narrower than “always reflects reality”: the dashboard reflects the manifests supplied to it, and states their absence when they are not.

33.7 Experimental Setup — MISSION CONTROL: canonical scenarios, parameters, and configuration

33.7.1 Configuration surface

The manuscript identity is loaded from `manuscript/config.yaml` by `src/bond_coordinator/manuscript_variables.py::load_config` and injected as tokens; the only hand-authored prose values are the tokens themselves, never hardcoded numbers. Identity fields (title **BOND-COORDINATOR**, subtitle **The Registry and Situation Room for the BOND Suite**, version **0.1.0**, codename **MISSION CONTROL**, keywords) live under `paper:` and `mission:`.

There is deliberately **no experiment: block:** unlike the exemplar, the coordinator has no analysis-output dependency. `generate_variables(..., require_outputs=...)` accepts the flag for pipeline parity but never gates on generated files — this is the one intentional divergence from the exemplar’s strict analysis-to-manuscript contract, documented in `docs/architecture.md`.

33.7.2 Dependencies

- **Runtime:** `pyyaml` (manuscript config parsing). Everything else is the Python standard library.
- **Developer:** `pytest`, `pytest-cov` (coverage gate), `ruff` (lint + format), `mypy` (types).

33.7.3 Quality gates

The enforced gates for this package are:

1. **Coverage:** `uv run pytest tests/ --cov=src --cov-fail-under=90` — line + branch coverage on `src/` at or above 90%, the same floor `status.py::COVERAGE_FLOOR` applies when grading the fleet.
2. **Zero mocks:** no `unittest.mock` / `MagicMock` / `@patch` anywhere in `tests/`; all tests use real data and real computation.
3. **Types/style:** `ruff check` and `mypy clean` on `src/bond_coordinator` and `scripts/`.
4. **Determinism:** fixed clocks and fixed seeds; no wall-clock tokens in generated artifacts.

33.7.4 Test data

Behavioral tests are seeded from a **deterministic five-film fixture** of representative packages (`dr_no`, `goldfinger`, `goldeneye`, `skyfall`, `no_time_to_die`) via `subset_roster`, so the suite exercises every rule without depending on all 27 packages being built. A full-registry test independently asserts completeness (27 unique, canon-ordered slugs).

Tests that touch the live fleet are written to assert **this package’s** contract rather than a sibling’s internals, because sibling packages are developed concurrently and may be mid-edit at any moment. Where a deterministic film is required — for the broken-film isolation

control and the gadget-purpose regression guard — the test materialises its own minimal film package in a temporary directory and points the CLI at it with `--bond-root`. This keeps the suite’s result a function of this package alone.

33.8 Reproducibility — MISSION CONTROL: certification, gates, and regeneration

Reproducibility is a first-order requirement for mission control, because the suite’s provenance story depends on it. The certification below follows the reproducibility-in-computational-research guidance of [Sandve et al., 2013, Peng, 2011].

33.8.1 Determinism

- **Injectable clocks:** every time-dependent path (`MissionLedger`, `build_status`, `is_stale`) accepts an injectable `now`; tests pin a fixed UTC reference clock and assert exact timestamps and byte-identical outputs.
- **No wall-clock tokens:** `manuscript_variables.py` emits no `GENERATION_TIMESTAMP`-style token, so manuscript bytes depend only on the config file and registry/ledger state. A test asserts the situation room is byte-identical across repeated builds with identical inputs.
- **Deterministic audit views:** `transition_summary`, `per_agent_summary`, and `phase_timeline` are pure functions of the append-only history — their output is stable for a given ledger, and the situation room’s recent-transitions table sorts with a deterministic tie-break (`((timestamp, slug))`).
- **Fixed seeds:** no random draws anywhere in `src/` or `tests/`.

33.8.2 Provenance

- The ledger is **append-only** and validated: every transition must carry a non-empty agent and version (a `@require`-style identity check), and a non-mutating `validate_transition` gates moves before they commit. Every recorded transition lands in the audit trail, and `to_json/from_json` round-trip state and history losslessly. A thin CLI (`scripts/advance_mission.py`) persists the ledger to `output/data/mission_ledger.json` after each move.
- The audit trail is *countable, not just capturable*: at a fresh state it holds exactly 27 entries (one per registered package), and every later move is visible in `transition_summary` and `per_agent_summary`.
- Generated artifacts — `output/situation_room.html`, `output/data/suite_status.json`, `output/data/manuscript_variables.json`, and the resolved manuscript tree — are gitignored and understood to be disposable: they are always regenerated from source, never hand-edited.

33.8.3 Verification

The certification commands (identical to the enforced gates) are:

```
uv run pytest tests/ --cov=src --cov-fail-under=90
uv run ruff check src/bond_coordinator scripts
uv run mypy src/bond_coordinator scripts
```

The availability of the Layer-1 package (`bond-api`) is detected at runtime by a `importlib.util.find_spec` probe in `scripts/` and reported in the status and situation-room output. The probe measures *importability in this environment*, not whether the package exists, and the rendered wording says so: a false result reads “not importable in this environment (pure-core mirror in use)”. The pure core never depends on the result — it works from its own protocol mirror either way (`docs/architecture.md`).

Reconciliation against the live fleet is likewise reproducible in the sense that matters for a fleet of independently developed packages: it is *order-independent and partial-failure-tolerant*. Each film is imported under its own guard, so a film that raises at import is skipped with its exception type and message on stderr while every other film still reconciles, and the harvest merge is idempotent (re-registering a `(film, gadget)` pair replaces it, so repeated runs converge on the same table).

33.9 Scope and Related Work — MISSION CONTROL: boundaries, positioning, and relationship to the literature

33.9.1 Scope

BOND-COORDINATOR is **Layer 2 (mission control)** of PROJECT BOND. Its scope is exactly the pure core under `src/bond_coordinator/` — `types.py`, `registry.py`, `reconcile.py`, `ledger.py`, `status.py`, `situation.py`, and `manuscript_variables.py` — and their thin `scripts/` orchestrators. It implements **no film algorithms** and makes **no cross-film computation**: those belong to the 27 film packages. It does not orchestrate missions across packages (that is Layer 3, `bond-orchestrator`) and does not expose a front door (that is Layer 4, `bond-cli`).

Within that boundary this package is deliberately standalone. Its `src/` core is pure — no `infrastructure.*`, no sibling-package imports (the **reconcile script** discovers films through `bond-api`, but the pure merge functions take plain data) — so the registry, ledger, status, and situation room can run in any Python 3.10+ environment with `pyyaml` installed, exactly as the exemplar’s mathematical core could.

33.9.2 Related work

The package inherits the research-exemplar’s standards — $\geq 90\%$ zero-mock coverage, pure tested cores, thin orchestrators, injected manuscript variables [Hunt and Thomas, 1999, Gamma et al., 1995] — and applies the design-pattern vocabulary of software architecture [Bass et al., 2012] and enterprise registries [Fowler, 2002] to a multi-package fleet. The layered topology (strict downward dependency, per-layer readiness) follows the layered-architecture guidance in the software-engineering body of knowledge [Bourque and Fairley, 2014], and the central claim — that a machine-checkable registry and append-only, validated ledger make fleet health an executable fact — follows the “essence vs. accidents” framing of software complexity [Brooks, 1987].

33.9.3 Positioning within the fleet

The coordinator declares and reports 6 topology layers (layer0 (Q-branch primitives), layer1 (The protocol), layer2 (Mission control), layer3 (The DAG runner), layer4 (Front door & fleet ops), films (Film packages)) and surfaces per-layer readiness in the status snapshot and situation room:

Layer	Package(s)	Coordinator report
Layer 0 (Q-branch primitives)	<code>bond-utilities</code>	readiness tri-state
Layer 1 (the protocol)	<code>bond-api</code>	readiness (tracks the runtime probe)
Layer 2 (mission control)	<code>bond-coordinator (this package)</code>	registry, ledger, status, situation room
Layer 3 (the DAG runner)	<code>bond-orchestrator</code>	readiness tri-state
Layer 4 (front door / fleet ops)	<code>bond-cli</code> , <code>bond-ops</code>	readiness tri-state
Films	27 film packages	tracked count, READY count, mean coverage

Layer 1 (`bond-api`) is **built**, and `scripts/reconcile.py` consumes its `MissionProvider` discovery directly. The pure-core mirror of the protocol shapes (`src/bond_coordinator/types.py`) nevertheless stays: it is an architectural boundary that keeps `src/` free of sibling imports, not a placeholder waiting for the real package. Layer 0 (`bond-utilities`) is **built** as well, and its `mission_io` module carries the fleet’s provenance-record conventions; this package has not yet adopted it — ledger provenance still uses local `stdlib` JSON codecs, and routing it through Layer 0 is the one open integration item (see `TODO.md`). Layer 4’s `bond-ops` produces the fleet gate aggregate that `src/bond_coordinator/gates.py` consumes, which is how a registry row acquires a measured status and coverage.

33.10 Limitations and Uncertainty — MISSION CONTROL: assumptions, threats to validity, and what the numbers do not claim

No claim below is a discovery about any film; every statement is about the registry, ledger, and status machinery this package ships, and about the boundaries of what those numbers do and do not certify.

33.10.1 The seeded registry is not a measurement

The `ROSTER` literal in `src/bond_coordinator/registry.py` ships every one of the 27 film rows with `status=scaffolded` and `coverage=None`. Those are *seed defaults*: they mean “this process has not applied gate output to this row”, never “the fleet is unbuilt” and never “this film passes at some coverage”. When a manuscript or dashboard is built with `--no-gate-manifest`, every figure that reads “*measured gate output on 2 of 27 registry rows*” is therefore a statement about the seed, and only about the seed. The number 2 “ready” / 25 “scaffolded” in such a build must not be read as a claim about any film package’s actual state.

33.10.2 Measured values are only as good as the manifest that supplies them

The single path by which a row becomes `ready/building/red` with a real number is `gates.apply_gate_results`, consuming the fleet aggregate that `bond-ops` writes at a *convention* location (`../output/aggregate/gate_manifest.json`). Three properties bound the uncertainty of anything reported as “measured”:

1. **The path is a convention, not a contract.** Nothing in this package forces `bond-ops` to keep writing that exact path; if the sibling relocates its aggregate, every consumer here degrades to the unmeasured seed. That degradation is *announced* (the provenance line says `measured gate output on 2 of 27 registry rows`, and the scripts print `no gate manifest at ... to stderr`), so it is not a silent lie — but a fleet whose sibling moved a file is indistinguishable, in these outputs, from a fleet that was never measured.
2. **Coverage is treated as a bounded percentage.** A row whose coverage is non-finite, negative, or above 100 is rejected loudly by `parse_gate_manifest` (`ValueError`) rather than coerced into the fleet mean. This closes the class of bug where a stray `NaN/Infinity` in a manifest silently poisons `CoverageSummary.mean`.
3. **Absence of a row is absence of evidence.** A film with no gate row keeps its seed state. We cannot distinguish “not yet gated” from “gated and skipped”, which is honest but makes 25 an upper bound on how much of the fleet the status report actually certifies.

33.10.3 Ledger truthfulness is bounded by its inputs

The forward-only transition rule is *exact and centralised*: the same `_require_forward` helper backs both the mutating path and the non-mutating `validate_transition`, so the two cannot drift, and the defensive invalid-jump guard is reachable and tested rather than excluded. But the ledger records what a caller tells it. It validates that a move is forward and attributable; it does not verify that a film objectively “is” a given phase, nor that the `updated_by` identity is real. A `touch` re-stamps the current phase, so the audit tail’s per-phase counts (`LEDGER_PHASE_COUNT` phases, `brief` → `recon` → `plan` → `execute` → `debrief`) include same-phase re-entries and can overcount *activity* relative to *movement*. Staleness (30 days) is a heuristic: a mission is “stale” purely because no transition was recorded within the window, which treats an abandoned mission and an idle-but-legitimate one identically.

33.10.4 Gadget purposes are derived shapes, not film confessions

`reconcile.gadget_purpose` deliberately refuses to read a builtin’s `__doc__`, because doing so recorded CPython’s own strings as a film’s declared purpose. The flip side is that a plain-data payload is summarised by its *shape* (“data payload: dict of 8 entries”), which is factual but conveys nothing about what the gadget does. Purposes derived this way understate semantic content by construction; they make no claim about the gadget’s real behaviour.

33.10.5 Reconciliation is partial by design

`scripts/reconcile.py` imports 27 independently developed packages, each guarded individually: a film that raises at import is skipped with its exception type and message reported, and the other films still reconcile. The cost is that a run’s completeness is a runtime fact, not a constant — the set of films that declared a codename or harvested a gadget varies with what is built on disk that day. Nothing here should be read as “all 27 films reconciled”; the skipped set is always reported.

33.10.6 The situation room reflects its inputs, exactly

`build_situation_room` renders whatever roster, ledger, gadget table, and provenance wording it is handed, and says which of the reconciled/measured manifests (if any) were loaded. It is deterministic and HTML-escaped, so it cannot *distort* its inputs — but it cannot *repair* them either. A dashboard built with stale manifests is a faithful rendering of stale facts.

33.10.7 Uncertainty statement

Because this package issues no scientific quantities of its own, most conventional uncertainty analysis (error bars, confidence intervals) does not apply. Its uncertainties are *modelling* and *provenance* uncertainties, and they are qualitative: how complete a reconciliation run was, whether a seed-vs-measured figure is being read as the other, and whether the fleet gate aggregate this run consumed is the one the current `bond-ops` produces. We insist on stating those explicitly in every rendered artifact rather than quantifying them, because a spurious precision here would be the most misleading output the package could produce.

The reproducibility bar is documented in §[sec:reproducibility] and follows the reproducibility-in-research-software guidance of [Sandve et al., 2013, Peng, 2011].

33.11 Sources — MISSION CONTROL: bibliography

Gamma et al. [1995]; Brooks [1987]; Bass et al. [2012]; Fowler [2002]; Hunt and Thomas [1999]; Bourque and Fairley [2014]; Sandve et al. [2013]; Peng [2011]

34 Bond Orchestrator — DAG 00

infra package · package codename DAG 00.

34.1 Concepts — DAG 00: domain and operational focus

Multi-package mission DAG runner + real-film binding

34.2 Abstract — DAG 00: mission summary

BOND-ORCHESTRATOR (codename **DAG 00**) is the Layer 3 mission orchestrator of the BOND suite: it executes multi-package film missions end-to-end over the `MissionProvider` protocol. A mission is a directed acyclic graph of film packages — dependency edges exist exactly where one film’s recon outputs feed another film’s recon — and the orchestrator executes every package through the five lifecycle phases in deterministic topological order, producing a single consolidated debrief. The flagship canned mission, **OPERATION_OMNIBUS**, chains `goldfinger` → `goldeneye` → `no_time_to_die`; its last measured execution order was `goldfinger` → `goldeneye` → `no_time_to_die`, with an all-verdicts outcome of **PASS**. Every outcome is recorded with provenance (an independent, deterministic record writer — this package does not yet delegate to `bond-utilities.mission_io`, see §7.4), and intermediate reports persist to a state directory so an interrupted mission resumes from its last completed step. The execution core is pure logic with zero infrastructure imports; determinism is load-bearing: identical inputs produce byte-identical reports, checkpoint files, and provenance records.

34.3 Introduction — DAG 00: mission framing, the operational problem, and how to read this chapter

The BOND suite splits multi-package research-and-response operations into six cooperating packages: `bond-utilities` (shared mission I/O and provenance), `bond-api` (the `MissionProvider` contract), `bond-coordinator` (fleet-level planning), `bond-orchestrator` (this package — Layer 3, the DAG runner), `bond-cli`, and `bond-ops`. Each film package (`GOLDFINGER`, `GOLDENEYE`, `NO TIME TO DIE`,...) is itself a self-contained mission capability that implements the provider protocol.

This package, **BOND-ORCHESTRATOR** (codename **DAG 00**), is responsible for one thing: running a mission across N providers end-to-end and reporting a single consolidated debrief. It does not implement any film’s business logic — that belongs to the providers — and it does not decide which missions exist — that belongs to the mission registry in `src/bond_orchestrator/missions.py`. It only guarantees *order*, *resumability*, and *provenance*.

The guarantees are enforced by construction:

- **Order** — `src/bond_orchestrator/graph.py` builds the film DAG from `depends_on` edges and computes a canonical topological order (Kahn’s algorithm with id tie-breaking), so execution order never depends on declaration order or dictionary iteration.
- **Resumability** — `src/bond_orchestrator/runner.py` persists every completed (`phase`, `package`) step to a state directory and replays it on resume, continuing from the last completed step.
- **Provenance** — `src/bond_orchestrator/provenance.py` writes one provenance record per executed plan step and per debrief, with a deterministic run id.
- **Bounded recovery** — `src/bond_orchestrator/retry.py` re-executes a failing `execute` step a bounded number of times, immediately and without wall-clock backoff, so recovery never costs determinism.
- **Explicit concurrency** — `src/bond_orchestrator/planning.py` decomposes the DAG into the wavefront stages a parallel executor could run, reporting achievable parallelism and critical path without changing the sequential execution contract.
- **Real-film binding** — `src/bond_orchestrator/film_providers.py` adapts packages written against the frozen `bond_api.MissionProvider` protocol to the runner-side contract, so the fleet’s real films run under all of the above rather than a parallel code path.

Two of those deserve their negative statement up front, because they are what the package deliberately does *not* do. It does not sleep between retries: a wall-clock delay would leak a non-reproducible quantity into an artifact whose byte-stability is the point. And it does not execute packages concurrently: concurrency is emitted as a plan and verified as a property, not performed, so that the guarantees above hold under a single, auditable execution order.

The reader’s guide: §2 details the DAG model, the five-phase lifecycle, the retry policy, stage planning, real-film binding, and provenance; §3 reports the measured **OPERATION OMNIBUS** execution and the real-film runs; §4 states what the design bought and what it deferred; §5 lists the configured mission parameters and the mission shapes under test; §6 documents reproducibility; §7 bounds the scope and places the work against prior art.

34.4 Methodology — DAG 00: the analytical models and algorithms that drive the mission

34.4.1 2.1 The mission DAG

A mission is an ordered collection of film packages (`src/bond_orchestrator/protocol.py::Mission`). Each package declares `depends_on`, the set of package ids whose recon outputs feed its own recon. `build_film_graph` (`src/bond_orchestrator/graph.p`

y) turns that declaration into a directed acyclic graph with an edge (**upstream**, **downstream**) for every dependency, validating that:

1. no package depends on itself,
2. every dependency names a package inside the mission,
3. The graph is acyclic.

Execution order is the **deterministic topological order** of the DAG: Kahn’s algorithm with a sorted ready-worklist, so the result is canonical for a given set of packages regardless of declaration order [Kahn, 1962, Cormen et al., 2009d]. The current flagship mission, OPERATION_OMNIBUS, declares 3 packages and is therefore executed in exactly one valid topological order.

Cycle detection. A mission whose dependencies close a loop is rejected before any provider runs: `FilmGraph.find_cycle` (`src/bond_orchestrator/graph.py`) runs a depth-first search over sorted node ids and returns the *ordered cycle path* (e.g. ("goldeneye", "goldfinger", "goldeneye")); the raised `CycleError` carries that path, so a miswired mission names its own loop (goldfinger $\leftrightarrow$ goldeneye) instead of a bare “cycle detected” message. The same validation gates the stage planner (§2.5) and therefore the **plan** CLI command.

34.4.2 2.2 The five-phase lifecycle

Every provider implements the `MissionProvider` protocol (`src/bond_orchestrator/protocol.py`): **brief**, **recon**, **plan**, **execute**, **debrief**. The runner (`src/bond_orchestrator/runner.py::MissionRunner`) executes phase-major, package-minor: for each phase, every package runs in topological order, sharing one `MissionContext` that accumulates recon reports. A downstream package’s **recon** therefore reads its dependencies’ **outputs** (the DAG edge semantics), and **plan/execute/debrief** see the full recon picture.

Execution is sequential and deterministic; the topological order is exactly what makes a later parallel implementation safe, because every dependency’s outputs are guaranteed present before a consumer runs (§2.5 makes that parallelism explicit without changing the sequential contract).

34.4.3 2.3 Checkpoint / resume

When a `checkpoint_dir` is supplied, every completed (**phase**, **package**) step is persisted as `<checkpoint_dir>/<mission>/<phase>_<package>.manifest.json` with a `manifest.json` tracking completed steps in order. Re-running the same mission with the same checkpoint dir replays completed steps (recon reports are restored into the context so downstream steps see identical inputs) and resumes from the last completed step. The manifest records a deterministic *fingerprnt of the mission definition* (name + every package and its **depends_on** edges, via `protocol.mission_fingerprint`); a resume against a checkpoint dir written for a different definition is refused loudly rather than replaying stale steps that describe a mission never executed (TODO C1). Execute steps additionally persist their attempt count, so a retried step (§2.4) resumes with an identical retry record. This follows the standard checkpoint/rollback-recovery pattern for long-running distributed work [Elnozahy et al., 2002, Chandy and Lamport, 1985], specialised here for deterministic single-node execution.

34.4.4 2.4 Retry policy

A `RetryPolicy` (`src/bond_orchestrator/retry.py`) bounds how many attempts a failing **execute** step is given (default 1 attempt in configuration; the CLI raises it via `--retry-max-attempts N`). When any execution result carries a retryable outcome ("failure" by default), the step is re-executed — immediately and deterministically, with **no wall-clock backoff**, because a sleep would break the byte-determinism of reports, checkpoint files, and provenance records. After `max_attempts` the final outcome stands. This is fault *tolerance* in the standard sense — a transient fault is masked before it becomes a mission failure, while a persistent one is allowed to surface [Avizienis et al., 2004] — with the backoff term removed because reproducibility outranks throughput here. Every step that needed more than one attempt records a `RetryRecord` (package, phase, attempts, succeeded) in the mission report and in its checkpoint record; retries are thus observable, resumable, and manuscript-token-able, while the consolidated debrief only annotates them when they occurred.

34.4.5 2.5 Parallel-stage planning

`src/bond_orchestrator/planning.py` decomposes the film DAG into **parallel stages** — the classic wavefront scheduling of directed task graphs [Kwok and Ahmad, 1999]. Stage 1 holds every package with no dependencies; stage `k` holds every package whose dependencies all sit in stages `< k`; within a stage, package ids are sorted, so the plan is canonical. The `StagePlan` reports the achievable parallelism (widest stage) and the critical path length (number of stages), and the **plan** CLI command renders it. Execution itself remains sequential — the stages are the *planning artifact* that makes a future parallel executor provably safe, since every package in a stage may run concurrently without violating dependency order.

34.4.6 2.6 Real-film binding

The orchestrator’s own protocol (§2.2) is the *runner-side* contract: it is package-aware, because the runner must hand a `MissionPackage` and the shared `MissionContext` to every call. The film packages of the BOND fleet implement a different, deliberately narrower contract — the **frozen bond_api MissionProvider**, whose methods take only a `ReconRequest`, a `MissionPlan`, or a `MissionOutcome` and know nothing about DAGs. Neither contract may move: the frozen protocol is consumed by 27 film packages, and the runner-side shapes (`MissionPackage`, `MissionContext`) are consumed by `bond-cli` and `bond-coordinator`.

src/bond_orchestrator/film_providers.py resolves that tension with an adapter [Gamma et al., 1994]. FilmProviderAdapter wraps one discovered bond_api provider and presents the runner-side interface, translating in both directions:

- **Down** — the DAG hand-off is folded into the film’s own vocabulary. The adapter renders each depends_on dependency’s recon outputs into the ReconRequest.goal text (_citation_text), so a film that has never heard of the DAG still receives its upstream intelligence; a dependency that produced no outputs contributes nothing rather than an empty citation.
- **Up** — the film’s Finding list becomes the adapter’s ReconReport: labels with detail become outputs (the downstream hand-off channel), every finding becomes a rendered findings entry, the film’s plan steps are renumbered from 1, and the MissionOutcome’s numeric results become the Debrief metrics (booleans excluded — bool is an int in Python and would otherwise be reported as a measurement).

load_film_providers(slugs, bond_root) discovers each film from <bond_root>/<slug>/src through bond_api.discovery.load_provider and is strict: a missing package directory, an absent mission.py, or a discovery failure raises FilmProviderError rather than dropping a package, because a silently short mission would still report all-verdicts-pass. load_mission_providers(mission, bond_root) binds every package a Mission declares. bond_api is imported lazily, inside the methods that need it, so the pure core of §2.1–§2.5 remains liftable into any environment where bond-api is not installed.

34.4.7 2.7 Provenance

For every outcome — each executed plan step and each debrief — the runner emits a ProvenanceRecord (src/bond_orchestrator/provenance.py) carrying a deterministic run id (sha256 over mission name + package ids), the package and provider, the phase, the outcome, artifact references, and numeric metrics. Records are written as outcomes.json and a flat provenance.csv. This is an independent, deterministic provenance writer, not yet a delegation to bond-utilities.mission_io (the integration and its schema mapping are tracked in TODO.md, §7.4).

34.5 Results — DAG 00: measured outcomes, headline numbers, and what they establish

34.5.1 3.1 OPERATION OMNIBUS execution

OPERATION_OMNIBUS ran to completion over the deterministic reference providers (src/bond_orchestrator/providers.py), which implement the real MissionProvider protocol with fixed logic — no mocks, no random draws.

The execution order was:

goldfinger → goldeneye → no_time_to_die

The mission executed 6 plan steps across all packages, and the consolidated debrief closed with an all-verdicts outcome of PASS.

The per-package debrief table below is read out of the live execution (output/data/mission_report.json) by src/bond_orchestrator/manuscript_variables.py::results_tokens — the same report the consolidated debrief prints. No cell is hand-typed; every verdict and metric is the engine’s own record, formed with the same %g formatting the runner uses so the table cannot disagree with the debrief it summarizes.

Package	Verdict	Metrics
goldfinger	pass	market_index=0.92, reserves_t=4200
goldeneye	pass	shield_rating=0.87, uplink_redundancy=2
no_time_to_die	pass	candidate_antidotes=3, neutralization_rating=0.99

Dependency-edge semantics were verified at recon time. The run carried 2 hand-off edges, each one an upstream package’s recorded recon outputs reaching its declared downstream consumer:

goldfinger → goldeneye: gold_reserves_t=4200, market_index=0.92; goldeneye → no_time_to_die: emp_shield_rating=0.87

Those payloads are read out of output/data/mission_report.json by src/bond_orchestrator/manuscript_variables.py::_handoff_tokens, not typed into this section, so the edge evidence cannot drift from the run that produced it. The DAG’s data hand-off therefore behaved exactly as specified in §2.1.

34.5.2 3.2 Determinism and resumability

Two fresh runs of OPERATION_OMNIBUS — executed independently into two separate directories — produced byte-identical report.json, manifest.json, checkpoint step files, outcomes.json and provenance.csv, compared as raw bytes rather than as decoded objects (tests/test_determinism_bytes.py). The same holds for OPERATION DIAMOND and OPERATION SINGLET. A run interrupted before the final debrief resumed from the last completed step and produced a report.json byte-identical to a fresh run’s — the resumability contract of §2.3 holds.

The byte comparison itself carries a positive control: a companion test perturbs one artifact by a single byte and requires the comparison to flag exactly that file, so a green result means the writers are deterministic rather than that the comparison is blind. Injecting a wall-clock value into any persisted record turns the suite red.

34.5.3 3.3 Real-film binding

The same runner drives the REAL film packages through the adapter of §2.6: `goldfinger`, `goldeneye`, `spectre`, and `no_time_to_die` are discovered from their own `src/` trees as frozen `bond_api` providers and executed as a mission. The canonical OPERATION OMNIBUS shape over real films runs in dependency order, every verdict passes, and `goldeneye`'s recon carries a citation of `goldfinger`'s outputs — the adapter's down-translation of the DAG edge (§2.6) survives the round trip into a package that has no DAG concept. Real films also exercise cycle rejection (a `goldfinger` ↔ `goldeneye` loop is caught with its ordered path) and the retry policy (a real film wrapped in a deterministic call-counting fault recovers within its attempt budget and resumes byte-identically). These are `tests/test_film_binding.py`; they are not fixtures standing in for films, they are the films.

That module carries no `skipif`. It previously skipped itself when the sibling film packages were absent, which meant this section's claim went green with no evidence behind it in any checkout lacking those packages. Absence is now a hard failure — `TestRealFilmEvidenceIsRequired` names this section in its failure message — so §3.3 is red whenever its evidence is missing. This was verified by repointing the module at an empty tree: the presence gate and the real-film binding tests went red together. (The count of tests that turn red is deliberately not quoted here — it drifts as the suite grows, and a stale figure would decay into a false claim about a control that was genuinely run.) A fork that cannot supply the film packages must delete this section, not silence the tests.

34.5.4 3.4 Provenance

The run emitted one provenance record per executed step and per debrief (`output/data/provenance/outcomes.json` + `provenance.csv`), every record carrying the deterministic run id, package, provider, phase, outcome, and numeric metrics.

34.5.5 3.5 Stage plan and retry outcome

OPERATION_OMNIBUS's wavefront decomposition (see §2.5) resolved into **3** parallel stages with an achievable parallelism of **1** and a critical path of **3** stages:

```
1: goldfinger | 2: goldeneye | 3: no_time_to_die
```

Because the chain is serial, every stage is single-package: no concurrency is available, and the critical path equals the package count — exactly the shape a serial chain must have. The diamond-shaped OPERATION DIAMOND is the contrasting case: its two dependency-free leaves share stage 1, so its achievable parallelism exceeds one while its critical path is shorter than its package count.

The run's retry ledger recorded **0** retried steps under the configured policy of **1** maximum attempts. A retry record exists only when a transient failure was actually re-executed (§2.4), so an empty ledger is evidence of a clean run rather than of a disabled mechanism — the mechanism itself is exercised under deliberate fault injection in §3.3.

34.5.6 3.6 The measured DAG (figure)

Figure 3.1 is a deterministic text figure assembled from the run's own JSON by `src/bond_orchestrator/manuscript_variables.py::dag_figure`: each package carries its wavefront stage (from `stage_plan`), its debrief verdict, and each declared hand-off edge is labelled with the upstream recon `outputs` that actually crossed it. It is the §2.1 DAG as *measured*, so the figure and the engine are the same object rather than two accounts of it.

```
measured DAG (from the live mission report)
stage 1 goldfinger verdict=pass
stage 2 goldeneye verdict=pass
stage 3 no_time_to_die verdict=pass
gold_reserves_t=4200, market_index=0.92 goldeneye
emp_shield_rating=0.87, uplink_redundancy=2 no_time_to_die
```

The stage 1 → 2 → 3 chain and the two non-empty edges mirror the `goldfinger` → `goldeneye` → `no_time_to_die` written in §3.1; the single-package-per-stage width confirms the serial shape §3.5 describes, and the labelled payloads are exactly the evidence §3.1 quoted in prose.

34.6 Conclusion — DAG 00: findings, verdict, and what the mission establishes

BOND-ORCHESTRATOR delivers its Layer 3 contract: multi-package missions run end-to-end over the `MissionProvider` protocol in deterministic topological order, with per-step checkpoint/resume and provenance on every outcome, and a single consolidated debrief per mission. The architecture keeps the protocol, the DAG machinery, and the runner free of infrastructure imports — pure logic that any BOND package can depend on — while the thin CLI and scripts remain glue.

The guarantees demonstrated for OPERATION OMNIBUS — order, resumability, byte-determinism, and full provenance — transfer to any mission expressed as a DAG of providers, and they are not confined to the reference providers: the adapter of §2.6 binds the

REAL film packages built to the frozen `bond_api.MissionProvider` protocol, and §3.3 runs them through the same runner with the same guarantees intact. The one integration point still declared rather than wired is provenance delegation to `bond-utilities.mission_io:src/bond_orchestrator/provenance.py` is today an independent, non-delegating record writer whose schema differs from `mission_io`'s, and the delegation mapping plus acceptance criteria are tracked in `TODO.md` and §7.4.

Two design choices carry the rest of the package. Retries were made *immediate* rather than backed off, because a wall-clock sleep would put a non-reproducible quantity inside an artifact that has to be byte-stable; bounded immediate re-attempts recover from a transient fault without giving that up. And parallel execution was deliberately not implemented: the stage planner of §2.5 emits the schedule a parallel executor would follow and proves the achievable concurrency, while the sequential runner remains the reference implementation of the same order. The plan is the artifact; the concurrency is a later, separately verifiable step.

34.7 Experimental Setup — DAG 00: canonical scenarios, parameters, and configuration

34.7.1 5.1 Identity and configuration

This manuscript was produced for **BOND-ORCHESTRATOR: A Deterministic Mission DAG Runner** (Multi-package film missions over the MissionProvider protocol, with cycle-path detection, checkpoint/resume, bounded deterministic retry, parallel-stage planning, real-film binding, and provenance). The package identity is **BOND-ORCHESTRATOR**, codename **DAG 00**, version **2.5.2**, first author Daniel Ari Friedman. All identity, publication, and mission parameters are declared in `manuscript/config.yaml` 1 and injected into this document through the manuscript-variable pipeline (`scripts/generate_manuscript_variables.py` → `src/bond_orchestrator/manuscript_variables.py`; syntax reference in `manuscript/SYNTAX.md`) — no metric is hand-authored in prose.

34.7.2 5.2 Mission parameters

Parameter	Value
Mission name	OPERATION_OMNIBUS
Package count	3
Package chain	goldfinger → goldeneye → no_time_to_die
Lifecycle phases	brief, recon, plan, execute, debrief
Retry max attempts (configured)	1

34.7.3 5.3 Software environment

- Python: the supported floor is `requires-python = ">=3.10"` (`pyproject.toml`). The interpreter each gate run actually used is recorded alongside the measured results in `docs/_generated/COUNTS.md` rather than asserted here, so this section cannot drift from the machine that ran it.
- `pytest + pytest-cov` for the enforced `>=90%` line+branch coverage gate on `src/` (`pyproject.toml: fail_under = 90, branch = true, source = ["src"]`)
- `ruff` (lint + format check) and `mypy` clean on `src/` and `scripts/`
- Runtime dependencies of the pure core: the standard library plus `pyyaml` (config parsing only). `bond-api` is a declared path dependency (a relative path to the sibling package), imported lazily and required only when real film packages are bound (§2.6). This sentence is enforced, not asserted: `tests/test_packaging.py` parses `pyproject.toml`, walks the import statements of `src/` and `scripts/` with the AST, and fails if a declared dependency is imported nowhere or an import is undeclared. Earlier revisions declared `numpy`, `matplotlib` and `defusedxml`, none of which this package imports; they were inherited from the exemplar it was forked from and have been removed.
- No external runtime services and no network access: the reference providers are in-process deterministic classes, and the real film packages are imported from their own `src/` trees.

34.7.4 5.4 Mission surface under test

Every number in this table is a token computed from the mission registry (`src/bond_orchestrator/missions.py`) through the same `build_stage_plan` the runner uses — see `src/bond_orchestrator/manuscript_variables.py::_mission_surface_token s`. Earlier revisions typed these counts by hand, contradicting §5.1 four lines above.

Mission	Shape	Packages	Hand-off edges	Stages	Max parallelism	Critical path
OPERATION_OMNIBUS	serial chain	3	2	3	1	3
OPERATION_DIAMOND	two leaves joining one sink	3	2	2	2	2
OPERATION_SINGLET	single package	1	0	1	1	1

- OPERATION_OMNIBUS is the flagship: a serial chain, so its critical path equals its package count and no concurrency is available.
- OPERATION_DIAMOND is the only shape whose achievable parallelism exceeds one; it exercises a multi-dependency join.
- OPERATION_SINGLET is the degenerate case, isolating retry and verdict behaviour from DAG effects.

The same three shapes are re-run over the REAL film packages through the adapter of §2.6, so the DAG results are not an artifact of the reference providers alone.

34.8 Reproducibility — DAG 00: verification gates, deterministic regeneration, and artifacts

34.8.1 6.1 Determinism policy

Determinism is a hard contract, not a preference:

- Providers are pure with respect to their inputs (fixed spec classes in `src/bond_orchestrator/providers.py`); there are no random draws and no wall-clock dependencies in any report artifact.
- Topological order is canonical (sorted ready-worklist), so execution order is declaration-order independent.
- Checkpoint step files, `manifest.json`, `report.json`, `outcomes.json`, and `provenance.csv` are written with `sort_keys=True` — byte-stable across runs for identical inputs. `tests/test_determinism_bytes.py` enforces this boundary at the byte level: it executes each of the three missions twice into two separate directories and compares every written file with `Path.read_bytes`, so key order, separators, indentation and trailing newlines are all in scope. It carries a positive control (`TestByteComparisonIsSensitive`) proving the comparison detects a one-byte difference, and injecting a wall-clock value into any persisted record turns it red. Earlier revisions compared `report.as_dict` — a Python `dict` equality that is blind to all of the above.
- Wall-clock timing is never part of a persisted artifact.

34.8.2 6.2 Verification commands

```
## Enforced gate: tests + 90% line+branch coverage on src/
uv run pytest tests/ --cov=src --cov-fail-under=90

## No mocks anywhere in tests/
grep -r "unittest.mock\|MagicMock\|@patch\|create_autospec" tests/ || echo "Clean"

## Packaging + lineage honesty gates (declared-vs-imported dependencies, a
## relative bond-api path, no forked-scaffold slug in any declared value)
uv run pytest tests/test_packaging.py -v

## Reproduce the flagship run + provenance
uv run python scripts/run_mission.py OPERATION_OMNIBUS --checkpoint-dir output/state

## Hydrate manuscript tokens (strict: requires the mission report above)
uv run python scripts/generate_manuscript_variables.py
```

The lineage check is a test, not a `rg` one-liner. The one-liner previously printed here never reported `Clean`: `rg` skips dotfiles by default, so it was blind to `.template-export.json` — a stale export manifest whose recorded `source_project` named the exemplar and whose 1,075 file hashes had decayed to 12 matches and 1,014 deleted paths. That file has been removed. The command also matched its own text in this manuscript and in `docs/AGENTS.md`, so it could not have printed `Clean` even on a genuinely clean tree.

What the replacement gate covers, stated so it is not read as more: it scans every `.py` file under `src/`, `scripts/` and `tests/` for the exemplar slug, and every scalar *value* of `pyproject.toml`, `manuscript/config.yaml`, `CITATION.cff`, `codemeta.json`, `.zenodo.json`, `domain_profile.yaml`, `experiment_plan.yaml` and `export.toml`. It does **not** scan Markdown prose or configuration comments, because this package’s own remediation notes name the fork source deliberately. Un-forked *narrative* text is therefore not gated and remains a review responsibility.

34.8.3 6.3 Artifact inventory

Artifact	Producer	Status
output/data/mission_report.json	scripts/run_mission.py	Regenerated
output/data/provenance/outcomes.json, provenance.csv	scripts/run_mission.py	Regenerated
output/state/<mission>/*.json	runner checkpoint layer	Regenerated
output/data/manuscript_variables.json	scripts/generate_manuscript_variables.py	Regenerated

For OPERATION_OMNIBUS (version 2.5.2, first author Daniel Ari Friedman) the all-verdicts outcome was **PASS**. Live test counts and measured coverage are tracked in `docs/_generated/COUNTS.md`.

34.9 Scope and Related Work — DAG 00: boundaries, positioning, and relationship to the literature

34.9.1 7.1 Scope

This package scopes to orchestration only:

- **In scope:** DAG construction and validation with ordered cycle-path detection, deterministic topological execution, checkpoint/resume, deterministic retry policy, parallel-stage planning, provenance recording, canned mission registry, real-film binding, thin CLI.
- **Out of scope:** film business logic (provider concern), mission *choice* (coordinator concern), protocol ownership (now `bond-api`'s — see §7.3), shared mission I/O once `bond-utilities.mission_io` is adopted (see `TODO.md`).

Parallel execution is deliberately deferred: the runner executes sequentially because determinism and resumability are the load-bearing guarantees. The stage planner (§2.5) makes the parallelism explicit without changing that contract — every package in a stage may run concurrently, and the sequential runner is the reference implementation of the same order.

34.9.2 7.2 Related work

Topological sorting of dependency graphs dates to Kahn's algorithm [Kahn, 1962] and is standard material in algorithms texts [Cormen et al., 2009d]. Long-running multi-stage workloads recover from interruption through checkpoint/rollback-recovery protocols, surveyed in [Elnozahy et al., 2002], with distributed snapshots for coordinated state capture [Chandy and Lamport, 1985]; the runner's manifest + per-step state files follow that pattern specialised for deterministic single-node execution. Scheduling directed task graphs onto parallel machines by level-by-level wavefront decomposition is a classic static-scheduling result [Kwok and Ahmad, 1999]; the orchestrator's stage plan is its deterministic, single-machine specialization. Bounded re-execution of a failing step is fault tolerance in the dependability sense — masking a transient fault so it does not become a system failure, while letting a persistent one surface [Avizienis et al., 2004]; here it is made deterministic by removing wall-clock backoff entirely, trading throughput for byte-reproducible artifacts. The binding between the runner-side contract and the frozen `bond_api` protocol is a textbook object adapter [Gamma et al., 1994]: neither interface may move, so a translating wrapper is the only structure that preserves both.

34.9.3 7.3 Integration points

Wired (phases 2–3): real film packages built to the frozen `bond-api` `MissionProvider` protocol are bound through the adapter in `src/bond_orchestrator/film_providers.py` (`bond-api` is a local path dependency, imported lazily so the pure core stays liftable without it).

Declared, not yet wired (each tracked with acceptance criteria in `TODO.md`):

- `bond-utilities.mission_io` provenance delegation — `provenance.py` is an independent, non-delegating provenance writer today, because its record schema differs from `mission_io`'s (key sets share only `codename` and `schema_version`). Delegation means writing a mapping and picking a canonical record shape first; see `TODO.md` V1/E6.
- `bond-coordinator` mission-spec handoff — coordinator-produced missions (same `Mission` shape) should execute unchanged.

34.9.4 7.4 Limitations and uncertainty

This section states, in one place, what the package does *not* claim. Each bullet is grounded in the current code and tracked where it is ongoing work.

- **Provenance is not yet `mission_io`.** `provenance.py` is an independent, deterministic writer; its records are the orchestrator's own schema, not `bond-utilities.mission_io`'s, and no test compares the two key sets. Until the delegation (TODO E6) and its schema mapping (TODO V1) exist, cross-package provenance is stored in this package's format, not the suite's shared one.
- **One interpreter is exercised.** Every gate this package has run used a single CPython (3.14.6, recorded in `docs/_generated/COUNTS.md`); the declared `requires-python = ">=3.10"` floor has not been run on 3.10–3.13 (TODO V2). The floor is a declaration, not a matrix.
- **Live counts are hand-maintained.** The measured test/coverage figures that §5.3 and §6 defer to live in `docs/_generated/COUNTS.md`, which no script regenerates (TODO V3). It is updated by hand on every pass and can silently go stale; the deferral chain is only as honest as that one leaf.
- **Resuming a changed mission requires a fresh checkpoint dir.** Checkpoint resume is bound to the exact mission definition by a recorded hash (TODO C1, §2.3): mutate a mission's `depends_on` or package set and the runner refuses to replay the old steps. This closes a stale-replay hole but makes an in-place, same-directory *edit-and-resume* workflow impossible by design — the safe workflow deletes or renames the state dir for a genuinely changed mission.
- **Real-film retry cannot rescue a deterministic film.** The adapter caches a film's `execute` outcome for the run (determinism safety), so a film that fails once yields the same failure on every bounded re-attempt (§2.6, §2.4). The retry mechanism therefore demonstrably recovers only *non-deterministic* or deliberately fault-injected providers — real films are exercised against it with a call-counting fault in `tests/test_film_binding.py`, not as a claim that it masks a persistent film bug.

- **Parallel execution is planned, not performed.** The stage plan (§2.5) is a scheduling artifact; the runner executes sequentially, so achievable concurrency is verified as a property, not delivered as speed. No parallel executor or wall-clock performance characterization exists (deliberately: wall-clock values would break byte-determinism).
- **Empty-plan packages pass vacuously.** A package whose provider returns an empty execution plan records zero execution steps and debriefs `pass` (there is nothing to fail). The runner now rejects an *empty mission* at graph build (§2.1) so the vacuum cannot hide a misbuilt mission, but a single empty-plan provider is still treated as a no-op success.
- **Determinism is single-node and single-interpreter.** Byte-reproducibility (§6.1) is demonstrated for fixed seeds and one interpreter on one machine; it does not by itself guarantee bit-identical output across Python versions or platforms, a question the single-interpreter gap above does not yet answer.

34.10 Sources — DAG 00: bibliography

[Kahn \[1962\]](#); [Cormen et al. \[2009d\]](#); [Elnozahy et al. \[2002\]](#); [Chandy and Lamport \[1985\]](#); [Kwok and Ahmad \[1999\]](#); [Avizienis et al. \[2004\]](#); [Gamma et al. \[1994\]](#); [Friedman \[2026g\]](#)

35 Bond CLI — THE FRONT DOOR

infra package · package codename THE FRONT DOOR.

35.1 Concepts — THE FRONT DOOR: domain and operational focus

Unified terminal front door (bond list/brief/.../run/status/gadgets)

35.2 Abstract — THE FRONT DOOR: mission summary

bond-cli (version 3.0.0, codename THE FRONT DOOR) is the Layer-4 terminal front door of the PROJECT BOND suite — a fleet of 27 film packages plus shared mission infrastructure, each film package implementing the frozen `MissionProvider` protocol. As the front door, bond-cli is a thin dispatch layer: it routes 12 verb-first commands — list, brief, recon, plan, execute, debrief, run, status, gadgets, missions, whois, roster-export — to the 4 built sibling layers (bond-utilities, bond-api, bond-coordinator, bond-orchestrator) and maps their outcomes to deterministic exit codes. It implements no mission logic and ships no `MissionProvider` of its own; every command either loads a real film provider through the frozen `bond-api` discovery contract or delegates to the coordinator’s registry and the orchestrator’s runner. The deepened surface adds machine-readable `--json` output on 4 commands (list, status, missions, whois), a JSON roster export, color-safe aligned tables that never emit ANSI, and reconciliation of the fleet’s measured gate manifest onto the registry so that reported status and coverage are measurements rather than intentions.

35.3 Introduction — THE FRONT DOOR: mission framing, the operational problem, and how to read this chapter

PROJECT BOND is a serious special-agent software fleet: 27 film packages, each faithful to a film’s central idea and implementing the frozen `MissionProvider` protocol, plus shared infrastructure — `bond-api` (the protocol + discovery + registries), `bond-coordinator` (mission control: registry, ledger, status, situation room), `bond-orchestrator` (the DAG mission runner with checkpoint/resume and provenance), and `bond-utilities` (Q-branch primitives and mission IO).

A fleet of that shape has an interface problem. Each film package is standalone by construction — its own repository, its own tests, its own manuscript — so an operator who wants to ask *what does this film do, is it built, does its mission pass* would otherwise need 27 entry points and as many local conventions. bond-cli is the answer to that problem: the one surface an operator talks to.

Its role, THE FRONT DOOR, accepts a verb and a target. `brief/recon/plan/execute/debrief` walk a single film through the protocol’s five stages; `run` executes a canned multi-package mission through the orchestrator; `list`, `whois`, and `roster-export` report the fleet from the coordinator registry; `status` renders the suite snapshot; `missions` enumerates what `run` can be given; and `gadgets` inspects the gadget payloads a film registers. The full command set is list, brief, recon, plan, execute, debrief, run, status, gadgets, missions, whois, roster-export.

Two design commitments shape everything that follows. First, **thinness**: argument shaping, provider loading, output formatting, and exit-code mapping live here, and nothing else does. The front door cannot drift from the layers it fronts because it implements none of their logic — it ships no `MissionProvider` and no mission mathematics. Second, **measurement over intention**: where the fleet has actually been gated, `status`, `whois`, and `roster-export` report the measured result rather than the registry’s declared state, and they say plainly when no measurement exists.

35.4 Methodology — THE FRONT DOOR: the analytical models and algorithms that drive the mission

35.4.1 Thin dispatch over built layers

`src/bond_cli/dispatch.py` builds an argparse surface whose 12 commands are:

list, brief, recon, plan, execute, debrief, run, status, gadgets, missions, whois, roster-export

That tuple, `COMMANDS`, is the single source of the command surface: it drives help ordering, the manuscript token above, and the ledger row that the test suite re-derives. Two sibling tuples do the same job for the rest of the surface — `JSON_COMMANDS` (list, status, missions, whois) names the commands accepting `--json`, and `DEPENDENCY_LAYERS` names the 4 built siblings (bond-utilities, bond-api, bond-coordinator, bond-orchestrator) in dependency order. No count in this manuscript is written by hand; each is derived from the tuple or registry it describes.

`load_film(slug)` uses the FROZEN `bond_api.discovery.load_provider` contract, searching the fleet directory (`projects/working/bo`) and bond-api’s reference `films/` directory, so a single code path serves both the 27 built films and the reference film. Only directories that exist are passed to discovery, because the contract validates every search path it is given. A missing or malformed film raises a `CliError`, which the CLI turns into exit code 1.

35.4.2 Per-command delegation

- **list** — reads the coordinator registry `ROSTER` and prints all 27 films in canonical order (rank, slug, title, year, codename).

- **brief/recon/plan/execute/debrief** — load the named film’s `MissionProvider` and call the corresponding protocol stage. `debrief` runs the full five-stage flow through `bond_api.protocol.run_mission`. `execute/debrief` exit 0 iff the outcome reports success.
- **run** — resolves a canned mission by name from `bond_orchestrator.missions` (there are 3: `OPERATION_DIAMOND`, `OPERATION_OMNIBUS`, `OPERATION_SINGLET`), binds each package id to its real film provider via `film_providers`. `load_film_providers` (falling back to the orchestrator’s built-in deterministic provider when a film is not built), executes it with the `MissionRunner`, prints the consolidated debrief, and exits 0 iff every verdict passes.
- **status** — reads the aggregate gate manifest written by the fleet gate runner (`output/aggregate/gate_manifest.json`) when present, reconciles each film’s measured status/coverage onto the coordinator registry, and renders the suite snapshot (counts by status, coverage summary, blocked packages). Without a manifest it falls back to the honest pre-measurement registry.
- **gadgets** — builds a `bond_api.GadgetRegistry` by discovering each film’s gadgets across every fleet convention (`register_gadgets(registry)`, zero-arg `register_gadgets`, module-level `GADGETS` dict or factory, a module-level `GadgetRegistry`, or provider-bound registration at construction), and prints the registered gadget names, for one film or the whole fleet. The three single-film outcomes are now distinguished: an **unknown slug** raises a `CliError` (stderr, exit 1) with the same “not available” wording as `brief/plan`; a **built film whose import or hook raises** is diagnosed to stderr (naming the exception) while still contributing 0 so a whole-fleet sweep never aborts; and a **clean film with no gadgets** keeps the empty-lookup message. All failure text goes to `stderr`, so a caller piping stdout into a parser never receives error text as data.
- **missions** — lists the orchestrator’s 3 canned missions and each mission’s package chain.
- **whois <film>** — prints a film’s registry identity (rank, film, year, codename, status, coverage, concepts), whether its provider module is built on disk, and its measured gate state when the manifest carries a row for it.
- **roster-export** — exports the fleet roster as key-sorted JSON (the measured roster when the manifest exists, else the plain registry), to stdout or to a file with `--output`.

35.4.3 Reconciling measurement onto the registry

The registry records what each film package *declares*; the aggregate gate manifest records what the fleet gate *measured*. Three functions keep the two apart and then combine them honestly.

`_manifest_rows` reads the manifest into rows keyed by slug and returns nothing at all when the file is absent, unreadable, or malformed — a broken manifest degrades the front door to registry-only reporting rather than failing a command. `_measured_coverage(row)` extracts a coverage percentage, treating a measured 0.0 as a real measurement (only a missing or non-numeric value is unknown; booleans are rejected, since `bool` subclasses `int` in Python). `_reconcile(record, row)` overlays one row onto one registry record, marking the package ready iff its gate returned 0 and red otherwise.

Composing these over the roster gives the measured view. A film with no row keeps its registry record untouched, because a manifest may cover only part of the fleet — and `whois` reports a measurement for such a film as `null` rather than echoing the registry’s declared status as though it had been observed. The distinction matters: a front door that reports intentions as measurements is worse than one that reports nothing.

35.4.4 JSON output and color-safe tables

`list`, `status`, `missions`, `whois` accept `--json` for machine-readable output; `roster-export` is JSON by construction and therefore takes no flag. All JSON is emitted with sorted keys and contains no wall-clock field, so two runs against an unchanged fleet are byte-identical.

Human tables are rendered by `src/bond_cli/table.py::format_table` with fixed-width column padding only — never ANSI escape codes. That is the color-safe contract: identical bytes in a terminal, a pipe, a log file, and test capture, and no escape sequences leaking into anything a downstream tool parses. Widths derive from the header and cell text; no cell is truncated or wrapped, so no information is lost to formatting.

35.4.5 Determinism and exit codes

Film providers are fixed-seed and side-effect free, output ordering is canonical (roster order, sorted result keys), and no command persists wall-clock data. Exit codes: 0 success, 1 command failure (unknown film, non-passing `execute/debrief/run`, empty or broken gadget lookup), 2 argparse misuse and unknown mission name. `main` catches only its own `CliError`; any other exception is a genuine defect and propagates rather than being flattened into a misleading exit code.

Gadget-hook arity is read from `inspect.signature` [Python Software Foundation, b] rather than guessed by catching `TypeError` on a registry call. That closes a subtle failure mode: a `register_gadgets(registry)` hook whose *body* raised `TypeError` was previously re-invoked as a zero-arg hook, escaping a second, misleading `TypeError: missing 1 required positional argument` that aborted the whole-fleet sweep and masked the film’s real defect. With arity from the signature, a hook-body exception is caught per-film, diagnosed to stderr, and the sweep continues.

35.5 Results — THE FRONT DOOR: measured outcomes, headline numbers, and what they establish

35.5.1 The delivered surface

`bond-cli` exposes 12 commands over the 4 built layers, all produced by real delegation:

- **bond list** prints the full 27-film roster from the coordinator registry, in canonical order (`--json` for rows).
- **bond brief <film>** / **bond recon <film>** / **bond plan <film>** / **bond execute <film>** / **bond debrief <film>** exercise a real film provider end-to-end: the brief’s identity, the recon’s findings, the plan’s steps, the executed outcome with provenance, and the debrief’s lessons.
- **bond run <mission>** executes any of the 3 canned orchestrator missions (OPERATION_DIAMOND, OPERATION_OMNIBUS, OPERATION_SINGLET) over the real film providers, with deterministic fallback for films not yet built, and prints the consolidated debrief; exit 0 iff all verdicts pass.
- **bond status** reports the suite snapshot — counts by status, the coverage summary, blocked packages — reconciled against the measured gate manifest when present (`--json` for a key-sorted snapshot).
- **bond gadgets** lists which films register gadgets and the gadget names each exposes, across every fleet registration convention.
- **bond missions** lists the canned missions and their package chains (`--json` for rows).
- **bond whois <film>** prints one film’s registry identity, built-on-disk state, and measured gate state (`--json` for a payload).
- **bond roster-export** emits the fleet roster as key-sorted JSON (stdout or `--output FILE`).

Of these, 4 accept `--json` (list, status, missions, whois); **roster-export** is JSON by construction.

35.5.2 Measurement is distinguishable from declaration

The reconciliation results are worth stating separately, because they are the front door’s only non-trivial computation. Against a full manifest, every film reports its gate verdict and measured coverage. Against a *partial* manifest, films with rows report measurements and films without rows report **measured: null** while retaining their declared registry status — the two are never conflated. Against a malformed or absent manifest, every command still succeeds and reports the registry view, flagged **measured: false**; the human status prints **min=N/A max=N/A mean=N/A** rather than a literal **None%**. A gate row of 0.0 coverage survives as 0.0, not as “unknown”: a package measured at zero has been measured.

35.5.3 Determinism

Because every command delegates to a frozen, deterministic layer and prints canonical output — roster order, sorted JSON keys, no wall-clock field anywhere — repeated invocations against an unchanged fleet are byte-identical. That property is a live determinism check on the layers beneath: if a film provider ever became seed-dependent, the front door’s own output would stop reproducing.

35.5.4 Verification

The suite is zero-mock: real providers, real importable film packages written to temporary directories, real gate manifests, and real subprocess invocations of the `scripts/` entry points. The live test count and the measured `src/` coverage percentage are recorded in `docs/_generated/COUNTS.md` and re-measured by the gate `uv run pytest tests/ --cov=src --cov-fail-under=90`; this manuscript deliberately quotes neither figure, so no prose here can go stale against the measurement. Every count it does quote — commands, JSON commands, layers, films, missions — is injected from the registry that defines it, and `data/claim_ledger.yaml` records the same values with their sources, re-derived on every test run.

Two of the claims above are not established by coverage and are therefore pinned by dedicated mutation-checked tests. The exit-code contract (0 iff the underlying execute/debrief/run passed) is implemented as a conditional expression, for which `coverage.py` emits no branch arc: replacing all three with an unconditional **return 0** left the entire suite green, because no test drove a *failing* mission. It is now driven by real film packages whose **execute** genuinely reports failure, each paired with a passing positive control. The sorted-JSON-keys contract was similarly invisible, since every assertion parsed the output back into a dictionary and discarded key order; it is now asserted against the raw emitted text with each handler’s literal construction order pinned, so the assertion cannot pass by coincidence. Both tests were validated by applying the corresponding mutation to `dispatch.py` and confirming they go red; the mutation table is recorded alongside the measurement in `docs/_generated/COUNTS.md`.

A third class of claim is bound by *repeatability* rather than coverage: the determinism guarantees in `06_reproducibility.md`. **satus --json** and **roster-export** are invoked twice in separate subprocesses and asserted byte-identical, so a `generated_at: time.time` field — executed, fully covered, and invisible to coverage — turns the suite read as a determinism regression. The installed `bond` console script (declared in `pyproject.toml`) is exercised in a subprocess and its stdout/exit code asserted equal to the `scripts/bond.py` bootstrap, so a mistyped entry-point target in `pyproject.toml` fails the suite. And the three mechanically checkable `cross_cutting` rules in `manuscript/layer_contract.yaml` — no `infrastructure.*` imports, no wall-clock calls, no terminal-width/environment reads — are asserted against the `src/` and `scripts/` trees, so adding `import time` to `dispatch.py` fails the suite instead of silently falsifying the manuscript.

35.5.5 The derived surface (bound to the live token map)

The counts this front door reports are never hand-typed into prose. Every one is produced from the registry or tuple that owns it by `src/bond_cli/manuscript_variables.py`, persisted to `output/variables/manuscript_variables.json` by `scripts/z_generate_manuscript_variables.py`, and re-derived on every test run:

Quantity	Live value
Films in the coordinator roster	27
Front-door subcommands	12
Subcommands accepting <code>--json</code>	4
Built sibling layers delegated to	4
Canned orchestrator missions	3

Each placeholder in the table above is regenerated from the live coordinator `ROSTER`, the orchestrator `MISSION_NAMES`, and `dispatch.COMMANDS / JSON_COMMANDS / DEPENDENCY_LAYERS`; the values cannot drift from the engines they count. `data/claim_ledger.yaml` records the same five quantities against the same five source symbols, and `tests/test_hardening.py::test_claim_ledger_matches_its_sources` re-derives them.

35.5.6 Limitations and uncertainty

- **Coverage proves execution, not verification.** A conditional expression (`return 0 if ok else 1`) emits no branch arc, so high line+branch coverage can coexist with a completely unverified non-zero path — that is exactly how both headline contracts were once fully “covered” and entirely unbound. The mitigation is zero-mock contract tests that drive the *failing* path and are mutation-validated (see above), not the coverage number itself.
- **Gadget discovery is best-effort by contract.** A film that fails to import, or whose hook body raises, contributes 0 and emits a diagnostic to `stderr` — the whole-fleet `bond gadgets` sweep must never be aborted by one broken film. The cost is that gadget coverage for the fleet is only as healthy as the individual films’ hooks; the front door reports the exception, it does not repair the film. This package does not install a film’s scientific dependencies (e.g. `die_another_day` needs `scipy` and is not importable here), which is correct — the front door depends only on the four built layers — even though it means one film realistically shows as gadget-broken in this environment.
- **format_table assumes rectangular input.** A row shorter than the header raises `IndexError`; a row longer is silently truncated to the header width. Every call site builds matched-width rows by construction and no public surface accepts user-supplied rows, so this is an unvalidated internal precondition rather than a reachable defect — but it would be the right place to add a guard if the front door ever grew a free-format command.
- **The measured gate is a snapshot, not a promise.** `status`, `whois`, and `roster-export` report the aggregate gate manifest as last written by the fleet runner. A stale or absent manifest degrades to the declared registry view, flagged `measured: false`; the front door reports the manifest honestly and does not try to re-measure the fleet itself.
- **Determinism is inherited, not re-derived.** The byte-identical repeat runs pass because the layers beneath are deterministic and the front door introduces no randomness, wall-clock, or unordered output. If a film provider ever became seed-dependent, the front door’s output would stop reproducing and the determinism tests above would turn red — but they would flag the *symptom*, not pinpoint the film.

35.6 Conclusion — THE FRONT DOOR: findings, verdict, and what the mission establishes

`bond-cli` gives the operator of the PROJECT BOND fleet a single, reliable terminal: 12 verbs in one grammar over the 4 built layers, real film providers loaded through the FROZEN contract, orchestrator missions run to a consolidated debrief, and a status surface that reports what the fleet gate actually measured. Its guarantees are the ones a front door must provide — thinness (no duplicated logic, no `MissionProvider` of its own), correctness through delegation, honesty about what has and has not been measured, and determinism (canonical output, sorted JSON keys, fixed seeds, no wall-clock state).

The design bet is that a front door earns its keep by being boring. Every capability stays where it was built; nothing is reimplemented one layer up for convenience. What `bond-cli` adds is uniformity — one grammar, one exit-code contract, one output format that pipes cleanly — and the discipline that keeps that uniformity true: every number it reports is derived from the registry that owns it, and every claim it makes is exercised against a real provider rather than a stand-in.

35.7 Experimental Setup — THE FRONT DOOR: canonical scenarios, parameters, and configuration

- **Runtime.** Python `>=3.10`; the 4 built layers (`bond-utilities`, `bond-api`, `bond-coordinator`, `bond-orchestrator`) wired via `[tool.uv.sources]` path sources, plus NumPy (film providers load it at import time) and PyYAML (manuscript config).
- **Entry points.** Console script `bond` (`bond = "bond_cli.dispatch:main"`) and the thin `scripts/bond.py` bootstrap — identical dispatch, both returning the same exit codes. Both are exercised: the console script by a subprocess smoke test that asserts its `stdout`/exit code equal the bootstrap’s, the bootstrap by the `test_scripts_smoke.py` suite.

- **Film discovery.** `bond_api.discovery.load_provider` over `<slug>/src` (fleet layout) and `bond-api/films` (reference layout), FROZEN contract; only existing directories are offered to it.
- **Mission execution.** `bond_orchestrator.runner.MissionRunner` over real film adapters, with the orchestrator’s built-in deterministic provider as fallback for films not yet built on disk.
- **Measured fleet state.** The aggregate gate manifest at `<bond root>/output/aggregate/gate_manifest.json`, written by the fleet gate runner. Absent, unreadable, or malformed $\Rightarrow$ registry-only reporting; the tests pin this path at a temporary file so no result depends on whether the workspace currently holds a manifest.
- **Output layer.** `src/bond_cli/table.py::format_table` for color-safe aligned tables (no ANSI); JSON output follows RFC 8259 [Bray, 2017], is key-sorted, and never embeds wall-clock values.
- **Token hydration.** `scripts/z_generate_manuscript_variables.py` $\rightarrow$ `src/bond_cli/manuscript_variables.py` $\rightarrow$ `output/variables/manuscript_variables.json`; identity from `manuscript/config.yaml`, counts from the live registries.
- **Toolchain** (dev extras): `pytest`, `pytest-cov`, `ruff`, `mypy`, `types-PyYAML`.
- **Gates.** `uv run pytest tests/ --cov=src --cov-fail-under=90` (line + branch), `uv run ruff check src/ scripts/ tests/`, `uv run ruff format --check src/ scripts/ tests/`, `uv run mypy src/ scripts/`. Measured results are recorded in `docs/_generated/COUNTS.md`, not in this prose.
- **Tests.** Real subprocess CLI smoke against the real packages plus in-process dispatch tests against real deterministic film providers and real temporary film packages; zero mocks.

35.8 Reproducibility — THE FRONT DOOR: verification gates, deterministic regeneration, and artifacts

`bond-cli` inherits the fleet’s determinism guarantees and adds a few of its own:

- **Delegation, not duplication.** Every command calls a built layer; there is no second implementation to drift from the source of truth.
- **Frozen discovery.** Films are located through the FROZEN `bond_api` contract; output uses canonical ordering (roster order, sorted JSON keys per RFC 8259 [Bray, 2017]).
- **Fixed seeds.** Film providers are deterministic; `bond run` reproduces an identical consolidated debrief for an unchanged fleet.
- **No wall-clock in persisted artifacts.** The CLI writes nothing time-based to files, the `--json` snapshots carry no `generated_at` field (a test asserts its absence), and the manuscript token map embeds no timestamps.
- **No hand-authored numbers.** Every quantity in this manuscript is an injected token generated by `src/bond_cli/manuscript_variables.py` from the registry that defines it — `COMMANDS`, `JSON_COMMANDS`, and `DEPENDENCY_LAYERS` in `dispatch`, the coordinator `ROSTER`, and the orchestrator’s mission registry. `data/claim_ledger.yaml` records the same values alongside the symbol each is derived from, and the test suite re-derives every ledger row from that symbol, so a stale number fails the gate instead of shipping. Measured evidence that changes with each run — test count and coverage % — is written only to `docs/_generated/COUNTS.md`.
- **Environment-independent tests.** The fleet’s aggregate gate manifest lives outside this package and appears or disappears as the fleet gate runs; every test that depends on it pins `dispatch.GATE_MANIFEST` at a temporary file, so the suite’s result and its branch coverage do not move with the workspace’s state.
- **Tests as evidence.** The suite runs real subprocess invocations against the real packages and asserts byte-stable command output wherever the layers beneath are deterministic. In particular, `status --json` and `roster-export` are each invoked twice in separate subprocesses and asserted byte-identical, binding the repeatability claim to a test rather than to inspection; and the installed `bond` console script is exercised in a subprocess and its stdout/exit code asserted equal to the `scripts/bond.py` bootstrap, so the declared `pyproject.toml` entry point cannot silently rot.

To reproduce the whole surface from a clean checkout:

```
uv sync --extra dev
uv run pytest tests/ --cov=src --cov-fail-under=90
uv run python scripts/z_generate_manuscript_variables.py
uv run python scripts/bond.py status --json
```

35.9 Scope and Related Work — THE FRONT DOOR: boundaries, positioning, and relationship to the literature

35.9.1 Scope

`bond-cli` covers the **front-door** surface only: routing verbs to films, missions, status, gadgets, and roster export. It does **not** implement the `MissionProvider` protocol (`bond-api`), the mission DAG and checkpoint machinery (`bond-orchestrator`), the registry/ledger/situation room (`bond-coordinator`), or Q-branch primitives (`bond-utilities`). A film is never executed outside its own frozen provider; a mission is never orchestrated outside the runner. The one computation the front door does own — overlaying the measured gate manifest onto the registry — is presentational: it changes what is reported, never what is executed.

Explicitly out of scope: interactive or TUI modes, persistent CLI state, network access, credential handling, and any writing to another package’s tree. The only file the CLI writes is the roster export the operator asks for with `--output`, plus its own manuscript

token map.

35.9.2 Relationship to the built layers

- **bond-api (THE PROTOCOL)** — the FROZEN `MissionProvider` contract + discovery + `GadgetRegistry` that the front door consumes for `brief/recon/plan/execute/debrief` and `gadgets`.
- **bond-coordinator (MISSION CONTROL)** — source of the 27-film `ROSTER` for `list/whois/roster-export` and of the `build_status` snapshot for `status`.
- **bond-orchestrator (DAG 00)** — source of the 3 canned missions and the `MissionRunner` for `run`.
- **bond-utilities (Q-BRANCH)** — transitively loaded by film providers; provenance is recorded by the orchestrator and merely displayed here, never recomputed.

35.9.3 Related work

The design is conventional by intent. The verb-first grammar and one-verb-per-subparser structure follow the standard-library `argparse` sub-command model [Bethard, 2009, Python Software Foundation, a]; the exit-code discipline (0 success, non-zero failure, distinct code for usage error) and the commitment to plain, pipeable output follow long-standing Unix utility conventions [IEE, 2018, Raymond, 2003]. Machine-readable output is JSON per RFC 8259 [Bray, 2017], key-sorted for byte-stability. The package version follows Semantic Versioning [Preston-Werner].

The color-safe table is the one place the front door deviates from common practice: many modern CLIs auto-detect a TTY and emit ANSI colour. This one never does. Uniform bytes across a terminal, a pipe, a log file, and test capture are worth more to a fleet whose output is read by other programs than colour is to a human reader — and a table that renders identically everywhere is a table whose tests mean something.

35.9.4 Related surface within the suite

`bond list` and `bond run` overlap conceptually with the orchestrator’s own `list/run` CLI; the front door re-exposes them under one grammar rather than relocating the logic. Where the two disagree, the orchestrator is authoritative — it holds the implementation.

35.10 Sources — THE FRONT DOOR: bibliography

Bethard [2009]; Python Software Foundation [a]; Bray [2017]; IEE [2018]; Raymond [2003]; Preston-Werner; Python Software Foundation [b]

36 Bond Operations — QUARTERMASTER

infra package · package codename QUARTERMASTER.

36.1 Concepts — QUARTERMASTER: domain and operational focus

Fleet inventory, gates, health, provisioning

36.2 Abstract — QUARTERMASTER: mission summary

PROJECT BOND is a **33**-package research suite: **6** infrastructure packages and **27** film packages. This manuscript documents **bond-ops** (*QUARTERMASTER*), the Layer-4 fleet-operations package that keeps the whole fleet measurable and gateable. The *QUARTERMASTER* owns the canonical suite roster as data ([Definition 1](#)), reconciles it against live suite directories, runs per-package quality gates (one package per subprocess — [Proposition 1](#)), reports per-repo git health, and emits the herdr provisioning roster. Fleet gate results are runtime diagnostics — never hand-authored metrics. The methodology section defines the gates, probes, and formalisms; the results section shows the measured fleet surface; the reproducibility section pins the determinism properties.

36.3 Introduction — QUARTERMASTER: mission framing, the operational problem, and how to read this chapter

The PROJECT BOND suite (**PROJECT BOND**) comprises **33** packages: **6** infrastructure packages forming the operational core (protocol, coordinator, orchestrator, CLI, ops, utilities) and **27** film packages implementing their respective mission providers. The roster composition figure shows the composition.

Definition 1 (Suite roster). The *suite roster* is the frozen, ordered set of the **33** package slugs — **6** infrastructure and **27** films — carried as data in `src/bond_ops/inventory.py` and reconciled against live directories by `inventory.reconcile`.

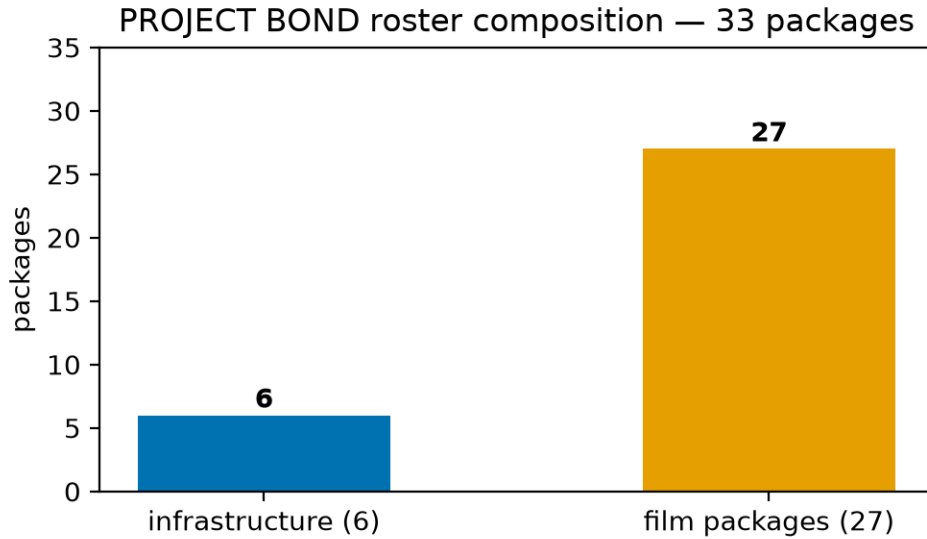


Figure 124: Roster composition of PROJECT BOND: 6 infrastructure packages and 27 film packages, totalling 33 packages.

The *QUARTERMASTER*'s premise: a fleet this large can only be trusted if it is inventoried, gated, and provisioned from a single, deterministic vantage point. **bond-ops** fills exactly that Layer-4 role; the machinery is defined in the methodology section.

36.4 Methodology — QUARTERMASTER: the analytical models and algorithms that drive the mission

36.4.1 Inventory

`src/bond_ops/inventory.py` carries the canonical roster as data ([Definition 1](#)) and reconciles it against the live suite root, flagging missing or unexpected packages ([Definition 2](#)).

Definition 2 (Reconciliation). `inventory.reconcile(root)` partitions the roster into **present** (slug directory exists under `root`), **missing** (roster slug absent), and **unexpected** (non-hidden directories on disk that are not roster slugs). The fleet is *complete* exactly when the missing set is empty (the reconcile verdict equation).

$$\text{complete}(r) \iff \text{missing}(r) = \emptyset$$

Beyond presence, the roster is also checked for *identity drift*: every present film package’s declared mission codename — read as data from its own `mission.py` / config, never by importing a sibling — must equal the matching 27-row roster codename. A film that renames itself (or a roster row that drifts) fails loudly rather than silently, so the QUARTERMASTER’s copy of fleet identity cannot rot unseen.

36.4.2 Aggregate gate

Definition 3 (Aggregate gate). The *aggregate gate* runs the canonical per-package gate — `uv run pytest tests/ --cov=src --cov-fail-under=90` — **once per package in its own subprocess**, plus ruff and mypy, and aggregates the verdicts into the machine-readable `bond-ops-gate` manifest.

Proposition 1 (One package per subprocess). Gating multiple packages in a single pytest process is forbidden: scaffold `conftest.py` files mutate `sys.path` and collide. Each package’s gate therefore runs with `cwd` set to that package’s own directory, one subprocess per package.

Coverage is parsed from either the pytest-cov TOTAL table or the ... **not reached. Total coverage: X%** failure line. The manifest summarises measured coverage across the fleet with the mean and extrema of the per-package values (the coverage mean equation, the coverage extrema equation).

$$\mu = \frac{1}{n} \sum_{i=1}^n c_i$$

$$\min_i c_i \leq c_i \leq \max_i c_i$$

A package’s gate verdict is the conjunction of the coverage gate and, when run, the ruff and mypy checks (the gate verdict equation):

$$G(p) = \text{pytest}(p) \wedge \text{ruff}(p) \wedge \text{mypy}(p)$$

The toolchain is standard and pinned via `uv.lock`: [Krekel et al.] drives every package’s tests, [Batchelder and contributors] measures line+branch coverage, [Marsh and Astral Software Inc.] lints and formats, and [Lehtosalo and contributors] checks types. The gate coverage figure and the gate verdicts figure visualise a real gate run; the toolchain table lists the tools.

36.4.3 Health

Definition 4 (Health verdict). A package is *healthy* when it is present, ships all 4 required files (README / AGENTS / TODO / pyproject), is free of lineage hits, and has a clean git tree (the health verdict equation).

$$H(p) = \text{present}(p) \wedge \text{files}(p) \wedge \neg \text{lineage}(p) \wedge \text{clean}(p)$$

Git state is read from `git status --porcelain` and reduced by a pure parser (`health.parse_porcelain`) into modified and untracked counts; the clean predicate is exactly “both counts are zero”. Branch, short SHA, and ahead/behind-upstream distances are recorded alongside, with ahead/behind reported as *undefined* rather than zero when no upstream is configured — the normal case in this local-only suite.

Proposition 2 (Probes are additive, not verdict-changing). The layout probes (`health.probe_layout`: [project] name equal to the slug, `tests/` present, `src/` present) and the toolchain probe (`health.tool_versions`) are reported alongside the health verdict but are deliberately **not** folded into $H(p)$ of the health verdict equation. A package whose pyproject name has drifted from its slug is a silent fork hazard worth surfacing, but folding a new condition into the verdict would retroactively change the contract that downstream consumers of `ops health --json` read. Diagnostics widen; verdicts do not move.

36.4.4 Command surface

Definition 5 (The ‘ops’ surface). `bond_ops.cli` exposes 4 subcommands — `inventory`, `gate`, `health`, `roster` — each with a human-readable form and a `--json` form emitting a versioned manifest. The gate overrides `--pytest-cmd` / `--ruff-cmd` / `--mypy-cmd` each accept a single shell-tokenized command (parsed with `shlex.split`), so a value containing `--`-prefixed arguments — e.g. `uv run pytest tests/ -q` — is usable; a test drives one through the CLI and asserts the `--`-prefixed token reached the subprocess. Exit codes: 0 on success, 1 when a `gate` verdict fails, 2 on usage error (unknown slug, negative `--limit`). Every default is read-only; only `roster --execute` starts anything.

36.4.5 Visualization

`bond_ops.figures` renders the manuscript figures headlessly (Agg backend, no display) from canonical data and real reports: the roster composition figure from the frozen roster constants, the fleet health figure from a real health sweep, and the gate coverage figure / the gate verdicts figure from a real gate report. Colours come from a colourblind-safe palette [Wong, 2011] and the plotting is done with Matplotlib [Hunter, 2007]. Because the inputs are fixed and no wall-clock or random draw enters the render path, the PNG bytes are reproducible for a given fleet state.

36.4.6 Provisioning

Definition 6 (Provisioning roster). The *provisioning roster* maps each package to a herdr provisioning recipe — `workspace create` (capturing the root pane id), `agent start` in that pane with a 90000 ms interactivity timeout, and a fire-and-forget `agent prompt` when a mission brief file exists. Rendering is pure and read-only; execution requires an explicit `--execute`.

`provision.roster_report` serialises the same roster as a versioned JSON manifest — totals by kind and by prompt-brief coverage, then one record per package — so fleet composition can be diffed mechanically rather than read out of a shell script.

See the results section for the measured fleet surface and the setup section for the experimental setup.

36.5 Results — QUARTERMASTER: measured outcomes, headline numbers, and what they establish

The fleet surface produced by `bond_ops` (codename QUARTERMASTER) is the 4-subcommand `ops` surface of Definition 5, all measured from real subprocess and filesystem state:

- **Inventory.** `ops inventory` reconciles the canonical roster (Definition 1) against `projects/working/bond/` and reports present, missing, and unexpected directories. A complete fleet shows `present 33/33 · missing 0 · complete True` (the reconcile verdict equation); the verdict is rendered as the Python boolean it is, and a test binds this quoted line to the CLI’s real output.
- **Fleet identity.** Every present film package’s declared mission codename is cross-checked against the roster as data; the coherence test fails loudly on a rename or drift, so the QUARTERMASTER’s copy of the 27-film roster matches what the fleet actually declares.
- **Aggregate gate.** `ops gate --json` emits a deterministic `bond-ops-gate` manifest (Definition 3) containing, for each package, its presence, pytest pass/fail, measured coverage, ruff and mypy verdicts, and the summary totals with coverage min/mean/max over the 90-floor gate (the coverage mean equation, the coverage extrema equation, the gate verdict equation). `ops gate` (human form) prints a per-package table; the exit code is 0 iff every present package passes.
- **Fleet health.** `ops health` reports per-repo clean/dirty, branch, ahead/behind, required-file coverage, lineage hits, layout probes, and the overall verdict (Definition 4); `--json` exposes `probes` and `layout_ok` per package as diagnostics that never move the verdict (Proposition 2). The fleet health figure shows a real sweep over the whole fleet.
- **Provisioning roster.** `ops roster` prints the herdr provisioning script; `ops roster --json` emits a machine-readable roster report (Definition 6); `ops roster --execute` runs it against the herdr daemon.

The gate manifest schema (version 1) is summarised in the gate manifest table.

Table 4: The `bond-ops-gate` manifest schema, version 1.

Field	Type	Meaning
<code>manifest</code>	string	manifest identifier (<code>bond-ops-gate</code>)
<code>schema_version</code>	integer	schema version (1)
<code>generated_by</code>	string	fleet identity (QUARTERMASTER)
<code>suite</code>	string	suite name (PROJECT BOND)
<code>summary</code>	object	totals (present / passed / failed / missing) + coverage measured / mean / min / max
<code>packages</code>	array	per-package verdict objects (presence, pytest / ruff / mypy, coverage)

Because the manifest carries no wall-clock timestamp and asks only real processes for their verdicts, re-running `ops gate` against an unchanged fleet yields an identical manifest — a live determinism check (Proposition 3).

36.6 Conclusion — QUARTERMASTER: findings, verdict, and what the mission establishes

`bond_ops` (*QUARTERMASTER*) delivers the fleet-operations surface of PROJECT BOND: a canonical roster (Definition 1), aggregate gates that respect one-package-per-subprocess isolation (Proposition 1), per-repo health inspection (Definition 4), and a safe, read-only provisioning path (Definition 6). Together these make a 33- package suite measurable and maintainable from a single QUARTERMASTER vantage point.

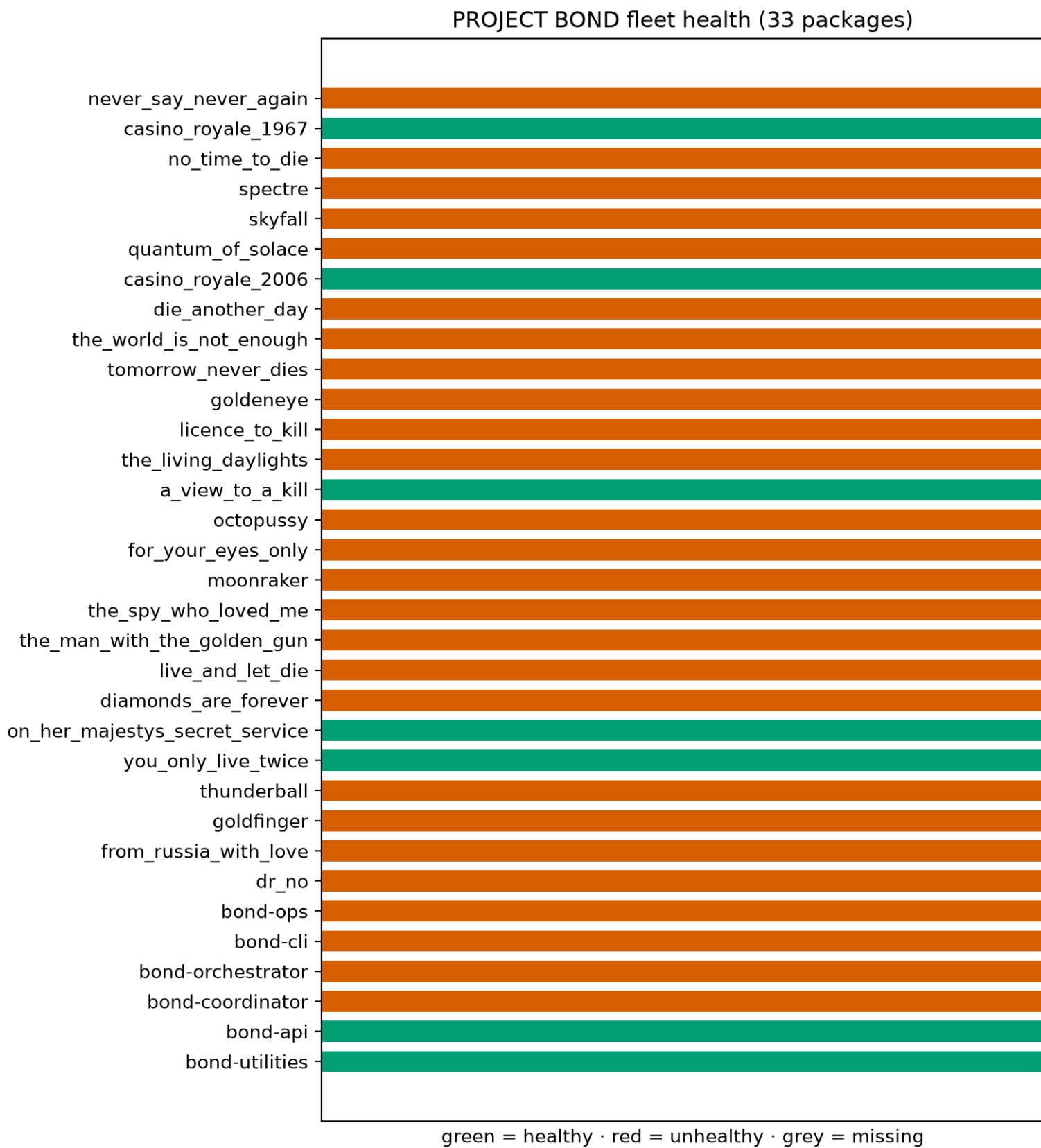


Figure 125: Fleet health across the whole roster: one row per package, green = healthy, red = unhealthy, grey = missing.

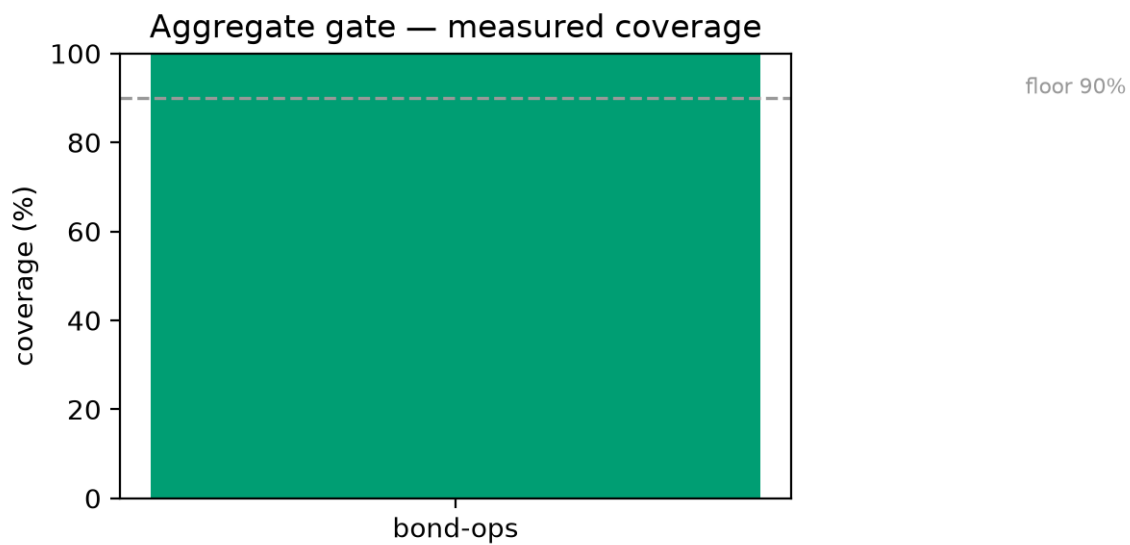


Figure 126: Measured gate coverage with the 90% floor reference line.

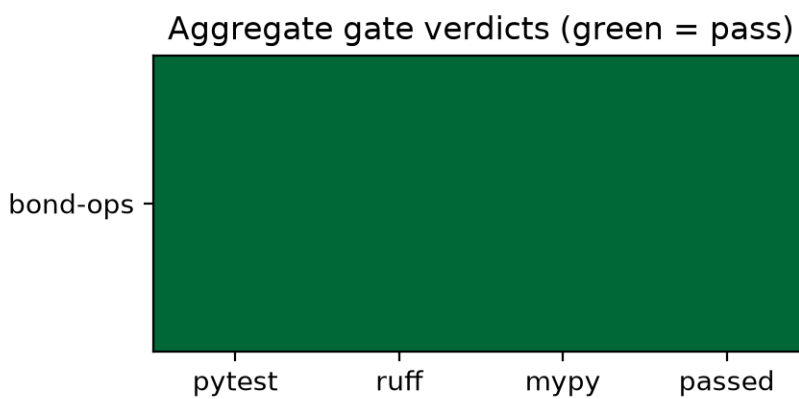


Figure 127: Aggregate gate verdict heatmap (pytest / ruff / mypy / passed).

36.7 Experimental Setup — QUARTERMASTER: canonical scenarios, parameters, and configuration

- **Runtime.** Python ≥ 3.12 (the lowest exercised floor; the toolchain is standard library + subprocess + pyyaml + matplotlib (figures only). No `infrastructure.*` and no sibling BOND import.
- **Gate command** invoked per package (one subprocess each): `uv run pytest tests/ --cov=src --cov-fail-under=90`, killed after 600 s so a hung package cannot stall the sweep (the timeout is recorded as a gate failure with its exception text, never as a pass).
- **Fleet root:** PROJECT BOND at `projects/working/bond/`, 33 packages (6 infra, 27 films).
- **Isolation contract:** each gate subprocess runs with `cwd` equal to the package’s own directory; never all packages in one pytest process ([Proposition 1](#)).
- **Figures:** rendered headlessly (Agg) into `../figures/` by `scripts/z_generate_figures.py`, byte-deterministic for fixed inputs.

The toolchain is pinned via `uv.lock` (the toolchain table).

Table 5: The QUARTERMASTER toolchain.

Tool	Role
pytest	test framework for every fleet package gate
pytest-cov	line+branch coverage at the 90% floor
ruff	lint + format in the aggregate gate
mypy	static type check in the aggregate gate
uv	environment + lockfile management

The package’s own test suite exercises the real subprocess machinery against deterministic tmp-path fake packages (and a fake `uv/herdr` on `PATH`), with zero mocks, and must hold $\geq 90\%$ line+branch coverage on `src/` (enforced by `--cov-fail-under=90` and the pyproject `fail_under`). A dedicated real-fleet test module additionally runs the gate runner and the health/layout probes against the live sibling packages under `projects/working/bond/`, giving the QUARTERMASTER an end-to-end gate it can trust on the fleet it operates.

36.8 Reproducibility — QUARTERMASTER: verification gates, deterministic regeneration, and artifacts

Proposition 3 (Manifest determinism). Gate manifests and manuscript variable maps embed no wall-clock timestamp, so for an unchanged fleet they are byte-identical across runs. The one place a clock reading can enter is the free-text **error** excerpt captured from a failing package’s own test output (pytest ends its summary with `1 failed in 0.10s`); elapsed-time readings are redacted from that excerpt before it is persisted, which is what extends the property from a passing fleet to a failing one.

`bond-ops` is engineered to be deterministic and reproducible:

- **Fixed roster data.** The 33-package roster (6 infra + 27 films) is a frozen, ordered data structure in `src/bond_ops/inventory.py`; order never varies across runs or platforms ([Definition 1](#)).
- **No wall-clock in persisted artifacts.** Gate manifests and manuscript variable maps embed no timestamps; for an unchanged fleet they are byte-identical across runs ([Proposition 3](#)).
- **Real subprocess, real files.** Every gate, git inspection, lineage sweep, and provisioning execution reads actual subprocess output and actual files — there is no mock or simulated state.
- **Determinism tests.** The test suite pins coverage parsing, manifest shape, roster order, provisioning script text, and figure bytes. The cross-run claim is checked by running the gate *twice* — two separate live invocations, each spawning its own real test subprocess against the same on-disk fleet — and diffing the two payloads, at both the module and the CLI surface. The CLI-level check runs against a package whose test output stamps a **different** elapsed time on every invocation, so the assertion fails if any clock reading survives into the manifest; it is not satisfied by serializing one report twice.
- **Recovery paths are tested, not assumed.** The three places where `bond-ops` degrades instead of raising — a malformed `manuscript/config.yaml`, a corrupt generated token file, and a gate subprocess that times out or cannot be launched — each carry a test that drives the real failure and asserts the fallback, so a silent-failure path cannot rot unobserved behind a coverage exclusion.
- **Filters cannot silently widen.** `--limit 0` gates nothing and a negative limit is a usage error, so a mistyped bound can never expand into a full-fleet run.
- **No untracked randomness.** No `random` draws; nothing depends on wall-clock ordering.

36.9 Scope and Related Work — QUARTERMASTER: boundaries, positioning, and relationship to the literature

36.9.1 Scope

`bond-ops` covers fleet *operations* only: knowing the 33-package roster ([Definition 1](#)), gating each package ([Definition 3](#)), inspecting health ([Definition 4](#)), and rendering provisioning ([Definition 6](#)). It does **not** dispatch multi-package missions (that is `bond-orchestrator`) and does **not** provide the user-facing front door (that is `bond-cli`). Its gate runs each package’s *own* tests; it never executes another package’s mission logic.

Crucially, `bond-ops` ships **no `mission.py` and no `MissionProvider` implementation**, and this is a design decision rather than an omission. The `MissionProvider` protocol is *defined* by `bond-api` and *implemented* by the 27 film packages, each of which answers `brief / recon / plan / execute / debrief` for its own film. A mission provider is a thing the fleet *runs*; the QUARTERMASTER is the thing that *measures whether the fleet can be run at all*. Making the gate runner itself a mission provider would put the measuring instrument inside the population it measures — the aggregate gate would then have to gate itself as a peer, and a provider failure would be indistinguishable from a gate failure. `bond-ops` therefore imports nothing from `bond-api` and treats every package, film or infrastructure, purely as a directory with tests, a git tree, and a layout to probe.

36.9.2 Related work within the fleet

- **`bond-coordinator` (MISSION CONTROL)** maintains the film-package registry and mission ledger. `bond-ops` currently carries the roster as mirror data; an integration point is recorded in `TODO.md` to consume `bond_coordinator.registry.ROSTER` for film metadata once the coordinator ships as a dependency, with the acceptance criterion that the reconciled slug set stays identical to the canonical 33.
- **`bond-cli` (THE FRONT DOOR)** may consume the gate manifest produced here (the gate manifest table) to surface fleet status to an end user.
- **`bond-api` (THE PROTOCOL)** defines the protocol the film packages implement; `bond-ops` does not depend on it, but a film package fails the aggregate gate if it stops meeting protocol-backed tests.

36.9.3 External dependencies

The fleet’s toolchain (the toolchain table) is standard and versioned via `uv.lock`; the manifest schema (the gate manifest table) and CLI surface are the stable interfaces documented in `docs/`.

36.10 Limitations and Uncertainty — QUARTERMASTER: assumptions, threats to validity, and what the numbers do not claim

36.10.1 Scope of a gate verdict

A 90% line+branch floor is a *necessary*, not a *sufficient*, condition for correctness. A package whose own suite passes can still be wrong where its tests do not look: the QUARTERMASTER reports that each package held its own gate, not that the gate is a proof. This is why measured counts live in `docs/_generated/COUNTS.md` as receipts rather than being folded into manuscript prose (the reproducibility section).

36.10.2 Verdicts are narrower than their diagnostics

`HealthReport.healthy` ([Definition 4](#)) deliberately excludes the layout probes ([Proposition 2](#)): a package with a clean tree and all 4 required files is `healthy` even if its `[project] name` has drifted from its slug, because folding the probe in would change the verdict contract downstream consumers read. The drift is surfaced as `layout_ok=False`, but that is a diagnostic, not a verdict — a reader who wants “identity is correct” must inspect `layout_ok`, not `healthy`.

36.10.3 Fleet identity is a reflection, not a ground truth

The roster ([Definition 1](#)) and each film’s mission codename are cross-checked as data against every present film package’s own declaration. That declaration is read from source and config files and can itself be stale: the coherence test catches a film that renamed itself without updating this roster, but cannot catch a film whose *own* source declares an outdated codename. The check is only as fresh as the packages it reads.

36.10.4 Determinism has a same-environment scope

Gate manifests are byte-identical for an unchanged fleet ([Proposition 3](#)), but figure bytes are deterministic *within a pinned environment*: changing the matplotlib/font stack can move PNG bytes even for identical inputs. The figure guarantee is therefore reproduced under a fixed toolchain rather than claimed unconditionally across every platform.

36.10.5 The suite is not portable to a root-run environment

One health probe reaches its unreadable-file branch with a real `chmod 000` file; a root user can read such a file, so under root that branch would go unexercised and the test fails loudly instead of silently dropping coverage (see `tests/test_health.py`). This is honest scoping, recorded so a later pass does not mistake it for a gap: the suite is operated as a non-root user.

36.10.6 No wall-clock, by design

Persisted artifacts carry no timestamp, which is what makes byte-identical re-runs meaningful, at the cost that a reader cannot tell *when* a manifest was produced from the artifact alone. Toolchain versioning comes from the `uv.lock` pin, not from a clock in the file.

36.10.7 Remaining drift risk: the coordinator registry

`bond-ops` carries the roster as its own data rather than consuming a shared registry. When `bond-coordinator` publishes a machine-readable export, a mismatch must be treated as *fleet drift*, not resolved by picking a winner. That integration point, and the current codename divergence from the coordinator’s provisional registry, are tracked in `TODO.md`.

36.11 Sources — QUARTERMASTER: bibliography

Friedman [2026d]; Krekel et al.; Batchelder and contributors; Marsh and Astral Software Inc.; Lehtosalo and contributors; Hunter [2007]; Wong [2011]

37 Closing

The preceding chapters constitute the complete PROJECT BOND suite: **33 packages (27 films plus 6 infrastructure packages)**, each derived from a common research scaffold, each independently tested to $\geq 90\%$ coverage with zero mocks, each implementing its film’s concepts as real algorithms, and each exposing a **MissionProvider** against the frozen protocol.

Every package chapter above carries that package’s **full manuscript** — not an abstract-only excerpt. The abstract, the introduction, the methodology, the results, the conclusion, the experimental setup, the reproducibility statement, and the scope are all imported from the package’s authored manuscript and token-hydrated from its own variables. The package’s figures are embedded in the combined PDF, with citations resolved against the unified bibliography.

The suite is more than the sum of its films. Through the orchestrator, missions run *across* films: OPERATION OMNIBUS chains **goldfinger** (market-attack recon) into **goldeneye** (EMP resilience) into **no_time_to_die** (bioweapon countermeasure), producing a single consolidated debrief with provenance on every outcome. The coordinator reconciles the fleet’s real codenames and gadgets into a situation room. The front-door CLI exposes the whole surface.

The quality bar and the evidence supporting it:

- **Comprehensiveness** — the compendium imports all 33 full manuscripts, figures included, so no package content is lost in the wrapper.
- **Determinism** — fixed seeds, no untracked randomness, byte-stable reports.
- **Integrity** — no mock framework; every test exercises real computation.
- **Coverage** — the aggregate gate ran `pytest --cov=src --cov-fail-under=90` over the fleet and this compendium itself (which measures its own gate): 33 rows measured, all passing, coverage well above the 90% floor.
- **Unified bibliography** — every package’s references merged into one `references.bib`, cited from the imported chapters.

PROJECT BOND is simultaneously a research fleet, a demonstration of the template’s forkable-project architecture at scale, and — in keeping with its source material — a thorough exercise in serious special-agent operations software.

38 References

Merged automatically from every package's `manuscript/references.bib` into the unified `references.bib`. Keys are cited from the package chapters.

38.1 octopussy

- **knoll2010radiation** — Knoll, Glenn F.. *Radiation Detection and Measurement*.

38.2 no_time_to_die

- **currie1968limits** — Currie, Lloyd A. *Limits for qualitative detection and quantitative determination: Application to radio-chemistry*.

38.3 dr_no

- **ensdf** — National Nuclear Data Center. *ENSDF*.

38.4 no_time_to_die

- **bateman1910solution** — Bateman, Harry. *Solution of a system of differential equations occurring in the theory of radioactive transformations*.

38.5 dr_no

- **hubbell1995tables** — Hubbell, J. H. and Seltzer, S. M.. *Tables of X*.
- **nistxcom** — Berger, M. J. and Hubbell, J. H. and Seltzer, S. M. and Chang, J. and Coursey, J. S. and Sukumar, R. and Zucker, D. S. and Olsen, K.. *XCOM*.
- **icrp119compendium** — International Commission on Radiological Protection. *Compendium of Dose Coefficients based on ICRP*.
- **icrp72** — International Commission on Radiological Protection. *Age-dependent Doses to Members of the Public from Intake of Radionuclides: Part 5, Compilation of Ingestion and Inhalation Dose Coefficients*.
- **smith2012exposure** — Smith, Daniel S. and Stabin, Michael G.. *Exposure Rate Constants and Lead Shielding Values for over 1,100 Radionuclides*.
- **pasquill1961estimation** — Pasquill, F.. *The Estimation of the Dispersion of Windborne Material*.
- **gifford1961use** — Gifford, Franklin A.. *Use of Routine Meteorological Observations for Estimating Atmospheric Dispersion*.
- **martin1976comment** — Martin, D. O.. *Comment on “The Change of Concentration Standard Deviations with Distance”*.
- **turner1994workbook** — Turner, D. Bruce. *Workbook of Atmospheric Dispersion Estimates: An Introduction to Dispersion Modeling*.

38.6 bond-api

- **bond_api** — Friedman, Daniel Ari. *BOND-API: The Frozen Mission Protocol Contract for the BOND Film Suite*.

38.7 never_say_never_again

- **bond_utilities** — Friedman, Daniel Ari. *BOND-Utilities*.

38.8 from_russia_with_love

- **foy1976position** — Foy, Wade H.. *Position-Location Solutions by Taylor*.
- **torrieri1984statistical** — Torrieri, Don J.. *Statistical Theory of Passive Location Systems*.
- **chan1994hyperbolic** — Chan, Y. T. and Ho, K. C.. *A Simple and Efficient Estimator for Hyperbolic Location*.

38.9 octopussy

- **nocedal2006numerical** — Nocedal, Jorge and Wright, Stephen J.. *Numerical Optimization*.

38.10 from_russia_with_love

- **martello1990knapsack** — Martello, Silvano and Toth, Paolo. *Knapsack Problems: Algorithms and Computer Implementations*.
- **bellman1957dynamic** — Bellman, Richard. *Dynamic Programming*.
- **kleinberg2005algorithm** — Kleinberg, Jon and Tardos, 'E. *Algorithm Design*.
- **garey1979computers** — Garey, Michael R. and Johnson, David S.. *Computers and Intractability: A Guide to the Theory of NP-Completeness*.
- **proakis2008digital** — Proakis, John G. and Salehi, Masoud. *Digital Communications*.
- **shannon1949secrecy** — Shannon, Claude E.. *Communication Theory of Secrecy Systems*.

38.11 never_say_never_again

- **nist2015fips180** — National Institute of Standards and Technology. *Secure Hash Standard (SHS)*.

38.12 from_russia_with_love

- **cox1958regression** — Cox, David R.. *The Regression Analysis of Binary Sequences*.
- **fleming1957russia** — Fleming, Ian. *From Russia, with Love*.

38.13 goldfinger

- **walras1874elements** — Walras, Lé. *É*.
- **hicks1946value** — Hicks, John R.. *Value and Capital: An Inquiry into Some Fundamental Principles of Economic Theory*.
- **kyle1985continuous** — Kyle, Albert S.. *Continuous Auctions and Insider Trading*.
- **krugman1979model** — Krugman, Paul. *A Model of Balance-of-Payments Crises*.
- **flood1984collapsing** — Flood, Robert P. and Garber, Peter M.. *Collapsing Exchange-Rate Regimes: Some Linear Examples*.
- **garcia2007design** — Garcia, Mary Lynn. *The Design and Evaluation of Physical Protection Systems*.

38.14 skyfall

- **dijkstra1959note** — Dijkstra, E. W.. *A Note on Two Problems in Connexion with Graphs*.

38.15 goldfinger

- **israeli2002shortest** — Israeli, Eitan and Wood, R. Kevin. *Shortest-Path Network Interdiction*.
- **steen2010laser** — Steen, William M. and Mazumder, Jyotirmoy. *Laser Material Processing*.
- **carslaw1959conduction** — Carslaw, Horatio S. and Jaeger, John C.. *Conduction of Heat in Solids*.
- **siegman1986lasers** — Siegman, Anthony E.. *Lasers*.
- **bondapi2026protocol** — PROJECT BOND suite. *BOND-API*.

38.16 thunderball

- **buehlmann1984** — Albert A. Buehlmann. *Decompression-Decompression Sickness*.
- **urick1983** — Robert J. Urick. *Principles of Underwater Sound*.
- **thorp1967** — William H. Thorp. *Analytic description of the low-frequency attenuation coefficient*.
- **bowditch2002** — Nathaniel Bowditch. *The American Practical Navigator*.
- **newman1977** — John Nicholas Newman. *Marine Hydrodynamics*.
- **cockcroft2011** — A. N. Cockcroft and J. N. F. Lameijer. *A Guide to the Collision Avoidance Rules*.
- **usnavydiving2016** — Naval Sea Systems Command. *U.S. Navy Diving Manual, Revision 7*.
- **fleming1961** — Ian Fleming. *Thunderball*.

38.17 the_living_daylights

- **bondapi2026** — PROJECT BOND. *bond-api: the frozen mission protocol contract*.

38.18 die_another_day

- **vallado2013** — Vallado, David A.. *Fundamentals of Astrodynamics and Applications*.

38.19 you_only_live_twice

- **bate1971** — Bate, Roger R. and Mueller, Donald D. and White, Jerry E.. *Fundamentals of Astrodynamics*.
- **wertz2011** — . *Space Mission Engineering: The New SMAD*.
- **fehse2003** — Fehse, Wigbert. *Automated Rendezvous and Docking of Spacecraft*.
- **reed1990** — Reed, I. S. and Yu, X.. *Adaptive Multiple-Band CFAR*.
- **chandola2009** — Chandola, Varun and Banerjee, Arindam and Kumar, Vipin. *Anomaly Detection: A Survey*.
- **pearl1988** — Pearl, Judea. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*.
- **stone1975** — Stone, Lawrence D.. *Theory of Optimal Search*.
- **waltz1990** — Waltz, Edward and Llinas, James. *Multisensor Data Fusion*.
- **hart1968** — Hart, Peter E. and Nilsson, Nils J. and Raphael, Bertram. *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*.

38.20 live_and_let_die

- **cormen2009** — Cormen, Thomas H. and Leiserson, Charles E. and Rivest, Ronald L. and Stein, Clifford. *Introduction to Algorithms*.

38.21 you_only_live_twice

- **foy1976** — Foy, Wade H.. *Position-Location Solutions by Taylor*.
- **chan1994** — Chan, Y. T. and Ho, K. C.. *A Simple and Efficient Estimator for Hyperbolic Location*.
- **skolnik2001** — Skolnik, Merrill I.. *Introduction to Radar Systems*.
- **daley2003** — Daley, D. J. and Vere-Jones, D.. *An Introduction to the Theory of Point Processes, Volume I: Elementary Theory and Methods*.

38.22 on_her_majestys_secret_service

- **arya1999dispersion** — Arya, S. Pal. *Air Pollution Meteorology and Dispersion*.
- **turner1970workbook** — Turner, D. Bruce. *Workbook of Atmospheric Dispersion Estimates*.
- **zannetti1990air** — Zannetti, Paolo. *Air Pollution Modeling: Theories, Computational Methods, and Available Software*.
- **sykes1998scipuff** — Sykes, R. Ian and Parker, Sonya F. and Henn, Donald S. and Lewellen, William S.. *SCIPUFF*.
- **morlock1989bobsled** — Morlock, M. M. and Zatsiorsky, V. M.. *Factors influencing performance in bobsledding: I*.
- **mcclung2006avalanche** — McClung, David and Schaerer, Peter. *The Avalanche Handbook*.

38.23 skyfall

- **cormen2009algorithms** — Cormen, Thomas H. and Leiserson, Charles E. and Rivest, Ronald L. and Stein, Clifford. *Introduction to Algorithms*.

38.24 diamonds_are_forever

- **friedman2026conflictstone** — Friedman, Daniel Ari. *Diamonds Are Forever: CONFLICT STONE — Special-Agent Mission Software*.
- **globalwitness1998** — Global Witness. *A Rough Trade: The Role of Companies and Governments in the Angolan Conflict*.
- **kimberley2003** — Kimberley Process. *Kimberley Process Certification Scheme*.

- **berman2017** — Berman, Nicolas and Couttenier, Mathieu and Rohner, Dominic and Thoenig, Mathias. *This Mine Is Mine! How Minerals Fuel Conflicts in Africa*.
- **webster1994** — Webster, Robert. *Gems: Their Sources, Descriptions and Identification*.

38.25 casino_royale_2006

- **dijkstra1959** — Dijkstra, Edsger W.. *A Note on Two Problems in Connexion with Graphs*.

38.26 diamonds_are_forever

- **levi2006** — Levi, Michael and Reuter, Peter. *Money Laundering*.
- **benford1938** — Benford, Frank. *The Law of Anomalous Numbers*.
- **bornwolf1999** — Born, Max and Wolf, Emil. *Principles of Optics: Electromagnetic Theory of Propagation, Interference and Diffraction of Light*.
- **newcomb1881** — Newcomb, Simon. *Note on the Frequency of Use of the Different Digits in Natural Numbers*.
- **pearson1900** — Pearson, Karl. *On the Criterion that a Given System of Deviations from the Probable in the Case of a Correlated System of Variables is Such that it Can be Reasonably Supposed to have Arisen from Random Sampling*.
- **nigrini2012** — Nigrini, Mark J.. *Benford's Law: Applications for Forensic Accounting, Auditing, and Fraud Detection*.
- **tarjan1973** — Tarjan, Robert. *Enumeration of the Elementary Circuits of a Directed Graph*.
- **freeman1978** — Freeman, Linton C.. *Centrality in Social Networks Conceptual Clarification*.
- **airy1835** — Airy, George Biddell. *On the Diffraction of an Object-glass with Circular Aperture*.
- **beer1852** — Beer, August. *Bestimmung der Absorption des rothen Lichts in farbigen Flüssigkeiten*.
- **johnson1975** — Johnson, Donald B.. *Finding All the Elementary Circuits of a Directed Graph*.

38.27 live_and_let_die

- **film_lald** — Mankiewicz, Tom. *Live and Let Die*.
- **wood1993** — Wood, R. Kevin. *Deterministic Network Interdiction*.
- **stinson2019** — Stinson, Douglas R. and Paterson, Maura B.. *Cryptography: Theory and Practice*.
- **itu2009** — International Telecommunication Union. *International Morse code*.
- **reuter1986** — Reuter, Peter and Kleiman, Mark A. R.. *Risks and Prices: An Economic Analysis of Drug Enforcement*.
- **caulkins2010** — Caulkins, Jonathan P. and Reuter, Peter. *How Drug Enforcement Affects Drug Prices*.

38.28 the_man_with_the_golden_gun

- **bond_protocol** — Friedman, Daniel Ari. *bond-api: the frozen MissionProvider protocol contract*.
- **golden_gun_film** — Hamilton, Guy. *The Man with the Golden Gun*.
- **duffie_beckman** — Duffie, John A. and Beckman, William A.. *Solar Engineering of Thermal Processes*.
- **spencer** — Spencer, J. W.. *Fourier series representation of the position of the sun*.
- **michalsky** — Michalsky, Joseph J.. *The Astronomical Almanac's algorithm for approximate solar position (1950–2050)*.
- **cormen** — Cormen, Thomas H. and Leiserson, Charles E. and Rivest, Ronald L. and Stein, Clifford. *Introduction to Algorithms*.

38.29 the_world_is_not_enough

- **ford_fulkerson** — Ford, Lester R. and Fulkerson, Delbert R.. *Maximal flow through a network*.
- **edmonds_karp** — Edmonds, Jack and Karp, Richard M.. *Theoretical improvements in algorithmic efficiency for network flow problems*.
- **dijkstra** — Dijkstra, Edsger W.. *A note on two problems in connexion with graphs*.

38.30 the_man_with_the_golden_gun

- **hopcroft_tarjan** — Hopcroft, John and Tarjan, Robert. *Algorithm 447: efficient algorithms for graph manipulation.*
- **kleinberg_tardos** — Kleinberg, Jon and Tardos, É. *Algorithm Design.*
- **tarjan_unionfind** — Tarjan, Robert Endre. *Efficiency of a good but not linear set union algorithm.*

38.31 the_spy_who_loved_me

- **kay_detection_1998** — Kay, Steven M.. *Fundamentals of Statistical Signal Processing: Detection Theory.*
- **nardone_aidala_1981** — Nardone, Steven C. and Aidala, Vincent J.. *Observability criteria for bearings-only target motion analysis.*
- **anderson_moore_1979** — Anderson, Brian D. O. and Moore, John B.. *Optimal Filtering.*
- **tupper_naval_2013** — Tupper, Eric C.. *Introduction to Naval Architecture.*
- **urick_principles_1983** — Urick, Robert J.. *Principles of Underwater Sound.*
- **thorp_1967** — Thorp, William H.. *Analytic description of the low-frequency attenuation coefficient.*
- **mackenzie_1981** — Mackenzie, Kenneth V.. *Nine-term equation for sound speed in the oceans.*
- **van_trees_2002** — Van Trees, Harry L.. *Optimum Array Processing: Part IV of Detection, Estimation, and Modulation Theory.*
- **timoshenko_gere_1961** — Timoshenko, Stephen P. and Gere, James M.. *Theory of Elastic Stability.*
- **young_roark_2011** — Young, Warren C. and Budynas, Richard G. and Sadegh, Ali M.. *Roark's Formulas for Stress and Strain.*

38.32 moonraker

- **kelley1959critical** — Kelley, James E. and Walker, Morgan R.. *Critical-Path Planning and Scheduling.*
- **vallado2013astrodynamics** — Vallado, David A.. *Fundamentals of Astrodynamics and Applications.*
- **stoll1956human** — Stoll, Alice M.. *Human Tolerance to Positive G as Determined by the Physiological End Points.*
- **moonraker1979** — Gilbert, Lewis. *Moonraker.*
- **hohmann1925** — Hohmann, Walter. *Die Erreichbarkeit der Himmelskörper.*
- **clohessy1960** — Clohessy, W. H. and Wiltshire, R. S.. *Terminal Guidance System for Satellite Rendezvous.*
- **tsiolkovsky1903** — Tsiolkovsky, Konstantin E.. *Exploration of Cosmic Space by Means of Reaction Devices.*
- **burton1988** — Burton, Russell R.. *G-Induced Loss of Consciousness: Definition, History, Current Status.*

38.33 for_your_eyes_only

- **shamir1979share** — Shamir, Adi. *How to Share a Secret.*
- **boycott1908prevention** — Boycott, A. E. and Damant, G. C. C. and Haldane, J. S.. *The Prevention of Compressed-air Illness.*
- **hart1968formal** — Hart, Peter E. and Nilsson, Nils J. and Raphael, Bertram. *A Formal Basis for the Heuristic Determination of Minimum Cost Paths.*
- **reed1960polynomial** — Reed, Irving S. and Solomon, Gustave. *Polynomial Codes over Certain Finite Fields.*
- **thorp1967analytic** — Thorp, William H.. *Analytic Description of the Low-Frequency Attenuation Coefficient.*
- **urick1983principles** — Urick, Robert J.. *Principles of Underwater Sound.*
- **wenz1962noise** — Wenz, Gordon M.. *Acoustic Ambient Noise in the Ocean: Spectra and Sources.*
- **blakley1979safeguarding** — Blakley, G. R.. *Safeguarding Cryptographic Keys.*
- **berlekamp1968algebraic** — Berlekamp, Elwyn R.. *Algebraic Coding Theory.*
- **buhlmann1984decompression** — Bü. *Decompression-Decompression Sickness.*

38.34 octopussy

- **lawson1974solving** — Lawson, Charles L. and Hanson, Richard J.. *Solving Least Squares Problems*.
- **keshava2002spectral** — Keshava, Nirmal and Mustard, John F.. *Spectral Unmixing*.
- **delarie1982fluorescence** — de la Rie, E. René. *Fluorescence of Paint and Varnish Layers (Part I)*.
- **feller1994accelerated** — Feller, Robert L.. *Accelerated Aging: Photochemical and Thermal Aspects*.
- **horie2010materials** — Horie, Charles Velson. *Materials for Conservation: Organic Consolidants, Adhesives and Coatings*.

38.35 skyfall

- **ford1962flows** — Ford, Lester Randolph and Fulkerson, Delbert Ray. *Flows in Networks*.

38.36 octopussy

- **ahuja1993network** — Ahuja, Ravindra K. and Magnanti, Thomas L. and Orlin, James B.. *Network Flows: Theory, Algorithms, and Applications*.
- **bellman1958routing** — Bellman, Richard. *On a Routing Problem*.
- **dantzig1954minimizing** — Dantzig, George B. and Fulkerson, D. R.. *Minimizing the Number of Tankers to Meet a Fixed Schedule*.
- **caprara2002train** — Caprara, Alberto and Fischetti, Matteo and Toth, Paolo. *Modeling and Solving the Train Timetabling Problem*.
- **hopcroft1973matching** — Hopcroft, John E. and Karp, Richard M.. *An $\mathcal{O}(n^{5/2})$* .
- **kuhn1955hungarian** — Kuhn, Harold W.. *The Hungarian Method for the Assignment Problem*.
- **kiefer1953sequential** — Kiefer, J.. *Sequential Minimax Search for a Maximum*.
- **marquardt1963algorithm** — Marquardt, Donald W.. *An Algorithm for Least-Squares Estimation of Nonlinear Parameters*.

38.37 the_world_is_not_enough

- **brandes** — Brandes, Ulrik. *A faster algorithm for betweenness centrality*.

38.38 a_view_to_a_kill

- **hhi** — U.S. Department of Justice and Federal Trade Commission. *Horizontal Merger Guidelines*.
- **film** — Glen, John (dir.) and Eon Productions. *A View to a Kill*.
- **dixon** — Mark J. Dixon and Stuart G. Coles. *Modelling Association Football Scores and Inefficiencies in the Football Betting Market*.
- **ast** — Ravindra K. Ahuja and Thomas L. Magnanti and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*.
- **kelly1956** — John L. Kelly. *A New Interpretation of Information Rate*.
- **christopher2004** — Martin Christopher and Helen Peck. *Building the Resilient Supply Chain*.
- **kreibich2008** — Heidi Kreibich and Annegret H. Thieken. *Assessment of damage caused by high groundwater inundation*.
- **greenswets1966** — David M. Green and John A. Swets. *Signal Detection Theory and Psychophysics*.

38.39 the_living_daylights

- **friedman2026sniper** — Friedman, Daniel A.. *The Living Daylights: SNIPER'S NEST — Special-Agent Mission Software*.
- **fm2310sniper** — U.S. Department of the Army. *FM 23-10: Sniper Training*.
- **duckworth1996counter** — Duckworth, Gregory L. and Gilbert, David C. and Barger, James E.. *Acoustic counter-sniper system*.
- **astmg173** — ASTM International. *ASTM G173-03(2020): Standard Tables for Reference Solar Spectral Irradiances: Direct Normal and Hemispherical on 37°rc Tilted Surface*.
- **vantrees1968** — Van Trees, Harry L.. *Detection, Estimation, and Modulation Theory, Part I*.
- **fisher1996viewshed** — Fisher, Peter F.. *Extending the applicability of viewsheds in landscape planning*.

- **kinsler2000** — Kinsler, Lawrence E. and Frey, Austin R. and Coppens, Alan B. and Sanders, James V.. *Fundamentals of Acoustics*.
- **ledecz2005countersniper** — Ledecz, Akos and Nadas, Andras and Volgyesi, Peter and Balogh, Gyorgy and Kusy, Branislav and Sallai, Janos and Pap, Gyula and Dora, Sebestyen and Molnar, Karoly and Maroti, Miklos and Simon, Gyorgy. *Counter-sniper system for urban warfare*.
- **mccoy1999** — McCoy, Robert L.. *Modern Exterior Ballistics: The Launch and Flight Dynamics of Symmetric Projectiles*.
- **bresenham1965** — Bresenham, Jack E.. *Algorithm for computer control of a digital plotter*.

38.40 spectre

- **tarjan1972** — Robert Endre Tarjan. *Depth-first search and linear graph algorithms*.

38.41 licence_to_kill

- **imo1969** — International Maritime Organization. *International Convention on Tonnage Measurement of Ships, 1969*.
- **bellman1957** — Bellman, Richard. *Dynamic Programming*.
- **licence1989** — Maibaum, Richard and Wilson, Michael G.. *Licence to Kill*.
- **fatf2020** — Financial Action Task Force. *International Standards on Combating Money Laundering and the Financing of Terrorism & Proliferation (the FATF Recommendations)*.
- **kellerer2004** — Kellerer, Hans and Pfersch, Ulrich and Pisinger, David. *Knapsack Problems*.
- **smith1956** — Smith, Wayne E.. *Various Optimizers for Single-Stage Production*.
- **bep_currency** — United States Bureau of Engraving and Printing. *Currency Facts: Denominations and Dimensions*.
- **dea_prices** — United States Drug Enforcement Administration. *Domestic Drug Prices and Pure-Purity Analyses (STRIDE / System to Retrieve Information from Drug Evidence)*.
- **shannon1948** — Shannon, Claude Elwood. *A Mathematical Theory of Communication*.

38.42 goldeneye

- **glasstone1977effects** — Glasstone, Samuel and Dolan, Philip J.. *The Effects of Nuclear Weapons*.
- **iec6100029** — . *Electromagnetic compatibility (EMC) – Part 2: Environment – Section 9: Description of HEMP environment – Radiated disturbance*.
- **milstd188125** — . *High-altitude electromagnetic pulse (HEMP) protection for ground-based facilities performing critical, time-urgent missions*.
- **froehlich1995peak** — Froehlich, David C.. *Peak outflow from breached embankment dam*.
- **ritter1892fortpflanzung** — Ritter, August. *Die Fortpflanzung der Wasserwellen*.
- **stoker1957water** — Stoker, James Johnston. *Water Waves: The Mathematical Theory with Applications*.
- **bate1971fundamentals** — Bate, Roger R. and Mueller, Donald D. and White, Jerry E.. *Fundamentals of Astrodynamics*.
- **ott2009electromagnetic** — Ott, Henry W.. *Electromagnetic Compatibility Engineering*.
- **vance1978coupling** — Vance, Edward F.. *Coupling to Shielded Cables*.
- **froehlich2008breach** — Froehlich, David C.. *Embankment dam breach parameters and their uncertainties*.
- **buldyrev2010cascading** — Buldyrev, Sergey V. and Parshani, Roni and Paul, Gerald and Stanley, H. Eugene and Havlin, Shlomo. *Catastrophic cascade of failures in interdependent networks*.

38.43 skyfall

- **edmonds1972theoretical** — Edmonds, Jack and Karp, Richard M.. *Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems*.

38.44 goldeneye

- **schelkunoff1943electromagnetic** — Schelkunoff, Sergei A.. *Electromagnetic Waves*.

38.45 tomorrow_never_dies

- **marcum1960statistical** — Marcum, J. I.. *A statistical theory of target detection by pulsed radar.*
- **swerling1960probability** — Swerling, P.. *Probability of detection for fluctuating targets.*
- **kempe2003maximizing** — Kempe, David and Kleinberg, Jon and Tardos, É. *Maximizing the spread of influence through a social network.*
- **misra2006gps** — Misra, Pratap and Enge, Per. *Global Positioning System: Signals, Measurements, and Performance.*
- **skolnik2001radar** — Skolnik, Merrill I.. *Introduction to Radar Systems.*
- **tnd1997** — . *Tomorrow Never Dies.*
- **hegselmann2002opinion** — Hegselmann, Rainer and Krause, Ulrich. *Opinion dynamics and bounded confidence: models, analysis and simulation.*
- **degroot1974reaching** — DeGroot, Morris H.. *Reaching a consensus.*
- **barker1953group** — Barker, R. H.. *Group synchronizing of binary digital systems.*
- **parkinson1988autonomous** — Parkinson, Bradford W. and Axelrad, Penina. *Autonomous GPS.*
- **brown1992baseline** — Brown, R. Grover. *A baseline GPS.*

38.46 the_world_is_not_enough

- **urick** — Urick, Robert J.. *Principles of Underwater Sound.*
- **namnyak** — Namnyak, Michelle and Tufton, Norra and Szekely, Rebecca and Toal, Matthew and Worboys, Sarah and Sampson, Eleanor L.. *“Stockholm syndrome”: psychiatric diagnosis or urban myth?*
- **de_fabrique** — de Fabrique, Nathalie and Romano, Stephen J. and Vecchi, Gregory M. and van Hasselt, Vincent B.. *Understanding Stockholm syndrome.*
- **stott** — Stott, Brian. *Review of load-flow calculation methods.*
- **dobson_op** — Dobson, Ian and Carreras, Benjamin A. and Lynch, Vickie E. and Newman, David E.. *An initial model for complex dynamics in electric power system blackouts.*
- **rubinstein** — Rubinstein, Ariel. *Perfect equilibrium in a bargaining model.*
- **kalman** — Kalman, Rudolph E.. *A new approach to linear filtering and prediction problems.*

38.47 die_another_day

- **mahalanobis1936** — Mahalanobis, Prasanta Chandra. *On the generalized distance in statistics.*
- **levenshtein1966** — Levenshtein, Vladimir Iosifovich. *Binary codes capable of correcting deletions, insertions, and reversals.*
- **gold1971** — Gold, Lorne W.. *Use of ice covers for transportation.*
- **sodhi1995** — Sodhi, Devinder S.. *Breakthrough loads of floating ice sheets.*
- **hobbs1974** — Hobbs, Peter V.. *Ice Physics.*
- **tipton1989** — Tipton, Michael J.. *The initial responses to cold-water immersion in man.*
- **golden2002** — Golden, Frank and Tipton, Michael J.. *Essentials of Sea Survival.*
- **newton1701** — Newton, Isaac. *Scala Graduum Caloris: Calorum Descriptiones & Signa.*
- **battin1999** — Battin, Richard H.. *An Introduction to the Mathematics and Methods of Astrodynamics.*
- **glaser1968** — Glaser, Peter E.. *Power from the Sun: Its Future.*
- **ondov2016** — Ondov, Brian D. and Treangen, Todd J. and Melsted, Pá. *Mash: fast genome and metagenome distance estimation using MinHash.*

38.48 casino_royale_2006

- **bond_api_protocol** — Friedman, Daniel Ari. *BOND-API: The Frozen MissionProvider Protocol (THE PROTOCOL).*
- **casino_royale_2006** — Friedman, Daniel Ari. *Casino Royale (2006) – LE CHIFFRE Special-Agent Mission Software.*
- **zinkevich2007cfr** — Zinkevich, Martin and Johanson, Michael and Bowling, Michael and Piccione, Carmelo. *Regret Minimization in Games with Incomplete Information.*

- **billings2002poker** — Billings, Darse and Davidson, Aaron and Schaeffer, Jonathan and Szafron, Duane. *The Challenge of Poker*.
- **chen_ankenman_2006** — Chen, Bill and Ankenman, Jerrod. *The Mathematics of Poker*.
- **ganzfried2011opponent** — Ganzfried, Sam and Sandholm, Tuomas. *Game Theory-Based Opponent Modeling in Large Imperfect-Information Games*.
- **minka2000dirichlet** — Minka, Thomas P.. *Estimating a Dirichlet*.
- **page1999pagerank** — Page, Lawrence and Brin, Sergey and Motwani, Rajeev and Winograd, Terry. *The PageRank*.
- **raghavan2007labelprop** — Raghavan, Usha Nandini and Albert, Ré. *Near Linear Time Algorithm to Detect Community Structures in Large-Scale Networks*.

38.49 spectre

- **brandes2001** — Ulrik Brandes. *A faster algorithm for betweenness centrality*.
- **hirschman1964** — Albert O. Hirschman. *The paternity of an index*.

38.50 casino_royale_2006

- **weber2019aml** — Weber, Mark and Domeniconi, Giacomo and Chen, Jie and Weidele, Daniel Karl I. and Bellei, Claudio and Robinson, Tom and Leiserson, Charles E.. *Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics*.
- **fatf2009casinos** — Financial Action Task Force. *Vulnerabilities of Casinos and Gaming Sector*.
- **antman1990fab** — Antman, Elliott M. and Wenger, Thomas L. and Butler, Vincent P. and Haber, Edgar and Smith, Thomas W.. *Treatment of 150 Cases of Life-Threatening Digitalis Intoxication with Digoxin-Specific Fab*.

38.51 quantum_of_solace

- **hirschman1945national** — Hirschman, Albert O.. *National Power and the Structure of Foreign Trade*.
- **shapley1953value** — Shapley, Lloyd S.. *A value for n -person games*.
- **banzhaf1965weighted** — Banzhaf, John F.. *Weighted voting doesn't work: a mathematical analysis*.
- **ford1956maximal** — Ford, Lester R. and Fulkerson, Delbert R.. *Maximal flow through a network*.
- **herfindahl1950concentration** — Herfindahl, Orris C.. *Concentration in the Steel Industry*.
- **doj2010merger** — U.S. Department of Justice. *Horizontal Merger Guidelines*.
- **dubey1979mathematical** — Dubey, Pradeep and Shapley, Lloyd S.. *Mathematical properties of the Banzhaf*.
- **wood1993deterministic** — Wood, R. Kevin. *Deterministic network interdiction*.
- **espey1997price** — Espey, Molly and Espey, James and Shaw, W. Douglass. *Price elasticity of residential demand for water: A meta-analysis*.
- **loucks2017water** — Loucks, Daniel P. and van Beek, Eelco. *Water Resource Systems Planning and Management: An Introduction to Methods, Models, and Applications*.

38.52 skyfall

- **even2011graph** — Even, Shimon. *Graph Algorithms*.
- **page1954continuous** — Page, E. S.. *Continuous Inspection Schemes*.
- **pinedo2016scheduling** — Pinedo, Michael L.. *Scheduling: Theory, Algorithms, and Systems*.
- **hawkins1999cumulative** — Hawkins, Douglas M. and Olwell, David H.. *Cumulative Sum Charts and Charting for Quality Improvement*.
- **tarjan1972dfs** — Tarjan, Robert. *Depth-First Search and Linear Graph Algorithms*.
- **hopcroft1973algorithm447** — Hopcroft, John and Tarjan, Robert. *Algorithm 447: Efficient Algorithms for Graph Manipulation*.
- **phillips1998graphbased** — Phillips, Cynthia and Swiler, Laura Painton. *A Graph-Based System for Network-Vulnerability Analysis*.

- **sheyner2002attackgraphs** — Sheyner, Oleg and Haines, Joshua and Jha, Somesh and Lippmann, Richard and Wing, Jeanette M.. *Automated Generation and Analysis of Attack Graphs*.
- **mell2007cvss** — Mell, Peter and Scarfone, Karen and Romanosky, Sasha. *A Complete Guide to the Common Vulnerability Scoring System Version 2.0*.
- **beattie2002timing** — Beattie, Steve and Arnold, Seth and Cowan, Crispin and Wagle, Perry and Wright, Chris and Shostack, Adam. *Timing the Application of Security Patches for Optimal Uptime*.
- **killick2012changepoints** — Killick, Rebecca and Fearnhead, Paul and Eckley, Idris A.. *Optimal Detection of Changepoints with a Linear Computational Cost*.
- **montgomery2012spc** — Montgomery, Douglas C.. *Introduction to Statistical Quality Control*.
- **brandes2001algorithm** — Brandes, Ulrik. *A Faster Algorithm for Betweenness Centrality*.

38.53 spectre

- **freeman1979** — Linton C. Freeman. *Centrality in social networks: Conceptual clarification*.
- **newman2010** — Mark E. J. Newman. *Networks: An Introduction*.
- **nemhauser1978** — George L. Nemhauser and Laurence A. Wolsey and Marshall L. Fisher. *An analysis of approximations for maximizing submodular set functions—I*.
- **johnson1974** — David S. Johnson. *Approximation algorithms for combinatorial problems*.
- **feige1998** — Uriel Feige. *A threshold of nn for approximating set cover*.
- **newman2002** — Mark E. J. Newman. *Spread of epidemic disease on networks*.
- **hopcrofttarjan1973** — John Hopcroft and Robert Endre Tarjan. *Algorithm 447: Efficient algorithms for graph manipulation*.
- **latora2001** — Vito Latora and Massimo Marchiori. *Efficient behavior of small-world networks*.
- **albert2000** — Ré. *Error and attack tolerance of complex networks*.
- **borgatti2006** — Stephen P. Borgatti. *Identifying sets of key players in a social network*.
- **kempe2003** — David Kempe and Jon Kleinberg and É. *Maximizing the spread of influence through a social network*.
- **leskovec2007** — Jure Leskovec and Andreas Krause and Carlos Guestrin and Christos Faloutsos and Jeanne VanBriesen and Natalie Glance. *Cost-effective outbreak detection in networks*.

38.54 no_time_to_die

- **bond_api_2026** — PROJECT BOND. *bond-api: the frozen mission contract for the BOND film suite*.
- **bond_orchestrator_2026** — PROJECT BOND. *bond-orchestrator: the BOND suite DAG runner*.
- **bond_utilities_2026** — PROJECT BOND. *bond-utilities: Q-BRANCH ciphers, codenames, and mission_io provenance*.
- **hill1910possible** — Hill, Archibald Vivian. *The possible effects of the aggregation of the molecules of haemoglobin on its dissociation curves*.
- **no_time_to_die_2021** — Fukunaga, Cary Joji and Purvis, Neal and Wade, Robert. *No Time to Die*.
- **doench2014rational** — Doench, John G and Hartenian, Ella and Graham, Daniel B and Tothova, Zuzana and Hegde, Mudra and Smith, Ian and Sullender, Meagan and Ebert, Benjamin L and Xavier, Ramnik J and Root, David E. *Rational design of highly active sgRNA*.
- **hsu2013dna** — Hsu, Patrick D and Scott, David A and Weinstein, Jason A and Ran, F Ann and Konermann, Silvana and Agarwala, Vineeta and Li, Yinqing and Fine, Eli J and Wu, Xuebing and Shalem, Ophir and others. *DNA*.
- **cheng1973relationship** — Cheng, Yung-Chi and Prusoff, William H. *Relationship between the inhibition constant (K)*.
- **nrc1983risk** — National Research Council. *Risk Assessment in the Federal Government: Managing the Process*.
- **gibaldi1982pharmacokinetics** — Gibaldi, Milo and Perrier, Donald. *Pharmacokinetics*.
- **langmuir1918adsorption** — Langmuir, Irving. *The adsorption of gases on plane surfaces of glass, mica and platinum*.
- **bliss1939toxicity** — Bliss, Chester I. *The toxicity of poisons applied jointly*.
- **lawsonhanson1974** — Lawson, Charles L and Hanson, Richard J. *Solving Least Squares Problems*.

38.55 casino_royale_1967

- **shannon1948communication** — Shannon, Claude E.. *A Mathematical Theory of Communication*.
- **kemeny1960finite** — Kemeny, John G. and Snell, J. Laurie. *Finite Markov Chains*.
- **ethier2010doctrine** — Ethier, Stewart N.. *The Doctrine of Chances: Probabilistic Aspects of Gambling*.
- **wizardofodds_baccarat** — Shackelford, Michael. *Baccarat Basics*.
- **casinoroyale1967film** — Huston, John and Hughes, Ken and McGrath, Joseph and Parrish, Robert and Guest, Val. *Casino Royale*.
- **cover2006elements** — Cover, Thomas M. and Thomas, Joy A.. *Elements of Information Theory*.
- **strehl2002cluster** — Strehl, Alexander and Ghosh, Joydeep. *Cluster Ensembles — A Knowledge Reuse Framework for Combining Multiple Partitions*.

38.56 never_say_never_again

- **nemhauser1978submodular** — Nemhauser, George L. and Wolsey, Laurence A. and Fisher, Marshall L.. *An Analysis of Approximations for Maximizing Submodular Set Functions—I*.
- **wright1936learning** — Wright, T. P.. *Factors Affecting the Cost of Airplanes*.
- **barlow1975reliability** — Barlow, Richard E. and Proschan, Frank. *Statistical Theory of Reliability and Life Testing: Probability Models*.
- **ross2019probability** — Ross, Sheldon M.. *Introduction to Probability Models*.
- **saltelli2008sensitivity** — Saltelli, Andrea and Ratto, Marco and Andres, Terry and Campolongo, Francesca and Cariboni, Jessica and Gatelli, Debora and Saisana, Michaela and Tarantola, Stefano. *Global Sensitivity Analysis. The Primer*.
- **verhulst1838notice** — Verhulst, Pierre-Francc. *Notice sur la loi que la population suit dans son accroissement*.
- **unicode2023uax15** — The Unicode Consortium. *Unicode Standard Annex #15: Unicode Normalization Forms*.

38.57 bond-utilities

- **kahn_codebreakers** — Kahn, David. *The Codebreakers: The Comprehensive History of Secret Communication from Ancient Times to the Internet*.
- **shannon_1949** — Shannon, Claude Elwood. *Communication Theory of Secrecy Systems*.
- **matsumoto_1998** — Matsumoto, Makoto and Nishimura, Takuji. *Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudo-Random Number Generator*.
- **friedman_1922** — Friedman, William F.. *The Index of Coincidence and Its Applications in Cryptography*.
- **kasiski_1863** — Kasiski, Friedrich. *Die Geheimschriften und die Dechiffir-Kunst*.
- **stinson_paterson_2018** — Stinson, Douglas R. and Paterson, Maura B.. *Cryptography: Theory and Practice*.
- **iso8601** — . *ISO*.
- **unicode_standard** — . *The Unicode Standard*.

38.58 bond-api

- **pep544** — Levkivskyi, Ivan and Lehtosalo, Jukka and Langa, Ł. *PEP 544 – Protocols: Structural subtyping (static duck typing)*.
- **pep557** — Smith, Eric V.. *PEP 557 – Data Classes*.
- **pep484** — van Rossum, Guido and Lehtosalo, Jukka and Langa, Ł. *PEP 484 – Type Hints*.

38.59 bond-cli

- **rfc8259** — Bray, Tim. *The JavaScript Object Notation (JSON) Data Interchange Format*.

38.60 bond-api

- **nist_fips180_4** — National Institute of Standards and Technology. *FIPS 180-4: Secure Hash Standard (SHA-1, SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224 and SHA-512/256)*.

38.61 bond-coordinator

- **gamma1995designpatterns** — Gamma, Erich and Helm, Richard and Johnson, Ralph and Vlissides, John. *Design Patterns: Elements of Reusable Object-Oriented Software*.
- **brooks1987nosilverbullet** — Brooks, Frederick P.. *No Silver Bullet: Essence and Accidents of Software Engineering*.
- **bass2012softwarearchitecture** — Bass, Len and Clements, Paul and Kazman, Rick. *Software Architecture in Practice*.
- **fowler2002patterns** — Fowler, Martin. *Patterns of Enterprise Application Architecture*.
- **hunt1999pragmatic** — Hunt, Andrew and Thomas, David. *The Pragmatic Programmer: From Journeyman to Master*.
- **swebok2014** — Bourque, Pierre and Fairley, Richard E.. *Guide to the Software Engineering Body of Knowledge (SWEBOK), Version 3.0*.
- **sandve2013tensimplerules** — Sandve, Geir Kjetil and Nekrutenko, Anton and Taylor, James and Hovig, Eivind. *Ten Simple Rules for Reproducible Computational Research*.
- **peng2011reproducibleresearch** — Peng, Roger D.. *Reproducible Research in Computational Science*.

38.62 bond-orchestrator

- **kahn1962topological** — Kahn, Arthur B.. *Topological Sorting of Large Networks*.
- **cormen2009introduction** — Cormen, Thomas H. and Leiserson, Charles E. and Rivest, Ronald L. and Stein, Clifford. *Introduction to Algorithms*.
- **elnozahy2002survey** — Elnozahy, Elmootazbellah N. and Alvisi, Lorenzo and Wang, Yi-Min and Johnson, David B.. *A Survey of Rollback-Recovery Protocols in Message-Passing Systems*.
- **chandy1985distributed** — Chandy, K. Mani and Lamport, Leslie. *Distributed Snapshots: Determining Global States of Distributed Systems*.
- **kwok1999static** — Kwok, Yu-Kwong and Ahmad, Ishfaq. *Static Scheduling Algorithms for Allocating Directed Task Graphs to Multiprocessors*.
- **avizienis2004basic** — Aviz. *Basic Concepts and Taxonomy of Dependable and Secure Computing*.
- **gamma1994design** — Gamma, Erich and Helm, Richard and Johnson, Ralph and Vlissides, John. *Design Patterns: Elements of Reusable Object-Oriented Software*.
- **bondorchestrator2026** — Friedman, Daniel Ari. *bond-orchestrator: A Deterministic Mission DAG Runner (BOND-ORCHESTRATOR, codename DAG 00)*.

38.63 bond-cli

- **bethard2009argparse** — Bethard, Steven. *PEP 389 — argparse: New Command Line Parsing Module*.
- **pythonargparse** — Python Software Foundation. *argparse — Parser for command-line options, arguments and sub-commands*.
- **posix2018** — . *The Open Group Base Specifications Issue 7, 2018 edition (IEEE Std 1003.1-2017), Chapter 12: Utility Conventions*.
- **raymond2003unix** — Raymond, Eric S.. *The Art of Unix Programming*.
- **semver2** — Preston-Werner, Tom. *Semantic Versioning 2.0.0*.
- **pythoninspect** — Python Software Foundation. *inspect — Inspect live objects*.

38.64 bond-ops

- **bond_ops** — Friedman, Daniel Ari. *BOND-OPS: Fleet Health, Aggregate Gates, and Provisioning for PROJECT BOND*.
- **pytest** — Krekel, Holger and Oliveira, Bruno and Pfannschmidt, Ronny and Bruynooghe, Floris and Laughner, Brianna and Bruhin, Florian. *pytest: helps you write better programs*.
- **coveragepy** — Batchelder, Ned and contributors. *Coverage.py*.
- **ruff** — Marsh, Charlie and Astral Software Inc.. *Ruff: An extremely fast Python linter and code formatter*.
- **mypy** — Lehtosalo, Jukka and contributors. *mypy: Optional Static Typing for Python*.
- **matplotlib** — Hunter, John D.. *Matplotlib: A 2D Graphics Environment*.
- **wong2011** — Wong, Bang. *Points of view: Color blindness*.

- Electromagnetic compatibility (emc) – part 2: Environment – section 9: Description of hemp environment – radiated disturbance. IEC 61000-2-9, 1996.
- Tomorrow never dies. Motion picture, MGM / United Artists, 1997. Directed by Roger Spottiswoode.
- High-altitude electromagnetic pulse (hemp) protection for ground-based facilities performing critical, time-urgent missions. MIL-STD-188-125-1, 1998.
- ISO 8601-1:2019 — date and time — representations for information interchange. International Organization for Standardization, 2019.
- The unicode standard. Unicode Consortium, 2025. Defines which characters carry the Alphabetic property; cited for the deliberate ASCII-only scope of this package’s cipher alphabet.
- Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Upper Saddle River, NJ, 1993a.
- Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993b.
- George Biddell Airy. On the diffraction of an object-glass with circular aperture. *Transactions of the Cambridge Philosophical Society*, 5:283–291, 1835. The Airy pattern; source of the 1.22 first-null constant in the spot-radius model.
- Réka Albert, Hawoong Jeong, and Albert-László Barabási. Error and attack tolerance of complex networks. *Nature*, 406(6794):378–382, 2000. doi: 10.1038/35019019.
- Brian D. O. Anderson and John B. Moore. *Optimal Filtering*. Prentice Hall, 1979.
- Elliott M. Antman, Thomas L. Wenger, Vincent P. Butler, Edgar Haber, and Thomas W. Smith. Treatment of 150 cases of life-threatening digitalis intoxication with digoxin-specific Fab antibody fragments: Final report of a multicenter study. *Circulation*, 81(6):1744–1752, 1990. Digoxin-specific Fab fragments as definitive therapy for life-threatening digitalis toxicity; the severe-grade intervention in poison__detect.antidote__plan.
- S. Pal Arya. *Air Pollution Meteorology and Dispersion*. Oxford University Press, 1999.
- ASTM International. Astm g173-03(2020): Standard tables for reference solar spectral irradiances: Direct normal and hemispherical on 37° tilted surface, 2020. Reference solar irradiance of 1000 W/m².
- Algirdas Avizienis, Jean-Claude Laprie, Brian Randell, and Carl Landwehr. Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, 1(1):11–33, 2004. doi: 10.1109/TDSC.2004.2.
- John F. Banzhaf. Weighted voting doesn’t work: a mathematical analysis. *Rutgers Law Review*, 19(2):317–343, 1965.
- R. H. Barker. Group synchronizing of binary digital systems. In Willis Jackson, editor, *Communication Theory*, pages 273–287. Academic Press, 1953.
- Richard E. Barlow and Frank Proschan. *Statistical Theory of Reliability and Life Testing: Probability Models*. Holt, Rinehart and Winston, New York, 1975.
- Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley, Upper Saddle River, NJ, USA, 3rd edition, 2012. ISBN 978-0-321-81573-6.
- Ned Batchelder and contributors. Coverage.py. <https://coverage.readthedocs.io>. Software; version pinned by this package’s uv.lock. Line and branch coverage measurement enforced at the gate floor.
- Roger R. Bate, Donald D. Mueller, and Jerry E. White. *Fundamentals of Astrodynamics*. Dover Publications, New York, 1971a.
- Roger R. Bate, Donald D. Mueller, and Jerry E. White. *Fundamentals of Astrodynamics*. Dover Publications, New York, 1971b.
- Harry Bateman. Solution of a system of differential equations occurring in the theory of radioactive transformations. *Proceedings of the Cambridge Philosophical Society*, 15:423–427, 1910.
- Richard H. Battin. *An Introduction to the Mathematics and Methods of Astrodynamics*. AIAA Education Series, Reston, VA, 1999.
- Steve Beattie, Seth Arnold, Crispin Cowan, Perry Wagle, Chris Wright, and Adam Shostack. Timing the application of security patches for optimal uptime. In *Proceedings of the 16th USENIX Conference on System Administration (LISA ’02)*, pages 233–242. USENIX Association, 2002.
- August Beer. Bestimmung der absorption des rothen lichts in farbigen flüssigkeiten. *Annalen der Physik und Chemie*, 162(5):78–88, 1852. Exponential absorption law used for the atmospheric transmittance term.
- Richard Bellman. *Dynamic Programming*. Princeton University Press, Princeton, NJ, 1957a.
- Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957b.
- Richard Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.

- Frank Benford. The law of anomalous numbers. *Proceedings of the American Philosophical Society*, 78(4):551–572, 1938. Empirical law of anomalous first digits; basis of the ledger audit.
- M. J. Berger, J. H. Hubbell, S. M. Seltzer, J. Chang, J. S. Coursey, R. Sukumar, D. S. Zucker, and K. Olsen. XCOM: Photon cross sections database. NIST Standard Reference Database 8 (XGAM), 2010. Photon cross sections cross-checking the shielding tables.
- Elwyn R. Berlekamp. *Algebraic Coding Theory*. McGraw-Hill, 1968.
- Nicolas Berman, Mathieu Couttenier, Dominic Rohner, and Mathias Thoenig. This mine is mine! how minerals fuel conflicts in africa. *American Economic Review*, 107(6):1564–1610, 2017. Economics of mineral-fueled conflict; motivates conflict-resource provenance.
- Steven Bethard. Pep 389 — argparse: New command line parsing module. Python Enhancement Proposal 389, 2009. URL <https://peps.python.org/pep-0389/>. The proposal that added argparse to the Python standard library, including the sub-command (sub-parser) model this CLI’s verb-first grammar uses.
- Darse Billings, Aaron Davidson, Jonathan Schaeffer, and Duane Szafron. The challenge of poker. *Artificial Intelligence*, 134(1–2): 201–240, 2002. Survey of hand evaluation, simulation-based equity, and opponent modelling in computer poker.
- G. R. Blakley. Safeguarding cryptographic keys. In *Proceedings of the 1979 AFIPS National Computer Conference*, volume 48, pages 313–317. AFIPS Press, 1979.
- Chester I Bliss. The toxicity of poisons applied jointly. *Annals of Applied Biology*, 26(3):585–615, 1939.
- PROJECT BOND. bond-api: the frozen mission contract for the bond film suite. Local-only PROJECT BOND suite, Layer 1, 2026a.
- PROJECT BOND. bond-orchestrator: the bond suite dag runner. Local-only PROJECT BOND suite, Layer 3, 2026b.
- PROJECT BOND. bond-utilities: Q-branch ciphers, codenames, and mission_io provenance. Local-only PROJECT BOND suite, Layer 0, 2026c.
- PROJECT BOND. bond-api: the frozen mission protocol contract. PROJECT BOND suite, Active Inference Institute, 2026d. Phase 1 reference; films implement its MissionProvider.
- Stephen P. Borgatti. Identifying sets of key players in a social network. *Computational and Mathematical Organization Theory*, 12 (1):21–34, 2006.
- Max Born and Emil Wolf. *Principles of Optics: Electromagnetic Theory of Propagation, Interference and Diffraction of Light*. Cambridge University Press, 7th edition, 1999. Diffraction limits and atmospheric propagation used in the beam physics.
- Pierre Bourque and Richard E. Fairley. *Guide to the Software Engineering Body of Knowledge (SWEBOK), Version 3.0*. IEEE Computer Society, Los Alamitos, CA, USA, 2014. ISBN 978-0-7695-5166-1.
- Nathaniel Bowditch. *The American Practical Navigator*. National Imagery and Mapping Agency, Bethesda, MD, 2002. Pub. No. 9.
- A. E. Boycott, G. C. C. Damant, and J. S. Haldane. The prevention of compressed-air illness. *The Journal of Hygiene*, 8(3):342–443, 1908. PMCID: PMC2167126.
- Ulrik Brandes. A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2):163–177, 2001a. doi: 10.1080/0022250X.2001.9990249.
- Ulrik Brandes. A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2):163–177, 2001b. doi: 10.1080/0022250X.2001.9990249.
- Ulrik Brandes. A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2):163–177, 2001c.
- Tim Bray. The javascript object notation (json) data interchange format. Request for Comments 8259, Internet Engineering Task Force, December 2017. URL <https://www.rfc-editor.org/rfc/rfc8259>. Internet Standard 90. The interchange format emitted by `--json` and `roster-export`.
- Jack E. Bresenham. Algorithm for computer control of a digital plotter. *IBM Systems Journal*, 4(1):25–30, 1965. doi: 10.1147/sj.41.0025.
- Frederick P. Brooks. No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4):10–19, 1987. doi: 10.1109/MC.1987.1663532.
- R. Grover Brown. A baseline GPS RAIM scheme and a note on the equivalence of three RAIM methods. *Navigation*, 39(3):301–316, 1992.
- Albert A. Buehlmann. *Decompression–Decompression Sickness*. Springer-Verlag, Berlin, 1984.
- Albert A. Bühlmann. *Decompression–Decompression Sickness*. Springer-Verlag, Berlin, 1984.
- Sergey V. Buldyrev, Roni Parshani, Gerald Paul, H. Eugene Stanley, and Shlomo Havlin. Catastrophic cascade of failures in interdependent networks. *Nature*, 464(7291):1025–1028, 2010.

- Russell R. Burton. G-induced loss of consciousness: Definition, history, current status. *Aviation, Space, and Environmental Medicine*, 59(1):2–5, 1988.
- Alberto Caprara, Matteo Fischetti, and Paolo Toth. Modeling and solving the train timetabling problem. *Operations Research*, 50(5):851–861, 2002.
- Horatio S. Carslaw and John C. Jaeger. *Conduction of Heat in Solids*. Oxford University Press, Oxford, 2 edition, 1959. The constant-surface-flux temperature solution used by `laser_thermal.surface_melt_time`.
- Jonathan P. Caulkins and Peter Reuter. How drug enforcement affects drug prices. *Crime and Justice*, 39(1):213–271, 2010. Producer share and price spread in narcotics markets.
- Y. T. Chan and K. C. Ho. A simple and efficient estimator for hyperbolic location. *IEEE Transactions on Signal Processing*, 42(8):1905–1915, 1994a.
- Y. T. Chan and K. C. Ho. A simple and efficient estimator for hyperbolic location. *IEEE Transactions on Signal Processing*, 42(8):1905–1915, 1994b. Closed-form, non-iterative two-stage weighted least squares – an alternative to the iterative scheme; NOT implemented here.
- Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):1–58, 2009.
- K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985. doi: 10.1145/214451.214456.
- Bill Chen and Jerrod Ankenman. *The Mathematics of Poker*. ConJelCo, 2006. Pot odds, equity, and the Independent Chip Model: chip stacks to tournament prize equity; the basis for `icm.py`.
- Yung-Chi Cheng and William H Prusoff. Relationship between the inhibition constant (K_1) and the concentration of inhibitor which causes 50 per cent inhibition (I_{50}) of an enzymatic reaction. *Biochemical Pharmacology*, 22(23):3099–3108, 1973.
- Martin Christopher and Helen Peck. Building the resilient supply chain. *The International Journal of Logistics Management*, 15(2):1–14, 2004.
- W. H. Clohessy and R. S. Wiltshire. Terminal guidance system for satellite rendezvous. *Journal of the Aerospace Sciences*, 27(9):653–658, 1960.
- A. N. Cockcroft and J. N. F. Lameijer. *A Guide to the Collision Avoidance Rules*. Butterworth-Heinemann, Oxford, 7th edition, 2011.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3rd edition, 2009a. Ch. 24 Dijkstra; ch. 26 maximum flow, Edmonds–Karp and max-flow min-cut; sec. 16.5 unit-time task scheduling with deadlines as a matroid, which is the exactness argument for `deadline_scheduler`.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009b. Dijkstra, Edmonds–Karp max-flow, residual min-cut theorem.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3 edition, 2009c.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, USA, 3 edition, 2009d. ISBN 978-0-262-03384-8.
- Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, Hoboken, NJ, 2 edition, 2006. Entropy, conditional entropy and mutual information as used by the identity-confusion model.
- David R. Cox. The regression analysis of binary sequences. *Journal of the Royal Statistical Society, Series B*, 20(2):215–242, 1958.
- Lloyd A Currie. Limits for qualitative detection and quantitative determination: Application to radiochemistry. *Analytical Chemistry*, 40(3):586–593, 1968.
- D. J. Daley and D. Vere-Jones. *An Introduction to the Theory of Point Processes, Volume I: Elementary Theory and Methods*. Springer, New York, 2nd edition, 2003.
- George B. Dantzig and D. R. Fulkerson. Minimizing the number of tankers to meet a fixed schedule. *Naval Research Logistics Quarterly*, 1(3):217–222, 1954.
- Nathalie de Fabrique, Stephen J. Romano, Gregory M. Vecchi, and Vincent B. van Hasselt. Understanding stockholm syndrome. *FBI Law Enforcement Bulletin*, 76(7):10–15, 2007.
- E. René de la Rie. Fluorescence of paint and varnish layers (part i). *Studies in Conservation*, 27(1):1–7, 1982.
- Morris H. DeGroot. Reaching a consensus. *Journal of the American Statistical Association*, 69(345):118–121, 1974.
- E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959a.

- Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959b. doi: 10.1007/BF01386390.
- Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959c. Single-source shortest paths; run over feasibility-filtered parkour edges in `urban_routes.py`.
- Mark J. Dixon and Stuart G. Coles. Modelling association football scores and inefficiencies in the football betting market. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 46(2):265–280, 1997.
- Ian Dobson, Benjamin A. Carreras, Vickie E. Lynch, and David E. Newman. An initial model for complex dynamics in electric power system blackouts. In *Proceedings of the 34th Annual Hawaii International Conference on System Sciences (HICSS)*, 2001. doi: 10.1109/HICSS.2001.926274.
- John G Doench, Ella Hartenian, Daniel B Graham, Zuzana Tothova, Mudra Hegde, Ian Smith, Meagan Sullender, Benjamin L Ebert, Ramnik J Xavier, and David E Root. Rational design of highly active sgRNAs for CRISPR–Cas9-mediated gene inactivation. *Nature Biotechnology*, 32(12):1262–1267, 2014.
- Pradeep Dubey and Lloyd S. Shapley. Mathematical properties of the Banzhaf power index. *Mathematics of Operations Research*, 4(2):99–131, 1979. doi: 10.1287/moor.4.2.99.
- Gregory L. Duckworth, David C. Gilbert, and James E. Barger. Acoustic counter-sniper system. In *Proc. SPIE 2938: Command, Control, Communications, and Intelligence Systems for Law Enforcement*, pages 262–275. SPIE, 1996. doi: 10.1117/12.266747.
- John A. Duffie and William A. Beckman. *Solar Engineering of Thermal Processes*. John Wiley & Sons, Hoboken, NJ, 4th edition, 2013. Ch. 1: solar position, the sunrise hour angle, and the closed-form daily extraterrestrial irradiation on a horizontal plane implemented in `solar_tracking.daily_extraterrestrial_insolation_kwh_m2`.
- Jack Edmonds and Richard M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264, 1972a.
- Jack Edmonds and Richard M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264, 1972b. doi: 10.1145/321694.321699.
- Elmootazbellah N. Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys*, 34(3):375–408, 2002. doi: 10.1145/568522.568525.
- Molly Espey, James Espey, and W. Douglass Shaw. Price elasticity of residential demand for water: A meta-analysis. *Water Resources Research*, 33(6):1369–1374, 1997. doi: 10.1029/97WR00571.
- Stewart N. Ethier. *The Doctrine of Chances: Probabilistic Aspects of Gambling*. Probability and its Applications. Springer, 2010. URL <https://doi.org/10.1007/978-3-540-78783-9>.
- Shimon Even. *Graph Algorithms*. Cambridge University Press, 2 edition, 2011.
- Wigbert Fehse. *Automated Rendezvous and Docking of Spacecraft*. Cambridge University Press, Cambridge, 2003.
- Uriel Feige. A threshold of $\ln n$ for approximating set cover. *Journal of the ACM*, 45(4):634–652, 1998. doi: 10.1145/285055.285059.
- Robert L. Feller. *Accelerated Aging: Photochemical and Thermal Aspects*. Getty Conservation Institute, Los Angeles, 1994.
- Financial Action Task Force. Vulnerabilities of casinos and gaming sector. Technical report, FATF/OECD, Paris, 2009. Casino-sector money-laundering typologies: chip purchase and redemption, structuring, and minimal-play cash-out - the behaviours laundering_detect.py scores.
- Financial Action Task Force. International standards on combating money laundering and the financing of terrorism & proliferation (the fatf recommendations). FATF, Paris, 2012, updated 2020.
- Peter F. Fisher. Extending the applicability of viewsheds in landscape planning. *Photogrammetric Engineering & Remote Sensing*, 62(11):1297–1302, 1996.
- Ian Fleming. *From Russia, with Love*. Jonathan Cape, London, 1957. The 1963 Eon film is an adaptation of this 1957 novel.
- Ian Fleming. *Thunderball*. Jonathan Cape, London, 1961.
- Robert P. Flood and Peter M. Garber. Collapsing exchange-rate regimes: Some linear examples. *Journal of International Economics*, 17(1–2):1–13, 1984. The linear shadow-exchange-rate and attack-time closed forms implemented in `reserve_attack.attack_time`.
- Lester R. Ford and Delbert R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399–404, 1956a. doi: 10.4153/CJM-1956-045-5.
- Lester R. Ford and Delbert R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399–404, 1956b. doi: 10.4153/CJM-1956-045-5.
- Lester Randolph Ford and Delbert Ray Fulkerson. *Flows in Networks*. Princeton University Press, 1962.
- Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, Boston, MA, USA, 2002. ISBN 978-0-321-12742-6.

- Wade H. Foy. Position-location solutions by Taylor-series estimation. *IEEE Transactions on Aerospace and Electronic Systems*, AES-12(2):187–194, 1976a.
- Wade H. Foy. Position-location solutions by Taylor-series estimation. *IEEE Transactions on Aerospace and Electronic Systems*, AES-12(2):187–194, 1976b. The iterative Taylor-series (Gauss–Newton) estimator this package implements.
- Linton C. Freeman. Centrality in social networks conceptual clarification. *Social Networks*, 1(3):215–239, 1978. Degree-style centrality; the sense in which the gross-flow hub is a network hub.
- Linton C. Freeman. Centrality in social networks: Conceptual clarification. *Social Networks*, 1(3):215–239, 1978/1979. doi: 10.1016/0378-8733(78)90021-7.
- Daniel A. Friedman. The living daylights: Sniper’s nest — special-agent mission software, 2026a. Local-only working tree.
- Daniel Ari Friedman. Bond-api: The frozen mission protocol contract for the bond film suite. Software repository, 2026b. URL <https://github.com/docxology/bond-api>.
- Daniel Ari Friedman. Bond-api: The frozen missionprovider protocol (the protocol), 2026c. Frozen contract every BOND film package implements: protocol dataclasses, discovery, gadget registry. This package’s only non-stdlib suite dependency.
- Daniel Ari Friedman. Bond-ops: Fleet health, aggregate gates, and provisioning for project bond, 2026d. PROJECT BOND suite, Layer 4 (codename QUARTERMASTER). Local-only working tree with no remote; unpublished software record.
- Daniel Ari Friedman. bond-api: the frozen missionprovider protocol contract, 2026e. PROJECT BOND suite, Layer 1 (THE PROTOCOL). Local-only working tree.
- Daniel Ari Friedman. BOND-Utilities (Q-BRANCH): Layer 0 foundation. Unpublished software, PROJECT BOND local working tree, 2026f. No remote and no DOI; named as a declared, not-yet-bound integration point.
- Daniel Ari Friedman. bond-orchestrator: A deterministic mission dag runner (bond-orchestrator, codename dag 00). Unpublished software, developed in the PROJECT BOND working tree; not deposited and not publicly hosted, 2026g.
- Daniel Ari Friedman. Casino royale (2006) – le chiffre special-agent mission software, 2026h. This package: hold’em game theory, CFR, ICM, Bayesian opponent inference, chip-flow and network-level laundering detection, parkour mobility routing, digitalis poison triage.
- Daniel Ari Friedman. Diamonds are forever: Conflict stone — special-agent mission software. PROJECT BOND film package, local-only working tree, 2026i. Deterministic provenance-chain, casino-network, and orbital-mirror analyses.
- William F. Friedman. The index of coincidence and its applications in cryptography. *Riverbank Laboratories Publication*, (22), 1922.
- David C. Froehlich. Peak outflow from breached embankment dam. *Journal of Water Resources Planning and Management*, 121(1): 90–97, 1995.
- David C. Froehlich. Embankment dam breach parameters and their uncertainties. *Journal of Hydraulic Engineering*, 134(12): 1708–1721, 2008.
- Cary Joji Fukunaga, Neal Purvis, and Robert Wade. No time to die, 2021. Film. Eon Productions / Metro-Goldwyn-Mayer.
- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Reading, MA, USA, 1994. ISBN 978-0-201-63361-0.
- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional Computing Series. Addison-Wesley, Reading, MA, USA, 1995. ISBN 978-0-201-63361-0.
- Sam Ganzfried and Tuomas Sandholm. Game theory-based opponent modeling in large imperfect-information games. In *Proceedings of the 10th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 533–540, 2011. Bayesian/game-theoretic opponent modelling; the motivation for the Dirichlet-multinomial archetype posterior in `opponent_bayes.py`.
- Mary Lynn Garcia. *The Design and Evaluation of Physical Protection Systems*. Butterworth-Heinemann, Burlington, MA, 2 edition, 2007. Layered physical protection, adversary sequence diagrams, and detection/delay path analysis — the standard reference behind both `vault_security` and `vault_threat_network`.
- Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979. Problem [SS1], sequencing with release times and deadlines: NP-complete in the strong sense.
- Milo Gibaldi and Donald Perrier. *Pharmacokinetics*. Marcel Dekker, New York, 2nd edition, 1982.
- Franklin A. Gifford. Use of routine meteorological observations for estimating atmospheric dispersion. *Nuclear Safety*, 2(4):47–51, 1961. The dispersion curves paired with Pasquill’s stability classes.
- Lewis Gilbert. Moonraker. Motion picture. Eon Productions / United Artists, 1979. Directed by Lewis Gilbert; screenplay by Christopher Wood; source concept for the DRAX mission.
- Peter E. Glaser. Power from the sun: Its future. *Science*, 162(3856):857–861, 1968.

- Samuel Glasstone and Philip J. Dolan. *The Effects of Nuclear Weapons*. US Department of Defense / ERDA, Washington, D.C., 3rd edition, 1977.
- John (dir.) Glen and Eon Productions. A view to a kill. Feature film; mission codename ZORIN, 1985.
- Global Witness. A rough trade: The role of companies and governments in the angolan conflict. *Global Witness Report*, 1998. Foundational documentation of conflict-diamond trade routes.
- Lorne W. Gold. Use of ice covers for transportation. *Canadian Geotechnical Journal*, 8(2):170–181, 1971.
- Frank Golden and Michael J. Tipton. *Essentials of Sea Survival*. Human Kinetics, Champaign, IL, 2002.
- David M. Green and John A. Swets. *Signal Detection Theory and Psychophysics*. John Wiley and Sons, New York, 1966.
- Guy Hamilton. The man with the golden gun. Eon Productions / United Artists, 1974. Film. Directed by Guy Hamilton; screenplay by Richard Maibaum and Tom Mankiewicz. Narrative source for the Solex agitator, the funhouse lair, and the assassin’s contract book.
- Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968a.
- Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968b.
- Douglas M. Hawkins and David H. Olwell. *Cumulative Sum Charts and Charting for Quality Improvement*. Springer, New York, 1998.
- Rainer Hegselmann and Ulrich Krause. Opinion dynamics and bounded confidence: models, analysis and simulation. *Journal of Artificial Societies and Social Simulation*, 5(3):1–24, 2002.
- Orris C. Herfindahl. *Concentration in the Steel Industry*. Phd dissertation, Columbia University, 1950.
- John R. Hicks. *Value and Capital: An Inquiry into Some Fundamental Principles of Economic Theory*. Oxford University Press, Oxford, 2 edition, 1946. Constant-elasticity demand and market clearing behind the shocked-equilibrium price.
- Archibald Vivian Hill. The possible effects of the aggregation of the molecules of haemoglobin on its dissociation curves. *The Journal of Physiology*, 40:iv–vii, 1910.
- Albert O. Hirschman. *National Power and the Structure of Foreign Trade*. University of California Press, 1945.
- Albert O. Hirschman. The paternity of an index. *The American Economic Review*, 54(5):761–762, 1964.
- Peter V. Hobbs. *Ice Physics*. Clarendon Press, Oxford, 1974.
- Walter Hohmann. *Die Erreichbarkeit der Himmelskörper*. R. Oldenbourg, 1925.
- John Hopcroft and Robert Tarjan. Algorithm 447: Efficient algorithms for graph manipulation. *Communications of the ACM*, 16(6):372–378, 1973a.
- John Hopcroft and Robert Tarjan. Algorithm 447: efficient algorithms for graph manipulation. *Communications of the ACM*, 16(6):372–378, 1973b. Linear-time biconnected-components / cut-vertex algorithm implemented in `lair_topology.articulation_points`.
- John Hopcroft and Robert Endre Tarjan. Algorithm 447: Efficient algorithms for graph manipulation. *Communications of the ACM*, 16(6):372–378, 1973c. doi: 10.1145/362248.362272.
- John E. Hopcroft and Richard M. Karp. An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs. *SIAM Journal on Computing*, 2(4):225–231, 1973.
- Charles Velson Horie. *Materials for Conservation: Organic Consolidants, Adhesives and Coatings*. Butterworth-Heinemann, Oxford, 2nd edition, 2010.
- Patrick D Hsu, David A Scott, Jason A Weinstein, F Ann Ran, Silvana Konermann, Vineeta Agarwala, Yinqing Li, Eli J Fine, Xuebing Wu, Ophir Shalem, et al. DNA targeting specificity of RNA-guided Cas9 nucleases. *Nature Biotechnology*, 31(9):827–832, 2013.
- J. H. Hubbell and S. M. Seltzer. Tables of X-ray mass attenuation coefficients and mass energy-absorption coefficients from 1 keV to 20 MeV for elements $Z = 1$ to 92 and 48 additional substances of dosimetric interest. Technical Report NISTIR 5632, National Institute of Standards and Technology, 1995. Source of the μ/ρ values in `MASS_ATTENUATION_CM2_G`.
- Andrew Hunt and David Thomas. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley, Reading, MA, USA, 1999. ISBN 978-0-201-61622-4.
- John D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55. plotting library used by the deterministic Agg figure renderers.

- John Huston, Ken Hughes, Joseph McGrath, Robert Parrish, and Val Guest. *Casino royale*, 1967. Satirical source of the five-Bond premise; five credited directors.
- The Open Group Base Specifications Issue 7, 2018 edition (IEEE Std 1003.1-2017), Chapter 12: Utility Conventions*. IEEE and The Open Group, 2018. URL https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/V1_chap12.html. The utility argument and exit-status conventions the front door’s option handling and 0/1/2 exit codes follow.
- International Commission on Radiological Protection. Age-dependent doses to members of the public from intake of radionuclides: Part 5, compilation of ingestion and inhalation dose coefficients. ICRP Publication 72, *Annals of the ICRP* 26(1), 1996.
- International Commission on Radiological Protection. Compendium of dose coefficients based on ICRP publication 60. ICRP Publication 119, *Annals of the ICRP* 41 (Suppl.), 2012. Committed effective dose coefficients behind INHALATION_DCF_SV_BQ.
- International Maritime Organization. International convention on tonnage measurement of ships, 1969. IMO, London, 1969.
- International Telecommunication Union. International morse code. Recommendation ITU-R M.1677-1, ITU Radiocommunication Sector (ITU-R), 2009. Normative dot/dash durations and intra-letter, inter-letter and inter-word gap ratios used by the drum codec.
- Eitan Israeli and R. Kevin Wood. Shortest-path network interdiction. *Networks*, 40(2):97–111, 2002. The shortest-path interdiction / improvement problem the greedy hardening allocation in `harden_network` approximates.
- David S. Johnson. Approximation algorithms for combinatorial problems. *Journal of Computer and System Sciences*, 9(3):256–278, 1974. doi: 10.1016/S0022-0000(74)80044-9.
- Donald B. Johnson. Finding all the elementary circuits of a directed graph. *SIAM Journal on Computing*, 4(1):77–84, 1975. Complete elementary-circuit enumeration used for the money-cycle count and laundering closure.
- Arthur B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11):558–562, 1962. doi: 10.1145/368996.369025.
- David Kahn. *The Codebreakers: The Comprehensive History of Secret Communication from Ancient Times to the Internet*. Scribner, revised edition, 1996.
- Rudolph E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960. doi: 10.1115/1.3662552.
- Friedrich Kasiski. *Die Geheimschriften und die Dechiffrier-Kunst*. E. S. Mittler und Sohn, Berlin, 1863.
- Steven M. Kay. *Fundamentals of Statistical Signal Processing: Detection Theory*, volume 2. Prentice Hall, 1998.
- Hans Kellerer, Ulrich Pferschy, and David Pisinger. *Knapsack Problems*. Springer, Berlin, Heidelberg, 2004. doi: 10.1007/978-3-540-24777-7.
- James E. Kelley and Morgan R. Walker. Critical-path planning and scheduling. *Proceedings of the Eastern Joint Computer Conference*, pages 160–173, 1959.
- John L. Kelly. A new interpretation of information rate. *Bell System Technical Journal*, 35(4):917–926, 1956.
- John G. Kemeny and J. Laurie Snell. *Finite Markov Chains*. D. Van Nostrand, Princeton, NJ, 1960.
- David Kempe, Jon Kleinberg, and Éva Tardos. Maximizing the spread of influence through a social network. In *Proceedings of the 9th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD ’03)*, pages 137–146, 2003a. doi: 10.1145/956750.956769.
- David Kempe, Jon Kleinberg, and Éva Tardos. Maximizing the spread of influence through a social network. In *Proceedings of the 9th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 137–146, 2003b.
- Nirmal Keshava and John F. Mustard. Spectral unmixing. *IEEE Signal Processing Magazine*, 19(1):44–57, 2002.
- J. Kiefer. Sequential minimax search for a maximum. *Proceedings of the American Mathematical Society*, 4(3):502–506, 1953.
- Rebecca Killick, Paul Fearnhead, and Idris A. Eckley. Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107(500):1590–1598, 2012.
- Kimberley Process. Kimberley process certification scheme. Certification scheme launched January 2003; adopted by the Interlaken Declaration of 5 November 2002, 2003. Supply-chain provenance standard for conflict-diamond traceability.
- Lawrence E. Kinsler, Austin R. Frey, Alan B. Coppers, and James V. Sanders. *Fundamentals of Acoustics*. John Wiley & Sons, New York, 4th edition, 2000.
- Jon Kleinberg and Éva Tardos. *Algorithm Design*. Addison-Wesley, 2005. Chapter 4: greedy interval scheduling.
- Jon Kleinberg and Éva Tardos. *Algorithm Design*. Pearson / Addison-Wesley, Boston, MA, 2006. Sec. 6.1: weighted interval scheduling by dynamic programming with the `p(j)` predecessor table, implemented in `duel_scheduler.max_bounty_schedule`; sec. 4.1: the earliest-finish greedy for interval scheduling, implemented in `duel_scheduler.earliest_finish_schedule`.

- Glenn F. Knoll. *Radiation Detection and Measurement*. Wiley, Hoboken, NJ, 4th edition, 2010.
- Heidi Kreibich and Annegret H. Thieken. Assessment of damage caused by high groundwater inundation. *Water Resources Research*, 44(9):W09409, 2008.
- Holger Krekel, Bruno Oliveira, Ronny Pfannschmidt, Floris Bruynooghe, Brianna Laughner, and Florian Bruhin. pytest: helps you write better programs. <https://docs.pytest.org>. Software; version pinned by this package’s uv.lock. Test framework used by every fleet package gate.
- Paul Krugman. A model of balance-of-payments crises. *Journal of Money, Credit and Banking*, 11(3):311–325, 1979. The first-generation speculative-attack setup underlying `reserve_attack`.
- Harold W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.
- Yu-Kwong Kwok and Ishfaq Ahmad. Static scheduling algorithms for allocating directed task graphs to multiprocessors. *ACM Computing Surveys*, 31(4):406–471, 1999. doi: 10.1145/344588.344618.
- Albert S. Kyle. Continuous auctions and insider trading. *Econometrica*, 53(6):1315–1335, 1985. Canonical treatment of the price impact of a strategic trader — the microstructure counterpart to the corner’s supply shock.
- Irving Langmuir. The adsorption of gases on plane surfaces of glass, mica and platinum. *Journal of the American Chemical Society*, 40(9):1361–1403, 1918.
- Vito Latora and Massimo Marchiori. Efficient behavior of small-world networks. *Physical Review Letters*, 87(19):198701, 2001. doi: 10.1103/PhysRevLett.87.198701.
- Charles L. Lawson and Richard J. Hanson. *Solving Least Squares Problems*. Prentice-Hall, Englewood Cliffs, NJ, 1974a.
- Charles L. Lawson and Richard J. Hanson. *Solving Least Squares Problems*. Prentice-Hall, Englewood Cliffs, NJ, 1974b. Classical reference for non-negative least-squares fitting.
- Akos Ledecz, Andras Nadas, Peter Volgyesi, Gyorgy Balogh, Branislav Kusy, Janos Sallai, Gyula Pap, Sebestyen Dora, Karoly Molnar, Miklos Maroti, and Gyorgy Simon. Countersniper system for urban warfare. *ACM Transactions on Sensor Networks*, 1(2):153–177, 2005. doi: 10.1145/1105688.1105689.
- Jukka Lehtosalo and contributors. mypy: Optional static typing for python. <https://mypy.readthedocs.io>. Software; version pinned by this package’s uv.lock. Static type check run per package in the aggregate gate.
- Jure Leskovec, Andreas Krause, Carlos Guestrin, Christos Faloutsos, Jeanne VanBriesen, and Natalie Glance. Cost-effective outbreak detection in networks. In *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD ’07)*, pages 420–429, 2007. doi: 10.1145/1281192.1281239.
- Vladimir Iosifovich Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966.
- Michael Levi and Peter Reuter. Money laundering. *Crime and Justice*, 34(1):289–375, 2006. Money-laundering detection methods over financial networks and ledgers.
- Ivan Levkivskyi, Jukka Lehtosalo, and Łukasz Langa. Pep 544 – protocols: Structural subtyping (static duck typing). Python Enhancement Proposal, 2017. URL <https://peps.python.org/pep-0544/>.
- Daniel P. Loucks and Eelco van Beek. *Water Resource Systems Planning and Management: An Introduction to Methods, Models, and Applications*. Springer, 2017. doi: 10.1007/978-3-319-44234-1.
- Kenneth V. Mackenzie. Nine-term equation for sound speed in the oceans. *Journal of the Acoustical Society of America*, 70(3):807–812, 1981.
- Prasanta Chandra Mahalanobis. On the generalized distance in statistics. *Proceedings of the National Institute of Sciences of India*, 2:49–55, 1936.
- Richard Maibaum and Michael G. Wilson. Licence to kill. Film. Directed by John Glen. Eon Productions, 1989. Bond film: mission codename ROGUE.
- Tom Mankiewicz. Live and let die. Eon Productions / United Artists, 1973. Source film for the SAN MONIQUE mission scenario.
- J. I. Marcum. A statistical theory of target detection by pulsed radar. *IRE Transactions on Information Theory*, 6(2):59–267, 1960.
- Donald W. Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2):431–441, 1963.
- Charlie Marsh and Astral Software Inc. Ruff: An extremely fast python linter and code formatter. <https://docs.astral.sh/ruff>. Software; version pinned by this package’s uv.lock. Lint and format checks run per package in the aggregate gate.
- Silvano Martello and Paolo Toth. *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, 1990.

- D. O. Martin. Comment on “the change of concentration standard deviations with distance”. *Journal of the Air Pollution Control Association*, 26(2):145–147, 1976. Power-law fits to the Pasquill-Gifford curves. The a/b/c/d/f values in plume_model.STABILITY_TABLE are taken from this paper verbatim (x in km, sigma in m); only the near-field $x \leq 1$ km sigma_z branch is implemented. A phase-6 review found the table had previously held neither this paper’s coefficients nor a consistent unit convention; tests/test_plume_model.py now binds both coefficients to published Pasquill-Gifford values.
- Makoto Matsumoto and Takuji Nishimura. Mersenne twister: A 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1):3–30, 1998. doi: 10.1145/272991.272995.
- David McClung and Peter Schaerer. *The Avalanche Handbook*. The Mountaineers, 3 edition, 2006.
- Robert L. McCoy. *Modern Exterior Ballistics: The Launch and Flight Dynamics of Symmetric Projectiles*. Schiffer Publishing, Atglen, PA, 1999. ISBN 9780764307201.
- Peter Mell, Karen Scarfone, and Sasha Romanosky. A complete guide to the common vulnerability scoring system version 2.0. Technical report, Forum of Incident Response and Security Teams (FIRST), 2007.
- Joseph J. Michalsky. The astronomical almanac’s algorithm for approximate solar position (1950–2050). *Solar Energy*, 40(3):227–235, 1988. Higher-accuracy alternative to the Spencer series; cited in the scope section as the upgrade path, deliberately not implemented here.
- Thomas P. Minka. Estimating a Dirichlet distribution. Technical report, Massachusetts Institute of Technology, 2000. Dirichlet and Dirichlet-multinomial machinery; the closed-form log-space likelihood used in opponent_bayes.py.
- Pratap Misra and Per Enge. *Global Positioning System: Signals, Measurements, and Performance*. Ganga-Jamuna Press, 2nd edition, 2006.
- Douglas C. Montgomery. *Introduction to Statistical Quality Control*. Wiley, Hoboken, NJ, 7 edition, 2012.
- M. M. Morlock and V. M. Zatsiorsky. Factors influencing performance in bobsledding: I: Influences of the bobsled crew and the environment. *International Journal of Sport Biomechanics*, 5(2):208–221, 1989.
- Michelle Namnyak, Norra Tufton, Rebecca Szekely, Matthew Toal, Sarah Worboys, and Eleanor L. Sampson. “stockholm syndrome”: psychiatric diagnosis or urban myth? *Acta Psychiatrica Scandinavica*, 117(1):4–11, 2008. doi: 10.1111/j.1600-0447.2007.01112.x.
- Steven C. Nardone and Vincent J. Aidala. Observability criteria for bearings-only target motion analysis. *IEEE Transactions on Aerospace and Electronic Systems*, AES-17(2):162–166, 1981.
- National Institute of Standards and Technology. Secure hash standard (SHS). Federal Information Processing Standards Publication FIPS PUB 180-4, U.S. Department of Commerce, 2015.
- National Nuclear Data Center. ENSDF: Evaluated nuclear structure data file. Brookhaven National Laboratory. Gamma-ray energies, emission intensities, and half-lives underlying the isotope library and decay-chain table.
- National Research Council. *Risk Assessment in the Federal Government: Managing the Process*. National Academies Press, Washington, DC, 1983.
- Naval Sea Systems Command. *U.S. Navy Diving Manual, Revision 7*. Naval Sea Systems Command, Washington, DC, 2016. SS521-AG-PRO-010.
- George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14(1):265–294, 1978a. doi: 10.1007/BF01588971.
- George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14(1):265–294, 1978b. doi: 10.1007/BF01588971.
- Simon Newcomb. Note on the frequency of use of the different digits in natural numbers. *American Journal of Mathematics*, 4(1): 39–40, 1881. The first published observation of the leading-digit law later named for Benford.
- John Nicholas Newman. *Marine Hydrodynamics*. MIT Press, Cambridge, MA, 1977.
- Mark E. J. Newman. Spread of epidemic disease on networks. *Physical Review E*, 66:016128, 2002. doi: 10.1103/PhysRevE.66.016128.
- Mark E. J. Newman. *Networks: An Introduction*. Oxford University Press, 2010. doi: 10.1093/acprof:oso/9780199206650.001.0001.
- Isaac Newton. Scala graduum caloris: Calorum descriptiones & signa. *Philosophical Transactions of the Royal Society of London*, 22: 824–829, 1701.
- Mark J. Nigrini. *Benford’s Law: Applications for Forensic Accounting, Auditing, and Fraud Detection*. John Wiley & Sons, Hoboken, NJ, 2012. Applied digit analysis in forensic accounting; the practice this ledger audit models.
- Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, New York, 2nd edition, 2006.

- National Institute of Standards and Technology. Fips 180-4: Secure hash standard (sha-1, sha-224, sha-256, sha-384, sha-512, sha-512/224 and sha-512/256). Federal Information Processing Standards Publication, 2015. URL <https://csrc.nist.gov/pubs/fips/180-4/upd1/final>.
- Brian D. Ondov, Todd J. Treangen, Páll Melsted, Adam B. Mallonee, Nicholas H. Bergman, Sergey Koren, and Adam M. Phillippy. Mash: fast genome and metagenome distance estimation using minhash. *Genome Biology*, 17:132, 2016. doi: 10.1186/s13059-016-0997-x.
- Henry W. Ott. *Electromagnetic Compatibility Engineering*. John Wiley and Sons, Hoboken, NJ, 2009.
- E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.
- Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The PageRank citation ranking: Bringing order to the web. Technical Report 1999-66, Stanford InfoLab, 1999. PageRank; used in `laundering_network.py` to rank the collection account that centrally-placed senders feed.
- Bradford W. Parkinson and Penina Axelrad. Autonomous GPS integrity monitoring using the pseudorange residual. *Navigation*, 35(2):255–274, 1988.
- F. Pasquill. The estimation of the dispersion of windborne material. *The Meteorological Magazine*, 90(1063):33–49, 1961. Origin of the A–F atmospheric stability classification.
- Judea Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, San Mateo, CA, 1988.
- Karl Pearson. On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 50(302):157–175, 1900. The chi-square goodness-of-fit statistic used to score the ledger against Benford.
- Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi: 10.1126/science.1213847.
- Cynthia Phillips and Laura Painton Swiler. A graph-based system for network-vulnerability analysis. In *Proceedings of the 1998 Workshop on New Security Paradigms (NSPW '98)*, pages 71–79. ACM, 1998.
- Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, 5 edition, 2016.
- Tom Preston-Werner. Semantic versioning 2.0.0. URL <https://semver.org/spec/v2.0.0.html>. The versioning scheme used by this package’s `pyproject.toml` and `manuscript/config.yaml`.
- John G. Proakis and Masoud Salehi. *Digital Communications*. McGraw-Hill, 5 edition, 2008. Additive white Gaussian noise channel; symbol decision.
- PROJECT BOND suite. BOND-API: the frozen MissionProvider protocol. `bond-api` package, local-only working tree, 2026. The mission contract this film package implements (brief / recon / plan / execute / debrief).
- Python Software Foundation. `argparse` — parser for command-line options, arguments and sub-commands. The Python Standard Library documentation, a. URL <https://docs.python.org/3/library/argparse.html>. Reference documentation for the parser API used by `bond_cli.dispatch.build_parser`, including the exit status 2 on usage error.
- Python Software Foundation. `inspect` — inspect live objects. The Python Standard Library documentation, b. URL <https://docs.python.org/3/library/inspect.html>. Source of `inspect.signature`, used to read a `register_gadgets` hook’s arity so a body `TypeError` is diagnosed rather than misread as a zero-arg hook.
- Usha Nandini Raghavan, Réka Albert, and Soundar Kumara. Near linear time algorithm to detect community structures in large-scale networks. *Physical Review E*, 76(3):036106, 2007. Label-propagation community detection; implemented deterministically in `laundering_network.py`.
- Eric S. Raymond. *The Art of Unix Programming*. Addison-Wesley, 2003. ISBN 978-0131429017. Source of the composition and transparency rules behind the plain, pipeable, ANSI-free output this CLI commits to.
- I. S. Reed and X. Yu. Adaptive multiple-band CFAR detection of an optical pattern with unknown spectral distribution. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 38(10):1760–1770, 1990.
- Irving S. Reed and Gustave Solomon. Polynomial codes over certain finite fields. *Journal of the Society for Industrial and Applied Mathematics*, 8(2):300–304, 1960.
- Peter Reuter and Mark A. R. Kleiman. Risks and prices: An economic analysis of drug enforcement. *Crime and Justice*, 7:289–340, 1986. Pricing and purity along the narcotics supply chain.
- August Ritter. Die fortpflanzung der wasserwellen. *Zeitschrift des Vereines deutscher Ingenieure*, 36(33):947–954, 1892.
- Sheldon M. Ross. *Introduction to Probability Models*. Academic Press, 12 edition, 2019.
- Ariel Rubinstein. Perfect equilibrium in a bargaining model. *Econometrica*, 50(1):97–109, 1982. doi: 10.2307/1912531.

- Andrea Saltelli, Marco Ratto, Terry Andres, Francesca Campolongo, Jessica Cariboni, Debora Gatelli, Michaela Saisana, and Stefano Tarantola. *Global Sensitivity Analysis. The Primer*. John Wiley & Sons, Chichester, 2008. doi: 10.1002/9780470725184.
- Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLOS Computational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.1003285.
- Sergei A. Schelkunoff. *Electromagnetic Waves*. D. Van Nostrand, New York, 1943.
- Michael Shackelford. Baccarat basics. Wizard of Odds, 2023. URL <https://wizardofodds.com/games/baccarat/basics/>.
- Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
- Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948a. URL <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Claude E. Shannon. Communication theory of secrecy systems. *Bell System Technical Journal*, 28(4):656–715, 1949a.
- Claude Elwood Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3–4):379–423 and 623–656, 1948b. doi: 10.1002/j.1538-7305.1948.tb01338.x.
- Claude Elwood Shannon. Communication theory of secrecy systems. *Bell System Technical Journal*, 28(4):656–715, 1949b. doi: 10.1002/j.1538-7305.1949.tb00928.x.
- Lloyd S. Shapley. A value for n-person games. In *Contributions to the Theory of Games (II)*, volume 28 of *Annals of Mathematics Studies*, pages 307–317. Princeton University Press, 1953.
- Oleg Sheyner, Joshua Haines, Somesh Jha, Richard Lippmann, and Jeannette M. Wing. Automated generation and analysis of attack graphs. In *Proceedings of the 2002 IEEE Symposium on Security and Privacy*, pages 273–284. IEEE, 2002.
- Anthony E. Siegman. *Lasers*. University Science Books, Mill Valley, CA, 1986. Gaussian-beam propagation, waist, and Rayleigh range used by `laser_thermal.spot_radius_at_distance`.
- Merrill I. Skolnik. *Introduction to Radar Systems*. McGraw-Hill, New York, 3rd edition, 2001a.
- Merrill I. Skolnik. *Introduction to Radar Systems*. McGraw-Hill, 3rd edition, 2001b.
- Daniel S. Smith and Michael G. Stabin. Exposure rate constants and lead shielding values for over 1,100 radionuclides. *Health Physics*, 102(3):271–291, 2012. Specific gamma-ray constants behind `GAMMA_CONSTANT_MSV_M2_H_GBQ`.
- Eric V. Smith. Pep 557 – data classes. Python Enhancement Proposal, 2017. URL <https://peps.python.org/pep-0557/>.
- Wayne E. Smith. Various optimizers for single-stage production. *Naval Research Logistics Quarterly*, 3(1–2):59–66, 1956. doi: 10.1002/nav.3800030106.
- Devinder S. Sodhi. Breakthrough loads of floating ice sheets. *Journal of Cold Regions Engineering*, 9(1):4–22, 1995.
- J. W. Spencer. Fourier series representation of the position of the sun. *Search*, 2(5):172, 1971. The declination and equation-of-time Fourier series implemented in `solar_tracking.solar_declination_deg` and `equation_of_time_min`.
- William M. Steen and Jyotirmoy Mazumder. *Laser Material Processing*. Springer, London, 4 edition, 2010. Continuous-wave laser energy-balance and cutting models behind `laser_physics.burn_through_time`.
- Douglas R. Stinson and Maura B. Paterson. *Cryptography: Theory and Practice*. CRC Press, fourth edition, 2018.
- Douglas R. Stinson and Maura B. Paterson. *Cryptography: Theory and Practice*. CRC Press, 4 edition, 2019. Monoalphabetic substitution ciphers and frequency analysis.
- James Johnston Stoker. *Water Waves: The Mathematical Theory with Applications*. Interscience Publishers, New York, 1957.
- Alice M. Stoll. Human tolerance to positive g as determined by the physiological end points. *Journal of Aviation Medicine*, 27(4): 356–367, 1956.
- Lawrence D. Stone. *Theory of Optimal Search*. Academic Press, New York, 1975.
- Brian Stott. Review of load-flow calculation methods. *Proceedings of the IEEE*, 62(7):916–929, 1974. doi: 10.1109/PROC.1974.9534.
- Alexander Strehl and Joydeep Ghosh. Cluster ensembles — a knowledge reuse framework for combining multiple partitions. *Journal of Machine Learning Research*, 3:583–617, 2002. Normalized mutual information; the normalization convention used for the routing NMI.
- P. Swerling. Probability of detection for fluctuating targets. *IRE Transactions on Information Theory*, 6(2):269–308, 1960.
- R. Ian Sykes, Sonya F. Parker, Donald S. Henn, and William S. Lewellen. SCIPUFF – a generalized lagrangian dispersion modeling system for plume rise, dispersion and deposition. In *NATO CCMS 21st International Technical Meeting on Air Pollution Modelling and Its Application*, 1998.
- Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972a.

- Robert Tarjan. Enumeration of the elementary circuits of a directed graph. *SIAM Journal on Computing*, 2(3):211–216, 1973. Directed-circuit enumeration; the classical statement of the money-cycle problem.
- Robert Endre Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972b. doi: 10.1137/0201010.
- Robert Endre Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975. Path-compressed disjoint-set analysis; the structure used for the “latest free slot” query in `deadline_scheduler._SlotUnionFind`.
- The Unicode Consortium. Unicode standard annex #15: Unicode normalization forms. <https://www.unicode.org/reports/tr15/>, 2023.
- William H. Thorp. Analytic description of the low-frequency attenuation coefficient. *Journal of the Acoustical Society of America*, 42(1):270, 1967a.
- William H. Thorp. Analytic description of the low-frequency attenuation coefficient. *The Journal of the Acoustical Society of America*, 42(1):270, 1967b.
- William H. Thorp. Analytic description of the low-frequency attenuation coefficient. *Journal of the Acoustical Society of America*, 42(1):270, 1967c.
- Stephen P. Timoshenko and James M. Gere. *Theory of Elastic Stability*. McGraw-Hill, 2nd edition, 1961.
- Michael J. Tipton. The initial responses to cold-water immersion in man. *Clinical Science*, 77(6):581–588, 1989.
- Don J. Torrieri. Statistical theory of passive location systems. *IEEE Transactions on Aerospace and Electronic Systems*, AES-20(2): 183–198, 1984. Iterative least-squares TDOA location and its error analysis.
- Konstantin E. Tsiolkovsky. Exploration of cosmic space by means of reaction devices. *Nauchnoye Obozreniye (Scientific Review)*, 1903.
- Eric C. Tupper. *Introduction to Naval Architecture*. Butterworth-Heinemann, 5th edition, 2013.
- D. Bruce Turner. *Workbook of Atmospheric Dispersion Estimates*. U.S. Environmental Protection Agency, 1970.
- D. Bruce Turner. *Workbook of Atmospheric Dispersion Estimates: An Introduction to Dispersion Modeling*. Lewis Publishers / CRC Press, Boca Raton, FL, 2nd edition, 1994. ISBN 978-1-56670-023-6. Standard treatment of the Gaussian plume with ground reflection.
- United States Bureau of Engraving and Printing. Currency facts: Denominations and dimensions. BEP, Washington, DC, 2020. US note mass 1 g; length 155.956 mm, width 66.294 mm, thickness 0.10922 mm.
- United States Drug Enforcement Administration. Domestic drug prices and pure-purity analyses (stride / system to retrieve information from drug evidence). DEA, Arlington, VA, 1990s–2020. Retail/wholesale cocaine price ranges used as order-of-magnitude anchors.
- Robert J. Urick. *Principles of Underwater Sound*. McGraw-Hill, New York, 3rd edition, 1983a.
- Robert J. Urick. *Principles of Underwater Sound*. McGraw-Hill, New York, 3rd edition, 1983b.
- Robert J. Urick. *Principles of Underwater Sound*. McGraw-Hill, 3rd edition, 1983c.
- Robert J. Urick. *Principles of Underwater Sound*. McGraw-Hill, 3rd edition, 1983d.
- U.S. Department of Justice and Federal Trade Commission. Horizontal merger guidelines. Technical report, U.S. Department of Justice and Federal Trade Commission, 2010. Section 5.3 defines the unconcentrated / moderately concentrated / highly concentrated HHI bands.
- U.S. Department of Justice and Federal Trade Commission. Horizontal merger guidelines. <https://www.justice.gov/atr/horizontal-merger-guidelines-08192010>, 2010.
- U.S. Department of the Army. *FM 23-10: Sniper Training*. Headquarters, Department of the Army, Washington, DC, 1994. Mil-dot ranging and field firing procedures.
- David A. Vallado. *Fundamentals of Astrodynamics and Applications*. Microcosm Press, Hawthorne, CA, 4 edition, 2013a.
- David A. Vallado. *Fundamentals of Astrodynamics and Applications*. Microcosm Press, 4th edition, 2013b.
- Guido van Rossum, Jukka Lehtosalo, and Łukasz Langa. Pep 484 – type hints. Python Enhancement Proposal, 2014. URL <https://peps.python.org/pep-0484/>.
- Harry L. Van Trees. *Detection, Estimation, and Modulation Theory, Part I*. John Wiley & Sons, New York, 1968.
- Harry L. Van Trees. *Optimum Array Processing: Part IV of Detection, Estimation, and Modulation Theory*. Wiley-Interscience, 2002.
- Edward F. Vance. *Coupling to Shielded Cables*. Wiley-Interscience, New York, 1978.

- Pierre-François Verhulst. Notice sur la loi que la population suit dans son accroissement. *Correspondance Mathématique et Physique*, 10:113–121, 1838.
- Léon Walras. *Éléments d'économie politique pure, ou théorie de la richesse sociale*. L. Corbaz & Cie, Lausanne, 1874. Origin of the tâtonnement price-adjustment process used by `market_attack.simulate_corner`.
- Edward Waltz and James Llinas. *Multisensor Data Fusion*. Artech House, Boston, MA, 1990.
- Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. In *KDD Workshop on Anomaly Detection in Finance*, 2019. Graph-based AML over transaction networks; the framing this package’s structural layer follows.
- Robert Webster. *Gems: Their Sources, Descriptions and Identification*. Butterworth-Heinemann, Oxford, 5th edition, 1994. Gemo-logical identification practice; basis of the density forensic test.
- Gordon M. Wenz. Acoustic ambient noise in the ocean: Spectra and sources. *The Journal of the Acoustical Society of America*, 34(12):1936–1956, 1962.
- James R. Wertz, David F. Everett, and Jeffery J. Puschell, editors. *Space Mission Engineering: The New SMAD*. Microcosm Press, Hawthorne, CA, 2011.
- Bang Wong. Points of view: Color blindness. *Nature Methods*, 8(6):441, 2011. doi: 10.1038/nmeth.1618. source of the colourblind-safe palette used by the figures.
- R. Kevin Wood. Deterministic network interdiction. *Mathematical and Computer Modelling*, 17(2):1–18, 1993a. Budgeted network interdiction formulation.
- R. Kevin Wood. Deterministic network interdiction. *Mathematical and Computer Modelling*, 17(2):1–18, 1993b. doi: 10.1016/0895-7177(93)90236-R.
- T. P. Wright. Factors affecting the cost of airplanes. *Journal of the Aeronautical Sciences*, 3(4):122–128, 1936. doi: 10.2514/8.155.
- Warren C. Young, Richard G. Budynas, and Ali M. Sadegh. *Roark’s Formulas for Stress and Strain*. McGraw-Hill, 8th edition, 2011.
- Paolo Zannetti. *Air Pollution Modeling: Theories, Computational Methods, and Available Software*. Van Nostrand Reinhold, 1990.
- Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems 20 (NIPS)*, pages 1729–1736, 2007. Counterfactual regret minimization; the algorithm implemented in `poker_cfr.py`.